

ゲームグラフィックステ論

2026年09月04日(金) 10時19分05秒作成 - ゲームグラフィックステ論 / 構成: Doxygen
1.15.0

2026年09月04日(金) 10時19分05秒

1	ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版.	1
2	名前空間索引	3
2.1	名前空間一覧	3
3	階層索引	5
3.1	クラス階層	5
4	クラス索引	7
4.1	クラス一覧	7
5	ファイル索引	9
5.1	ファイル一覧	9
6	名前空間詳解	11
6.1	gg 名前空間	11
6.1.1	詳解	14
6.1.2	列挙型詳解	14
6.1.2.1	BindingPoints	14
6.1.3	関数詳解	14
6.1.3.1	_ggError()	14
6.1.3.2	_ggFBOError()	15
6.1.3.3	ggArraysObj()	15
6.1.3.4	ggConjugate()	16
6.1.3.5	ggCreateComputeShader()	16
6.1.3.6	ggCreateNormalMap()	17
6.1.3.7	ggCreateShader()	18
6.1.3.8	ggCross() [1/2]	19
6.1.3.9	ggCross() [2/2]	20
6.1.3.10	ggDistance3() [1/2]	20
6.1.3.11	ggDistance3() [2/2]	21
6.1.3.12	ggDistance4() [1/2]	22
6.1.3.13	ggDistance4() [2/2]	22
6.1.3.14	ggDot3() [1/2]	23
6.1.3.15	ggDot3() [2/2]	24
6.1.3.16	ggDot4() [1/2]	25
6.1.3.17	ggDot4() [2/2]	25
6.1.3.18	ggElementsMesh()	26
6.1.3.19	ggElementsObj()	27
6.1.3.20	ggElementsSphere()	28
6.1.3.21	ggEllipse()	29
6.1.3.22	ggEulerQuaternion() [1/3]	29
6.1.3.23	ggEulerQuaternion() [2/3]	30
6.1.3.24	ggEulerQuaternion() [3/3]	31

6.1.3.25 ggFrustum()	32
6.1.3.26 ggIdentity()	33
6.1.3.27 ggIdentityQuaternion()	33
6.1.3.28 ggInit()	34
6.1.3.29 ggInvert() [1/2]	34
6.1.3.30 ggInvert() [2/2]	35
6.1.3.31 ggLength3() [1/2]	35
6.1.3.32 ggLength3() [2/2]	36
6.1.3.33 ggLength4() [1/2]	37
6.1.3.34 ggLength4() [2/2]	37
6.1.3.35 ggLoadComputeShader()	38
6.1.3.36 ggLoadHeight()	39
6.1.3.37 ggLoadImage()	40
6.1.3.38 ggLoadShader() [1/2]	40
6.1.3.39 ggLoadShader() [2/2]	41
6.1.3.40 ggLoadSimpleObj() [1/2]	42
6.1.3.41 ggLoadSimpleObj() [2/2]	43
6.1.3.42 ggLoadTexture()	44
6.1.3.43 ggLookat() [1/3]	45
6.1.3.44 ggLookat() [2/3]	46
6.1.3.45 ggLookat() [3/3]	46
6.1.3.46 ggMatrixQuaternion() [1/2]	47
6.1.3.47 ggMatrixQuaternion() [2/2]	48
6.1.3.48 ggNorm()	49
6.1.3.49 ggNormal()	49
6.1.3.50 ggNormalize()	50
6.1.3.51 ggNormalize3() [1/4]	50
6.1.3.52 ggNormalize3() [2/4]	51
6.1.3.53 ggNormalize3() [3/4]	51
6.1.3.54 ggNormalize3() [4/4]	52
6.1.3.55 ggNormalize4() [1/4]	52
6.1.3.56 ggNormalize4() [2/4]	53
6.1.3.57 ggNormalize4() [3/4]	54
6.1.3.58 ggNormalize4() [4/4]	54
6.1.3.59 ggOrthogonal()	55
6.1.3.60 ggPerspective()	56
6.1.3.61 ggPointsCube()	56
6.1.3.62 ggPointsSphere()	57
6.1.3.63 ggQuaternionMatrix()	57
6.1.3.64 ggQuaternionTransposeMatrix()	58
6.1.3.65 ggReadImage()	59
6.1.3.66 ggRectangle()	60

6.1.3.67 ggRotate() [1/5]	60
6.1.3.68 ggRotate() [2/5]	61
6.1.3.69 ggRotate() [3/5]	62
6.1.3.70 ggRotate() [4/5]	62
6.1.3.71 ggRotate() [5/5]	63
6.1.3.72 ggRotateQuaternion() [1/3]	64
6.1.3.73 ggRotateQuaternion() [2/3]	64
6.1.3.74 ggRotateQuaternion() [3/3]	65
6.1.3.75 ggRotateX()	65
6.1.3.76 ggRotateY()	66
6.1.3.77 ggRotateZ()	67
6.1.3.78 ggSaveColor()	68
6.1.3.79 ggSaveDepth()	69
6.1.3.80 ggSaveTga()	70
6.1.3.81 ggScale() [1/3]	71
6.1.3.82 ggScale() [2/3]	72
6.1.3.83 ggScale() [3/3]	72
6.1.3.84 ggSlerp() [1/4]	73
6.1.3.85 ggSlerp() [2/4]	74
6.1.3.86 ggSlerp() [3/4]	75
6.1.3.87 ggSlerp() [4/4]	76
6.1.3.88 ggTranslate() [1/3]	77
6.1.3.89 ggTranslate() [2/3]	78
6.1.3.90 ggTranslate() [3/3]	79
6.1.3.91 ggTranspose()	79
6.1.3.92 operator*()	80
6.1.3.93 operator+() [1/2]	80
6.1.3.94 operator+() [2/2]	81
6.1.3.95 operator-() [1/2]	81
6.1.3.96 operator-() [2/2]	81
6.1.3.97 operator/()	82
6.1.4 変数詳解	82
6.1.4.1 ggBufferAlignment	82
7 クラス詳解	83
7.1 GgApp クラス	83
7.1.1 詳解	83
7.1.2 構築子と解体子	83
7.1.2.1 GgApp() [1/3]	83
7.1.2.2 GgApp() [2/3]	84
7.1.2.3 GgApp() [3/3]	85
7.1.2.4 ~GgApp()	85

7.1.3 関数詳解	85
7.1.3.1 getUsername()	85
7.1.3.2 main()	85
7.1.3.3 operator=() [1/2]	86
7.1.3.4 operator=() [2/2]	86
7.2 gg::GgBuffer< T > クラステンプレート	87
7.2.1 詳解	88
7.2.2 構築子と解体子	88
7.2.2.1 GgBuffer() [1/3]	88
7.2.2.2 GgBuffer() [2/3]	89
7.2.2.3 GgBuffer() [3/3]	89
7.2.2.4 ~GgBuffer()	90
7.2.3 関数詳解	90
7.2.3.1 bind()	90
7.2.3.2 copy()	90
7.2.3.3 getBuffer()	91
7.2.3.4 getCount()	91
7.2.3.5 getStride()	92
7.2.3.6 getTarget()	92
7.2.3.7 map() [1/2]	93
7.2.3.8 map() [2/2]	93
7.2.3.9 operator=()	94
7.2.3.10 read()	94
7.2.3.11 send()	95
7.2.3.12 unbind()	96
7.2.3.13 unmap()	96
7.3 gg::GgColorTexture クラス	96
7.3.1 詳解	97
7.3.2 構築子と解体子	97
7.3.2.1 GgColorTexture() [1/3]	97
7.3.2.2 GgColorTexture() [2/3]	97
7.3.2.3 GgColorTexture() [3/3]	98
7.3.2.4 ~GgColorTexture()	98
7.3.3 関数詳解	99
7.3.3.1 bind()	99
7.3.3.2 get()	99
7.3.3.3 getHeight()	99
7.3.3.4 getSize() [1/2]	99
7.3.3.5 getSize() [2/2]	99
7.3.3.6 getTexture()	100
7.3.3.7 getWidth()	100
7.3.3.8 load() [1/2]	100

7.3.3.9 load() [2/2]	101
7.3.3.10 operator bool()	102
7.3.3.11 operator"!()	102
7.3.3.12 unbind()	102
7.4 gg::GgElements クラス	103
7.4.1 詳解	104
7.4.2 構築子と解体子	104
7.4.2.1 GgElements() [1/2]	104
7.4.2.2 GgElements() [2/2]	105
7.4.2.3 ~GgElements()	106
7.4.3 関数詳解	106
7.4.3.1 draw()	106
7.4.3.2 getIndexBuffer()	107
7.4.3.3 getIndexCount()	107
7.4.3.4 load()	107
7.4.3.5 send()	108
7.5 gg::GgMatrix クラス	109
7.5.1 詳解	112
7.5.2 構築子と解体子	112
7.5.2.1 GgMatrix() [1/6]	112
7.5.2.2 GgMatrix() [2/6]	112
7.5.2.3 GgMatrix() [3/6]	113
7.5.2.4 GgMatrix() [4/6]	113
7.5.2.5 GgMatrix() [5/6]	114
7.5.2.6 GgMatrix() [6/6]	114
7.5.2.7 ~GgMatrix()	115
7.5.3 関数詳解	115
7.5.3.1 frustum()	115
7.5.3.2 get() [1/2]	116
7.5.3.3 get() [2/2]	118
7.5.3.4 invert()	118
7.5.3.5 loadFrustum()	119
7.5.3.6 loadIdentity()	119
7.5.3.7 loadInvert() [1/2]	120
7.5.3.8 loadInvert() [2/2]	120
7.5.3.9 loadLookat() [1/3]	121
7.5.3.10 loadLookat() [2/3]	122
7.5.3.11 loadLookat() [3/3]	122
7.5.3.12 loadNormal() [1/2]	123
7.5.3.13 loadNormal() [2/2]	124
7.5.3.14 loadOrthogonal()	125
7.5.3.15 loadPerspective()	125

7.5.3.16 loadRotate() [1/5]	126
7.5.3.17 loadRotate() [2/5]	127
7.5.3.18 loadRotate() [3/5]	128
7.5.3.19 loadRotate() [4/5]	129
7.5.3.20 loadRotate() [5/5]	130
7.5.3.21 loadRotateX()	131
7.5.3.22 loadRotateY()	132
7.5.3.23 loadRotateZ()	132
7.5.3.24 loadScale() [1/3]	133
7.5.3.25 loadScale() [2/3]	134
7.5.3.26 loadScale() [3/3]	134
7.5.3.27 loadTranslate() [1/3]	135
7.5.3.28 loadTranslate() [2/3]	136
7.5.3.29 loadTranslate() [3/3]	136
7.5.3.30 loadTranspose() [1/2]	137
7.5.3.31 loadTranspose() [2/2]	138
7.5.3.32 lookat() [1/3]	138
7.5.3.33 lookat() [2/3]	139
7.5.3.34 lookat() [3/3]	140
7.5.3.35 normal()	141
7.5.3.36 operator*() [1/3]	142
7.5.3.37 operator*() [2/3]	143
7.5.3.38 operator*() [3/3]	144
7.5.3.39 operator*=() [1/2]	145
7.5.3.40 operator*=() [2/2]	146
7.5.3.41 operator+() [1/2]	147
7.5.3.42 operator+() [2/2]	148
7.5.3.43 operator+=() [1/2]	149
7.5.3.44 operator+=() [2/2]	149
7.5.3.45 operator-() [1/2]	150
7.5.3.46 operator-() [2/2]	151
7.5.3.47 operator-=() [1/2]	152
7.5.3.48 operator-=() [2/2]	152
7.5.3.49 operator/() [1/2]	153
7.5.3.50 operator/() [2/2]	154
7.5.3.51 operator/=() [1/2]	154
7.5.3.52 operator/=() [2/2]	155
7.5.3.53 operator=() [1/3]	156
7.5.3.54 operator=() [2/3]	156
7.5.3.55 operator=() [3/3]	157
7.5.3.56 orthogonal()	157
7.5.3.57 perspective()	158

7.5.3.58 projection() [1/4]	159
7.5.3.59 projection() [2/4]	159
7.5.3.60 projection() [3/4]	159
7.5.3.61 projection() [4/4]	159
7.5.3.62 rotate() [1/5]	160
7.5.3.63 rotate() [2/5]	160
7.5.3.64 rotate() [3/5]	161
7.5.3.65 rotate() [4/5]	161
7.5.3.66 rotate() [5/5]	162
7.5.3.67 rotateX()	163
7.5.3.68 rotateY()	164
7.5.3.69 rotateZ()	164
7.5.3.70 scale() [1/3]	165
7.5.3.71 scale() [2/3]	166
7.5.3.72 scale() [3/3]	167
7.5.3.73 translate() [1/3]	168
7.5.3.74 translate() [2/3]	169
7.5.3.75 translate() [3/3]	169
7.5.3.76 transpose()	170
7.6 gg::GgNormalTexture クラス	171
7.6.1 詳解	171
7.6.2 構築子と解体子	172
7.6.2.1 GgNormalTexture() [1/3]	172
7.6.2.2 GgNormalTexture() [2/3]	172
7.6.2.3 GgNormalTexture() [3/3]	172
7.6.2.4 ~GgNormalTexture()	173
7.6.3 関数詳解	173
7.6.3.1 bind()	173
7.6.3.2 get()	173
7.6.3.3 getHeight()	174
7.6.3.4 getSize() [1/2]	174
7.6.3.5 getSize() [2/2]	174
7.6.3.6 getTexture()	174
7.6.3.7 getWidth()	175
7.6.3.8 load() [1/2]	175
7.6.3.9 load() [2/2]	176
7.6.3.10 operator bool()	176
7.6.3.11 operator"!()"	177
7.6.3.12 unbind()	177
7.7 gg::GgPoints クラス	177
7.7.1 詳解	178
7.7.2 構築子と解体子	179

7.7.2.1 GgPoints() [1/2]	179
7.7.2.2 GgPoints() [2/2]	179
7.7.2.3 ~GgPoints()	180
7.7.3 関数詳解	180
7.7.3.1 draw()	180
7.7.3.2 getBuffer()	181
7.7.3.3 getCount()	181
7.7.3.4 load()	182
7.7.3.5 operator bool()	182
7.7.3.6 operator"!()	182
7.7.3.7 send()	183
7.8 gg::GgPointShader クラス	183
7.8.1 詳解	184
7.8.2 構築子と解体子	184
7.8.2.1 GgPointShader() [1/3]	184
7.8.2.2 GgPointShader() [2/3]	185
7.8.2.3 GgPointShader() [3/3]	186
7.8.2.4 ~GgPointShader()	186
7.8.3 関数詳解	187
7.8.3.1 get()	187
7.8.3.2 load() [1/2]	187
7.8.3.3 load() [2/2]	188
7.8.3.4 loadMatrix() [1/2]	189
7.8.3.5 loadMatrix() [2/2]	190
7.8.3.6 loadModelviewMatrix() [1/2]	191
7.8.3.7 loadModelviewMatrix() [2/2]	192
7.8.3.8 loadProjectionMatrix() [1/2]	193
7.8.3.9 loadProjectionMatrix() [2/2]	193
7.8.3.10 unuse()	194
7.8.3.11 use() [1/5]	194
7.8.3.12 use() [2/5]	195
7.8.3.13 use() [3/5]	196
7.8.3.14 use() [4/5]	196
7.8.3.15 use() [5/5]	197
7.9 gg::GgQuaternion クラス	198
7.9.1 詳解	201
7.9.2 構築子と解体子	201
7.9.2.1 GgQuaternion() [1/7]	201
7.9.2.2 GgQuaternion() [2/7]	202
7.9.2.3 GgQuaternion() [3/7]	203
7.9.2.4 GgQuaternion() [4/7]	203
7.9.2.5 GgQuaternion() [5/7]	204

7.9.2.6 GgQuaternion() [6/7]	204
7.9.2.7 GgQuaternion() [7/7]	205
7.9.2.8 ~GgQuaternion()	205
7.9.3 関数詳解	205
7.9.3.1 add() [1/4]	205
7.9.3.2 add() [2/4]	206
7.9.3.3 add() [3/4]	206
7.9.3.4 add() [4/4]	207
7.9.3.5 conjugate()	208
7.9.3.6 divide() [1/4]	209
7.9.3.7 divide() [2/4]	209
7.9.3.8 divide() [3/4]	210
7.9.3.9 divide() [4/4]	211
7.9.3.10 euler() [1/3]	212
7.9.3.11 euler() [2/3]	212
7.9.3.12 euler() [3/3]	213
7.9.3.13 get()	214
7.9.3.14 getConjugateMatrix() [1/3]	214
7.9.3.15 getConjugateMatrix() [2/3]	215
7.9.3.16 getConjugateMatrix() [3/3]	215
7.9.3.17 getMatrix() [1/3]	216
7.9.3.18 getMatrix() [2/3]	217
7.9.3.19 getMatrix() [3/3]	218
7.9.3.20 invert()	219
7.9.3.21 loadAdd() [1/4]	219
7.9.3.22 loadAdd() [2/4]	220
7.9.3.23 loadAdd() [3/4]	220
7.9.3.24 loadAdd() [4/4]	221
7.9.3.25 loadConjugate() [1/2]	222
7.9.3.26 loadConjugate() [2/2]	223
7.9.3.27 loadDivide() [1/4]	224
7.9.3.28 loadDivide() [2/4]	225
7.9.3.29 loadDivide() [3/4]	226
7.9.3.30 loadDivide() [4/4]	227
7.9.3.31 loadEuler() [1/2]	228
7.9.3.32 loadEuler() [2/2]	229
7.9.3.33 loadIdentity()	230
7.9.3.34 loadInvert() [1/2]	230
7.9.3.35 loadInvert() [2/2]	231
7.9.3.36 loadMatrix() [1/2]	232
7.9.3.37 loadMatrix() [2/2]	232
7.9.3.38 loadMultiply() [1/4]	233

7.9.3.39 loadMultiply() [2/4]	234
7.9.3.40 loadMultiply() [3/4]	234
7.9.3.41 loadMultiply() [4/4]	235
7.9.3.42 loadNormalize() [1/2]	236
7.9.3.43 loadNormalize() [2/2]	237
7.9.3.44 loadRotate() [1/3]	238
7.9.3.45 loadRotate() [2/3]	238
7.9.3.46 loadRotate() [3/3]	239
7.9.3.47 loadRotateX()	240
7.9.3.48 loadRotateY()	240
7.9.3.49 loadRotateZ()	241
7.9.3.50 loadSlerp() [1/4]	242
7.9.3.51 loadSlerp() [2/4]	243
7.9.3.52 loadSlerp() [3/4]	243
7.9.3.53 loadSlerp() [4/4]	244
7.9.3.54 loadSubtract() [1/4]	245
7.9.3.55 loadSubtract() [2/4]	245
7.9.3.56 loadSubtract() [3/4]	246
7.9.3.57 loadSubtract() [4/4]	247
7.9.3.58 multiply() [1/4]	248
7.9.3.59 multiply() [2/4]	249
7.9.3.60 multiply() [3/4]	250
7.9.3.61 multiply() [4/4]	251
7.9.3.62 norm()	252
7.9.3.63 normalize()	252
7.9.3.64 operator*() [1/3]	253
7.9.3.65 operator*() [2/3]	254
7.9.3.66 operator*() [3/3]	254
7.9.3.67 operator*=() [1/3]	255
7.9.3.68 operator*=() [2/3]	255
7.9.3.69 operator*=() [3/3]	255
7.9.3.70 operator+() [1/3]	256
7.9.3.71 operator+() [2/3]	256
7.9.3.72 operator+() [3/3]	257
7.9.3.73 operator+=() [1/3]	257
7.9.3.74 operator+=() [2/3]	258
7.9.3.75 operator+=() [3/3]	258
7.9.3.76 operator-() [1/3]	258
7.9.3.77 operator-() [2/3]	259
7.9.3.78 operator-() [3/3]	259
7.9.3.79 operator-=() [1/3]	260
7.9.3.80 operator-=() [2/3]	260

7.9.3.81 operator-=() [3 / 3]	260
7.9.3.82 operator/() [1 / 3]	261
7.9.3.83 operator/() [2 / 3]	261
7.9.3.84 operator/() [3 / 3]	262
7.9.3.85 operator/=() [1 / 3]	262
7.9.3.86 operator/=() [2 / 3]	263
7.9.3.87 operator/=() [3 / 3]	263
7.9.3.88 operator=() [1 / 4]	263
7.9.3.89 operator=() [2 / 4]	264
7.9.3.90 operator=() [3 / 4]	265
7.9.3.91 operator=() [4 / 4]	265
7.9.3.92 rotate() [1 / 3]	265
7.9.3.93 rotate() [2 / 3]	266
7.9.3.94 rotate() [3 / 3]	267
7.9.3.95 rotateX()	268
7.9.3.96 rotateY()	268
7.9.3.97 rotateZ()	269
7.9.3.98 slerp() [1 / 2]	269
7.9.3.99 slerp() [2 / 2]	270
7.9.3.100 subtract() [1 / 4]	270
7.9.3.101 subtract() [2 / 4]	271
7.9.3.102 subtract() [3 / 4]	271
7.9.3.103 subtract() [4 / 4]	272
7.10 gg::GgShader クラス	273
7.10.1 詳解	273
7.10.2 構築子と解体子	273
7.10.2.1 GgShader() [1 / 4]	273
7.10.2.2 GgShader() [2 / 4]	274
7.10.2.3 GgShader() [3 / 4]	275
7.10.2.4 GgShader() [4 / 4]	275
7.10.2.5 ~GgShader()	276
7.10.3 関数詳解	276
7.10.3.1 get()	276
7.10.3.2 operator=() [1 / 2]	276
7.10.3.3 operator=() [2 / 2]	277
7.10.3.4 unuse()	277
7.10.3.5 use()	278
7.11 gg::GgShape クラス	278
7.11.1 詳解	279
7.11.2 構築子と解体子	279
7.11.2.1 GgShape()	279
7.11.2.2 ~GgShape()	279

7.11.3 関数詳解	279
7.11.3.1 draw()	279
7.11.3.2 get()	280
7.11.3.3 getMode()	281
7.11.3.4 operator bool()	281
7.11.3.5 operator"!()	281
7.11.3.6 setMode()	282
7.12 gg::GgSimpleObj クラス	282
7.12.1 詳解	282
7.12.2 構築子と解体子	282
7.12.2.1 GgSimpleObj()	282
7.12.2.2 ~GgSimpleObj()	283
7.12.3 関数詳解	283
7.12.3.1 draw()	283
7.12.3.2 get()	284
7.12.3.3 operator bool()	284
7.12.3.4 operator"!()	284
7.13 gg::GgSimpleShader クラス	285
7.13.1 詳解	286
7.13.2 構築子と解体子	287
7.13.2.1 GgSimpleShader() [1/4]	287
7.13.2.2 GgSimpleShader() [2/4]	287
7.13.2.3 GgSimpleShader() [3/4]	288
7.13.2.4 GgSimpleShader() [4/4]	289
7.13.2.5 ~GgSimpleShader()	289
7.13.3 関数詳解	289
7.13.3.1 load() [1/2]	289
7.13.3.2 load() [2/2]	290
7.13.3.3 loadMatrix() [1/4]	291
7.13.3.4 loadMatrix() [2/4]	292
7.13.3.5 loadMatrix() [3/4]	292
7.13.3.6 loadMatrix() [4/4]	293
7.13.3.7 loadModelviewMatrix() [1/4]	294
7.13.3.8 loadModelviewMatrix() [2/4]	295
7.13.3.9 loadModelviewMatrix() [3/4]	295
7.13.3.10 loadModelviewMatrix() [4/4]	296
7.13.3.11 operator=()	297
7.13.3.12 use() [1/13]	298
7.13.3.13 use() [2/13]	299
7.13.3.14 use() [3/13]	300
7.13.3.15 use() [4/13]	301
7.13.3.16 use() [5/13]	301

7.13.3.17 use() [6/13]	302
7.13.3.18 use() [7/13]	303
7.13.3.19 use() [8/13]	304
7.13.3.20 use() [9/13]	305
7.13.3.21 use() [10/13]	305
7.13.3.22 use() [11/13]	306
7.13.3.23 use() [12/13]	306
7.13.3.24 use() [13/13]	307
7.14 gg::GgTexture クラス	308
7.14.1 詳解	308
7.14.2 構築子と解体子	308
7.14.2.1 GgTexture() [1/3]	308
7.14.2.2 GgTexture() [2/3]	309
7.14.2.3 GgTexture() [3/3]	310
7.14.2.4 ~GgTexture()	310
7.14.3 関数詳解	311
7.14.3.1 bind()	311
7.14.3.2 getHeight()	311
7.14.3.3 getSize() [1/2]	311
7.14.3.4 getSize() [2/2]	311
7.14.3.5 getTexture()	312
7.14.3.6 getWidth()	312
7.14.3.7 operator=() [1/2]	313
7.14.3.8 operator=() [2/2]	313
7.14.3.9 swapRandB()	314
7.14.3.10 unbind()	314
7.15 gg::GgTrackball クラス	314
7.15.1 詳解	318
7.15.2 構築子と解体子	318
7.15.2.1 GgTrackball()	318
7.15.2.2 ~GgTrackball()	319
7.15.3 関数詳解	319
7.15.3.1 begin()	319
7.15.3.2 end()	320
7.15.3.3 get()	320
7.15.3.4 getMatrix()	320
7.15.3.5 getQuaternion()	321
7.15.3.6 getScale() [1/3]	321
7.15.3.7 getScale() [2/3]	321
7.15.3.8 getScale() [3/3]	321
7.15.3.9 getStart() [1/3]	322
7.15.3.10 getStart() [2/3]	322

7.15.3.11	getStart() [3/3]	322
7.15.3.12	motion()	322
7.15.3.13	operator=()	323
7.15.3.14	region() [1/2]	324
7.15.3.15	region() [2/2]	324
7.15.3.16	reset()	325
7.15.3.17	rotate()	326
7.16	gg::GgTriangles クラス	326
7.16.1	詳解	327
7.16.2	構築子と解体子	327
7.16.2.1	GgTriangles() [1/2]	327
7.16.2.2	GgTriangles() [2/2]	328
7.16.2.3	~GgTriangles()	329
7.16.3	関数詳解	329
7.16.3.1	draw()	329
7.16.3.2	getBuffer()	330
7.16.3.3	getCount()	330
7.16.3.4	load()	330
7.16.3.5	send()	331
7.17	gg::GgUniformBuffer< T > クラステンプレート	332
7.17.1	詳解	332
7.17.2	構築子と解体子	332
7.17.2.1	GgUniformBuffer() [1/3]	332
7.17.2.2	GgUniformBuffer() [2/3]	333
7.17.2.3	GgUniformBuffer() [3/3]	334
7.17.2.4	~GgUniformBuffer()	335
7.17.3	関数詳解	335
7.17.3.1	bind()	335
7.17.3.2	copy()	336
7.17.3.3	fill()	336
7.17.3.4	getBuffer()	337
7.17.3.5	getCount()	338
7.17.3.6	getStride()	339
7.17.3.7	getTarget()	339
7.17.3.8	load() [1/2]	340
7.17.3.9	load() [2/2]	340
7.17.3.10	map() [1/2]	341
7.17.3.11	map() [2/2]	342
7.17.3.12	read()	342
7.17.3.13	send()	343
7.17.3.14	unbind()	344
7.17.3.15	unmap()	345

7.18 gg::GgVector クラス	345
7.18.1 詳解	346
7.18.2 構築子と解体子	346
7.18.2.1 GgVector() [1/6]	346
7.18.2.2 GgVector() [2/6]	347
7.18.2.3 GgVector() [3/6]	348
7.18.2.4 GgVector() [4/6]	348
7.18.2.5 GgVector() [5/6]	349
7.18.2.6 GgVector() [6/6]	349
7.18.2.7 ~GgVector()	350
7.18.3 関数詳解	350
7.18.3.1 distance3()	350
7.18.3.2 distance4()	351
7.18.3.3 dot3()	351
7.18.3.4 dot4()	352
7.18.3.5 length3()	353
7.18.3.6 length4()	353
7.18.3.7 normalize3()	354
7.18.3.8 normalize4()	354
7.18.3.9 operator*() [1/2]	354
7.18.3.10 operator*() [2/2]	355
7.18.3.11 operator*=() [1/2]	356
7.18.3.12 operator*=() [2/2]	357
7.18.3.13 operator+() [1/2]	358
7.18.3.14 operator+() [2/2]	359
7.18.3.15 operator+=() [1/2]	360
7.18.3.16 operator+=() [2/2]	361
7.18.3.17 operator-() [1/2]	362
7.18.3.18 operator-() [2/2]	363
7.18.3.19 operator-=() [1/2]	364
7.18.3.20 operator-=() [2/2]	365
7.18.3.21 operator/() [1/2]	366
7.18.3.22 operator/() [2/2]	367
7.18.3.23 operator/=() [1/2]	368
7.18.3.24 operator/=() [2/2]	369
7.18.3.25 operator=() [1/2]	370
7.18.3.26 operator=() [2/2]	370
7.19 gg::GgVertex 構造体	370
7.19.1 詳解	371
7.19.2 構築子と解体子	372
7.19.2.1 GgVertex() [1/4]	372
7.19.2.2 GgVertex() [2/4]	372

7.19.2.3 GgVertex() [3/4]	372
7.19.2.4 GgVertex() [4/4]	373
7.19.3 メンバ詳解	373
7.19.3.1 normal	373
7.19.3.2 position	374
7.20 gg::GgVertexArray クラス	374
7.20.1 詳解	374
7.20.2 構築子と解体子	374
7.20.2.1 GgVertexArray() [1/3]	374
7.20.2.2 GgVertexArray() [2/3]	375
7.20.2.3 GgVertexArray() [3/3]	376
7.20.2.4 ~GgVertexArray()	377
7.20.3 関数詳解	377
7.20.3.1 bind()	377
7.20.3.2 get()	377
7.20.3.3 operator=() [1/2]	378
7.20.3.4 operator=() [2/2]	379
7.21 gg::GgSimpleShader::Light 構造体	380
7.21.1 詳解	380
7.21.2 メンバ詳解	381
7.21.2.1 ambient	381
7.21.2.2 diffuse	381
7.21.2.3 position	381
7.21.2.4 specular	381
7.22 gg::GgSimpleShader::LightBuffer クラス	382
7.22.1 詳解	383
7.22.2 構築子と解体子	383
7.22.2.1 LightBuffer() [1/3]	383
7.22.2.2 LightBuffer() [2/3]	384
7.22.2.3 LightBuffer() [3/3]	385
7.22.2.4 ~LightBuffer()	385
7.22.3 関数詳解	385
7.22.3.1 load() [1/2]	385
7.22.3.2 load() [2/2]	386
7.22.3.3 loadAmbient() [1/3]	387
7.22.3.4 loadAmbient() [2/3]	388
7.22.3.5 loadAmbient() [3/3]	388
7.22.3.6 loadColor()	389
7.22.3.7 loadDiffuse() [1/3]	390
7.22.3.8 loadDiffuse() [2/3]	391
7.22.3.9 loadDiffuse() [3/3]	392
7.22.3.10 loadPosition() [1/4]	392

7.22.3.11 loadPosition() [2/4]	393
7.22.3.12 loadPosition() [3/4]	394
7.22.3.13 loadPosition() [4/4]	395
7.22.3.14 loadSpecular() [1/3]	396
7.22.3.15 loadSpecular() [2/3]	396
7.22.3.16 loadSpecular() [3/3]	397
7.22.3.17 select()	398
7.23 gg::GgSimpleShader::Material 構造体	399
7.23.1 詳解	400
7.23.2 メンバ詳解	400
7.23.2.1 ambient	400
7.23.2.2 diffuse	400
7.23.2.3 shininess	400
7.23.2.4 specular	401
7.24 gg::GgSimpleShader::MaterialBuffer クラス	401
7.24.1 詳解	402
7.24.2 構築子と解体子	403
7.24.2.1 MaterialBuffer() [1/3]	403
7.24.2.2 MaterialBuffer() [2/3]	403
7.24.2.3 MaterialBuffer() [3/3]	404
7.24.2.4 ~MaterialBuffer()	405
7.24.3 関数詳解	405
7.24.3.1 load() [1/2]	405
7.24.3.2 load() [2/2]	405
7.24.3.3 loadAmbient() [1/3]	406
7.24.3.4 loadAmbient() [2/3]	407
7.24.3.5 loadAmbient() [3/3]	408
7.24.3.6 loadAmbientAndDiffuse() [1/3]	408
7.24.3.7 loadAmbientAndDiffuse() [2/3]	409
7.24.3.8 loadAmbientAndDiffuse() [3/3]	410
7.24.3.9 loadDiffuse() [1/3]	411
7.24.3.10 loadDiffuse() [2/3]	412
7.24.3.11 loadDiffuse() [3/3]	413
7.24.3.12 loadShininess() [1/2]	413
7.24.3.13 loadShininess() [2/2]	414
7.24.3.14 loadSpecular() [1/3]	415
7.24.3.15 loadSpecular() [2/3]	416
7.24.3.16 loadSpecular() [3/3]	417
7.24.3.17 select()	417
7.25 GgApp::Window クラス	418
7.25.1 詳解	420
7.25.2 構築子と解体子	420

7.25.2.1 Window() [1/3]	420
7.25.2.2 Window() [2/3]	421
7.25.2.3 Window() [3/3]	422
7.25.2.4 ~Window()	422
7.25.3 関数詳解	422
7.25.3.1 get()	422
7.25.3.2 getAltArrow()	422
7.25.3.3 getAltArrowX()	423
7.25.3.4 getAltArrowY()	424
7.25.3.5 getArrow() [1/2]	424
7.25.3.6 getArrow() [2/2]	425
7.25.3.7 getArrowX()	426
7.25.3.8 getArrowY()	426
7.25.3.9 getAspect()	427
7.25.3.10 getControlArrow()	427
7.25.3.11 getControlArrowX()	428
7.25.3.12 getControlArrowY()	429
7.25.3.13 getFboHeight()	429
7.25.3.14 getFboSize() [1/2]	430
7.25.3.15 getFboSize() [2/2]	430
7.25.3.16 getFboWidth()	430
7.25.3.17 getHeight()	431
7.25.3.18 getKey()	431
7.25.3.19 getLastKey()	432
7.25.3.20 getMouse() [1/3]	432
7.25.3.21 getMouse() [2/3]	432
7.25.3.22 getMouse() [3/3]	432
7.25.3.23 getMouseX()	433
7.25.3.24 getMouseY()	433
7.25.3.25 getRotation()	433
7.25.3.26 getRotationMatrix()	433
7.25.3.27 getScrollMatrix()	434
7.25.3.28 getShiftArrow()	434
7.25.3.29 getShiftArrowX()	435
7.25.3.30 getShiftArrowY()	435
7.25.3.31 getSize() [1/2]	436
7.25.3.32 getSize() [2/2]	436
7.25.3.33 getTranslation()	437
7.25.3.34 getTranslationMatrix()	438
7.25.3.35 getUserPointer()	438
7.25.3.36 getWheel() [1/3]	438
7.25.3.37 getWheel() [2/3]	438

7.25.3.38	getWheel() [3/3]	439
7.25.3.39	getWheelX()	439
7.25.3.40	getWheelY()	439
7.25.3.41	getWidth()	440
7.25.3.42	operator bool()	440
7.25.3.43	operator=() [1/2]	440
7.25.3.44	operator=() [2/2]	441
7.25.3.45	reset()	442
7.25.3.46	resetRotation()	442
7.25.3.47	resetTranslation()	442
7.25.3.48	restoreViewport()	443
7.25.3.49	selectInterface()	443
7.25.3.50	setClose()	443
7.25.3.51	setKeyboardFunc()	443
7.25.3.52	setMouseFunc()	444
7.25.3.53	setResizeFunc()	444
7.25.3.54	setUserPointer()	445
7.25.3.55	setVelocity()	445
7.25.3.56	setWheelFunc()	445
7.25.3.57	shouldClose()	446
7.25.3.58	swapBuffers()	446
7.25.3.59	updateViewport()	447
8	ファイル詳解	449
8.1	gg.cpp ファイル	449
8.1.1	詳解	449
8.2	gg.cpp	450
8.3	gg.h ファイル	526
8.3.1	詳解	530
8.3.2	マクロ定義詳解	531
8.3.2.1	ggError	531
8.3.2.2	ggFBOError	531
8.3.3	型定義詳解	531
8.3.3.1	pathChar	531
8.3.3.2	pathString	531
8.3.4	関数詳解	531
8.3.4.1	TCharToUtf8()	531
8.3.4.2	Utf8ToTChar()	532
8.4	gg.h	532
8.5	GgApp.cpp ファイル	589
8.5.1	詳解	589
8.6	GgApp.cpp	590

8.7 GgApp.h ファイル	613
8.7.1 詳解	614
8.7.2 マクロ定義詳解	615
8.7.2.1 GG_BUTTON_COUNT	615
8.7.2.2 GG_INTERFACE_COUNT	615
8.8 GgApp.h	615
Index	627

Chapter 1

ゲームグラフィックス特論の宿題用補助 プログラム **GLFW3** 版.

著作権所有

Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies or substantial portions of the Software.

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

Chapter 2

名前空間索引

2.1 名前空間一覧

全名前空間の一覧です。

99		11
----	-------	----

Chapter 3

階層索引

3.1 クラス階層

クラス階層一覧です。大雑把に文字符号順で並べられています。

std::array	
gg::GgMatrix	109
gg::GgVector	345
gg::GgQuaternion	198
gg::GgTrackball	314
GgApp	83
gg::GgBuffer< T >	87
gg::GgColorTexture	96
gg::GgNormalTexture	171
gg::GgPointShader	183
gg::GgSimpleShader	285
gg::GgShader	273
gg::GgShape	278
gg::GgPoints	177
gg::GgTriangles	326
gg::GgElements	103
gg::GgSimpleObj	282
gg::GgTexture	308
gg::GgUniformBuffer< T >	332
gg::GgUniformBuffer< Light >	332
gg::GgSimpleShader::LightBuffer	382
gg::GgUniformBuffer< Material >	332
gg::GgSimpleShader::MaterialBuffer	401
gg::GgVertex	370
gg::GgVertexArray	374
gg::GgSimpleShader::Light	380
gg::GgSimpleShader::Material	399
GgApp::Window	418

Chapter 4

クラス索引

4.1 クラス一覧

クラス・構造体・共用体・インターフェースの一覧です。

GgApp	83
gg::GgBuffer< T >	87
gg::GgColorTexture	96
gg::GgElements	103
gg::GgMatrix	109
gg::GgNormalTexture	171
gg::GgPoints	177
gg::GgPointShader	183
gg::GgQuaternion	198
gg::GgShader	273
gg::GgShape	278
gg::GgSimpleObj	282
gg::GgSimpleShader	285
gg::GgTexture	308
gg::GgTrackball	314
gg::GgTriangles	326
gg::GgUniformBuffer< T >	332
gg::GgVector	345
gg::GgVertex	370
gg::GgVertexArray	374
gg::GgSimpleShader::Light	380
gg::GgSimpleShader::LightBuffer	382
gg::GgSimpleShader::Material	399
gg::GgSimpleShader::MaterialBuffer	401
GgApp::Window	418

Chapter 5

ファイル索引

5.1 ファイル一覧

ファイル一覧です。

gg.cpp	449
gg.h	526
GgApp.cpp	589
GgApp.h	613

Chapter 6

名前空間詳解

6.1 gg 名前空間

クラス

- class [GgVector](#)
- class [GgMatrix](#)
- class [GgQuaternion](#)
- class [GgTrackball](#)
- class [GgTexture](#)
- class [GgColorTexture](#)
- class [GgNormalTexture](#)
- class [GgBuffer](#)
- class [GgUniformBuffer](#)
- class [GgVertexArray](#)
- class [GgShape](#)
- class [GgPoints](#)
- struct [GgVertex](#)
- class [GgTriangles](#)
- class [GgElements](#)
- class [GgShader](#)
- class [GgPointShader](#)
- class [GgSimpleShader](#)
- class [GgSimpleObj](#)

列挙型

- enum [BindingPoints](#) { [LightBindingPoint](#) = 0 , [MaterialBindingPoint](#) }

関数

- void `ggInit` ()
- void `_ggError` (const std::string &name="", unsigned int line=0)
- void `_ggFBOError` (const std::string &name="", unsigned int line=0)
- void `ggCross` (GLfloat *c, const GLfloat *a, const GLfloat *b)
- GLfloat `ggDot3` (const GLfloat *a, const GLfloat *b)
- GLfloat `ggLength3` (const GLfloat *a)
- GLfloat `ggDistance3` (const GLfloat *a, const GLfloat *b)
- void `ggNormalize3` (const GLfloat *a, GLfloat *b)
- void `ggNormalize3` (GLfloat *a)
- GLfloat `ggDot4` (const GLfloat *a, const GLfloat *b)
- GLfloat `ggLength4` (const GLfloat *a)
- GLfloat `ggDistance4` (const GLfloat *a, const GLfloat *b)
- void `ggNormalize4` (const GLfloat *a, GLfloat *b)
- void `ggNormalize4` (GLfloat *a)
- const `GgVector` & `operator+` (const `GgVector` &v)
- `GgVector` `operator+` (GLfloat a, const `GgVector` &b)
- const `GgVector` `operator-` (const `GgVector` &v)
- `GgVector` `operator-` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator*` (GLfloat a, const `GgVector` &b)
- `GgVector` `operator/` (GLfloat a, const `GgVector` &b)
- `GgVector` `ggCross` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggDot3` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength3` (const `GgVector` &a)
- GLfloat `ggDistance3` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize3` (const `GgVector` &a)
- void `ggNormalize3` (`GgVector` *a)
- GLfloat `ggDot4` (const `GgVector` &a, const `GgVector` &b)
- GLfloat `ggLength4` (const `GgVector` &a)
- GLfloat `ggDistance4` (const `GgVector` &a, const `GgVector` &b)
- `GgVector` `ggNormalize4` (const `GgVector` &a)
- void `ggNormalize4` (`GgVector` *a)
- `GgMatrix` `ggIdentity` ()
- `GgMatrix` `ggTranslate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggTranslate` (const GLfloat *t)
- `GgMatrix` `ggTranslate` (const `GgVector` &t)
- `GgMatrix` `ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix` `ggScale` (const GLfloat *s)
- `GgMatrix` `ggScale` (const `GgVector` &s)
- `GgMatrix` `ggRotateX` (GLfloat a)
- `GgMatrix` `ggRotateY` (GLfloat a)
- `GgMatrix` `ggRotateZ` (GLfloat a)
- `GgMatrix` `ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix` `ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix` `ggRotate` (const GLfloat *r)
- `GgMatrix` `ggRotate` (const `GgVector` &r)
- `GgMatrix` `ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix` `ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix` `ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix` `ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix` `ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)

- [GgMatrix ggTranspose](#) (const [GgMatrix](#) &m)
- [GgMatrix ggInvert](#) (const [GgMatrix](#) &m)
- [GgMatrix ggNormal](#) (const [GgMatrix](#) &m)
- [GgQuaternion ggIdentityQuaternion](#) ()
- [GgQuaternion ggMatrixQuaternion](#) (const GLfloat *a)
- [GgQuaternion ggMatrixQuaternion](#) (const [GgMatrix](#) &m)
- [GgMatrix ggQuaternionMatrix](#) (const [GgQuaternion](#) &q)
- [GgMatrix ggQuaternionTransposeMatrix](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggRotateQuaternion](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgQuaternion ggRotateQuaternion](#) (const GLfloat *v, GLfloat a)
- [GgQuaternion ggRotateQuaternion](#) (const GLfloat *v)
- [GgQuaternion ggEulerQuaternion](#) (GLfloat heading, GLfloat pitch, GLfloat roll)
- [GgQuaternion ggEulerQuaternion](#) (const GLfloat *e)
- [GgQuaternion ggEulerQuaternion](#) (const [GgVector](#) &e)
- [GgQuaternion ggSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion ggSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion ggSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion ggSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- GLfloat [ggNorm](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion ggInvert](#) (const [GgQuaternion](#) &q)
- bool [ggSaveTga](#) (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool [ggSaveColor](#) (const std::string &name)
- bool [ggSaveDepth](#) (const std::string &name)
- bool [ggReadImage](#) (const std::string &name, std::vector< GLubyte > &image, GLsizei *pWidth, GLsizei *pHeight, GLenum *pFormat)
- GLuint [ggLoadTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- GLuint [ggLoadImage](#) (const std::string &name, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE)
- void [ggCreateNormalMap](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format, GLfloat nz, GLenum internal, std::vector< [GgVector](#) > &nmap)
- GLuint [ggLoadHeight](#) (const std::string &name, GLfloat nz, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA)
- GLuint [ggCreateShader](#) (const std::string &vsrsrc, const std::string &fsrc="", const std::string &gsrsrc="", GLint nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std::string &ftext="fragment shader", const std::string >ext="geometry shader")
- GLuint [ggLoadShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [ggLoadShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [ggCreateComputeShader](#) (const std::string &csrsrc, const std::string &ctext="compute shader")
- GLuint [ggLoadComputeShader](#) (const std::string &comp)
- std::shared_ptr< [GgPoints](#) > [ggPointsCube](#) (GLsizei countv, GLfloat length=1.0f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgPoints](#) > [ggPointsSphere](#) (GLsizei countv, GLfloat radius=0.5f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgTriangles](#) > [ggRectangle](#) (GLfloat width=1.0f, GLfloat height=1.0f)
- std::shared_ptr< [GgTriangles](#) > [ggEllipse](#) (GLfloat width=1.0f, GLfloat height=1.0f, GLuint slices=16)
- std::shared_ptr< [GgTriangles](#) > [ggArraysObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [ggElementsObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [ggElementsMesh](#) (GLuint slices, GLuint stacks, const GLfloat(*pos)[3], const GLfloat(*norm)[3]=nullptr)

- `std::shared_ptr< GgElements > ggElementsSphere` (GLfloat radius=1.0f, int slices=16, int stacks=8)
- `bool ggLoadSimpleObj` (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< GgSimpleShader::Material > &material, std::vector< GgVertex > &vert, bool normalize=false)
- `bool ggLoadSimpleObj` (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< GgSimpleShader::Material > &material, std::vector< GgVertex > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint `ggBufferAlignment`
使用している GPU のバッファアライメント。

6.1.1 詳解

ゲームグラフィックス特論の宿題用補助プログラムの名前空間

6.1.2 列挙型詳解

6.1.2.1 BindingPoints

```
enum gg::BindingPoints
```

光源と材質の uniform buffer object の結合ポイント。

列挙値

LightBindingPoint	光源の uniform buffer object の結合ポイント。
MaterialBindingPoint	材質の uniform buffer object の結合ポイント。

`gg.h` の 1355 行目に定義があります。

6.1.3 関数詳解

6.1.3.1 _ggError()

```
void gg::_ggError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

OpenGL のエラーをチェックする。

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, nullptr なら何も出力しない。
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない。

覚え書き

OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する。

`gg.cpp` の 2609 行目に定義があります。

6.1.3.2 _ggFBOError()

```
void gg::_ggFBOError (
    const std::string & name = "",
    unsigned int line = 0) [extern]
```

FBO のエラーをチェックする.

引数

<i>name</i>	エラー発生時に標準エラー出力に出力する文字列, nullptr なら何も出力しない.
<i>line</i>	エラー発生時に標準エラー出力に出力する数値, 0 なら何も出力しない.

覚え書き

FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する.

[gg.cpp](#) の 2653 行目に定義があります。

6.1.3.3 ggArraysObj()

```
std::shared_ptr< gg::GgTriangles > gg::ggArraysObj (
    const std::string & name,
    bool normalize = false) [extern]
```

Wavefront OBJ ファイルを読み込む (Arrays 形式)

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

[GgTriangles](#) 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイルを読み込んで GgArrays 形式の三角形データを生成する.

[gg.cpp](#) の 5301 行目に定義があります。

呼び出し関係図:



6.1.3.4 ggConjugate()

```
GgQuaternion gg::ggConjugate (
    const GgQuaternion & q) [inline]
```

共役四元数を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* の共役四元数.

gg.h の 4787 行目に定義があります.

呼び出し関係図:



6.1.3.5 ggCreateComputeShader()

```
GLuint gg::ggCreateComputeShader (
    const std::string & csrc,
    const std::string & ctext = "compute shader") [extern]
```

コンピュータシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する.

引数

<i>csrc</i>	コンピュータシェーダのソースプログラムの文字列.
<i>ctext</i>	コンピュータシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 5072 行目に定義があります。

被呼び出し関係図:



6.1.3.6 ggCreateNormalMap()

```

void gg::ggCreateNormalMap (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format,
    GLfloat nz,
    GLenum internal,
    std::vector< GgVector > & nmap) [extern]
  
```

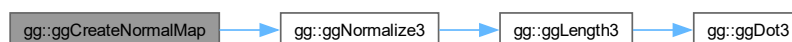
グレースケール画像 (8bit) から法線マップのデータを作成する。

引数

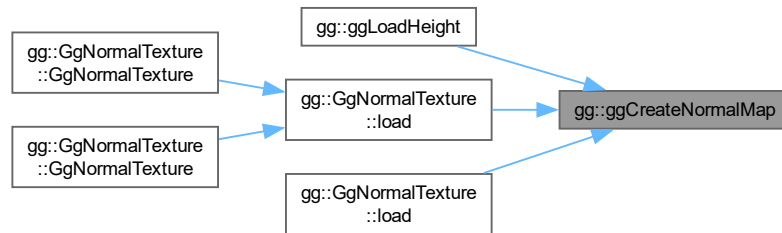
<i>hmap</i>	グレースケール画像のデータ.
<i>width</i>	高さマップのグレースケール画像 <i>hmap</i> の横の画素数.
<i>height</i>	高さマップのグレースケール画像 <i>hmap</i> の縦の画素数.
<i>format</i>	データの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線の z 成分の割合.
<i>internal</i>	法線マップを格納するテクスチャの内部フォーマット.
<i>nmap</i>	法線マップを格納する vector.

gg.cpp の 3896 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.7 ggCreateShader()

```

GLuint gg::ggCreateShader (
    const std::string & vsrc,
    const std::string & fsrc = "",
    const std::string & gsrc = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr,
    const std::string & vtext = "vertex shader",
    const std::string & ftext = "fragment shader",
    const std::string & gtext = "geometry shader") [extern]
  
```

シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する。

引数

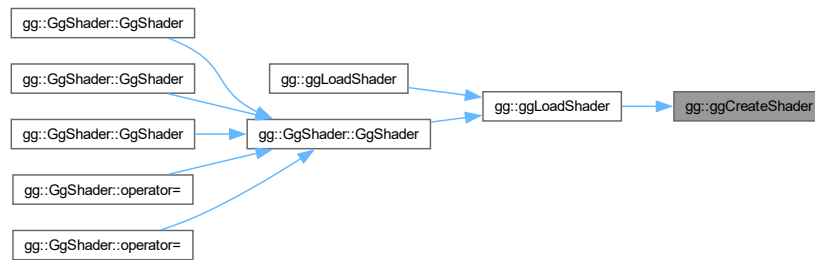
<i>vsrc</i>	バーテックスシェーダのソースプログラムの文字列.
<i>fsrc</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>gsrc</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).
<i>vtext</i>	バーテックスシェーダのコンパイル時のメッセージに追加する文字列.
<i>ftext</i>	フラグメントシェーダのコンパイル時のメッセージに追加する文字列.
<i>gtext</i>	ジオメトリシェーダのコンパイル時のメッセージに追加する文字列.

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

[gg.cpp](#) の 4896 行目に定義があります。

被呼び出し関係図:



6.1.3.8 ggCross() [1/2]

```
GgVector gg::ggCross (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 3 要素の外積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と *b* の外積.

覚え書き

戻り値の *w* (第4) 要素は 0.

gg.h の 2025 行目に定義があります。

呼び出し関係図:



6.1.3.9 ggCross() [2/2]

```
void gg::ggCross (
    GLfloat * c,
    const GLfloat * a,
    const GLfloat * b) [inline]
```

3 要素の外積.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	GLfloat 型の 3 要素の配列変数.
<i>c</i>	結果を格納する GLfloat 型の 3 要素の配列変数.

[gg.h](#) の 1435 行目に定義があります。

被呼び出し関係図:



6.1.3.10 ggDistance3() [1/2]

```
GLfloat gg::ggDistance3 (
    const GgVector & a,
    const GgVector & b) [inline]
```

[GgVector](#) 型の 3 要素の距離.

引数

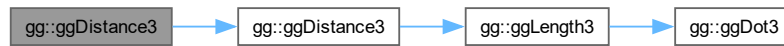
<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と b の距離.

gg.h の 2062 行目に定義があります。

呼び出し関係図:



6.1.3.11 ggDistance3() [2/2]

```
GLfloat gg::ggDistance3 (  
    const GLfloat * a,  
    const GLfloat * b) [inline]
```

3 要素の距離.

引数

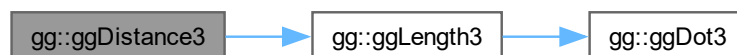
a	GLfloat 型の 3 要素の配列変数.
b	GLfloat 型の 3 要素の配列変数.

戻り値

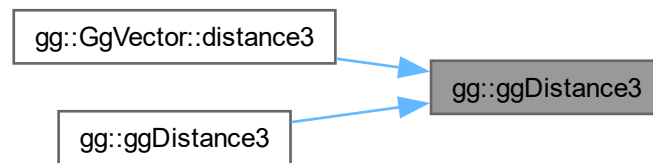
a と b の距離.

gg.h の 1472 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.12 ggDistance4() [1/2]

```

GLfloat gg::ggDistance4 (
    const GgVector & a,
    const GgVector & b) [inline]
  
```

GgVector 型の 4 要素の距離.

引数

<i>a</i>	GgVector 型の変数.
<i>b</i>	GgVector 型の変数.

戻り値

2 つの **GgVector** *a*, *b* の距離.

gg.h の 2126 行目に定義があります。

呼び出し関係図:



6.1.3.13 ggDistance4() [2/2]

```

GLfloat gg::ggDistance4 (
    const GLfloat * a,
    const GLfloat * b) [inline]
  
```

GLfloat 型の 4 要素の距離.

引数

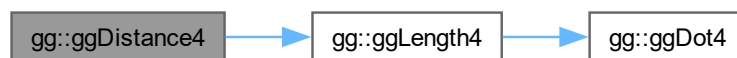
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	GLfloat 型の 4 要素の配列変数.

戻り値

a と *b* のそれぞれの 4 要素の距離.

gg.h の 1537 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.14 ggDot3() [1/2]

```

GLfloat gg::ggDot3 (
    const GgVector & a,
    const GgVector & b) [inline]
  
```

GgVector 型の 3 要素の内積.

引数

<i>a</i>	GgVector 型のベクトル.
<i>b</i>	GgVector 型のベクトル.

戻り値

a と b のそれぞれの 3 要素の内積.

gg.h の 2039 行目に定義があります。

呼び出し関係図:



6.1.3.15 ggDot3() [2/2]

```

GLfloat gg::ggDot3 (
    const GLfloat * a,
    const GLfloat * b) [inline]
  
```

3 要素の内積.

引数

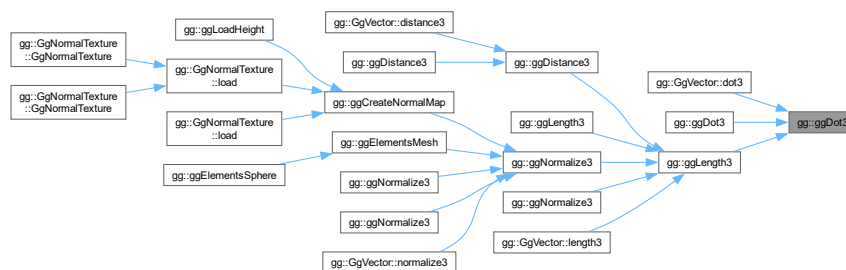
a	GLfloat 型の 3 要素の配列変数.
b	GLfloat 型の 3 要素の配列変数.

戻り値

a と b のそれぞれの 3 要素の内積.

gg.h の 1449 行目に定義があります。

被呼び出し関係図:



6.1.3.16 ggDot4() [1/2]

```
GLfloat gg::ggDot4 (
    const GgVector & a,
    const GgVector & b) [inline]
```

GgVector 型の 4 要素の内積.

引数

<i>a</i>	GgVector 型の変数.
<i>b</i>	GgVector 型の変数.

戻り値

a と *b* のそれぞれの 4 要素の内積.

gg.h の 2103 行目に定義があります。

呼び出し関係図:



6.1.3.17 ggDot4() [2/2]

```
GLfloat gg::ggDot4 (
    const GLfloat * a,
    const GLfloat * b) [inline]
```

GLfloat 型の 4 要素の内積

引数

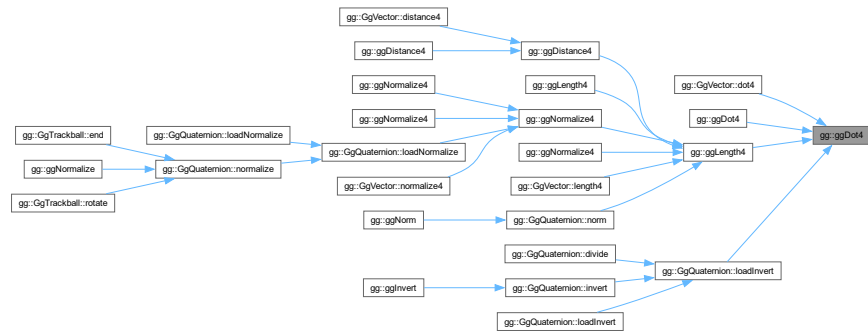
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	GLfloat 型の 4 要素の配列変数.

戻り値

a と b のそれぞれの 4 要素の内積.

gg.h の 1514 行目に定義があります.

被呼び出し関係図:



6.1.3.18 ggElementsMesh()

```
std::shared_ptr< gg::GgElements > gg::ggElementsMesh (
    GLuint slices,
    GLuint stacks,
    const GLfloat(*) pos[3],
    const GLfloat(*) norm[3] = nullptr) [extern]
```

メッシュ形状を作成する (Elements 形式).

引数

<i>slices</i>	メッシュの横方向の分割数.
<i>stacks</i>	メッシュの縦方向の分割数.
<i>pos</i>	メッシュの頂点の位置.
<i>norm</i>	メッシュの頂点の法線, <code>nullptr</code> なら頂点の位置から算出する.

戻り値

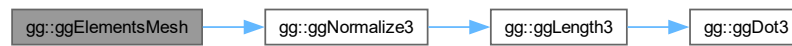
GgElements 型のポインタ.

覚え書き

メッシュ状に **GgElements** 形式の三角形データを生成する.

gg.cpp の 5335 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.19 ggElementsObj()

```

std::shared_ptr< gg::GgElements > gg::ggElementsObj (
    const std::string & name,
    bool normalize = false) [extern]
  
```

Wavefront OBJ ファイル を読み込む (Elements 形式).

引数

<i>name</i>	ファイル名.
<i>normalize</i>	true なら大きさを正規化.

戻り値

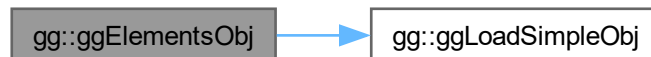
GgElements 型のポインタ.

覚え書き

三角形分割された Wavefront OBJ ファイル を読み込んで **GgElements** 形式の三角形データを生成する.

gg.cpp の 5317 行目に定義があります。

呼び出し関係図:



6.1.3.20 ggElementsSphere()

```

std::shared_ptr< gg::GgElements > gg::GgElementsSphere (
    GLfloat radius = 1.0f,
    int slices = 16,
    int stacks = 8) [extern]
  
```

球状に三角形データを生成する (Elements 形式).

引数

<i>radius</i>	球の半径.
<i>slices</i>	球の経度方向の分割数.
<i>stacks</i>	球の緯度方向の分割数.

戻り値

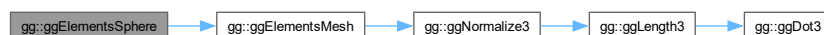
GgElements 型のポインタ.

覚え書き

球状に **GgElements** 形式の三角形データを生成する.

gg.cpp の 5430 行目に定義があります。

呼び出し関係図:



6.1.3.21 ggEllipse()

```
std::shared_ptr< gg::GgTriangles > gg::ggEllipse (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f,
    GLuint slices = 16) [extern]
```

楕円状に三角形を生成する。

引数

<i>width</i>	楕円の横幅.
<i>height</i>	楕円の高さ.
<i>slices</i>	楕円の分割数.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5276 行目に定義があります。

6.1.3.22 ggEulerQuaternion() [1/3]

```
GgQuaternion gg::ggEulerQuaternion (
    const GgVector & e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す。

引数

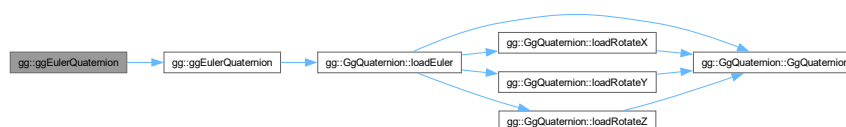
<i>e</i>	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
----------	--

戻り値

回転を表す四元数.

[gg.h](#) の 4701 行目に定義があります。

呼び出し関係図:



6.1.3.23 ggEulerQuaternion() [2/3]

```
GgQuaternion gg::ggEulerQuaternion (  
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を返す.

引数

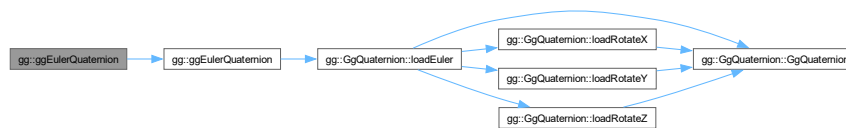
<i>e</i>	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

回転を表す四元数.

gg.h の 4690 行目に定義があります。

呼び出し関係図:



6.1.3.24 ggEulerQuaternion() [3/3]

```

GgQuaternion gg::ggEulerQuaternion (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) [inline]
  
```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を返す.

引数

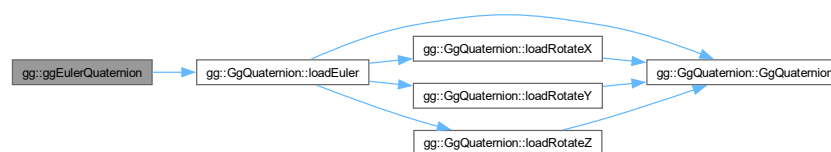
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

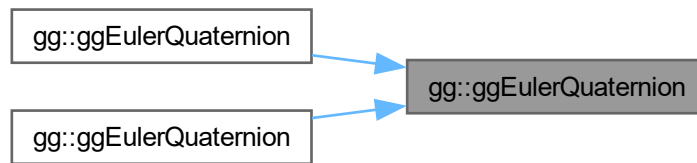
回転を表す四元数.

gg.h の 4678 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.25 ggFrustum()

```
GgMatrix gg::ggFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

透視投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3419 行目に定義があります。

呼び出し関係図:



6.1.3.26 ggIdentity()

```
GgMatrix gg::ggIdentity () [inline]
```

単位行列を返す.

戻り値

単位行列.

gg.h の 3144 行目に定義があります。

呼び出し関係図:



6.1.3.27 ggIdentityQuaternion()

```
GgQuaternion gg::ggIdentityQuaternion () [inline]
```

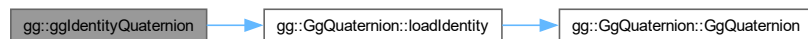
単位四元数を返す.

戻り値

単位四元数.

gg.h の 4576 行目に定義があります。

呼び出し関係図:



6.1.3.28 ggInit()

```
void gg::ggInit () [extern]
```

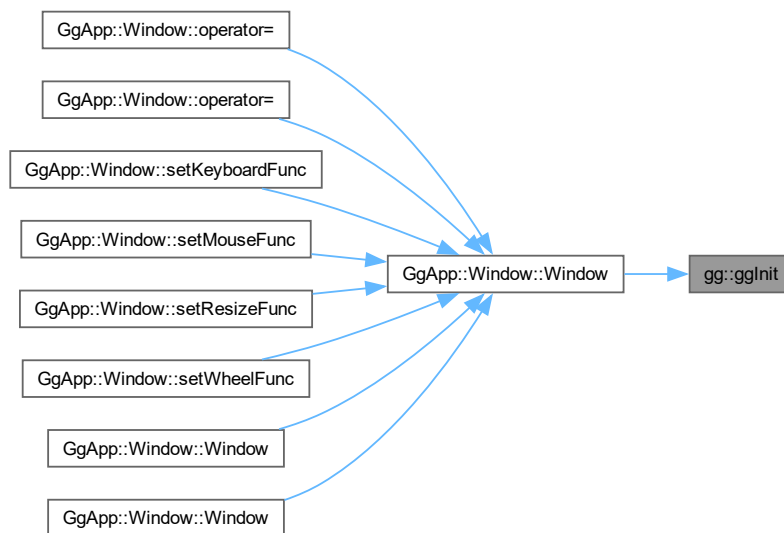
ゲームグラフィックス特論の都合にもとづく初期化を行う。

覚え書き

Windows において OpenGL 1.2 以降の API を有効化する。

gg.cpp の 1354 行目に定義があります。

被呼び出し関係図:



6.1.3.29 ggInvert() [1/2]

```
GgMatrix gg::ggInvert (
    const GgMatrix & m) [inline]
```

逆行列を返す。

引数

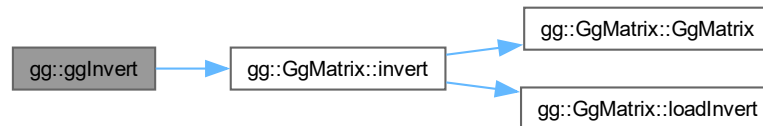
<code>m</code>	元の変換行列.
----------------	---------

戻り値

m の逆行列.

gg.h の 3464 行目に定義があります。

呼び出し関係図:



6.1.3.30 ggInvert() [2/2]

```
GgQuaternion gg::ggInvert (
    const GgQuaternion & q) [inline]
```

四元数の逆元を求める.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 q の逆元.

gg.h の 4798 行目に定義があります。

呼び出し関係図:



6.1.3.31 ggLength3() [1/2]

```
GLfloat gg::ggLength3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の長さ.

引数

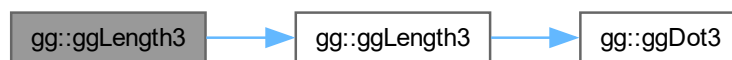
<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

a の 3 要素の長さ.

gg.h の 2050 行目に定義があります。

呼び出し関係図:



6.1.3.32 ggLength3() [2/2]

```
GLfloat gg::ggLength3 (  
    const GLfloat * a) [inline]
```

3 要素の長さ.

引数

<i>a</i>	GLfloat 型の 3 要素の配列変数.
----------	-----------------------

戻り値

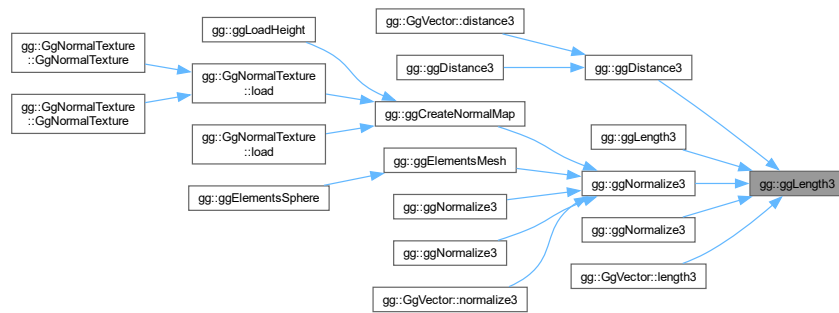
a の 3 要素の長さ.

gg.h の 1460 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.33 ggLength4() [1/2]

```
GLfloat gg::ggLength4 (
    const GgVector & a) [inline]
```

GgVector 型の 4 要素の長さ.

引数

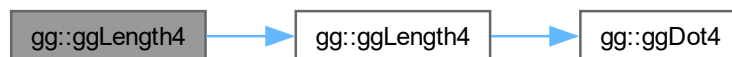
<i>a</i>	GgVector 型の変数.
----------	-----------------------

戻り値

a の 4 要素の長さ.

gg.h の 2114 行目に定義があります.

呼び出し関係図:



6.1.3.34 ggLength4() [2/2]

```
GLfloat gg::ggLength4 (
    const GLfloat * a) [inline]
```

GLfloat 型の 4 要素の長さ.

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

戻り値

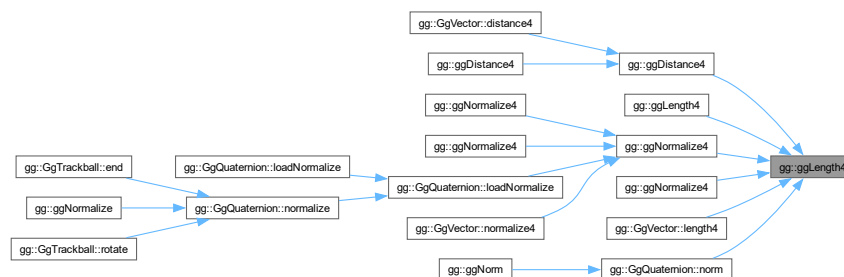
a の 4 要素の長さ.

[gg.h](#) の 1525 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.35 ggLoadComputeShader()

```

GLuint gg::ggLoadComputeShader (
    const std::string & comp) [extern]
  
```

コンピュータシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>comp</i>	コンピュータシェーダのソースファイル名.
-------------	----------------------

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 5113 行目に定義があります。

呼び出し関係図:



6.1.3.36 ggLoadHeight()

```

GLuint gg::ggLoadHeight (
    const std::string & name,
    GLfloat nz,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA) [extern]
  
```

TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する。

引数

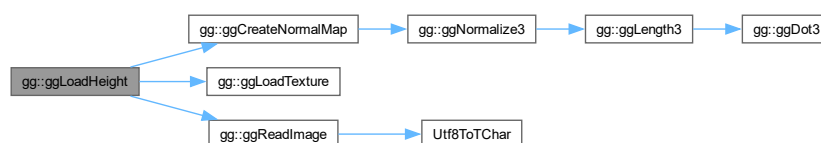
<i>name</i>	読み込むファイル名.
<i>nz</i>	法線の z 成分の割合.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3981 行目に定義があります。

呼び出し関係図:



6.1.3.37 ggLoadImage()

```
GLuint gg::ggLoadImage (
    const std::string & name,
    GLsizei * pWidth = nullptr,
    GLsizei * pHeight = nullptr,
    GLenum internal = GL_RGBA,
    GLenum wrap = GL_CLAMP_TO_EDGE) [extern]
```

テクスチャを作成して TGA フォーマットの画像ファイルを読み込む。

引数

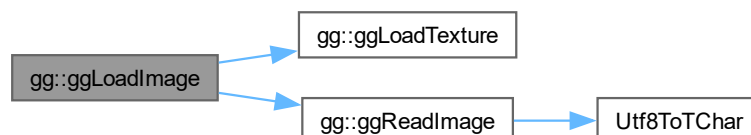
<i>name</i>	読み込むファイル名.
<i>pWidth</i>	読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない).
<i>pHeight</i>	読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない).
<i>internal</i>	テクスチャの内部フォーマット, デフォルトは GL_RGBA. 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3847 行目に定義があります。

呼び出し関係図:



6.1.3.38 ggLoadShader() [1/2]

```
GLuint gg::ggLoadShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.h の 5187 行目に定義があります。

呼び出し関係図:



6.1.3.39 ggLoadShader() [2/2]

```

GLuint gg::ggLoadShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [extern]
  
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する。

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

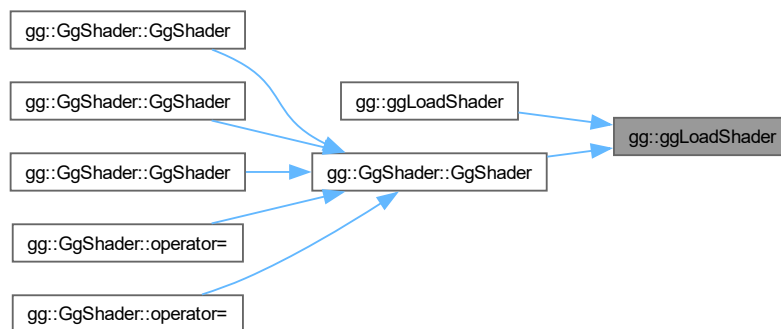
プログラムオブジェクトのプログラム名 (作成できなければ 0).

gg.cpp の 5039 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.40 ggLoadSimpleObj() [1/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルと MTL ファイルを読み込む (Arrays 形式)

引数

<i>name</i>	読み込む Wavefront OBJ ファイル名.
-------------	---------------------------

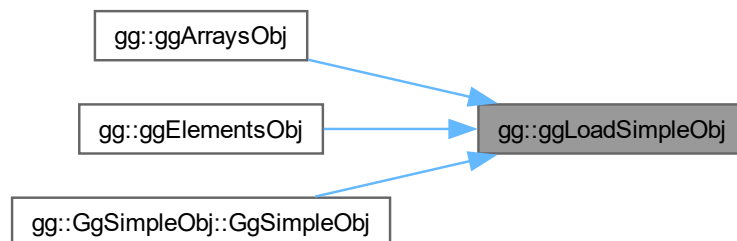
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの <code>GgSimpleShader::Material</code> 型の材質.
<i>vert</i>	読み込んだデータの頂点属性.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら true.

`gg.cpp` の 4648 行目に定義があります。

被呼び出し関係図:



6.1.3.41 ggLoadSimpleObj() [2/2]

```

bool gg::ggLoadSimpleObj (
    const std::string & name,
    std::vector< std::array< GLuint, 3 > > & group,
    std::vector< GgSimpleShader::Material > & material,
    std::vector< GgVertex > & vert,
    std::vector< GLuint > & face,
    bool normalize = false) [extern]
  
```

三角形分割された OBJ ファイルを読み込む (Elements 形式).

引数

<i>name</i>	読み込む Wavefront OBJ ファイル名.
<i>group</i>	読み込んだデータのポリゴングループごとの最初の三角形の番号と三角形数・材質番号.
<i>material</i>	読み込んだデータのポリゴングループごとの材質.
<i>vert</i>	読み込んだデータの頂点属性.

<i>face</i>	読み込んだデータの三角形の頂点インデックス.
<i>normalize</i>	true なら読み込んだデータの大きさを正規化する.

戻り値

ファイルの読み込みに成功したら true.

gg.cpp の 4737 行目に定義があります。

6.1.3.42 ggLoadTexture()

```
GLuint gg::ggLoadTexture (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = false) [extern]
```

テクスチャを作成して確保して画像データをテクスチャとして読み込む.

引数

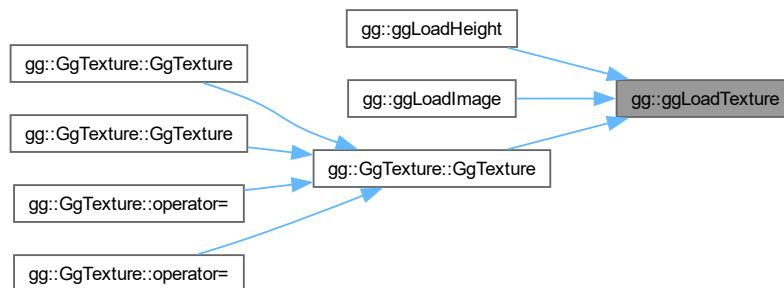
<i>image</i>	テクスチャとして読み込むデータ, nullptr ならテクスチャの作成のみを行う.
<i>width</i>	テクスチャとして読み込むデータ image の横の画素数.
<i>height</i>	テクスチャとして読み込むデータ image の縦の画素数.
<i>format</i>	image のフォーマット.
<i>type</i>	image のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL_CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

戻り値

テクスチャの作成に成功すればテクスチャ名, 失敗すれば 0.

gg.cpp の 3796 行目に定義があります。

被呼び出し関係図:



6.1.3.43 ggLookat() [1/3]

```
GgMatrix gg::ggLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]
```

ビュー変換行列を返す.

引数

<i>e</i>	視点の位置を格納した <code>GgVector</code> 型の変数.
<i>t</i>	目標点の位置を格納した <code>GgVector</code> 型の変数.
<i>u</i>	上方向のベクトルを格納した <code>GgVector</code> 型の変数.

戻り値

求めたビュー変換行列.

`gg.h` の 3379 行目に定義があります。

呼び出し関係図:



6.1.3.44 ggLookat() [2/3]

```
GgMatrix gg::ggLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
```

ビュー変換行列を返す.

引数

<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

求めたビュー変換行列.

[gg.h](#) の 3361 行目に定義があります。

呼び出し関係図:



6.1.3.45 ggLookat() [3/3]

```
GgMatrix gg::ggLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz) [inline]
```

ビュー変換行列を返す.

引数

<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

求めたビュー変換行列.

gg.h の 3343 行目に定義があります。

呼び出し関係図:

gg::ggLookat



gg::GgMatrix::loadLookat

6.1.3.46 ggMatrixQuaternion() [1/2]

```
GgQuaternion gg::ggMatrixQuaternion (  
    const GgMatrix & m) [inline]
```

回転の変換行列 m を表す四元数を返す.

引数

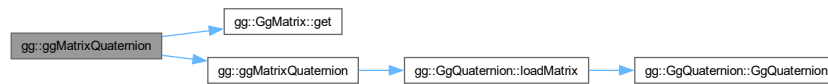
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

`m` による回転の変換に相当する四元数.

`gg.h` の 4600 行目に定義があります。

呼び出し関係図:



6.1.3.47 ggMatrixQuaternion() [2/2]

```
GgQuaternion gg::ggMatrixQuaternion (
    const GLfloat * a) [inline]
```

回転の変換行列 `m` を表す四元数を返す.

引数

<code>a</code>	GLfloat 型の 16 要素の配列変数.
----------------	------------------------

戻り値

`a` による回転の変換に相当する四元数.

`gg.h` の 4588 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.48 ggNorm()

```
GLfloat gg::ggNorm (
    const GgQuaternion & q) [inline]
```

四元数のノルムを返す。

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* のノルム.

gg.h の 4765 行目に定義があります。

呼び出し関係図:



6.1.3.49 ggNormal()

```
GgMatrix gg::ggNormal (
    const GgMatrix & m) [inline]
```

法線変換行列を返す。

引数

<i>m</i>	元の変換行列.
----------	---------

戻り値

m の法線変換行列.

gg.h の 3475 行目に定義があります。

呼び出し関係図:



6.1.3.50 ggNormalize()

```
GgQuaternion gg::ggNormalize (
    const GgQuaternion & q) [inline]
```

正規化した四元数を返す.

引数

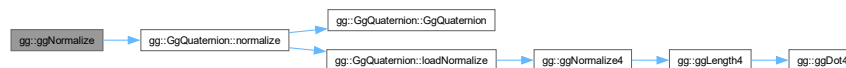
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

四元数 *q* を正規化した四元数.

gg.h の 4776 行目に定義があります.

呼び出し関係図:



6.1.3.51 ggNormalize3() [1/4]

```
GgVector gg::ggNormalize3 (
    const GgVector & a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

<i>a</i>	GgVector 型のベクトル.
----------	------------------

戻り値

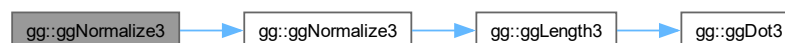
a の 3 要素を正規化したもの.

覚え書き

戻り値の *w* (第4) 要素は操作しない.

gg.h の 2076 行目に定義があります.

呼び出し関係図:



6.1.3.52 ggNormalize3() [2/4]

```
void gg::ggNormalize3 (
    const GLfloat * a,
    GLfloat * b) [inline]
```

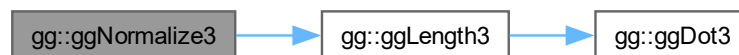
3 要素の正規化.

引数

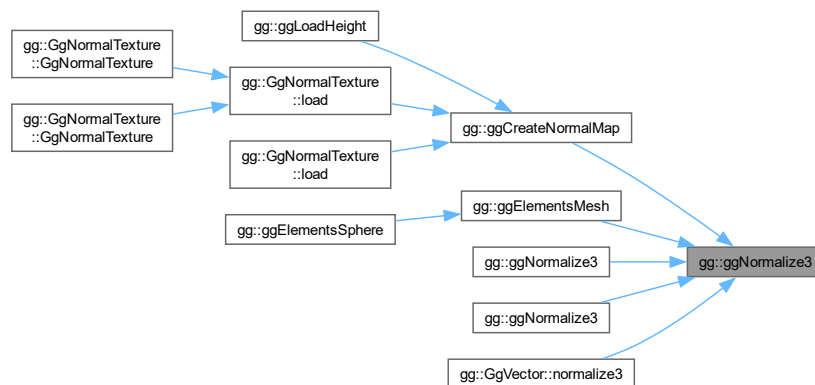
<i>a</i>	GLfloat 型の 3 要素の配列変数.
<i>b</i>	<i>a</i> の 3 要素を正規化した結果を格納する GLfloat 型の 3 要素の配列変数.

gg.h の 1484 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



6.1.3.53 ggNormalize3() [3/4]

```
void gg::ggNormalize3 (
    GgVector * a) [inline]
```

GgVector 型の 3 要素の正規化.

引数

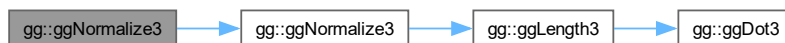
<i>a</i>	<code>GgVector</code> 型の変数のポインタ.
----------	----------------------------------

覚え書き

a の *w* (第4) 要素は操作しない.

`gg.h` の 2091 行目に定義があります。

呼び出し関係図:



6.1.3.54 ggNormalize3() [4/4]

```
void gg::ggNormalize3 (
    GLfloat * a) [inline]
```

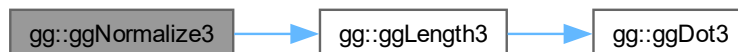
3 要素の正規化.

引数

<i>a</i>	<code>GLfloat</code> 型の 3 要素の配列変数.
----------	------------------------------------

`gg.h` の 1498 行目に定義があります。

呼び出し関係図:



6.1.3.55 ggNormalize4() [1/4]

```
GgVector gg::ggNormalize4 (
    const GgVector & a) [inline]
```

`GgVector` 型の 4 要素の正規化.

引数

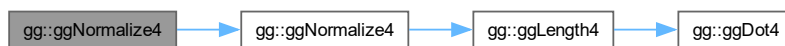
<i>a</i>	GgVector 型の変数.
----------	----------------

戻り値

a の 4 要素を正規化した結果.

gg.h の 2137 行目に定義があります。

呼び出し関係図:



6.1.3.56 ggNormalize4() [2/4]

```
void gg::ggNormalize4 (  
    const GLfloat * a,  
    GLfloat * b) [inline]
```

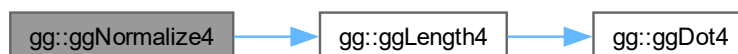
GLfloat 型の 4 要素の正規化.

引数

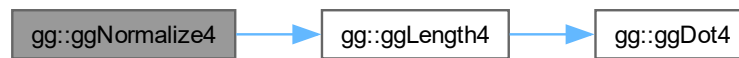
<i>a</i>	GLfloat 型の 4 要素の配列変数.
<i>b</i>	<i>a</i> を正規化した結果を格納する GLfloat 型の 4 要素の配列変数.

gg.h の 1549 行目に定義があります。

呼び出し関係図:



呼び出し関係図:



6.1.3.59 ggOrthogonal()

```
GgMatrix gg::ggOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

直交投影変換行列を返す.

引数

<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた直交投影変換行列.

gg.h の 3400 行目に定義があります。

呼び出し関係図:



6.1.3.60 ggPerspective()

```
GgMatrix gg::ggPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) [inline]
```

画角を指定して透視投影変換行列を返す.

引数

<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

求めた透視投影変換行列.

[gg.h](#) の 3438 行目に定義があります。

呼び出し関係図:



6.1.3.61 ggPointsCube()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsCube (
    GLsizei countv,
    GLfloat length = 1.0f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を立方体状に生成する.

引数

<i>countv</i>	生成する点の数.
---------------	----------

<i>length</i>	点群を生成する立方体の一辺の長さ.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5200 行目に定義があります。

6.1.3.62 ggPointsSphere()

```
std::shared_ptr< gg::GgPoints > gg::ggPointsSphere (
    GLsizei countv,
    GLfloat radius = 0.5f,
    GLfloat cx = 0.0f,
    GLfloat cy = 0.0f,
    GLfloat cz = 0.0f) [extern]
```

点群を球状に生成する.

引数

<i>countv</i>	生成する点の数.
<i>radius</i>	点群を生成する半径.
<i>cx</i>	点群の中心の x 座標.
<i>cy</i>	点群の中心の y 座標.
<i>cz</i>	点群の中心の z 座標.

戻り値

[GgPoints](#) 型の ポインタ.

[gg.cpp](#) の 5227 行目に定義があります。

6.1.3.63 ggQuaternionMatrix()

```
GgMatrix gg::ggQuaternionMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の変換行列を返す.

引数

q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する **GgMatrix** 型の変換行列.

gg.h の 4611 行目に定義があります。

呼び出し関係図:



6.1.3.64 ggQuaternionTransposeMatrix()

```
GgMatrix gg::ggQuaternionTransposeMatrix (
    const GgQuaternion & q) [inline]
```

四元数 q の回転の転置した変換行列を返す.

引数

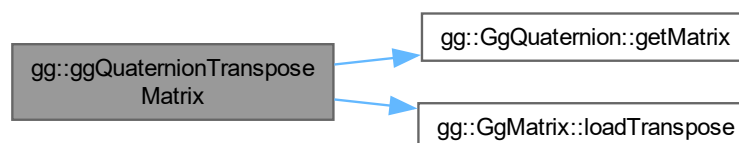
q	元の四元数.
-----	--------

戻り値

四元数 q が表す回転に相当する転置した **GgMatrix** 型の変換行列.

gg.h の 4624 行目に定義があります。

呼び出し関係図:



6.1.3.65 ggReadImage()

```
bool gg::ggReadImage (
    const std::string & name,
    std::vector< GLubyte > & image,
    GLsizei * pWidth,
    GLsizei * pHeight,
    GLenum * pFormat) [extern]
```

TGA ファイル (8/16/24/32bit) をメモリに読み込む。

引数

<i>name</i>	読み込むファイル名.
<i>image</i>	読み込んだデータを格納する vector.
<i>pWidth</i>	読み込んだ画像の横の画素数の格納先のポインタ, nullptr なら格納しない.
<i>pHeight</i>	読み込んだ画像の縦の画素数の格納先のポインタ, nullptr なら格納しない.
<i>pFormat</i>	読み込んだファイルの書式 (GL_RED, GL_RG, GL_RGB, GL_RGBA) の格納先のポインタ, nullptr なら格納しない.

戻り値

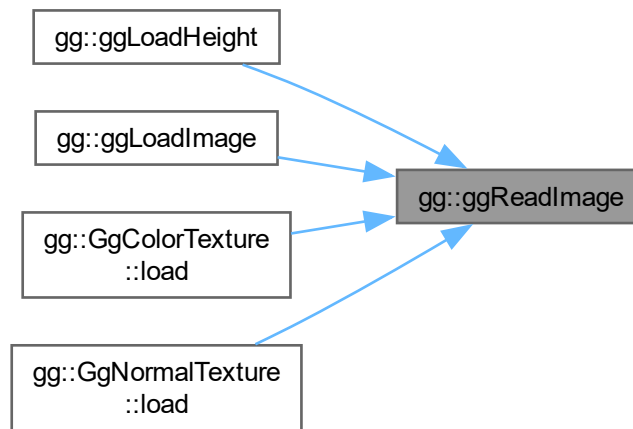
読み込みに成功すれば true, 失敗すれば false.

gg.cpp の 3682 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.66 ggRectangle()

```
std::shared_ptr< gg::GgTriangles > gg::ggRectangle (
    GLfloat width = 1.0f,
    GLfloat height = 1.0f) [extern]
```

矩形状に 2 枚の三角形を生成する.

引数

<i>width</i>	矩形の横幅.
<i>height</i>	矩形の高さ.

戻り値

[GgTriangles](#) 型のポインタ.

[gg.cpp](#) の 5255 行目に定義があります。

6.1.3.67 ggRotate() [1/5]

```
GgMatrix gg::ggRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<code>r</code>	回転軸のベクトルと回転角を表す <code>GgVector</code> 型の変数.
----------------	---

戻り値

(`r[0]`, `r[1]`, `r[2]`) を軸に `r[3]` だけ回転する変換行列.

`gg.h` の 3323 行目に定義があります。

呼び出し関係図:



6.1.3.68 ggRotate() [2/5]

```

GgMatrix gg::ggRotate (
    const GgVector & r,
    GLfloat a) [inline]
  
```

`r` 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<code>r</code>	回転軸のベクトルを表す <code>GgVector</code> 型の変数.
<code>a</code>	回転角.

戻り値

`r` を軸に `a` だけ回転する変換行列.

`gg.h` の 3299 行目に定義があります。

呼び出し関係図:



6.1.3.69 ggRotate() [3/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルと回転角を表す GLfloat 型の 4 要素の配列変数.
-----	---------------------------------------

戻り値

($r[0]$, $r[1]$, $r[2]$) を軸に $r[3]$ だけ回転する変換行列.

gg.h の 3311 行目に定義があります。

呼び出し関係図:



6.1.3.70 ggRotate() [4/5]

```
GgMatrix gg::ggRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

r	回転軸のベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

r を軸に a だけ回転する変換行列.

gg.h の 3286 行目に定義があります。

呼び出し関係図:



6.1.3.71 ggRotate() [5/5]

```
GgMatrix gg::ggRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

x	回転軸の x 成分.
y	回転軸の y 成分.
z	回転軸の z 成分.
a	回転角.

戻り値

(x, y, z) を軸にさらに a 回転する変換行列.

gg.h の 3273 行目に定義があります。

呼び出し関係図:



6.1.3.72 ggRotateQuaternion() [1/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を返す.

引数

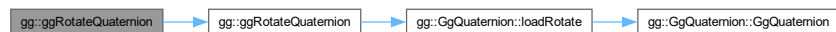
v	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

回転を表す四元数.

gg.h の 4665 行目に定義があります。

呼び出し関係図:



6.1.3.73 ggRotateQuaternion() [2/3]

```
GgQuaternion gg::ggRotateQuaternion (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を返す.

引数

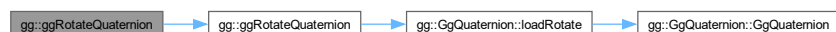
v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

回転を表す四元数.

gg.h の 4654 行目に定義があります。

呼び出し関係図:



6.1.3.74 ggRotateQuaternion() [3/3]

```
GgQuaternion gg::ggRotateQuaternion (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) [inline]
```

(x, y, z) を軸として角度 a 回転する四元数を返す.

引数

x	軸ベクトルの x 成分.
y	軸ベクトルの y 成分.
z	軸ベクトルの z 成分.
a	回転角.

戻り値

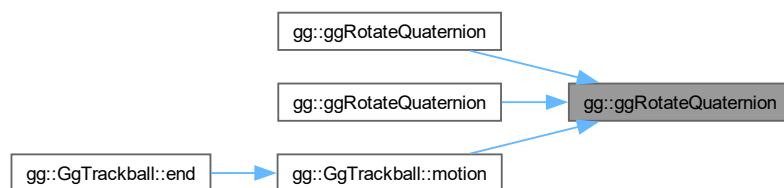
回転を表す四元数.

gg.h の 4641 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.75 ggRotateX()

```
GgMatrix gg::ggRotateX (
    GLfloat a) [inline]
```

x 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

x 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3234 行目に定義があります。

呼び出し関係図:



6.1.3.76 ggRotateY()

```
GgMatrix gg::ggRotateY (  
    GLfloat a) [inline]
```

y 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

y 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3246 行目に定義があります。

呼び出し関係図:



6.1.3.77 ggRotateZ()

```
GgMatrix gg::ggRotateZ (  
    GLfloat a) [inline]
```

z 軸中心の回転の変換行列を返す.

引数

<i>a</i>	回転角.
----------	------

戻り値

z 軸中心に *a* だけ回転する変換行列.

[gg.h](#) の 3258 行目に定義があります。

呼び出し関係図:



6.1.3.78 ggSaveColor()

```
bool gg::ggSaveColor (
    const std::string & name) [extern]
```

カラーバッファの内容を TGA ファイルに保存する.

引数

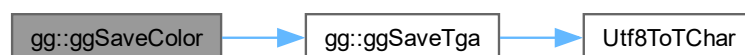
<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

[gg.cpp](#) の 3626 行目に定義があります。

呼び出し関係図:



6.1.3.79 ggSaveDepth()

```
bool gg::ggSaveDepth (
    const std::string & name) [extern]
```

デプスバッファの内容を TGA ファイルに保存する.

引数

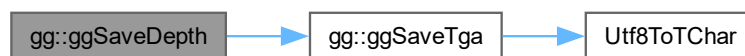
<i>name</i>	保存するファイル名.
-------------	------------

戻り値

保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3652 行目に定義があります。

呼び出し関係図:



6.1.3.80 ggSaveTga()

```
bool gg::ggSaveTga (  
    const std::string & name,  
    const void * buffer,  
    unsigned int width,  
    unsigned int height,  
    unsigned int depth) [extern]
```

配列の内容を TGA ファイルに保存する。

引数

<i>name</i>	保存するファイル名.
<i>buffer</i>	画像データを格納した配列.
<i>width</i>	画像の横の画素数.
<i>height</i>	画像の縦の画素数.
<i>depth</i>	1画素のバイト数.

戻り値

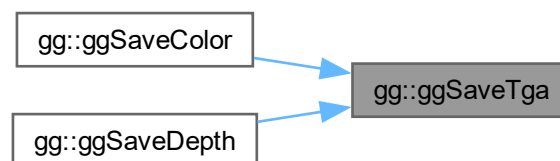
保存に成功すれば `true`, 失敗すれば `false`.

`gg.cpp` の 3550 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.81 ggScale() [1/3]

```
GgMatrix gg::ggScale (  
    const GgVector & s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の GgVector 型の変数.
---	-------------------------------------

戻り値

拡大縮小の変換行列.

[gg.h](#) の 3222 行目に定義があります.

呼び出し関係図:



6.1.3.82 ggScale() [2/3]

```
GgMatrix gg::ggScale (  
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を返す.

引数

s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小の変換行列.

gg.h の 3210 行目に定義があります。

呼び出し関係図:



6.1.3.83 ggScale() [3/3]

```
GgMatrix gg::ggScale (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w = 1.0f) [inline]
```

拡大縮小の変換行列を返す.

引数

x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	拡大率のスケールファクタ (= 1.0f).

戻り値

拡大縮小の変換行列.

[gg.h](#) の 3198 行目に定義があります。

呼び出し関係図:



6.1.3.84 ggSlerp() [1/4]

```
GgQuaternion gg::ggSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

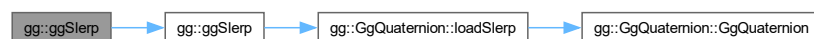
<i>q</i>	GgQuaternion 型の四元数.
<i>r</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

q, *r* を *t* で内分した四元数.

[gg.h](#) の 4728 行目に定義があります。

呼び出し関係図:



6.1.3.85 ggSlerp() [2/4]

```
GgQuaternion gg::ggSlerp (  
    const GgQuaternion & q,  
    const GLfloat * a,  
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

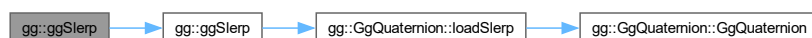
<i>q</i>	GgQuaternion 型の四元数.
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

q, *a* を *t* で内分した四元数.

[gg.h](#) の 4741 行目に定義があります。

呼び出し関係図:



6.1.3.86 ggSlerp() [3/4]

```

GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
  
```

二つの四元数の球面線形補間の結果を返す.

引数

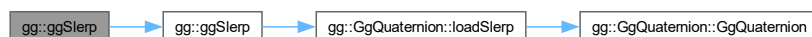
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

a, *q* を *t* で内分した四元数.

[gg.h](#) の 4754 行目に定義があります。

呼び出し関係図:



6.1.3.87 ggSlerp() [4/4]

```
GgQuaternion gg::ggSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

二つの四元数の球面線形補間の結果を返す.

引数

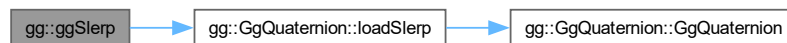
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

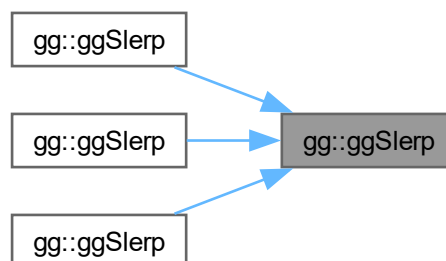
a, *b* を *t* で内分した四元数.

gg.h の 4714 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



6.1.3.88 ggTranslate() [1/3]

```
GgMatrix gg::ggTranslate (  
    const GgVector & t) [inline]
```

平行移動の変換行列を返す.

引数

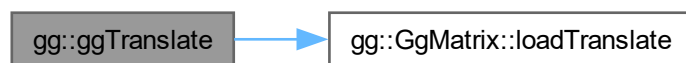
<code>t</code>	移動量の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

平行移動の変換行列.

`gg.h` の 3183 行目に定義があります。

呼び出し関係図:



6.1.3.89 ggTranslate() [2/3]

```
GgMatrix gg::ggTranslate (  
    const GLfloat * t) [inline]
```

平行移動の変換行列を返す.

引数

<code>t</code>	移動量の <code>GLfloat</code> 型の 3 要素の配列変数 (x, y, z).
----------------	---

戻り値

平行移動の変換行列.

`gg.h` の 3171 行目に定義があります。

呼び出し関係図:



6.1.3.90 ggTranslate() [3/3]

```
GgMatrix gg::ggTranslate (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w = 1.0f) [inline]
```

平行移動の変換行列を返す.

引数

x	x 方向の移動量.
y	y 方向の移動量.
z	z 方向の移動量.
w	移動量のスケールファクタ (= 1.0f).

戻り値

平行移動の変換行列.

gg.h の 3159 行目に定義があります。

呼び出し関係図:



6.1.3.91 ggTranspose()

```
GgMatrix gg::ggTranspose (  
    const GgMatrix & m) [inline]
```

転置行列を返す.

引数

m	元の変換行列.
-----	---------

戻り値

`m` の転置行列.

`gg.h` の 3453 行目に定義があります。

呼び出し関係図:



6.1.3.92 operator*()

```
GgVector gg::operator* (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに `GgVector` 型の各要素を乗じた積を返す.

引数

<code>a</code>	GLfloat 型の値.
<code>b</code>	<code>GgVector</code> 型のベクトル.

戻り値

`a` に `b` の各要素を乗じた積.

`gg.h` の 1998 行目に定義があります。

6.1.3.93 operator+() [1/2]

```
const GgVector & gg::operator+ (
    const GgVector & v) [inline]
```

単項の `+` 演算子なので何もしない.

引数

<code>v</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

`v` の値.

`gg.h` の 1951 行目に定義があります。

6.1.3.94 operator+() [2/2]

```
GgVector gg::operator+ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーに **GgVector** 型の各要素を足した和を返す.

引数

a	GLfloat 型の値.
b	GgVector 型のベクトル.

戻り値

a に **b** の各要素を足した和.

gg.h の 1963 行目に定義があります。

6.1.3.95 operator-() [1/2]

```
const GgVector gg::operator- (
    const GgVector & v) [inline]
```

符号の反転.

引数

v	GgVector 型のベクトル.
----------	-------------------------

戻り値

v の値の符号を反転した結果.

gg.h の 1974 行目に定義があります。

6.1.3.96 operator-() [2/2]

```
GgVector gg::operator- (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーから **GgVector** 型の各要素を引いた差を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a から *b* の各要素を引いた差.

gg.h の 1986 行目に定義があります。

6.1.3.97 operator/()

```
GgVector gg::operator/ (
    GLfloat a,
    const GgVector & b) [inline]
```

スカラーを GgVector 型の各要素で割った商を返す.

引数

<i>a</i>	GLfloat 型の値.
<i>b</i>	GgVector 型のベクトル.

戻り値

a を *b* の各要素で割った商.

gg.h の 2010 行目に定義があります。

6.1.4 変数詳解

6.1.4.1 ggBufferAlignment

```
GLint gg::ggBufferAlignment [extern]
```

使用している GPU のバッファアライメント.

使用している GPU のバッファオブジェクトのアライメント, 初期化に取得される.

Chapter 7

クラス詳解

7.1 GgApp クラス

```
#include <GgApp.h>
```

クラス

- class [Window](#)

公開メンバ関数

- [GgApp](#) (int major=0, int minor=1)
- [GgApp](#) (const [GgApp](#) &w)=delete
- [GgApp](#) ([GgApp](#) &&w)=default
- virtual [~GgApp](#) ()
- [GgApp](#) & [operator=](#) (const [GgApp](#) &w)=delete
- [GgApp](#) & [operator=](#) ([GgApp](#) &&w)=default
- int [main](#) (int argc, const char *const *argv)

静的公開メンバ関数

- static std::string [getUsername](#) ()

7.1.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラス。

[GgApp.h](#) の 108 行目に定義があります。

7.1.2 構築子と解体子

7.1.2.1 GgApp() [1/3]

```
GgApp::GgApp (  
    int major = 0,  
    int minor = 1)
```

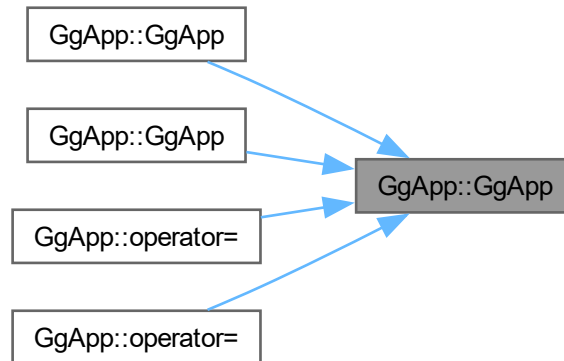
コンストラクタ。

引数

<i>major</i>	使用する OpenGL の major 番号, 0 なら無指定.
<i>minor</i>	使用する OpenGL の minor 番号, major 番号が 0 なら無視.

[GgApp.cpp](#) の 49 行目に定義があります。

被呼び出し関係図:



7.1.2.2 GgApp() [2/3]

```
GgApp::GgApp (
    const GgApp & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のオブジェクト.
----------	--------------

呼び出し関係図:



7.1.2.3 GgApp() [3/3]

```
GgApp::GgApp (
    GgApp && w) [default]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ元のオブジェクト.
----------	--------------

呼び出し関係図:



7.1.2.4 ~GgApp()

```
GgApp::~GgApp () [virtual]
```

デストラクタ.

[GgApp.cpp](#) の 91 行目に定義があります。

7.1.3 関数詳解

7.1.3.1 getUsername()

```
std::string GgApp::getUsername () [static]
```

ユーザ名を得る.

戻り値

ユーザ名の文字列.

[GgApp.cpp](#) の 1997 行目に定義があります。

7.1.3.2 main()

```
int GgApp::main (
    int argc,
    const char *const * argv)
```

アプリケーション本体.

引数

<i>argc</i>	コマンドライン引数の数.
<i>argv</i>	コマンドライン引数の配列.

戻り値

アプリケーションの終了ステータス.

7.1.3.3 operator=() [1/2]

```
GgApp & GgApp::operator= (
    const GgApp & w) [delete]
```

代入演算子は使用しない.

引数

<i>w</i>	代入元のオブジェクト.
----------	-------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



7.1.3.4 operator=() [2/2]

```
GgApp & GgApp::operator= (
    GgApp && w) [default]
```

ムーブ代入演算子.

引数

<i>w</i>	ムーブ代入元のオブジェクト.
----------	----------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [GgApp.cpp](#)

7.2 gg::GgBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgBuffer](#) (GLenum target, const T *data, GLsizei stride, GLsizei count, GLenum usage)
- [GgBuffer](#) (const GgBuffer< T > &buffer)=delete
- [GgBuffer](#) (GgBuffer< T > &&o) noexcept
- virtual [~GgBuffer](#) ()
- [GgBuffer](#)< T > & [operator=](#) (const [GgBuffer](#)< T > &buffer)=delete
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLint first, GLsizei count) const
- void [unmap](#) () const
- void [send](#) (const T *data, GLint first, GLsizei count) const
- void [read](#) (T *data, GLint first, GLsizei count) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

7.2.1 詳解

```
template<typename T>
class gg::GgBuffer< T >
```

バッファオブジェクト.

覚え書き

頂点属性／頂点インデックス／ユニフォーム変数を格納するバッファオブジェクトの基底クラス.

gg.h の 5822 行目に定義があります。

7.2.2 構築子と解体子

7.2.2.1 GgBuffer() [1/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GLenum target,
    const T * data,
    GLsizei stride,
    GLsizei count,
    GLenum usage) [inline]
```

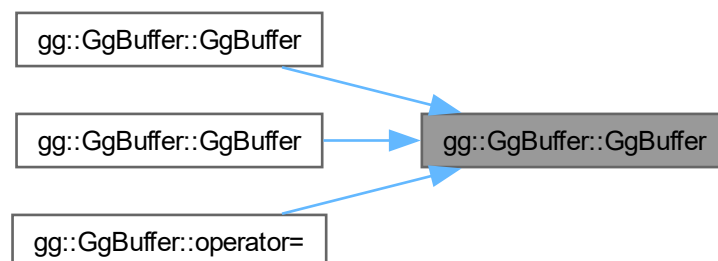
コンストラクタ.

引数

<i>target</i>	バッファオブジェクトのターゲット.
<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>stride</i>	データの間隔.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 5847 行目に定義があります。

被呼び出し関係図:



7.2.2.2 GgBuffer() [2/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    const GgBuffer< T > & buffer) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>buffer</i>	コピー元のバッファ.
---------------	------------

呼び出し関係図:



7.2.2.3 GgBuffer() [3/3]

```
template<typename T>
gg::GgBuffer< T >::GgBuffer (
    GgBuffer< T > && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

<i>o</i>	ムーブ元のバッファ.
----------	------------

覚え書き

バッファオブジェクト名の所有権を移動し, ムーブ元のバッファオブジェクト名を 0 にする. ムーブ元のデストラクタが同じバッファオブジェクトを削除しないようにするため.

gg.h の 5881 行目に定義があります.

呼び出し関係図:



7.2.2.4 ~GgBuffer()

```
template<typename T>
virtual gg::GgBuffer< T >::~~GgBuffer () [inline], [virtual]
```

デストラクタ.

gg.h の 5893 行目に定義があります。

7.2.3 関数詳解

7.2.3.1 bind()

```
template<typename T>
void gg::GgBuffer< T >::bind () const [inline]
```

バッファオブジェクトを結合する.

gg.h の 5951 行目に定義があります。

7.2.3.2 copy()

```
template<typename T>
void gg::GgBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
```

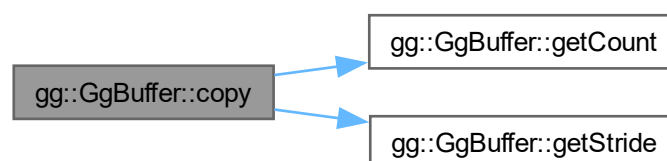
別のバッファオブジェクトからデータを複写する.

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (<i>src_buffer</i>) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6056 行目に定義があります。

呼び出し関係図:



7.2.3.3 getBuffer()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getBuffer () const [inline]
```

バッファオブジェクト名を取り出す。

戻り値

このバッファオブジェクト名。

gg.h の 5943 行目に定義があります。

7.2.3.4 getCount()

```
template<typename T>
const GLsizei & gg::GgBuffer< T >::getCount () const [inline]
```

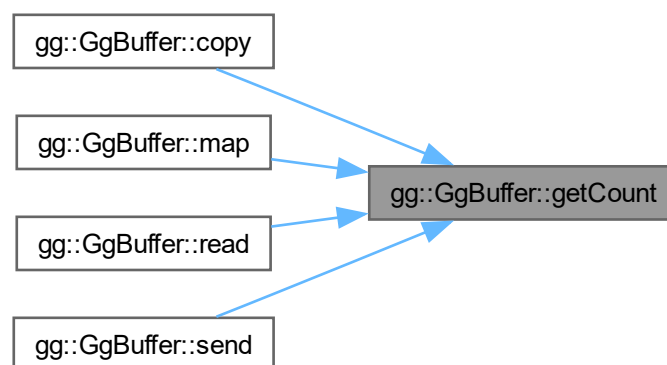
バッファオブジェクトが保持するデータの数を取り出す。

戻り値

このバッファオブジェクトが保持するデータの数。

gg.h の 5933 行目に定義があります。

被呼び出し関係図:



7.2.3.5 getStride()

```
template<typename T>
GLsizeiptr gg::GgBuffer< T >::getStride () const [inline]
```

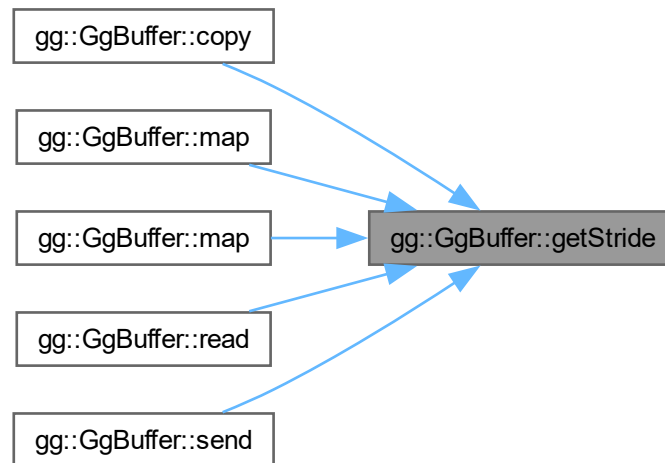
バッファオブジェクトのアライメントを考慮したデータの間隔を取り出す。

戻り値

このバッファオブジェクトのデータの間隔。

gg.h の 5923 行目に定義があります。

被呼び出し関係図:



7.2.3.6 getTarget()

```
template<typename T>
const GLuint & gg::GgBuffer< T >::getTarget () const [inline]
```

バッファオブジェクトのターゲットを取り出す。

戻り値

このバッファオブジェクトのターゲット。

gg.h の 5913 行目に定義があります。

7.2.3.7 map() [1/2]

```
template<typename T>
void * gg::GgBuffer< T >::map () const [inline]
```

バッファオブジェクトをマップする.

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5969 行目に定義があります。

呼び出し関係図:



7.2.3.8 map() [2/2]

```
template<typename T>
void * gg::GgBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

バッファオブジェクトの指定した範囲をマップする.

引数

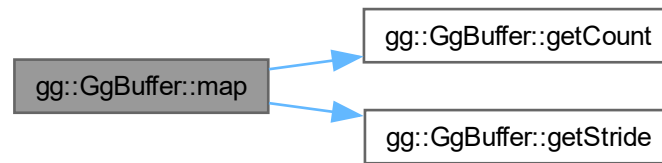
<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 5986 行目に定義があります。

呼び出し関係図:



7.2.3.9 operator=()

```
template<typename T>
GgBuffer< T > & gg::GgBuffer< T >::operator= (
    const GgBuffer< T > & buffer) [delete]
```

代入演算子は使用しない.

引数

<i>buffer</i>	代入元のバッファ.
---------------	-----------

戻り値

代入後のこのバッファの参照.

呼び出し関係図:



7.2.3.10 read()

```
template<typename T>
void gg::GgBuffer< T >::read (
    T * data,
    GLint first,
    GLsizei count) const [inline]
```

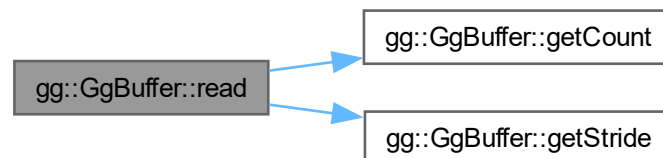
バッファオブジェクトのデータから抽出する.

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>first</i>	抽出元のバッファオブジェクトの取り出すデータの領域の先頭の要素番号.
<i>count</i>	抽出するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6029 行目に定義があります。

呼び出し関係図:



7.2.3.11 send()

```
template<typename T>
void gg::GgBuffer< T >::send (
    const T * data,
    GLint first,
    GLsizei count) const [inline]
```

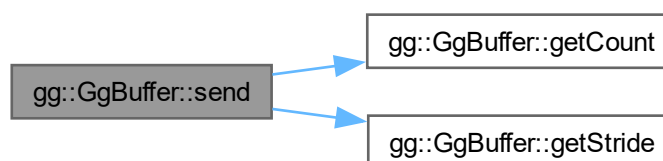
すでに確保したバッファオブジェクトにデータを転送する。

引数

<i>data</i>	転送元のデータが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送するデータの数 (0 ならバッファオブジェクト全体).

gg.h の 6011 行目に定義があります。

呼び出し関係図:



7.2.3.12 unbind()

```
template<typename T>
void gg::GgBuffer< T >::unbind () const [inline]
```

バッファオブジェクトを解放する。

gg.h の 5959 行目に定義があります。

7.2.3.13 unmap()

```
template<typename T>
void gg::GgBuffer< T >::unmap () const [inline]
```

バッファオブジェクトをアンマップする。

gg.h の 5999 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

7.3 gg::GgColorTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgColorTexture \(\)](#)
- [GgColorTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- [GgColorTexture](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- virtual [~GgColorTexture \(\)](#)
- void [load](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=true)
- void [load](#) (const std::string &name, GLenum internal=0, GLenum wrap=GL_CLAMP_TO_EDGE)
- [operator bool \(\)](#) const noexcept
- bool [operator!](#) () const noexcept
- const [GgTexture](#) * [get](#) () const
- GLuint [getTexture](#) () const
- GLsizei [getWidth](#) () const
- GLsizei [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- void [bind](#) () const
- void [unbind](#) () const

7.3.1 詳解

カラーマップ.

覚え書き

カラー画像を読み込んでテクスチャを作成する.

gg.h の 5409 行目に定義があります。

7.3.2 構築子と解体子

7.3.2.1 GgColorTexture() [1/3]

```
gg::GgColorTexture::GgColorTexture () [inline]
```

コンストラクタ.

gg.h の 5419 行目に定義があります。

7.3.2.2 GgColorTexture() [2/3]

```
gg::GgColorTexture::GgColorTexture (  
    const GLvoid * image,  
    GLsizei width,  
    GLsizei height,  
    GLenum format = GL_RGB,  
    GLenum type = GL_UNSIGNED_BYTE,  
    GLenum internal = GL_RGB,  
    GLenum wrap = GL_CLAMP_TO_EDGE,  
    bool swizzle = true) [inline]
```

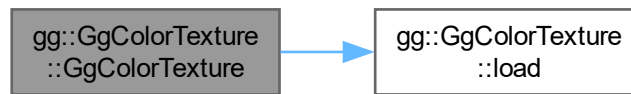
メモリ上のデータからカラーのテクスチャを作成するコンストラクタ.

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	読み込む画像の横の画素数.
<i>height</i>	読み込む画像の縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

gg.h の 5435 行目に定義があります。

呼び出し関係図:



7.3.2.3 GgColorTexture() [3/3]

```

gg::GgColorTexture::GgColorTexture (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE) [inline]
  
```

TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成するコンストラクタ。

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 なら外部フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード, GL_TEXTURE_WRAP_S および GL_TEXTURE_WRAP_T に設定する値.

[gg.h](#) の 5456 行目に定義があります。

呼び出し関係図:



7.3.2.4 ~GgColorTexture()

```

virtual gg::GgColorTexture::~~GgColorTexture () [inline], [virtual]
  
```

デストラクタ。

[gg.h](#) の 5468 行目に定義があります。

7.3.3 関数詳解

7.3.3.1 bind()

```
void gg::GgColorTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

gg.h の 5595 行目に定義があります。

7.3.3.2 get()

```
const GgTexture * gg::GgColorTexture::get () const [inline]
```

テクスチャオブジェクトを取り出す.

戻り値

GgTexture オブジェクトのポインタ.

gg.h の 5537 行目に定義があります。

7.3.3.3 getHeight()

```
GLsizei gg::GgColorTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す.

戻り値

テクスチャの縦の画素数.

gg.h の 5567 行目に定義があります。

7.3.3.4 getSize() [1/2]

```
const GLsizei * gg::GgColorTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す.

戻り値

テクスチャのサイズを格納した配列へのポインタ.

gg.h の 5587 行目に定義があります。

7.3.3.5 getSize() [2/2]

```
void gg::GgColorTexture::getSize (  
    GLsizei * size) const [inline]
```

使用しているテクスチャのサイズを取り出す.

引数

<i>size</i>	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数.
-------------	--------------------------------------

[gg.h](#) の 5577 行目に定義があります。

7.3.3.6 getTexture()

```
GLuint gg::GgColorTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る.

戻り値

テクスチャ名.

[gg.h](#) の 5547 行目に定義があります。

7.3.3.7 getWidth()

```
GLsizei gg::GgColorTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す.

戻り値

テクスチャの横の画素数.

[gg.h](#) の 5557 行目に定義があります。

7.3.3.8 load() [1/2]

```
void gg::GgColorTexture::load (
    const GLvoid * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RGB,
    GLenum type = GL_UNSIGNED_BYTE,
    GLenum internal = GL_RGB,
    GLenum wrap = GL_CLAMP_TO_EDGE,
    bool swizzle = true) [inline]
```

メモリ上のデータを読み込んでテクスチャを作成する.

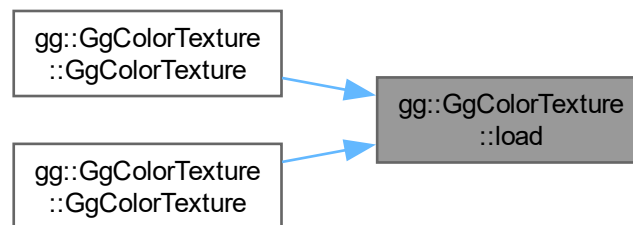
引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.

<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	読み込む画像のデータ型.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは true.

gg.h の 5484 行目に定義があります。

被呼び出し関係図:



7.3.3.9 load() [2/2]

```
void gg::GgColorTexture::load (
    const std::string & name,
    GLenum internal = 0,
    GLenum wrap = GL_CLAMP_TO_EDGE)
```

TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する.

引数

<i>name</i>	読み込むファイル名.
<i>internal</i>	glTexImage2D() に指定するテクスチャの内部フォーマット, 0 ならファイルの画像フォーマットに合わせる.
<i>wrap</i>	テクスチャのラッピングモード (GL_CLAMP_TO_EDGE, GL_CLAMP_TO_BORDER, GL_REPEAT, GL_MIRRORED_REPEAT).

gg.cpp の 4043 行目に定義があります。

呼び出し関係図:



7.3.3.10 operator bool()

```
gg::GgColorTexture::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる.

戻り値

オブジェクトが有効なら **true**.

[gg.h](#) の [5517](#) 行目に定義があります。

7.3.3.11 operator!()

```
bool gg::GgColorTexture::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する.

戻り値

オブジェクトが有効なら **false**, 無効なら **true**.

[gg.h](#) の [5527](#) 行目に定義があります。

7.3.3.12 unbind()

```
void gg::GgColorTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

[gg.h](#) の [5603](#) 行目に定義があります。

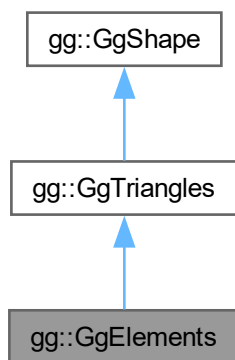
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

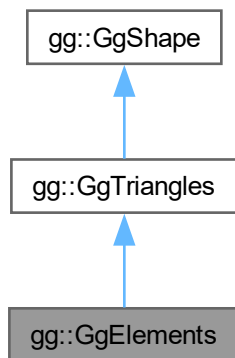
7.4 gg::GgElements クラス

```
#include <gg.h>
```

gg::GgElements の継承関係図



gg::GgElements 連携図



公開メンバ関数

- [GgElements](#) (GLenum mode=GL_TRIANGLES)
- [GgElements](#) (const [GgVertex](#) *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgElements](#) ()
- const GLsizei & [getIndexCount](#) () const

- const GLuint & [getIndexBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLuint firstv, GLsizei countv, const GLuint *face=NULLPTR, GLuint firstf=0, GLsizei countf=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei countv, const GLuint *face, GLsizei countf, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLuint first=0, GLsizei count=0) const

基底クラス [gg::GgTriangles](#) に属する継承公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLuint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

7.4.1 詳解

三角形で表した形状データ (Elements 形式).

[gg.h](#) の 6889 行目に定義があります。

7.4.2 構築子と解体子

7.4.2.1 GgElements() [1/2]

```
gg::GgElements::GgElements (
    GLenum mode = GL_TRIANGLES) [inline]
```

コンストラクタ.

引数

<i>mode</i>	描画する基本図形の種類.
-------------	--------------

[gg.h](#) の 6902 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.4.2.2 GgElements() [2/2]

```

gg::GgElements::GgElements (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

コンストラクタ.

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>face</i>	三角形の頂点インデックス.
<i>countf</i>	三角形の頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6917 行目に定義があります。

呼び出し関係図:



7.4.2.3 ~GgElements()

```
virtual gg::GgElements::~~GgElements () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6933 行目に定義があります。

7.4.3 関数詳解

7.4.3.1 draw()

```
void gg::GgElements::draw (  
    GLint first = 0,  
    GLsizei count = 0) const [virtual]
```

インデックスを使った三角形の描画.

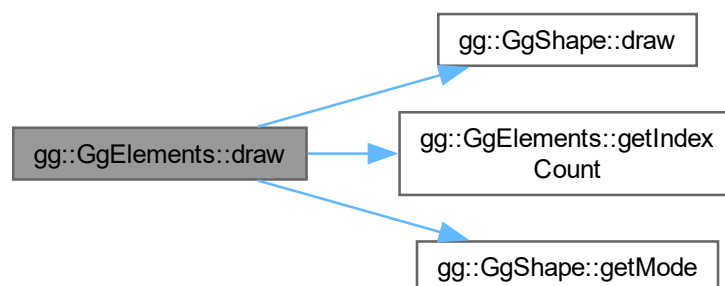
引数

<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

[gg::GgTriangles](#) を再実装しています。

[gg.cpp](#) の 5187 行目に定義があります。

呼び出し関係図:



7.4.3.2 getIndexBuffer()

```
const GLuint & gg::GgElements::getIndexBuffer () const [inline]
```

三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名を取り出す。

戻り値

この図形の三角形の頂点インデックスデータを格納した頂点バッファオブジェクト名。

gg.h の 6951 行目に定義があります。

7.4.3.3 getIndexCount()

```
const GLsizei & gg::GgElements::getIndexCount () const [inline]
```

データの数を取り出す。

戻り値

この図形の三角形数。

gg.h の 6941 行目に定義があります。

被呼び出し関係図:



7.4.3.4 load()

```
void gg::GgElements::load (
    const GgVertex * vert,
    GLsizei countv,
    const GLuint * face,
    GLsizei countf,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

バッファオブジェクトを確保して頂点属性と三角形の頂点インデックスデータを格納する。

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>countv</i>	頂点のデータの数 (頂点数).
<i>face</i>	三角形の頂点インデックスデータ.
<i>countf</i>	三角形の頂点数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6988 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.4.3.5 send()

```

void gg::GgElements::send (
    const GgVertex * vert,
    GLuint firstv,
    GLsizei countv,
    const GLuint * face = nullptr,
    GLuint firstf = 0,
    GLsizei countf = 0) const [inline]
  
```

既存のバッファオブジェクトに頂点属性と三角形の頂点インデックスデータを転送する。

引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ.
<i>firstv</i>	頂点属性の転送先のバッファオブジェクトの先頭の要素番号.
<i>countv</i>	頂点のデータの数 (頂点数).

<i>face</i>	三角形の頂点インデックスデータ.
<i>firstf</i>	インデックスの転送先のバッファオブジェクトの先頭の要素番号.
<i>countf</i>	三角形の頂点数.

gg.h の 6966 行目に定義があります。

呼び出し関係図:



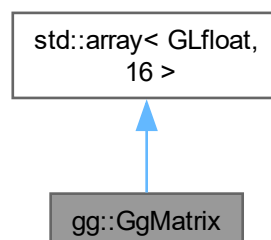
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

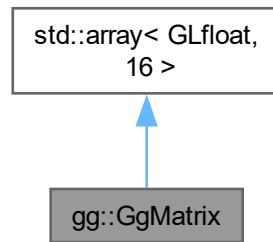
7.5 gg::GgMatrix クラス

```
#include <gg.h>
```

gg::GgMatrix の継承関係図



gg::GgMatrix 連携図



公開メンバ関数

- `GgMatrix ()=default`
- `constexpr GgMatrix (GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03, GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13, GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23, GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33)`
- `constexpr GgMatrix (const GLfloat *a)`
- `constexpr GgMatrix (GLfloat c)`
- `GgMatrix (const GgMatrix &m)=default`
- `GgMatrix (GgMatrix &&m)=default`
- `~GgMatrix ()=default`
- `GgMatrix & operator= (const GgMatrix &m)=default`
- `GgMatrix & operator= (GgMatrix &&m)=default`
- `GgMatrix & operator= (const GLfloat *a)`
- `GgMatrix operator+ (const GLfloat *a) const`
- `GgMatrix operator+ (const GgMatrix &m) const`
- `GgMatrix & operator+= (const GLfloat *a)`
- `GgMatrix & operator+= (const GgMatrix &m)`
- `GgMatrix operator- (const GLfloat *a) const`
- `GgMatrix operator- (const GgMatrix &m) const`
- `GgMatrix & operator-= (const GLfloat *a)`
- `GgMatrix & operator-= (const GgMatrix &m)`
- `GgMatrix operator* (const GLfloat *a) const`
- `GgMatrix operator* (const GgMatrix &m) const`
- `GgMatrix & operator*= (const GLfloat *a)`
- `GgMatrix & operator*= (const GgMatrix &m)`
- `GgMatrix operator/ (const GLfloat *a) const`
- `GgMatrix operator/ (const GgMatrix &m) const`
- `GgMatrix & operator/= (const GLfloat *a)`
- `GgMatrix & operator/= (const GgMatrix &m)`
- `GgMatrix & loadIdentity ()`
- `GgMatrix & loadTranslate (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)`
- `GgMatrix & loadTranslate (const GLfloat *t)`
- `GgMatrix & loadTranslate (const GgVector &t)`
- `GgMatrix & loadScale (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)`
- `GgMatrix & loadScale (const GLfloat *s)`
- `GgMatrix & loadScale (const GgVector &s)`

- `GgMatrix & loadRotateX` (GLfloat a)
- `GgMatrix & loadRotateY` (GLfloat a)
- `GgMatrix & loadRotateZ` (GLfloat a)
- `GgMatrix & loadRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix & loadRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix & loadRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix & loadRotate` (const GLfloat *r)
- `GgMatrix & loadRotate` (const `GgVector` &r)
- `GgMatrix & loadLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix & loadLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix & loadLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix & loadOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix & loadTranspose` (const GLfloat *a)
- `GgMatrix & loadTranspose` (const `GgMatrix` &m)
- `GgMatrix & loadInvert` (const GLfloat *a)
- `GgMatrix & loadInvert` (const `GgMatrix` &m)
- `GgMatrix & loadNormal` (const GLfloat *a)
- `GgMatrix & loadNormal` (const `GgMatrix` &m)
- `GgMatrix translate` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix translate` (const GLfloat *t) const
- `GgMatrix translate` (const `GgVector` &t) const
- `GgMatrix scale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f) const
- `GgMatrix scale` (const GLfloat *s) const
- `GgMatrix scale` (const `GgVector` &s) const
- `GgMatrix rotateX` (GLfloat a) const
- `GgMatrix rotateY` (GLfloat a) const
- `GgMatrix rotateZ` (GLfloat a) const
- `GgMatrix rotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r, GLfloat a) const
- `GgMatrix rotate` (const `GgVector` &r, GLfloat a) const
- `GgMatrix rotate` (const GLfloat *r) const
- `GgMatrix rotate` (const `GgVector` &r) const
- `GgMatrix lookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz) const
- `GgMatrix lookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u) const
- `GgMatrix lookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u) const
- `GgMatrix orthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix frustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar) const
- `GgMatrix perspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar) const
- `GgMatrix transpose` () const
- `GgMatrix invert` () const
- `GgMatrix normal` () const
- void `projection` (GLfloat *c, const GLfloat *v) const
- void `projection` (GLfloat *c, const `GgVector` &v) const
- void `projection` (`GgVector` &c, const GLfloat *v) const
- void `projection` (`GgVector` &c, const `GgVector` &v) const
- `GgVector operator*` (const `GgVector` &v) const
- const GLfloat * `get` () const
- void `get` (GLfloat *a) const

7.5.1 詳解

変換行列.

[gg.h](#) の 2157 行目に定義があります。

7.5.2 構築子と解体子

7.5.2.1 GgMatrix() [1/6]

```
gg::GgMatrix::GgMatrix () [default]
```

コンストラクタ.

7.5.2.2 GgMatrix() [2/6]

```
gg::GgMatrix::GgMatrix (  
    GLfloat m00,  
    GLfloat m01,  
    GLfloat m02,  
    GLfloat m03,  
    GLfloat m10,  
    GLfloat m11,  
    GLfloat m12,  
    GLfloat m13,  
    GLfloat m20,  
    GLfloat m21,  
    GLfloat m22,  
    GLfloat m23,  
    GLfloat m30,  
    GLfloat m31,  
    GLfloat m32,  
    GLfloat m33) [inline], [constexpr]
```

コンストラクタ.

引数

<i>m00</i>	GLfloat 型の値.
<i>m01</i>	GLfloat 型の値.
<i>m02</i>	GLfloat 型の値.
<i>m03</i>	GLfloat 型の値.
<i>m10</i>	GLfloat 型の値.
<i>m11</i>	GLfloat 型の値.
<i>m12</i>	GLfloat 型の値.
<i>m13</i>	GLfloat 型の値.
<i>m20</i>	GLfloat 型の値.
<i>m21</i>	GLfloat 型の値.

<i>m22</i>	GLfloat 型の値.
<i>m23</i>	GLfloat 型の値.
<i>m30</i>	GLfloat 型の値.
<i>m31</i>	GLfloat 型の値.
<i>m32</i>	GLfloat 型の値.
<i>m33</i>	GLfloat 型の値.

gg.h の 2192 行目に定義があります。

7.5.2.3 GgMatrix() [3/6]

```
gg::GgMatrix::GgMatrix (
    const GLfloat * a) [inline], [constexpr]
```

コンストラクタ.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

gg.h の 2207 行目に定義があります。

呼び出し関係図:



7.5.2.4 GgMatrix() [4/6]

```
gg::GgMatrix::GgMatrix (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

gg.h の 2217 行目に定義があります。

呼び出し関係図:



7.5.2.5 GgMatrix() [5/6]

```
gg::GgMatrix::GgMatrix (  
    const GgMatrix & m) [default]
```

コピーコンストラクタ.

引数

<i>m</i>	コピー元の変換行列.
----------	------------

呼び出し関係図:



7.5.2.6 GgMatrix() [6/6]

```
gg::GgMatrix::GgMatrix (  
    GgMatrix && m) [default]
```

ムーブコンストラクタ.

引数

<i>m</i>	ムーブ元の変換行列.
----------	------------

呼び出し関係図:



7.5.2.7 ~GgMatrix()

```
gg::GgMatrix::~~GgMatrix () [default]
```

デストラクタ.

7.5.3 関数詳解

7.5.3.1 frustum()

```
GgMatrix gg::GgMatrix::frustum (  
    GLfloat left,  
    GLfloat right,  
    GLfloat bottom,  
    GLfloat top,  
    GLfloat zNear,  
    GLfloat zFar) const [inline]
```

透視投影変換を乗じた結果を返す.

引数

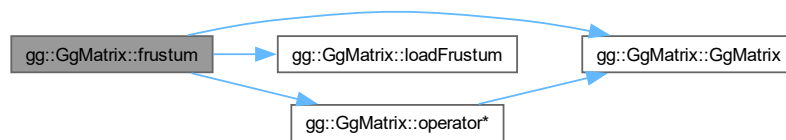
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

[gg.h](#) の 3000 行目に定義があります。

呼び出し関係図:



7.5.3.2 get() [1/2]

```
const GLfloat * gg::GgMatrix::get () const [inline]
```

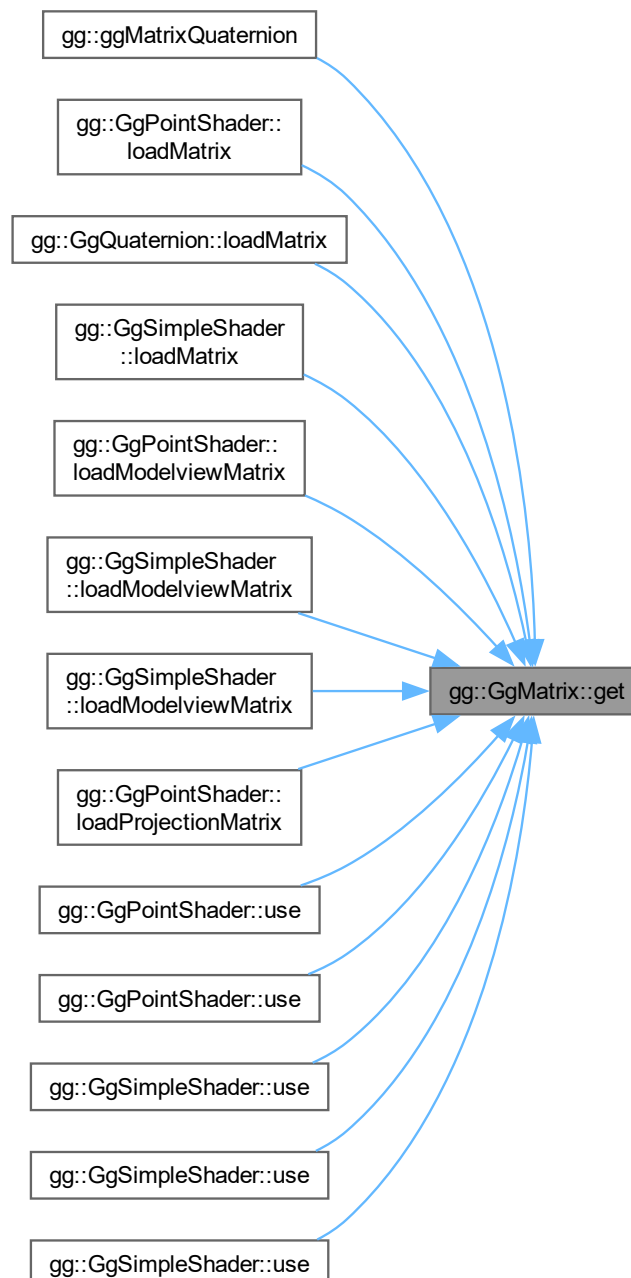
変換行列を取り出す.

戻り値

変換行列を格納した GLfloat 型の 16 要素の配列変数.

gg.h の 3123 行目に定義があります。

被呼び出し関係図:



7.5.3.3 get() [2/2]

```
void gg::GgMatrix::get (  
    GLfloat * a) const [inline]
```

変換行列を取り出す.

引数

<i>a</i>	変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

[gg.h](#) の 3133 行目に定義があります。

7.5.3.4 invert()

```
GgMatrix gg::GgMatrix::invert () const [inline]
```

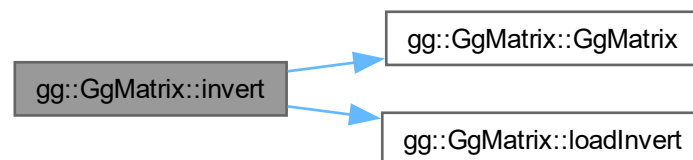
逆行列を返す.

戻り値

逆行列.

[gg.h](#) の 3044 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.5 loadFrustum()

```

gg::GgMatrix & gg::GgMatrix::loadFrustum (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)

```

透視投影変換行列を格納する.

引数

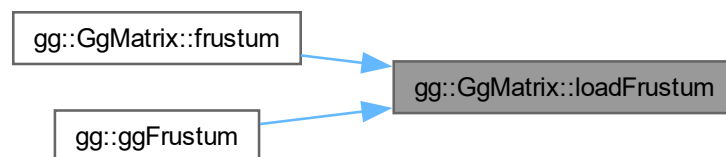
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

gg.cpp の 3123 行目に定義があります。

被呼び出し関係図:



7.5.3.6 loadIdentity()

```

gg::GgMatrix & gg::GgMatrix::loadIdentity ()

```

単位行列を格納する.

gg.cpp の 2734 行目に定義があります。

被呼び出し関係図:



7.5.3.7 loadInvert() [1/2]

```
GgMatrix & gg::GgMatrix::loadInvert (
    const GgMatrix & m) [inline]
```

逆行列を格納する.

引数

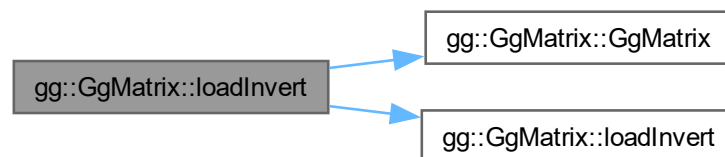
<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

設定した *m* の逆行列.

`gg.h` の 2723 行目に定義があります。

呼び出し関係図:



7.5.3.8 loadInvert() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadInvert (
    const GLfloat * a)
```

逆行列を格納する.

引数

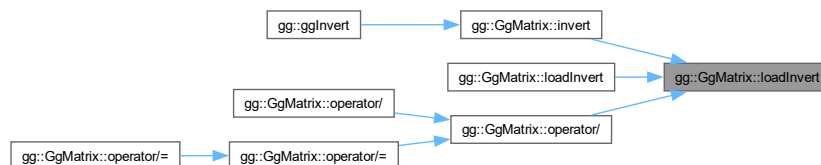
<i>a</i>	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した *a* の逆行列.

gg.cpp の 2909 行目に定義があります。

被呼び出し関係図:



7.5.3.9 loadLookat() [1/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) [inline]
```

ビュー変換行列を格納する.

引数

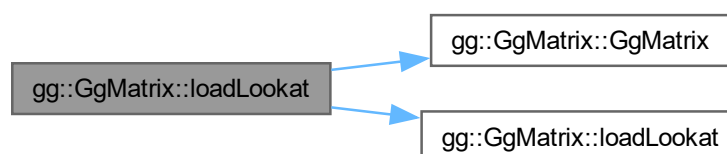
<i>e</i>	視点の位置の GgVector 型の変数.
<i>t</i>	目標点の位置の GgVector 型の変数.
<i>u</i>	上方向のベクトルの GgVector 型の変数.

戻り値

設定したビュー変換行列.

gg.h の 2643 行目に定義があります。

呼び出し関係図:



7.5.3.10 loadLookat() [2/3]

```
GgMatrix & gg::GgMatrix::loadLookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) [inline]
```

ビュー変換行列を格納する.

引数

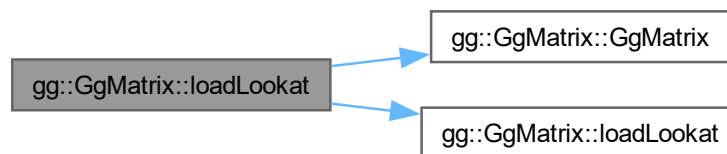
<i>e</i>	視点の位置の配列変数.
<i>t</i>	目標点の位置の配列変数.
<i>u</i>	上方向のベクトルの配列変数.

戻り値

設定したビュー変換行列.

[gg.h](#) の 2630 行目に定義があります。

呼び出し関係図:



7.5.3.11 loadLookat() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadLookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz)
```

ビュー変換行列を格納する.

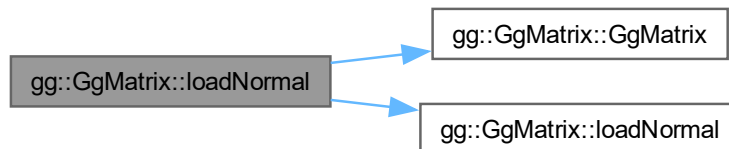
引数

戻り値

設定した m の法線変換行列.

[gg.h](#) の 2742 行目に定義があります。

呼び出し関係図:



7.5.3.13 loadNormal() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadNormal (
    const GLfloat * a)
```

法線変換行列を格納する.

引数

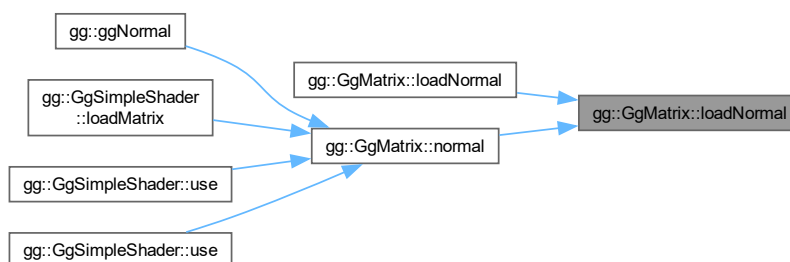
a	GLfloat 型の 16 要素の変換行列.
-----	------------------------

戻り値

設定した a の法線変換行列.

[gg.cpp](#) の 2992 行目に定義があります。

被呼び出し関係図:



7.5.3.14 loadOrthogonal()

```
gg::GgMatrix & gg::GgMatrix::loadOrthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar)
```

直交投影変換行列を格納する。

引数

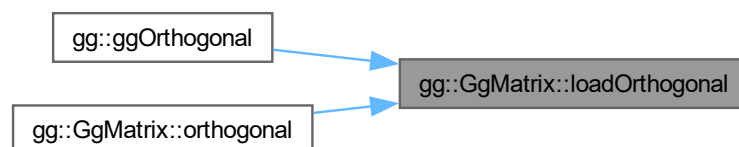
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した直交投影変換行列.

gg.cpp の 3082 行目に定義があります。

被呼び出し関係図:



7.5.3.15 loadPerspective()

```
gg::GgMatrix & gg::GgMatrix::loadPerspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar)
```

画角を指定して透視投影変換行列を格納する。

引数

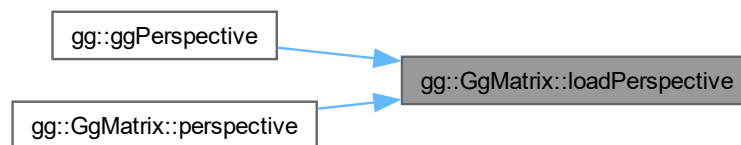
<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

設定した透視投影変換行列.

`gg.cpp` の 3164 行目に定義があります。

被呼び出し関係図:



7.5.3.16 loadRotate() [1/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

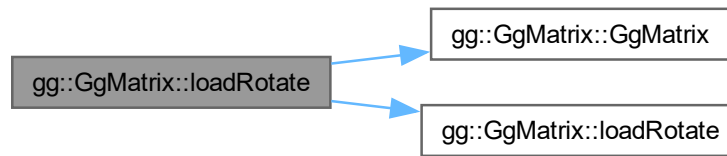
<i>r</i>	回転軸の方向ベクトルと回転角を格納した <code>GgVector</code> 型のベクトル, 第 4 要素を回転角に用いる.
----------	---

戻り値

設定した変換行列.

`gg.h` の 2599 行目に定義があります。

呼び出し関係図:



7.5.3.17 loadRotate() [2/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GgVector & r,
    GLfloat a) [inline]
```

`r` 方向のベクトルを軸とする回転の変換行列を格納する.

引数

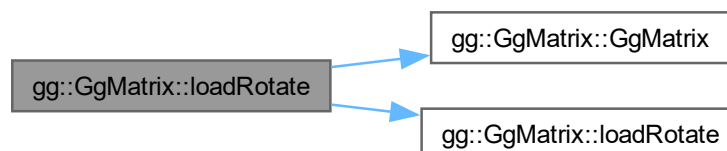
<i>r</i>	回転軸の方向ベクトルを格納した <code>GgVector</code> 型のベクトル, 第 4 要素は無視する.
<i>a</i>	回転角.

戻り値

設定した変換行列.

`gg.h` の 2577 行目に定義があります。

呼び出し関係図:



7.5.3.18 loadRotate() [3/5]

```
GgMatrix & gg::GgMatrix::loadRotate (  
    const GLfloat * r) [inline]
```

r方向のベクトルを軸とする回転の変換行列を格納する.

引数

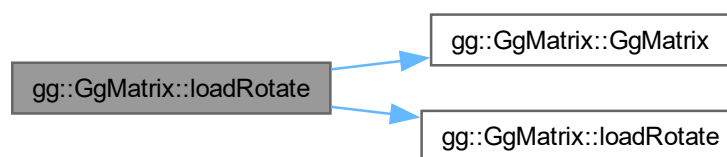
<i>r</i>	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数, 第 4 要素を回転角に用いる.
----------	---

戻り値

設定した変換行列.

gg.h の 2588 行目に定義があります。

呼び出し関係図:



7.5.3.19 loadRotate() [4/5]

```
GgMatrix & gg::GgMatrix::loadRotate (
    const GLfloat * r,
    GLfloat a) [inline]
```

r 方向のベクトルを軸とする回転の変換行列を格納する.

引数

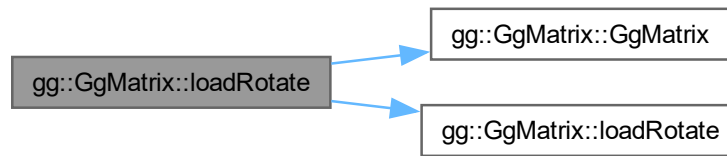
<i>r</i>	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
<i>a</i>	回転角.

戻り値

設定した変換行列.

gg.h の 2565 行目に定義があります。

呼び出し関係図:



7.5.3.20 loadRotate() [5/5]

```
gg::GgMatrix & gg::GgMatrix::loadRotate (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat a)
```

(x, y, z) 方向のベクトルを軸とする回転の変換行列を格納する.

引数

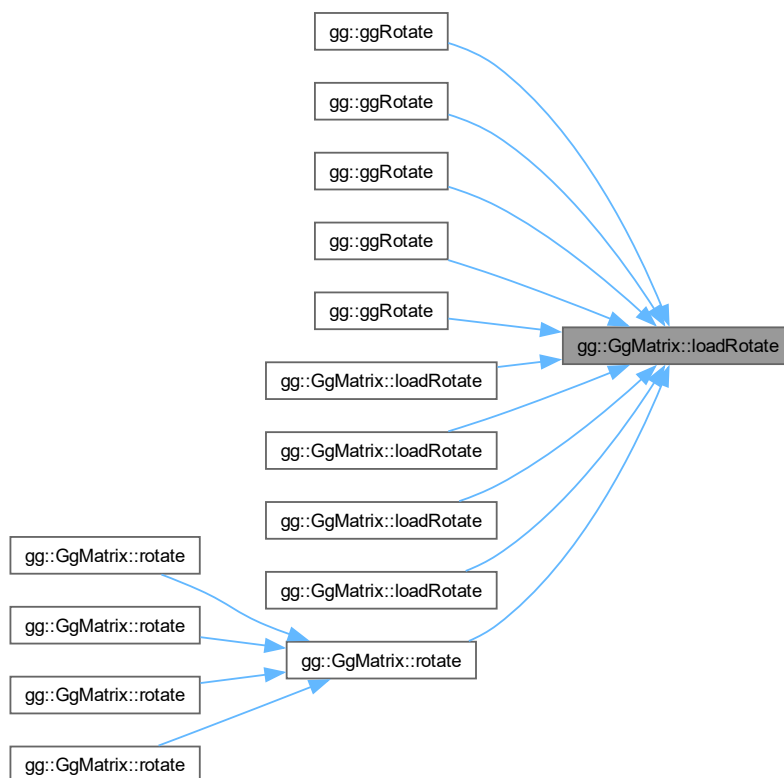
<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

設定した変換行列.

[gg.cpp](#) の [2839](#) 行目に定義があります。

被呼び出し関係図:



7.5.3.21 loadRotateX()

```
gg::GgMatrix & gg::GgMatrix::loadRotateX (
    GLfloat a)
```

x 軸中心の回転の変換行列を格納する.

引数

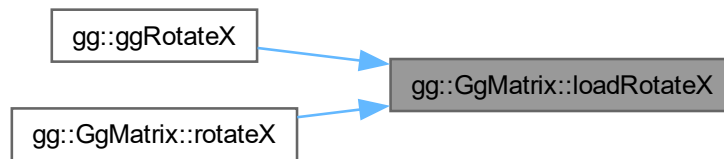
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

gg.cpp の 2782 行目に定義があります。

被呼び出し関係図:



7.5.3.22 loadRotateY()

```
gg::GgMatrix & gg::GgMatrix::loadRotateY (
    GLfloat a)
```

y 軸中心の回転の変換行列を格納する.

引数

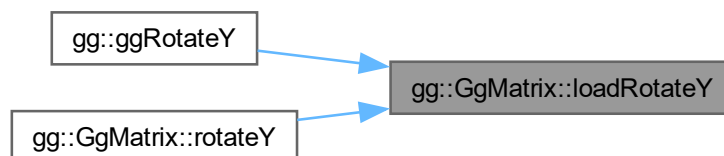
<i>a</i>	回転角.
----------	------

戻り値

設定した変換行列.

[gg.cpp](#) の 2801 行目に定義があります。

被呼び出し関係図:



7.5.3.23 loadRotateZ()

```
gg::GgMatrix & gg::GgMatrix::loadRotateZ (
    GLfloat a)
```

z 軸中心の回転の変換行列を格納する.

引数

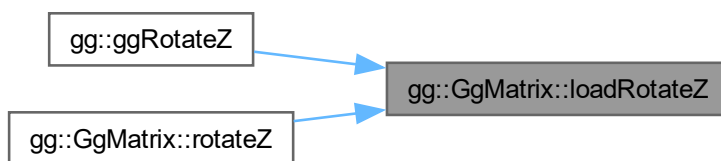
<code>a</code>	回転角.
----------------	------

戻り値

設定した変換行列.

`gg.cpp` の 2820 行目に定義があります。

被呼び出し関係図:



7.5.3.24 loadScale() [1/3]

```
GgMatrix & gg::GgMatrix::loadScale (  
    const GgVector & s) [inline]
```

拡大縮小の変換行列を格納する.

引数

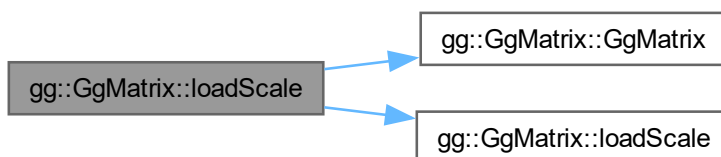
<code>s</code>	拡大率の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

設定した変換行列.

`gg.h` の 2518 行目に定義があります。

呼び出し関係図:



7.5.3.25 loadScale() [2/3]

```
GgMatrix & gg::GgMatrix::loadScale (
    const GLfloat * s) [inline]
```

拡大縮小の変換行列を格納する.

引数

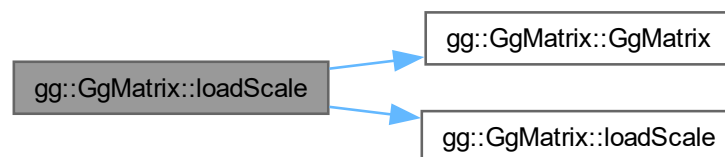
s	拡大率の GLfloat 型の配列 (x, y, z).
---	------------------------------

戻り値

設定した変換行列.

gg.h の 2507 行目に定義があります。

呼び出し関係図:



7.5.3.26 loadScale() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadScale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

拡大縮小の変換行列を格納する.

引数

x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	w 拡大率のスケールファクタ (= 1.0f).

7.5.3.28 loadTranslate() [2/3]

```
GgMatrix & gg::GgMatrix::loadTranslate (
    const GLfloat * t) [inline]
```

平行移動の変換行列を格納する.

引数

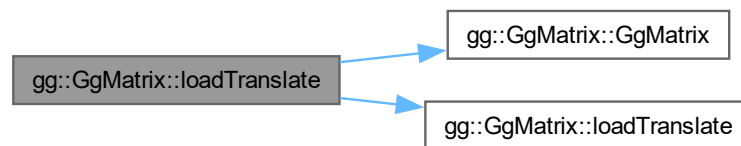
<i>t</i>	移動量の GLfloat 型の配列 (x, y, z).
----------	------------------------------

戻り値

設定した変換行列.

gg.h の 2474 行目に定義があります。

呼び出し関係図:



7.5.3.29 loadTranslate() [3/3]

```
gg::GgMatrix & gg::GgMatrix::loadTranslate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f)
```

平行移動の変換行列を格納する.

引数

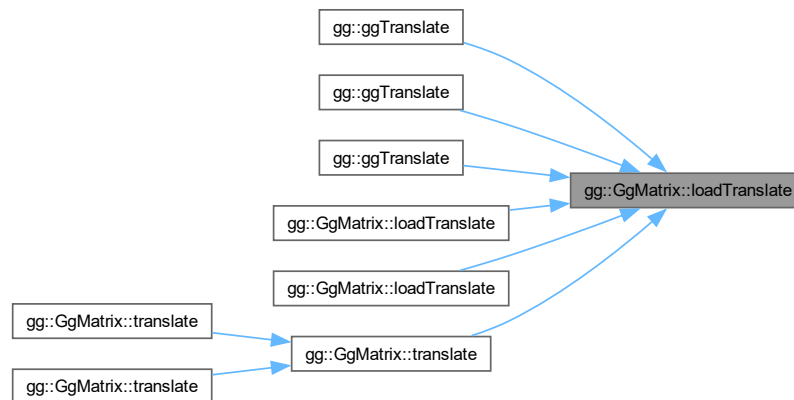
<i>x</i>	x 方向の移動量.
<i>y</i>	y 方向の移動量.
<i>z</i>	z 方向の移動量.
<i>w</i>	w 移動量のスケールファクタ (= 1.0f).

戻り値

設定した変換行列.

gg.cpp の 2750 行目に定義があります。

被呼び出し関係図:



7.5.3.30 loadTranspose() [1/2]

```
GgMatrix & gg::GgMatrix::loadTranspose (
    const GgMatrix & m) [inline]
```

転置行列を格納する.

引数

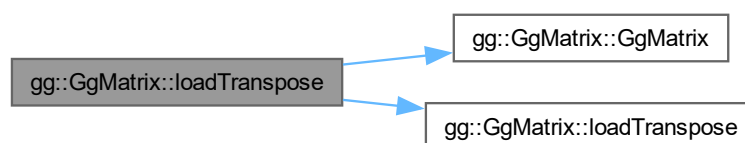
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

設定した *m* の転置行列.

gg.h の 2704 行目に定義があります。

呼び出し関係図:



7.5.3.31 loadTranspose() [2/2]

```
gg::GgMatrix & gg::GgMatrix::loadTranspose (
    const GLfloat * a)
```

転置行列を格納する.

引数

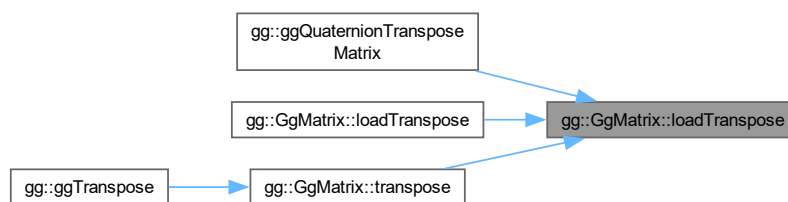
a	GLfloat 型の 16 要素の変換行列.
----------	------------------------

戻り値

設定した **a** の転置行列.

[gg.cpp](#) の 2881 行目に定義があります。

被呼び出し関係図:



7.5.3.32 lookat() [1/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GgVector & e,
    const GgVector & t,
    const GgVector & u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

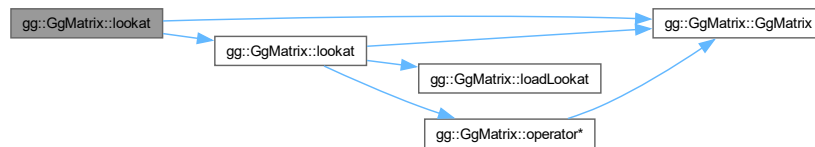
e	視点の位置を格納した GgVector 型の変数.
t	目標点の位置を格納した GgVector 型の変数.
u	上方向のベクトルを格納した GgVector 型の変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2963 行目に定義があります。

呼び出し関係図:



7.5.3.33 lookat() [2/3]

```
GgMatrix gg::GgMatrix::lookat (
    const GLfloat * e,
    const GLfloat * t,
    const GLfloat * u) const [inline]
```

ビュー変換を乗じた結果を返す.

引数

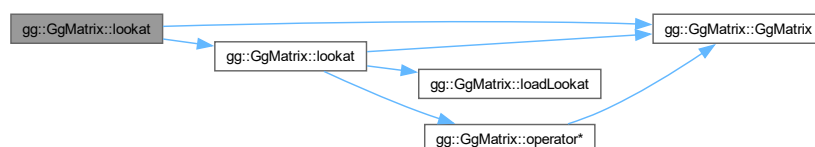
<i>e</i>	視点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>t</i>	目標点の位置を格納した GLfloat 型の 3 要素の配列変数.
<i>u</i>	上方向のベクトルを格納した GLfloat 型の 3 要素の配列変数.

戻り値

ビュー変換行列を乗じた変換行列.

gg.h の 2950 行目に定義があります。

呼び出し関係図:



7.5.3.34 lookat() [3/3]

```
GgMatrix gg::GgMatrix::lookat (
    GLfloat ex,
    GLfloat ey,
    GLfloat ez,
    GLfloat tx,
    GLfloat ty,
    GLfloat tz,
    GLfloat ux,
    GLfloat uy,
    GLfloat uz) const [inline]
```

ビュー変換を乗じた結果を返す。

引数

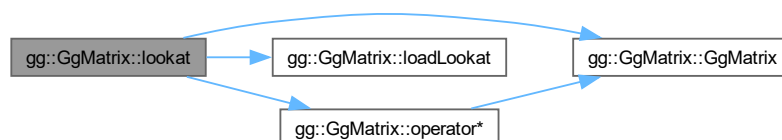
<i>ex</i>	視点の位置の x 座標値.
<i>ey</i>	視点の位置の y 座標値.
<i>ez</i>	視点の位置の z 座標値.
<i>tx</i>	目標点の位置の x 座標値.
<i>ty</i>	目標点の位置の y 座標値.
<i>tz</i>	目標点の位置の z 座標値.
<i>ux</i>	上方向のベクトルの x 成分.
<i>uy</i>	上方向のベクトルの y 成分.
<i>uz</i>	上方向のベクトルの z 成分.

戻り値

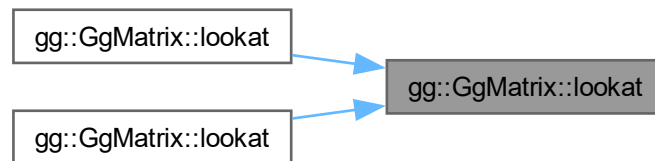
ビュー変換行列を乗じた変換行列。

[gg.h](#) の 2932 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.35 normal()

```
GgMatrix gg::GgMatrix::normal () const [inline]
```

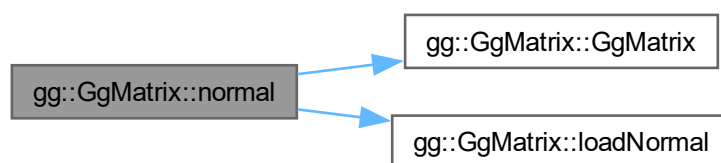
法線変換行列を返す.

戻り値

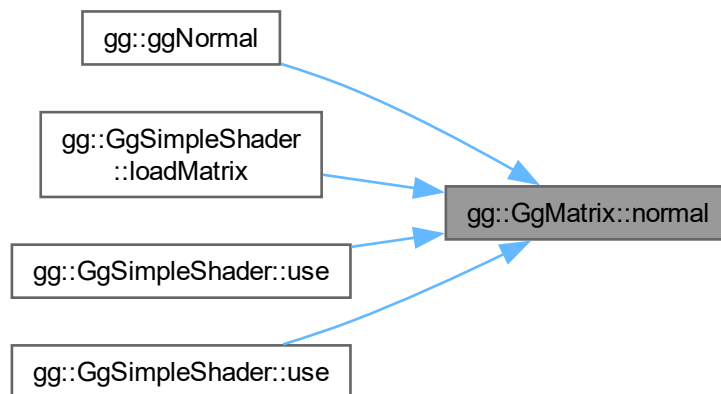
法線変換行列.

gg.h の 3055 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.36 operator*() [1/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を乗算した値を返す.

引数

<i>m</i>	<code>GgMatrix</code> 型の変換行列.
----------	-------------------------------

戻り値

変換行列に *m* を乗じた `GgMatrix` 型の変換行列.

`gg.h` の 2376 行目に定義があります。

呼び出し関係図:



7.5.3.37 operator*() [2/3]

```
GgVector gg::GgMatrix::operator* (  
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<code>v</code>	元のベクトルの <code>GgVector</code> 型の変数.
----------------	-------------------------------------

戻り値

`v` 変換結果の `GgVector` 型のベクトル.

`gg.h` の 3111 行目に定義があります。

7.5.3.38 operator*() [3/3]

```
GgMatrix gg::GgMatrix::operator* (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を乗算した値を返す.

引数

<code>a</code>	変換行列を格納した <code>GLfloat</code> 型の 16 要素の配列変数.
----------------	---

戻り値

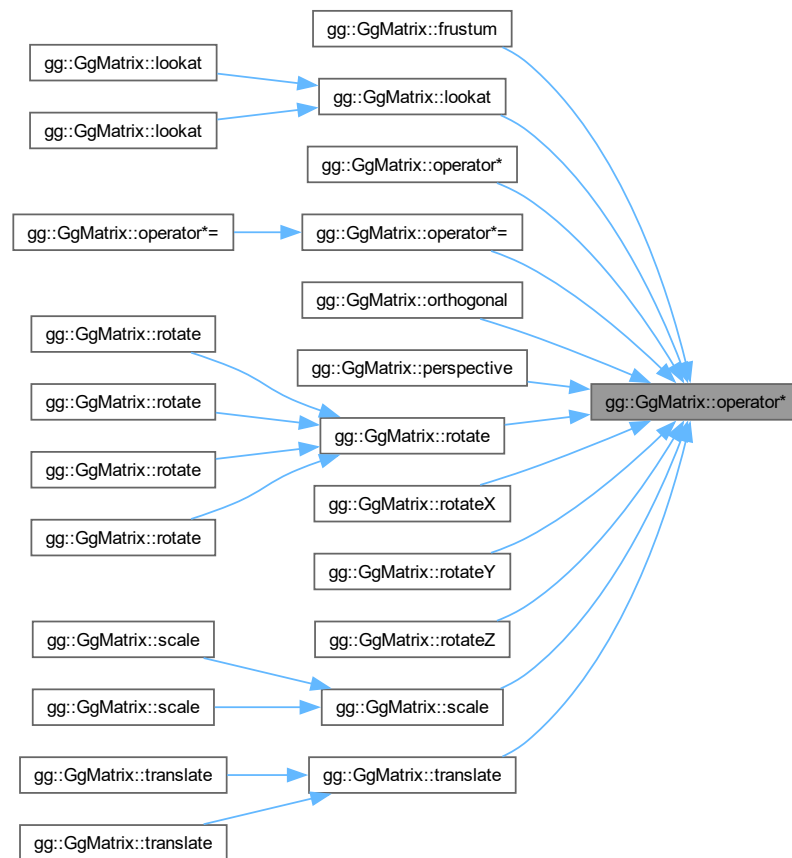
変換行列に `a` を乗じた `GgMatrix` 型の変換行列.

`gg.h` の 2363 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.39 operator*=() [1/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を乗算した結果を格納する。

引数

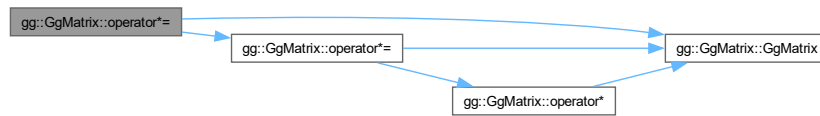
<i>m</i>	GgMatrix 型の変換行列。
----------	------------------

戻り値

m を掛けた変換行列の参照。

gg.h の 2399 行目に定義があります。

呼び出し関係図:



7.5.3.40 operator*=() [2/2]

```
GgMatrix & gg::GgMatrix::operator*= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を乗算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を掛けた変換行列の参照.

gg.h の 2387 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.41 operator+() [1/2]

```
GgMatrix gg::GgMatrix::operator+ (  
    const GgMatrix & m) const [inline]
```

変換行列に別の変換行列を加算した値を返す.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列に *m* を加えた GgMatrix 型の変換行列.

gg.h の 2282 行目に定義があります。

呼び出し関係図:



7.5.3.42 operator+() [2/2]

```
GgMatrix gg::GgMatrix::operator+ (
    const GLfloat * a) const [inline]
```

変換行列に配列に格納した変換行列を加算した値を返す.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列に *a* を加えた GgMatrix 型の変換行列.

gg.h の 2269 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.43 operator+=() [1/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を加算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を加えた変換行列の参照.

gg.h の 2305 行目に定義があります。

呼び出し関係図:



7.5.3.44 operator+=() [2/2]

```
GgMatrix & gg::GgMatrix::operator+= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を加算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を加えた変換行列の参照.

gg.h の 2293 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.45 operator-() [1/2]

```
GgMatrix gg::GgMatrix::operator- (  
    const GgMatrix & m) const [inline]
```

変換行列から別の変換行列を減算した値を返す.

引数

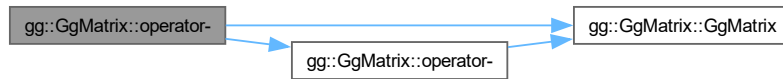
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列から m を引いた `GgMatrix` 型の変換行列.

`gg.h` の 2329 行目に定義があります。

呼び出し関係図:



7.5.3.46 operator-() [2/2]

```
GgMatrix gg::GgMatrix::operator- (
    const GLfloat * a) const [inline]
```

変換行列から配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

変換行列から a を引いた `GgMatrix` 型の変換行列.

`gg.h` の 2316 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.47 operator-=() [1/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GgMatrix & m) [inline]
```

変換行列に別の変換行列を減算した結果を格納する.

引数

<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m を引いた変換行列の参照.

gg.h の 2352 行目に定義があります。

呼び出し関係図:



7.5.3.48 operator-=() [2/2]

```
GgMatrix & gg::GgMatrix::operator-= (
    const GLfloat * a) [inline]
```

変換行列に配列に格納した変換行列を減算した結果を格納する.

引数

<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a を引いた変換行列の参照.

gg.h の 2340 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.49 operator/() [1/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GgMatrix & m) const [inline]
```

変換行列を変換行列で除算した値を返す.

引数

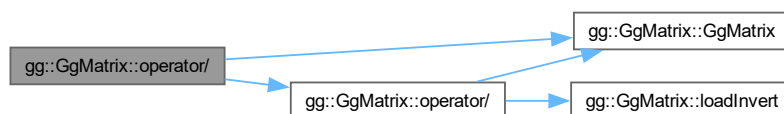
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

変換行列を m で割った GgMatrix 型の変換行列.

gg.h の 2424 行目に定義があります。

呼び出し関係図:



7.5.3.50 operator/() [2/2]

```
GgMatrix gg::GgMatrix::operator/ (
    const GLfloat * a) const [inline]
```

変換行列を配列に格納した変換行列で除算した値を返す.

引数

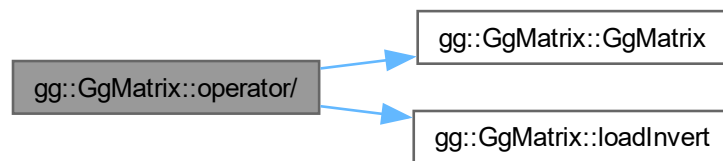
a	変換行列を格納した GLfloat 型の 16 要素の配列変数.
---	----------------------------------

戻り値

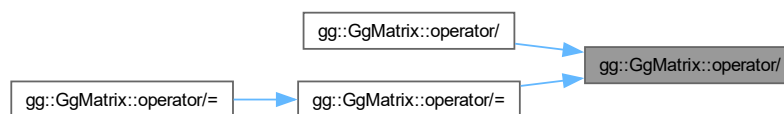
変換行列を a で割った GgMatrix 型の変換行列.

gg.h の 2410 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.51 operator/=() [1/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GgMatrix & m) [inline]
```

変換行列を別の変換行列で除算した結果を格納する.

引数

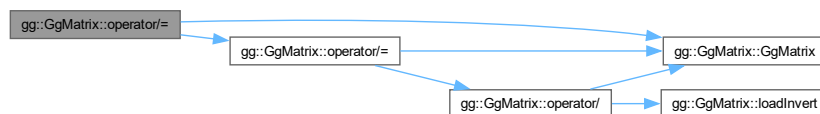
<i>m</i>	GgMatrix 型の変換行列.
----------	------------------

戻り値

m で割った変換行列の参照.

gg.h の 2447 行目に定義があります。

呼び出し関係図:



7.5.3.52 operator/=() [2/2]

```
GgMatrix & gg::GgMatrix::operator/= (
    const GLfloat * a) [inline]
```

変換行列を配列に格納した変換行列で除算した結果を格納する.

引数

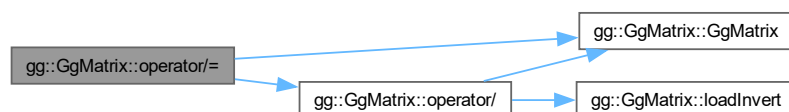
<i>a</i>	変換行列を格納した GLfloat 型の 16 要素の配列変数.
----------	----------------------------------

戻り値

a で割った変換行列の参照.

gg.h の 2435 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.53 operator=() [1/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GgMatrix & m) [default]
```

代入演算子. 呼び出し関係図:



7.5.3.54 operator=() [2/3]

```
GgMatrix & gg::GgMatrix::operator= (
    const GLfloat * a) [inline]
```

配列変数の値を格納する.

引数

<i>a</i>	GLfloat 型の 16 要素の配列変数.
----------	------------------------

戻り値

a を代入したこのオブジェクトの参照.

[gg.h](#) の 2257 行目に定義があります。

呼び出し関係図:



7.5.3.55 operator=() [3/3]

```
GgMatrix & gg::GgMatrix::operator= (
    GgMatrix && m) [default]
```

ムーブ代入演算子. 呼び出し関係図:



7.5.3.56 orthogonal()

```
GgMatrix gg::GgMatrix::orthogonal (
    GLfloat left,
    GLfloat right,
    GLfloat bottom,
    GLfloat top,
    GLfloat zNear,
    GLfloat zFar) const [inline]
```

直交投影変換を乗じた結果を返す.

引数

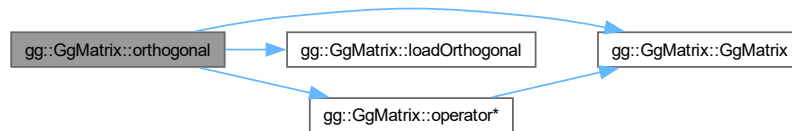
<i>left</i>	ウィンドウの左端の位置.
<i>right</i>	ウィンドウの右端の位置.
<i>bottom</i>	ウィンドウの下端の位置.
<i>top</i>	ウィンドウの上端の位置.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

直交投影変換行列を乗じた変換行列.

gg.h の 2979 行目に定義があります。

呼び出し関係図:



7.5.3.57 perspective()

```

GgMatrix gg::GgMatrix::perspective (
    GLfloat fovy,
    GLfloat aspect,
    GLfloat zNear,
    GLfloat zFar) const [inline]
  
```

画角を指定して透視投影変換を乗じた結果を返す.

引数

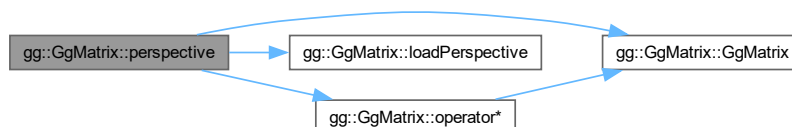
<i>fovy</i>	y 方向の画角.
<i>aspect</i>	縦横比.
<i>zNear</i>	視点から前方面までの位置.
<i>zFar</i>	視点から後方面までの位置.

戻り値

透視投影変換行列を乗じた変換行列.

gg.h の 3019 行目に定義があります。

呼び出し関係図:



7.5.3.58 projection() [1/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GgVector 型の変数.
v	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3100 行目に定義があります。

7.5.3.59 projection() [2/4]

```
void gg::GgMatrix::projection (
    GgVector & c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GgVector 型の変数.
v	元のベクトルの GLfloat 型の 4 要素の配列変数.

[gg.h](#) の 3089 行目に定義があります。

7.5.3.60 projection() [3/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GgVector & v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

c	変換結果を格納する GLfloat 型の 4 要素の配列変数.
v	元のベクトルの GgVector 型の変数.

[gg.h](#) の 3078 行目に定義があります。

7.5.3.61 projection() [4/4]

```
void gg::GgMatrix::projection (
    GLfloat * c,
    const GLfloat * v) const [inline]
```

ベクトルに対して投影変換を行う。

引数

<code>c</code>	変換結果を格納する <code>GLfloat</code> 型の 4 要素の配列変数.
<code>v</code>	元のベクトルの <code>GLfloat</code> 型の 4 要素の配列変数.

`gg.h` の 3067 行目に定義があります。

7.5.3.62 rotate() [1/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r) const [inline]
```

`r` 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

<code>r</code>	回転軸の方向ベクトルと回転角を格納した <code>GgVector</code> 型の変数).
----------------	--

戻り値

(`r[0]`, `r[1]`, `r[2]`) を軸にさらに `r[3]` 回転した変換行列.

`gg.h` の 2913 行目に定義があります。

呼び出し関係図:



7.5.3.63 rotate() [2/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GgVector & r,
    GLfloat a) const [inline]
```

`r` 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

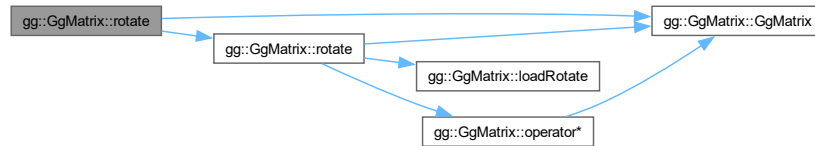
<code>r</code>	回転軸の方向ベクトルを格納した <code>GgVector</code> 型の変数.
<code>a</code>	回転角.

戻り値

(r[0], r[1], r[2]) を軸にさらに a 回転した変換行列.

gg.h の 2891 行目に定義があります。

呼び出し関係図:



7.5.3.64 rotate() [3/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r) const [inline]
```

r 方向のベクトルを軸とする回転の変換行列を乗じた結果を返す.

引数

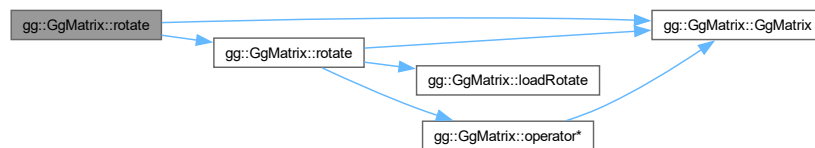
r	回転軸の方向ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数 (x, y, z, a).
---	--

戻り値

(r[0], r[1], r[2]) を軸にさらに r[3] 回転した変換行列.

gg.h の 2902 行目に定義があります。

呼び出し関係図:



7.5.3.65 rotate() [4/5]

```
GgMatrix gg::GgMatrix::rotate (
    const GLfloat * r,
    GLfloat a) const [inline]
```

r 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

<i>r</i>	回転軸の方向ベクトルを格納した GLfloat 型の 3 要素の配列変数 (x, y, z).
<i>a</i>	回転角.

戻り値

(*r*[0], *r*[1], *r*[2]) を軸にさらに *a* 回転した変換行列.

[gg.h](#) の 2879 行目に定義があります。

呼び出し関係図:



7.5.3.66 rotate() [5/5]

```
GgMatrix gg::GgMatrix::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

(*x*, *y*, *z*) 方向のベクトルを軸とする回転変換を乗じた結果を返す.

引数

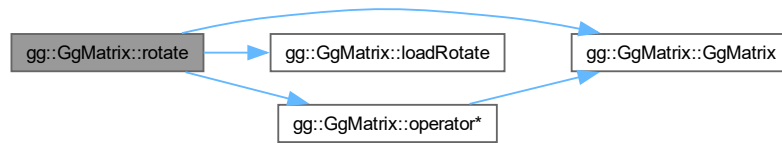
<i>x</i>	回転軸の x 成分.
<i>y</i>	回転軸の y 成分.
<i>z</i>	回転軸の z 成分.
<i>a</i>	回転角.

戻り値

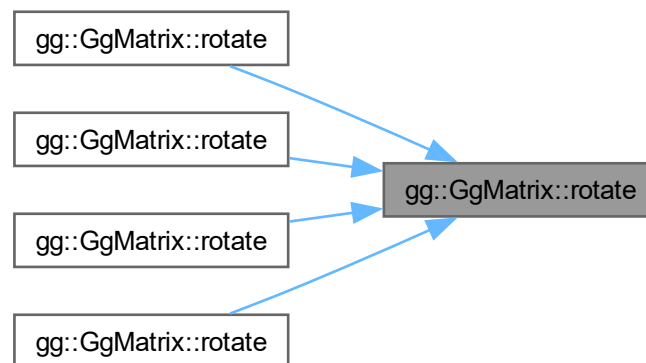
(*x*, *y*, *z*) を軸にさらに *a* 回転した変換行列.

[gg.h](#) の 2866 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.67 rotateX()

```
GgMatrix gg::GgMatrix::rotateX (
    GLfloat a) const [inline]
```

x 軸中心の回転変換を乗じた結果を返す.

引数

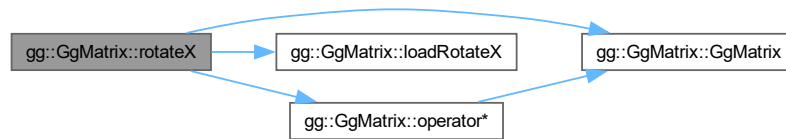
<i>a</i>	回転角.
----------	------

戻り値

x 軸中心にさらに *a* 回転した変換行列.

gg.h の 2827 行目に定義があります。

呼び出し関係図:



7.5.3.68 rotateY()

```
GgMatrix gg::GgMatrix::rotateY (
    GLfloat a) const [inline]
```

y 軸中心の回転変換を乗じた結果を返す.

引数

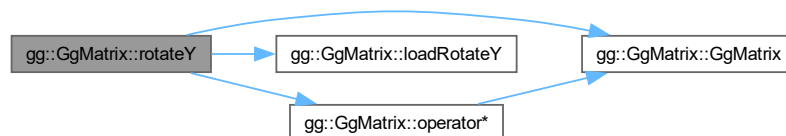
a	回転角.
---	------

戻り値

y 軸中心にさらに a 回転した変換行列.

gg.h の 2839 行目に定義があります。

呼び出し関係図:



7.5.3.69 rotateZ()

```
GgMatrix gg::GgMatrix::rotateZ (
    GLfloat a) const [inline]
```

z 軸中心の回転変換を乗じた結果を返す.

引数

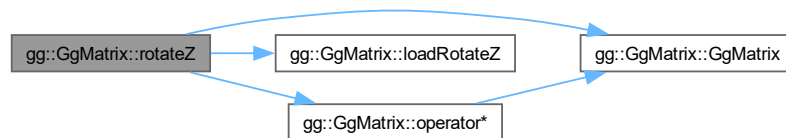
<code>a</code>	回転角.
----------------	------

戻り値

z 軸中心にさらに a 回転した変換行列.

gg.h の 2851 行目に定義があります。

呼び出し関係図:



7.5.3.70 scale() [1/3]

```
GgMatrix gg::GgMatrix::scale (
    const GgVector & s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

<code>s</code>	拡大率の <code>GgVector</code> 型の変数.
----------------	----------------------------------

戻り値

拡大縮小した結果の変換行列.

gg.h の 2816 行目に定義があります。

呼び出し関係図:



7.5.3.71 `scale()` [2/3]

```
GgMatrix gg::GgMatrix::scale (  
    const GLfloat * s) const [inline]
```

拡大縮小変換を乗じた結果を返す.

引数

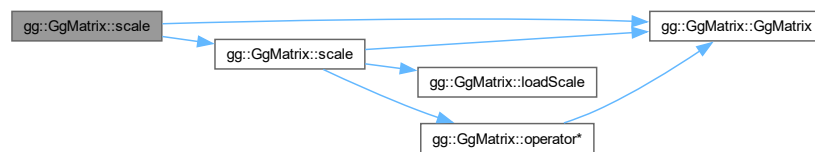
s	拡大率の GLfloat 型の 3 要素の配列変数 (x, y, z).
---	--------------------------------------

戻り値

拡大縮小した結果の変換行列.

gg.h の 2805 行目に定義があります。

呼び出し関係図:



7.5.3.72 scale() [3/3]

```

GgMatrix gg::GgMatrix::scale (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
  
```

拡大縮小変換を乗じた結果を返す.

引数

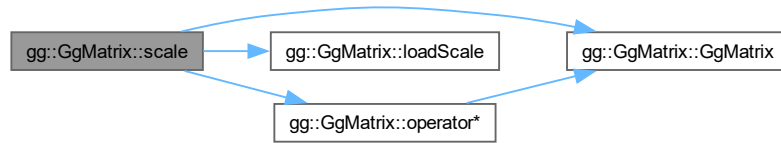
x	x 方向の拡大率.
y	y 方向の拡大率.
z	z 方向の拡大率.
w	w 移動量のスケールファクタ (= 1.0f).

戻り値

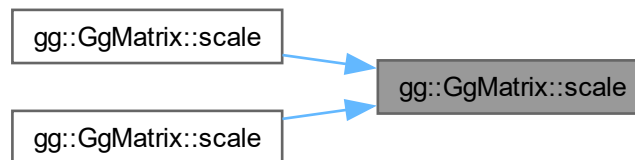
拡大縮小した結果の変換行列.

gg.h の 2793 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.73 translate() [1/3]

```
GgMatrix gg::GgMatrix::translate (
    const GgVector & t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

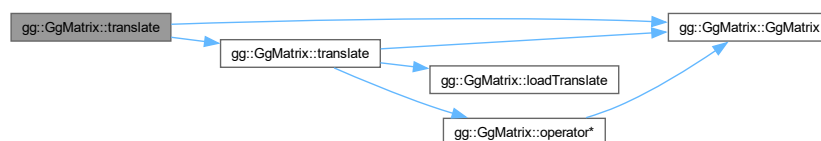
<i>t</i>	移動量の GgVector 型の変数.
----------	-------------------------------------

戻り値

平行移動した結果の変換行列.

[gg.h](#) の 2779 行目に定義があります。

呼び出し関係図:



7.5.3.74 translate() [2/3]

```
GgMatrix gg::GgMatrix::translate (
    const GLfloat * t) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

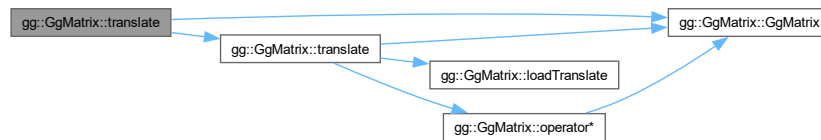
<i>t</i>	移動量の GLfloat 型の 3 要素の配列変数 (x, y, z).
----------	--------------------------------------

戻り値

平行移動した結果の変換行列.

gg.h の 2768 行目に定義があります。

呼び出し関係図:



7.5.3.75 translate() [3/3]

```
GgMatrix gg::GgMatrix::translate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f) const [inline]
```

平行移動変換を乗じた結果を返す.

引数

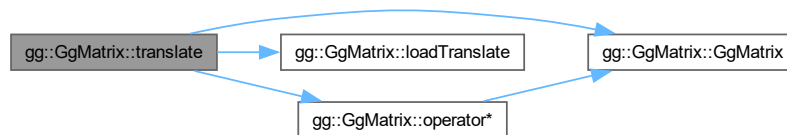
<i>x</i>	<i>x</i> 方向の移動量.
<i>y</i>	<i>y</i> 方向の移動量.
<i>z</i>	<i>z</i> 方向の移動量.
<i>w</i>	<i>w</i> 移動量のスケールファクタ (= 1.0f).

戻り値

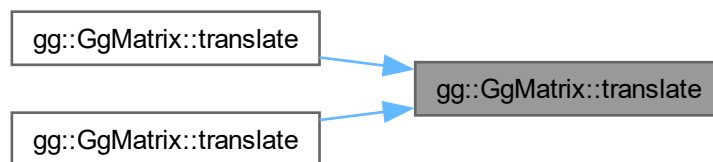
平行移動した結果の変換行列.

[gg.h](#) の 2756 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.5.3.76 transpose()

```
GgMatrix gg::GgMatrix::transpose () const [inline]
```

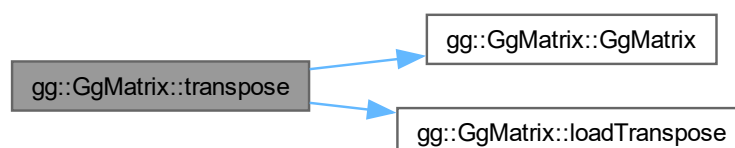
転置行列を返す.

戻り値

転置行列.

[gg.h](#) の 3033 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

7.6 gg::GgNormalTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgNormalTexture](#) ()
- [GgNormalTexture](#) (const GLubyte *image, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- [GgNormalTexture](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- virtual [~GgNormalTexture](#) ()
- void [load](#) (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format=GL_RED, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- void [load](#) (const std::string &name, GLfloat nz=1.0f, GLenum internal=GL_RGBA)
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const [GgTexture](#) * [get](#) () const
- GLuint [getTexture](#) () const
- GLsizei [getWidth](#) () const
- GLsizei [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- void [bind](#) () const
- void [unbind](#) () const

7.6.1 詳解

法線マップ.

覚え書き

高さマップ（グレイスケール画像）を読み込んで法線マップのテクスチャを作成する.

[gg.h](#) の 5615 行目に定義があります。

7.6.2 構築子と解体子

7.6.2.1 GgNormalTexture() [1/3]

```
gg::GgNormalTexture::GgNormalTexture () [inline]
```

コンストラクタ.

[gg.h](#) の 5625 行目に定義があります。

7.6.2.2 GgNormalTexture() [2/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const GLubyte * image,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

メモリ上のデータから法線マップのテクスチャを作成するコンストラクタ.

引数

<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャとして用いる画像データの横幅.
<i>height</i>	テクスチャとして用いる画像データの高さ.
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

[gg.h](#) の 5639 行目に定義があります。

呼び出し関係図:



7.6.2.3 GgNormalTexture() [3/3]

```
gg::GgNormalTexture::GgNormalTexture (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

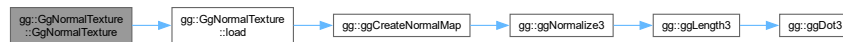
ファイルからデータを読み込んで法線マップのテクスチャを作成するコンストラクタ.

引数

<i>name</i>	画像ファイル名.
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

gg.h の 5659 行目に定義があります。

呼び出し関係図:



7.6.2.4 ~GgNormalTexture()

```
virtual gg::GgNormalTexture::~~GgNormalTexture () [inline], [virtual]
```

デストラクタ.

gg.h の 5672 行目に定義があります。

7.6.3 関数詳解

7.6.3.1 bind()

```
void gg::GgNormalTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

gg.h の 5801 行目に定義があります。

7.6.3.2 get()

```
const GgTexture * gg::GgNormalTexture::get () const [inline]
```

テクスチャオブジェクトを取り出す.

戻り値

GgTexture オブジェクトのポインタ.

gg.h の 5743 行目に定義があります。

7.6.3.3 getHeight()

```
GLsizei gg::GgNormalTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す。

戻り値

テクスチャの縦の画素数。

[gg.h](#) の 5773 行目に定義があります。

7.6.3.4 getSize() [1/2]

```
const GLsizei * gg::GgNormalTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す。

戻り値

テクスチャのサイズを格納した配列へのポインタ。

[gg.h](#) の 5793 行目に定義があります。

7.6.3.5 getSize() [2/2]

```
void gg::GgNormalTexture::getSize (  
    GLsizei * size) const [inline]
```

使用しているテクスチャのサイズを取り出す。

引数

size	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数。
------	--------------------------------------

[gg.h](#) の 5783 行目に定義があります。

7.6.3.6 getTexture()

```
GLuint gg::GgNormalTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る。

戻り値

テクスチャ名。

[gg.h](#) の 5753 行目に定義があります。

7.6.3.7 getWidth()

```
GLsizei gg::GgNormalTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す。

戻り値

テクスチャの横の画素数。

gg.h の 5763 行目に定義があります。

7.6.3.8 load() [1/2]

```
void gg::GgNormalTexture::load (
    const GLubyte * hmap,
    GLsizei width,
    GLsizei height,
    GLenum format = GL_RED,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA) [inline]
```

メモリ上のデータから法線マップのテクスチャを作成する。

引数

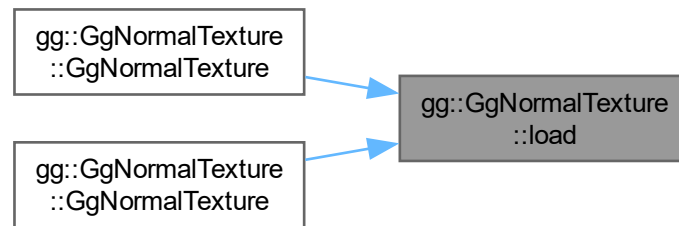
<i>hmap</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない。
<i>width</i>	テクスチャとして用いる画像データの横幅。
<i>height</i>	テクスチャとして用いる画像データの高さ。
<i>format</i>	テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA)。
<i>nz</i>	法線マップの z 成分の値。
<i>internal</i>	テクスチャの内部フォーマット。

gg.h の 5686 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.6.3.9 load() [2/2]

```
void gg::GgNormalTexture::load (
    const std::string & name,
    GLfloat nz = 1.0f,
    GLenum internal = GL_RGBA)
```

TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する.

引数

<i>name</i>	画像ファイル名 (1 チャンネルの TGA 画像).
<i>nz</i>	法線マップの z 成分の値.
<i>internal</i>	テクスチャの内部フォーマット.

[gg.cpp](#) の 4079 行目に定義があります。

呼び出し関係図:



7.6.3.10 operator bool()

```
gg::GgNormalTexture::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる.

戻り値

オブジェクトが有効なら true.

[gg.h](#) の 5723 行目に定義があります。

7.6.3.11 operator!()

```
bool gg::GgNormalTexture::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する.

戻り値

オブジェクトが有効なら **false**, 無効なら **true**.

[gg.h](#) の 5733 行目に定義があります。

7.6.3.12 unbind()

```
void gg::GgNormalTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

[gg.h](#) の 5809 行目に定義があります。

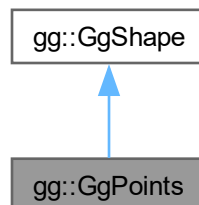
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

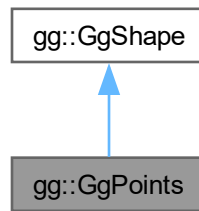
7.7 gg::GgPoints クラス

```
#include <gg.h>
```

gg::GgPoints の継承関係図



gg::GgPoints 連携図



公開メンバ関数

- [GgPoints](#) (GLenum mode=GL_POINTS)
- [GgPoints](#) (const [GgVector](#) *pos, GLsizei countv, GLenum mode=GL_POINTS, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgPoints](#) ()
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVector](#) *pos, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVector](#) *pos, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

7.7.1 詳解

点.

[gg.h](#) の 6616 行目に定義があります。

7.7.2 構築子と解体子

7.7.2.1 GgPoints() [1/2]

```
gg::GgPoints::GgPoints (
    GLenum mode = GL_POINTS) [inline]
```

コンストラクタ.

[gg.h](#) の 6627 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.7.2.2 GgPoints() [2/2]

```
gg::GgPoints::GgPoints (
    const GgVector * pos,
    GLsizei countv,
    GLenum mode = GL_POINTS,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

コンストラクタ.

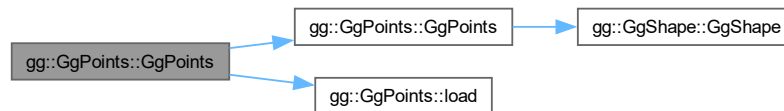
引数

<i>pos</i>	この図形の頂点の位置のデータの配列 (nullptr ならデータを転送しない).
<i>countv</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.

<i>usage</i>	バッファオブジェクトの使い方.
--------------	-----------------

[gg.h](#) の 6640 行目に定義があります。

呼び出し関係図:



7.7.2.3 ~GgPoints()

```
virtual gg::GgPoints::~~GgPoints () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6654 行目に定義があります。

7.7.3 関数詳解

7.7.3.1 draw()

```
void gg::GgPoints::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

点の描画.

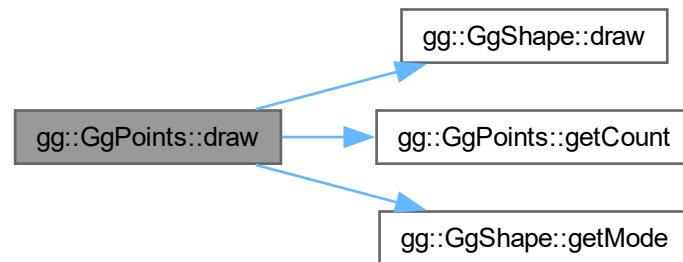
引数

<i>first</i>	描画を開始する最初の点の番号.
<i>count</i>	描画する点の数, 0 なら全部の点を描く.

[gg::GgShape](#) を再実装しています。

[gg.cpp](#) の 5144 行目に定義があります。

呼び出し関係図:



7.7.3.2 getBuffer()

```
const GLuint & gg::GgPoints::getBuffer () const [inline]
```

頂点の位置データを格納した頂点バッファオブジェクト名を取り出す.

戻り値

この図形の頂点の位置データを格納した頂点バッファオブジェクト名.

[gg.h](#) の 6693 行目に定義があります。

7.7.3.3 getCount()

```
const GLsizei & gg::GgPoints::getCount () const [inline]
```

データの数を取り出す.

戻り値

この図形の頂点の位置データの数 (頂点数).

[gg.h](#) の 6683 行目に定義があります。

被呼び出し関係図:



7.7.3.4 load()

```
void gg::GgPoints::load (
    const GgVector * pos,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点の位置データを格納する。

引数

<i>pos</i>	頂点の位置データが格納されている領域の先頭のポインタ。
<i>count</i>	頂点のデータの数 (頂点数)。
<i>usage</i>	バッファオブジェクトの使い方。

gg.cpp の 5131 行目に定義があります。

被呼び出し関係図:



7.7.3.5 operator bool()

```
gg::GgPoints::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

バッファが有効かどうか調べる。

戻り値

バッファが有効なら true

gg::GgShape を再実装しています。

gg.h の 6663 行目に定義があります。

7.7.3.6 operator"!()

```
bool gg::GgPoints::operator! () const [inline], [virtual], [noexcept]
```

バッファが有効かどうかの結果を反転する。

戻り値

バッファが有効なら false, 無効なら true.

gg::GgShape を再実装しています。

gg.h の 6673 行目に定義があります。

7.7.3.7 send()

```
void gg::GgPoints::send (
    const GgVector * pos,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点の位置データを転送する。

引数

<i>pos</i>	転送元の頂点の位置データが格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

gg.h の 6705 行目に定義があります。

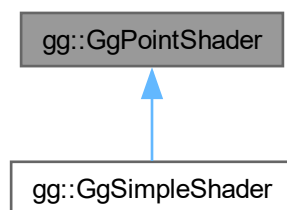
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

7.8 gg::GgPointShader クラス

```
#include <gg.h>
```

gg::GgPointShader の継承関係図



公開メンバ関数

- `GgPointShader` ()
- `GgPointShader` (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- `GgPointShader` (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual `~GgPointShader` ()
- bool `load` (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool `load` (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void `loadProjectionMatrix` (const GLfloat *mp) const
- virtual void `loadProjectionMatrix` (const `GgMatrix` &mp) const
- virtual void `loadModelviewMatrix` (const GLfloat *mv) const
- virtual void `loadModelviewMatrix` (const `GgMatrix` &mv) const
- virtual void `loadMatrix` (const GLfloat *mp, const GLfloat *mv) const
- virtual void `loadMatrix` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- virtual void `use` () const
- void `use` (const GLfloat *mp) const
- void `use` (const `GgMatrix` &mp) const
- void `use` (const GLfloat *mp, const GLfloat *mv) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- void `unuse` () const
- GLuint `get` () const

7.8.1 詳解

点のシェーダ.

`gg.h` の 7286 行目に定義があります。

7.8.2 構築子と解体子

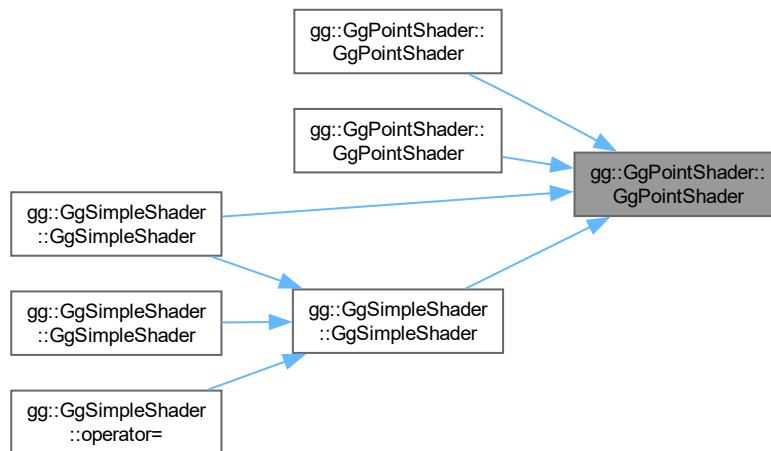
7.8.2.1 `GgPointShader()` [1/3]

```
gg::GgPointShader::GgPointShader () [inline]
```

コンストラクタ.

`gg.h` の 7302 行目に定義があります。

被呼び出し関係図:



7.8.2.2 GgPointShader() [2/3]

```

gg::GgPointShader::GgPointShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]

```

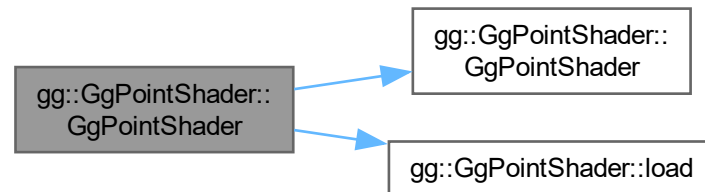
コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7317 行目に定義があります。

呼び出し関係図:



7.8.2.3 GgPointShader() [3/3]

```

gg::GgPointShader::GgPointShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

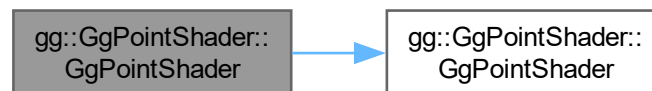
コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

[gg.h](#) の 7336 行目に定義があります。

呼び出し関係図:



7.8.2.4 ~GgPointShader()

```

virtual gg::GgPointShader::~GgPointShader () [inline], [virtual]
  
```

デストラクタ.

[gg.h](#) の 7348 行目に定義があります。

7.8.3 関数詳解

7.8.3.1 get()

```
GLuint gg::GgPointShader::get () const [inline]
```

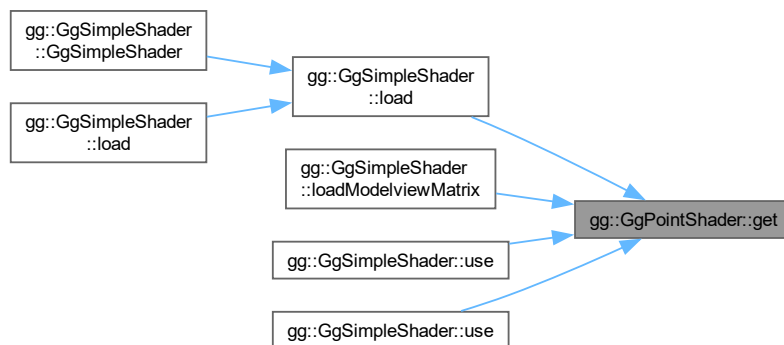
シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

gg.h の 7532 行目に定義があります。

被呼び出し関係図:



7.8.3.2 load() [1/2]

```
bool gg::GgPointShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら `true`.

`gg.h` の 7395 行目に定義があります。

呼び出し関係図:



7.8.3.3 load() [2/2]

```

bool gg::GgPointShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

シェーダのソースファイルを読み込む。

引数

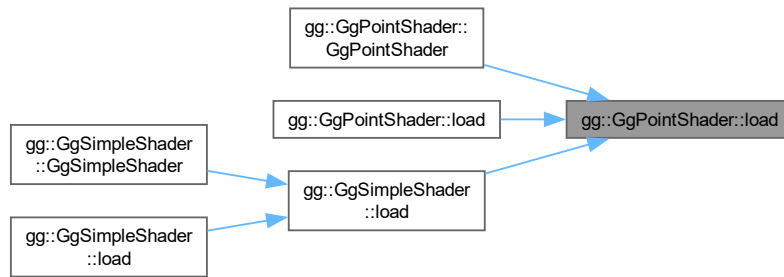
<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <code>varying</code> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <code>varying</code> 変数のリスト.

戻り値

プログラムオブジェクトが作成できれば `true`.

`gg.h` の 7362 行目に定義があります。

被呼び出し関係図:



7.8.3.4 loadMatrix() [1/2]

```
virtual void gg::GgPointShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する。

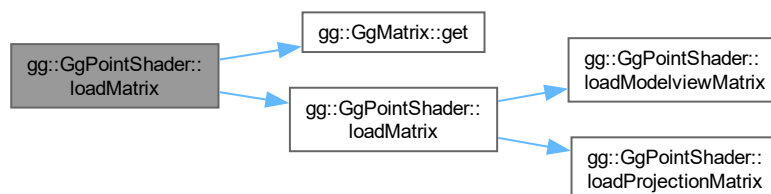
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg::GgSimpleShaderで再実装されています。

gg.h の 7462 行目に定義があります。

呼び出し関係図:



7.8.3.5 loadMatrix() [2/2]

```
virtual void gg::GgPointShader::loadMatrix (  
    const GLfloat * mp,  
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定する。

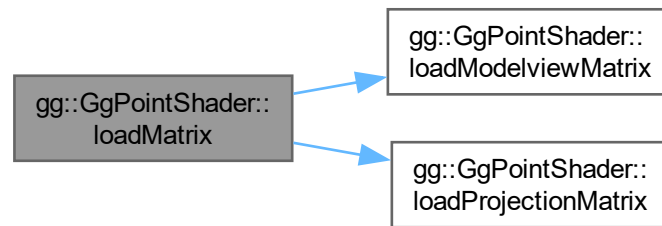
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

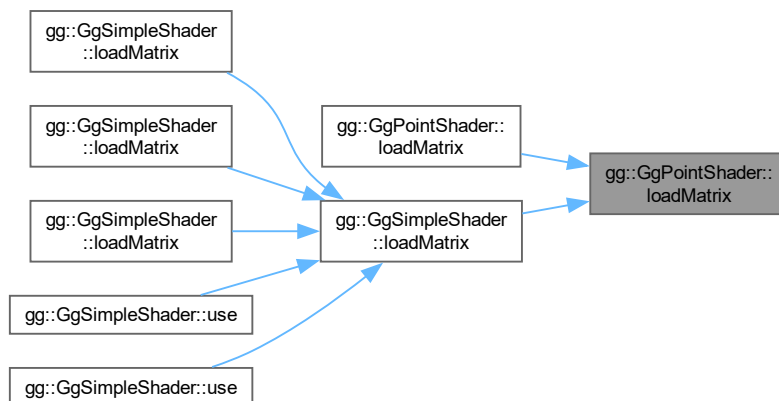
gg::GgSimpleShaderで再実装されています。

gg.h の 7450 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.8.3.6 loadModelviewMatrix() [1/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

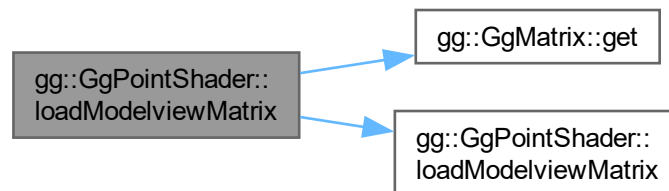
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	------------------------

gg::GgSimpleShaderで再実装されています。

gg.h の 7439 行目に定義があります。

呼び出し関係図:



7.8.3.7 loadModelviewMatrix() [2/2]

```
virtual void gg::GgPointShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列を設定する。

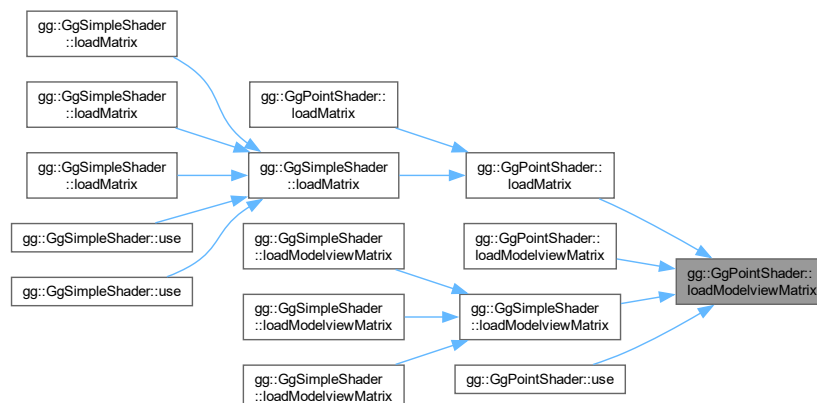
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgSimpleShaderで再実装されています。

gg.h の 7429 行目に定義があります。

被呼び出し関係図:



7.8.3.8 loadProjectionMatrix() [1/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (  
    const GgMatrix & mp) const [inline], [virtual]
```

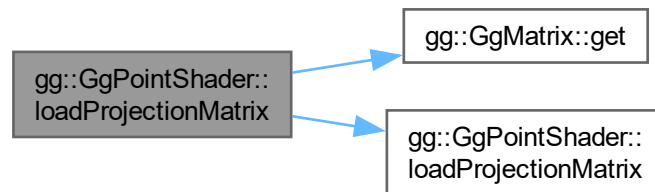
投影変換行列を設定する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
-----------	--------------------

gg.h の 7419 行目に定義があります。

呼び出し関係図:



7.8.3.9 loadProjectionMatrix() [2/2]

```
virtual void gg::GgPointShader::loadProjectionMatrix (  
    const GLfloat * mp) const [inline], [virtual]
```

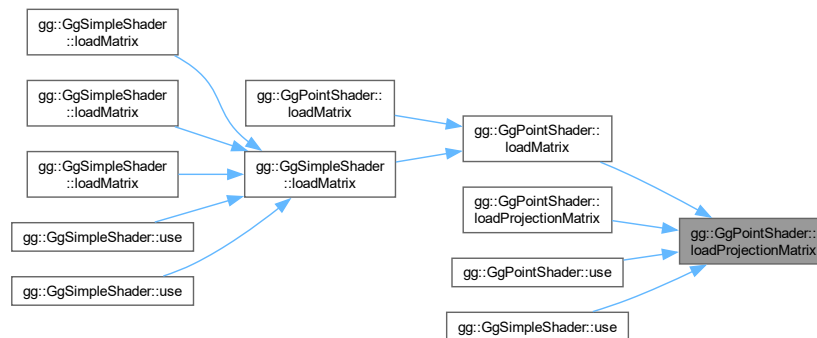
投影変換行列を設定する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

gg.h の 7409 行目に定義があります。

被呼び出し関係図:



7.8.3.10 `unuse()`

```
void gg::GgPointShader::unuse () const [inline]
```

シェーダプログラムの使用を終了する。

[gg.h](#) の 7522 行目に定義があります。

7.8.3.11 `use()` [1/5]

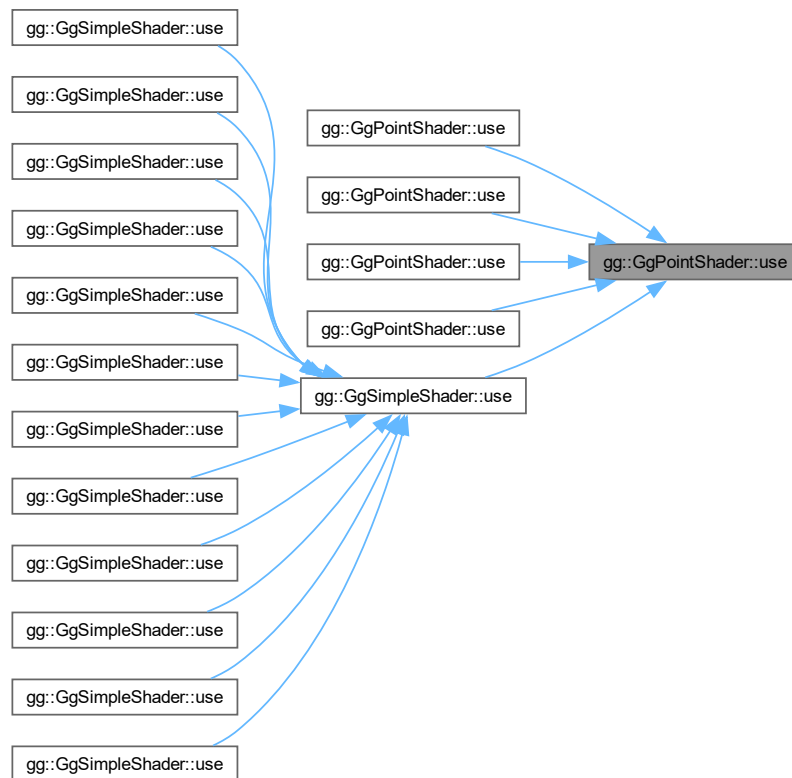
```
virtual void gg::GgPointShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する。

[gg::GgSimpleShader](#) で再実装されています。

[gg.h](#) の 7470 行目に定義があります。

被呼び出し関係図:



7.8.3.12 use() [2/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp) const [inline]
```

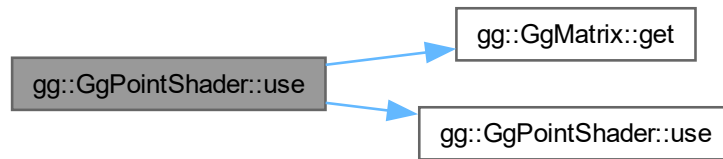
投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
-----------	--------------------

gg.h の 7491 行目に定義があります。

呼び出し関係図:



7.8.3.13 use() [3/5]

```
void gg::GgPointShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

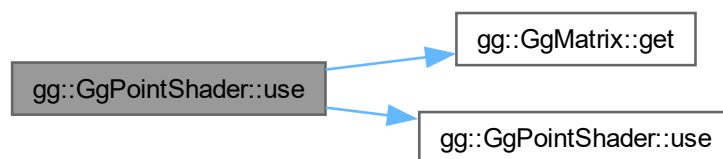
投影変換行列とモデルビューを設定してシェードプログラムの使用を開始する.

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

[gg.h](#) の 7514 行目に定義があります。

呼び出し関係図:



7.8.3.14 use() [4/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp) const [inline]
```

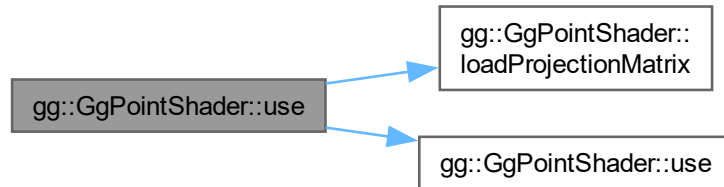
投影変換行列を設定してシェードプログラムの使用を開始する.

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
-----------	------------------------------------

gg.h の 7480 行目に定義があります。

呼び出し関係図:



7.8.3.15 use() [5/5]

```
void gg::GgPointShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

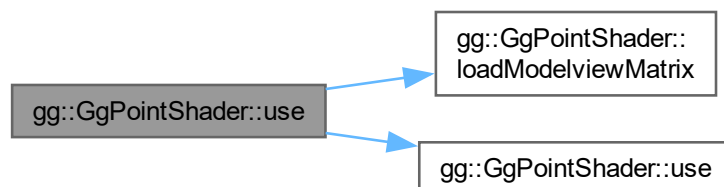
投影変換行列とモデルビューを設定してシェーダプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg.h の 7502 行目に定義があります。

呼び出し関係図:



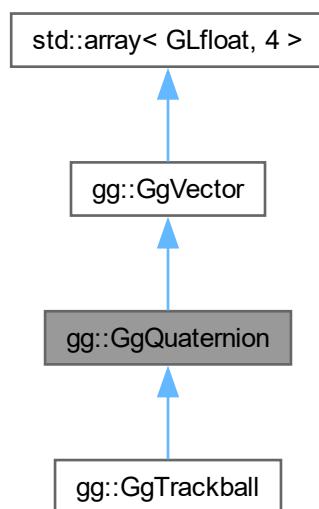
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

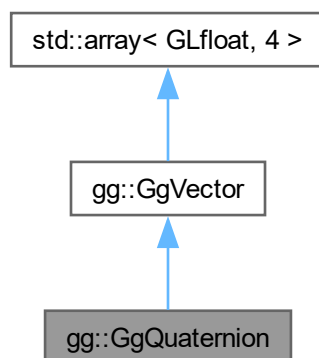
7.9 gg::GgQuaternion クラス

```
#include <gg.h>
```

gg::GgQuaternion の継承関係図



gg::GgQuaternion 連携図



公開メンバ関数

- `GgQuaternion()`=default

- constexpr [GgQuaternion](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- constexpr [GgQuaternion](#) (GLfloat c)
- [GgQuaternion](#) (const GLfloat *a)
- [GgQuaternion](#) (const [GgVector](#) &v)
- [GgQuaternion](#) (const [GgQuaternion](#) &q)=default
- [GgQuaternion](#) ([GgQuaternion](#) &&q)=default
- [~GgQuaternion](#) ()=default
- [GgQuaternion](#) & [operator=](#) (const [GgQuaternion](#) &q)=default
- [GgQuaternion](#) & [operator=](#) ([GgQuaternion](#) &&q)=default
- GLfloat [norm](#) () const
- [GgQuaternion](#) & [loadAdd](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadAdd](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadAdd](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadAdd](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadSubtract](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadSubtract](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadSubtract](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadMultiply](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadMultiply](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadMultiply](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [loadDivide](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat w)
- [GgQuaternion](#) & [loadDivide](#) (const GLfloat *a)
- [GgQuaternion](#) & [loadDivide](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [loadDivide](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) add (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) add (const GLfloat *a) const
- [GgQuaternion](#) add (const [GgVector](#) &v) const
- [GgQuaternion](#) add (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) subtract (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) subtract (const GLfloat *a) const
- [GgQuaternion](#) subtract (const [GgVector](#) &v) const
- [GgQuaternion](#) subtract (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) multiply (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) multiply (const GLfloat *a) const
- [GgQuaternion](#) multiply (const [GgVector](#) &v) const
- [GgQuaternion](#) multiply (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) divide (GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
- [GgQuaternion](#) divide (const GLfloat *a) const
- [GgQuaternion](#) divide (const [GgVector](#) &v) const
- [GgQuaternion](#) divide (const [GgQuaternion](#) &q) const
- [GgQuaternion](#) & [operator=](#) (const GLfloat *a)
- [GgQuaternion](#) & [operator=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator+=](#) (const GLfloat *a)
- [GgQuaternion](#) & [operator+=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator+=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [operator-=](#) (const GLfloat *a)
- [GgQuaternion](#) & [operator-=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator-=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [operator*=](#) (const GLfloat *a)
- [GgQuaternion](#) & [operator*=](#) (const [GgVector](#) &v)
- [GgQuaternion](#) & [operator*=](#) (const [GgQuaternion](#) &q)
- [GgQuaternion](#) & [operator/=](#) (const GLfloat *a)
- [GgQuaternion](#) & [operator/=](#) (const [GgVector](#) &v)

- `GgQuaternion & operator/=(const GgQuaternion &q)`
- `GgQuaternion operator+(const GLfloat *a) const`
- `GgQuaternion operator+(const GgVector &v) const`
- `GgQuaternion operator+(const GgQuaternion &q) const`
- `GgQuaternion operator-(const GLfloat *a) const`
- `GgQuaternion operator-(const GgVector &v) const`
- `GgQuaternion operator-(const GgQuaternion &q) const`
- `GgQuaternion operator*(const GLfloat *a) const`
- `GgQuaternion operator*(const GgVector &v) const`
- `GgQuaternion operator*(const GgQuaternion &q) const`
- `GgQuaternion operator/(const GLfloat *a) const`
- `GgQuaternion operator/(const GgVector &v) const`
- `GgQuaternion operator/(const GgQuaternion &q) const`
- `GgQuaternion & loadMatrix(const GLfloat *a)`
- `GgQuaternion & loadMatrix(const GgMatrix &m)`
- `GgQuaternion & loadIdentity()`
- `GgQuaternion & loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)`
- `GgQuaternion & loadRotate(const GLfloat *v, GLfloat a)`
- `GgQuaternion & loadRotate(const GLfloat *v)`
- `GgQuaternion & loadRotateX(GLfloat a)`
- `GgQuaternion & loadRotateY(GLfloat a)`
- `GgQuaternion & loadRotateZ(GLfloat a)`
- `GgQuaternion rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const`
- `GgQuaternion rotate(const GLfloat *v, GLfloat a) const`
- `GgQuaternion rotate(const GLfloat *v) const`
- `GgQuaternion rotateX(GLfloat a) const`
- `GgQuaternion rotateY(GLfloat a) const`
- `GgQuaternion rotateZ(GLfloat a) const`
- `GgQuaternion & loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll)`
- `GgQuaternion & loadEuler(const GLfloat *e)`
- `GgQuaternion euler(GLfloat heading, GLfloat pitch, GLfloat roll) const`
- `GgQuaternion euler(const GLfloat *e) const`
- `GgQuaternion euler(const GgVector &e) const`
- `GgQuaternion & loadSlerp(const GLfloat *a, const GLfloat *b, GLfloat t)`
- `GgQuaternion & loadSlerp(const GgQuaternion &q, const GgQuaternion &r, GLfloat t)`
- `GgQuaternion & loadSlerp(const GgQuaternion &q, const GLfloat *a, GLfloat t)`
- `GgQuaternion & loadSlerp(const GLfloat *a, const GgQuaternion &q, GLfloat t)`
- `GgQuaternion & loadNormalize(const GLfloat *a)`
- `GgQuaternion & loadNormalize(const GgQuaternion &q)`
- `GgQuaternion & loadConjugate(const GLfloat *a)`
- `GgQuaternion & loadConjugate(const GgQuaternion &q)`
- `GgQuaternion & loadInvert(const GLfloat *a)`
- `GgQuaternion & loadInvert(const GgQuaternion &q)`
- `GgQuaternion slerp(GLfloat *a, GLfloat t) const`
- `GgQuaternion slerp(const GgQuaternion &q, GLfloat t) const`
- `GgQuaternion normalize() const`
- `GgQuaternion conjugate() const`
- `GgQuaternion invert() const`
- `void get(GLfloat *a) const`
- `void getMatrix(GLfloat *a) const`
- `void getMatrix(GgMatrix &m) const`
- `GgMatrix getMatrix() const`
- `void getConjugateMatrix(GLfloat *a) const`
- `void getConjugateMatrix(GgMatrix &m) const`
- `GgMatrix getConjugateMatrix() const`

基底クラス **gg::GgVector** に属する継承公開メンバ関数

- **GgVector** ()=default
- constexpr **GgVector** (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr **GgVector** (const GLfloat *a)
- constexpr **GgVector** (GLfloat c)
- **GgVector** (const GgVector &v)=default
- **GgVector** (GgVector &&v)=default
- ~**GgVector** ()=default
- **GgVector** & operator= (const **GgVector** &v)=default
- **GgVector** & operator= (GgVector &&v)=default
- constexpr **GgVector** operator+ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator+ (GLfloat c) const noexcept
- constexpr **GgVector** & operator+= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator+= (GLfloat c) noexcept
- constexpr **GgVector** operator- (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator- (GLfloat c) const noexcept
- constexpr **GgVector** & operator-= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator-= (GLfloat c) noexcept
- constexpr **GgVector** operator* (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator* (GLfloat c) const noexcept
- constexpr **GgVector** & operator*= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator*= (GLfloat c) noexcept
- constexpr **GgVector** operator/ (const **GgVector** &v) const noexcept
- constexpr **GgVector** operator/ (GLfloat c) const noexcept
- constexpr **GgVector** & operator/= (const **GgVector** &v) noexcept
- constexpr **GgVector** & operator/= (GLfloat c) noexcept
- GLfloat dot3 (const **GgVector** &v) const
- GLfloat length3 () const
- GLfloat distance3 (const **GgVector** &v) const
- **GgVector** normalize3 () const
- GLfloat dot4 (const **GgVector** &v) const
- GLfloat length4 () const
- GLfloat distance4 (const **GgVector** &v) const
- **GgVector** normalize4 () const

7.9.1 詳解

四元数.

[gg.h](#) の 3483 行目に定義があります。

7.9.2 構築子と解体子

7.9.2.1 GgQuaternion() [1/7]

```
gg::GgQuaternion::GgQuaternion () [default]
```

コンストラクタ.

7.9.2.2 GgQuaternion() [2/7]

```
gg::GgQuaternion::GgQuaternion (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w) [inline], [constexpr]
```

コンストラクタ.

引数

<i>x</i>	四元数の <i>x</i> 要素.
<i>y</i>	四元数の <i>y</i> 要素.
<i>z</i>	四元数の <i>z</i> 要素.
<i>w</i>	四元数の <i>w</i> 要素.

gg.h の 3512 行目に定義があります。

呼び出し関係図:



7.9.2.3 GgQuaternion() [3/7]

```
gg::GgQuaternion::GgQuaternion (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

<i>c</i>	GLfloat 型の値.
----------	--------------

gg.h の 3522 行目に定義があります。

呼び出し関係図:



7.9.2.4 GgQuaternion() [4/7]

```
gg::GgQuaternion::GgQuaternion (
    const GLfloat * a) [inline]
```

コンストラクタ.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

[gg.h](#) の 3532 行目に定義があります。

呼び出し関係図:



7.9.2.5 GgQuaternion() [5/7]

```
gg::GgQuaternion::GgQuaternion (
    const GgVector & v) [inline]
```

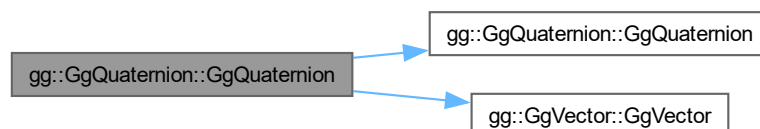
コンストラクタ.

引数

<i>v</i>	四元数を格納した GgVector 型の変数.
----------	---

[gg.h](#) の 3542 行目に定義があります。

呼び出し関係図:



7.9.2.6 GgQuaternion() [6/7]

```
gg::GgQuaternion::GgQuaternion (
    const GgQuaternion & q) [default]
```

コピーコンストラクタ.

引数

q	コピー元の四元数.
-----	-----------

呼び出し関係図:



7.9.2.7 GgQuaternion() [7/7]

```
gg::GgQuaternion::GgQuaternion (
    GgQuaternion && q) [default]
```

ムーブコンストラクタ.

引数

q	ムーブ元の四元数.
-----	-----------

呼び出し関係図:



7.9.2.8 ~GgQuaternion()

```
gg::GgQuaternion::~~GgQuaternion () [default]
```

デストラクタ.

7.9.3 関数詳解

7.9.3.1 add() [1/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgQuaternion & q) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

<i>q</i>	<code>GgQuaternion</code> 型の四元数.
----------	----------------------------------

戻り値

q を加えた四元数.

`gg.h` の 3835 行目に定義があります。

呼び出し関係図:



7.9.3.2 add() [2/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GgVector & v) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

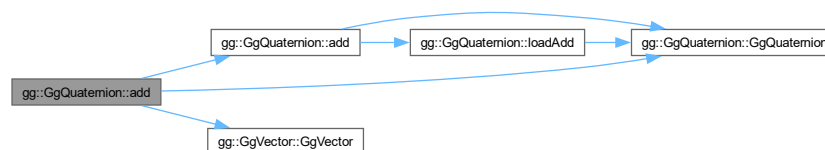
<i>v</i>	四元数を格納した <code>GgVector</code> 型の変数.
----------	--------------------------------------

戻り値

v を加えた四元数.

`gg.h` の 3824 行目に定義があります。

呼び出し関係図:



7.9.3.3 add() [3/4]

```
GgQuaternion gg::GgQuaternion::add (
    const GLfloat * a) const [inline]
```

四元数に別の四元数を加算した結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を加えた四元数.

gg.h の 3813 行目に定義があります。

呼び出し関係図:



7.9.3.4 add() [4/4]

```

GgQuaternion gg::GgQuaternion::add (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
  
```

四元数に別の四元数を加算した結果を返す.

引数

<i>x</i>	加える四元数の <i>x</i> 要素.
<i>y</i>	加える四元数の <i>y</i> 要素.
<i>z</i>	加える四元数の <i>z</i> 要素.
<i>w</i>	加える四元数の <i>w</i> 要素.

戻り値

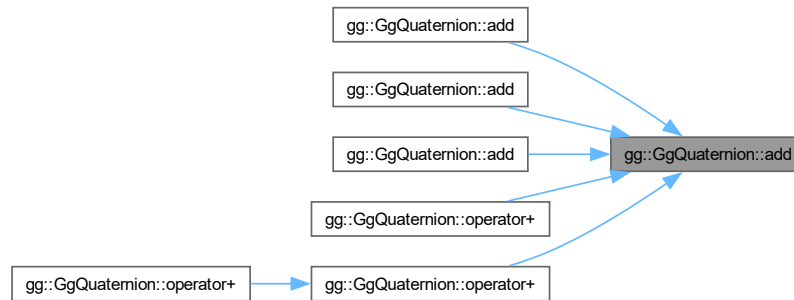
(*x*, *y*, *z*, *w*) を加えた四元数.

gg.h の 3801 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.5 conjugate()

```
GgQuaternion gg::GgQuaternion::conjugate () const [inline]
```

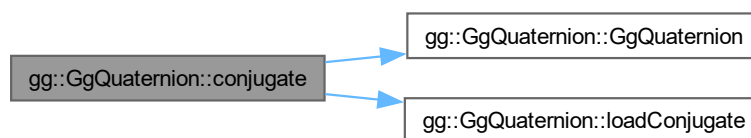
共役四元数に変換する.

戻り値

共役四元数.

[gg.h](#) の 4472 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.6 divide() [1/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgQuaternion & q) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

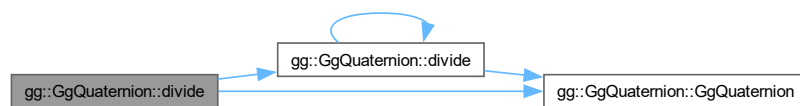
<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q で割った四元数.

gg.h の 3984 行目に定義があります。

呼び出し関係図:



7.9.3.7 divide() [2/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GgVector & v) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

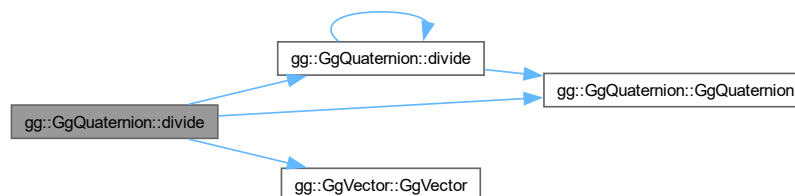
<i>v</i>	四元数を格納した GgVector 型の変数.
----------	-------------------------

戻り値

v で割った四元数.

[gg.h](#) の 3973 行目に定義があります。

呼び出し関係図:



7.9.3.8 divide() [3/4]

```
GgQuaternion gg::GgQuaternion::divide (
    const GLfloat * a) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

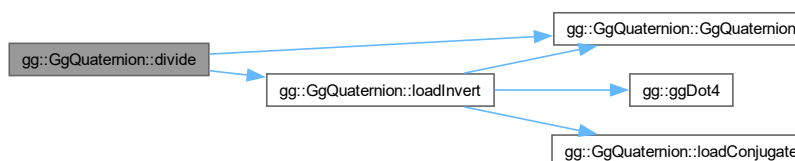
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a で割った四元数.

[gg.h](#) の 3959 行目に定義があります。

呼び出し関係図:



7.9.3.9 divide() [4/4]

```
GgQuaternion gg::GgQuaternion::divide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数を別の四元数で除算した結果を返す.

引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

戻り値

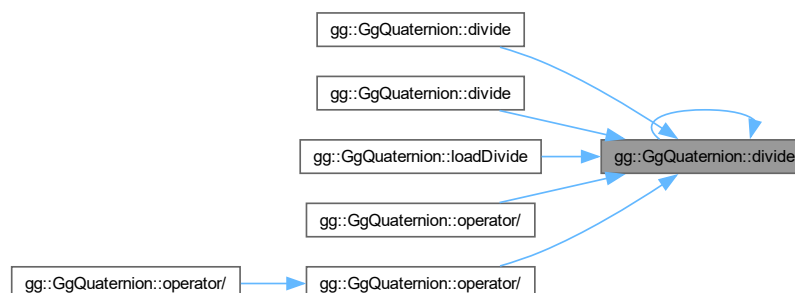
(x, y, z, w) を割った四元数.

gg.h の 3947 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.10 euler() [1/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GgVector & e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

e	オイラー角を表す GgVector 型の変数 (heading, pitch, roll) の参照.
---	---

戻り値

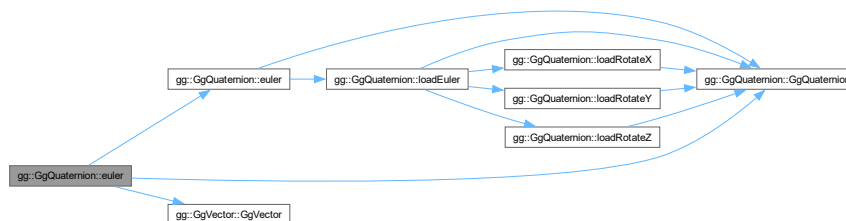
回転した四元数.

覚え書き

第 4 要素は無視する.

gg.h の 4312 行目に定義があります。

呼び出し関係図:



7.9.3.11 euler() [2/3]

```
GgQuaternion gg::GgQuaternion::euler (
    const GLfloat * e) const [inline]
```

四元数をオイラー角 (e[0], e[1], e[2]) で回転した四元数を返す.

引数

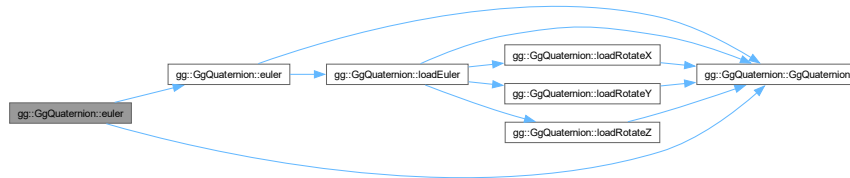
e	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
---	---

戻り値

回転した四元数.

gg.h の 4298 行目に定義があります。

呼び出し関係図:



7.9.3.12 euler() [3/3]

```

GgQuaternion gg::GgQuaternion::euler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll) const [inline]
  
```

四元数をオイラー角 (heading, pitch, roll) で回転した四元数を返す.

引数

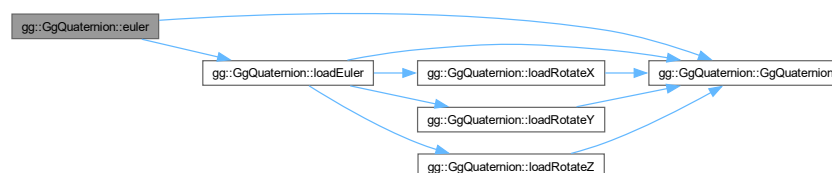
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

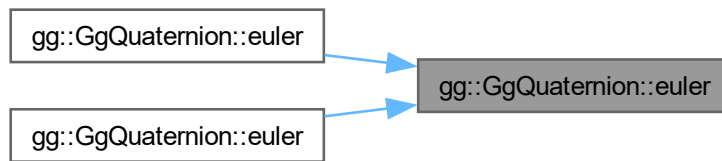
回転した四元数.

gg.h の 4286 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.13 get()

```
void gg::GgQuaternion::get (
    GLfloat * a) const [inline]
```

四元数を取り出す.

引数

<code>a</code>	四元数を格納する GLfloat 型の 4 要素の配列変数.
----------------	--------------------------------

[gg.h](#) の 4496 行目に定義があります。

7.9.3.14 getConjugateMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getConjugateMatrix () const [inline]
```

四元数の共役が表す回転の変換行列を取り出す.

戻り値

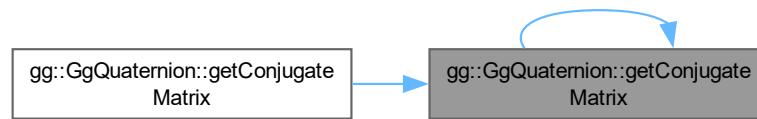
回転の変換を表す [GgMatrix](#) 型の変換行列.

[gg.h](#) の 4563 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.15 getConjugateMatrix() [2/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GgMatrix & m) const [inline]
```

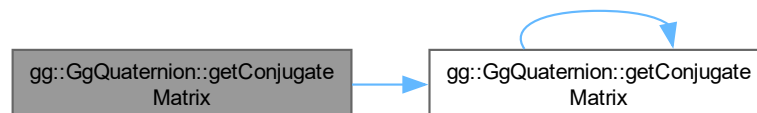
四元数の共役が表す回転の変換行列を m に求める。

引数

m	回転の変換行列を格納する <code>GgMatrix</code> 型の変数.
-----	--

`gg.h` の 4553 行目に定義があります。

呼び出し関係図:



7.9.3.16 getConjugateMatrix() [3/3]

```
void gg::GgQuaternion::getConjugateMatrix (
    GLfloat * a) const [inline]
```

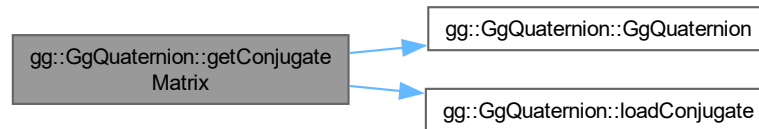
四元数の共役が表す回転の変換行列を a に求める。

引数

a	回転の変換行列を格納する <code>GLfloat</code> 型の 16 要素の配列変数.
-----	--

gg.h の 4541 行目に定義があります。

呼び出し関係図:



7.9.3.17 getMatrix() [1/3]

```
GgMatrix gg::GgQuaternion::getMatrix () const [inline]
```

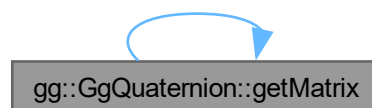
四元数が表す回転の変換行列を取り出す。

戻り値

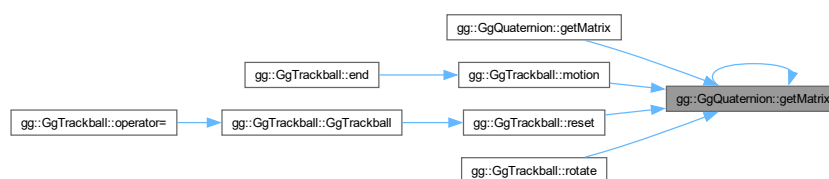
回転の変換を表す `GgMatrix` 型の変換行列。

gg.h の 4529 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.18 getMatrix() [2/3]

```
void gg::GgQuaternion::getMatrix (
    GgMatrix & m) const [inline]
```

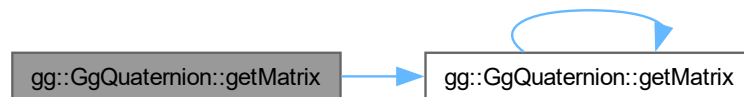
四元数が表す回転の変換行列を m に求める.

引数

<i>m</i>	回転の変換行列を格納する GgMatrix 型の変数.
----------	------------------------------------

[gg.h](#) の 4519 行目に定義があります。

呼び出し関係図:



7.9.3.19 getMatrix() [3/3]

```
void gg::GgQuaternion::getMatrix (
    GLfloat * a) const [inline]
```

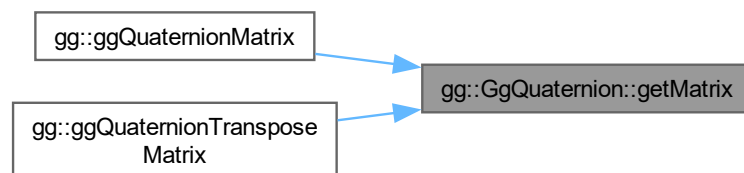
四元数が表す回転の変換行列を **a** に求める。

引数

<i>a</i>	回転の変換行列を格納する GLfloat 型の 16 要素の配列変数.
----------	-------------------------------------

[gg.h](#) の 4509 行目に定義があります。

被呼び出し関係図:



7.9.3.20 invert()

```
GgQuaternion gg::GgQuaternion::invert () const [inline]
```

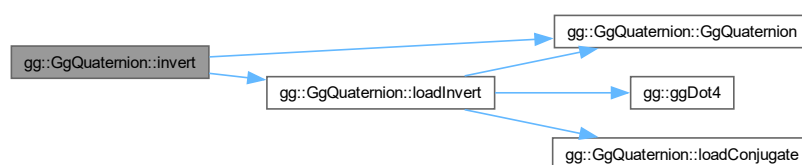
逆元に変換する.

戻り値

四元数の逆元.

gg.h の 4484 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.21 loadAdd() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	---------------------

戻り値

q を加えた四元数.

gg.h の 3639 行目に定義があります。

呼び出し関係図:



7.9.3.22 loadAdd() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GgVector & v) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

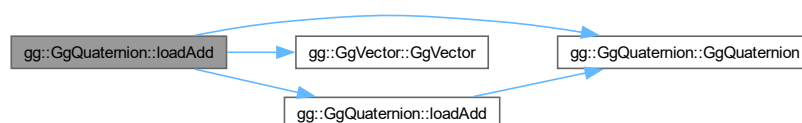
v	四元数を格納した GgVector 型の変数.
---	--------------------------------

戻り値

v を加えた四元数.

gg.h の 3628 行目に定義があります。

呼び出し関係図:



7.9.3.23 loadAdd() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadAdd (
    const GLfloat * a) [inline]
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を加えた四元数.

gg.h の 3617 行目に定義があります。

呼び出し関係図:



7.9.3.24 loadAdd() [4/4]

```

GgQuaternion & gg::GgQuaternion::loadAdd (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
  
```

四元数に別の四元数を加算した結果を格納する.

引数

<i>x</i>	加える四元数の <i>x</i> 要素.
<i>y</i>	加える四元数の <i>y</i> 要素.
<i>z</i>	加える四元数の <i>z</i> 要素.
<i>w</i>	加える四元数の <i>w</i> 要素.

戻り値

(*x*, *y*, *z*, *w*) を加えた四元数.

gg.h の 3601 行目に定義があります。

呼び出し関係図:



7.9.3.26 loadConjugate() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadConjugate (  
    const GLfloat * a)
```

引数に指定した四元数の共役四元数を格納する.

引数

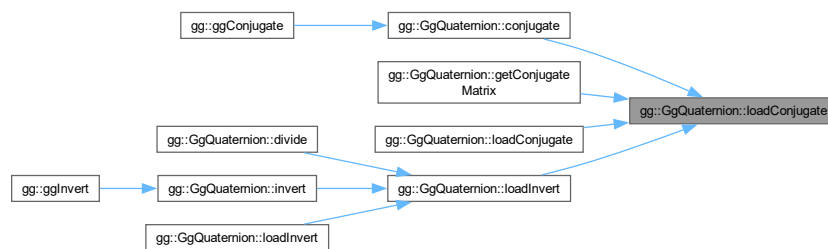
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

共役四元数.

[gg.cpp](#) の 3393 行目に定義があります。

被呼び出し関係図:



7.9.3.27 loadDivide() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GgQuaternion & q) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q で割った四元数.

[gg.h](#) の 3787 行目に定義があります。

呼び出し関係図:



7.9.3.28 loadDivide() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (  
    const GgVector & v) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

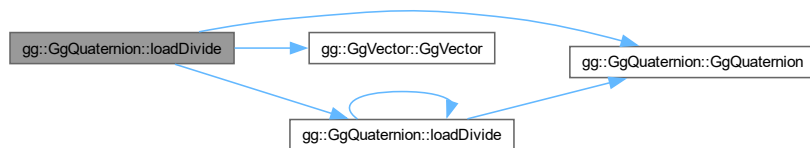
v	四元数を格納した GgVector 型の変数.
-----	--------------------------------

戻り値

v で割った四元数.

gg.h の 3776 行目に定義があります。

呼び出し関係図:



7.9.3.29 loadDivide() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    const GLfloat * a) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

a で割った四元数.

gg.h の 3765 行目に定義があります。

呼び出し関係図:



7.9.3.30 loadDivide() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadDivide (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元を別の四元数で除算した結果を格納する.

引数

x	割る四元数の x 要素.
y	割る四元数の y 要素.
z	割る四元数の z 要素.
w	割る四元数の w 要素.

戻り値

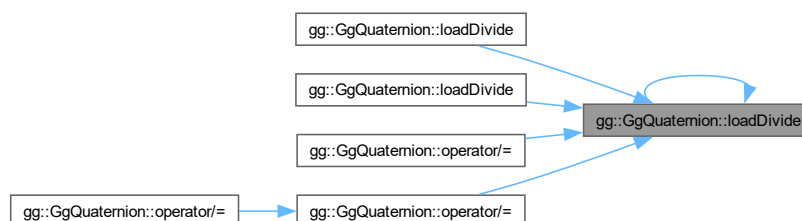
(x , y , z , w) を割った四元数.

gg.h の 3753 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.31 loadEuler() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadEuler (  
    const GLfloat * e) [inline]
```

オイラー角 (e[0], e[1], e[2]) で与えられた回転を表す四元数を格納する.

引数

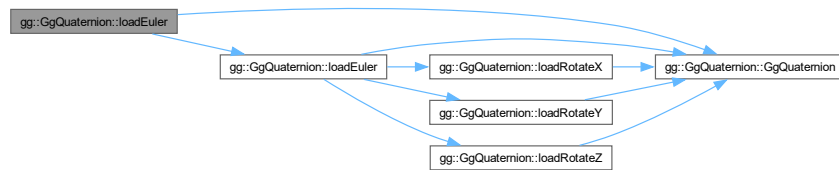
<i>e</i>	オイラー角を表す GLfloat 型の 3 要素の配列変数 (heading, pitch, roll).
----------	---

戻り値

格納した回転を表す四元数.

gg.h の 4273 行目に定義があります。

呼び出し関係図:



7.9.3.32 loadEuler() [2/2]

```

gg::GgQuaternion & gg::GgQuaternion::loadEuler (
    GLfloat heading,
    GLfloat pitch,
    GLfloat roll)

```

オイラー角 (heading, pitch, roll) で与えられた回転を表す四元数を格納する.

引数

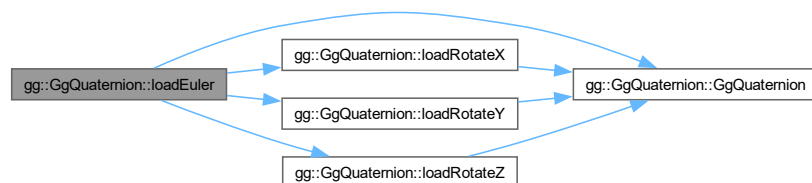
<i>heading</i>	y 軸中心の回転角.
<i>pitch</i>	x 軸中心の回転角.
<i>roll</i>	z 軸中心の回転角.

戻り値

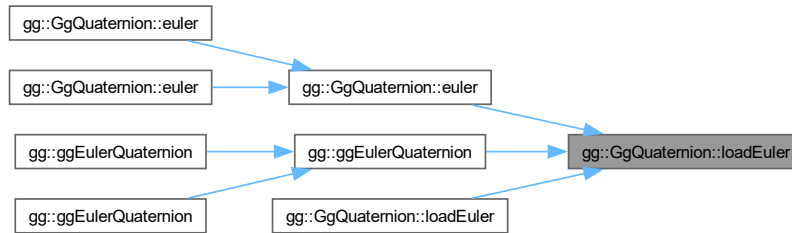
格納した回転を表す四元数.

gg.cpp の 3365 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.33 loadIdentity()

`GgQuaternion` & `gg::GgQuaternion::loadIdentity ()` [inline]

単位元を格納する.

戻り値

格納された単位元.

`gg.h` の 4123 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.34 loadInvert() [1/2]

`GgQuaternion` & `gg::GgQuaternion::loadInvert (`
`const GgQuaternion & q)` [inline]

引数に指定した四元数の逆元を格納する.

引数

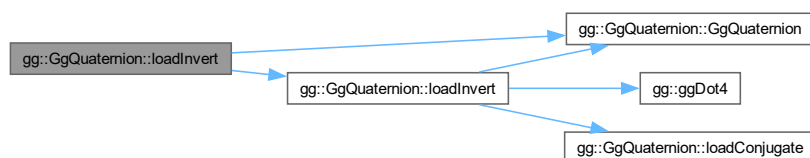
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

四元数の逆元.

gg.h の 4422 行目に定義があります。

呼び出し関係図:



7.9.3.35 loadInvert() [2/2]

```

gg::GgQuaternion & gg::GgQuaternion::loadInvert (
    const GLfloat * a)
  
```

引数に指定した四元数の逆元を格納する.

引数

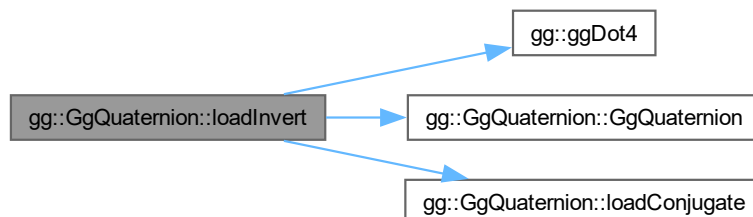
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

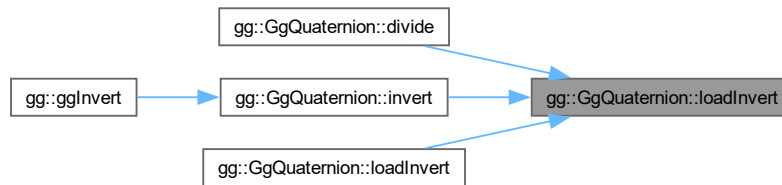
四元数の逆元.

gg.cpp の 3407 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.36 loadMatrix() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GgMatrix & m) [inline]
```

回転の変換行列 m を表す四元数を格納する.

引数

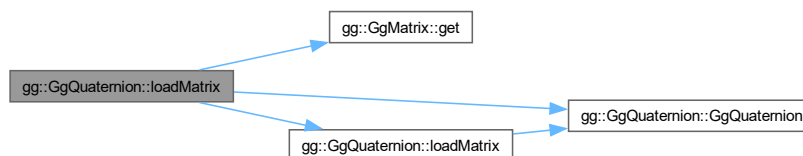
m	<code>GgMatrix</code> 型の変換行列.
-----	-------------------------------

戻り値

m による回転の変換に相当する四元数.

`gg.h` の 4113 行目に定義があります.

呼び出し関係図:



7.9.3.37 loadMatrix() [2/2]

```
GgQuaternion & gg::GgQuaternion::loadMatrix (
    const GLfloat * a) [inline]
```

回転の変換行列を表す四元数を格納する.

引数

a	GLfloat 型の 16 要素の変換行列.
---	------------------------

戻り値

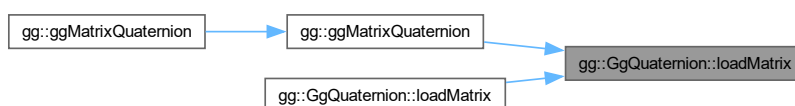
a による回転の変換に相当する四元数.

gg.h の 4101 行目に定義があります.

呼び出し関係図:



被呼び出し関係図:



7.9.3.38 loadMultiply() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgQuaternion & q) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

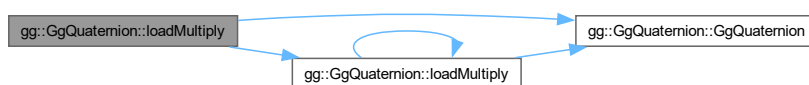
q	GgQuaternion 型の四元数.
---	---------------------

戻り値

q を乗じた四元数.

gg.h の 3739 行目に定義があります.

呼び出し関係図:



7.9.3.39 loadMultiply() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GgVector & v) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

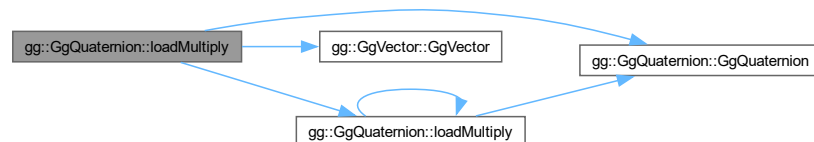
v	四元数を格納した GgVector 型の変数.
---	--------------------------------

戻り値

v を乗じた四元数.

gg.h の 3728 行目に定義があります。

呼び出し関係図:



7.9.3.40 loadMultiply() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    const GLfloat * a) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a を乗じた四元数.

gg.h の 3717 行目に定義があります。

呼び出し関係図:



7.9.3.41 loadMultiply() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadMultiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) [inline]
```

四元数に別の四元数を乗算した結果を格納する.

引数

x	掛ける四元数の x 要素.
y	掛ける四元数の y 要素.
z	掛ける四元数の z 要素.
w	掛ける四元数の w 要素.

戻り値

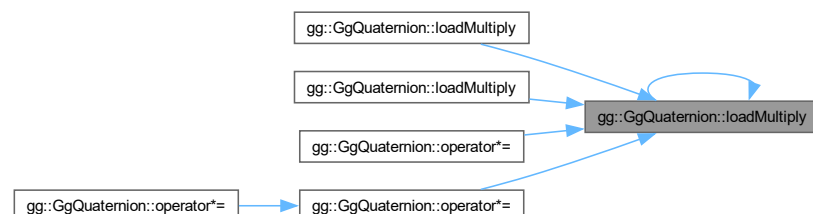
(x , y , z , w) を掛けた四元数.

gg.h の 3705 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.42 loadNormalize() [1/2]

```
GgQuaternion & gg::GgQuaternion::loadNormalize (  
    const GgQuaternion & q) [inline]
```

引数に指定した四元数を正規化して格納する.

引数

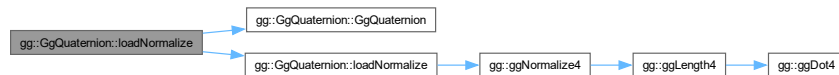
q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

正規化された四元数.

gg.h の 4384 行目に定義があります。

呼び出し関係図:



7.9.3.43 loadNormalize() [2/2]

```
gg::GgQuaternion & gg::GgQuaternion::loadNormalize (
    const GLfloat * a)
```

引数に指定した四元数を正規化して格納する.

引数

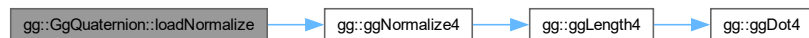
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

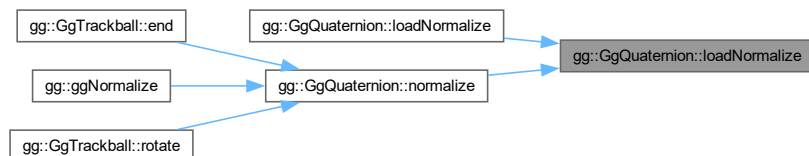
正規化された四元数.

gg.cpp の 3381 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.44 loadRotate() [1/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v) [inline]
```

(v[0], v[1], v[2]) を軸として角度 v[3] 回転する四元数を格納する.

引数

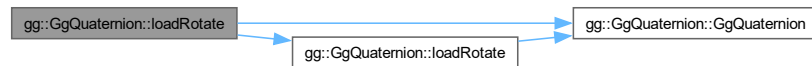
v	軸ベクトルと回転角を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------------

戻り値

格納された回転を表す四元数.

gg.h の 4157 行目に定義があります。

呼び出し関係図:



7.9.3.45 loadRotate() [2/3]

```
GgQuaternion & gg::GgQuaternion::loadRotate (
    const GLfloat * v,
    GLfloat a) [inline]
```

(v[0], v[1], v[2]) を軸として角度 a 回転する四元数を格納する.

引数

v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

格納された回転を表す四元数.

gg.h の 4146 行目に定義があります。

呼び出し関係図:



7.9.3.46 loadRotate() [3/3]

```
gg::GgQuaternion & gg::GgQuaternion::loadRotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a)
```

(x, y, z) を軸として角度 a 回転する四元数を格納する.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

戻り値

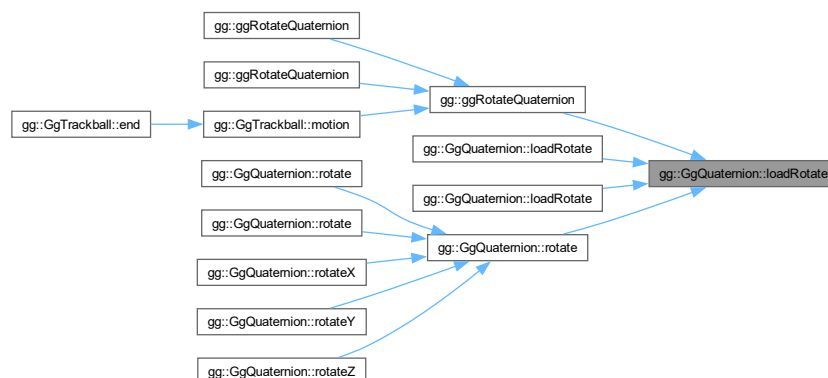
格納された回転を表す四元数.

gg.cpp の 3307 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.47 loadRotateX()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateX (
    GLfloat a)
```

x 軸中心に角度 **a** 回転する四元数を格納する.

引数

a	回転角.
----------	------

戻り値

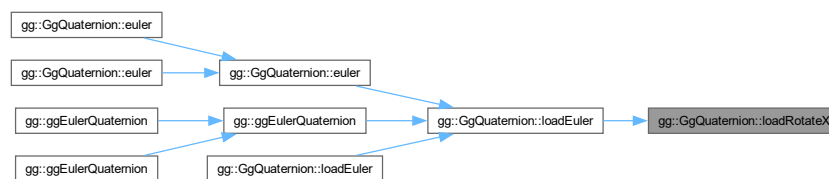
格納された回転を表す四元数.

[gg.cpp](#) の 3329 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.48 loadRotateY()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateY (
    GLfloat a)
```

y 軸中心に角度 **a** 回転する四元数を格納する.

引数

a	回轉角.
-----	------

戻り値

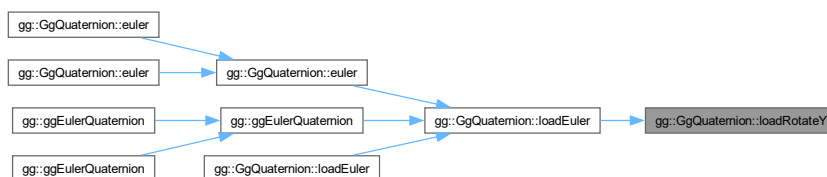
格納された回転を表す四元数.

gg.cpp の 3341 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.49 loadRotateZ()

```
gg::GgQuaternion & gg::GgQuaternion::loadRotateZ (
    GLfloat a)
```

z 軸中心に角度 a 回転する四元数を格納する.

引数

a	回轉角.
-----	------

戻り値

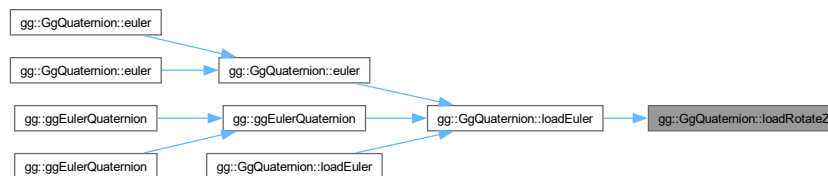
格納された回転を表す四元数.

`gg.cpp` の 3353 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.50 loadSlerp() [1/4]

```

GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GgQuaternion & r,
    GLfloat t) [inline]
  
```

球面線形補間の結果を格納する.

引数

<i>q</i>	<code>GgQuaternion</code> 型の四元数.
<i>r</i>	<code>GgQuaternion</code> 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した q, r を t で内分した四元数.

gg.h の 4339 行目に定義があります。

呼び出し関係図:



7.9.3.51 loadSlerp() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GgQuaternion & q,
    const GLfloat * a,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

q	GgQuaternion 型の四元数.
a	四元数を格納した GLfloat 型の 4 要素の配列変数.
t	補間パラメータ.

戻り値

格納した q, a を t で内分した四元数.

gg.h の 4352 行目に定義があります。

呼び出し関係図:



7.9.3.52 loadSlerp() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GgQuaternion & q,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

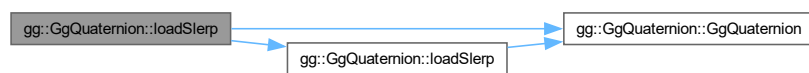
<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>q</i>	GgQuaternion 型の四元数.
<i>t</i>	補間パラメータ.

戻り値

格納した *a*, *q* を *t* で内分した四元数.

gg.h の 4365 行目に定義があります。

呼び出し関係図:



7.9.3.53 loadSlerp() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSlerp (
    const GLfloat * a,
    const GLfloat * b,
    GLfloat t) [inline]
```

球面線形補間の結果を格納する.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>b</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

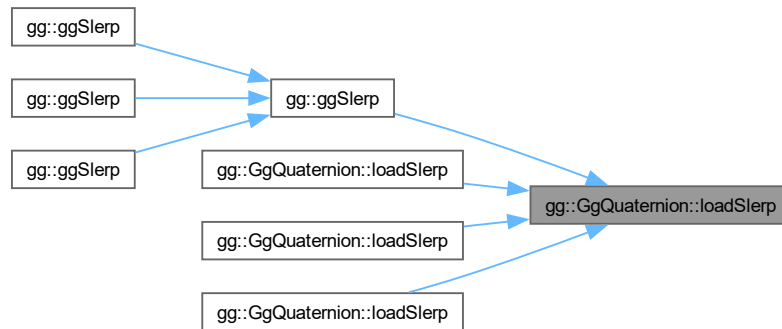
格納した *a*, *b* を *t* で内分した四元数.

gg.h の 4325 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.54 loadSubtract() [1/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgQuaternion & q) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

q	<code>GgQuaternion</code> 型の四元数.
-----	----------------------------------

戻り値

q を引いた四元数.

`gg.h` の 3691 行目に定義があります.

呼び出し関係図:



7.9.3.55 loadSubtract() [2/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GgVector & v) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

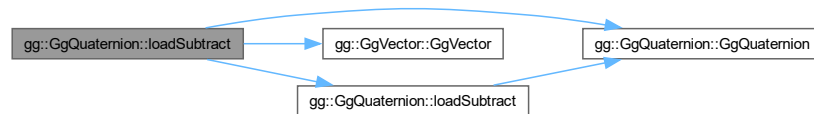
v	四元数を格納した GgVector 型の変数.
-----	--------------------------------

戻り値

v を引いた四元数.

gg.h の 3680 行目に定義があります。

呼び出し関係図:



7.9.3.56 loadSubtract() [3/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (
    const GLfloat * a) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
-----	---------------------------------------

戻り値

a を引いた四元数.

gg.h の 3669 行目に定義があります。

呼び出し関係図:



7.9.3.57 loadSubtract() [4/4]

```
GgQuaternion & gg::GgQuaternion::loadSubtract (  
    GLfloat x,  
    GLfloat y,  
    GLfloat z,  
    GLfloat w) [inline]
```

四元数から別の四元数を減算した結果を格納する.

引数

x	引く四元数の x 要素.
y	引く四元数の y 要素.
z	引く四元数の z 要素.
w	引く四元数の w 要素.

戻り値

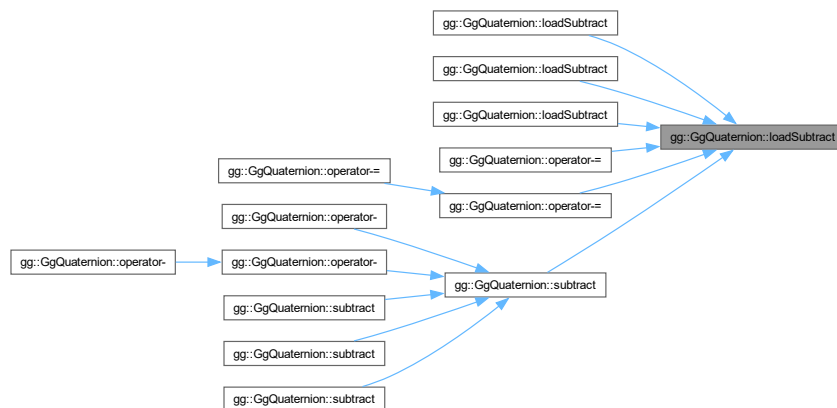
(x, y, z, w) を引いた四元数.

[gg.h](#) の 3653 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.58 multiply() [1/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GgQuaternion & q) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

q	GgQuaternion 型の四元数.
-----	---------------------

戻り値

q を掛けた四元数.

gg.h の 3933 行目に定義があります。

呼び出し関係図:



7.9.3.59 multiply() [2/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    const GgVector & v) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

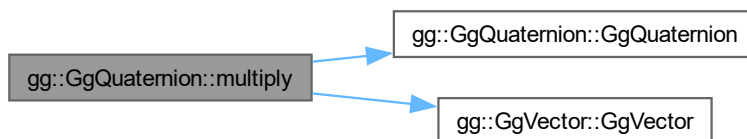
v	四元数を格納した GgVector 型の変数.
-----	-------------------------

戻り値

v を掛けた四元数.

gg.h の 3922 行目に定義があります。

呼び出し関係図:



7.9.3.60 multiply() [3/4]

```
GgQuaternion gg::GgQuaternion::multiply (  
    const GLfloat * a) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
----------	--------------------------------

戻り値

a を掛けた四元数.

gg.h の 3909 行目に定義があります。

呼び出し関係図:



7.9.3.61 multiply() [4/4]

```
GgQuaternion gg::GgQuaternion::multiply (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数に別の四元数を乗算した結果を返す.

引数

<i>x</i>	掛ける四元数の <i>x</i> 要素.
<i>y</i>	掛ける四元数の <i>y</i> 要素.
<i>z</i>	掛ける四元数の <i>z</i> 要素.
<i>w</i>	掛ける四元数の <i>w</i> 要素.

戻り値

(*x*, *y*, *z*, *w*) を掛けた四元数.

gg.h の 3897 行目に定義があります。

呼び出し関係図:



7.9.3.62 norm()

```
GLfloat gg::GgQuaternion::norm () const [inline]
```

四元数のノルムを求める。

戻り値

四元数のノルム。

[gg.h](#) の [3587](#) 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.63 normalize()

```
GgQuaternion gg::GgQuaternion::normalize () const [inline]
```

正規化する。

戻り値

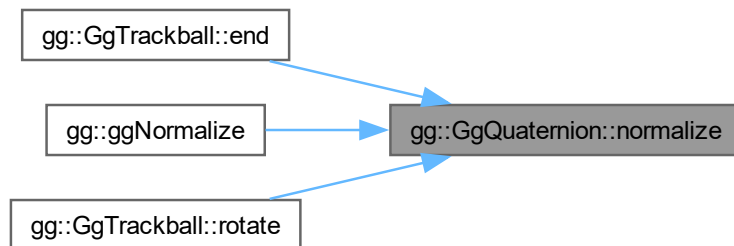
正規化された四元数.

gg.h の 4460 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.64 operator*() [1/3]

```
GgQuaternion gg::GgQuaternion::operator* (
    const GgQuaternion & q) const [inline]
```

gg.h の 4078 行目に定義があります。

呼び出し関係図:

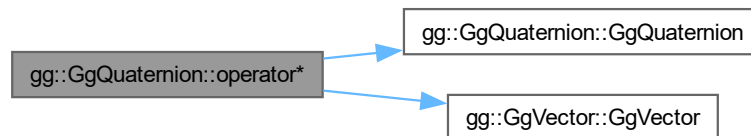


7.9.3.65 operator*() [2/3]

```
GgQuaternion gg::GgQuaternion::operator* (  
    const GgVector & v) const [inline]
```

gg.h の 4074 行目に定義があります。

呼び出し関係図:



7.9.3.66 operator*() [3/3]

```
GgQuaternion gg::GgQuaternion::operator* (  
    const GLfloat * a) const [inline]
```

gg.h の 4070 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.67 operator*=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgQuaternion & q) [inline]
```

gg.h の 4030 行目に定義があります。

呼び出し関係図:

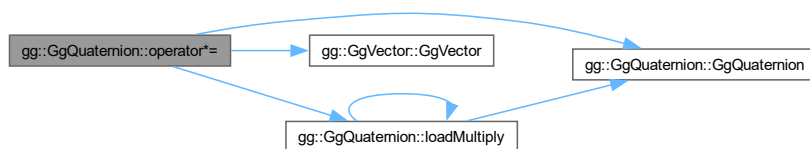


7.9.3.68 operator*=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GgVector & v) [inline]
```

gg.h の 4026 行目に定義があります。

呼び出し関係図:



7.9.3.69 operator*=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator*= (
    const GLfloat * a) [inline]
```

gg.h の 4022 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.70 operator+() [1/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgQuaternion & q) const [inline]
```

gg.h の 4054 行目に定義があります。

呼び出し関係図:

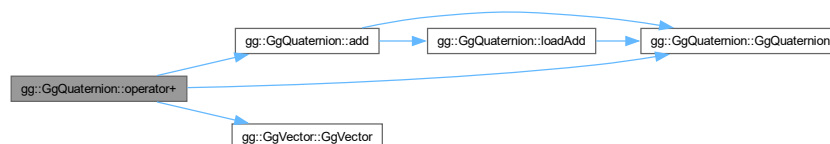


7.9.3.71 operator+() [2/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GgVector & v) const [inline]
```

gg.h の 4050 行目に定義があります。

呼び出し関係図:



7.9.3.72 operator+() [3/3]

```
GgQuaternion gg::GgQuaternion::operator+ (
    const GLfloat * a) const [inline]
```

gg.h の 4046 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

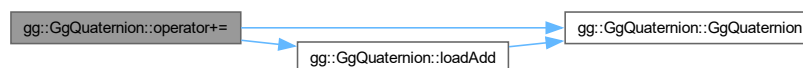


7.9.3.73 operator+=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgQuaternion & q) [inline]
```

gg.h の 4006 行目に定義があります。

呼び出し関係図:

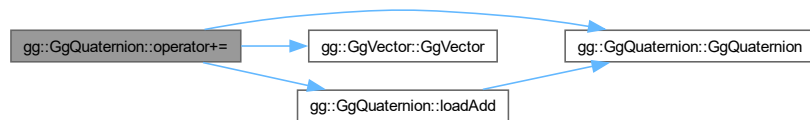


7.9.3.74 operator+=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GgVector & v) [inline]
```

gg.h の 4002 行目に定義があります。

呼び出し関係図:

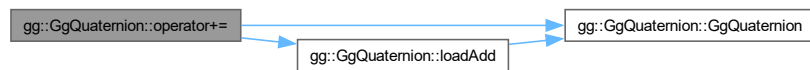


7.9.3.75 operator+=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator+= (
    const GLfloat * a) [inline]
```

gg.h の 3998 行目に定義があります。

呼び出し関係図:

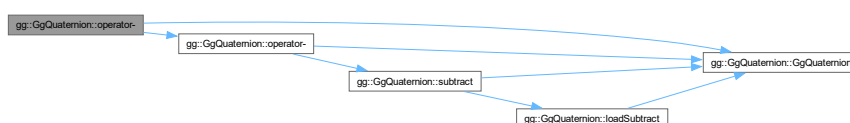


7.9.3.76 operator-() [1/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgQuaternion & q) const [inline]
```

gg.h の 4066 行目に定義があります。

呼び出し関係図:



7.9.3.77 operator-() [2/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GgVector & v) const [inline]
```

gg.h の 4062 行目に定義があります。

呼び出し関係図:



7.9.3.78 operator-() [3/3]

```
GgQuaternion gg::GgQuaternion::operator- (
    const GLfloat * a) const [inline]
```

gg.h の 4058 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.79 operator-=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgQuaternion & q) [inline]
```

gg.h の 4018 行目に定義があります。

呼び出し関係図:

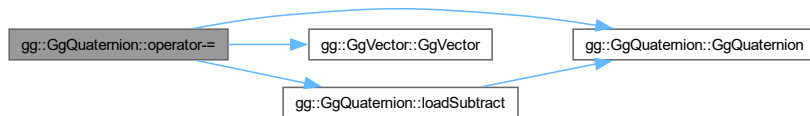


7.9.3.80 operator-=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GgVector & v) [inline]
```

gg.h の 4014 行目に定義があります。

呼び出し関係図:



7.9.3.81 operator-=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator-= (
    const GLfloat * a) [inline]
```

gg.h の 4010 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:

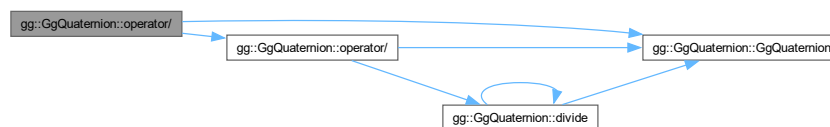


7.9.3.82 operator/() [1/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgQuaternion & q) const [inline]
```

`gg.h` の 4090 行目に定義があります。

呼び出し関係図:

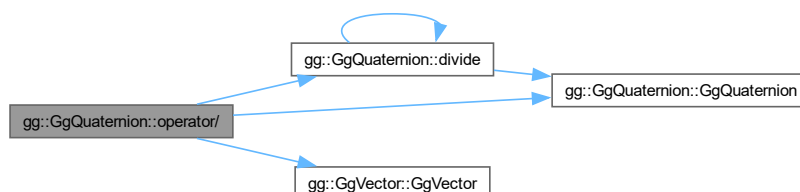


7.9.3.83 operator/() [2/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GgVector & v) const [inline]
```

`gg.h` の 4086 行目に定義があります。

呼び出し関係図:

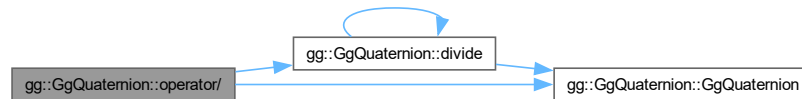


7.9.3.84 operator/() [3/3]

```
GgQuaternion gg::GgQuaternion::operator/ (
    const GLfloat * a) const [inline]
```

gg.h の 4082 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.85 operator/=() [1/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgQuaternion & q) [inline]
```

gg.h の 4042 行目に定義があります。

呼び出し関係図:

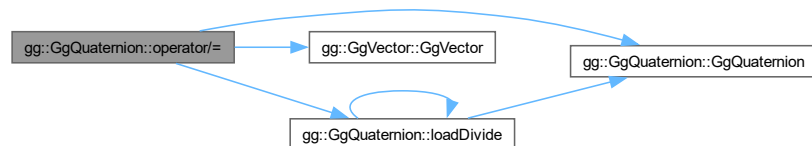


7.9.3.86 operator/=() [2/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GgVector & v) [inline]
```

gg.h の 4038 行目に定義があります。

呼び出し関係図:



7.9.3.87 operator/=() [3/3]

```
GgQuaternion & gg::GgQuaternion::operator/= (
    const GLfloat * a) [inline]
```

gg.h の 4034 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.88 operator=() [1/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GgQuaternion & q) [default]
```

代入演算子.

引数

q	代入元の四元数.
-----	----------

戻り値

代入後のこの四元数の参照.

呼び出し関係図:



被呼び出し関係図:

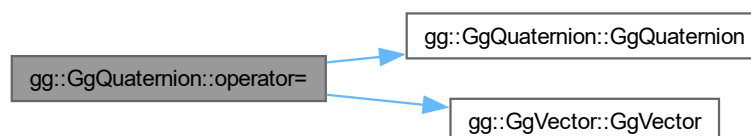


7.9.3.89 operator=() [2/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GgVector & v) [inline]
```

[gg.h](#) の 3994 行目に定義があります。

呼び出し関係図:



7.9.3.90 operator=() [3/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    const GLfloat * a) [inline]
```

gg.h の 3990 行目に定義があります。

呼び出し関係図:

**7.9.3.91 operator=()** [4/4]

```
GgQuaternion & gg::GgQuaternion::operator= (
    GgQuaternion && q) [default]
```

ムーブ代入演算子.

引数

<i>q</i>	ムーブ代入元の四元数.
----------	-------------

戻り値

ムーブ代入後のこの四元数の参照.

呼び出し関係図:

**7.9.3.92 rotate()** [1/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v) const [inline]
```

四元数を (v[0], v[1], v[2]) を軸として角度 v[3] 回転した四元数を返す.

引数

v	軸ベクトルを表す GLfloat 型の 4 要素の配列変数.
-----	--------------------------------

戻り値

回転した四元数.

gg.h の 4219 行目に定義があります.

呼び出し関係図:



7.9.3.93 rotate() [2/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    const GLfloat * v,
    GLfloat a) const [inline]
```

四元数を ($v[0]$, $v[1]$, $v[2]$) を軸として角度 a 回転した四元数を返す.

引数

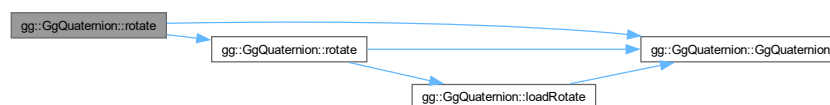
v	軸ベクトルを表す GLfloat 型の 3 要素の配列変数.
a	回転角.

戻り値

回転した四元数.

gg.h の 4208 行目に定義があります.

呼び出し関係図:



7.9.3.94 rotate() [3/3]

```
GgQuaternion gg::GgQuaternion::rotate (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat a) const [inline]
```

四元数を (x, y, z) を軸として角度 a 回転した四元数を返す.

引数

<i>x</i>	軸ベクトルの x 成分.
<i>y</i>	軸ベクトルの y 成分.
<i>z</i>	軸ベクトルの z 成分.
<i>a</i>	回転角.

戻り値

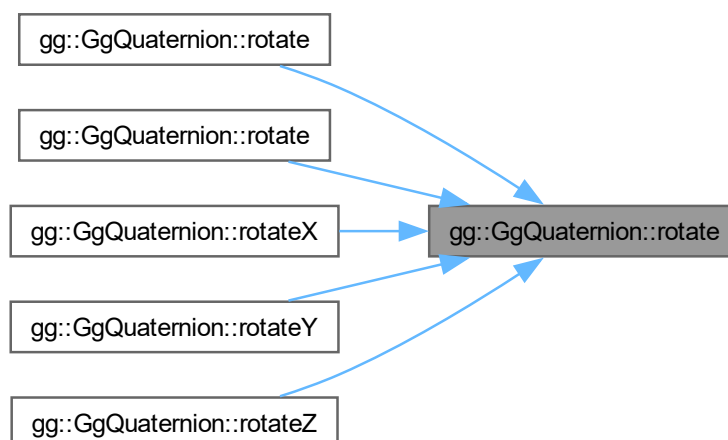
回転した四元数.

gg.h の 4195 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.9.3.95 rotateX()

```
GgQuaternion gg::GgQuaternion::rotateX (
    GLfloat a) const [inline]
```

四元数を x 軸中心に角度 **a** 回転した四元数を返す。

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4230 行目に定義があります。

呼び出し関係図:



7.9.3.96 rotateY()

```
GgQuaternion gg::GgQuaternion::rotateY (
    GLfloat a) const [inline]
```

四元数を y 軸中心に角度 **a** 回転した四元数を返す。

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4241 行目に定義があります。

呼び出し関係図:



7.9.3.97 rotateZ()

```
GgQuaternion gg::GgQuaternion::rotateZ (
    GLfloat a) const [inline]
```

四元数を z 軸中心に角度 **a** 回転した四元数を返す。

引数

a	回転角.
----------	------

戻り値

回転した四元数.

[gg.h](#) の 4252 行目に定義があります。

呼び出し関係図:



7.9.3.98 slerp() [1/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    const GgQuaternion & q,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す。

引数

q	GgQuaternion 型の四元数.
t	補間パラメータ.

戻り値

四元数を **q** に対して **t** で内分した結果.

[gg.h](#) の 4448 行目に定義があります。

呼び出し関係図:



7.9.3.99 slerp() [2/2]

```
GgQuaternion gg::GgQuaternion::slerp (
    GLfloat * a,
    GLfloat t) const [inline]
```

球面線形補間の結果を返す.

引数

<i>a</i>	四元数を格納した GLfloat 型の 4 要素の配列変数.
<i>t</i>	補間パラメータ.

戻り値

四元数を *a* に対して *t* で内分した結果.

[gg.h](#) の 4434 行目に定義があります。

呼び出し関係図:



7.9.3.100 subtract() [1/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgQuaternion & q) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>q</i>	GgQuaternion 型の四元数.
----------	-------------------------------------

戻り値

q を引いた四元数.

[gg.h](#) の 3883 行目に定義があります。

呼び出し関係図:



7.9.3.101 subtract() [2/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GgVector & v) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

v	四元数を格納した GgVector 型の変数.
---	--------------------------------

戻り値

v を引いた四元数.

gg.h の 3872 行目に定義があります.

呼び出し関係図:



7.9.3.102 subtract() [3/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    const GLfloat * a) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

a	四元数を格納した GLfloat 型の 4 要素の配列変数.
---	--------------------------------

戻り値

a を引いた四元数.

gg.h の 3861 行目に定義があります.

呼び出し関係図:



7.9.3.103 subtract() [4/4]

```
GgQuaternion gg::GgQuaternion::subtract (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w) const [inline]
```

四元数から別の四元数を減算した結果を返す.

引数

<i>x</i>	引く四元数の <i>x</i> 要素.
<i>y</i>	引く四元数の <i>y</i> 要素.
<i>z</i>	引く四元数の <i>z</i> 要素.
<i>w</i>	引く四元数の <i>w</i> 要素.

戻り値

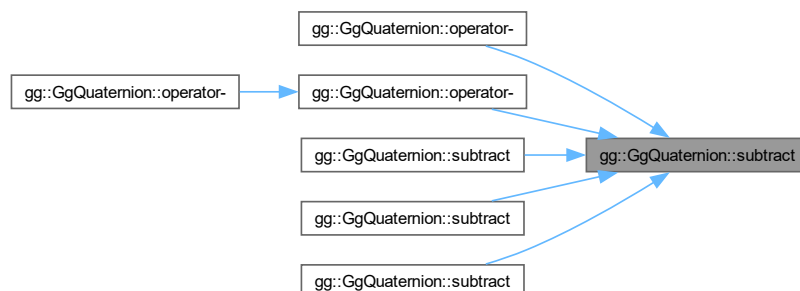
(*x*, *y*, *z*, *w*) を引いた四元数.

[gg.h](#) の 3849 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

7.10 gg::GgShader クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", int nvarying=0, const char *const *varyings=nullptr)
- [GgShader](#) (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- [GgShader](#) (const GgShader &shader)=delete
- [GgShader](#) (GgShader &&o) noexcept
- virtual ~[GgShader](#) ()
- [GgShader](#) & [operator=](#) (const [GgShader](#) &shader)=delete
- [GgShader](#) & [operator=](#) ([GgShader](#) &&o) noexcept
- void [use](#) () const
- void [unuse](#) () const
- GLuint [get](#) () const

7.10.1 詳解

シェーダの基底クラス.

覚え書き

シェーダのクラスはこのクラスを派生して作る.

[gg.h](#) の 7147 行目に定義があります。

7.10.2 構築子と解体子

7.10.2.1 GgShader() [1/4]

```
gg::GgShader::GgShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィールドバックする <i>varying</i> 変数の数 (0 なら不使用).

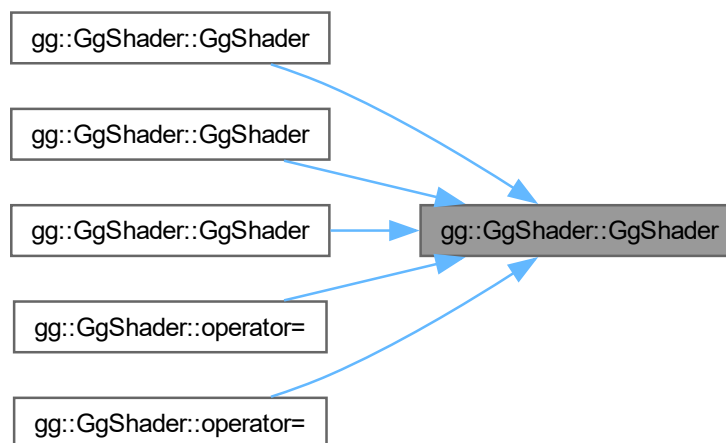
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.
-----------------	----------------------------------

[gg.h](#) の 7163 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.10.2.2 GgShader() [2/4]

```

gg::GgShader::GgShader (
    const std::array< std::string, 3 > & files,
    int nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

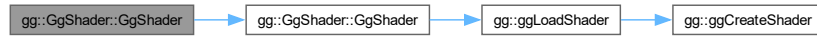
引数

<i>files</i>	パーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).

<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (<i>nullptr</i> なら不使用).
-----------------	--

gg.h の 7181 行目に定義があります。

呼び出し関係図:



7.10.2.3 GgShader() [3/4]

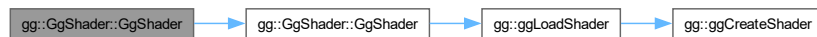
```
gg::GgShader::GgShader (
    const GgShader & shader) [delete]
```

コピーコンストラクタは使用しない。

引数

<i>shader</i>	コピー元のシェーダ.
---------------	------------

呼び出し関係図:



7.10.2.4 GgShader() [4/4]

```
gg::GgShader::GgShader (
    GgShader && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

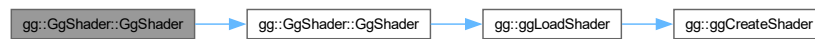
<i>o</i>	ムーブ元のシェーダ.
----------	------------

覚え書き

プログラム名の所有権を移動し, ムーブ元のプログラム名を 0 にする. ムーブ元のデストラクタが同じシェーダプログラムを削除しないようにするため.

gg.h の 7206 行目に定義があります。

呼び出し関係図:



7.10.2.5 ~GgShader()

```
virtual gg::GgShader::~GgShader () [inline], [virtual]
```

デストラクタ.

gg.h の 7215 行目に定義があります。

7.10.3 関数詳解

7.10.3.1 get()

```
GLuint gg::GgShader::get () const [inline]
```

シェーダのプログラム名を得る.

戻り値

シェーダのプログラム名.

gg.h の 7277 行目に定義があります。

7.10.3.2 operator=() [1/2]

```
GgShader & gg::GgShader::operator= (
    const GgShader & shader) [delete]
```

代入演算子は使用しない.

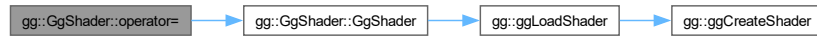
引数

<i>shader</i>	代入元のシェーダ.
---------------	-----------

戻り値

代入後のこのシェーダの参照.

呼び出し関係図:



7.10.3.3 operator=() [2/2]

```
GgShader & gg::GgShader::operator= (
    GgShader && o) [inline], [noexcept]
```

ムーブ代入演算子.

引数

<i>o</i>	ムーブ代入元のシェーダ.
----------	--------------

戻り値

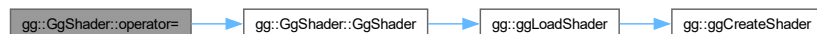
ムーブ代入後のこのシェーダの参照.

覚え書き

元々保持していたシェーダプログラムを削除してから所有権を移動し, ムーブ代入元のプログラム名を0にする.

[gg.h](#) の 7240 行目に定義があります。

呼び出し関係図:



7.10.3.4 unuse()

```
void gg::GgShader::unuse () const [inline]
```

シェーダプログラムの使用を終了する.

[gg.h](#) の 7267 行目に定義があります。

7.10.3.5 use()

```
void gg::GgShader::use () const [inline]
```

シェーダプログラムの使用を開始する。

[gg.h](#) の 7259 行目に定義があります。

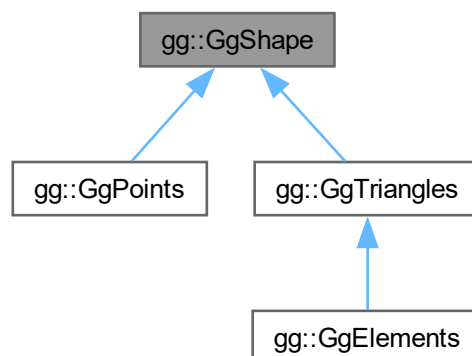
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

7.11 gg::GgShape クラス

```
#include <gg.h>
```

gg::GgShape の継承関係図



公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

7.11.1 詳解

形状データの基底クラス.

覚え書き

形状データのクラスはこのクラスを派生して作る. 基本図形の種類と頂点配列オブジェクトを保持する.

gg.h の 6523 行目に定義があります。

7.11.2 構築子と解体子

7.11.2.1 GgShape()

```
gg::GgShape::GgShape (
    GLenum mode = 0) [inline]
```

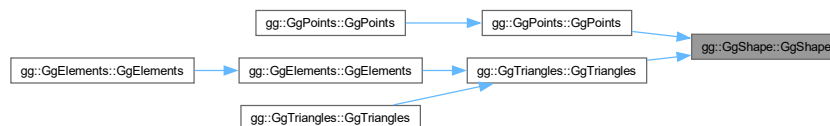
コンストラクタ.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

gg.h の 6538 行目に定義があります。

被呼び出し関係図:



7.11.2.2 ~GgShape()

```
virtual gg::GgShape::~~GgShape () [inline], [virtual]
```

デストラクタ.

gg.h の 6547 行目に定義があります。

7.11.3 関数詳解

7.11.3.1 draw()

```
virtual void gg::GgShape::draw (
    GLint first = 0,
    GLsizei count = 0) const [inline], [virtual]
```

図形の描画, 派生クラスでこの手続きをオーバーライドする.

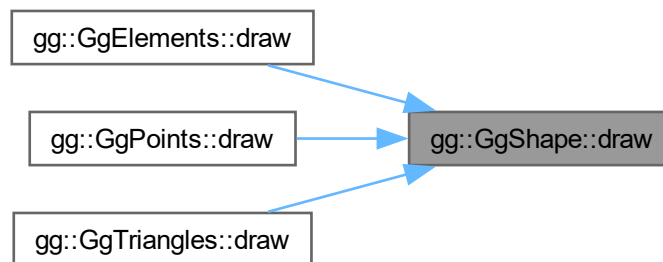
引数

<i>first</i>	描画する最初のアイテム.
<i>count</i>	描画するアイテムの数, 0 なら全部のアイテムを描画する.

[gg::GgElements](#), [gg::GgPoints](#), [gg::GgTriangles](#) で再実装されています。

[gg.h](#) の 6607 行目に定義があります。

被呼び出し関係図:



7.11.3.2 get()

```
const GLuint & gg::GgShape::get () const [inline]
```

頂点配列オブジェクト名を取り出す。

戻り値

頂点配列オブジェクト名。

[gg.h](#) の 6576 行目に定義があります。

被呼び出し関係図:



7.11.3.3 getMode()

```
const GEnum & gg::GgShape::getMode () const [inline]
```

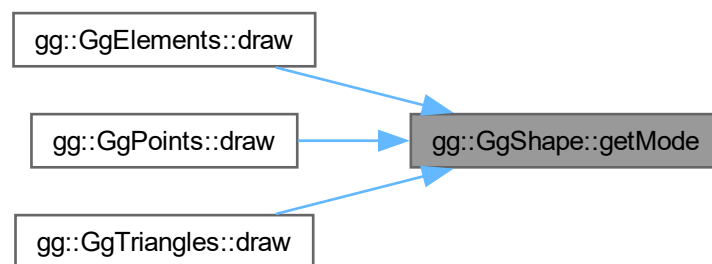
基本図形の検査.

戻り値

この頂点配列オブジェクトの基本図形の種類.

gg.h の 6596 行目に定義があります。

被呼び出し関係図:



7.11.3.4 operator bool()

```
virtual gg::GgShape::operator bool () const [inline], [explicit], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうか調べる.

戻り値

頂点配列オブジェクトが有効なら true

gg::GgPoints で再実装されています。

gg.h の 6556 行目に定義があります。

7.11.3.5 operator"!()

```
virtual bool gg::GgShape::operator! () const [inline], [virtual], [noexcept]
```

頂点配列オブジェクトが有効かどうかの結果を反転する.

戻り値

頂点配列オブジェクトが有効なら false, 無効なら true.

gg::GgPoints で再実装されています。

gg.h の 6566 行目に定義があります。

7.11.3.6 setMode()

```
void gg::GgShape::setMode (
    GLenum mode) [inline]
```

基本図形の設定.

引数

<i>mode</i>	基本図形の種類.
-------------	----------

[gg.h](#) の 6586 行目に定義があります。

このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

7.12 gg::GgSimpleObj クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgSimpleObj](#) (const std::string &name, bool normalize=false)
- virtual [~GgSimpleObj](#) ()
デストラクタ.
- [operator bool](#) () const noexcept
- bool [operator!](#) () const noexcept
- const [GgTriangles](#) * [get](#) () const
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

7.12.1 詳解

Wavefront OBJ 形式のファイル (Elements 形式).

[gg.h](#) の 8553 行目に定義があります。

7.12.2 構築子と解体子

7.12.2.1 GgSimpleObj()

```
gg::GgSimpleObj::GgSimpleObj (
    const std::string & name,
    bool normalize = false)
```

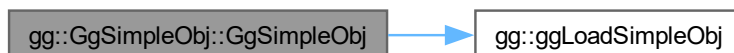
コンストラクタ.

引数

<i>name</i>	三角形分割された Alias OBJ 形式のファイルのファイル名.
<i>normalize</i>	true なら図形のサイズを [-1, 1] に正規化する.

gg.cpp の 6010 行目に定義があります。

呼び出し関係図:



7.12.2.2 ~GgSimpleObj()

```
virtual gg::GgSimpleObj::~~GgSimpleObj () [inline], [virtual]
```

デストラクタ.

gg.h の 8576 行目に定義があります。

7.12.3 関数詳解

7.12.3.1 draw()

```
void gg::GgSimpleObj::draw (  
    GLint first = 0,  
    GLsizei count = 0) const [virtual]
```

Wavefront OBJ 形式のデータを描画する手続き.

引数

<i>first</i>	描画する最初のパーツ番号.
<i>count</i>	描画するパーツの数, 0 なら全部のパーツを描く.

gg.cpp の 6038 行目に定義があります。

7.12.3.2 get()

```
const GgTriangles * gg::GgSimpleObj::get () const [inline]
```

形状データの取り出し。

戻り値

[GgTriangles](#) 型の形状データのポインタ。

[gg.h](#) の 8605 行目に定義があります。

呼び出し関係図:



7.12.3.3 operator bool()

```
gg::GgSimpleObj::operator bool () const [inline], [explicit], [noexcept]
```

オブジェクトが有効かどうか調べる。

戻り値

オブジェクトが有効なら true

[gg.h](#) の 8585 行目に定義があります。

7.12.3.4 operator"!()

```
bool gg::GgSimpleObj::operator! () const [inline], [noexcept]
```

オブジェクトが有効かどうかの結果を反転する。

戻り値

オブジェクトが有効なら false, 無効なら true.

[gg.h](#) の 8595 行目に定義があります。

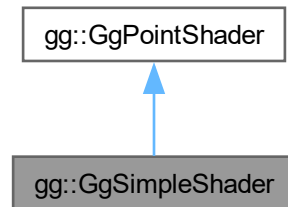
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

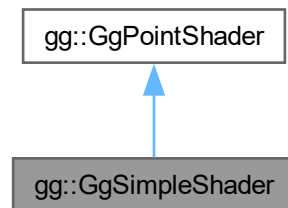
7.13 gg::GgSimpleShader クラス

```
#include <gg.h>
```

gg::GgSimpleShader の継承関係図



gg::GgSimpleShader 連携図



クラス

- struct [Light](#)
- class [LightBuffer](#)
- struct [Material](#)
- class [MaterialBuffer](#)

公開メンバ関数

- [GgSimpleShader](#) ()
- [GgSimpleShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- [GgSimpleShader](#) (const GgSimpleShader &o)

- virtual `~GgSimpleShader()`
- `GgSimpleShader & operator=` (const `GgSimpleShader &o`)
- bool `load` (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool `load` (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void `loadModelviewMatrix` (const GLfloat *mv, const GLfloat *mn) const
- virtual void `loadModelviewMatrix` (const `GgMatrix` &mv, const `GgMatrix` &mn) const
- virtual void `loadModelviewMatrix` (const GLfloat *mv) const
- virtual void `loadModelviewMatrix` (const `GgMatrix` &mv) const
- virtual void `loadMatrix` (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- virtual void `loadMatrix` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `GgMatrix` &mn) const
- virtual void `loadMatrix` (const GLfloat *mp, const GLfloat *mv) const
- virtual void `loadMatrix` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- void `use` () const
- void `use` (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `GgMatrix` &mn) const
- void `use` (const GLfloat *mp, const GLfloat *mv) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- void `use` (const `LightBuffer` *light, GLint i=0) const
- void `use` (const `LightBuffer` &light, GLint i=0) const
- void `use` (const GLfloat *mp, const GLfloat *mv, const GLfloat *mn, const `LightBuffer` *light, GLint i=0) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `GgMatrix` &mn, const `LightBuffer` &light, GLint i=0) const
- void `use` (const GLfloat *mp, const GLfloat *mv, const `LightBuffer` *light, GLint i=0) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv, const `LightBuffer` &light, GLint i=0) const
- void `use` (const GLfloat *mp, const `LightBuffer` *light, GLint i=0) const
- void `use` (const `GgMatrix` &mp, const `LightBuffer` &light, GLint i=0) const

基底クラス `gg::GgPointShader` に属する継承公開メンバ関数

- `GgPointShader()`
- `GgPointShader` (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- `GgPointShader` (const std::array< std::string, 3 > &files, int nvarying=0, const char *const *varyings=nullptr)
- virtual `~GgPointShader()`
- bool `load` (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- bool `load` (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- virtual void `loadProjectionMatrix` (const GLfloat *mp) const
- virtual void `loadProjectionMatrix` (const `GgMatrix` &mp) const
- void `use` (const GLfloat *mp) const
- void `use` (const `GgMatrix` &mp) const
- void `use` (const GLfloat *mp, const GLfloat *mv) const
- void `use` (const `GgMatrix` &mp, const `GgMatrix` &mv) const
- void `unuse` () const
- GLuint `get` () const

7.13.1 詳解

三角形に単純な陰影付けを行うシェーダ。

`gg.h` の 7541 行目に定義があります。

7.13.2 構築子と解体子

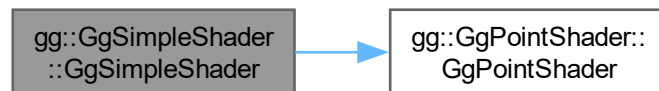
7.13.2.1 GgSimpleShader() [1/4]

```
gg::GgSimpleShader::GgSimpleShader () [inline]
```

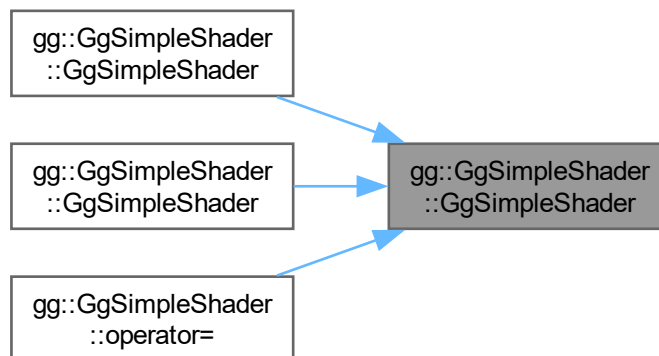
コンストラクタ.

[gg.h](#) の 7552 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.13.2.2 GgSimpleShader() [2/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

コンストラクタ.

引数

<i>vert</i>	バーテックスシェーダのソースファイル名.
<i>frag</i>	フラグメントシェーダのソースファイル名 (空文字列なら不使用).
<i>geom</i>	ジオメトリシェーダのソースファイル名 (空文字列なら不使用).
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト.

[gg.h](#) の 7567 行目に定義があります。

呼び出し関係図:



7.13.2.3 GgSimpleShader() [3/4]

```

gg::GgSimpleShader::GgSimpleShader (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
  
```

コンストラクタ.

引数

<i>files</i>	バーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

[gg.h](#) の 7585 行目に定義があります。

呼び出し関係図:



7.13.2.4 GgSimpleShader() [4/4]

```
gg::GgSimpleShader::GgSimpleShader (
    const GgSimpleShader & o) [inline]
```

コピーコンストラクタ.

gg.h の 7597 行目に定義があります。

呼び出し関係図:



7.13.2.5 ~GgSimpleShader()

```
virtual gg::GgSimpleShader::~~GgSimpleShader () [inline], [virtual]
```

デストラクタ.

gg.h の 7606 行目に定義があります。

7.13.3 関数詳解

7.13.3.1 load() [1/2]

```
bool gg::GgSimpleShader::load (
    const std::array< std::string, 3 > & files,
    GLint nvarying = 0,
    const char *const * varyings = nullptr) [inline]
```

シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する.

引数

<i>files</i>	パーテックスシェーダのソースファイル名の配列.
<i>nvarying</i>	フィードバックする varying 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする varying 変数のリスト (nullptr なら不使用).

戻り値

プログラムオブジェクトの作成に成功したら `true`.

[gg.h](#) の 7653 行目に定義があります。

呼び出し関係図:



7.13.3.2 load() [2/2]

```

bool gg::GgSimpleShader::load (
    const std::string & vert,
    const std::string & frag = "",
    const std::string & geom = "",
    GLint nvarying = 0,
    const char *const * varyings = nullptr)
  
```

シェーダのソースプログラムの文字列からプログラムオブジェクトを作成する。

引数

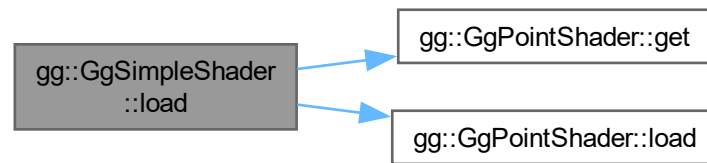
<i>vert</i>	バーテックスシェーダのソースプログラムの文字列.
<i>frag</i>	フラグメントシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>geom</i>	ジオメトリシェーダのソースプログラムの文字列 (空文字列なら不使用) .
<i>nvarying</i>	フィードバックする <i>varying</i> 変数の数 (0 なら不使用).
<i>varyings</i>	フィードバックする <i>varying</i> 変数のリスト (nullptr なら不使用).

戻り値

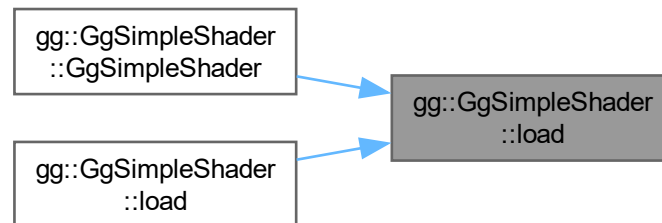
プログラムオブジェクトの作成に成功したら `true`.

[gg.cpp](#) の 5993 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.13.3.3 loadMatrix() [1/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline], [virtual]
  
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する。

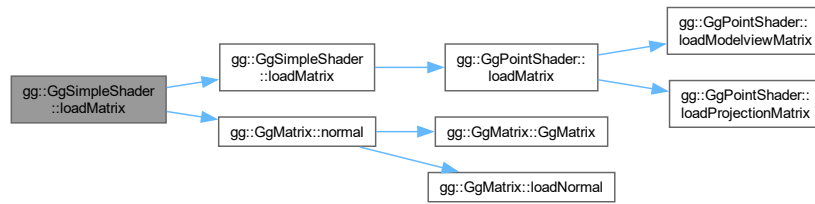
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

gg::GgPointShaderを再実装しています。

gg.h の 7747 行目に定義があります。

呼び出し関係図:



7.13.3.4 loadMatrix() [2/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]
```

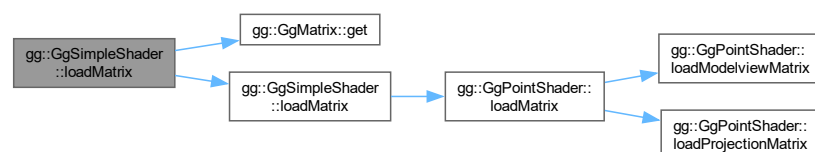
投影変換行列とモデルビュー変換行列と法線変換行列を設定する.

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7725 行目に定義があります.

呼び出し関係図:



7.13.3.5 loadMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv) const [inline], [virtual]
```

投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定する.

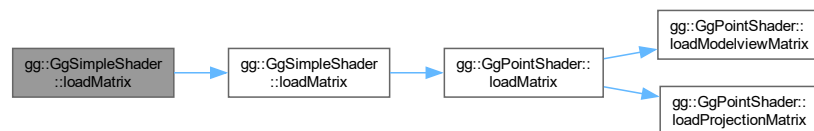
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg::GgPointShaderを再実装しています。

gg.h の 7736 行目に定義があります。

呼び出し関係図:



7.13.3.6 loadMatrix() [4/4]

```

virtual void gg::GgSimpleShader::loadMatrix (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]

```

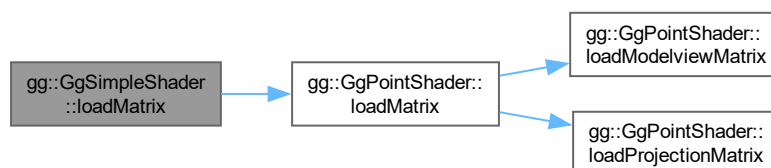
投影変換行列とモデルビュー変換行列と法線変換行列を設定する。

引数

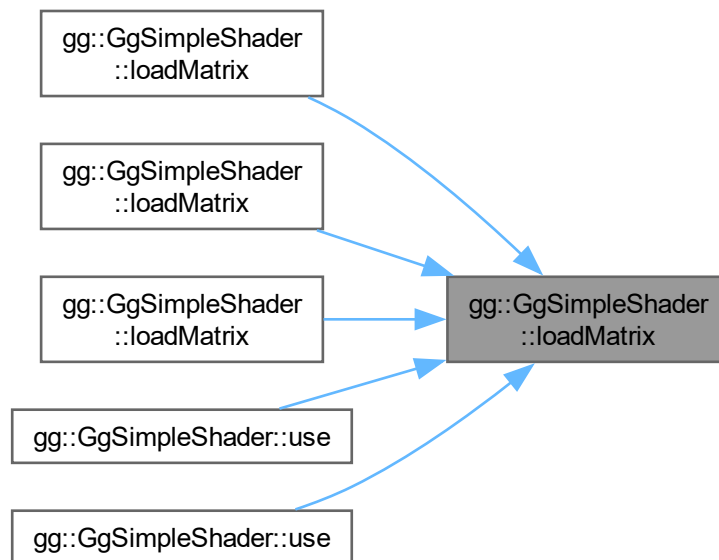
<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 7712 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.13.3.7 loadModelviewMatrix() [1/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

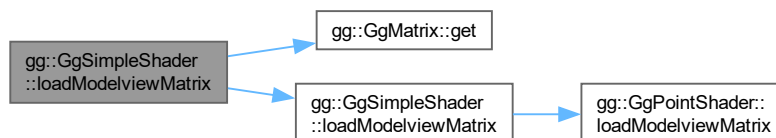
引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
-----------	--

[gg::GgPointShader](#)を再実装しています。

[gg.h](#) の 7700 行目に定義があります。

呼び出し関係図:



7.13.3.8 loadModelviewMatrix() [2/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline], [virtual]
```

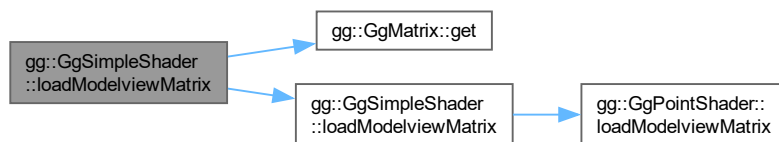
モデルビュー変換行列と法線変換行列を設定する。

引数

<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

gg.h の 7680 行目に定義があります。

呼び出し関係図:



7.13.3.9 loadModelviewMatrix() [3/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GLfloat * mv) const [inline], [virtual]
```

モデルビュー変換行列とそれから求めた法線変換行列を設定する。

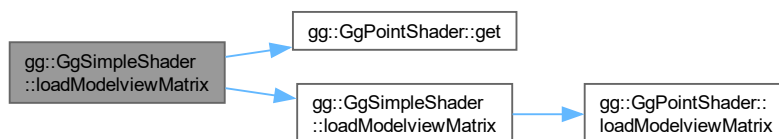
引数

<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
-----------	--

gg::GgPointShader を再実装しています。

gg.h の 7690 行目に定義があります。

呼び出し関係図:



7.13.3.10 loadModelviewMatrix() [4/4]

```
virtual void gg::GgSimpleShader::loadModelviewMatrix (
    const GLfloat * mv,
    const GLfloat * mn) const [inline], [virtual]
```

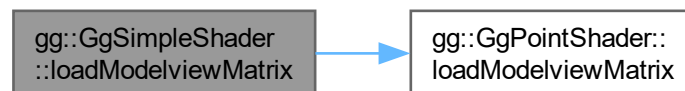
モデルビュー変換行列と法線変換行列を設定する。

引数

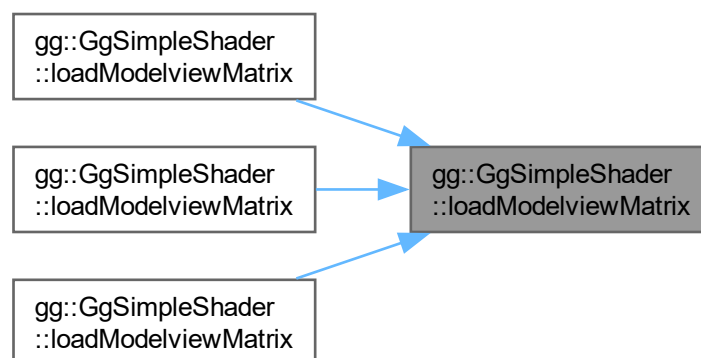
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 7668 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.13.3.11 operator=()

```
GgSimpleShader & gg::GgSimpleShader::operator= (  
    const GgSimpleShader & o) [inline]
```

代入演算子.

引数

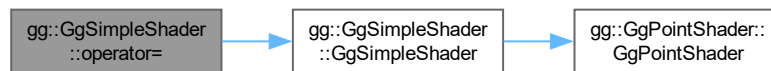
o 代入する `GgSimpleShader` 型のオブジェクト.

戻り値

代入後の `GgSimpleShader` 型のオブジェクトの参照.

`gg.h` の 7616 行目に定義があります。

呼び出し関係図:



7.13.3.12 use() [1/13]

```
void gg::GgSimpleShader::use () const [inline], [virtual]
```

シェーダプログラムの使用を開始する.

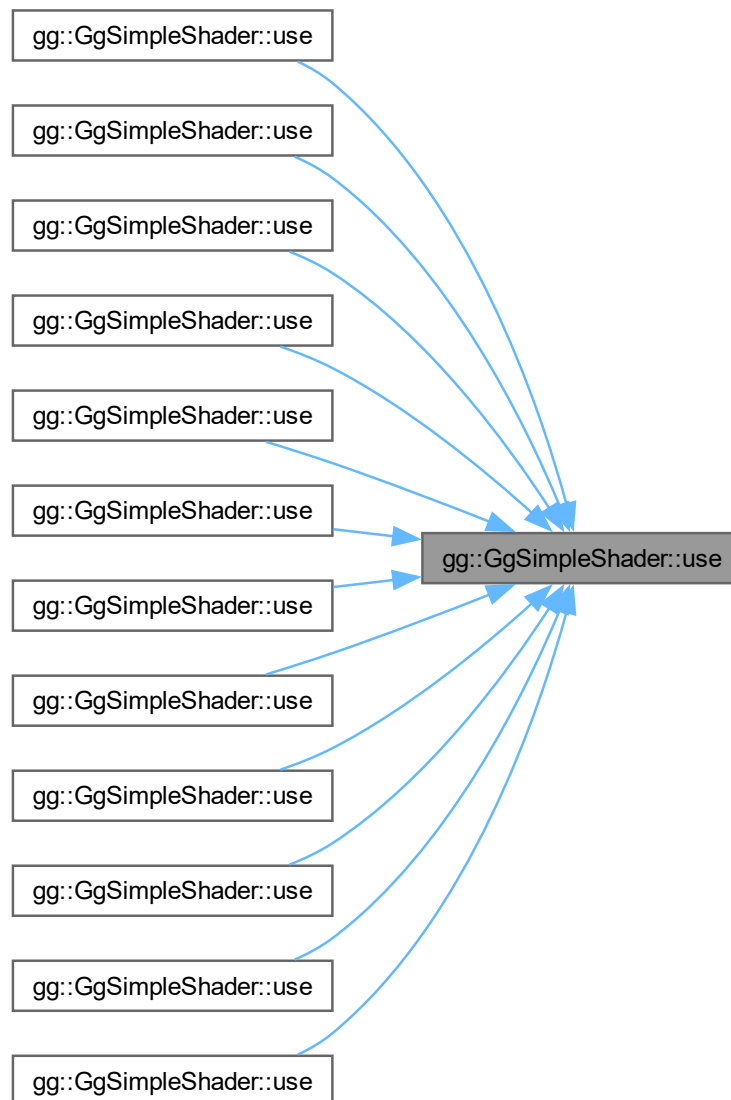
`gg::GgPointShader` を再実装しています。

`gg.h` の 8320 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.13.3.13 use() [2/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv) const [inline]
```

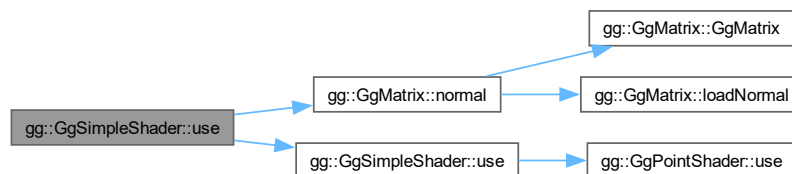
投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.

[gg.h](#) の 8371 行目に定義があります。

呼び出し関係図:



7.13.3.14 use() [3/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn) const [inline]
  
```

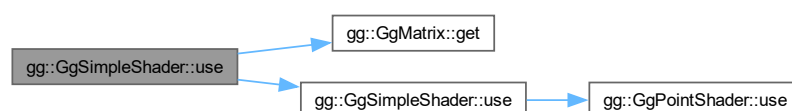
投影変換行列とモデルビュー変換行列と法線変換行列を指定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.

[gg.h](#) の 8349 行目に定義があります。

呼び出し関係図:



7.13.3.15 use() [4/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const GgMatrix & mn,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

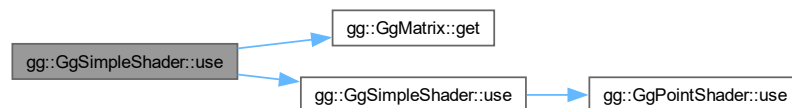
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>mn</i>	GgMatrix 型のモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8435 行目に定義があります。

呼び出し関係図:



7.13.3.16 use() [5/13]

```
void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const GgMatrix & mv,
    const LightBuffer & light,
    GLint i = 0) const [inline]
```

光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

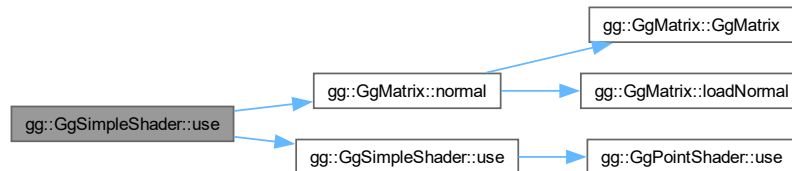
引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>mv</i>	GgMatrix 型のモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.

<i>i</i>	光源データの uniform block のインデックス.
----------	--------------------------------------

gg.h の 8472 行目に定義があります。

呼び出し関係図:



7.13.3.17 use() [6/13]

```

void gg::GgSimpleShader::use (
    const GgMatrix & mp,
    const LightBuffer & light,
    GLint i = 0) const [inline]
  
```

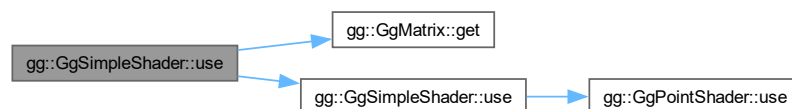
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GgMatrix 型の投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8505 行目に定義があります。

呼び出し関係図:



7.13.3.18 use() [7/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv) const [inline]
```

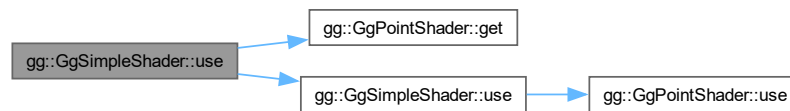
投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.

gg.h の 8360 行目に定義があります。

呼び出し関係図:



7.13.3.19 use() [8/13]

```

void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn) const [inline]
  
```

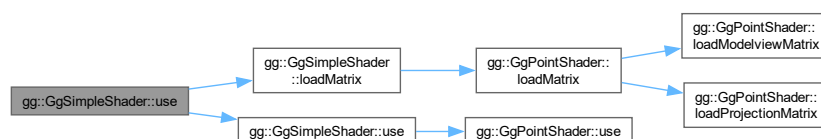
投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.

gg.h の 8333 行目に定義があります。

呼び出し関係図:



7.13.3.20 use() [9/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const GLfloat * mn,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

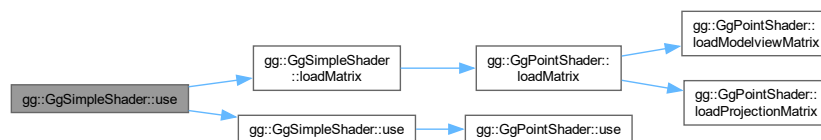
光源を指定し投影変換行列とモデルビュー変換行列と法線変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>mn</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列の法線変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8411 行目に定義があります。

呼び出し関係図:



7.13.3.21 use() [10/13]

```
void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const GLfloat * mv,
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

光源を指定し投影変換行列とモデルビュー変換行列を設定しモデルビュー変換行列から求めた法線変換行列を設定してシェードプログラムの使用を開始する。

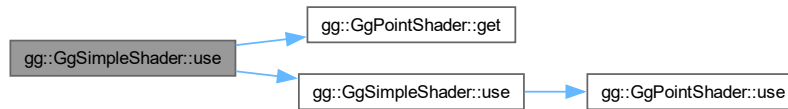
引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>mv</i>	GLfloat 型の 16 要素の配列変数に格納されたモデルビュー変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.

<i>i</i>	光源データの uniform block のインデックス.
----------	--------------------------------------

gg.h の 8454 行目に定義があります。

呼び出し関係図:



7.13.3.22 use() [11/13]

```

void gg::GgSimpleShader::use (
    const GLfloat * mp,
    const LightBuffer * light,
    GLint i = 0) const [inline]
  
```

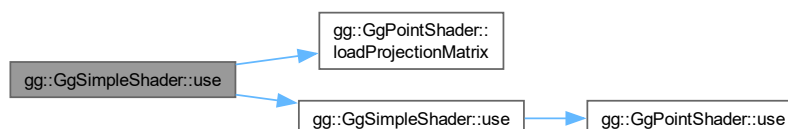
光源を指定し投影変換行列を設定してシェードプログラムの使用を開始する。

引数

<i>mp</i>	GLfloat 型の 16 要素の配列変数に格納された投影変換行列.
<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8489 行目に定義があります。

呼び出し関係図:



7.13.3.23 use() [12/13]

```

void gg::GgSimpleShader::use (
    const LightBuffer & light,
    GLint i = 0) const [inline]
  
```

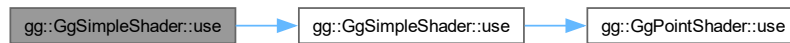
光源を指定してシェードプログラムの使用を開始する。

引数

<i>light</i>	光源の特性の gg::LightBuffer 構造体.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8397 行目に定義があります。

呼び出し関係図:



7.13.3.24 use() [13/13]

```
void gg::GgSimpleShader::use (
    const LightBuffer * light,
    GLint i = 0) const [inline]
```

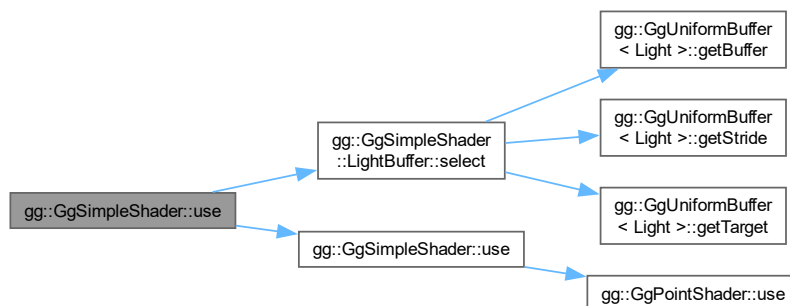
光源を指定してシェーダプログラムの使用を開始する。

引数

<i>light</i>	光源の特性の gg::LightBuffer 構造体のポインタ.
<i>i</i>	光源データの uniform block のインデックス.

gg.h の 8382 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

7.14 gg::GgTexture クラス

```
#include <gg.h>
```

公開メンバ関数

- [GgTexture](#) (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGB, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- [GgTexture](#) (const GgTexture &texture)=delete
- [GgTexture](#) (GgTexture &&o) noexcept
- virtual ~[GgTexture](#) ()
- [GgTexture](#) & operator= (const [GgTexture](#) &texture)=delete
- [GgTexture](#) & operator= ([GgTexture](#) &&o) noexcept
- void [bind](#) () const
- void [unbind](#) () const
- void [swapRandB](#) (bool swizzle) const
- const GLsizei & [getWidth](#) () const
- const GLsizei & [getHeight](#) () const
- void [getSize](#) (GLsizei *size) const
- const GLsizei * [getSize](#) () const
- const GLuint & [getTexture](#) () const

7.14.1 詳解

テクスチャ.

覚え書き

画像データを読み込んでテクスチャマップを作成する.

[gg.h](#) の 5224 行目に定義があります。

7.14.2 構築子と解体子

7.14.2.1 GgTexture() [1/3]

```
gg::GgTexture::GgTexture (  
    const GLvoid * image,  
    GLsizei width,  
    GLsizei height,  
    GLenum format = GL_RGB,  
    GLenum type = GL_UNSIGNED_BYTE,  
    GLenum internal = GL_RGBA,  
    GLenum wrap = GL_CLAMP_TO_EDGE,  
    bool swizzle = false) [inline]
```

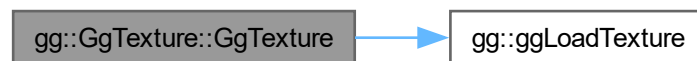
メモリ上のデータからテクスチャを作成するコンストラクタ.

引数

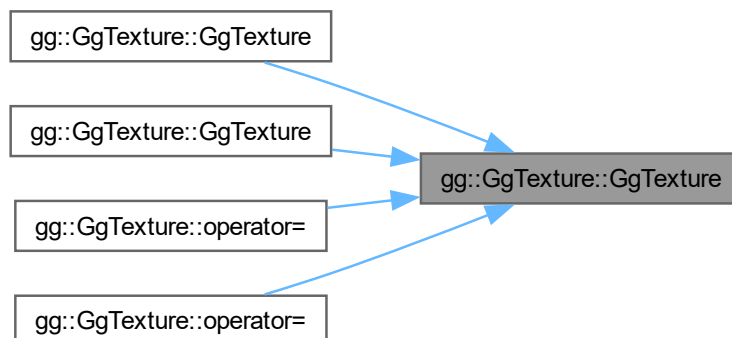
<i>image</i>	テクスチャとして用いる画像データ, nullptr ならデータを読み込まない.
<i>width</i>	テクスチャの横の画素数.
<i>height</i>	テクスチャの縦の画素数.
<i>format</i>	読み込む画像のフォーマット.
<i>type</i>	画像のデータ型.
<i>internal</i>	テクスチャの内部フォーマット.
<i>wrap</i>	テクスチャのラッピングモード, デフォルトは GL.CLAMP_TO_EDGE.
<i>swizzle</i>	true ならテクスチャの赤と青を入れ替える, デフォルトは false.

gg.h の 5246 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.14.2.2 GgTexture() [2/3]

```
gg::GgTexture::GgTexture (
    const GgTexture & texture) [delete]
```

コピーコンストラクタは使用しない。

引数

<code>texture</code>	コピー元のテクスチャ.
----------------------	-------------

呼び出し関係図:



7.14.2.3 GgTexture() [3/3]

```
gg::GgTexture::GgTexture (
    GgTexture && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

<code>o</code>	ムーブ元のテクスチャ.
----------------	-------------

覚え書き

テクスチャ名の所有権を移動し, ムーブ元のテクスチャ名を 0 にする. ムーブ元のデストラクタが同じテクスチャを削除しないようにするため.

[gg.h](#) の 5277 行目に定義があります。

呼び出し関係図:



7.14.2.4 ~GgTexture()

```
virtual gg::GgTexture::~~GgTexture () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 5287 行目に定義があります。

7.14.3 関数詳解

7.14.3.1 bind()

```
void gg::GgTexture::bind () const [inline]
```

テクスチャの使用開始 (このテクスチャを使用する際に呼び出す).

gg.h の 5331 行目に定義があります。

7.14.3.2 getHeight()

```
const GLsizei & gg::GgTexture::getHeight () const [inline]
```

使用しているテクスチャの縦の画素数を取り出す.

戻り値

テクスチャの縦の画素数.

gg.h の 5366 行目に定義があります。

被呼び出し関係図:



7.14.3.3 getSize() [1/2]

```
const GLsizei * gg::GgTexture::getSize () const [inline]
```

使用しているテクスチャのサイズを取り出す.

戻り値

テクスチャのサイズを格納した配列へのポインタ.

gg.h の 5387 行目に定義があります。

7.14.3.4 getSize() [2/2]

```
void gg::GgTexture::getSize (  
    GLsizei * size) const [inline]
```

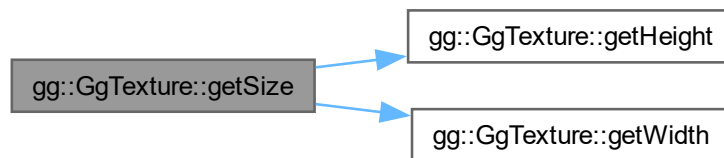
使用しているテクスチャのサイズを取り出す.

引数

size	テクスチャのサイズを格納する GLsizei 型の 2 要素の配列変数.
------	--------------------------------------

gg.h の 5376 行目に定義があります。

呼び出し関係図:



7.14.3.5 getTexture()

```
const GLuint & gg::GgTexture::getTexture () const [inline]
```

使用しているテクスチャのテクスチャ名を得る.

戻り値

テクスチャ名.

gg.h の 5397 行目に定義があります。

7.14.3.6 getWidth()

```
const GLsizei & gg::GgTexture::getWidth () const [inline]
```

使用しているテクスチャの横の画素数を取り出す.

戻り値

テクスチャの横の画素数.

gg.h の 5356 行目に定義があります。

被呼び出し関係図:



7.14.3.7 operator=() [1/2]

```
GgTexture & gg::GgTexture::operator= (
    const GgTexture & texture) [delete]
```

代入演算子は使用しない。

引数

<i>texture</i>	代入元のテクスチャ。
----------------	------------

戻り値

代入後のこのテクスチャの参照。

呼び出し関係図:



7.14.3.8 operator=() [2/2]

```
GgTexture & gg::GgTexture::operator= (
    GgTexture && o) [inline], [noexcept]
```

ムーブ代入演算子。

引数

<i>o</i>	ムーブ代入元のテクスチャ。
----------	---------------

戻り値

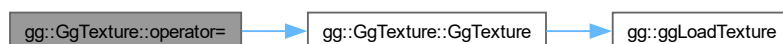
ムーブ代入後のこのテクスチャの参照。

覚え書き

元々保持していたテクスチャを削除してから所有権を移動する。

[gg.h](#) の 5310 行目に定義があります。

呼び出し関係図:



7.14.3.9 swapRandB()

```
void gg::GgTexture::swapRandB (
    bool swizzle) const
```

テクスチャの赤と青を交換する

引数

<code>swizzle</code>	赤と青を交換するなら true
----------------------	-----------------

[gg.cpp](#) の 4021 行目に定義があります。

7.14.3.10 unbind()

```
void gg::GgTexture::unbind () const [inline]
```

テクスチャの使用終了 (このテクスチャを使用しなくなったら呼び出す).

[gg.h](#) の 5339 行目に定義があります。

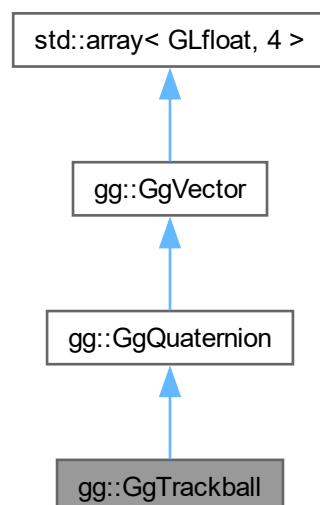
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

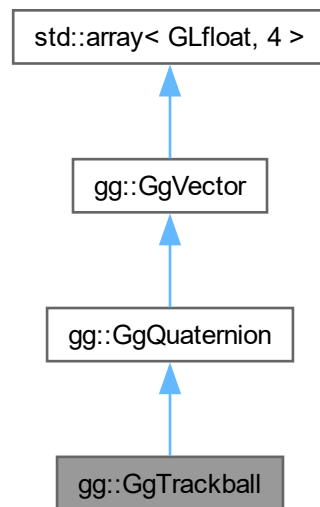
7.15 gg::GgTrackball クラス

```
#include <gg.h>
```

gg::GgTrackball の継承関係図



gg::GgTrackball 連携図



公開メンバ関数

- `GgTrackball` (`const GgQuaternion &q=ggIdentityQuaternion()`)
- `~GgTrackball` `()=default`
- `GgTrackball & operator=` (`const GgQuaternion &q`)
- `void region` (`GLfloat w, GLfloat h`)
- `void region` (`int w, int h`)
- `void begin` (`GLfloat x, GLfloat y`)
- `void motion` (`GLfloat x, GLfloat y`)
- `void rotate` (`const GgQuaternion &q`)
- `void end` (`GLfloat x, GLfloat y`)
- `void reset` (`const GgQuaternion &q=ggIdentityQuaternion()`)
- `const GLfloat * getStart` `() const`
- `const GLfloat & getStart` (`int direction`) `const`
- `void getStart` (`GLfloat *position`) `const`
- `const GLfloat * getScale` `() const`
- `const GLfloat getScale` (`int direction`) `const`
- `void getScale` (`GLfloat *factor`) `const`
- `const GgQuaternion & getQuaternion` `() const`
- `const GgMatrix & getMatrix` `() const`
- `const GLfloat * get` `() const`

基底クラス `gg::GgQuaternion` に属する継承公開メンバ関数

- `GgQuaternion` `()=default`
- `constexpr GgQuaternion` (`GLfloat x, GLfloat y, GLfloat z, GLfloat w`)
- `constexpr GgQuaternion` (`GLfloat c`)
- `GgQuaternion` (`const GLfloat *a`)

- `GgQuaternion` (const `GgVector` &v)
- `GgQuaternion` (const `GgQuaternion` &q)=default
- `GgQuaternion` (`GgQuaternion` &&q)=default
- `~GgQuaternion` ()=default
- `GgQuaternion` & `operator=` (const `GgQuaternion` &q)=default
- `GgQuaternion` & `operator=` (`GgQuaternion` &&q)=default
- `GLfloat` `norm` () const
- `GgQuaternion` & `loadAdd` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w)
- `GgQuaternion` & `loadAdd` (const `GLfloat` *a)
- `GgQuaternion` & `loadAdd` (const `GgVector` &v)
- `GgQuaternion` & `loadAdd` (const `GgQuaternion` &q)
- `GgQuaternion` & `loadSubtract` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w)
- `GgQuaternion` & `loadSubtract` (const `GLfloat` *a)
- `GgQuaternion` & `loadSubtract` (const `GgVector` &v)
- `GgQuaternion` & `loadSubtract` (const `GgQuaternion` &q)
- `GgQuaternion` & `loadMultiply` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w)
- `GgQuaternion` & `loadMultiply` (const `GLfloat` *a)
- `GgQuaternion` & `loadMultiply` (const `GgVector` &v)
- `GgQuaternion` & `loadMultiply` (const `GgQuaternion` &q)
- `GgQuaternion` & `loadDivide` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w)
- `GgQuaternion` & `loadDivide` (const `GLfloat` *a)
- `GgQuaternion` & `loadDivide` (const `GgVector` &v)
- `GgQuaternion` & `loadDivide` (const `GgQuaternion` &q)
- `GgQuaternion` `add` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w) const
- `GgQuaternion` `add` (const `GLfloat` *a) const
- `GgQuaternion` `add` (const `GgVector` &v) const
- `GgQuaternion` `add` (const `GgQuaternion` &q) const
- `GgQuaternion` `subtract` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w) const
- `GgQuaternion` `subtract` (const `GLfloat` *a) const
- `GgQuaternion` `subtract` (const `GgVector` &v) const
- `GgQuaternion` `subtract` (const `GgQuaternion` &q) const
- `GgQuaternion` `multiply` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w) const
- `GgQuaternion` `multiply` (const `GLfloat` *a) const
- `GgQuaternion` `multiply` (const `GgVector` &v) const
- `GgQuaternion` `multiply` (const `GgQuaternion` &q) const
- `GgQuaternion` `divide` (`GLfloat` x, `GLfloat` y, `GLfloat` z, `GLfloat` w) const
- `GgQuaternion` `divide` (const `GLfloat` *a) const
- `GgQuaternion` `divide` (const `GgVector` &v) const
- `GgQuaternion` `divide` (const `GgQuaternion` &q) const
- `GgQuaternion` & `operator=` (const `GLfloat` *a)
- `GgQuaternion` & `operator=` (const `GgVector` &v)
- `GgQuaternion` & `operator+=` (const `GLfloat` *a)
- `GgQuaternion` & `operator+=` (const `GgVector` &v)
- `GgQuaternion` & `operator+=` (const `GgQuaternion` &q)
- `GgQuaternion` & `operator-=` (const `GLfloat` *a)
- `GgQuaternion` & `operator-=` (const `GgVector` &v)
- `GgQuaternion` & `operator-=` (const `GgQuaternion` &q)
- `GgQuaternion` & `operator*=` (const `GLfloat` *a)
- `GgQuaternion` & `operator*=` (const `GgVector` &v)
- `GgQuaternion` & `operator*=` (const `GgQuaternion` &q)
- `GgQuaternion` & `operator/=` (const `GLfloat` *a)
- `GgQuaternion` & `operator/=` (const `GgVector` &v)
- `GgQuaternion` & `operator/=` (const `GgQuaternion` &q)
- `GgQuaternion` `operator+` (const `GLfloat` *a) const
- `GgQuaternion` `operator+` (const `GgVector` &v) const

- [GgQuaternion operator+](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator-](#) (const GLfloat *a) const
- [GgQuaternion operator-](#) (const [GgVector](#) &v) const
- [GgQuaternion operator-](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator*](#) (const GLfloat *a) const
- [GgQuaternion operator*](#) (const [GgVector](#) &v) const
- [GgQuaternion operator*](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion operator/](#) (const GLfloat *a) const
- [GgQuaternion operator/](#) (const [GgVector](#) &v) const
- [GgQuaternion operator/](#) (const [GgQuaternion](#) &q) const
- [GgQuaternion & loadMatrix](#) (const GLfloat *a)
- [GgQuaternion & loadMatrix](#) (const [GgMatrix](#) &m)
- [GgQuaternion & loadIdentity](#) ()
- [GgQuaternion & loadRotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v, GLfloat a)
- [GgQuaternion & loadRotate](#) (const GLfloat *v)
- [GgQuaternion & loadRotateX](#) (GLfloat a)
- [GgQuaternion & loadRotateY](#) (GLfloat a)
- [GgQuaternion & loadRotateZ](#) (GLfloat a)
- [GgQuaternion rotate](#) (GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v, GLfloat a) const
- [GgQuaternion rotate](#) (const GLfloat *v) const
- [GgQuaternion rotateX](#) (GLfloat a) const
- [GgQuaternion rotateY](#) (GLfloat a) const
- [GgQuaternion rotateZ](#) (GLfloat a) const
- [GgQuaternion & loadEuler](#) (GLfloat heading, GLfloat pitch, GLfloat roll)
- [GgQuaternion & loadEuler](#) (const GLfloat *e)
- [GgQuaternion euler](#) (GLfloat heading, GLfloat pitch, GLfloat roll) const
- [GgQuaternion euler](#) (const GLfloat *e) const
- [GgQuaternion euler](#) (const [GgVector](#) &e) const
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const GLfloat *b, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const [GgQuaternion](#) &r, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const [GgQuaternion](#) &q, const GLfloat *a, GLfloat t)
- [GgQuaternion & loadSlerp](#) (const GLfloat *a, const [GgQuaternion](#) &q, GLfloat t)
- [GgQuaternion & loadNormalize](#) (const GLfloat *a)
- [GgQuaternion & loadNormalize](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadConjugate](#) (const GLfloat *a)
- [GgQuaternion & loadConjugate](#) (const [GgQuaternion](#) &q)
- [GgQuaternion & loadInvert](#) (const GLfloat *a)
- [GgQuaternion & loadInvert](#) (const [GgQuaternion](#) &q)
- [GgQuaternion slerp](#) (GLfloat *a, GLfloat t) const
- [GgQuaternion slerp](#) (const [GgQuaternion](#) &q, GLfloat t) const
- [GgQuaternion normalize](#) () const
- [GgQuaternion conjugate](#) () const
- [GgQuaternion invert](#) () const
- void [get](#) (GLfloat *a) const
- void [getMatrix](#) (GLfloat *a) const
- void [getMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getMatrix](#) () const
- void [getConjugateMatrix](#) (GLfloat *a) const
- void [getConjugateMatrix](#) ([GgMatrix](#) &m) const
- [GgMatrix getConjugateMatrix](#) () const

基底クラス `gg::GgVector` に属する継承公開メンバ関数

- `GgVector` ()=default
- constexpr `GgVector` (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr `GgVector` (const GLfloat *a)
- constexpr `GgVector` (GLfloat c)
- `GgVector` (const GgVector &v)=default
- `GgVector` (GgVector &&v)=default
- `~GgVector` ()=default
- `GgVector & operator=` (const `GgVector` &v)=default
- `GgVector & operator=` (`GgVector` &&v)=default
- constexpr `GgVector operator+` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator+` (GLfloat c) const noexcept
- constexpr `GgVector & operator+=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator+=` (GLfloat c) noexcept
- constexpr `GgVector operator-` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator-` (GLfloat c) const noexcept
- constexpr `GgVector & operator-=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator-=` (GLfloat c) noexcept
- constexpr `GgVector operator*` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator*` (GLfloat c) const noexcept
- constexpr `GgVector & operator*=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator*=` (GLfloat c) noexcept
- constexpr `GgVector operator/` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator/` (GLfloat c) const noexcept
- constexpr `GgVector & operator/=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator/=` (GLfloat c) noexcept
- GLfloat `dot3` (const `GgVector` &v) const
- GLfloat `length3` () const
- GLfloat `distance3` (const `GgVector` &v) const
- `GgVector normalize3` () const
- GLfloat `dot4` (const `GgVector` &v) const
- GLfloat `length4` () const
- GLfloat `distance4` (const `GgVector` &v) const
- `GgVector normalize4` () const

7.15.1 詳解

簡易トラックボール処理.

`gg.h` の 4806 行目に定義があります。

7.15.2 構築子と解体子

7.15.2.1 GgTrackball()

```
gg::GgTrackball::GgTrackball (
    const GgQuaternion & q = ggIdentityQuaternion()) [inline]
```

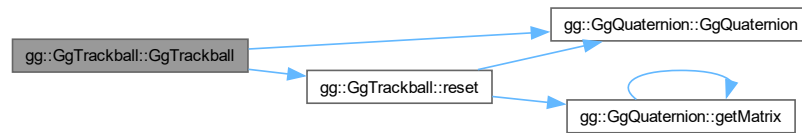
コンストラクタ.

引数

q	トラックボールの回転の初期値の四元数.
-----	---------------------

gg.h の 4821 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.15.2.2 ~GgTrackball()

```
gg::GgTrackball::~~GgTrackball () [default]
```

デストラクタ.

7.15.3 関数詳解

7.15.3.1 begin()

```
void gg::GgTrackball::begin (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を開始する.

引数

x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ開始時 (マウスボタンを押したとき) に呼び出す.

gg.cpp の 3462 行目に定義があります。

7.15.3.2 end()

```
void gg::GgTrackball::end (
    GLfloat x,
    GLfloat y)
```

トラックボール処理を停止する.

引数

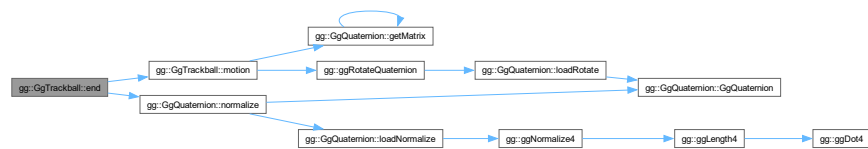
<i>x</i>	現在のマウスの <i>x</i> 座標.
<i>y</i>	現在のマウスの <i>y</i> 座標.

覚え書き

マウスのドラッグ終了時 (マウスボタンを離したとき) に呼び出す.

[gg.cpp](#) の 3528 行目に定義があります。

呼び出し関係図:



7.15.3.3 get()

```
const GLfloat * gg::GgTrackball::get () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GLfloat 型の 16 要素の配列.

[gg.h](#) の 4999 行目に定義があります。

7.15.3.4 getMatrix()

```
const GgMatrix & gg::GgTrackball::getMatrix () const [inline]
```

現在の回転の変換行列を取り出す.

戻り値

回転の変換を表す GgMatrix 型の変換行列.

[gg.h](#) の 4989 行目に定義があります。

7.15.3.5 getQuaternion()

```
const GgQuaternion & gg::GgTrackball::getQuaternion () const [inline]
```

現在の回転の四元数を取り出す.

戻り値

回転の変換を表す Quaternion 型の四元数.

gg.h の 4979 行目に定義があります。

呼び出し関係図:



7.15.3.6 getScale() [1/3]

```
const GLfloat * gg::GgTrackball::getScale () const [inline]
```

トラックボール処理の換算係数を取り出す.

戻り値

トラックボールの換算係数のポインタ.

gg.h の 4948 行目に定義があります。

7.15.3.7 getScale() [2/3]

```
void gg::GgTrackball::getScale (
    GLfloat * factor) const [inline]
```

トラックボール処理の換算係数を取り出す.

引数

<i>factor</i>	トラックボールの換算係数を格納する 2 要素の配列.
---------------	----------------------------

gg.h の 4968 行目に定義があります。

7.15.3.8 getScale() [3/3]

```
const GLfloat gg::GgTrackball::getScale (
    int direction) const [inline]
```

トラックボール処理の換算係数を取り出す.

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

[gg.h](#) の 4958 行目に定義があります。

7.15.3.9 `getStart()` [1/3]

```
const GLfloat * gg::GgTrackball::getStart () const [inline]
```

トラックボール処理の開始位置を取り出す。

戻り値

トラックボールの開始位置のポインタ。

[gg.h](#) の 4919 行目に定義があります。

7.15.3.10 `getStart()` [2/3]

```
void gg::GgTrackball::getStart (
    GLfloat * position) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>position</i>	トラックボールの開始位置を格納する 2 要素の配列.
-----------------	----------------------------

[gg.h](#) の 4937 行目に定義があります。

7.15.3.11 `getStart()` [3/3]

```
const GLfloat & gg::GgTrackball::getStart (
    int direction) const [inline]
```

トラックボール処理の開始位置を取り出す。

引数

<i>direction</i>	0 なら x 方向, 1 なら y 方向.
------------------	-----------------------

[gg.h](#) の 4927 行目に定義があります。

7.15.3.12 `motion()`

```
void gg::GgTrackball::motion (
    GLfloat x,
    GLfloat y)
```

回転の変換行列を計算する。

引数

x	現在のマウスの x 座標.
y	現在のマウスの y 座標.

覚え書き

マウスのドラッグ中に呼び出す.

gg.cpp の 3478 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.15.3.13 operator=()

```
GgTrackball & gg::GgTrackball::operator= (
    const GgQuaternion & q) [inline]
```

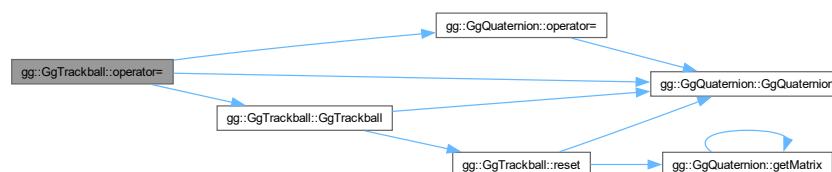
代入.

引数

q	トラックボールの回転の初期値の四元数.
-----	---------------------

gg.h の 4836 行目に定義があります。

呼び出し関係図:



7.15.3.14 region() [1/2]

```
void gg::GgTrackball::region (
    GLfloat w,
    GLfloat h)
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す.

[gg.cpp](#) の 3449 行目に定義があります。

被呼び出し関係図:



7.15.3.15 region() [2/2]

```
void gg::GgTrackball::region (
    int w,
    int h) [inline]
```

トラックボール処理するマウスの移動範囲を指定する.

引数

<i>w</i>	領域の横幅.
<i>h</i>	領域の高さ.

覚え書き

ウィンドウのリサイズ時に呼び出す。

gg.h の 4862 行目に定義があります。

呼び出し関係図:



7.15.3.16 reset()

```
void gg::GgTrackball::reset (
    const GgQuaternion & q = ggIdentityQuaternion())
```

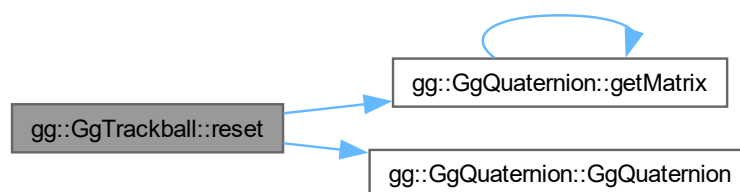
トラックボールをリセットする。

引数

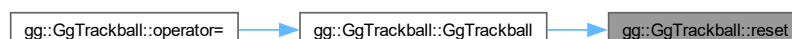
<i>q</i>	トラックボールの回転の初期値の四元数.
----------	---------------------

gg.cpp の 3431 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.15.3.17 rotate()

```
void gg::GgTrackball::rotate (
    const GgQuaternion & q)
```

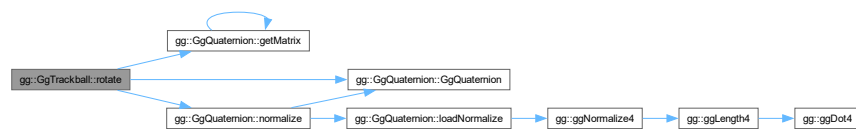
トラックボールの回転角を修正する。

引数

<i>q</i>	修正分の回転角の四元数.
----------	--------------

[gg.cpp](#) の 3507 行目に定義があります。

呼び出し関係図:



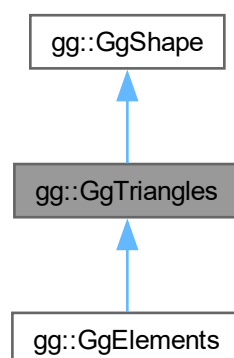
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

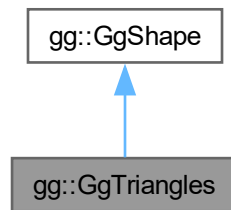
7.16 gg::GgTriangles クラス

```
#include <gg.h>
```

gg::GgTriangles の継承関係図



gg::GgTriangles 連携図



公開メンバ関数

- [GgTriangles](#) (GLenum mode=GL_TRIANGLES)
- [GgTriangles](#) (const [GgVertex](#) *vert, GLsizei count, GLenum mode=GL_TRIANGLES, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgTriangles](#) ()
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [send](#) (const [GgVertex](#) *vert, GLint first=0, GLsizei count=0) const
- void [load](#) (const [GgVertex](#) *vert, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual void [draw](#) (GLint first=0, GLsizei count=0) const

基底クラス [gg::GgShape](#) に属する継承公開メンバ関数

- [GgShape](#) (GLenum mode=0)
- virtual [~GgShape](#) ()
- virtual [operator bool](#) () const noexcept
- virtual bool [operator!](#) () const noexcept
- const GLuint & [get](#) () const
- void [setMode](#) (GLenum mode)
- const GLenum & [getMode](#) () const

7.16.1 詳解

三角形で表した形状データ (Arrays 形式).

[gg.h](#) の 6792 行目に定義があります。

7.16.2 構築子と解体子

7.16.2.1 GgTriangles() [1/2]

```
gg::GgTriangles::GgTriangles (
    GLenum mode = GL_TRIANGLES) [inline]
```

コンストラクタ.

引数

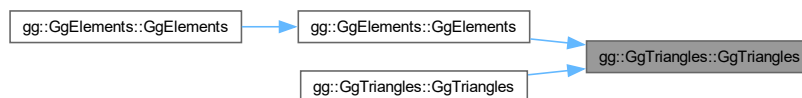
<i>mode</i>	描画する基本図形の種類.
-------------	--------------

gg.h の 6805 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.16.2.2 GgTriangles() [2/2]

```

gg::GgTriangles::GgTriangles (
    const GgVertex * vert,
    GLsizei count,
    GLenum mode = GL_TRIANGLES,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

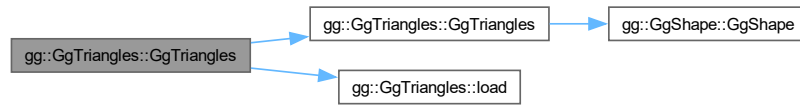
コンストラクタ.

引数

<i>vert</i>	この図形の頂点属性の配列 (nullptr ならデータを転送しない).
<i>count</i>	頂点数.
<i>mode</i>	描画する基本図形の種類.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6818 行目に定義があります。

呼び出し関係図:



7.16.2.3 ~GgTriangles()

```
virtual gg::GgTriangles::~~GgTriangles () [inline], [virtual]
```

デストラクタ.

[gg.h](#) の 6832 行目に定義があります。

7.16.3 関数詳解

7.16.3.1 draw()

```
void gg::GgTriangles::draw (
    GLint first = 0,
    GLsizei count = 0) const [virtual]
```

三角形の描画.

引数

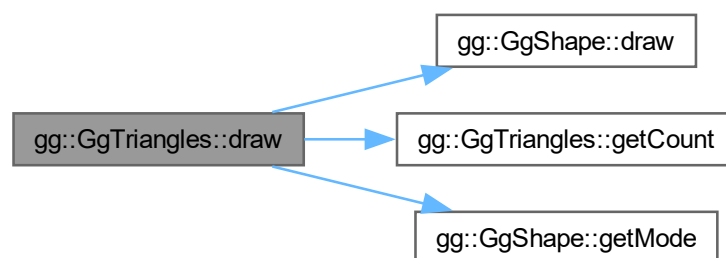
<i>first</i>	描画を開始する最初の三角形番号.
<i>count</i>	描画する三角形数, 0 なら全部の三角形を描く.

[gg::GgShape](#) を再実装しています。

[gg::GgElements](#) で再実装されています。

[gg.cpp](#) の 5175 行目に定義があります。

呼び出し関係図:



7.16.3.2 getBuffer()

```
const GLuint & gg::GgTriangles::getBuffer () const [inline]
```

頂点属性を格納した頂点バッファオブジェクト名を取り出す。

戻り値

この図形の頂点属性を格納した頂点バッファオブジェクト名。

[gg.h](#) の [6851](#) 行目に定義があります。

7.16.3.3 getCount()

```
const GLsizei & gg::GgTriangles::getCount () const [inline]
```

データの数を取り出す。

戻り値

この図形の頂点属性の数 (頂点数)。

[gg.h](#) の [6841](#) 行目に定義があります。

被呼び出し関係図:



7.16.3.4 load()

```
void gg::GgTriangles::load (
    const GgVertex * vert,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW)
```

バッファオブジェクトを確保して頂点属性を格納する。

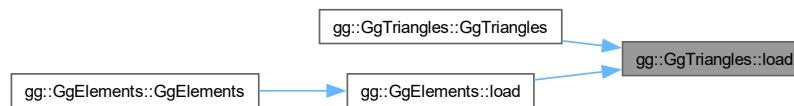
引数

<i>vert</i>	頂点属性が格納されている領域の先頭のポインタ。
-------------	-------------------------

<i>count</i>	頂点のデータの数 (頂点数).
<i>usage</i>	バッファオブジェクトの使い方.

gg.cpp の 5156 行目に定義があります。

被呼び出し関係図:



7.16.3.5 send()

```
void gg::GgTriangles::send (
    const GgVertex * vert,
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

既存のバッファオブジェクトに頂点属性を転送する。

引数

<i>vert</i>	転送元の頂点属性が格納されている領域の先頭のポインタ.
<i>first</i>	転送先のバッファオブジェクトの先頭の要素番号.
<i>count</i>	転送する頂点の位置データの数 (0 ならバッファオブジェクト全体).

gg.h の 6863 行目に定義があります。

被呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

7.17 gg::GgUniformBuffer< T > クラステンプレート

```
#include <gg.h>
```

公開メンバ関数

- [GgUniformBuffer](#) ()
- [GgUniformBuffer](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- [GgUniformBuffer](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- virtual [~GgUniformBuffer](#) ()
- const GLuint & [getTarget](#) () const
- GLsizeiptr [getStride](#) () const
- const GLsizei & [getCount](#) () const
- const GLuint & [getBuffer](#) () const
- void [bind](#) () const
- void [unbind](#) () const
- void * [map](#) () const
- void * [map](#) (GLuint first, GLsizei count) const
- void [unmap](#) () const
- void [load](#) (const T *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [load](#) (const T &data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void [send](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [fill](#) (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [read](#) (GLvoid *data, GLint offset=0, GLsizei size=sizeof(T), GLint first=0, GLsizei count=0) const
- void [copy](#) (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

7.17.1 詳解

```
template<typename T>
class gg::GgUniformBuffer< T >
```

ユニフォームバッファオブジェクト。

覚え書き

ユニフォーム変数を格納するバッファオブジェクトの基底クラス。

[gg.h](#) の 6082 行目に定義があります。

7.17.2 構築子と解体子

7.17.2.1 GgUniformBuffer() [1/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer () [inline]
```

コンストラクタ。

[gg.h](#) の 6092 行目に定義があります。

7.17.2.2 GgUniformBuffer() [2/3]

```
template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

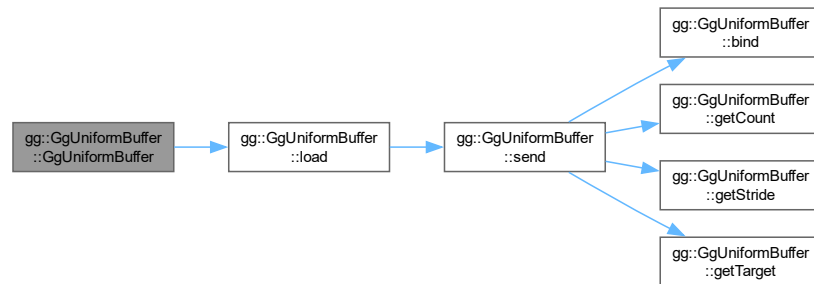
ユニフォームバッファオブジェクトのブロックごとにデータを転送するコンストラクタ.

引数

<i>data</i>	データが格納されている領域の先頭のポインタ (<code>nullptr</code> ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

`gg.h` の 6103 行目に定義があります。

呼び出し関係図:



7.17.2.3 GgUniformBuffer() [3/3]

```

template<typename T>
gg::GgUniformBuffer< T >::GgUniformBuffer (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

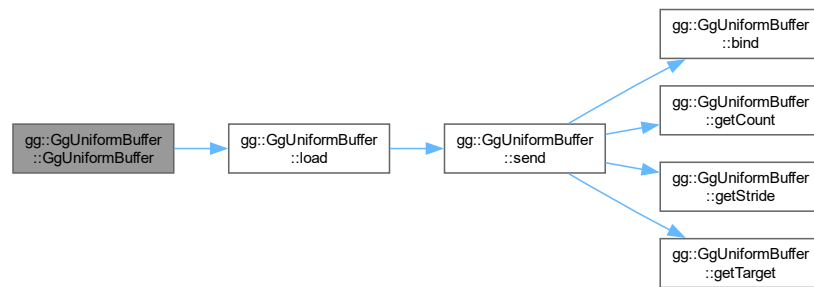
ユニフォームバッファオブジェクトの全ブロックに同じデータを格納するコンストラクタ。

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

`gg.h` の 6115 行目に定義があります。

呼び出し関係図:



7.17.2.4 ~GgUniformBuffer()

```
template<typename T>
virtual gg::GgUniformBuffer< T >::~~GgUniformBuffer () [inline], [virtual]
```

デストラクタ.

gg.h の 6123 行目に定義があります。

7.17.3 関数詳解

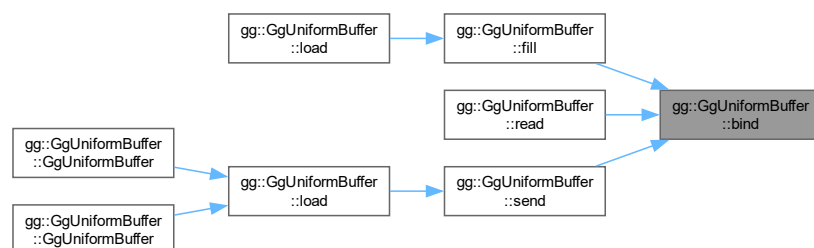
7.17.3.1 bind()

```
template<typename T>
void gg::GgUniformBuffer< T >::bind () const [inline]
```

ユニフォームバッファオブジェクトを結合する.

gg.h の 6170 行目に定義があります。

被呼び出し関係図:



7.17.3.2 copy()

```
template<typename T>
void gg::GgUniformBuffer< T >::copy (
    GLuint src_buffer,
    GLint src_first = 0,
    GLint dst_first = 0,
    GLsizei count = 0) const [inline]
```

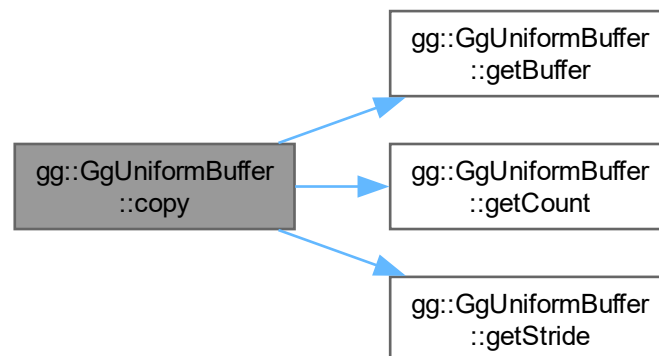
別のバッファオブジェクトからデータを複写する.

引数

<i>src_buffer</i>	複写元のバッファオブジェクト名.
<i>src_first</i>	複写元 (buffer) の先頭のデータの位置.
<i>dst_first</i>	複写先 (getBuffer()) の先頭のデータの位置.
<i>count</i>	複写するデータの数 (0 ならバッファオブジェクト全体).

[gg.h](#) の 6381 行目に定義があります.

呼び出し関係図:



7.17.3.3 fill()

```
template<typename T>
void gg::GgUniformBuffer< T >::fill (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

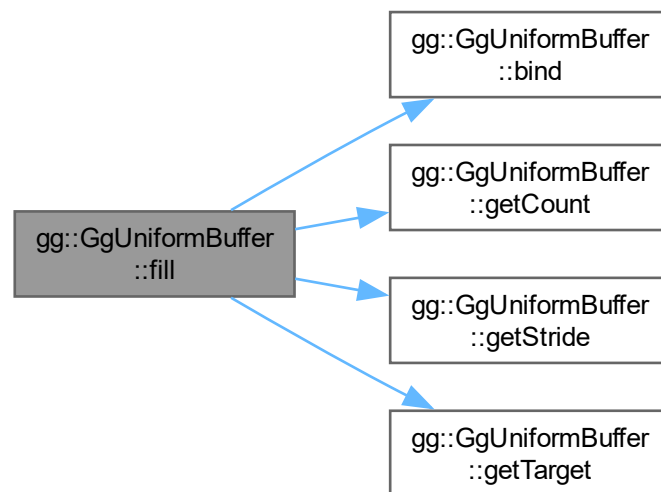
ユニフォームバッファオブジェクトの全ブロックのメンバーを同じデータを格納する.

引数

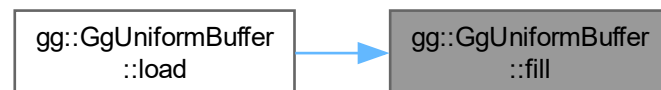
<i>data</i>	格納するデータ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

gg.h の 6298 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.17.3.4 getBuffer()

```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getBuffer () const [inline]
```

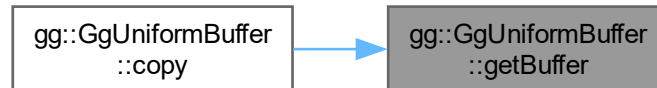
ユニフォームバッファオブジェクト名を取り出す。

戻り値

このユニフォームバッファオブジェクト名.

gg.h の 6162 行目に定義があります。

被呼び出し関係図:



7.17.3.5 getCount()

```
template<typename T>
const GLsizei & gg::GgUniformBuffer< T >::getCount () const [inline]
```

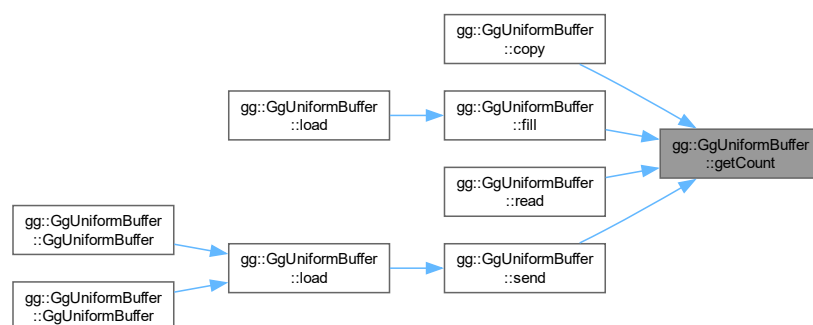
データの数を取り出す。

戻り値

このユニフォームバッファオブジェクトのデータの数.

gg.h の 6152 行目に定義があります。

被呼び出し関係図:



7.17.3.6 getStride()

```
template<typename T>
GLsizeiptr gg::GgUniformBuffer< T >::getStride () const [inline]
```

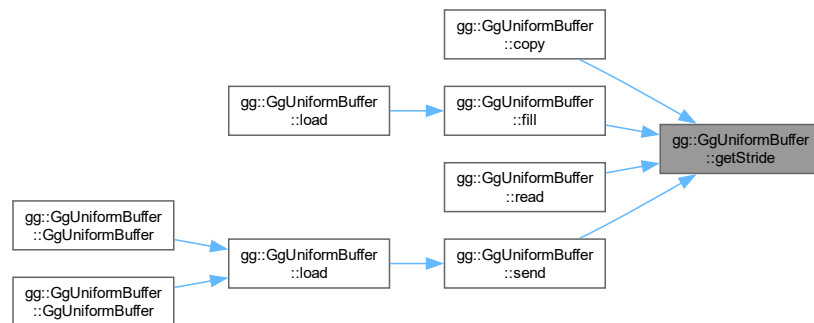
ユニフォームバッファオブジェクトのアライメントを考慮したデータの間隔を取り出す。

戻り値

このユニフォームバッファオブジェクトのデータの間隔。

gg.h の 6142 行目に定義があります。

被呼び出し関係図:



7.17.3.7 getTarget()

```
template<typename T>
const GLuint & gg::GgUniformBuffer< T >::getTarget () const [inline]
```

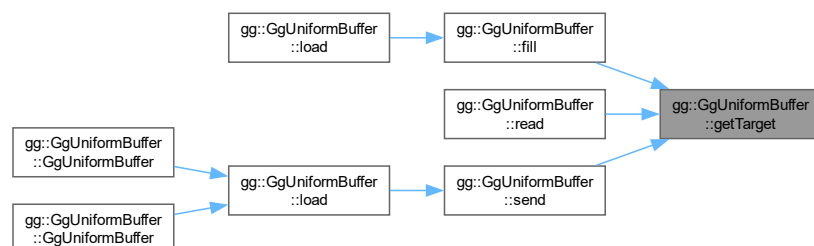
ユニフォームバッファオブジェクトのターゲットを取り出す。

戻り値

このユニフォームバッファオブジェクトのターゲット。

gg.h の 6132 行目に定義があります。

被呼び出し関係図:



7.17.3.8 load() [1/2]

```
template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T & data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

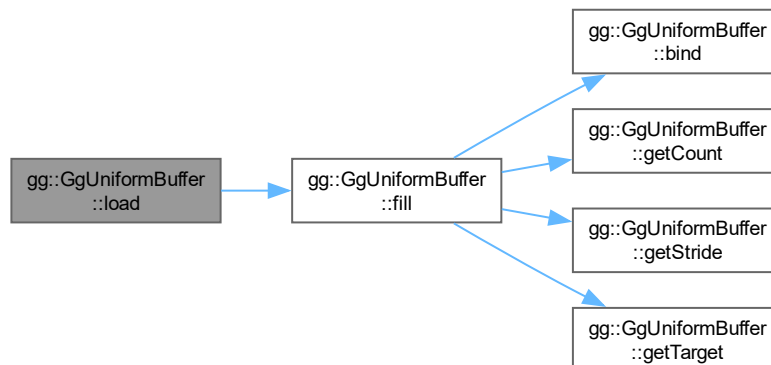
ユニフォームバッファオブジェクトを確保して全てのブロックに同じデータを格納する。

引数

<i>data</i>	格納するデータ.
<i>count</i>	格納する数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6239 行目に定義があります。

呼び出し関係図:



7.17.3.9 load() [2/2]

```
template<typename T>
void gg::GgUniformBuffer< T >::load (
    const T * data,
    GLsizei count,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

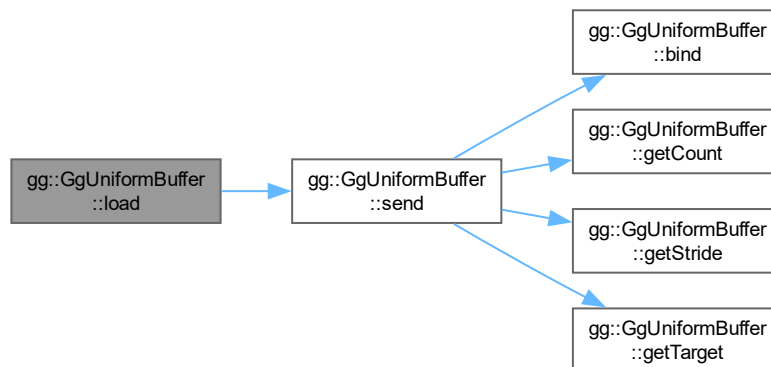
ユニフォームバッファオブジェクトを確保してブロックごとにデータを転送する。

引数

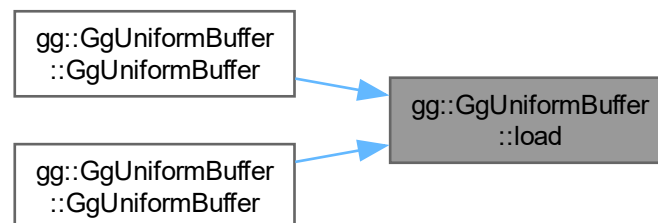
<i>data</i>	データが格納されている領域の先頭のポインタ (nullptr ならデータを転送しない).
<i>count</i>	データの数.
<i>usage</i>	バッファオブジェクトの使い方.

gg.h の 6220 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.17.3.10 map() [1/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map () const [inline]
```

ユニフォームバッファオブジェクトをマップする.

戻り値

マップしたメモリの先頭のポインタ.

gg.h の 6188 行目に定義があります。

7.17.3.11 map() [2/2]

```
template<typename T>
void * gg::GgUniformBuffer< T >::map (
    GLint first,
    GLsizei count) const [inline]
```

ユニフォームバッファオブジェクトの指定した範囲をマップする.

引数

<i>first</i>	マップする範囲のバッファオブジェクトの先頭からの位置.
<i>count</i>	マップするデータの数 (0 ならバッファオブジェクト全体).

戻り値

マップしたメモリの先頭のポインタ.

[gg.h](#) の 6200 行目に定義があります。

7.17.3.12 read()

```
template<typename T>
void gg::GgUniformBuffer< T >::read (
    GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
```

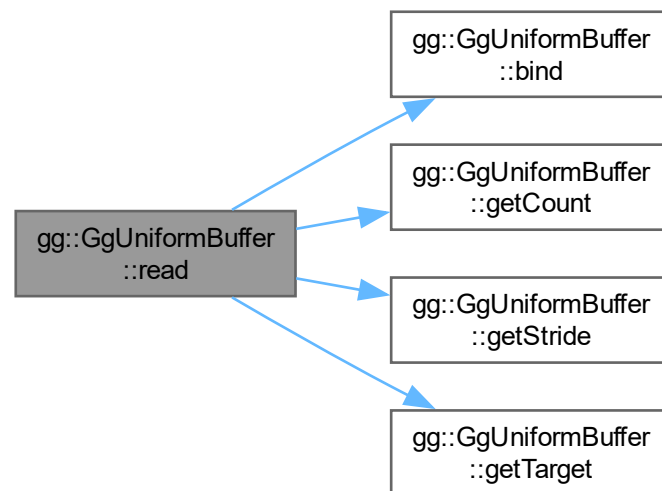
ユニフォームバッファオブジェクトからデータを抽出する.

引数

<i>data</i>	抽出先の領域の先頭のポインタ.
<i>offset</i>	抽出元のユニフォームバッファオブジェクトのメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	抽出するデータの一個あたりのバイト数.
<i>first</i>	抽出元のユニフォームバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	抽出するデータの数 (0 ならユニフォームバッファオブジェクト全体).

[gg.h](#) の 6333 行目に定義があります。

呼び出し関係図:



7.17.3.13 send()

```

template<typename T>
void gg::GgUniformBuffer< T >::send (
    const GLvoid * data,
    GLint offset = 0,
    GLsizei size = sizeof(T),
    GLint first = 0,
    GLsizei count = 0) const [inline]
  
```

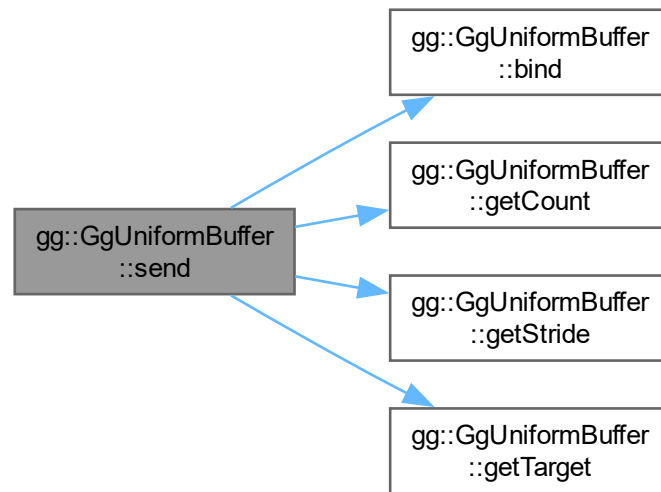
ユニフォームバッファオブジェクトを確保してユニフォームバッファオブジェクトのブロックごとのメンバを同じデータで埋める。

引数

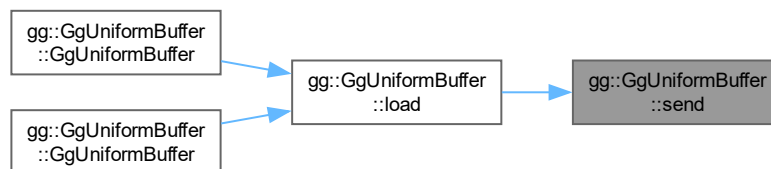
<i>data</i>	データが格納されている領域の先頭のポインタ.
<i>offset</i>	格納先のメンバのブロックの先頭からのバイトオフセット.
<i>size</i>	格納するデータの一個あたりのバイト数.
<i>first</i>	格納先のバッファオブジェクトのブロックの先頭の番号.
<i>count</i>	格納するデータの数.

[gg.h](#) の 6260 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.17.3.14 unbind()

```
template<typename T>
void gg::GgUniformBuffer< T >::unbind () const [inline]
```

ユニフォームバッファオブジェクトを解放する。

`gg.h` の 6178 行目に定義があります。

7.17.3.15 unmap()

```
template<typename T>
void gg::GgUniformBuffer< T >::unmap () const [inline]
```

バッファオブジェクトをアンマップする。

gg.h の 6208 行目に定義があります。

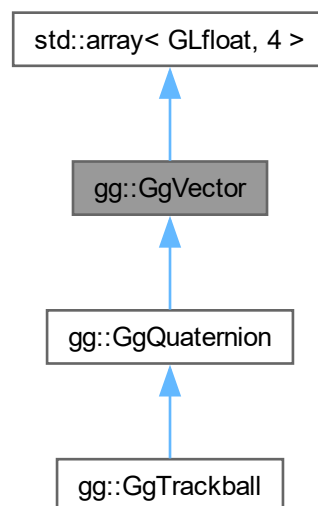
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

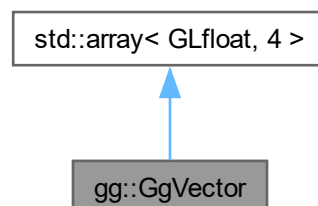
7.18 gg::GgVector クラス

```
#include <gg.h>
```

gg::GgVector の継承関係図



gg::GgVector 連携図



公開メンバ関数

- `GgVector()`=default
- constexpr `GgVector` (GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3)
- constexpr `GgVector` (const GLfloat *a)
- constexpr `GgVector` (GLfloat c)
- `GgVector` (const GgVector &v)=default
- `GgVector` (GgVector &&v)=default
- `~GgVector()`=default
- `GgVector & operator=` (const `GgVector` &v)=default
- `GgVector & operator=` (GgVector &&v)=default
- constexpr `GgVector operator+` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator+` (GLfloat c) const noexcept
- constexpr `GgVector & operator+=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator+=` (GLfloat c) noexcept
- constexpr `GgVector operator-` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator-` (GLfloat c) const noexcept
- constexpr `GgVector & operator-=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator-=` (GLfloat c) noexcept
- constexpr `GgVector operator*` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator*` (GLfloat c) const noexcept
- constexpr `GgVector & operator*=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator*=` (GLfloat c) noexcept
- constexpr `GgVector operator/` (const `GgVector` &v) const noexcept
- constexpr `GgVector operator/` (GLfloat c) const noexcept
- constexpr `GgVector & operator/=` (const `GgVector` &v) noexcept
- constexpr `GgVector & operator/=` (GLfloat c) noexcept
- GLfloat `dot3` (const `GgVector` &v) const
- GLfloat `length3` () const
- GLfloat `distance3` (const `GgVector` &v) const
- `GgVector normalize3` () const
- GLfloat `dot4` (const `GgVector` &v) const
- GLfloat `length4` () const
- GLfloat `distance4` (const `GgVector` &v) const
- `GgVector normalize4` () const

7.18.1 詳解

単精度実数の4要素のベクトル。

`gg.h` の 1577 行目に定義があります。

7.18.2 構築子と解体子

7.18.2.1 `GgVector()` [1/6]

```
gg::GgVector::GgVector () [default]
```



```
GLfloat v2,
GLfloat v3) [inline], [constexpr]
```

コンストラクタ.

引数

<i>v0</i>	GLfloat 型の値.
<i>v1</i>	GLfloat 型の値.
<i>v2</i>	GLfloat 型の値.
<i>v3</i>	GLfloat 型の値.

[gg.h](#) の 1594 行目に定義があります。

7.18.2.3 GgVector() [3/6]

```
gg::GgVector::GgVector (
    const GLfloat * a) [inline], [constexpr]
```

コンストラクタ.

引数

<i>a</i>	GLfloat 型の 4 要素の配列変数.
----------	-----------------------

[gg.h](#) の 1604 行目に定義があります。

呼び出し関係図:



7.18.2.4 GgVector() [4/6]

```
gg::GgVector::GgVector (
    GLfloat c) [inline], [constexpr]
```

コンストラクタ.

引数

<code>c</code>	GLfloat 型の値.
----------------	--------------

gg.h の 1614 行目に定義があります。

呼び出し関係図:



7.18.2.5 GgVector() [5/6]

```
gg::GgVector::GgVector (
    const GgVector & v) [default]
```

コピーコンストラクタ.

引数

<code>v</code>	コピー元のベクトル.
----------------	------------

呼び出し関係図:



7.18.2.6 GgVector() [6/6]

```
gg::GgVector::GgVector (
    GgVector && v) [default]
```

ムーブコンストラクタ.

引数

v	ムーブ元のベクトル.
-----	------------

呼び出し関係図:



7.18.2.7 ~GgVector()

```
gg::GgVector::~~GgVector () [default]
```

デストラクタ.

7.18.3 関数詳解

7.18.3.1 distance3()

```
GLfloat gg::GgVector::distance3 (
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の距離.

引数

v	GgVector 型のベクトル.
-----	-------------------------

戻り値

オブジェクトと v の 3 要素の距離.

gg.h の 1883 行目に定義があります。

呼び出し関係図:



7.18.3.2 distance4()

```
GLfloat gg::GgVector::distance4 (  
    const GgVector & v) const [inline]
```

GgVector 型の 4 要素の距離.

引数

v	GgVector 型のベクトル.
---	------------------

戻り値

オブジェクトと v の 4 要素の距離.

gg.h の 1927 行目に定義があります。

呼び出し関係図:



7.18.3.3 dot3()

```
GLfloat gg::GgVector::dot3 (  
    const GgVector & v) const [inline]
```

GgVector 型の 3 要素の内積.

引数

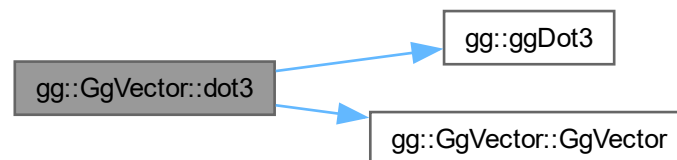
v	GgVector 型のベクトル.
---	------------------

戻り値

オブジェクトと `v` のそれぞれの 3 要素の内積.

`gg.h` の 1862 行目に定義があります。

呼び出し関係図:



7.18.3.4 dot4()

```
GLfloat gg::GgVector::dot4 (  
    const GgVector & v) const [inline]
```

`GgVector` 型の 4 要素の内積.

引数

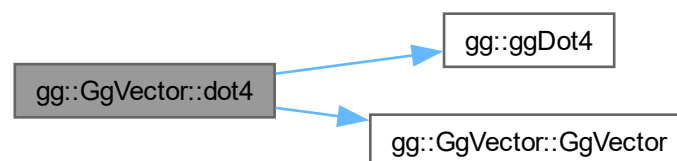
<code>v</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

オブジェクトと `v` のそれぞれの 4 要素の内積.

`gg.h` の 1906 行目に定義があります。

呼び出し関係図:



7.18.3.5 length3()

```
GLfloat gg::GgVector::length3 () const [inline]
```

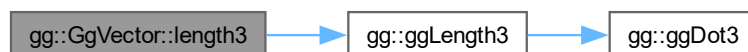
GgVector 型の 3 要素の長さ.

戻り値

オブジェクトの 3 要素の長さ.

gg.h の 1872 行目に定義があります。

呼び出し関係図:



7.18.3.6 length4()

```
GLfloat gg::GgVector::length4 () const [inline]
```

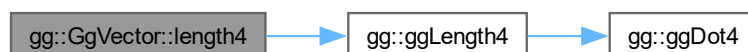
GgVector 型の 4 要素の長さ.

戻り値

オブジェクトの 4 要素の長さ.

gg.h の 1916 行目に定義があります。

呼び出し関係図:



7.18.3.7 normalize3()

`GgVector` `gg::GgVector::normalize3 () const [inline]`

`GgVector` 型の 3 要素の正規化.

戻り値

`GLfloat` 型の 4 要素の配列変数.

`gg.h` の 1893 行目に定義があります.

呼び出し関係図:



7.18.3.8 normalize4()

`GgVector` `gg::GgVector::normalize4 () const [inline]`

`GgVector` 型の 4 要素の正規化.

戻り値

`GLfloat` 型の 4 要素の配列変数.

`gg.h` の 1937 行目に定義があります.

呼び出し関係図:



7.18.3.9 operator*() [1/2]

`GgVector` `gg::GgVector::operator* (const GgVector & v) const [inline], [constexpr], [noexcept]`

`GgVector` 型の積を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの積のオブジェクト.

`gg.h` の 1758 行目に定義があります.

呼び出し関係図:



7.18.3.10 operator*() [2/2]

```
GgVector gg::GgVector::operator* (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを乗じた積を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素に `c` を乗じたオブジェクト.

`gg.h` の 1769 行目に定義があります.

呼び出し関係図:



7.18.3.11 operator*=() [1/2]

```
GgVector & gg::GgVector::operator*= (  
    const GgVector & v) [inline], [constexpr], [noexcept]
```

`GgVector` 型を乗算する.

引数

v	GgVector 型の変数.
----------	----------------

戻り値

オブジェクトの各要素に **v** の各要素をそれぞれ乗算したオブジェクトの参照.

gg.h の 1780 行目に定義があります。

呼び出し関係図:



7.18.3.12 operator*=() [2/2]

```
GgVector & gg::GgVector::operator*= (  
    GLfloat c) [inline], [constexpr], [noexcept]
```

GgVector 型の各要素にスカラーを乗じる.

引数

c	GLfloat 型の値.
----------	--------------

戻り値

オブジェクトの各要素に **c** を乗じたオブジェクトの参照.

gg.h の 1795 行目に定義があります。

呼び出し関係図:



7.18.3.13 operator+() [1/2]

```
GgVector gg::GgVector::operator+ (  
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の和を返す.

引数

<code>v</code>	<code>GgVector</code> 型のベクトル.
----------------	-------------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの和のオブジェクト.

`gg.h` の 1654 行目に定義があります.

呼び出し関係図:



7.18.3.14 operator+() [2/2]

```
GgVector gg::GgVector::operator+ (  
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを足した和を返す.

引数

<code>c</code>	GLfloat 型の値.
----------------	--------------

戻り値

`a` の各要素に `b` を足した和のオブジェクト.

`gg.h` の 1665 行目に定義があります.

呼び出し関係図:



7.18.3.15 operator+=() [1/2]

```
GgVector & gg::GgVector::operator+= (  
    const GgVector & v) [inline], [constexpr], [noexcept]
```

`GgVector` 型を加算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ加算したオブジェクトの参照.

`gg.h` の 1676 行目に定義があります.

呼び出し関係図:



7.18.3.16 operator+=() [2/2]

```
GgVector & gg::GgVector::operator+= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素にスカラーを加算する.

引数

<code>c</code>	<code>GLfloat</code> 型の値.
----------------	---------------------------

戻り値

オブジェクトの各要素に `c` を足したオブジェクトの参照.

`gg.h` の 1691 行目に定義があります.

呼び出し関係図:



7.18.3.17 operator-() [1/2]

```
GgVector gg::GgVector::operator- (
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の差を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素と `v` の各要素の要素ごとの差のオブジェクト.

`gg.h` の 1706 行目に定義があります.

呼び出し関係図:



7.18.3.18 operator-() [2/2]

```
GgVector gg::GgVector::operator- (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素からスカラーを引いた差を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素から `c` を引いたオブジェクト.

`gg.h` の 1717 行目に定義があります.

呼び出し関係図:



7.18.3.19 operator-=() [1/2]

```
GgVector & gg::GgVector::operator-= (  
    const GgVector & v) [inline], [constexpr], [noexcept]
```

`GgVector` 型を減算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素に `v` の各要素をそれぞれ減算したオブジェクトの参照.

`gg.h` の 1728 行目に定義があります.

呼び出し関係図:



7.18.3.20 operator-=() [2/2]

```
GgVector & gg::GgVector::operator-= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素からスカラーを引く.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素から `c` を引いたオブジェクトの参照.

`gg.h` の 1743 行目に定義があります.

呼び出し関係図:



7.18.3.21 operator/() [1/2]

```
GgVector gg::GgVector::operator/ (
    const GgVector & v) const [inline], [constexpr], [noexcept]
```

GgVector 型の商を返す.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素を `v` の各要素で要素ごとに割った結果のオブジェクト.

`gg.h` の 1810 行目に定義があります.

呼び出し関係図:



7.18.3.22 operator/() [2/2]

```
GgVector gg::GgVector::operator/ (
    GLfloat c) const [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素をスカラーで割った商を返す.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素を `c` で割ったオブジェクト.

`gg.h` の 1821 行目に定義があります.

呼び出し関係図:



7.18.3.23 operator/=() [1/2]

```
GgVector & gg::GgVector::operator/= (  
    const GgVector & v) [inline], [constexpr], [noexcept]
```

`GgVector` 型を除算する.

引数

<code>v</code>	<code>GgVector</code> 型の変数.
----------------	-----------------------------

戻り値

オブジェクトの各要素を `v` の各要素でそれぞれ除算したオブジェクトの参照.

`gg.h` の 1832 行目に定義があります.

呼び出し関係図:



7.18.3.24 operator/=() [2/2]

```
GgVector & gg::GgVector::operator/= (
    GLfloat c) [inline], [constexpr], [noexcept]
```

`GgVector` 型の各要素をスカラーで割る.

引数

<code>c</code>	<code>GLfloat</code> 型の変数.
----------------	----------------------------

戻り値

オブジェクトの各要素から `c` で割ったオブジェクトの参照.

`gg.h` の 1847 行目に定義があります.

呼び出し関係図:



7.18.3.25 operator=() [1/2]

```
GgVector & gg::GgVector::operator= (  
    const GgVector & v) [default]
```

代入演算子. 呼び出し関係図:



7.18.3.26 operator=() [2/2]

```
GgVector & gg::GgVector::operator= (  
    GgVector && v) [default]
```

ムーブ代入演算子. 呼び出し関係図:



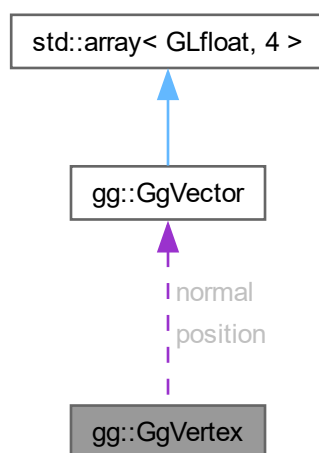
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

7.19 gg::GgVertex 構造体

```
#include <gg.h>
```

gg::GgVertex 連携図



公開メンバ関数

- `GgVertex ()`
- `GgVertex (const GgVector &pos, const GgVector &norm)`
- `GgVertex (GLfloat px, GLfloat py, GLfloat pz, GLfloat nx, GLfloat ny, GLfloat nz)`
- `GgVertex (const GLfloat *pos, const GLfloat *norm)`

公開変数類

- `GgVector position`
位置.
- `GgVector normal`
法線.

7.19.1 詳解

三角形の頂点データ.

`gg.h` の 6731 行目に定義があります。

7.19.2 構築子と解体子

7.19.2.1 GgVertex() [1/4]

```
gg::GgVertex::GgVertex () [inline]
```

コンストラクタ.

[gg.h](#) の 6742 行目に定義があります。

被呼び出し関係図:



7.19.2.2 GgVertex() [2/4]

```
gg::GgVertex::GgVertex (
    const GgVector & pos,
    const GgVector & norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	GgVector 型の位置データ.
<i>norm</i>	GgVector 型の法線データ.

[gg.h](#) の 6752 行目に定義があります。

7.19.2.3 GgVertex() [3/4]

```
gg::GgVertex::GgVertex (
    GLfloat px,
    GLfloat py,
    GLfloat pz,
    GLfloat nx,
    GLfloat ny,
    GLfloat nz) [inline]
```

コンストラクタ.

引数

<i>px</i>	GgVector 型の位置データの x 成分.
<i>py</i>	GgVector 型の位置データの y 成分.
<i>pz</i>	GgVector 型の位置データの z 成分.
<i>nx</i>	GgVector 型の法線データの x 成分.
<i>ny</i>	GgVector 型の法線データの y 成分.
<i>nz</i>	GgVector 型の法線データの z 成分.

gg.h の 6768 行目に定義があります。

7.19.2.4 GgVertex() [4/4]

```
gg::GgVertex::GgVertex (
    const GLfloat * pos,
    const GLfloat * norm) [inline]
```

コンストラクタ.

引数

<i>pos</i>	3 要素の GLfloat 型の位置データのポインタ.
<i>norm</i>	3 要素の GLfloat 型の法線データのポインタ.

gg.h の 6783 行目に定義があります。

呼び出し関係図:



7.19.3 メンバ詳解

7.19.3.1 normal

GgVector gg::GgVertex::normal

法線.

gg.h の 6737 行目に定義があります。

7.19.3.2 position

`GgVector gg::GgVertex::position`

位置.

`gg.h` の 6734 行目に定義があります。

この構造体詳解は次のファイルから抽出されました:

- `gg.h`

7.20 gg::GgVertexArray クラス

```
#include <gg.h>
```

公開メンバ関数

- `GgVertexArray` (`GLenum mode=0`)
- `GgVertexArray` (`const GgVertexArray &array`)=`delete`
- `GgVertexArray` (`GgVertexArray &&o`) `noexcept`
- `virtual ~GgVertexArray` ()
- `GgVertexArray & operator=` (`const GgVertexArray &array`)=`delete`
- `GgVertexArray & operator=` (`GgVertexArray &&o`) `noexcept`
- `const GLuint & get` () `const`
- `void bind` () `const`

7.20.1 詳解

頂点配列 クラス.

`gg.h` の 6412 行目に定義があります。

7.20.2 構築子と解体子

7.20.2.1 GgVertexArray() [1/3]

```
gg::GgVertexArray::GgVertexArray (
    GLenum mode = 0) [inline]
```

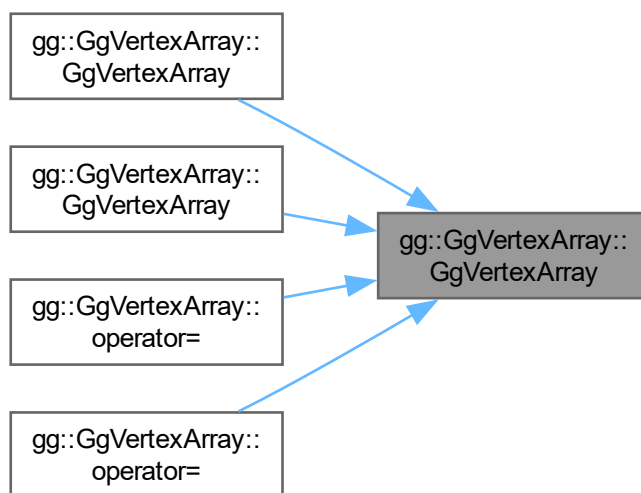
コンストラクタ.

引数

<code>mode</code>	基本図形の種類.
-------------------	----------

`gg.h` の 6424 行目に定義があります。

被呼び出し関係図:



7.20.2.2 GgVertexArray() [2/3]

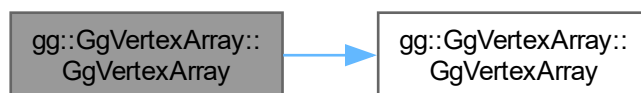
```
gg::GgVertexArray::GgVertexArray (
    const GgVertexArray & array) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>array</i>	コピー元の頂点配列オブジェクト.
--------------	------------------

呼び出し関係図:



7.20.2.3 GgVertexArray() [3/3]

```
gg::GgVertexArray::GgVertexArray (  
    GgVertexArray && o) [inline], [noexcept]
```

ムーブコンストラクタ.

引数

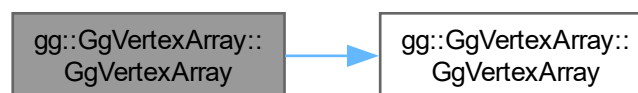
o	ムーブ元の頂点配列オブジェクト.
---	------------------

覚え書き

頂点配列オブジェクト名の所有権を移動し, ムーブ元の頂点配列オブジェクト名を 0 にする. ムーブ元のデストラクタが同じ頂点配列オブジェクトを削除しないようにするため.

gg.h の 6447 行目に定義があります。

呼び出し関係図:



7.20.2.4 ~GgVertexArray()

```
virtual gg::GgVertexArray::~~GgVertexArray () [inline], [virtual]
```

デストラクタ.

gg.h の 6456 行目に定義があります。

7.20.3 関数詳解

7.20.3.1 bind()

```
void gg::GgVertexArray::bind () const [inline]
```

頂点配列オブジェクトを結合する.

gg.h の 6510 行目に定義があります。

7.20.3.2 get()

```
const GLuint & gg::GgVertexArray::get () const [inline]
```

頂点配列オブジェクト名を取り出す.

戻り値

頂点配列オブジェクト名.

gg.h の 6502 行目に定義があります。

7.20.3.3 operator=() [1/2]

```
GgVertexArray & gg::GgVertexArray::operator= (  
    const GgVertexArray & array) [delete]
```

代入演算子は使用しない.

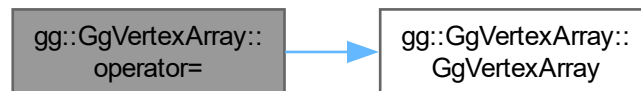
引数

<code>array</code>	代入元の頂点配列オブジェクト.
--------------------	-----------------

戻り値

代入後のこの頂点配列オブジェクトの参照.

呼び出し関係図:



7.20.3.4 operator=() [2/2]

```
GgVertexArray & gg::GgVertexArray::operator= (
    GgVertexArray && o) [inline], [noexcept]
```

ムーブ代入演算子.

引数

<code>o</code>	ムーブ代入元の頂点配列オブジェクト.
----------------	--------------------

戻り値

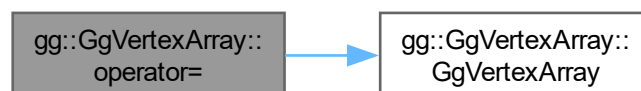
ムーブ代入後のこの頂点配列オブジェクトの参照.

覚え書き

元々保持していた頂点配列オブジェクトを削除してから所有権を移動し, ムーブ代入元の頂点配列オブジェクト名を 0 にする.

gg.h の 6481 行目に定義があります。

呼び出し関係図:



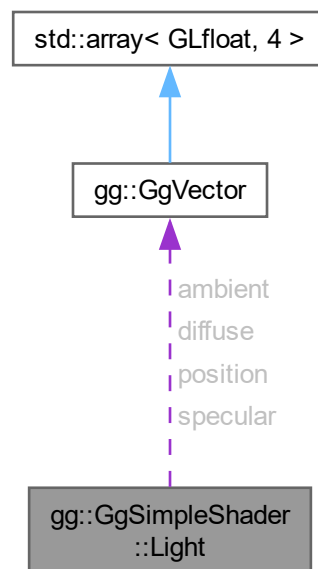
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)

7.21 gg::GgSimpleShader::Light 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Light 連携図



公開変数類

- [GgVector ambient](#)
光源強度の環境光成分.
- [GgVector diffuse](#)
光源強度の拡散反射光成分.
- [GgVector specular](#)
光源強度の鏡面反射光成分.
- [GgVector position](#)
光源の位置.

7.21.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データ.

[gg.h](#) の 7755 行目に定義があります。

7.21.2 メンバ詳解

7.21.2.1 ambient

`GgVector gg::GgSimpleShader::Light::ambient`

光源強度の環境光成分.

[gg.h](#) の 7757 行目に定義があります。

7.21.2.2 diffuse

`GgVector gg::GgSimpleShader::Light::diffuse`

光源強度の拡散反射光成分.

[gg.h](#) の 7758 行目に定義があります。

7.21.2.3 position

`GgVector gg::GgSimpleShader::Light::position`

光源の位置.

[gg.h](#) の 7760 行目に定義があります。

7.21.2.4 specular

`GgVector gg::GgSimpleShader::Light::specular`

光源強度の鏡面反射光成分.

[gg.h](#) の 7759 行目に定義があります。

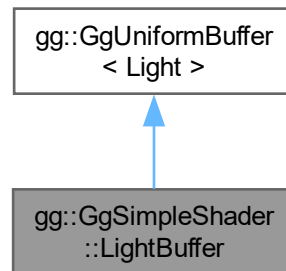
この構造体詳解は次のファイルから抽出されました:

- [gg.h](#)

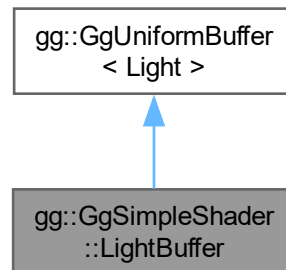
7.22 gg::GgSimpleShader::LightBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::LightBuffer の継承関係図



gg::GgSimpleShader::LightBuffer 連携図



公開メンバ関数

- `LightBuffer` (const `Light` *light=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `LightBuffer` (const `Light` &light, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `LightBuffer` (`GgVector` ambient, `GgVector` diffuse, `GgVector` specular, `GgVector` position, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual `~LightBuffer` ()
- void `loadAmbient` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const `GgVector` &ambient, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const `GgVector` &diffuse, GLint first=0, GLsizei count=1) const

- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GgVector &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadColor` (const Light &color, GLint first=0, GLsizei count=1) const
- void `loadPosition` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GgVector &position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GLfloat *position, GLint first=0, GLsizei count=1) const
- void `loadPosition` (const GgVector *position, GLint first=0, GLsizei count=1) const
- void `load` (const Light *light, GLint first=0, GLsizei count=1) const
- void `load` (const Light &light, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Light >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const Light *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Light), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

7.22.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する光源データのユニフォームバッファオブジェクト。

`gg.h` の 7766 行目に定義があります。

7.22.2 構築子と解体子

7.22.2.1 LightBuffer() [1/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light * light = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

デフォルトコンストラクタ。

引数

<i>light</i>	GgSimpleShader::Light 型の光源データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7778 行目に定義があります。

呼び出し関係図:



7.22.2.2 LightBuffer() [2/3]

```

gg::GgSimpleShader::LightBuffer::LightBuffer (
    const Light & light,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

同じデータで埋めるコンストラクタ.

引数

<i>light</i>	GgSimpleShader::Light 型の光源データ.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 7794 行目に定義があります。

呼び出し関係図:



7.22.2.3 LightBuffer() [3/3]

```
gg::GgSimpleShader::LightBuffer::LightBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GgVector position,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

同じ引数で埋めるコンストラクタ。

引数

<i>ambient</i>	光源強度の環境光成分.
<i>diffuse</i>	光源強度の拡散反射光成分.
<i>specular</i>	光源強度の鏡面反射光成分.
<i>position</i>	光源の位置.
<i>count</i>	バッファ中の GgSimpleShader::Light 型の光源データの数.
<i>usage</i>	バッファの使い方のパターン, glBufferData() の第 4 引数の <i>usage</i> に渡される.

[gg.h](#) の 7813 行目に定義があります。

呼び出し関係図:



7.22.2.4 ~LightBuffer()

```
virtual gg::GgSimpleShader::LightBuffer::~~LightBuffer () [inline], [virtual]
```

デストラクタ。

[gg.h](#) の 7828 行目に定義があります。

7.22.3 関数詳解

7.22.3.1 load() [1/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light & light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

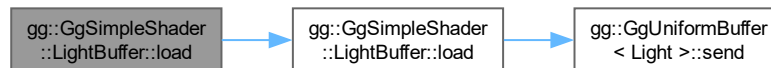
光源の色と位置を設定する。

引数

<i>light</i>	光源の特性の GgSimpleShader::Light 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 8020 行目に定義があります。

呼び出し関係図:



7.22.3.2 load() [2/2]

```
void gg::GgSimpleShader::LightBuffer::load (
    const Light * light,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の色と位置を設定する。

引数

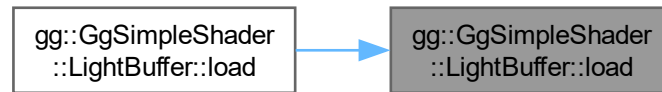
<i>light</i>	光源の特性の GgSimpleShader::Light 構造体のポインタ.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 8008 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.22.3.3 loadAmbient() [1/3]

```

void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
  
```

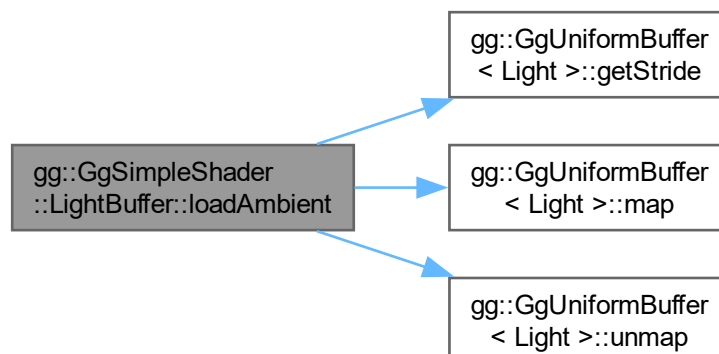
光源の強度の環境光成分を設定する.

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5503 行目に定義があります。

呼び出し関係図:



7.22.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の環境光成分を設定する。

引数

<i>ambient</i>	光源の強度の環境光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7863 行目に定義があります。

呼び出し関係図:



7.22.3.5 loadAmbient() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

光源の強度の環境光成分を設定する。

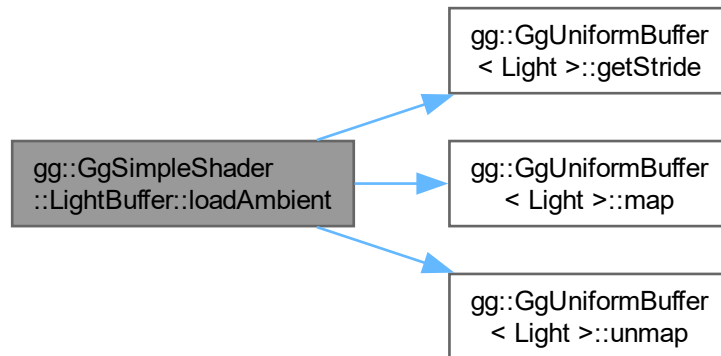
引数

<i>r</i>	光源の強度の環境光成分の赤成分.
<i>g</i>	光源の強度の環境光成分の緑成分.
<i>b</i>	光源の強度の環境光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.

<i>count</i>	値を設定する光源データの数, デフォルトは 1.
--------------	--------------------------

gg.cpp の 5477 行目に定義があります。

呼び出し関係図:



7.22.3.6 loadColor()

```
void gg::GgSimpleShader::LightBuffer::loadColor (
    const Light & color,
    GLint first = 0,
    GLsizei count = 1) const
```

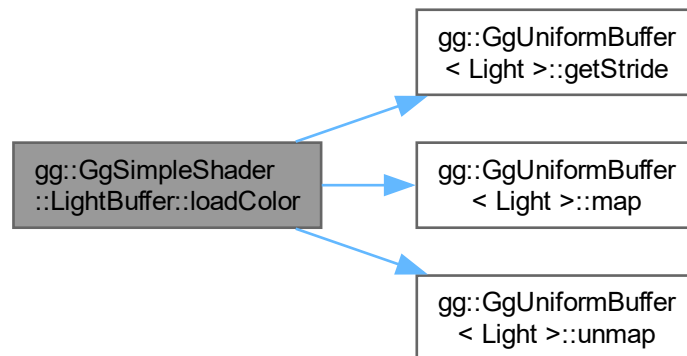
光源の色を設定するが位置は変更しない。

引数

<i>color</i>	光源の特性の <code>GgSimpleShader::Light</code> 構造体.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5630 行目に定義があります。

呼び出し関係図:



7.22.3.7 loadDiffuse() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (  
    const GgVector & diffuse,  
    GLint first = 0,  
    GLsizei count = 1) const
```

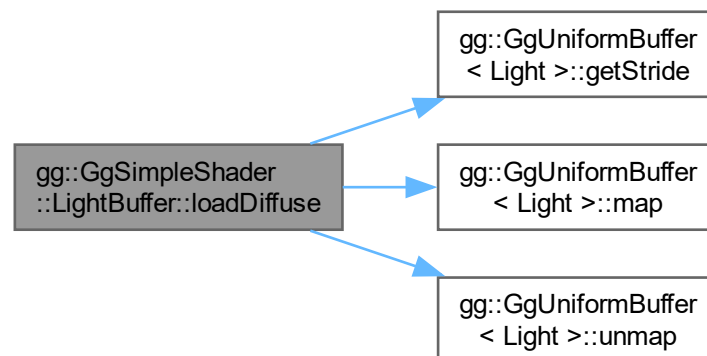
光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

`gg.cpp` の 5555 行目に定義があります。

呼び出し関係図:



7.22.3.8 loadDiffuse() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の拡散反射光成分を設定する.

引数

<i>diffuse</i>	光源の強度の拡散反射光成分を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7900 行目に定義があります。

呼び出し関係図:



7.22.3.9 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::LightBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

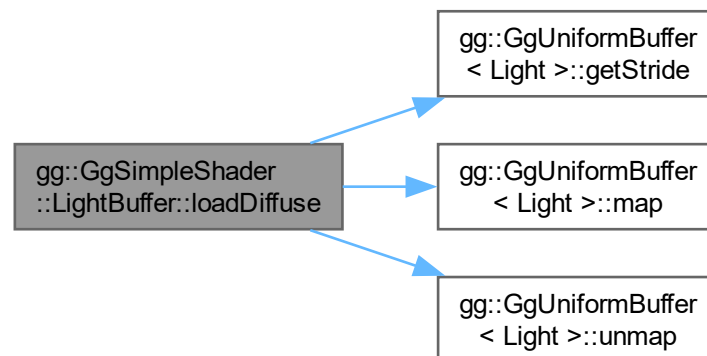
光源の強度の拡散反射光成分を設定する。

引数

<i>r</i>	光源の強度の拡散反射光成分の赤成分.
<i>g</i>	光源の強度の拡散反射光成分の緑成分.
<i>b</i>	光源の強度の拡散反射光成分の青成分.
<i>a</i>	光源の強度の拡散反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5529 行目に定義があります。

呼び出し関係図:



7.22.3.10 loadPosition() [1/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector & position,
    GLint first = 0,
    GLsizei count = 1) const
```

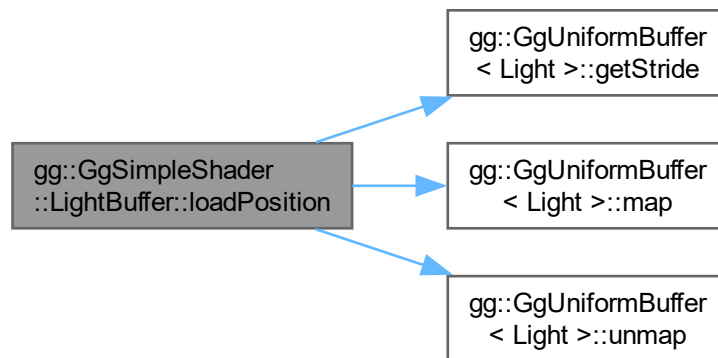
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.cpp](#) の 5684 行目に定義があります。

呼び出し関係図:



7.22.3.11 loadPosition() [2/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GgVector * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

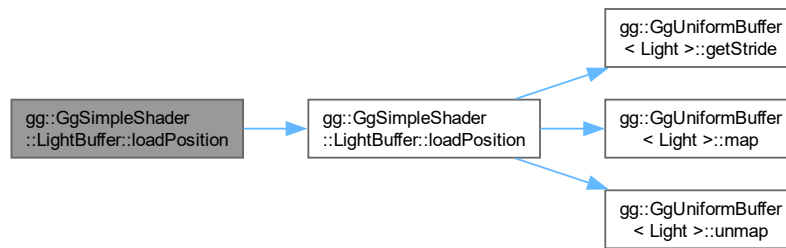
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GgVector 型の配列.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7996 行目に定義があります。

呼び出し関係図:



7.22.3.12 loadPosition() [3/4]

```

void gg::GgSimpleShader::LightBuffer::loadPosition (
    const GLfloat * position,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

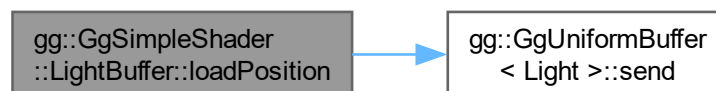
光源の位置を設定する。

引数

<i>position</i>	光源の位置の同次座標を格納した GLfloat 型の 4 要素の配列変数。
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

[gg.h](#) の 7983 行目に定義があります。

呼び出し関係図:



7.22.3.13 loadPosition() [4/4]

```
void gg::GgSimpleShader::LightBuffer::loadPosition (
    GLfloat x,
    GLfloat y,
    GLfloat z,
    GLfloat w = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

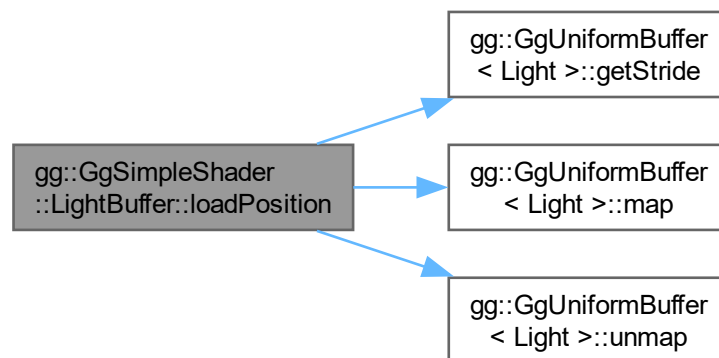
光源の位置を設定する。

引数

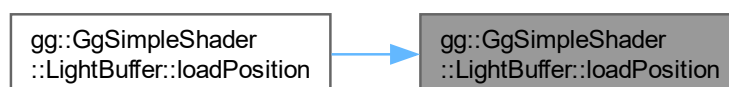
<i>x</i>	光源の位置の <i>x</i> 座標.
<i>y</i>	光源の位置の <i>y</i> 座標.
<i>z</i>	光源の位置の <i>z</i> 座標.
<i>w</i>	光源の位置の <i>w</i> 座標, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5658 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.22.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

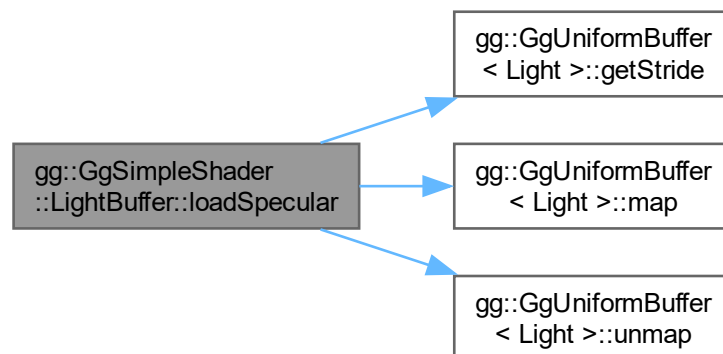
光源の強度の鏡面反射光成分を設定する。

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GgVector 型の変数.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5607 行目に定義があります。

呼び出し関係図:



7.22.3.15 loadSpecular() [2/3]

```
void gg::GgSimpleShader::LightBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

光源の強度の鏡面反射光成分を設定する。

引数

<i>specular</i>	光源の強度の鏡面反射光成分を格納した GLfloat 型の 4 要素の配列変数.
-----------------	---

<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.h の 7937 行目に定義があります。

呼び出し関係図:



7.22.3.16 loadSpecular() [3/3]

```

void gg::GgSimpleShader::LightBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

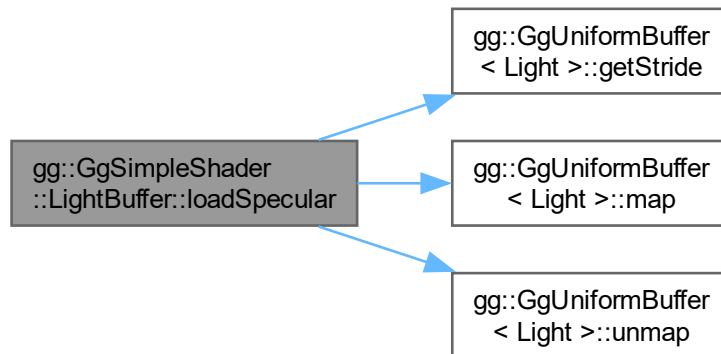
光源の強度の鏡面反射光成分を設定する。

引数

<i>r</i>	光源の強度の鏡面反射光成分の赤成分.
<i>g</i>	光源の強度の鏡面反射光成分の緑成分.
<i>b</i>	光源の強度の鏡面反射光成分の青成分.
<i>a</i>	光源の強度の鏡面反射光成分の不透明度, デフォルトは 1.
<i>first</i>	値を設定する光源データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する光源データの数, デフォルトは 1.

gg.cpp の 5581 行目に定義があります。

呼び出し関係図:



7.22.3.17 select()

```
void gg::GgSimpleShader::LightBuffer::select (
    GLint i = 0) const [inline]
```

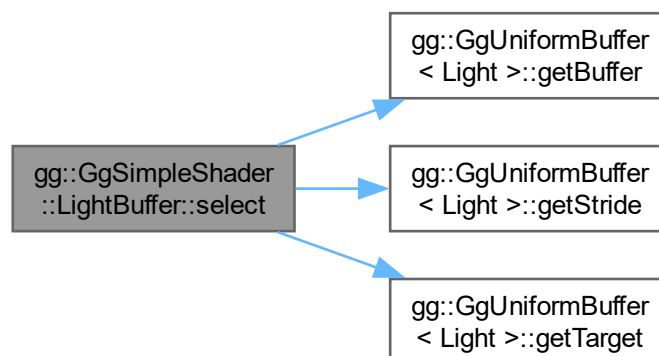
光源を選択する.

引数

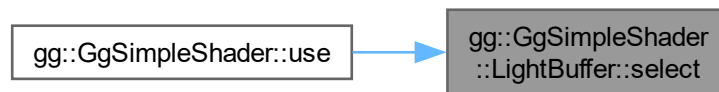
<i>i</i>	光源データの uniform block のインデックス.
----------	-------------------------------

[gg.h](#) の 8030 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



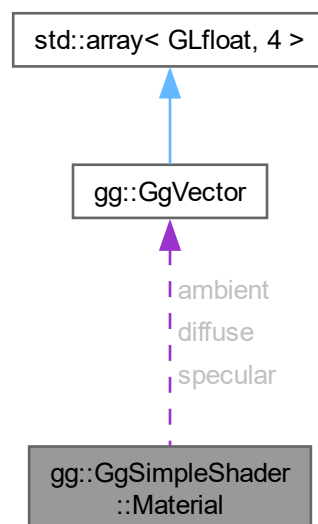
このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

7.23 gg::GgSimpleShader::Material 構造体

```
#include <gg.h>
```

gg::GgSimpleShader::Material 連携図



公開変数類

- [GgVector ambient](#)
環境光に対する反射係数.
- [GgVector diffuse](#)
拡散反射係数.
- [GgVector specular](#)
鏡面反射係数.
- [GLfloat shininess](#)
輝き係数.

7.23.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データ.

[gg.h](#) の 8041 行目に定義があります。

7.23.2 メンバ詳解

7.23.2.1 ambient

```
GgVector gg::GgSimpleShader::Material::ambient
```

環境光に対する反射係数.

[gg.h](#) の 8043 行目に定義があります。

7.23.2.2 diffuse

```
GgVector gg::GgSimpleShader::Material::diffuse
```

拡散反射係数.

[gg.h](#) の 8044 行目に定義があります。

7.23.2.3 shininess

```
GLfloat gg::GgSimpleShader::Material::shininess
```

輝き係数.

[gg.h](#) の 8046 行目に定義があります。

7.23.2.4 specular

GgVector gg::GgSimpleShader::Material::specular

鏡面反射係数.

gg.h の 8045 行目に定義があります。

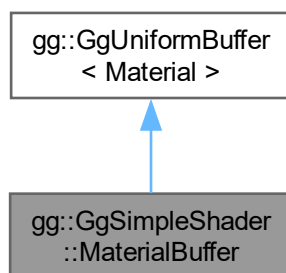
この構造体詳解は次のファイルから抽出されました:

- gg.h

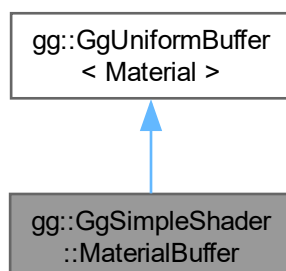
7.24 gg::GgSimpleShader::MaterialBuffer クラス

```
#include <gg.h>
```

gg::GgSimpleShader::MaterialBuffer の継承関係図



gg::GgSimpleShader::MaterialBuffer 連携図



公開メンバ関数

- `MaterialBuffer` (const `Material` *material=nullptr, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `MaterialBuffer` (const `Material` &material, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- `MaterialBuffer` (`GgVector` ambient, `GgVector` diffuse, `GgVector` specular, GLfloat shininess, GLsizei count=1, GLenum usage=GL_STATIC_DRAW)
- virtual `~MaterialBuffer` ()
- void `loadAmbient` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const `GgVector` &ambient, GLint first=0, GLsizei count=1) const
- void `loadAmbient` (const GLfloat *ambient, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const `GgVector` &diffuse, GLint first=0, GLsizei count=1) const
- void `loadDiffuse` (const GLfloat *diffuse, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const `GgVector` &color, GLint first=0, GLsizei count=1) const
- void `loadAmbientAndDiffuse` (const GLfloat *color, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (GLfloat r, GLfloat g, GLfloat b, GLfloat a=1.0f, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const `GgVector` &specular, GLint first=0, GLsizei count=1) const
- void `loadSpecular` (const GLfloat *specular, GLint first=0, GLsizei count=1) const
- void `loadShininess` (GLfloat shininess, GLint first=0, GLsizei count=1) const
- void `loadShininess` (const GLfloat *shininess, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` *material, GLint first=0, GLsizei count=1) const
- void `load` (const `Material` &material, GLint first=0, GLsizei count=1) const
- void `select` (GLint i=0) const

基底クラス `gg::GgUniformBuffer< Material >` に属する継承公開メンバ関数

- `GgUniformBuffer` ()
- virtual `~GgUniformBuffer` ()
- const GLuint & `getTarget` () const
- GLsizeiptr `getStride` () const
- const GLsizei & `getCount` () const
- const GLuint & `getBuffer` () const
- void `bind` () const
- void `unbind` () const
- void * `map` () const
- void `unmap` () const
- void `load` (const `Material` *data, GLsizei count, GLenum usage=GL_STATIC_DRAW)
- void `send` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `fill` (const GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `read` (GLvoid *data, GLint offset=0, GLsizei size=sizeof(Material), GLint first=0, GLsizei count=0) const
- void `copy` (GLuint src_buffer, GLint src_first=0, GLint dst_first=0, GLsizei count=0) const

7.24.1 詳解

三角形に単純な陰影付けを行うシェーダが参照する材質データのユニフォームバッファオブジェクト。

`gg.h` の 8052 行目に定義があります。

7.24.2 構築子と解体子

7.24.2.1 MaterialBuffer() [1/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material * material = nullptr,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

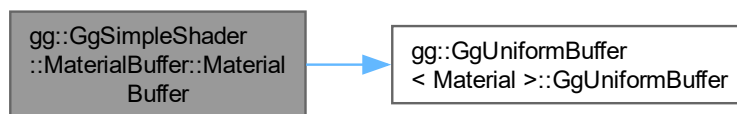
デフォルトコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データのポインタ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <i>usage</i> に渡される.

[gg.h](#) の 8064 行目に定義があります.

呼び出し関係図:



7.24.2.2 MaterialBuffer() [2/3]

```
gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    const Material & material,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
```

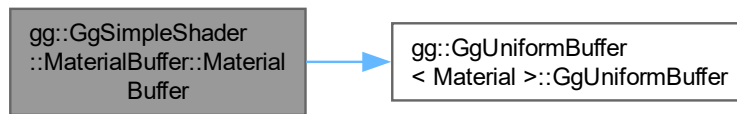
同じデータで埋めるコンストラクタ.

引数

<i>material</i>	GgSimpleShader::Material 型の材質データ.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <i>usage</i> に渡される.

[gg.h](#) の 8080 行目に定義があります.

呼び出し関係図:



7.24.2.3 MaterialBuffer() [3/3]

```

gg::GgSimpleShader::MaterialBuffer::MaterialBuffer (
    GgVector ambient,
    GgVector diffuse,
    GgVector specular,
    GLfloat shininess,
    GLsizei count = 1,
    GLenum usage = GL_STATIC_DRAW) [inline]
  
```

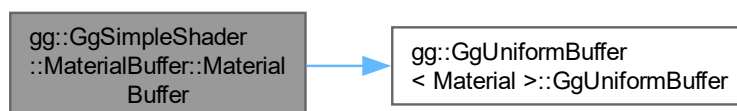
同じ引数で埋めるコンストラクタ.

引数

<i>ambient</i>	環境光に対する反射係数.
<i>diffuse</i>	拡散反射係数.
<i>specular</i>	鏡面反射係数.
<i>shininess</i>	輝き係数.
<i>count</i>	バッファ中の GgSimpleShader::Material 型の材質データの数.
<i>usage</i>	バッファの使い方のパターン, <code>glBufferData()</code> の第 4 引数の <code>usage</code> に渡される.

[gg.h](#) の 8099 行目に定義があります。

呼び出し関係図:



7.24.2.4 ~MaterialBuffer()

```
virtual gg::GgSimpleShader::MaterialBuffer::~MaterialBuffer () [inline], [virtual]
```

デストラクタ.

gg.h の 8114 行目に定義があります。

7.24.3 関数詳解

7.24.3.1 load() [1/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material & material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

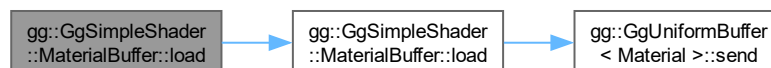
材質を設定する.

引数

<i>material</i>	材質の特性の GgSimpleShader::Material 構造体.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 8299 行目に定義があります。

呼び出し関係図:



7.24.3.2 load() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::load (
    const Material * material,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

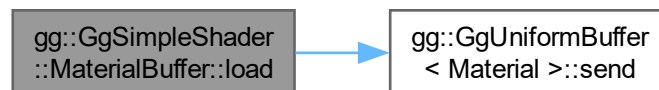
材質を設定する.

引数

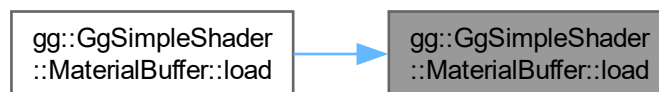
<i>material</i>	材質の特性の GgSimpleShader::Material 構造体のポインタ.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8287 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.24.3.3 loadAmbient() [1/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GgVector & ambient,
    GLint first = 0,
    GLsizei count = 1) const
  
```

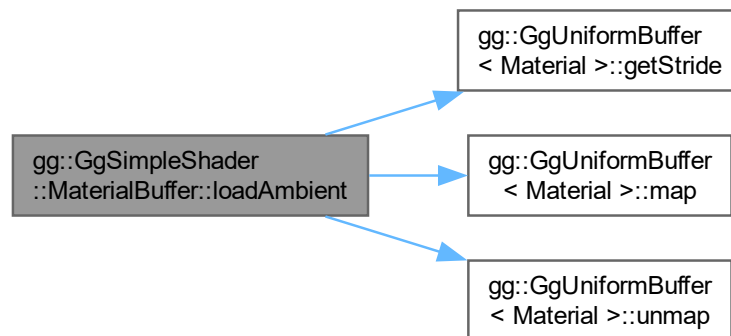
三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する。

引数

<i>ambient</i>	環境光に対する反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5736 行目に定義があります。

呼び出し関係図:



7.24.3.4 loadAmbient() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    const GLfloat * ambient,
    GLint first = 0,
    GLsizei count = 1) const [inline]
```

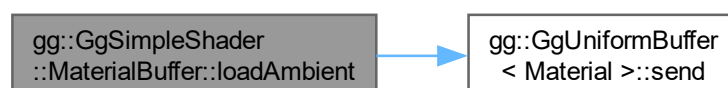
環境光に対する反射係数を設定する。

引数

<i>ambient</i>	環境光に対する反射係数を格納した GLfloat 型の 4 要素の配列変数。
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 8149 行目に定義があります。

呼び出し関係図:



7.24.3.5 loadAmbient() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbient (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

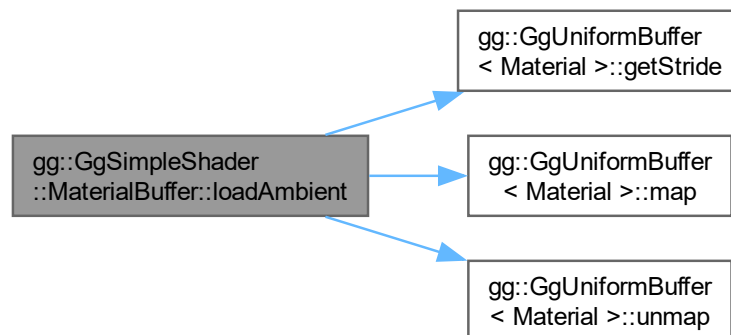
環境光に対する反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数の赤成分.
<i>g</i>	環境光に対する反射係数の緑成分.
<i>b</i>	環境光に対する反射係数の青成分.
<i>a</i>	環境光に対する反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5710 行目に定義があります。

呼び出し関係図:



7.24.3.6 loadAmbientAndDiffuse() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GgVector & color,
    GLint first = 0,
    GLsizei count = 1) const
```

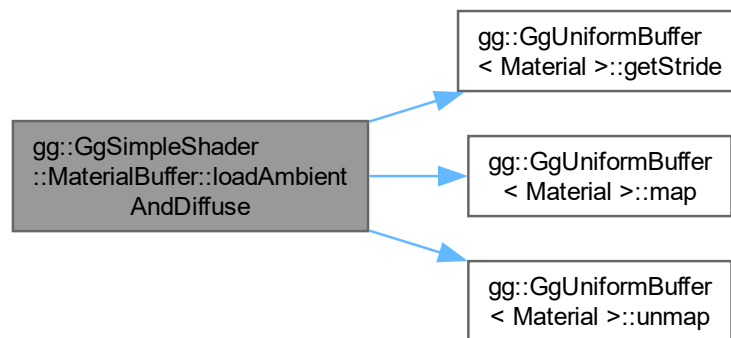
三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5840 行目に定義があります。

呼び出し関係図:



7.24.3.7 loadAmbientAndDiffuse() [2/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    const GLfloat * color,
    GLint first = 0,
    GLsizei count = 1) const
```

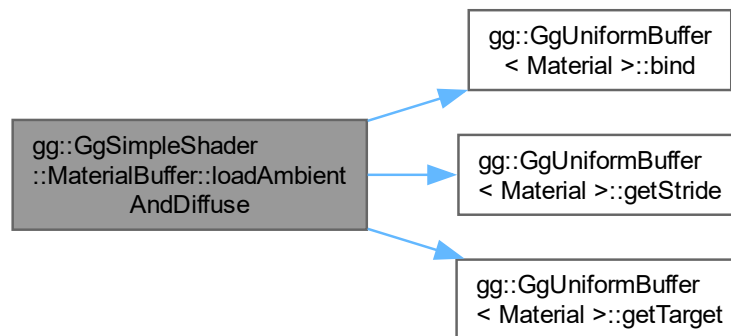
環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>color</i>	環境光に対する反射係数と拡散反射係数を格納した <code>GLfloat</code> 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5863 行目に定義があります。

呼び出し関係図:



7.24.3.8 loadAmbientAndDiffuse() [3/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
  
```

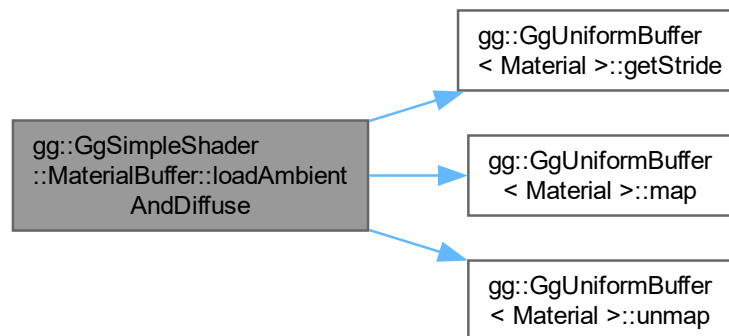
環境光に対する反射係数と拡散反射係数を設定する。

引数

<i>r</i>	環境光に対する反射係数と拡散反射係数の赤成分.
<i>g</i>	環境光に対する反射係数と拡散反射係数の緑成分.
<i>b</i>	環境光に対する反射係数と拡散反射係数の青成分.
<i>a</i>	環境光に対する反射係数と拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5814 行目に定義があります。

呼び出し関係図:



7.24.3.9 loadDiffuse() [1/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GgVector & diffuse,
    GLint first = 0,
    GLsizei count = 1) const
  
```

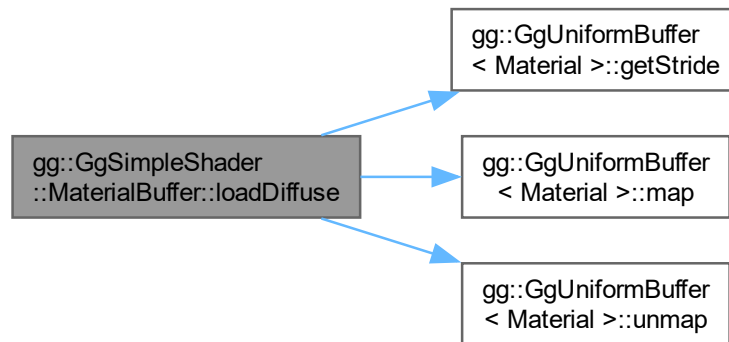
三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する。

引数

<i>diffuse</i>	拡散反射係数を格納した GgVector 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5788 行目に定義があります。

呼び出し関係図:



7.24.3.10 loadDiffuse() [2/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    const GLfloat * diffuse,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

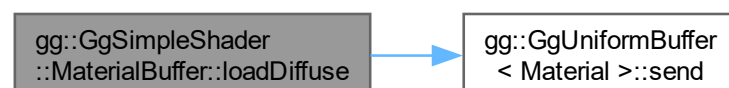
拡散反射係数を設定する.

引数

<i>diffuse</i>	拡散反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.h](#) の 8186 行目に定義があります。

呼び出し関係図:



7.24.3.11 loadDiffuse() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadDiffuse (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

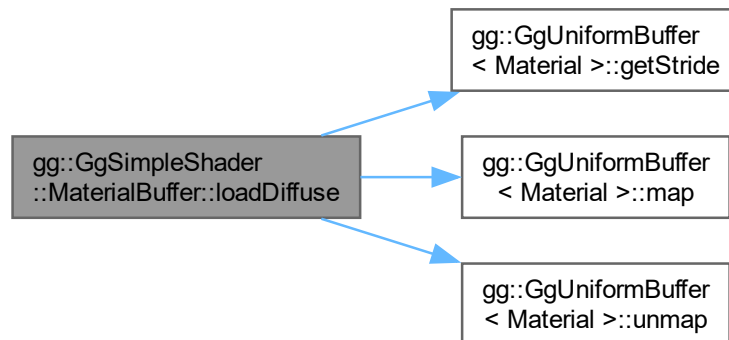
拡散反射係数を設定する。

引数

<i>r</i>	拡散反射係数の赤成分.
<i>g</i>	拡散反射係数の緑成分.
<i>b</i>	拡散反射係数の青成分.
<i>a</i>	拡散反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5762 行目に定義があります。

呼び出し関係図:



7.24.3.12 loadShininess() [1/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    const GLfloat * shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

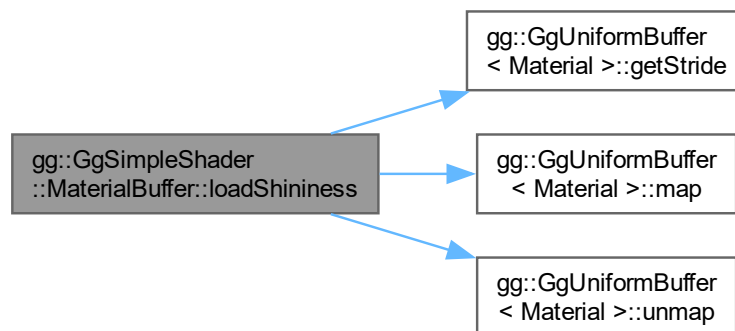
輝き係数を設定する。

引数

<i>shininess</i>	輝き 係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5976 行目に定義があります。

呼び出し関係図:



7.24.3.13 loadShininess() [2/2]

```
void gg::GgSimpleShader::MaterialBuffer::loadShininess (
    GLfloat shininess,
    GLint first = 0,
    GLsizei count = 1) const
```

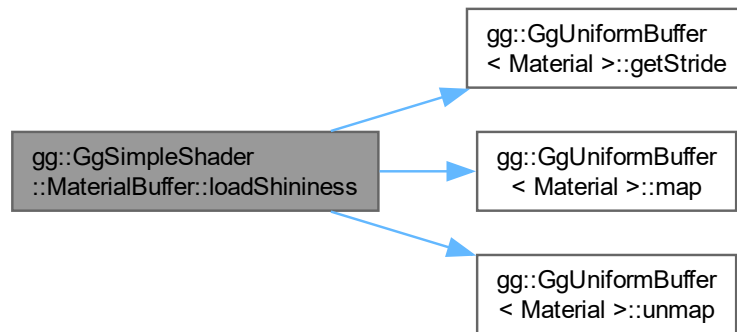
輝き係数を設定する.

引数

<i>shininess</i>	輝き 係数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

[gg.cpp](#) の 5955 行目に定義があります。

呼び出し関係図:



7.24.3.14 loadSpecular() [1/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GgVector & specular,
    GLint first = 0,
    GLsizei count = 1) const
```

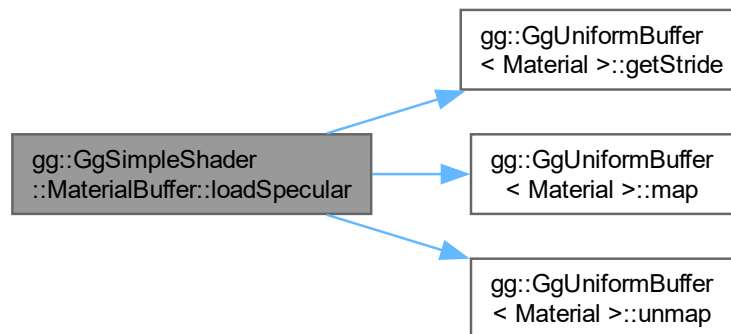
三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する。

引数

<i>specular</i>	鏡面反射係数を格納した <code>GgVector</code> 型の変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

`gg.cpp` の 5932 行目に定義があります。

呼び出し関係図:



7.24.3.15 loadSpecular() [2/3]

```

void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    const GLfloat * specular,
    GLint first = 0,
    GLsizei count = 1) const [inline]
  
```

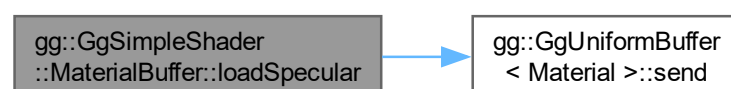
鏡面反射係数を設定する.

引数

<i>specular</i>	鏡面反射係数を格納した GLfloat 型の 4 要素の配列変数.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.h の 8256 行目に定義があります。

呼び出し関係図:



7.24.3.16 loadSpecular() [3/3]

```
void gg::GgSimpleShader::MaterialBuffer::loadSpecular (
    GLfloat r,
    GLfloat g,
    GLfloat b,
    GLfloat a = 1.0f,
    GLint first = 0,
    GLsizei count = 1) const
```

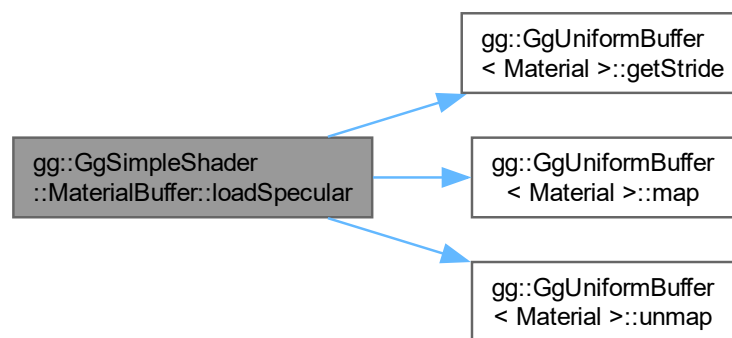
鏡面反射係数を設定する。

引数

<i>r</i>	鏡面反射係数の赤成分.
<i>g</i>	鏡面反射係数の緑成分.
<i>b</i>	鏡面反射係数の青成分.
<i>a</i>	鏡面反射係数の不透明度, デフォルトは 1.
<i>first</i>	値を設定する材質データの最初の番号, デフォルトは 0.
<i>count</i>	値を設定する材質データの数, デフォルトは 1.

gg.cpp の 5906 行目に定義があります。

呼び出し関係図:



7.24.3.17 select()

```
void gg::GgSimpleShader::MaterialBuffer::select (
    GLint i = 0) const [inline]
```

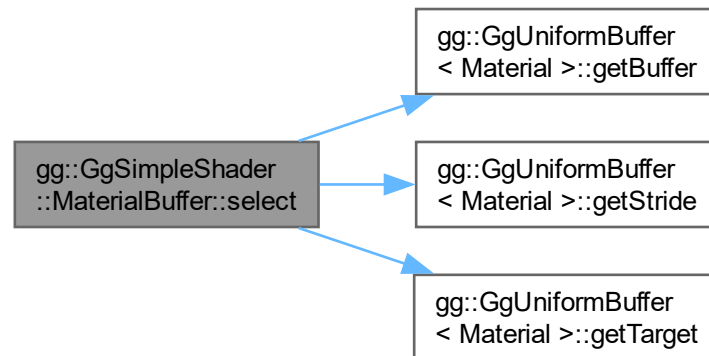
材質を選択する。

引数

<i>i</i>	材質データの uniform block のインデックス.
----------	-------------------------------

[gg.h](#) の 8309 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [gg.h](#)
- [gg.cpp](#)

7.25 GgApp::Window クラス

```
#include <GgApp.h>
```

公開メンバ関数

- [Window](#) (const std::string &title="GLFW Window", int width=640, int height=480, int fullscreen=0, GLFWwindow *share=nullptr)
- [Window](#) (const Window &w)=delete
- [Window](#) (Window &&w) noexcept
- virtual ~[Window](#) ()
- [Window](#) & operator= (const [Window](#) &w)=delete
- [Window](#) & operator= ([Window](#) &&w) noexcept
- auto * [get](#) () const
- void [setClose](#) (int flag=GLFW_TRUE) const
- bool [shouldClose](#) () const
- operator bool ()
- void [swapBuffers](#) () const
- void [restoreViewport](#) () const
- void [updateViewport](#) ()

- auto [getWidth](#) () const
- auto [getHeight](#) () const
- auto [getFboWidth](#) () const
- auto [getFboHeight](#) () const
- const auto & [getSize](#) () const
- void [getSize](#) (GLsizei *size) const
- const auto & [getFboSize](#) () const
- void [getFboSize](#) (GLsizei *fboSize) const
- auto [getAspect](#) () const
- bool [getKey](#) (int key) const
- void [selectInterface](#) (int no)
- void [setVelocity](#) (GLfloat vx, GLfloat vy, GLfloat vz=0.1f)
- int [getLastKey](#) ()
- auto [getArrow](#) (int direction=0, int mods=0) const
- auto [getArrowX](#) (int mods=0) const
- auto [getArrowY](#) (int mods=0) const
- void [getArrow](#) (GLfloat *arrow, int mods=0) const
- auto [getShiftArrowX](#) () const
- auto [getShiftArrowY](#) () const
- void [getShiftArrow](#) (GLfloat *shift_arrow) const
- auto [getControlArrowX](#) () const
- auto [getControlArrowY](#) () const
- void [getControlArrow](#) (GLfloat *control_arrow) const
- auto [getAltArrowX](#) () const
- auto [getAltArrowY](#) () const
- void [getAltArrow](#) (GLfloat *alt_arrow) const
- const auto * [getMouse](#) () const
- void [getMouse](#) (GLfloat *position) const
- auto [getMouse](#) (int direction) const
- auto [getMouseX](#) () const
- auto [getMouseY](#) () const
- const auto * [getWheel](#) () const
- void [getWheel](#) (GLfloat *rotation) const
- auto [getWheel](#) (int direction) const
- auto [getWheelX](#) () const
- auto [getWheelY](#) () const
- const auto & [getTranslation](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getTranslationMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getScrollMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getRotation](#) (int button=GLFW_MOUSE_BUTTON_1) const
- auto [getRotationMatrix](#) (int button=GLFW_MOUSE_BUTTON_1) const
- void [resetRotation](#) ()
- void [resetTranslation](#) ()
- void [reset](#) ()
- void * [getUserPointer](#) () const
- void [setUserPointer](#) (void *pointer)
- void [setResizeFunc](#) (void(*func)(const [Window](#) *window, int width, int height))
- void [setKeyboardFunc](#) (void(*func)(const [Window](#) *window, int key, int scancode, int action, int mods))
- void [setMouseFunc](#) (void(*func)(const [Window](#) *window, int button, int action, int mods))
- void [setWheelFunc](#) (void(*func)(const [Window](#) *window, double x, double y))

7.25.1 詳解

ウィンドウ関連の処理.

覚え書き

GLFW を使って OpenGL のウィンドウを操作するラッパークラス.

GgApp.h の 170 行目に定義があります。

7.25.2 構築子と解体子

7.25.2.1 Window() [1/3]

```
GgApp::Window::Window (
    const std::string & title = "GLFW Window",
    int width = 640,
    int height = 480,
    int fullscreen = 0,
    GLFWwindow * share = nullptr)
```

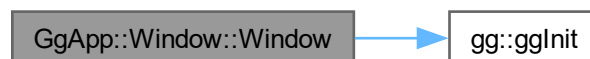
コンストラクタ.

引数

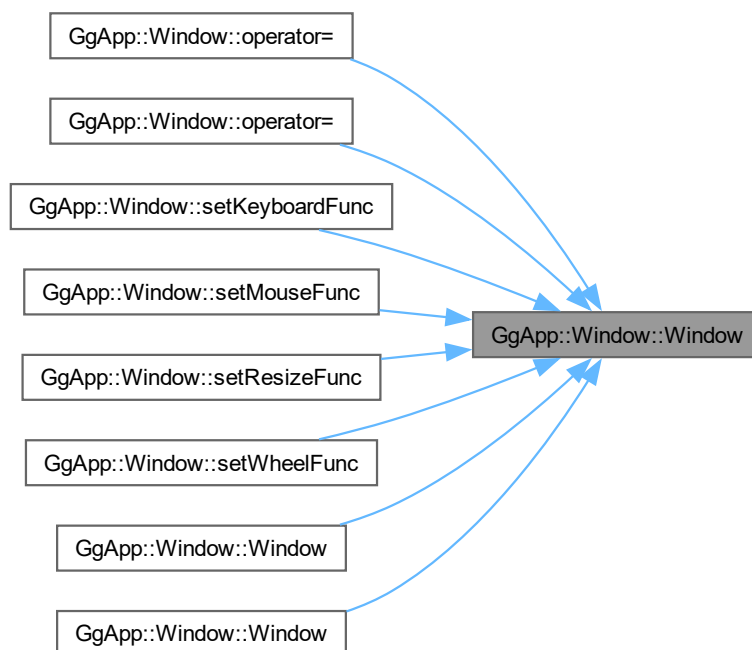
<i>title</i>	ウィンドウタイトルの文字列.
<i>width</i>	開くウィンドウの幅, フルスクリーン時は無視され実際のディスプレイの幅が使われる.
<i>height</i>	開くウィンドウの高さ, フルスクリーン時は無視され実際のディスプレイの高さが使われる.
<i>fullscreen</i>	フルスクリーン表示を行うディスプレイ番号, 0 ならフルスクリーン表示を行わない.
<i>share</i>	共有するコンテキスト, nullptr ならコンテキストを共有しない.

GgApp.cpp の 344 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.2.2 Window() [2/3]

```
GgApp::Window::Window (
    const Window & w) [delete]
```

コピーコンストラクタは使用しない.

引数

<i>w</i>	コピー元のウィンドウ.
----------	-------------

呼び出し関係図:



7.25.2.3 Window() [3/3]

```
GgApp::Window::Window (
    Window && w) [noexcept]
```

ムーブコンストラクタ.

引数

<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

[GgApp.cpp](#) の 424 行目に定義があります。

呼び出し関係図:



7.25.2.4 ~Window()

```
virtual GgApp::Window::~~Window () [inline], [virtual]
```

デストラクタ.

[GgApp.h](#) の 299 行目に定義があります。

7.25.3 関数詳解

7.25.3.1 get()

```
auto * GgApp::Window::get () const [inline]
```

ウィンドウの識別子のポインタを取得する.

戻り値

GLFWwindow 型のウィンドウ識別子のポインタ.

[GgApp.h](#) の 329 行目に定義があります。

7.25.3.2 getAltArrow()

```
void GgApp::Window::getAltArrow (
    GLfloat * alt_arrow) const [inline]
```

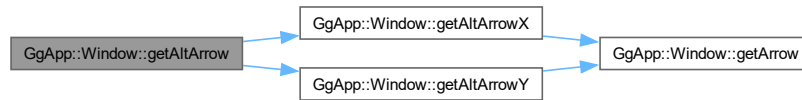
ALT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<code>alt_arrow</code>	ALT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
------------------------	---

GgApp.h の 673 行目に定義があります。

呼び出し関係図:



7.25.3.3 getAltArrowX()

```
auto GgApp::Window::getAltArrowX () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の X 値.

GgApp.h の 653 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.4 getAltArrowY()

```
auto GgApp::Window::getAltArrowY () const [inline]
```

ALT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

ALT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 663 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.5 getArrow() [1/2]

```
void GgApp::Window::getArrow (  
    GLfloat * arrow,  
    int mods = 0) const [inline]
```

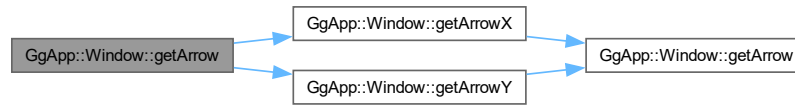
矢印キーの現在の値を得る.

引数

<i>arrow</i>	矢印キーの値を格納する GLfloat[2] の配列.
<i>mods</i>	修飾キーの状態 (0:なし, 1, SHIFT, 2: CTRL, 3: ALT).

[GgApp.h](#) の 580 行目に定義があります。

呼び出し関係図:



7.25.3.6 getArrow() [2/2]

```

auto GgApp::Window::getArrow (
    int direction = 0,
    int mods = 0) const [inline]
  
```

矢印キーの現在の値を得る.

引数

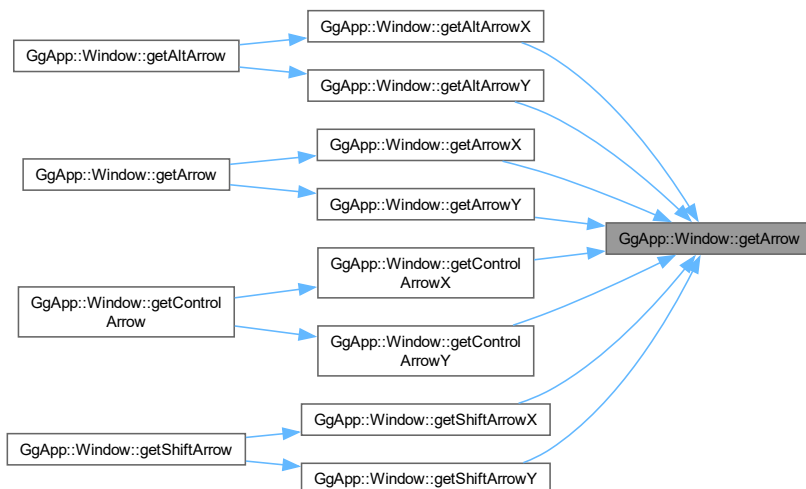
<i>direction</i>	方向 (0: X, 1:Y).
<i>mods</i>	修飾キーの状態 (0:なし, 1, SHIFT, 2: CTRL, 3: ALT).

戻り値

矢印キーの値.

[GgApp.h](#) の 546 行目に定義があります。

被呼び出し関係図:



7.25.3.7 getArrowX()

```
auto GgApp::Window::getArrowX (
    int mods = 0) const [inline]
```

矢印キーの現在の X 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの X 値.

[GgApp.h](#) の 558 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.8 getArrowY()

```
auto GgApp::Window::getArrowY (
    int mods = 0) const [inline]
```

矢印キーの現在の Y 値を得る.

引数

<i>mods</i>	修飾キーの状態 (0:なし, 1: SHIFT, 2: CTRL, 3: ALT).
-------------	--

戻り値

矢印キーの Y 値.

GgApp.h の 569 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.9 getAspect()

```
auto GgApp::Window::getAspect () const [inline]
```

ウィンドウの縦横比を得る.

戻り値

ウィンドウの縦横比.

GgApp.h の 483 行目に定義があります。

7.25.3.10 getControlArrow()

```
void GgApp::Window::getControlArrow (
    GLfloat * control_arrow) const [inline]
```

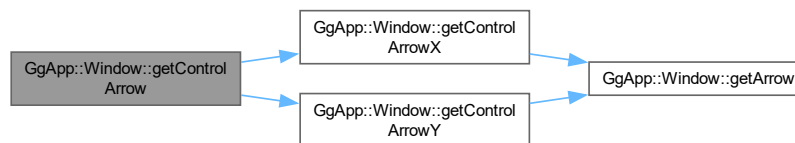
CTRL キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<code>control_arrow</code>	CTRL キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
----------------------------	--

GgApp.h の 642 行目に定義があります。

呼び出し関係図:



7.25.3.11 getControlArrowX()

```
auto GgApp::Window::getControlArrowX () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

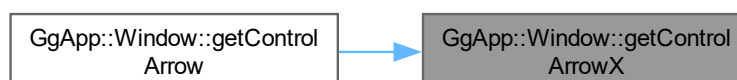
CTRL キーを押しながら矢印キーを押したときの現在の X 値.

GgApp.h の 622 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.12 getControlArrowY()

```
auto GgApp::Window::getControlArrowY () const [inline]
```

CTRL キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

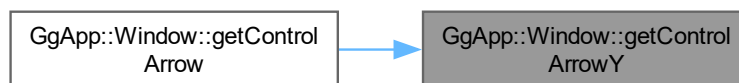
CTRL キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 632 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.13 getFboHeight()

```
auto GgApp::Window::getFboHeight () const [inline]
```

FBO の高さを得る.

戻り値

FBO の高さ.

[GgApp.h](#) の 431 行目に定義があります。

被呼び出し関係図:



7.25.3.14 getFboSize() [1/2]

```
const auto & GgApp::Window::getFboSize () const [inline]
```

FBO のサイズを得る.

戻り値

FBO の幅と高さを格納した GLsizei 型の 2 要素の配列.

[GgApp.h](#) の 462 行目に定義があります。

7.25.3.15 getFboSize() [2/2]

```
void GgApp::Window::getFboSize (  
    GLsizei * fboSize) const [inline]
```

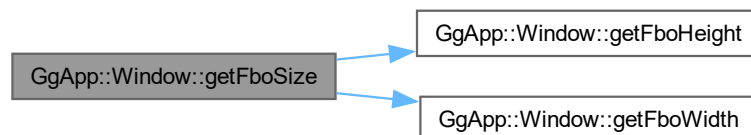
FBO のサイズを得る.

引数

<i>fboSize</i>	FBO の幅と高さを格納する GLsizei 型の 2 要素の配列.
----------------	------------------------------------

[GgApp.h](#) の 472 行目に定義があります。

呼び出し関係図:



7.25.3.16 getFboWidth()

```
auto GgApp::Window::getFboWidth () const [inline]
```

FBO の横幅を得る.

戻り値

FBO の横幅.

GgApp.h の 421 行目に定義があります。

被呼び出し関係図:



7.25.3.17 getHeight()

```
auto GgApp::Window::getHeight () const [inline]
```

ウィンドウの高さを得る.

戻り値

ウィンドウの高さ.

GgApp.h の 411 行目に定義があります。

被呼び出し関係図:



7.25.3.18 getKey()

```
bool GgApp::Window::getKey (  
    int key) const [inline]
```

キーが押されているかどうかを判定する.

戻り値

キーが押されていれば true.

GgApp.h の 493 行目に定義があります。

7.25.3.19 getLastKey()

```
int GgApp::Window::getLastKey () [inline]
```

最後にタイプしたキーを得る.

戻り値

最後にタイプしたキーの文字.

[GgApp.h](#) の [531](#) 行目に定義があります。

7.25.3.20 getMouse() [1/3]

```
const auto * GgApp::Window::getMouse () const [inline]
```

マウスカーソルの現在位置を得る.

戻り値

マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.

[GgApp.h](#) の [684](#) 行目に定義があります。

7.25.3.21 getMouse() [2/3]

```
void GgApp::Window::getMouse (
    GLfloat * position) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>position</i>	マウスカーソルの現在位置を格納した GLfloat 型の 2 要素の配列.
-----------------	---------------------------------------

[GgApp.h](#) の [695](#) 行目に定義があります。

7.25.3.22 getMouse() [3/3]

```
auto GgApp::Window::getMouse (
    int direction) const [inline]
```

マウスカーソルの現在位置を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスカーソルの現在位置.

[GgApp.h](#) の 708 行目に定義があります。

7.25.3.23 getMouseX()

```
auto GgApp::Window::getMouseX () const [inline]
```

マウスカーソルの現在位置の X 座標を得る.

戻り値

direction 方向のマウスカーソルの X 方向の現在位置.

[GgApp.h](#) の 719 行目に定義があります。

7.25.3.24 getMouseY()

```
auto GgApp::Window::getMouseY () const [inline]
```

マウスカーソルの現在位置の Y 座標を得る.

戻り値

direction 方向のマウスカーソルの Y 方向の現在位置.

[GgApp.h](#) の 730 行目に定義があります。

7.25.3.25 getRotation()

```
auto GgApp::Window::getRotation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW.MOUSE.BUTTON_[1,2]).
---------------	--

戻り値

回転を行う [GgQuaternion](#) 型の四元数.

[GgApp.h](#) の 858 行目に定義があります。

7.25.3.26 getRotationMatrix()

```
auto GgApp::Window::getRotationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボールの回転変換行列を得る.

引数

<i>button</i>	回転変換行列を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	--

戻り値

回転を行う **GgMatrix** 型の変換行列.

GgApp.h の 871 行目に定義があります。

7.25.3.27 getScrollMatrix()

```
auto GgApp::Window::getScrollMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによってオブジェクトの平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

クリッピング座標系で平行移動を行う **GgMatrix** 型の変換行列.

GgApp.h の 835 行目に定義があります。

7.25.3.28 getShiftArrow()

```
void GgApp::Window::getShiftArrow (
    GLfloat * shift_arrow) const [inline]
```

SHIFT キーを押しながら矢印キーを押したときの現在の値を得る.

引数

<i>shift_arrow</i>	SHIFT キーを押しながら矢印キーを押したときの値を格納する GLfloat 型の 2 要素の配列.
--------------------	---

GgApp.h の 611 行目に定義があります。

呼び出し関係図:



7.25.3.29 getShiftArrowX()

```
auto GgApp::Window::getShiftArrowX () const [inline]
```

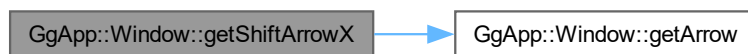
SHIFT キーを押しながら矢印キーを押したときの現在の X 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の X 値.

[GgApp.h](#) の 591 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.30 getShiftArrowY()

```
auto GgApp::Window::getShiftArrowY () const [inline]
```

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値を得る.

戻り値

SHIFT キーを押しながら矢印キーを押したときの現在の Y 値.

[GgApp.h](#) の 601 行目に定義があります。

呼び出し関係図:



被呼び出し関係図:



7.25.3.31 getSize() [1/2]

```
const auto & GgApp::Window::getSize () const [inline]
```

ウィンドウのサイズを得る.

戻り値

ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列の参照.

[GgApp.h](#) の 441 行目に定義があります.

7.25.3.32 getSize() [2/2]

```
void GgApp::Window::getSize (
    GLsizei * size) const [inline]
```

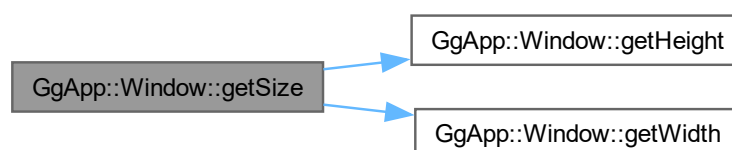
ウィンドウのサイズを得る.

引数

<i>size</i>	ウィンドウの幅と高さを格納した GLsizei 型の 2 要素の配列.
-------------	-------------------------------------

[GgApp.h](#) の 451 行目に定義があります.

呼び出し関係図:



7.25.3.33 getTranslation()

```
const auto & GgApp::Window::getTranslation (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

トラックボール処理を考慮したマウスによるスクロールの変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

平行移動量を格納した GLfloat[3] の配列のポインタ.

GgApp.h の 799 行目に定義があります。

7.25.3.34 getTranslationMatrix()

```
auto GgApp::Window::getTranslationMatrix (
    int button = GLFW_MOUSE_BUTTON_1) const [inline]
```

マウスによって視点の平行移動の変換行列を得る.

引数

<i>button</i>	平行移動量を取得するマウスボタン (GLFW_MOUSE_BUTTON_[1,2]).
---------------	---

戻り値

視点座標系で平行移動を行う GgMatrix 型の変換行列.

GgApp.h の 812 行目に定義があります。

7.25.3.35 getUserPointer()

```
void * GgApp::Window::getUserPointer () const [inline]
```

ユーザーポインタを取り出す.

戻り値

保存されているユーザポインタ.

GgApp.h の 913 行目に定義があります。

7.25.3.36 getWheel() [1/3]

```
const auto * GgApp::Window::getWheel () const [inline]
```

マウスホイールの回転量を得る.

戻り値

マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.

GgApp.h の 741 行目に定義があります。

7.25.3.37 getWheel() [2/3]

```
void GgApp::Window::getWheel (
    GLfloat * rotation) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>rotation</i>	マウスホイールの回転量を格納した GLfloat 型の 2 要素の配列.
-----------------	--------------------------------------

[GgApp.h](#) の 752 行目に定義があります。

7.25.3.38 getWheel() [3/3]

```
auto GgApp::Window::getWheel (
    int direction) const [inline]
```

マウスホイールの回転量を得る.

引数

<i>direction</i>	方向 (0:X, 1:Y).
------------------	----------------

戻り値

direction 方向のマウスホイールの回転量.

[GgApp.h](#) の 765 行目に定義があります。

7.25.3.39 getWheelX()

```
auto GgApp::Window::getWheelX () const [inline]
```

マウスホイールの X 方向の回転量を得る.

戻り値

マウスホイールの X 方向の回転量.

[GgApp.h](#) の 776 行目に定義があります。

7.25.3.40 getWheelY()

```
auto GgApp::Window::getWheelY () const [inline]
```

マウスホイールの Y 方向の回転量を得る.

戻り値

マウスホイールの Y 方向の回転量.

[GgApp.h](#) の 787 行目に定義があります。

7.25.3.41 getWidth()

```
auto GgApp::Window::getWidth () const [inline]
```

ウィンドウの横幅を得る.

戻り値

ウィンドウの横幅.

[GgApp.h](#) の 401 行目に定義があります。

被呼び出し関係図:



7.25.3.42 operator bool()

```
GgApp::Window::operator bool () [explicit]
```

イベントを取得してループを継続すべきかどうか調べる.

戻り値

ループを継続すべきなら true.

[GgApp.cpp](#) の 477 行目に定義があります。

呼び出し関係図:



7.25.3.43 operator=() [1/2]

```
Window & GgApp::Window::operator= (  
    const Window & w) [delete]
```

代入演算子を使用しない.

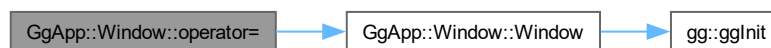
引数

<i>w</i>	代入元のウィンドウ.
----------	------------

戻り値

代入後のこのオブジェクトの参照.

呼び出し関係図:



7.25.3.44 operator=() [2/2]

```
GgApp::Window & GgApp::Window::operator= (  
    Window && w) [noexcept]
```

ムーブ代入演算子.

引数

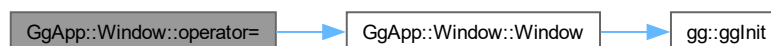
<i>w</i>	ムーブ代入元のウィンドウ.
----------	---------------

戻り値

ムーブ代入後のこのオブジェクトの参照.

[GgApp.cpp](#) の 446 行目に定義があります。

呼び出し関係図:



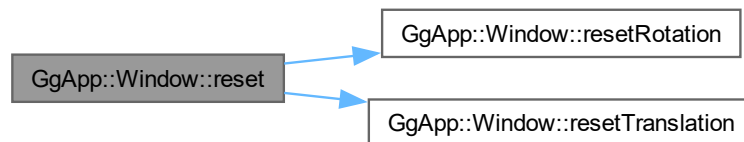
7.25.3.45 reset()

```
void GgApp::Window::reset () [inline]
```

トラックボール・マウスホイール・矢印キーの値を初期化する。

[GgApp.h](#) の 899 行目に定義があります。

呼び出し関係図:



7.25.3.46 resetRotation()

```
void GgApp::Window::resetRotation () [inline]
```

トラックボール処理を初期化する。

[GgApp.h](#) の 881 行目に定義があります。

被呼び出し関係図:



7.25.3.47 resetTranslation()

```
void GgApp::Window::resetTranslation () [inline]
```

平行移動量を初期化する。

[GgApp.h](#) の 890 行目に定義があります。

被呼び出し関係図:



7.25.3.48 restoreViewport()

```
void GgApp::Window::restoreViewport () const [inline]
```

ビューポートを元のサイズに復帰する.

GgApp.h の 370 行目に定義があります.

被呼び出し関係図:



7.25.3.49 selectInterface()

```
void GgApp::Window::selectInterface (
    int no) [inline]
```

インタフェースを選択する.

引数

<i>no</i>	インターフェース番号.
-----------	-------------

GgApp.h の 508 行目に定義があります.

7.25.3.50 setClose()

```
void GgApp::Window::setClose (
    int flag = GLFW_TRUE) const [inline]
```

ウィンドウのクローズフラグを設定する.

引数

<i>flag</i>	クローズフラグ, 0 (GLFW.FALSE) 以外ならウィンドウを閉じる.
-------------	--

GgApp.h の 339 行目に定義があります.

7.25.3.51 setKeyboardFunc()

```
void GgApp::Window::setKeyboardFunc (
    void(* func ) (const Window *window, int key, int scancode, int action, int mods))
[inline]
```

ユーザ定義の keyboard 関数を設定する.

引数

<i>func</i>	ユーザ定義の keyboard 関数, キーボードの操作時に呼び出される.
-------------	--

GgApp.h の 943 行目に定義があります。

呼び出し関係図:



7.25.3.52 setMouseFunc()

```
void GgApp::Window::setMouseFunc (  
    void(* func )(const Window *window, int button, int action, int mods)) [inline]
```

ユーザ定義の **mouse** 関数を設定する.

引数

<i>func</i>	ユーザ定義の mouse 関数, マウスボタンの操作時に呼び出される.
-------------	--

GgApp.h の 953 行目に定義があります。

呼び出し関係図:



7.25.3.53 setResizeFunc()

```
void GgApp::Window::setResizeFunc (  
    void(* func )(const Window *window, int width, int height)) [inline]
```

ユーザ定義の **resize** 関数を設定する.

引数

<i>func</i>	ユーザ定義の resize 関数, ウィンドウのサイズ変更時に呼び出される.
-------------	---

[GgApp.h](#) の 933 行目に定義があります。

呼び出し関係図:



7.25.3.54 setUserPointer()

```
void GgApp::Window::setUserPointer (
    void * pointer) [inline]
```

任意のユーザポインタを保存する。

引数

<i>pointer</i>	保存するユーザポインタ.
----------------	--------------

[GgApp.h](#) の 923 行目に定義があります。

7.25.3.55 setVelocity()

```
void GgApp::Window::setVelocity (
    GLfloat vx,
    GLfloat vy,
    GLfloat vz = 0.1f) [inline]
```

マウスの移動速度を設定する。

引数

<i>vx</i>	x 方向の移動速度.
<i>vy</i>	y 方向の移動速度.
<i>vz</i>	z 方向の移動速度.

[GgApp.h](#) の 521 行目に定義があります。

7.25.3.56 setWheelFunc()

```
void GgApp::Window::setWheelFunc (
    void(* func )(const Window *window, double x, double y)) [inline]
```

ユーザ定義の wheel 関数を設定する。

引数

<i>func</i>	ユーザ定義の <code>wheel</code> 関数, マウスホイールの操作時に呼び出される.
-------------	---

[GgApp.h](#) の 963 行目に定義があります。

呼び出し関係図:



7.25.3.57 shouldClose()

```
bool GgApp::Window::shouldClose () const [inline]
```

ウィンドウを閉じるべきかどうか調べる.

戻り値

ウィンドウを閉じるべきなら `true`.

[GgApp.h](#) の 349 行目に定義があります。

被呼び出し関係図:



7.25.3.58 swapBuffers()

```
void GgApp::Window::swapBuffers () const
```

カラーバッファを入れ替える.

[GgApp.cpp](#) の 528 行目に定義があります。

7.25.3.59 updateViewport()

```
void GgApp::Window::updateViewport ()
```

ビューポートのサイズを更新する.

[GgApp.cpp](#) の 547 行目に定義があります。

呼び出し関係図:



このクラス詳解は次のファイルから抽出されました:

- [GgApp.h](#)
- [GgApp.cpp](#)

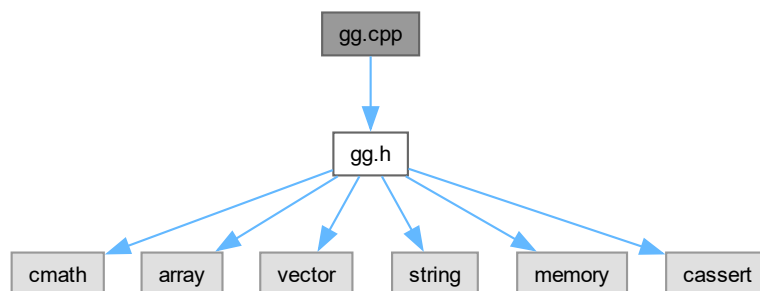
Chapter 8

ファイル詳解

8.1 gg.cpp ファイル

```
#include "gg.h"
```

gg.cpp の依存先関係図:



8.1.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の定義.

著者

Kohe Tokoi

日付

July 17, 2025

[gg.cpp](#) に定義があります。

8.2 gg.cpp

[詳解]

```

00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00025 #include "gg.h"
00026
00027 // 標準ライブラリ
00028 #include <cmath>
00029 #include <cstdlib>
00030 #include <iostream>
00031 #include <fstream>
00032 #include <sstream>
00033 #include <limits>
00034 #include <map>
00035
00036 #define READ_TEXTURE_COORDINATE_FROM_OBJ 0
00037
00038 // Windows (Visual Studio) のとき
00039 #if defined(MSC_VER)
00040 // リリースビルドではコンソールを出さない
00041 # if !defined(DEBUG)
00042 #   pragma comment(linker, "/subsystem:\"windows\" /entry:\"mainCRTStartup\"")
00043 # endif
00044 // リンクするライブラリ
00045 // # pragma comment(lib, "lib\\\" GLFW3_PLATFORM \"\\\" GLFW3_CONFIGURATION \"\\glfw3.lib")
00046
00047 //
00048 // For VC++ MFC Convert UTF-8 to TCHAR, or Convert TCHAR to UTF-8. VC++ MFC用 UTF-8⇔TCHARの変換処理
00049 //
00050 // Original author: mt-u
00051 // Copyright (c) 2013 mt-u
00052 // https://gist.github.com/mt-u/6878251
00053 //
00054 // Modified by: Kohe Tokoi
00055 //
00056 //
00057 // UTF-8 文字列を CString に変換する
00058 //
00059 //
00060 pathString Utf8ToTChar(const std::string& string)
00061 {
00062     // UTF-8 文字列を UTF-16 に変換した後の文字列の長さを求める
00063     const INT length{ MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, NULL, 0) };
00064
00065     // 変換結果の格納に必要な長さのメモリを確保する
00066     std::vector<WCHAR> utf16(length);
00067
00068     // UTF-8 文字列を UTF-16 に変換する
00069     MultiByteToWideChar(CP_UTF8, 0, string.c_str(), -1, utf16.data(), length);
00070
00071     // 変換した文字列を CString にして返す
00072     return CString{ utf16.data(), static_cast<int>(wcslen(utf16.data())) };
00073 }
00074
00075 //
00076 // CString を UTF-8 文字列に変換する
00077 //
00078 std::string TCharToUtf8(const pathString& cstring)
00079 {
00080     // 与えられた文字列を一旦 UTF-16 に変換する
00081     CStringW cstringw{ cstring };

```

```

00092
00093 // UTF-16 を UTF-8 に変換した後の文字列の長さを求める
00094 const INT length{ WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, NULL, 0, NULL, NULL) };
00095
00096 // 変換結果の格納に必要な長さのメモリを確保する
00097 std::vector<CHAR> utf8(length);
00098
00099 // UTF-16 を UTF-8 に変換する
00100 WideCharToMultiByte(CP_UTF8, 0, cstringw, -1, utf8.data(), length, NULL, NULL);
00101
00102 // 変換した文字列を std::string にして返す
00103 return std::string{ utf8.data(), strlen(utf8.data()) };
00104 }
00105 #endif
00106
00107 // OpenGL 3.2 の API のエントリポイント
00108 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00109 PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00110 PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00111 PFNGLACTIVEVTEXTUREPROC glActiveTexture;
00112 PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00113 PFNGLATTACHSHADERPROC glAttachShader;
00114 PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00115 PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00116 PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00117 PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00118 PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00119 PFNGLBEGINQUERYPROC glBeginQuery;
00120 PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00121 PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00122 PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00123 PFNGLBINDBUFFERPROC glBindBuffer;
00124 PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00125 PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00126 PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00127 PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00128 PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00129 PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00130 PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00131 PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00132 PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00133 PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00134 PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00135 PFNGLBINDSAMPLERPROC glBindSampler;
00136 PFNGLBINDSAMPLERSPROC glBindSamplers;
00137 PFNGLBINDTEXTUREPROC glBindTexture;
00138 PFNGLBINDTEXTURESPROC glBindTextures;
00139 PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00140 PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00141 PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00142 PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00143 PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00144 PFNGLBLENDARRIERKHRPROC glBlendBarrierKHR;
00145 PFNGLBLENDARRIERNVPROC glBlendBarrierNV;
00146 PFNGLBLENDCOLORPROC glBlendColor;
00147 PFNGLBLEND EQUATIONIARBPROC glBlendEquationiARB;
00148 PFNGLBLEND EQUATIONIPROC glBlendEquationi;
00149 PFNGLBLEND EQUATIONPROC glBlendEquation;
00150 PFNGLBLEND EQUATIONSEPARATEIARBPROC glBlendEquationSeparateiARB;
00151 PFNGLBLEND EQUATIONSEPARATEIPROC glBlendEquationSeparatei;
00152 PFNGLBLEND EQUATIONSEPARATEPROC glBlendEquationSeparate;
00153 PFNGLBLEND FUNC IARBPROC glBlendFunciARB;
00154 PFNGLBLEND FUNC IPROC glBlendFunci;
00155 PFNGLBLEND FUNC PROC glBlendFunc;
00156 PFNGLBLEND FUNC SEPARATEIARBPROC glBlendFuncSeparateiARB;
00157 PFNGLBLEND FUNC SEPARATEIPROC glBlendFuncSeparatei;
00158 PFNGLBLEND FUNC SEPARATEPROC glBlendFuncSeparate;
00159 PFNGLBLEND PARAMETER INVPROC glBlendParameteriNV;
00160 PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00161 PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00162 PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00163 PFNGLBUFFERDATAPROC glBufferData;
00164 PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00165 PFNGLBUFFERSTORAGEPROC glBufferStorage;
00166 PFNGLBUFFERSUBDATAPROC glBufferSubData;
00167 PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00168 PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00169 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00170 PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00171 PFNGLCLAMP COLORPROC glClampColor;
00172 PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00173 PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00174 PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00175 PFNGLCLEARBUFFERIVPROC glClearBufferiv;
00176 PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00177 PFNGLCLEARBUFFERUIVPROC glClearBufferuiv;
00178 PFNGLCLEARCOLORPROC glClearColor;

```

```
00179 PFNGLCLEARDEPTHFPROC glClearDepthf;
00180 PFNGLCLEARDEPTHFPROC glClearDepth;
00181 PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00182 PFNGLCLEARNAMEDBUFFERDATAAPROC glClearNamedBufferData;
00183 PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00184 PFNGLCLEARNAMEDBUFFERSUBDATAAPROC glClearNamedBufferSubData;
00185 PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00186 PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00187 PFNGLCLEARNAMEDFRAMEBUFFERIVPROC glClearNamedFramebufferiv;
00188 PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC glClearNamedFramebufferuiv;
00189 PFNGLCLEARPROC glClear;
00190 PFNGLCLEARSTENCILPROC glClearStencil;
00191 PFNGLCLEARTEXIMAGEPROC glClearTexImage;
00192 PFNGLCLEARTEXSUBIMAGEPROC glClearTexSubImage;
00193 PFNGLCLIENTATTRIBDEFAULTPROC glClientAttribDefaultEXT;
00194 PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
00195 PFNGLCLIPCONTROLPROC glClipControl;
00196 PFNGLCOLORFORMATNVPROC glColorFormatNV;
00197 PFNGLCOLORMASKIPROC glColorMaski;
00198 PFNGLCOLORMASKPROC glColorMask;
00199 PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00200 PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00201 PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00202 PFNGLCOMPILESHADERPROC glCompileShader;
00203 PFNGLCOMPRESSEDMULTITEXIMAGE1DXTPROC glCompressedMultiTexImage1DXT;
00204 PFNGLCOMPRESSEDMULTITEXIMAGE2DXTPROC glCompressedMultiTexImage2DXT;
00205 PFNGLCOMPRESSEDMULTITEXIMAGE3DXTPROC glCompressedMultiTexImage3DXT;
00206 PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DXTPROC glCompressedMultiTexSubImage1DXT;
00207 PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DXTPROC glCompressedMultiTexSubImage2DXT;
00208 PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DXTPROC glCompressedMultiTexSubImage3DXT;
00209 PFNGLCOMPRESSEDTEXIMAGE1DPROC glCompressedTexImage1D;
00210 PFNGLCOMPRESSEDTEXIMAGE2DPROC glCompressedTexImage2D;
00211 PFNGLCOMPRESSEDTEXIMAGE3DPROC glCompressedTexImage3D;
00212 PFNGLCOMPRESSEDTEXSUBIMAGE1DPROC glCompressedTexSubImage1D;
00213 PFNGLCOMPRESSEDTEXSUBIMAGE2DPROC glCompressedTexSubImage2D;
00214 PFNGLCOMPRESSEDTEXSUBIMAGE3DPROC glCompressedTexSubImage3D;
00215 PFNGLCOMPRESSEDTEXTUREIMAGE1DXTPROC glCompressedTextureImage1DXT;
00216 PFNGLCOMPRESSEDTEXTUREIMAGE2DXTPROC glCompressedTextureImage2DXT;
00217 PFNGLCOMPRESSEDTEXTUREIMAGE3DXTPROC glCompressedTextureImage3DXT;
00218 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DXTPROC glCompressedTextureSubImage1DXT;
00219 PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00220 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DXTPROC glCompressedTextureSubImage2DXT;
00221 PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00222 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DXTPROC glCompressedTextureSubImage3DXT;
00223 PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00224 PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00225 PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00226 PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00227 PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00228 PFNGLCOPYMULTITEXIMAGE1DXTPROC glCopyMultiTexImage1DXT;
00229 PFNGLCOPYMULTITEXIMAGE2DXTPROC glCopyMultiTexImage2DXT;
00230 PFNGLCOPYMULTITEXSUBIMAGE1DXTPROC glCopyMultiTexSubImage1DXT;
00231 PFNGLCOPYMULTITEXSUBIMAGE2DXTPROC glCopyMultiTexSubImage2DXT;
00232 PFNGLCOPYMULTITEXSUBIMAGE3DXTPROC glCopyMultiTexSubImage3DXT;
00233 PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00234 PFNGLCOPYPATHNVPROC glCopyPathNV;
00235 PFNGLCOPYTEXIMAGE1DPROC glCopyTexImage1D;
00236 PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00237 PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00238 PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00239 PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00240 PFNGLCOPYTEXTUREIMAGE1DXTPROC glCopyTextureImage1DXT;
00241 PFNGLCOPYTEXTUREIMAGE2DXTPROC glCopyTextureImage2DXT;
00242 PFNGLCOPYTEXTURESUBIMAGE1DXTPROC glCopyTextureSubImage1DXT;
00243 PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00244 PFNGLCOPYTEXTURESUBIMAGE2DXTPROC glCopyTextureSubImage2DXT;
00245 PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00246 PFNGLCOPYTEXTURESUBIMAGE3DXTPROC glCopyTextureSubImage3DXT;
00247 PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00248 PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00249 PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00250 PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00251 PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00252 PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00253 PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00254 PFNGLCREATEBUFFERSPROC glCreateBuffers;
00255 PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00256 PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00257 PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00258 PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00259 PFNGLCREATEPROGRAMPROC glCreateProgram;
00260 PFNGLCREATEQUERIESPROC glCreateQueries;
00261 PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00262 PFNGLCREATESAMPLERSPROC glCreateSamplers;
00263 PFNGLCREATESHADERPROC glCreateShader;
00264 PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00265 PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
```

```
00266 PFNGLCREATESTATESNVPROC glCreateStatesNV;
00267 PFNGLCREATESENDERFROMCLEVENTARBPROC glCreateSenderFromCLEventARB;
00268 PFNGLCREATETEXTURESPROC glCreateTextures;
00269 PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00270 PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00271 PFNGLCULLFACEPROC glCullFace;
00272 PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00273 PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00274 PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00275 PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00276 PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00277 PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00278 PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00279 PFNGLDELETECOMMANDLISTSNVPROC glDeleteCommandListsNV;
00280 PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00281 PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00282 PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00283 PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00284 PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00285 PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00286 PFNGLDELETEPROGRAMPROC glDeleteProgram;
00287 PFNGLDELETEQUERIESPROC glDeleteQueries;
00288 PFNGLDELETERENDERBUFFERSPROC glDeleteRenderbuffers;
00289 PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00290 PFNGLDELETESHADERPROC glDeleteShader;
00291 PFNGLDELETESSTATESNVPROC glDeleteStatesNV;
00292 PFNGLDELETESYNCPROC glDeleteSync;
00293 PFNGLDELETETEXTURESPROC glDeleteTextures;
00294 PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00295 PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00296 PFNGLDEPTHFUNCPROC glDepthFunc;
00297 PFNGLDEPTHMASKPROC glDepthMask;
00298 PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00299 PFNGLDEPTHRANGEFPROC glDepthRangef;
00300 PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00301 PFNGLDEPTHRANGEPROC glDepthRange;
00302 PFNGLDETACHSHADERPROC glDetachShader;
00303 PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00304 PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00305 PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00306 PFNGLDISABLEIPROC glDisablei;
00307 PFNGLDISABLEPROC glDisable;
00308 PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00309 PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00310 PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00311 PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00312 PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00313 PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00314 PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00315 PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00316 PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00317 PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00318 PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00319 PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00320 PFNGLDRAWARRAYSPROC glDrawArrays;
00321 PFNGLDRAWBUFFERPROC glDrawBuffer;
00322 PFNGLDRAWBUFFERSPROC glDrawBuffers;
00323 PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00324 PFNGLDRAWCOMMANDSNVPROC glDrawCommandsNV;
00325 PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00326 PFNGLDRAWCOMMANDSSTATESNVPROC glDrawCommandsStatesNV;
00327 PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00328 PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00329 PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00330 PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00331 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC glDrawElementsInstancedBaseVertexBaseInstance;
00332 PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00333 PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00334 PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00335 PFNGLDRAWELEMENTSPROC glDrawElements;
00336 PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00337 PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00338 PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00339 PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00340 PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00341 PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00342 PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00343 PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00344 PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00345 PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00346 PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00347 PFNGLENABLEIPROC glEnablei;
00348 PFNGLENABLEPROC glEnable;
00349 PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00350 PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00351 PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00352 PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
```

```
00353 PFNGLENDCONDITIONALRENDERNVPROC glEndConditionalRenderNV;
00354 PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00355 PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00356 PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00357 PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00358 PFNGLENDQUERYPROC glEndQuery;
00359 PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00360 PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00361 PFNGLFENCESYNCPROC glFenceSync;
00362 PFNGLFINISHPROC glFinish;
00363 PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00364 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00365 PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00366 PFNGLFLUSHPROC glFlush;
00367 PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00368 PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00369 PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00370 PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00371 PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00372 PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00373 PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00374 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00375 PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00376 PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00377 PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00378 PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00379 PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00380 PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00381 PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00382 PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00383 PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00384 PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00385 PFNGLFRONTFACEPROC glFrontFace;
00386 PFNGLGENBUFFERSPROC glGenBuffers;
00387 PFNGLGENERATEMIPMAPPROC glGenerateMipmap;
00388 PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenerateMultiTexMipmapEXT;
00389 PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenerateTextureMipmapEXT;
00390 PFNGLGENERATETEXTUREMIPMAPPROC glGenerateTextureMipmap;
00391 PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00392 PFNGLGENPATHSNVPROC glGenPathsNV;
00393 PFNGLGENPERFMONITORAMDPROC glGenPerfMonitorsAMD;
00394 PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00395 PFNGLGENQUERIESPROC glGenQueries;
00396 PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00397 PFNGLGENSAMPLERSPROC glGenSamplers;
00398 PFNGLGENTEXTURESPROC glGenTextures;
00399 PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00400 PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00401 PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00402 PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00403 PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00404 PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00405 PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00406 PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00407 PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00408 PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;
00409 PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00410 PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00411 PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00412 PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00413 PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00414 PFNGLGETBOOLEANI_VPROC glGetBooleani_v;
00415 PFNGLGETBOOLEANVPROC glGetBooleanv;
00416 PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00417 PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00418 PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00419 PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00420 PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00421 PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00422 PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00423 PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00424 PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00425 PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00426 PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00427 PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00428 PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00429 PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00430 PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00431 PFNGLGETDOUBLEI_VEXTPROC glGetDoublei_vEXT;
00432 PFNGLGETDOUBLEI_VPROC glGetDoublei_v;
00433 PFNGLGETDOUBLEVPROC glGetDoublev;
00434 PFNGLGETERRORPROC glGetError;
00435 PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00436 PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00437 PFNGLGETFLOATI_VEXTPROC glGetFloati_vEXT;
00438 PFNGLGETFLOATI_VPROC glGetFloati_v;
00439 PFNGLGETFLOATVPROC glGetFloatv;
```



```
00440 PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00441 PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00442 PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00443 PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00444 PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00445 PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00446 PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00447 PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00448 PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00449 PFNGLGETINTEGER64I_VPROC glGetInteger64i_v;
00450 PFNGLGETINTEGER64VPROC glGetInteger64v;
00451 PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00452 PFNGLGETINTEGERI_VPROC glGetIntegeri_v;
00453 PFNGLGETINTEGERUI64I_VNVPROC glGetIntegerui64i_vNV;
00454 PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00455 PFNGLGETINTEGERVPROC glGetIntegeriv;
00456 PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00457 PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00458 PFNGLGETINTERNALFORMATSAMPLEI_VNVPROC glGetInternalformatSampleivNV;
00459 PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00460 PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00461 PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00462 PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00463 PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00464 PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00465 PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00466 PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00467 PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00468 PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00469 PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameterIivEXT;
00470 PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameterIuivEXT;
00471 PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00472 PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00473 PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00474 PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00475 PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00476 PFNGLGETNAMEDBUFFERPOINTINTERVEXTPROC glGetNamedBufferPointervEXT;
00477 PFNGLGETNAMEDBUFFERPOINTINTERVPROC glGetNamedBufferPointerv;
00478 PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00479 PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00480 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC glGetNamedFramebufferAttachmentParameterivEXT;
00481 PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00482 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00483 PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00484 PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00485 PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00486 PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00487 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00488 PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00489 PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00490 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00491 PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00492 PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00493 PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00494 PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
00495 PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00496 PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00497 PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00498 PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00499 PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvarb;
00500 PFNGLGETNUNIFORMDVPROC glGetnUniformdv;
00501 PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvarb;
00502 PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00503 PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64varb;
00504 PFNGLGETNUNIFORMIVARBPROC glGetnUniformivarb;
00505 PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00506 PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64varb;
00507 PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivarb;
00508 PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00509 PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00510 PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00511 PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00512 PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00513 PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00514 PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00515 PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00516 PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00517 PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00518 PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00519 PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00520 PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00521 PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00522 PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00523 PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00524 PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00525 PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00526 PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
```

```
00527 PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00528 PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00529 PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00530 PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00531 PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00532 PFNGLGETPOINTERI_VEXTPROC glGetPointeri_vEXT;
00533 PFNGLGETPOINTERVPROC glGetPointerv;
00534 PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00535 PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00536 PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00537 PFNGLGETPROGRAMIVPROC glGetProgramiv;
00538 PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00539 PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00540 PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00541 PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00542 PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00543 PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00544 PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00545 PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00546 PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00547 PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00548 PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00549 PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00550 PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00551 PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00552 PFNGLGETQUERYIVPROC glGetQueryiv;
00553 PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00554 PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00555 PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00556 PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00557 PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00558 PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00559 PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameterIiv;
00560 PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameterIuiv;
00561 PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00562 PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00563 PFNGLGETSHADERIVPROC glGetShaderiv;
00564 PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00565 PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00566 PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00567 PFNGLGETSTRINGIPROC glGetStringi;
00568 PFNGLGETSTRINGPROC glGetString;
00569 PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00570 PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00571 PFNGLGETSYNClVPROC glGetSynciv;
00572 PFNGLGETTEXIMAGEPROC glGetTexImage;
00573 PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00574 PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00575 PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00576 PFNGLGETTEXPARAMETERIIVPROC glGetTexParameterIiv;
00577 PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameterIuiv;
00578 PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00579 PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00580 PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00581 PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;
00582 PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00583 PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00584 PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00585 PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00586 PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00587 PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00588 PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00589 PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00590 PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00591 PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00592 PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00593 PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00594 PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00595 PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00596 PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00597 PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00598 PFNGLGETTRANSFORMFEEDBACKI64_VPROC glGetTransformFeedbacki64_v;
00599 PFNGLGETTRANSFORMFEEDBACKIVPROC glGetTransformFeedbackiv;
00600 PFNGLGETTRANSFORMFEEDBACKI_VPROC glGetTransformFeedbacki_v;
00601 PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00602 PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00603 PFNGLGETUNIFORMDVPROC glGetUniformdv;
00604 PFNGLGETUNIFORMFVPROC glGetUniformfv;
00605 PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00606 PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00607 PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00608 PFNGLGETUNIFORMIVPROC glGetUniformiv;
00609 PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00610 PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00611 PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00612 PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00613 PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
```

```
00614 PFNGLGETVERTEXARRAYINDEXED64IVPROC   glGetVertexArrayIndexed64iv;
00615 PFNGLGETVERTEXARRAYINDEXEDIVPROC   glGetVertexArrayIndexediv;
00616 PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC   glGetVertexArrayIntegeri_vEXT;
00617 PFNGLGETVERTEXARRAYINTEGERVEXTPROC   glGetVertexArrayInteger_vEXT;
00618 PFNGLGETVERTEXARRAYI_VPROC   glGetVertexArrayiv;
00619 PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC   glGetVertexArrayPointeri_vEXT;
00620 PFNGLGETVERTEXARRAYPOINTERVEXTPROC   glGetVertexArrayPointervEXT;
00621 PFNGLGETVERTEXATTRIBDVPROC   glGetVertexAttribdv;
00622 PFNGLGETVERTEXATTRIBFVPROC   glGetVertexAttribfv;
00623 PFNGLGETVERTEXATTRIBIIVPROC   glGetVertexAttribIiv;
00624 PFNGLGETVERTEXATTRIBIUIVPROC   glGetVertexAttribIuiv;
00625 PFNGLGETVERTEXATTRIBIVPROC   glGetVertexAttribiv;
00626 PFNGLGETVERTEXATTRIBLDVPROC   glGetVertexAttribLdv;
00627 PFNGLGETVERTEXATTRIBLI64VNVPROC   glGetVertexAttribLi64vNV;
00628 PFNGLGETVERTEXATTRIBLUI64VARBPROC   glGetVertexAttribLui64vARB;
00629 PFNGLGETVERTEXATTRIBLUI64VNVPROC   glGetVertexAttribLui64vNV;
00630 PFNGLGETVERTEXATTRIBPOINTERVPROC   glGetVertexAttribPointerv;
00631 PFNGLGETVKPROCADDRNVPROC   glGetVkProcAddrNV;
00632 PFNGLHINTPROC   glHint;
00633 PFNGLINDEXFORMATNVPROC   glIndexFormatNV;
00634 PFNGLINSERTEVENTMARKEREXTPROC   glInsertEventMarkerEXT;
00635 PFNGLINTERPOLATEPATHSNVPROC   glInterpolatePathsNV;
00636 PFNGLINVALIDATEBUFFERDATAPROC   glInvalidateBufferData;
00637 PFNGLINVALIDATEBUFFERSUBDATAPROC   glInvalidateBufferSubData;
00638 PFNGLINVALIDATEFRAMEBUFFERPROC   glInvalidateFramebuffer;
00639 PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC   glInvalidateNamedFramebufferData;
00640 PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC   glInvalidateNamedFramebufferSubData;
00641 PFNGLINVALIDATESUBFRAMEBUFFERPROC   glInvalidateSubFramebuffer;
00642 PFNGLINVALIDATETEXIMAGEPROC   glInvalidateTexImage;
00643 PFNGLINVALIDATETEXSUBIMAGEPROC   glInvalidateTexSubImage;
00644 PFNGLISBUFFERPROC   glIsBuffer;
00645 PFNGLISBUFFERRESIDENTNVPROC   glIsBufferResidentNV;
00646 PFNGLISCOMMANDLISTNVPROC   glIsCommandListNV;
00647 PFNGLISENABLEDINDEXEDEXTPROC   glIsEnabledIndexedEXT;
00648 PFNGLISENABLEDIPROC   glIsEnabledi;
00649 PFNGLISENABLEDPROC   glIsEnabled;
00650 PFNGLISFRAMEBUFFERPROC   glIsFramebuffer;
00651 PFNGLISIMAGEHANDLERESIDENTARBPROC   glIsImageHandleResidentARB;
00652 PFNGLISIMAGEHANDLERESIDENTNVPROC   glIsImageHandleResidentNV;
00653 PFNGLISNAMEDBUFFERRESIDENTNVPROC   glIsNamedBufferResidentNV;
00654 PFNGLISNAMEDSTRINGARBPROC   glIsNamedStringARB;
00655 PFNGLISPATHNVPROC   glIsPathNV;
00656 PFNGLISPOINTINFILLPATHNVPROC   glIsPointInFillPathNV;
00657 PFNGLISPOINTINSTROKEPATHNVPROC   glIsPointInStrokePathNV;
00658 PFNGLISPROGRAMPIPELINEPROC   glIsProgramPipeline;
00659 PFNGLISPROGRAMPROC   glIsProgram;
00660 PFNGLISQUERYPROC   glIsQuery;
00661 PFNGLISRENDERBUFFERPROC   glIsRenderbuffer;
00662 PFNGLISSAMPLERPROC   glIsSampler;
00663 PFNGLISSHADERPROC   glIsShader;
00664 PFNGLISSTATENVPROC   glIsStateNV;
00665 PFNGLISSYNCPROC   glIsSync;
00666 PFNGLISTEXTUREHANDLERESIDENTARBPROC   glIsTextureHandleResidentARB;
00667 PFNGLISTEXTUREHANDLERESIDENTNVPROC   glIsTextureHandleResidentNV;
00668 PFNGLISTEXTUREPROC   glIsTexture;
00669 PFNGLISTRANSFORMFEEDBACKPROC   glIsTransformFeedback;
00670 PFNGLISVERTEXARRAYPROC   glIsVertexArray;
00671 PFNGLLABELOBJECTEXTPROC   glLabelObjectEXT;
00672 PFNGLLINEWIDTHPROC   glLineWidth;
00673 PFNGLLINKPROGRAMPROC   glLinkProgram;
00674 PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC   glListDrawCommandsStatesClientNV;
00675 PFNGLLOGICOPPROC   glLogicOp;
00676 PFNGLMAKEBUFFERNONRESIDENTNVPROC   glMakeBufferNonResidentNV;
00677 PFNGLMAKEBUFFERRESIDENTNVPROC   glMakeBufferResidentNV;
00678 PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC   glMakeImageHandleNonResidentARB;
00679 PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC   glMakeImageHandleNonResidentNV;
00680 PFNGLMAKEIMAGEHANDLERESIDENTARBPROC   glMakeImageHandleResidentARB;
00681 PFNGLMAKEIMAGEHANDLERESIDENTNVPROC   glMakeImageHandleResidentNV;
00682 PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC   glMakeNamedBufferNonResidentNV;
00683 PFNGLMAKENAMEDBUFFERRESIDENTNVPROC   glMakeNamedBufferResidentNV;
00684 PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC   glMakeTextureHandleNonResidentARB;
00685 PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC   glMakeTextureHandleNonResidentNV;
00686 PFNGLMAKETEXTUREHANDLERESIDENTARBPROC   glMakeTextureHandleResidentARB;
00687 PFNGLMAKETEXTUREHANDLERESIDENTNVPROC   glMakeTextureHandleResidentNV;
00688 PFNGLMAPBUFFERPROC   glMapBuffer;
00689 PFNGLMAPBUFFERRANGEPROC   glMapBufferRange;
00690 PFNGLMAPNAMEDBUFFEREXTPROC   glMapNamedBufferEXT;
00691 PFNGLMAPNAMEDBUFFERPROC   glMapNamedBuffer;
00692 PFNGLMAPNAMEDBUFFERRANGEEXTPROC   glMapNamedBufferRangeEXT;
00693 PFNGLMAPNAMEDBUFFERRANGEPROC   glMapNamedBufferRange;
00694 PFNGLMATRIXFRUSTUMEXTPROC   glMatrixFrustumEXT;
00695 PFNGLMATRIXLOAD3X2FNVPROC   glMatrixLoad3x2fNV;
00696 PFNGLMATRIXLOAD3X3FNVPROC   glMatrixLoad3x3fNV;
00697 PFNGLMATRIXLOADDEXTPROC   glMatrixLoadEXT;
00698 PFNGLMATRIXLOADFEXTPROC   glMatrixLoadfEXT;
00699 PFNGLMATRIXLOADIDENTITYEXTPROC   glMatrixLoadIdentityEXT;
00700 PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC   glMatrixLoadTranspose3x3fNV;
```

```
00701 PFNGLMATRIXLOADTRANPOSEDEXTPROC glMatrixLoadTransposedEXT;
00702 PFNGLMATRIXLOADTRANPOSEFEXTPROC glMatrixLoadTransposefEXT;
00703 PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00704 PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00705 PFNGLMATRIXMULTDEXTPROC glMatrixMultdEXT;
00706 PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00707 PFNGLMATRIXMULTTRANPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00708 PFNGLMATRIXMULTTRANPOSEDEXTPROC glMatrixMultTransposedEXT;
00709 PFNGLMATRIXMULTTRANPOSEFEXTPROC glMatrixMultTransposefEXT;
00710 PFNGLMATRIXORTHOEXTPROC glMatrixOrthoEXT;
00711 PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00712 PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00713 PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00714 PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00715 PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00716 PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00717 PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00718 PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00719 PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00720 PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00721 PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00722 PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00723 PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00724 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00725 PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00726 PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00727 PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00728 PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00729 PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00730 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00731 PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00732 PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00733 PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00734 PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00735 PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00736 PFNGLMULTITEXCOORDPOINTEREEXTPROC glMultiTexCoordPointerEXT;
00737 PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00738 PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00739 PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00740 PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00741 PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00742 PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00743 PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00744 PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00745 PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00746 PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00747 PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DEXT;
00748 PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DEXT;
00749 PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DEXT;
00750 PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00751 PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00752 PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00753 PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameterIivEXT;
00754 PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameterIuivEXT;
00755 PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
00756 PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00757 PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1DEXT;
00758 PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2DEXT;
00759 PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3DEXT;
00760 PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00761 PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00762 PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00763 PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00764 PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00765 PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00766 PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00767 PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00768 PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00769 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00770 PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00771 PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00772 PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00773 PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00774 PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00775 PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00776 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00777 PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00778 PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1DEXT;
00779 PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2DEXT;
00780 PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3DEXT;
00781 PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00782 PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00783 PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00784 PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00785 PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00786 PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00787 PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
```

```
00788 PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00789 PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00790 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00791 PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00792 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00793 PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00794 PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00795 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IEXTPROC glNamedProgramLocalParametersI4iEXT;
00796 PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00797 PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00798 PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00799 PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEEXTPROC
    glNamedRenderbufferStorageMultisampleCoverageEXT;
00800 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00801 PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00802 PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00803 PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00804 PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00805 PFNGLOBJEC TLABELPROC glObjectLabel;
00806 PFNGLOBJEC TPTRLABELPROC glObjectPtrLabel;
00807 PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00808 PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00809 PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00810 PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00811 PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00812 PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00813 PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00814 PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00815 PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00816 PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00817 PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00818 PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00819 PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00820 PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00821 PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00822 PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00823 PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00824 PFNGLPATHSTRINGNVPROC glPathStringNV;
00825 PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00826 PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00827 PFNGLPAUSESETTRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00828 PFNGLPIXELSTOREFPROC glPixelStoref;
00829 PFNGLPIXELSTOREIPROC glPixelStorei;
00830 PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00831 PFNGLPOINTPARAMETERFPROC glPointParameterf;
00832 PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00833 PFNGLPOINTPARAMETERIPROC glPointParameteri;
00834 PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00835 PFNGLPOINTSIZEPROC glPointSize;
00836 PFNGLPOLYGONMODEPROC glPolygonMode;
00837 PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00838 PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00839 PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00840 PFNGLPOPDEBUGGROUPMARKEREXTPROC glPopGroupMarkerEXT;
00841 PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
00842 PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00843 PFNGLPROGRAMBINARYPROC glProgramBinary;
00844 PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00845 PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00846 PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00847 PFNGLPROGRAMUNIFORM1DEXTPROC glProgramUniform1dEXT;
00848 PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00849 PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00850 PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00851 PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00852 PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00853 PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00854 PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00855 PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00856 PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00857 PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00858 PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00859 PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00860 PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00861 PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00862 PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00863 PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00864 PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00865 PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00866 PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00867 PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00868 PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00869 PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00870 PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00871 PFNGLPROGRAMUNIFORM2DEXTPROC glProgramUniform2dEXT;
00872 PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00873 PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
```

```
00874 PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00875 PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00876 PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00877 PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00878 PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00879 PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00880 PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00881 PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00882 PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00883 PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00884 PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00885 PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00886 PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00887 PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00888 PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00889 PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00890 PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00891 PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00892 PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00893 PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00894 PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00895 PFNGLPROGRAMUNIFORM3DEXTPROC glProgramUniform3dEXT;
00896 PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00897 PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00898 PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00899 PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00900 PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00901 PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00902 PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00903 PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00904 PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00905 PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00906 PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00907 PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00908 PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00909 PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00910 PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00911 PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00912 PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00913 PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00914 PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00915 PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00916 PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00917 PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00918 PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00919 PFNGLPROGRAMUNIFORM4DEXTPROC glProgramUniform4dEXT;
00920 PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00921 PFNGLPROGRAMUNIFORM4DVEXTPROC glProgramUniform4dvEXT;
00922 PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00923 PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00924 PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00925 PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00926 PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00927 PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00928 PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;
00929 PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00930 PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00931 PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00932 PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00933 PFNGLPROGRAMUNIFORM4IVEXTPROC glProgramUniform4ivEXT;
00934 PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00935 PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00936 PFNGLPROGRAMUNIFORM4UI64NVPROC glProgramUniform4ui64NV;
00937 PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00938 PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00939 PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00940 PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00941 PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00942 PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00943 PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00944 PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC glProgramUniformHandleui64NV;
00945 PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00946 PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00947 PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00948 PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00949 PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00950 PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00951 PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00952 PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00953 PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00954 PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00955 PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00956 PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00957 PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00958 PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00959 PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00960 PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
```

```
00961 PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00962 PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00963 PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00964 PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00965 PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00966 PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00967 PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00968 PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00969 PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00970 PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00971 PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00972 PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00973 PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00974 PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00975 PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00976 PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00977 PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00978 PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00979 PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00980 PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00981 PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00982 PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00983 PFNGLPROGRAMUNIFORMUI64NVPROC glProgramUniformui64NV;
00984 PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00985 PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00986 PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC glPushClientAttribDefaultEXT;
00987 PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00988 PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00989 PFNGLQUERYCOUNTERPROC glQueryCounter;
00990 PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00991 PFNGLREADBUFFERPROC glReadBuffer;
00992 PFNGLREADNPXELARBPROC glReadnPixelsARB;
00993 PFNGLREADNPXELSPROC glReadnPixels;
00994 PFNGLREADPIXELSPROC glReadPixels;
00995 PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
00996 PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGENVPROC glRenderbufferStorageMultisampleCoverageNV;
00997 PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
00998 PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
00999 PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
01000 PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01001 PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01002 PFNGLSAMPLEMASKIPROC glSampleMaski;
01003 PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01004 PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
01005 PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01006 PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01007 PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01008 PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01009 PFNGLSCISSORARRAYVPROC glScissorArrayv;
01010 PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01011 PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01012 PFNGLSCISSORPROC glScissor;
01013 PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01014 PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01015 PFNGLSHADERBINARYPROC glShaderBinary;
01016 PFNGLSHADERSOURCEPROC glShaderSource;
01017 PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01018 PFNGLSIGNALVKFENCENVPROC glSignalVkFenceNV;
01019 PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01020 PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01021 PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01022 PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01023 PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01024 PFNGLSTENCILFUNCPROC glStencilFunc;
01025 PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01026 PFNGLSTENCILMASKPROC glStencilMask;
01027 PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01028 PFNGLSTENCILOPPROC glStencilOp;
01029 PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01030 PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01031 PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01032 PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01033 PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01034 PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01035 PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01036 PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01037 PFNGLTEXBUFFERARBPROC glTexBufferARB;
01038 PFNGLTEXBUFFERPROC glTexBuffer;
01039 PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01040 PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01041 PFNGLTEXIMAGE1DPROC glTexImage1D;
01042 PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01043 PFNGLTEXIMAGE2DPROC glTexImage2D;
01044 PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01045 PFNGLTEXIMAGE3DPROC glTexImage3D;
01046 PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01047 PFNGLTEXPARAMETERFPROC glTexParameterf;
```

```
01048 PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01049 PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01050 PFNGLTEXPARAMETERIPROC glTexParameteri;
01051 PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01052 PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01053 PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01054 PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01055 PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01056 PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01057 PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01058 PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01059 PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01060 PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01061 PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01062 PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01063 PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01064 PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01065 PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01066 PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01067 PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01068 PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01069 PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01070 PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01071 PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01072 PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01073 PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01074 PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01075 PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01076 PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01077 PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01078 PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01079 PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01080 PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01081 PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01082 PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01083 PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01084 PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01085 PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01086 PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01087 PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01088 PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01089 PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01090 PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01091 PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01092 PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01093 PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01094 PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01095 PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01096 PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01097 PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01098 PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01099 PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01100 PFNGLTEXTUREVIEWPROC glTextureView;
01101 PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01102 PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;
01103 PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01104 PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01105 PFNGLUNIFORM1DPROC glUniform1d;
01106 PFNGLUNIFORM1DVPROC glUniform1dv;
01107 PFNGLUNIFORM1FPROC glUniform1f;
01108 PFNGLUNIFORM1FVPROC glUniform1fv;
01109 PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01110 PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01111 PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01112 PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01113 PFNGLUNIFORM1IPROC glUniform1i;
01114 PFNGLUNIFORM1IVPROC glUniform1iv;
01115 PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01116 PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01117 PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01118 PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01119 PFNGLUNIFORM1UIPROC glUniform1ui;
01120 PFNGLUNIFORM1UIVPROC glUniform1uiv;
01121 PFNGLUNIFORM2DPROC glUniform2d;
01122 PFNGLUNIFORM2DVPROC glUniform2dv;
01123 PFNGLUNIFORM2FPROC glUniform2f;
01124 PFNGLUNIFORM2FVPROC glUniform2fv;
01125 PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01126 PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01127 PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01128 PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01129 PFNGLUNIFORM2IPROC glUniform2i;
01130 PFNGLUNIFORM2IVPROC glUniform2iv;
01131 PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01132 PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01133 PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01134 PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
```



```
01135 PFNGLUNIFORM2UIPROC glUniform2ui;
01136 PFNGLUNIFORM2UIVPROC glUniform2uiv;
01137 PFNGLUNIFORM3DPROC glUniform3d;
01138 PFNGLUNIFORM3DVPROC glUniform3dv;
01139 PFNGLUNIFORM3FPROC glUniform3f;
01140 PFNGLUNIFORM3FVPROC glUniform3fv;
01141 PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01142 PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01143 PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01144 PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01145 PFNGLUNIFORM3IPROC glUniform3i;
01146 PFNGLUNIFORM3IVPROC glUniform3iv;
01147 PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01148 PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01149 PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01150 PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01151 PFNGLUNIFORM3UIPROC glUniform3ui;
01152 PFNGLUNIFORM3UIVPROC glUniform3uiv;
01153 PFNGLUNIFORM4DPROC glUniform4d;
01154 PFNGLUNIFORM4DVPROC glUniform4dv;
01155 PFNGLUNIFORM4FPROC glUniform4f;
01156 PFNGLUNIFORM4FVPROC glUniform4fv;
01157 PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01158 PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01159 PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01160 PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01161 PFNGLUNIFORM4IPROC glUniform4i;
01162 PFNGLUNIFORM4IVPROC glUniform4iv;
01163 PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01164 PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01165 PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01166 PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01167 PFNGLUNIFORM4UIPROC glUniform4ui;
01168 PFNGLUNIFORM4UIVPROC glUniform4uiv;
01169 PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01170 PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01171 PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01172 PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01173 PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01174 PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01175 PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01176 PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01177 PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01178 PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01179 PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01180 PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01181 PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01182 PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01183 PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01184 PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01185 PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01186 PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01187 PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01188 PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01189 PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
01190 PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01191 PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01192 PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01193 PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01194 PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01195 PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01196 PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01197 PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01198 PFNGLUSEPROGRAMPROC glUseProgram;
01199 PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01200 PFNGLUSESHADERPROGRAMEXTPROC glUseProgramEXT;
01201 PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01202 PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01203 PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01204 PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01205 PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01206 PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01207 PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01208 PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01209 PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01210 PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01211 PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01212 PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01213 PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01214 PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01215 PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01216 PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01217 PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01218 PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01219 PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01220 PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01221 PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
```

```
01222 PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01223 PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01224 PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01225 PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01226 PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01227 PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01228 PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01229 PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01230 PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01231 PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01232 PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01233 PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01234 PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01235 PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01236 PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01237 PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01238 PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01239 PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01240 PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01241 PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01242 PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01243 PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01244 PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01245 PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01246 PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01247 PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01248 PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01249 PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01250 PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01251 PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01252 PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01253 PFNGLVERTEXATTRIB4IPROC glVertexAttrib4iv;
01254 PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01255 PFNGLVERTEXATTRIB4NIPROC glVertexAttrib4Niv;
01256 PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01257 PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01258 PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubv;
01259 PFNGLVERTEXATTRIB4NUIPROC glVertexAttrib4Nuiv;
01260 PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01261 PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01262 PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01263 PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01264 PFNGLVERTEXATTRIB4UIPROC glVertexAttrib4uiv;
01265 PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01266 PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01267 PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01268 PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01269 PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01270 PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01271 PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01272 PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01273 PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01274 PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01275 PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01276 PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;
01277 PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01278 PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01279 PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01280 PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01281 PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01282 PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01283 PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01284 PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01285 PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01286 PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01287 PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01288 PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01289 PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01290 PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01291 PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01292 PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01293 PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01294 PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01295 PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01296 PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01297 PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01298 PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01299 PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01300 PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01301 PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01302 PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01303 PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01304 PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01305 PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01306 PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01307 PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01308 PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
```

```

01309 PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01310 PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01311 PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01312 PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01313 PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01314 PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01315 PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01316 PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01317 PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01318 PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01319 PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01320 PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01321 PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01322 PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01323 PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01324 PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01325 PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01326 PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01327 PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01328 PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01329 PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01330 PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01331 PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01332 PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01333 PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01334 PFNGLVIEWPORTARRAYVPROC glViewportArrayv;
01335 PFNGLVIEWPORTINDEXEDFPROC glViewportIndexdf;
01336 PFNGLVIEWPORTINDEXEDFVPROC glViewportIndexdfv;
01337 PFNGLVIEWPORTPOSITIONWSCALENVPROC glViewportPositionWScaleNV;
01338 PFNGLVIEWPORTPROC glViewport;
01339 PFNGLVIEWPORTSWIZZLENVPROC glViewportSwizzleNV;
01340 PFNGLWAITSYNCPROC glWaitSync;
01341 PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01342 PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01343 PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01344 #endif
01345
01347
01349 GLint gg::ggBufferAlignment (0);
01350
01351 //
01352 // ゲームグラフィックス特論の都合にもとづく初期化
01353 //
01354 void gg::ggInit ()
01355 {
01356     // すでにこの関数が実行されていたら以降の処理を行わない
01357     if (ggBufferAlignment) return;
01358
01359     // macOS 以外で OpenGL 3.2 以降の API を取得する
01360     #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
01361         glActiveProgramEXT = PFNGLACTIVEPROGRAMEXTPROC (glfwGetProcAddress ("glActiveProgramEXT"));
01362         glActiveShaderProgram = PFNGLACTIVESHADERPROGRAMPROC (glfwGetProcAddress ("glActiveShaderProgram"));
01363         glActiveTexture = PFNGLACTIVETEXTUREPROC (glfwGetProcAddress ("glActiveTexture"));
01364         glApplyFramebufferAttachmentCMAAINTEL =
01365             PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC (glfwGetProcAddress ("glApplyFramebufferAttachmentCMAAINTEL"));
01366         glAttachShader = PFNGLATTACHSHADERPROC (glfwGetProcAddress ("glAttachShader"));
01367         glBeginConditionalRender =
01368             PFNGLBEGINCONDITIONALRENDERPROC (glfwGetProcAddress ("glBeginConditionalRender"));
01369         glBeginConditionalRenderNV =
01370             PFNGLBEGINCONDITIONALRENDERNVPROC (glfwGetProcAddress ("glBeginConditionalRenderNV"));
01371         glBeginPerfMonitorAMD = PFNGLBEGINPERFMONITORAMDPROC (glfwGetProcAddress ("glBeginPerfMonitorAMD"));
01372         glBeginPerfQueryINTEL = PFNGLBEGINPERFQUERYINTELPROC (glfwGetProcAddress ("glBeginPerfQueryINTEL"));
01373         glBeginQuery = PFNGLBEGINQUERYPROC (glfwGetProcAddress ("glBeginQuery"));
01374         glBeginQueryIndexed = PFNGLBEGINQUERYINDEXEDPROC (glfwGetProcAddress ("glBeginQueryIndexed"));
01375         glBeginTransformFeedback =
01376             PFNGLBEGINTRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glBeginTransformFeedback"));
01377         glBindAttribLocation = PFNGLBINDATTRIBLOCATIONPROC (glfwGetProcAddress ("glBindAttribLocation"));
01378         glBindBuffer = PFNGLBINDBUFFERPROC (glfwGetProcAddress ("glBindBuffer"));
01379         glBindBufferBase = PFNGLBINDBUFFERBASEPROC (glfwGetProcAddress ("glBindBufferBase"));
01380         glBindBufferRange = PFNGLBINDBUFFERRANGEPROC (glfwGetProcAddress ("glBindBufferRange"));
01381         glBindBuffersBase = PFNGLBINDBUFFERSBASEPROC (glfwGetProcAddress ("glBindBuffersBase"));
01382         glBindBuffersRange = PFNGLBINDBUFFERSRANGEPROC (glfwGetProcAddress ("glBindBuffersRange"));
01383         glBindFragDataLocation =
01384             PFNGLBINDFRAGDATALOCATIONPROC (glfwGetProcAddress ("glBindFragDataLocation"));
01385         glBindFragDataLocationIndexed =
01386             PFNGLBINDFRAGDATALOCATIONINDEXEDPROC (glfwGetProcAddress ("glBindFragDataLocationIndexed"));
01387         glBindFramebuffer = PFNGLBINDFRAMEBUFFERPROC (glfwGetProcAddress ("glBindFramebuffer"));
01388         glBindImageTexture = PFNGLBINDIMAGETEXTUREPROC (glfwGetProcAddress ("glBindImageTexture"));
01389         glBindImageTextures = PFNGLBINDIMAGETEXTURESPROC (glfwGetProcAddress ("glBindImageTextures"));
01390         glBindMultiTextureEXT = PFNGLBINDMULTITEXTUREEXTPROC (glfwGetProcAddress ("glBindMultiTextureEXT"));
01391         glBindProgramPipeline = PFNGLBINDPROGRAMPIPELINEPROC (glfwGetProcAddress ("glBindProgramPipeline"));
01392         glBindRenderbuffer = PFNGLBINDRENDERBUFFERPROC (glfwGetProcAddress ("glBindRenderbuffer"));
01393         glBindSampler = PFNGLBINDSAMPLERPROC (glfwGetProcAddress ("glBindSampler"));
01394         glBindSamplers = PFNGLBINDSAMPLERSPROC (glfwGetProcAddress ("glBindSamplers"));
01395         glBindTexture = PFNGLBINDTEXTUREPROC (glfwGetProcAddress ("glBindTexture"));
01396         glBindTextureUnit = PFNGLBINDTEXTUREUNITPROC (glfwGetProcAddress ("glBindTextureUnit"));
01397         glBindTextures = PFNGLBINDTEXTURESPROC (glfwGetProcAddress ("glBindTextures"));

```

```

01392     glBindTransformFeedback =
PFNGLBINDTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glBindTransformFeedback"));
01393     glBindVertexArray = PFNGLBINDVERTEXARRAYPROC (glfwGetProcAddress("glBindVertexArray"));
01394     glBindVertexBuffer = PFNGLBINDVERTEXBUFFERPROC (glfwGetProcAddress("glBindVertexBuffer"));
01395     glBindVertexBuffers = PFNGLBINDVERTEXBUFFERSPROC (glfwGetProcAddress("glBindVertexBuffers"));
01396     glBlendBarrierKHR = PFNGLBLENDARRIERKHRPROC (glfwGetProcAddress("glBlendBarrierKHR"));
01397     glBlendBarrierNV = PFNGLBLENDBARRIERNVPROC (glfwGetProcAddress("glBlendBarrierNV"));
01398     glBlendColor = PFNGLBLENDCOLORPROC (glfwGetProcAddress("glBlendColor"));
01399     glBlendEquation = PFNGLBLEND EQUATIONPROC (glfwGetProcAddress("glBlendEquation"));
01400     glBlendEquationSeparate =
PFNGLBLEND EQUATIONSEPARATEPROC (glfwGetProcAddress("glBlendEquationSeparate"));
01401     glBlendEquationSeparatei =
PFNGLBLEND EQUATIONSEPARATEIPROC (glfwGetProcAddress("glBlendEquationSeparatei"));
01402     glBlendEquationSeparateiARB =
PFNGLBLEND EQUATIONSEPARATEIARBPROC (glfwGetProcAddress("glBlendEquationSeparateiARB"));
01403     glBlendEquationi = PFNGLBLEND EQUATIONIPROC (glfwGetProcAddress("glBlendEquationi"));
01404     glBlendEquationiARB = PFNGLBLEND EQUATIONIARBPROC (glfwGetProcAddress("glBlendEquationiARB"));
01405     glBlendFunc = PFNGLBLEND FUNCPROC (glfwGetProcAddress("glBlendFunc"));
01406     glBlendFuncSeparate = PFNGLBLEND FUNCSEPARATEPROC (glfwGetProcAddress("glBlendFuncSeparate"));
01407     glBlendFuncSeparatei = PFNGLBLEND FUNCSEPARATEIPROC (glfwGetProcAddress("glBlendFuncSeparatei"));
01408     glBlendFuncSeparateiARB =
PFNGLBLEND FUNCSEPARATEIARBPROC (glfwGetProcAddress("glBlendFuncSeparateiARB"));
01409     glBlendFunci = PFNGLBLEND FUNCIPROC (glfwGetProcAddress("glBlendFunci"));
01410     glBlendFunciARB = PFNGLBLEND FUNCIARBPROC (glfwGetProcAddress("glBlendFunciARB"));
01411     glBlendParameteriNV = PFNGLBLEND PARAMETERINVPROC (glfwGetProcAddress("glBlendParameteriNV"));
01412     glBlitFramebuffer = PFNGLBLITFRAMEBUFFERPROC (glfwGetProcAddress("glBlitFramebuffer"));
01413     glBlitNamedFramebuffer =
PFNGLBLITNAMEDFRAMEBUFFERPROC (glfwGetProcAddress("glBlitNamedFramebuffer"));
01414     glBufferAddressRangeNV =
PFNGLBUFFERADDRESSRANGENVPROC (glfwGetProcAddress("glBufferAddressRangeNV"));
01415     glBufferData = PFNGLBUFFERDATAPROC (glfwGetProcAddress("glBufferData"));
01416     glBufferPageCommitmentARB =
PFNGLBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress("glBufferPageCommitmentARB"));
01417     glBufferStorage = PFNGLBUFFERSTORAGEPROC (glfwGetProcAddress("glBufferStorage"));
01418     glBufferSubData = PFNGLBUFFERSUBDATAPROC (glfwGetProcAddress("glBufferSubData"));
01419     glCallCommandListNV = PFNGLCALLCOMMANDLISTNVPROC (glfwGetProcAddress("glCallCommandListNV"));
01420     glCheckFramebufferStatus =
PFNGLCHECKFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckFramebufferStatus"));
01421     glCheckNamedFramebufferStatus =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC (glfwGetProcAddress("glCheckNamedFramebufferStatus"));
01422     glCheckNamedFramebufferStatusEXT =
PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC (glfwGetProcAddress("glCheckNamedFramebufferStatusEXT"));
01423     glClampColor = PFNGLCLAMP COLORPROC (glfwGetProcAddress("glClampColor"));
01424     glClear = PFNGLCLEARPROC (glfwGetProcAddress("glClear"));
01425     glClearBufferData = PFNGLCLEARBUFFERDATAPROC (glfwGetProcAddress("glClearBufferData"));
01426     glClearBufferSubData = PFNGLCLEARBUFFERSUBDATAPROC (glfwGetProcAddress("glClearBufferSubData"));
01427     glClearBufferfi = PFNGLCLEARBUFFERFIPROC (glfwGetProcAddress("glClearBufferfi"));
01428     glClearBufferfv = PFNGLCLEARBUFFERFVPROC (glfwGetProcAddress("glClearBufferfv"));
01429     glClearBufferiv = PFNGLCLEARBUFFERIVPROC (glfwGetProcAddress("glClearBufferiv"));
01430     glClearBufferuiv = PFNGLCLEARBUFFERUIVPROC (glfwGetProcAddress("glClearBufferuiv"));
01431     glClearColor = PFNGLCLEARCOLORPROC (glfwGetProcAddress("glClearColor"));
01432     glClearDepth = PFNGLCLEARDEPTHPROC (glfwGetProcAddress("glClearDepth"));
01433     glClearDepthf = PFNGLCLEARDEPTHFPROC (glfwGetProcAddress("glClearDepthf"));
01434     glClearNamedBufferData =
PFNGLCLEARNAMEDBUFFERDATAPROC (glfwGetProcAddress("glClearNamedBufferData"));
01435     glClearNamedBufferDataEXT =
PFNGLCLEARNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferDataEXT"));
01436     glClearNamedBufferSubData =
PFNGLCLEARNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress("glClearNamedBufferSubData"));
01437     glClearNamedBufferSubDataEXT =
PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress("glClearNamedBufferSubDataEXT"));
01438     glClearNamedFramebufferfi =
PFNGLCLEARNAMEDFRAMEBUFFERFIPROC (glfwGetProcAddress("glClearNamedFramebufferfi"));
01439     glClearNamedFramebufferfv =
PFNGLCLEARNAMEDFRAMEBUFFERFVPROC (glfwGetProcAddress("glClearNamedFramebufferfv"));
01440     glClearNamedFramebufferiv =
PFNGLCLEARNAMEDFRAMEBUFFERIVPROC (glfwGetProcAddress("glClearNamedFramebufferiv"));
01441     glClearNamedBufferuiv =
PFNGLCLEARNAMEDFRAMEBUFFERUIVPROC (glfwGetProcAddress("glClearNamedBufferuiv"));
01442     glClearStencil = PFNGLCLEARSTENCILPROC (glfwGetProcAddress("glClearStencil"));
01443     glClearTexImage = PFNGLCLEARTEXIMAGEPROC (glfwGetProcAddress("glClearTexImage"));
01444     glClearTexSubImage = PFNGLCLEARTEXSUBIMAGEPROC (glfwGetProcAddress("glClearTexSubImage"));
01445     glClientAttribDefaultEXT =
PFNGLCLIENTATTRIBDEFAULTEXTPROC (glfwGetProcAddress("glClientAttribDefaultEXT"));
01446     glClientWaitSync = PFNGLCLIENTWAITSYNCPROC (glfwGetProcAddress("glClientWaitSync"));
01447     glClipControl = PFNGLCLIPCONTROLPROC (glfwGetProcAddress("glClipControl"));
01448     glColorFormatNV = PFNGLCOLORFORMATNVPROC (glfwGetProcAddress("glColorFormatNV"));
01449     glColorMask = PFNGLCOLORMASKPROC (glfwGetProcAddress("glColorMask"));
01450     glColorMaski = PFNGLCOLORMASKIPROC (glfwGetProcAddress("glColorMaski"));
01451     glCommandListSegmentsNV =
PFNGLCOMMANDLISTSEGMENTSNVPROC (glfwGetProcAddress("glCommandListSegmentsNV"));
01452     glCompileCommandListNV =
PFNGLCOMPILECOMMANDLISTNVPROC (glfwGetProcAddress("glCompileCommandListNV"));
01453     glCompileShader = PFNGLCOMPILESHADERPROC (glfwGetProcAddress("glCompileShader"));
01454     glCompileShaderIncludeARB =
PFNGLCOMPILESHADERINCLUDEARBPROC (glfwGetProcAddress("glCompileShaderIncludeARB"));
01455     glCompressedMultiTexImage1DEXT =

```

```
PFNGLCOMPRESSEDMULTITEXIMAGE1DTEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage1DTEXT"));
01456 glCompressedMultiTexImage2DTEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE2DTEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage2DTEXT"));
01457 glCompressedMultiTexImage3DTEXT =
PFNGLCOMPRESSEDMULTITEXIMAGE3DTEXTPROC (glfwGetProcAddress ("glCompressedMultiTexImage3DTEXT"));
01458 glCompressedMultiTexSubImage1DTEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DTEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage1DTEXT"));
01459 glCompressedMultiTexSubImage2DTEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DTEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage2DTEXT"));
01460 glCompressedMultiTexSubImage3DTEXT =
PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DTEXTPROC (glfwGetProcAddress ("glCompressedMultiTexSubImage3DTEXT"));
01461 glCompressedTexImage1D =
PFNGLCOMPRESSEDTEXTIMAGE1DPROC (glfwGetProcAddress ("glCompressedTexImage1D"));
01462 glCompressedTexImage2D =
PFNGLCOMPRESSEDTEXTIMAGE2DPROC (glfwGetProcAddress ("glCompressedTexImage2D"));
01463 glCompressedTexImage3D =
PFNGLCOMPRESSEDTEXTIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexImage3D"));
01464 glCompressedTexSubImage1D =
PFNGLCOMPRESSEDTEXTSUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTexSubImage1D"));
01465 glCompressedTexSubImage2D =
PFNGLCOMPRESSEDTEXTSUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTexSubImage2D"));
01466 glCompressedTexSubImage3D =
PFNGLCOMPRESSEDTEXTSUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTexSubImage3D"));
01467 glCompressedTextureImage1DTEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE1DTEXTPROC (glfwGetProcAddress ("glCompressedTextureImage1DTEXT"));
01468 glCompressedTextureImage2DTEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE2DTEXTPROC (glfwGetProcAddress ("glCompressedTextureImage2DTEXT"));
01469 glCompressedTextureImage3DTEXT =
PFNGLCOMPRESSEDTEXTUREIMAGE3DTEXTPROC (glfwGetProcAddress ("glCompressedTextureImage3DTEXT"));
01470 glCompressedTextureSubImage1D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCompressedTextureSubImage1D"));
01471 glCompressedTextureSubImage1DTEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE1DTEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage1DTEXT"));
01472 glCompressedTextureSubImage2D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCompressedTextureSubImage2D"));
01473 glCompressedTextureSubImage2DTEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE2DTEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage2DTEXT"));
01474 glCompressedTextureSubImage3D =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCompressedTextureSubImage3D"));
01475 glCompressedTextureSubImage3DTEXT =
PFNGLCOMPRESSEDTEXTURESUBIMAGE3DTEXTPROC (glfwGetProcAddress ("glCompressedTextureSubImage3DTEXT"));
01476 glConservativeRasterParameterfNV =
PFNGLCONSERVATIVERASTERPARAMETERFNVPROC (glfwGetProcAddress ("glConservativeRasterParameterfNV"));
01477 glConservativeRasterParameteriNV =
PFNGLCONSERVATIVERASTERPARAMETERINVPROC (glfwGetProcAddress ("glConservativeRasterParameteriNV"));
01478 glCopyBufferSubData = PFNGLCOPYBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyBufferSubData"));
01479 glCopyImageSubData = PFNGLCOPYIMAGESUBDATAPROC (glfwGetProcAddress ("glCopyImageSubData"));
01480 glCopyMultiTexImage1DTEXT =
PFNGLCOPYMULTITEXIMAGE1DTEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage1DTEXT"));
01481 glCopyMultiTexImage2DTEXT =
PFNGLCOPYMULTITEXIMAGE2DTEXTPROC (glfwGetProcAddress ("glCopyMultiTexImage2DTEXT"));
01482 glCopyMultiTexSubImage1DTEXT =
PFNGLCOPYMULTITEXSUBIMAGE1DTEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage1DTEXT"));
01483 glCopyMultiTexSubImage2DTEXT =
PFNGLCOPYMULTITEXSUBIMAGE2DTEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage2DTEXT"));
01484 glCopyMultiTexSubImage3DTEXT =
PFNGLCOPYMULTITEXSUBIMAGE3DTEXTPROC (glfwGetProcAddress ("glCopyMultiTexSubImage3DTEXT"));
01485 glCopyNamedBufferSubData =
PFNGLCOPYNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glCopyNamedBufferSubData"));
01486 glCopyPathNV = PFNGLCOPYPATHNVPROC (glfwGetProcAddress ("glCopyPathNV"));
01487 glCopyTexImage1D = PFNGLCOPYTEXIMAGE1DPROC (glfwGetProcAddress ("glCopyTexImage1D"));
01488 glCopyTexImage2D = PFNGLCOPYTEXIMAGE2DPROC (glfwGetProcAddress ("glCopyTexImage2D"));
01489 glCopyTexSubImage1D = PFNGLCOPYTEXSUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTexSubImage1D"));
01490 glCopyTexSubImage2D = PFNGLCOPYTEXSUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTexSubImage2D"));
01491 glCopyTexSubImage3D = PFNGLCOPYTEXSUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTexSubImage3D"));
01492 glCopyTextureImage1DTEXT =
PFNGLCOPYTEXTUREIMAGE1DTEXTPROC (glfwGetProcAddress ("glCopyTextureImage1DTEXT"));
01493 glCopyTextureImage2DTEXT =
PFNGLCOPYTEXTUREIMAGE2DTEXTPROC (glfwGetProcAddress ("glCopyTextureImage2DTEXT"));
01494 glCopyTextureSubImage1D =
PFNGLCOPYTEXTURESUBIMAGE1DPROC (glfwGetProcAddress ("glCopyTextureSubImage1D"));
01495 glCopyTextureSubImage1DTEXT =
PFNGLCOPYTEXTURESUBIMAGE1DTEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage1DTEXT"));
01496 glCopyTextureSubImage2D =
PFNGLCOPYTEXTURESUBIMAGE2DPROC (glfwGetProcAddress ("glCopyTextureSubImage2D"));
01497 glCopyTextureSubImage2DTEXT =
PFNGLCOPYTEXTURESUBIMAGE2DTEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage2DTEXT"));
01498 glCopyTextureSubImage3D =
PFNGLCOPYTEXTURESUBIMAGE3DPROC (glfwGetProcAddress ("glCopyTextureSubImage3D"));
01499 glCopyTextureSubImage3DTEXT =
PFNGLCOPYTEXTURESUBIMAGE3DTEXTPROC (glfwGetProcAddress ("glCopyTextureSubImage3DTEXT"));
01500 glCoverFillPathInstancedNV =
PFNGLCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glCoverFillPathInstancedNV"));
01501 glCoverFillPathNV = PFNGLCOVERFILLPATHNVPROC (glfwGetProcAddress ("glCoverFillPathNV"));
01502 glCoverStrokePathInstancedNV =
PFNGLCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glCoverStrokePathInstancedNV"));
01503 glCoverStrokePathNV = PFNGLCOVERSTROKEPATHNVPROC (glfwGetProcAddress ("glCoverStrokePathNV"));
```



```

01504     glCoverageModulationNV =
PFNGLCOVERAGEMODULATIONNVPROC (glfwGetProcAddress("glCoverageModulationNV"));
01505     glCoverageModulationTableNV =
PFNGLCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress("glCoverageModulationTableNV"));
01506     glCreateBuffers = PFNGLCREATEBUFFERSPROC (glfwGetProcAddress("glCreateBuffers"));
01507     glCreateCommandListsNV =
PFNGLCREATECOMMANDLISTSNVPROC (glfwGetProcAddress("glCreateCommandListsNV"));
01508     glCreateFramebuffers = PFNGLCREATEFRAMEBUFFERSPROC (glfwGetProcAddress("glCreateFramebuffers"));
01509     glCreatePerfQueryINTEL =
PFNGLCREATEPERFQUERYINTELPROC (glfwGetProcAddress("glCreatePerfQueryINTEL"));
01510     glCreateProgram = PFNGLCREATEPROGRAMPROC (glfwGetProcAddress("glCreateProgram"));
01511     glCreateProgramPipelines =
PFNGLCREATEPROGRAMPIPELINESPROC (glfwGetProcAddress("glCreateProgramPipelines"));
01512     glCreateQueries = PFNGLCREATEQUERIESPROC (glfwGetProcAddress("glCreateQueries"));
01513     glCreateRenderbuffers = PFNGLCREATERENDERBUFFERSPROC (glfwGetProcAddress("glCreateRenderbuffers"));
01514     glCreateSamplers = PFNGLCREATESAMPLERSPROC (glfwGetProcAddress("glCreateSamplers"));
01515     glCreateShader = PFNGLCREATESHADERPROC (glfwGetProcAddress("glCreateShader"));
01516     glCreateShaderProgramEXT =
PFNGLCREATESHADERPROGRAMEXTPROC (glfwGetProcAddress("glCreateShaderProgramEXT"));
01517     glCreateShaderProgramv =
PFNGLCREATESHADERPROGRAMVPROC (glfwGetProcAddress("glCreateShaderProgramv"));
01518     glCreateStatesNV = PFNGLCREATESTATESNVPROC (glfwGetProcAddress("glCreateStatesNV"));
01519     glCreateSyncFromCLeventARB =
PFNGLCREATESYNCFROMCLEVENTARBPROC (glfwGetProcAddress("glCreateSyncFromCLeventARB"));
01520     glCreateTextures = PFNGLCREATETEXTURESPROC (glfwGetProcAddress("glCreateTextures"));
01521     glCreateTransformFeedbacks =
PFNGLCREATETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glCreateTransformFeedbacks"));
01522     glCreateVertexArrays = PFNGLCREATEVERTEXARRAYSPROC (glfwGetProcAddress("glCreateVertexArrays"));
01523     glCullFace = PFNGLCULLFACEPROC (glfwGetProcAddress("glCullFace"));
01524     glDebugMessageCallback =
PFNGLDEBUGMESSAGECALLBACKPROC (glfwGetProcAddress("glDebugMessageCallback"));
01525     glDebugMessageCallbackARB =
PFNGLDEBUGMESSAGECALLBACKARBPROC (glfwGetProcAddress("glDebugMessageCallbackARB"));
01526     glDebugMessageControl = PFNGLDEBUGMESSAGECONTROLPROC (glfwGetProcAddress("glDebugMessageControl"));
01527     glDebugMessageControlARB =
PFNGLDEBUGMESSAGECONTROLARBPROC (glfwGetProcAddress("glDebugMessageControlARB"));
01528     glDebugMessageInsert = PFNGLDEBUGMESSAGEINSERTPROC (glfwGetProcAddress("glDebugMessageInsert"));
01529     glDebugMessageInsertARB =
PFNGLDEBUGMESSAGEINSERTARBPROC (glfwGetProcAddress("glDebugMessageInsertARB"));
01530     glDeleteBuffers = PFNGLDELETEBUFFERSPROC (glfwGetProcAddress("glDeleteBuffers"));
01531     glDeleteCommandListsNV =
PFNGLDELETETEXTURESPROC (glfwGetProcAddress("glDeleteCommandListsNV"));
01532     glDeleteFramebuffers = PFNGLDELETEFRAMEBUFFERSPROC (glfwGetProcAddress("glDeleteFramebuffers"));
01533     glDeleteNamedStringARB =
PFNGLDELETENAMEDSTRINGARBPROC (glfwGetProcAddress("glDeleteNamedStringARB"));
01534     glDeletePathsNV = PFNGLDELETEPATHSNVPROC (glfwGetProcAddress("glDeletePathsNV"));
01535     glDeletePerfMonitorsAMD =
PFNGLDELETEPERFMONITORSAMDPROC (glfwGetProcAddress("glDeletePerfMonitorsAMD"));
01536     glDeletePerfQueryINTEL =
PFNGLDELETEPERFQUERYINTELPROC (glfwGetProcAddress("glDeletePerfQueryINTEL"));
01537     glDeleteProgram = PFNGLDELETEPROGRAMPROC (glfwGetProcAddress("glDeleteProgram"));
01538     glDeleteProgramPipelines =
PFNGLDELETEPROGRAMPIPELINESPROC (glfwGetProcAddress("glDeleteProgramPipelines"));
01539     glDeleteQueries = PFNGLDELETEQUERIESPROC (glfwGetProcAddress("glDeleteQueries"));
01540     glDeleteRenderbuffers = PFNGLDELETETEXTURESPROC (glfwGetProcAddress("glDeleteRenderbuffers"));
01541     glDeleteSamplers = PFNGLDELETESAMPLERSPROC (glfwGetProcAddress("glDeleteSamplers"));
01542     glDeleteShader = PFNGLDELETESHADERPROC (glfwGetProcAddress("glDeleteShader"));
01543     glDeleteStatesNV = PFNGLDELETETEXTURESPROC (glfwGetProcAddress("glDeleteStatesNV"));
01544     glDeleteSync = PFNGLDELETESYNCPROC (glfwGetProcAddress("glDeleteSync"));
01545     glDeleteTextures = PFNGLDELETETEXTURESPROC (glfwGetProcAddress("glDeleteTextures"));
01546     glDeleteTransformFeedbacks =
PFNGLDELETETRANSFORMFEEDBACKSPROC (glfwGetProcAddress("glDeleteTransformFeedbacks"));
01547     glDeleteVertexArrays = PFNGLDELETEVERTEXARRAYSPROC (glfwGetProcAddress("glDeleteVertexArrays"));
01548     glDepthFunc = PFNGLDEPTHFUNCPROC (glfwGetProcAddress("glDepthFunc"));
01549     glDepthMask = PFNGLDEPTHMASKPROC (glfwGetProcAddress("glDepthMask"));
01550     glDepthRange = PFNGLDEPTHRANGEPROC (glfwGetProcAddress("glDepthRange"));
01551     glDepthRangeArrayv = PFNGLDEPTHRANGEARRAYVPROC (glfwGetProcAddress("glDepthRangeArrayv"));
01552     glDepthRangeIndexed = PFNGLDEPTHRANGEINDEXEDPROC (glfwGetProcAddress("glDepthRangeIndexed"));
01553     glDepthRangef = PFNGLDEPTHRANGEFPROC (glfwGetProcAddress("glDepthRangef"));
01554     glDetachShader = PFNGLDETACHSHADERPROC (glfwGetProcAddress("glDetachShader"));
01555     glDisable = PFNGLDISABLEPROC (glfwGetProcAddress("glDisable"));
01556     glDisableClientStateIndexedEXT =
PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glDisableClientStateIndexedEXT"));
01557     glDisableClientStateiEXT =
PFNGLDISABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glDisableClientStateiEXT"));
01558     glDisableIndexedEXT = PFNGLDISABLEINDEXEDEXTPROC (glfwGetProcAddress("glDisableIndexedEXT"));
01559     glDisableVertexArrayAttrib =
PFNGLDISABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glDisableVertexArrayAttrib"));
01560     glDisableVertexArrayAttribEXT =
PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glDisableVertexArrayAttribEXT"));
01561     glDisableVertexArrayEXT =
PFNGLDISABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glDisableVertexArrayEXT"));
01562     glDisableVertexAttribArray =
PFNGLDISABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glDisableVertexAttribArray"));
01563     glDisablei = PFNGLDISABLEIPROC (glfwGetProcAddress("glDisablei"));
01564     glDispatchCompute = PFNGLDISPATCHCOMPUTEPROC (glfwGetProcAddress("glDispatchCompute"));
01565     glDispatchComputeGroupSizeARB =

```

```
PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC (glfwGetProcAddress("glDispatchComputeGroupSizeARB"));
01566 glDispatchComputeIndirect =
PFNGLDISPATCHCOMPUTEINDIRECTPROC (glfwGetProcAddress("glDispatchComputeIndirect"));
01567 glDrawArrays = PFNGLDRAWARRAYSPROC (glfwGetProcAddress("glDrawArrays"));
01568 glDrawArraysIndirect = PFNGLDRAWARRAYSINDIRECTPROC (glfwGetProcAddress("glDrawArraysIndirect"));
01569 glDrawArraysInstanced = PFNGLDRAWARRAYSINSTANCEDPROC (glfwGetProcAddress("glDrawArraysInstanced"));
01570 glDrawArraysInstancedARB =
PFNGLDRAWARRAYSINSTANCEDARBPROC (glfwGetProcAddress("glDrawArraysInstancedARB"));
01571 glDrawArraysInstancedBaseInstance =
PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress("glDrawArraysInstancedBaseInstance"));
01572 glDrawArraysInstancedEXT =
PFNGLDRAWARRAYSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawArraysInstancedEXT"));
01573 glDrawBuffer = PFNGLDRAWBUFFERPROC (glfwGetProcAddress("glDrawBuffer"));
01574 glDrawBuffers = PFNGLDRAWBUFFERSPROC (glfwGetProcAddress("glDrawBuffers"));
01575 glDrawCommandsAddressNV =
PFNGLDRAWCOMMANDSADDRESSNVPROC (glfwGetProcAddress("glDrawCommandsAddressNV"));
01576 glDrawCommandsNV = PFNGLDRAWCOMMANDSNVPROC (glfwGetProcAddress("glDrawCommandsNV"));
01577 glDrawCommandsStatesAddressNV =
PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC (glfwGetProcAddress("glDrawCommandsStatesAddressNV"));
01578 glDrawCommandsStatesNV =
PFNGLDRAWCOMMANDSSTATESNVPROC (glfwGetProcAddress("glDrawCommandsStatesNV"));
01579 glDrawElements = PFNGLDRAWELEMENTSPROC (glfwGetProcAddress("glDrawElements"));
01580 glDrawElementsBaseVertex =
PFNGLDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glDrawElementsBaseVertex"));
01581 glDrawElementsIndirect =
PFNGLDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress("glDrawElementsIndirect"));
01582 glDrawElementsInstanced =
PFNGLDRAWELEMENTSINSTANCEDPROC (glfwGetProcAddress("glDrawElementsInstanced"));
01583 glDrawElementsInstancedARB =
PFNGLDRAWELEMENTSINSTANCEDARBPROC (glfwGetProcAddress("glDrawElementsInstancedARB"));
01584 glDrawElementsInstancedBaseInstance =
PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC (glfwGetProcAddress("glDrawElementsInstancedBaseInstance"));
01585 glDrawElementsInstancedBaseVertex =
PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC (glfwGetProcAddress("glDrawElementsInstancedBaseVertex"));
01586 glDrawElementsInstancedBaseVertexBaseInstance =
PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC (glfwGetProcAddress("glDrawElementsInstancedBaseVertexBaseInstance"));
01587 glDrawElementsInstancedEXT =
PFNGLDRAWELEMENTSINSTANCEDEXTPROC (glfwGetProcAddress("glDrawElementsInstancedEXT"));
01588 glDrawRangeElements = PFNGLDRAWRANGELEMENTSPROC (glfwGetProcAddress("glDrawRangeElements"));
01589 glDrawRangeElementsBaseVertex =
PFNGLDRAWRANGELEMENTSBASEVERTEXPROC (glfwGetProcAddress("glDrawRangeElementsBaseVertex"));
01590 glDrawTransformFeedback =
PFNGLDRAWTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glDrawTransformFeedback"));
01591 glDrawTransformFeedbackInstanced =
PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackInstanced"));
01592 glDrawTransformFeedbackStream =
PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC (glfwGetProcAddress("glDrawTransformFeedbackStream"));
01593 glDrawTransformFeedbackStreamInstanced =
PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC (glfwGetProcAddress("glDrawTransformFeedbackStreamInstanced"));
01594 glDrawVkImageNV = PFNGLDRAWVKIMAGENVPROC (glfwGetProcAddress("glDrawVkImageNV"));
01595 glEdgeFlagFormatNV = PFNGLEDGEFLAGFORMATNVPROC (glfwGetProcAddress("glEdgeFlagFormatNV"));
01596 glEnable = PFNGLENABLEPROC (glfwGetProcAddress("glEnable"));
01597 glEnableClientStateIndexedEXT =
PFNGLENABLECLIENTSTATEINDEXEDEXTPROC (glfwGetProcAddress("glEnableClientStateIndexedEXT"));
01598 glEnableClientStateiEXT =
PFNGLENABLECLIENTSTATEIEXTPROC (glfwGetProcAddress("glEnableClientStateiEXT"));
01599 glEnableIndexedEXT = PFNGLENABLEINDEXEDEXTPROC (glfwGetProcAddress("glEnableIndexedEXT"));
01600 glEnableVertexArrayAttrib =
PFNGLENABLEVERTEXARRAYATTRIBPROC (glfwGetProcAddress("glEnableVertexArrayAttrib"));
01601 glEnableVertexArrayAttribEXT =
PFNGLENABLEVERTEXARRAYATTRIBEXTPROC (glfwGetProcAddress("glEnableVertexArrayAttribEXT"));
01602 glEnableVertexArrayEXT =
PFNGLENABLEVERTEXARRAYEXTPROC (glfwGetProcAddress("glEnableVertexArrayEXT"));
01603 glEnableVertexAttribArray =
PFNGLENABLEVERTEXATTRIBARRAYPROC (glfwGetProcAddress("glEnableVertexAttribArray"));
01604 glEnablei = PFNGLENABLEIPROC (glfwGetProcAddress("glEnablei"));
01605 glEndConditionalRender =
PFNGLENDCONDITIONALRENDERPROC (glfwGetProcAddress("glEndConditionalRender"));
01606 glEndConditionalRenderNV =
PFNGLENDCONDITIONALRENDERNVPROC (glfwGetProcAddress("glEndConditionalRenderNV"));
01607 glEndPerfMonitorAMD = PFNGLENDPERFMONITORAMDPROC (glfwGetProcAddress("glEndPerfMonitorAMD"));
01608 glEndPerfQueryINTEL = PFNGLENDPERFQUERYINTELPROC (glfwGetProcAddress("glEndPerfQueryINTEL"));
01609 glEndQuery = PFNGLENDQUERYPROC (glfwGetProcAddress("glEndQuery"));
01610 glEndQueryIndexed = PFNGLENDQUERYINDEXEDPROC (glfwGetProcAddress("glEndQueryIndexed"));
01611 glEndTransformFeedback =
PFNGLENDTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glEndTransformFeedback"));
01612 glEvaluateDepthValuesARB =
PFNGLEVALUATEDEPTHVALUESARBPROC (glfwGetProcAddress("glEvaluateDepthValuesARB"));
01613 glFenceSync = PFNGLFENCESYNCPROC (glfwGetProcAddress("glFenceSync"));
01614 glFinish = PFNGLFINISHPROC (glfwGetProcAddress("glFinish"));
01615 glFlush = PFNGLFLUSHPROC (glfwGetProcAddress("glFlush"));
01616 glFlushMappedBufferRange =
PFNGLFLUSHMAPPEDBUFFERRANGEPROC (glfwGetProcAddress("glFlushMappedBufferRange"));
01617 glFlushMappedNamedBufferRange =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC (glfwGetProcAddress("glFlushMappedNamedBufferRange"));
01618 glFlushMappedNamedBufferRangeEXT =
PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress("glFlushMappedNamedBufferRangeEXT"));
```

```

01619     glFogCoordFormatNV = PFNGLFOGCOORDFORMATNVPROC (glfwGetProcAddress ("glFogCoordFormatNV"));
01620     glFragmentCoverageColorNV =
PFNGLFRAGMENTCOVERAGECOLORNVPROC (glfwGetProcAddress ("glFragmentCoverageColorNV"));
01621     glFramebufferDrawBufferEXT =
PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC (glfwGetProcAddress ("glFramebufferDrawBufferEXT"));
01622     glFramebufferDrawBuffersEXT =
PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC (glfwGetProcAddress ("glFramebufferDrawBuffersEXT"));
01623     glFramebufferParameteri =
PFNGLFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress ("glFramebufferParameteri"));
01624     glFramebufferReadBufferEXT =
PFNGLFRAMEBUFFERREADBUFFEREXTPROC (glfwGetProcAddress ("glFramebufferReadBufferEXT"));
01625     glFramebufferRenderbuffer =
PFNGLFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress ("glFramebufferRenderbuffer"));
01626     glFramebufferSampleLocationsfvARB =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress ("glFramebufferSampleLocationsfvARB"));
01627     glFramebufferSampleLocationsfvNV =
PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress ("glFramebufferSampleLocationsfvNV"));
01628     glFramebufferTexture = PFNGLFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress ("glFramebufferTexture"));
01629     glFramebufferTexture1D =
PFNGLFRAMEBUFFERTEXTURE1DPROC (glfwGetProcAddress ("glFramebufferTexture1D"));
01630     glFramebufferTexture2D =
PFNGLFRAMEBUFFERTEXTURE2DPROC (glfwGetProcAddress ("glFramebufferTexture2D"));
01631     glFramebufferTexture3D =
PFNGLFRAMEBUFFERTEXTURE3DPROC (glfwGetProcAddress ("glFramebufferTexture3D"));
01632     glFramebufferTextureARB =
PFNGLFRAMEBUFFERTEXTUREARBPROC (glfwGetProcAddress ("glFramebufferTextureARB"));
01633     glFramebufferTextureFaceARB =
PFNGLFRAMEBUFFERTEXTUREFACEARBPROC (glfwGetProcAddress ("glFramebufferTextureFaceARB"));
01634     glFramebufferTextureLayer =
PFNGLFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress ("glFramebufferTextureLayer"));
01635     glFramebufferTextureLayerARB =
PFNGLFRAMEBUFFERTEXTURELAYERARBPROC (glfwGetProcAddress ("glFramebufferTextureLayerARB"));
01636     glFramebufferTextureMultiviewOVR =
PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC (glfwGetProcAddress ("glFramebufferTextureMultiviewOVR"));
01637     glFrontFace = PFNGLFRONTFACEPROC (glfwGetProcAddress ("glFrontFace"));
01638     glGenBuffers = PFNGLGENBUFFERSPROC (glfwGetProcAddress ("glGenBuffers"));
01639     glGenFramebuffers = PFNGLGENFRAMEBUFFERSPROC (glfwGetProcAddress ("glGenFramebuffers"));
01640     glGenPathsNV = PFNGLGENPATHSNVPROC (glfwGetProcAddress ("glGenPathsNV"));
01641     glGenPerfMonitorsAMD = PFNGLGENPERFMONITORSAMDPROC (glfwGetProcAddress ("glGenPerfMonitorsAMD"));
01642     glGenProgramPipelines = PFNGLGENPROGRAMPIPELINESPROC (glfwGetProcAddress ("glGenProgramPipelines"));
01643     glGenQueries = PFNGLGENQUERIESPROC (glfwGetProcAddress ("glGenQueries"));
01644     glGenRenderbuffers = PFNGLGENRENDERBUFFERSPROC (glfwGetProcAddress ("glGenRenderbuffers"));
01645     glGenSamplers = PFNGLGENSAMPLERSPROC (glfwGetProcAddress ("glGenSamplers"));
01646     glGenTextures = PFNGLGENTEXTURESPROC (glfwGetProcAddress ("glGenTextures"));
01647     glGenTransformFeedbacks =
PFNGLGENTRANSFORMFEEDBACKSPROC (glfwGetProcAddress ("glGenTransformFeedbacks"));
01648     glGenVertexArrays = PFNGLGENVERTEXARRAYSPROC (glfwGetProcAddress ("glGenVertexArrays"));
01649     glGenerateMipmap = PFNGLGENERATEMIPMAPPROC (glfwGetProcAddress ("glGenerateMipmap"));
01650     glGenerateMultiTexMipmapEXT =
PFNGLGENERATEMULTITEXMIPMAPEXTPROC (glfwGetProcAddress ("glGenerateMultiTexMipmapEXT"));
01651     glGenerateTextureMipmap =
PFNGLGENERATETEXTUREMIPMAPPROC (glfwGetProcAddress ("glGenerateTextureMipmap"));
01652     glGenerateTextureMipmapEXT =
PFNGLGENERATETEXTUREMIPMAPEXTPROC (glfwGetProcAddress ("glGenerateTextureMipmapEXT"));
01653     glGetActiveAtomicCounterBufferiv =
PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC (glfwGetProcAddress ("glGetActiveAtomicCounterBufferiv"));
01654     glGetActiveAttrib = PFNGLGETACTIVEATTRIBPROC (glfwGetProcAddress ("glGetActiveAttrib"));
01655     glGetActiveSubroutineName =
PFNGLGETACTIVESUBROUTINENAMEPROC (glfwGetProcAddress ("glGetActiveSubroutineName"));
01656     glGetActiveSubroutineUniformName =
PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC (glfwGetProcAddress ("glGetActiveSubroutineUniformName"));
01657     glGetActiveSubroutineUniformiv =
PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC (glfwGetProcAddress ("glGetActiveSubroutineUniformiv"));
01658     glGetActiveUniform = PFNGLGETACTIVEUNIFORMPROC (glfwGetProcAddress ("glGetActiveUniform"));
01659     glGetActiveUniformBlockName =
PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC (glfwGetProcAddress ("glGetActiveUniformBlockName"));
01660     glGetActiveUniformBlockiv =
PFNGLGETACTIVEUNIFORMBLOCKIVPROC (glfwGetProcAddress ("glGetActiveUniformBlockiv"));
01661     glGetActiveUniformName =
PFNGLGETACTIVEUNIFORMNAMEPROC (glfwGetProcAddress ("glGetActiveUniformName"));
01662     glGetActiveUniformsiv = PFNGLGETACTIVEUNIFORMSIVPROC (glfwGetProcAddress ("glGetActiveUniformsiv"));
01663     glGetAttachedShaders = PFNGLGETATTACHEDSHADERSPROC (glfwGetProcAddress ("glGetAttachedShaders"));
01664     glGetAttribLocation = PFNGLGETATTRIBLOCATIONPROC (glfwGetProcAddress ("glGetAttribLocation"));
01665     glGetBooleanIndexedvEXT =
PFNGLGETBOOLEANINDEXEDVEXTPROC (glfwGetProcAddress ("glGetBooleanIndexedvEXT"));
01666     glGetBooleani_v = PFNGLGETBOOLEANI_VPROC (glfwGetProcAddress ("glGetBooleani_v"));
01667     glGetBooleany_v = PFNGLGETBOOLEANYVPROC (glfwGetProcAddress ("glGetBooleany_v"));
01668     glGetBufferParameteri64v =
PFNGLGETBUFFERPARAMETERI64VPROC (glfwGetProcAddress ("glGetBufferParameteri64v"));
01669     glGetBufferParameteriv =
PFNGLGETBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetBufferParameteriv"));
01670     glGetBufferParameterui64vNV =
PFNGLGETBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetBufferParameterui64vNV"));
01671     glGetBufferPointerv = PFNGLGETBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetBufferPointerv"));
01672     glGetBufferSubData = PFNGLGETBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetBufferSubData"));
01673     glGetCommandHeaderNV = PFNGLGETCOMMANDHEADERNVPROC (glfwGetProcAddress ("glGetCommandHeaderNV"));
01674     glGetCompressedMultiTexImageEXT =

```



```

PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC (glfwGetProcAddress ("glGetCompressedMultiTexImageEXT"));
01675   glGetCompressedTexImage =
PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetCompressedTexImage"));
01676   glGetCompressedTextureImage =
PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetCompressedTextureImage"));
01677   glGetCompressedTextureImageEXT =
PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC (glfwGetProcAddress ("glGetCompressedTextureImageEXT"));
01678   glGetCompressedTextureSubImage =
PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC (glfwGetProcAddress ("glGetCompressedTextureSubImage"));
01679   glGetCoverageModulationTableNV =
PFNGLGETCOVERAGEMODULATIONTABLENVPROC (glfwGetProcAddress ("glGetCoverageModulationTableNV"));
01680   glGetDebugMessageLog = PFNGLGETDEBUGMESSAGELOGPROC (glfwGetProcAddress ("glGetDebugMessageLog"));
01681   glGetDebugMessageLogARB =
PFNGLGETDEBUGMESSAGELOGARBPROC (glfwGetProcAddress ("glGetDebugMessageLogARB"));
01682   glGetDoubleIndexeddvEXT =
PFNGLGETDOUBLEINDEXEDVEXTPROC (glfwGetProcAddress ("glGetDoubleIndexeddvEXT"));
01683   glGetDoublei_v = PFNGLGETDOUBLEI_VPROC (glfwGetProcAddress ("glGetDoublei_v"));
01684   glGetDoublei_vEXT = PFNGLGETDOUBLEI_VEXTPROC (glfwGetProcAddress ("glGetDoublei_vEXT"));
01685   glGetDoublev = PFNGLGETDOUBLEVPROC (glfwGetProcAddress ("glGetDoublev"));
01686   glGetError = PFNGLGETERRORPROC (glfwGetProcAddress ("glGetError"));
01687   glGetFirstPerfQueryIdINTEL =
PFNGLGETFIRSTPERFQUERYIDINTELPROC (glfwGetProcAddress ("glGetFirstPerfQueryIdINTEL"));
01688   glGetFloatIndexeddvEXT = PFNGLGETFLOATINDEXEDVEXTPROC (glfwGetProcAddress ("glGetFloatIndexeddvEXT"));
01689   glGetFloati_v = PFNGLGETFLOATI_VPROC (glfwGetProcAddress ("glGetFloati_v"));
01690   glGetFloati_vEXT = PFNGLGETFLOATI_VEXTPROC (glfwGetProcAddress ("glGetFloati_vEXT"));
01691   glGetFloatv = PFNGLGETFLOATVPROC (glfwGetProcAddress ("glGetFloatv"));
01692   glGetFragDataIndex = PFNGLGETFRAGDATAINDEXPROC (glfwGetProcAddress ("glGetFragDataIndex"));
01693   glGetFragDataLocation = PFNGLGETFRAGDATALOCATIONPROC (glfwGetProcAddress ("glGetFragDataLocation"));
01694   glGetFramebufferAttachmentParameteriv =
PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC (glfwGetProcAddress ("glGetFramebufferAttachmentParameteriv"));
01695   glGetFramebufferParameteriv =
PFNGLGETFRAMEBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetFramebufferParameteriv"));
01696   glGetFramebufferParameterivEXT =
PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetFramebufferParameterivEXT"));
01697   glGetGraphicsResetStatus =
PFNGLGETGRAPHICSRESETSTATUSPROC (glfwGetProcAddress ("glGetGraphicsResetStatus"));
01698   glGetGraphicsResetStatusARB =
PFNGLGETGRAPHICSRESETSTATUSARBPROC (glfwGetProcAddress ("glGetGraphicsResetStatusARB"));
01699   glGetImageHandleARB = PFNGLGETIMAGEHANDLEARBPROC (glfwGetProcAddress ("glGetImageHandleARB"));
01700   glGetImageHandleNV = PFNGLGETIMAGEHANDLENVPROC (glfwGetProcAddress ("glGetImageHandleNV"));
01701   glGetInteger64i_v = PFNGLGETINTEGER64I_VPROC (glfwGetProcAddress ("glGetInteger64i_v"));
01702   glGetInteger64v = PFNGLGETINTEGER64VPROC (glfwGetProcAddress ("glGetInteger64v"));
01703   glGetIntegerIndexeddvEXT =
PFNGLGETINTEGERINDEXEDVEXTPROC (glfwGetProcAddress ("glGetIntegerIndexeddvEXT"));
01704   glGetIntegeri_v = PFNGLGETINTEGERI_VPROC (glfwGetProcAddress ("glGetIntegeri_v"));
01705   glGetIntegerui64i_vNV = PFNGLGETINTEGERUI64I_VNVPROC (glfwGetProcAddress ("glGetIntegerui64i_vNV"));
01706   glGetIntegerui64vNV = PFNGLGETINTEGERUI64VNVPROC (glfwGetProcAddress ("glGetIntegerui64vNV"));
01707   glGetIntegerv = PFNGLGETINTEGERVPROC (glfwGetProcAddress ("glGetIntegerv"));
01708   glGetInternalformatSampleivNV =
PFNGLGETINTERNALFORMATSAMPLEIVNVPROC (glfwGetProcAddress ("glGetInternalformatSampleivNV"));
01709   glGetInternalformati64v =
PFNGLGETINTERNALFORMATI64VPROC (glfwGetProcAddress ("glGetInternalformati64v"));
01710   glGetInternalformativ = PFNGLGETINTERNALFORMATIVPROC (glfwGetProcAddress ("glGetInternalformativ"));
01711   glGetMultiTexEnvfvEXT = PFNGLGETMULTITEXENVFVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvfvEXT"));
01712   glGetMultiTexEnvivEXT = PFNGLGETMULTITEXENVIVEXTPROC (glfwGetProcAddress ("glGetMultiTexEnvivEXT"));
01713   glGetMultiTexGendvEXT = PFNGLGETMULTITEXGENDVEXTPROC (glfwGetProcAddress ("glGetMultiTexGendvEXT"));
01714   glGetMultiTexGenfvEXT = PFNGLGETMULTITEXGENFVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenfvEXT"));
01715   glGetMultiTexGenivEXT = PFNGLGETMULTITEXGENIVEXTPROC (glfwGetProcAddress ("glGetMultiTexGenivEXT"));
01716   glGetMultiTexImageEXT = PFNGLGETMULTITEXIMAGEEXTPROC (glfwGetProcAddress ("glGetMultiTexImageEXT"));
01717   glGetMultiTexLevelParameterfvEXT =
PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterfvEXT"));
01718   glGetMultiTexLevelParameterivEXT =
PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexLevelParameterivEXT"));
01719   glGetMultiTexParameterIivEXT =
PFNGLGETMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIivEXT"));
01720   glGetMultiTexParameterIuiEXT =
PFNGLGETMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterIuiEXT"));
01721   glGetMultiTexParameterfvEXT =
PFNGLGETMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterfvEXT"));
01722   glGetMultiTexParameterivEXT =
PFNGLGETMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetMultiTexParameterivEXT"));
01723   glGetMultisamplefv = PFNGLGETMULTISAMPLEFVPROC (glfwGetProcAddress ("glGetMultisamplefv"));
01724   glGetNamedBufferParameteri64v =
PFNGLGETNAMEDBUFFERPARAMETERI64VPROC (glfwGetProcAddress ("glGetNamedBufferParameteri64v"));
01725   glGetNamedBufferParameteriv =
PFNGLGETNAMEDBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetNamedBufferParameteriv"));
01726   glGetNamedBufferParameterivEXT =
PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetNamedBufferParameterivEXT"));
01727   glGetNamedBufferParameterui64vNV =
PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC (glfwGetProcAddress ("glGetNamedBufferParameterui64vNV"));
01728   glGetNamedBufferPointerv =
PFNGLGETNAMEDBUFFERPOINTERVPROC (glfwGetProcAddress ("glGetNamedBufferPointerv"));
01729   glGetNamedBufferPointervEXT =
PFNGLGETNAMEDBUFFERPOINTERVEXTPROC (glfwGetProcAddress ("glGetNamedBufferPointervEXT"));
01730   glGetNamedBufferSubData =
PFNGLGETNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glGetNamedBufferSubData"));
01731   glGetNamedBufferSubDataEXT =

```

2026年09月04日(金) 10時19分05秒作成 - ゲームグラフィックス特論 / 構成: Doxygen

```

        PFNGLGETPROGRAMRESOURCEIVPROC (glfwGetProcAddress ("glGetProgramResourceiv"));
01783     glGetProgramStageiv = PFNGLGETPROGRAMSTAGEIVPROC (glfwGetProcAddress ("glGetProgramStageiv"));
01784     glGetProgramiv = PFNGLGETPROGRAMIVPROC (glfwGetProcAddress ("glGetProgramiv"));
01785     glGetQueryBufferObjecti64v =
        PFNGLGETQUERYBUFFEROBJECTI64VPROC (glfwGetProcAddress ("glGetQueryBufferObjecti64v"));
01786     glGetQueryBufferObjectiv =
        PFNGLGETQUERYBUFFEROBJECTIVPROC (glfwGetProcAddress ("glGetQueryBufferObjectiv"));
01787     glGetQueryBufferObjectui64v =
        PFNGLGETQUERYBUFFEROBJECTUI64VPROC (glfwGetProcAddress ("glGetQueryBufferObjectui64v"));
01788     glGetQueryBufferObjectuiv =
        PFNGLGETQUERYBUFFEROBJECTUIVPROC (glfwGetProcAddress ("glGetQueryBufferObjectuiv"));
01789     glGetQueryIndexediv = PFNGLGETQUERYINDEXEDIVPROC (glfwGetProcAddress ("glGetQueryIndexediv"));
01790     glGetQueryObjecti64v = PFNGLGETQUERYOBJECTI64VPROC (glfwGetProcAddress ("glGetQueryObjecti64v"));
01791     glGetQueryObjectiv = PFNGLGETQUERYOBJECTIVPROC (glfwGetProcAddress ("glGetQueryObjectiv"));
01792     glGetQueryObjectui64v = PFNGLGETQUERYOBJECTUI64VPROC (glfwGetProcAddress ("glGetQueryObjectui64v"));
01793     glGetQueryObjectuiv = PFNGLGETQUERYOBJECTUIVPROC (glfwGetProcAddress ("glGetQueryObjectuiv"));
01794     glGetQueryiv = PFNGLGETQUERYIVPROC (glfwGetProcAddress ("glGetQueryiv"));
01795     glGetRenderbufferParameteriv =
        PFNGLGETRENDERBUFFERPARAMETERIVPROC (glfwGetProcAddress ("glGetRenderbufferParameteriv"));
01796     glGetSamplerParameterIiv =
        PFNGLGETSAMPLERPARAMETERIIVPROC (glfwGetProcAddress ("glGetSamplerParameterIiv"));
01797     glGetSamplerParameterIuiv =
        PFNGLGETSAMPLERPARAMETERUIVPROC (glfwGetProcAddress ("glGetSamplerParameterIuiv"));
01798     glGetSamplerParameterfv =
        PFNGLGETSAMPLERPARAMETERFVPROC (glfwGetProcAddress ("glGetSamplerParameterfv"));
01799     glGetSamplerParameteriv =
        PFNGLGETSAMPLERPARAMETERIVPROC (glfwGetProcAddress ("glGetSamplerParameteriv"));
01800     glGetShaderInfoLog = PFNGLGETSHADERINFOLOGPROC (glfwGetProcAddress ("glGetShaderInfoLog"));
01801     glGetShaderPrecisionFormat =
        PFNGLGETSHADERPRECISIONFORMATPROC (glfwGetProcAddress ("glGetShaderPrecisionFormat"));
01802     glGetShaderSource = PFNGLGETSHADERSOURCEPROC (glfwGetProcAddress ("glGetShaderSource"));
01803     glGetShaderiv = PFNGLGETSHADERIVPROC (glfwGetProcAddress ("glGetShaderiv"));
01804     glGetStageIndexNV = PFNGLGETSTAGEINDEXNVPROC (glfwGetProcAddress ("glGetStageIndexNV"));
01805     glGetString = PFNGLGETSTRINGPROC (glfwGetProcAddress ("glGetString"));
01806     glGetStringi = PFNGLGETSTRINGIPROC (glfwGetProcAddress ("glGetStringi"));
01807     glGetSubroutineIndex = PFNGLGETSUBROUTINEINDEXPROC (glfwGetProcAddress ("glGetSubroutineIndex"));
01808     glGetSubroutineUniformLocation =
        PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC (glfwGetProcAddress ("glGetSubroutineUniformLocation"));
01809     glGetSynciv = PFNGLGETSYNClVPROC (glfwGetProcAddress ("glGetSynciv"));
01810     glGetTexImage = PFNGLGETTEXIMAGEPROC (glfwGetProcAddress ("glGetTexImage"));
01811     glGetTexLevelParameterfv =
        PFNGLGETTEXLEVELPARAMETERFVPROC (glfwGetProcAddress ("glGetTexLevelParameterfv"));
01812     glGetTexLevelParameteriv =
        PFNGLGETTEXLEVELPARAMETERIVPROC (glfwGetProcAddress ("glGetTexLevelParameteriv"));
01813     glGetTexParameterIiv = PFNGLGETTEXPARAMETERIIVPROC (glfwGetProcAddress ("glGetTexParameterIiv"));
01814     glGetTexParameterIuiv = PFNGLGETTEXPARAMETERUIVPROC (glfwGetProcAddress ("glGetTexParameterIuiv"));
01815     glGetTexParameterfv = PFNGLGETTEXPARAMETERFVPROC (glfwGetProcAddress ("glGetTexParameterfv"));
01816     glGetTexParameteriv = PFNGLGETTEXPARAMETERIVPROC (glfwGetProcAddress ("glGetTexParameteriv"));
01817     glGetTextureHandleARB = PFNGLGETTEXTUREHANDLEARBPROC (glfwGetProcAddress ("glGetTextureHandleARB"));
01818     glGetTextureHandleNV = PFNGLGETTEXTUREHANDLENVPROC (glfwGetProcAddress ("glGetTextureHandleNV"));
01819     glGetTextureImage = PFNGLGETTEXTUREIMAGEPROC (glfwGetProcAddress ("glGetTextureImage"));
01820     glGetTextureImageEXT = PFNGLGETTEXTUREIMAGEEXTPROC (glfwGetProcAddress ("glGetTextureImageEXT"));
01821     glGetTextureLevelParameterfv =
        PFNGLGETTEXTURELEVELPARAMETERFVPROC (glfwGetProcAddress ("glGetTextureLevelParameterfv"));
01822     glGetTextureLevelParameterfvEXT =
        PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetTextureLevelParameterfvEXT"));
01823     glGetTextureLevelParameteriv =
        PFNGLGETTEXTURELEVELPARAMETERIVPROC (glfwGetProcAddress ("glGetTextureLevelParameteriv"));
01824     glGetTextureLevelParameterivEXT =
        PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetTextureLevelParameterivEXT"));
01825     glGetTextureParameterIiv =
        PFNGLGETTEXTUREPARAMETERIIVPROC (glfwGetProcAddress ("glGetTextureParameterIiv"));
01826     glGetTextureParameterIivEXT =
        PFNGLGETTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterIivEXT"));
01827     glGetTextureParameterIuiv =
        PFNGLGETTEXTUREPARAMETERUIVPROC (glfwGetProcAddress ("glGetTextureParameterIuiv"));
01828     glGetTextureParameterIuivEXT =
        PFNGLGETTEXTUREPARAMETERUIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterIuivEXT"));
01829     glGetTextureParameterfv =
        PFNGLGETTEXTUREPARAMETERFVPROC (glfwGetProcAddress ("glGetTextureParameterfv"));
01830     glGetTextureParameterfvEXT =
        PFNGLGETTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress ("glGetTextureParameterfvEXT"));
01831     glGetTextureParameteriv =
        PFNGLGETTEXTUREPARAMETERIVPROC (glfwGetProcAddress ("glGetTextureParameteriv"));
01832     glGetTextureParameterivEXT =
        PFNGLGETTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress ("glGetTextureParameterivEXT"));
01833     glGetTextureSamplerHandleARB =
        PFNGLGETTEXTURESAMPLERHANDLEARBPROC (glfwGetProcAddress ("glGetTextureSamplerHandleARB"));
01834     glGetTextureSamplerHandleNV =
        PFNGLGETTEXTURESAMPLERHANDLENVPROC (glfwGetProcAddress ("glGetTextureSamplerHandleNV"));
01835     glGetTextureSubImage = PFNGLGETTEXTURESUBIMAGEPROC (glfwGetProcAddress ("glGetTextureSubImage"));
01836     glGetTransformFeedbackVarying =
        PFNGLGETTRANSFORMFEEDBACKVARYINGPROC (glfwGetProcAddress ("glGetTransformFeedbackVarying"));
01837     glGetTransformFeedbacki64_v =
        PFNGLGETTRANSFORMFEEDBACKI64_VPROC (glfwGetProcAddress ("glGetTransformFeedbacki64_v"));
01838     glGetTransformFeedbacki_v =
        PFNGLGETTRANSFORMFEEDBACKI_VPROC (glfwGetProcAddress ("glGetTransformFeedbacki_v"));

```

```

01839     glGetTransformFeedbackiv =
PFNGLGETTRANSFORMFEEDBACKIVPROC (glfwGetProcAddress("glGetTransformFeedbackiv"));
01840     glGetUniformBlockIndex =
PFNGLGETUNIFORMBLOCKINDEXPROC (glfwGetProcAddress("glGetUniformBlockIndex"));
01841     glGetUniformIndices = PFNGLGETUNIFORMINDICESPROC (glfwGetProcAddress("glGetUniformIndices"));
01842     glGetUniformLocation = PFNGLGETUNIFORMLOCATIONPROC (glfwGetProcAddress("glGetUniformLocation"));
01843     glGetUniformSubroutineuiv =
PFNGLGETUNIFORMSUBROUTINEUIVPROC (glfwGetProcAddress("glGetUniformSubroutineuiv"));
01844     glGetUniformdv = PFNGLGETUNIFORMDVPROC (glfwGetProcAddress("glGetUniformdv"));
01845     glGetUniformfv = PFNGLGETUNIFORMFVPROC (glfwGetProcAddress("glGetUniformfv"));
01846     glGetUniformi64vARB = PFNGLGETUNIFORMI64VARBPROC (glfwGetProcAddress("glGetUniformi64vARB"));
01847     glGetUniformi64vNV = PFNGLGETUNIFORMI64VNVPROC (glfwGetProcAddress("glGetUniformi64vNV"));
01848     glGetUniformiv = PFNGLGETUNIFORMIVPROC (glfwGetProcAddress("glGetUniformiv"));
01849     glGetUniformui64vARB = PFNGLGETUNIFORMUI64VARBPROC (glfwGetProcAddress("glGetUniformui64vARB"));
01850     glGetUniformui64vNV = PFNGLGETUNIFORMUI64VNVPROC (glfwGetProcAddress("glGetUniformui64vNV"));
01851     glGetUniformuiv = PFNGLGETUNIFORMUIVPROC (glfwGetProcAddress("glGetUniformuiv"));
01852     glGetVertexArrayIndexed64iv =
PFNGLGETVERTEXARRAYINDEXED64IVPROC (glfwGetProcAddress("glGetVertexArrayIndexed64iv"));
01853     glGetVertexArrayIndexediv =
PFNGLGETVERTEXARRAYINDEXEDIVPROC (glfwGetProcAddress("glGetVertexArrayIndexediv"));
01854     glGetVertexArrayIntegeri_vEXT =
PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC (glfwGetProcAddress("glGetVertexArrayIntegeri_vEXT"));
01855     glGetVertexArrayInteger_vEXT =
PFNGLGETVERTEXARRAYINTEGERVEXTPROC (glfwGetProcAddress("glGetVertexArrayInteger_vEXT"));
01856     glGetVertexArrayPointeri_vEXT =
PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC (glfwGetProcAddress("glGetVertexArrayPointeri_vEXT"));
01857     glGetVertexArrayPointervEXT =
PFNGLGETVERTEXARRAYPOINTERVEXTPROC (glfwGetProcAddress("glGetVertexArrayPointervEXT"));
01858     glGetVertexArrayiv = PFNGLGETVERTEXARRAYIVPROC (glfwGetProcAddress("glGetVertexArrayiv"));
01859     glGetVertexAttribiiv = PFNGLGETVERTEXATTRIBIIVPROC (glfwGetProcAddress("glGetVertexAttribiiv"));
01860     glGetVertexAttribiuiiv = PFNGLGETVERTEXATTRIBUIIVPROC (glfwGetProcAddress("glGetVertexAttribiuiiv"));
01861     glGetVertexAttribLdv = PFNGLGETVERTEXATTRIBLDVPROC (glfwGetProcAddress("glGetVertexAttribLdv"));
01862     glGetVertexAttribLi64vNV =
PFNGLGETVERTEXATTRIBLI64VNVPROC (glfwGetProcAddress("glGetVertexAttribLi64vNV"));
01863     glGetVertexAttribLui64vARB =
PFNGLGETVERTEXATTRIBLUI64VARBPROC (glfwGetProcAddress("glGetVertexAttribLui64vARB"));
01864     glGetVertexAttribLui64vNV =
PFNGLGETVERTEXATTRIBLUI64VNVPROC (glfwGetProcAddress("glGetVertexAttribLui64vNV"));
01865     glGetVertexAttribPointerv =
PFNGLGETVERTEXATTRIBPOINTERVPROC (glfwGetProcAddress("glGetVertexAttribPointerv"));
01866     glGetVertexAttribdv = PFNGLGETVERTEXATTRIBDVPROC (glfwGetProcAddress("glGetVertexAttribdv"));
01867     glGetVertexAttribfv = PFNGLGETVERTEXATTRIBFVPROC (glfwGetProcAddress("glGetVertexAttribfv"));
01868     glGetVertexAttribiv = PFNGLGETVERTEXATTRIBIVPROC (glfwGetProcAddress("glGetVertexAttribiv"));
01869     glGetVkProcAddrNV = PFNGLGETVKPROCADDRNVPROC (glfwGetProcAddress("glGetVkProcAddrNV"));
01870     glGetnCompressedTexImage =
PFNGLGETNCOMPRESSEDTEXIMAGEPROC (glfwGetProcAddress("glGetnCompressedTexImage"));
01871     glGetnCompressedTexImageARB =
PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC (glfwGetProcAddress("glGetnCompressedTexImageARB"));
01872     glGetnTexImage = PFNGLGETNTEXIMAGEPROC (glfwGetProcAddress("glGetnTexImage"));
01873     glGetnTexImageARB = PFNGLGETNTEXIMAGEARBPROC (glfwGetProcAddress("glGetnTexImageARB"));
01874     glGetnUniformdv = PFNGLGETNUNIFORMDVPROC (glfwGetProcAddress("glGetnUniformdv"));
01875     glGetnUniformdvARB = PFNGLGETNUNIFORMDVARBPROC (glfwGetProcAddress("glGetnUniformdvARB"));
01876     glGetnUniformfv = PFNGLGETNUNIFORMFVPROC (glfwGetProcAddress("glGetnUniformfv"));
01877     glGetnUniformfvARB = PFNGLGETNUNIFORMFVARBPROC (glfwGetProcAddress("glGetnUniformfvARB"));
01878     glGetnUniformi64vARB = PFNGLGETNUNIFORMI64VARBPROC (glfwGetProcAddress("glGetnUniformi64vARB"));
01879     glGetnUniformiv = PFNGLGETNUNIFORMIVPROC (glfwGetProcAddress("glGetnUniformiv"));
01880     glGetnUniformivARB = PFNGLGETNUNIFORMIVARBPROC (glfwGetProcAddress("glGetnUniformivARB"));
01881     glGetnUniformui64vARB = PFNGLGETNUNIFORMUI64VARBPROC (glfwGetProcAddress("glGetnUniformui64vARB"));
01882     glGetnUniformuiv = PFNGLGETNUNIFORMUIVPROC (glfwGetProcAddress("glGetnUniformuiv"));
01883     glGetnUniformuiivARB = PFNGLGETNUNIFORMUIVARBPROC (glfwGetProcAddress("glGetnUniformuiivARB"));
01884     glHint = PFNGLHINTPROC (glfwGetProcAddress("glHint"));
01885     glIndexFormatNV = PFNGLINDEXFORMATNVPROC (glfwGetProcAddress("glIndexFormatNV"));
01886     glInsertEventMarkerEXT =
PFNGLINSERTEVENTMARKEREXTPROC (glfwGetProcAddress("glInsertEventMarkerEXT"));
01887     glInterpolatePathsNV = PFNGLINTERPOLATEPATHSNVPROC (glfwGetProcAddress("glInterpolatePathsNV"));
01888     glInvalidateBufferData =
PFNGLINVALIDATEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateBufferData"));
01889     glInvalidateBufferSubData =
PFNGLINVALIDATEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateBufferSubData"));
01890     glInvalidateFramebuffer =
PFNGLINVALIDATEFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateFramebuffer"));
01891     glInvalidateNamedFramebufferData =
PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferData"));
01892     glInvalidateNamedFramebufferSubData =
PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC (glfwGetProcAddress("glInvalidateNamedFramebufferSubData"));
01893     glInvalidateSubFramebuffer =
PFNGLINVALIDATESUBFRAMEBUFFERPROC (glfwGetProcAddress("glInvalidateSubFramebuffer"));
01894     glInvalidateTexImage = PFNGLINVALIDATETEXIMAGEPROC (glfwGetProcAddress("glInvalidateTexImage"));
01895     glInvalidateTexSubImage =
PFNGLINVALIDATETEXSUBIMAGEPROC (glfwGetProcAddress("glInvalidateTexSubImage"));
01896     glIsBuffer = PFNGLISBUFFERPROC (glfwGetProcAddress("glIsBuffer"));
01897     glIsBufferResidentNV = PFNGLISBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsBufferResidentNV"));
01898     glIsCommandListNV = PFNGLISCOMMANDLISTNVPROC (glfwGetProcAddress("glIsCommandListNV"));
01899     glIsEnabled = PFNGLISENABLEDPROC (glfwGetProcAddress("glIsEnabled"));
01900     glIsEnabledIndexedEXT = PFNGLISENABLEDINDEXEDEXTPROC (glfwGetProcAddress("glIsEnabledIndexedEXT"));
01901     glIsEnabledi = PFNGLISENABLEDIPROC (glfwGetProcAddress("glIsEnabledi"));
01902     glIsFramebuffer = PFNGLISFRAMEBUFFERPROC (glfwGetProcAddress("glIsFramebuffer"));

```



```

01903     glIsImageHandleResidentARB =
PFNGLISIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsImageHandleResidentARB"));
01904     glIsImageHandleResidentNV =
PFNGLISIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsImageHandleResidentNV"));
01905     glIsNamedBufferResidentNV =
PFNGLISNAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glIsNamedBufferResidentNV"));
01906     glIsNamedStringARB = PFNGLISNAMEDSTRINGARBPROC (glfwGetProcAddress("glIsNamedStringARB"));
01907     glIsPathNV = PFNGLISPATHNVPROC (glfwGetProcAddress("glIsPathNV"));
01908     glIsPointInFillPathNV = PFNGLISPOINTINFILLPATHNVPROC (glfwGetProcAddress("glIsPointInFillPathNV"));
01909     glIsPointInStrokePathNV =
PFNGLISPOINTINSTROKEPATHNVPROC (glfwGetProcAddress("glIsPointInStrokePathNV"));
01910     glIsProgram = PFNGLISPROGRAMPROC (glfwGetProcAddress("glIsProgram"));
01911     glIsProgramPipeline = PFNGLISPROGRAMPIPELINEPROC (glfwGetProcAddress("glIsProgramPipeline"));
01912     glIsQuery = PFNGLISQUERYPROC (glfwGetProcAddress("glIsQuery"));
01913     glIsRenderbuffer = PFNGLISRENDERBUFFERPROC (glfwGetProcAddress("glIsRenderbuffer"));
01914     glIsSampler = PFNGLISSAMPLERPROC (glfwGetProcAddress("glIsSampler"));
01915     glIsShader = PFNGLISSHADERPROC (glfwGetProcAddress("glIsShader"));
01916     glIsStateNV = PFNGLISSTATENVPROC (glfwGetProcAddress("glIsStateNV"));
01917     glIsSync = PFNGLISSYNCPROC (glfwGetProcAddress("glIsSync"));
01918     glIsTexture = PFNGLISTEXTUREPROC (glfwGetProcAddress("glIsTexture"));
01919     glIsTextureHandleResidentARB =
PFNGLISTEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress("glIsTextureHandleResidentARB"));
01920     glIsTextureHandleResidentNV =
PFNGLISTEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress("glIsTextureHandleResidentNV"));
01921     glIsTransformFeedback = PFNGLISTRANSFORMFEEDBACKPROC (glfwGetProcAddress("glIsTransformFeedback"));
01922     glIsVertexArray = PFNGLISVERTEXARRAYPROC (glfwGetProcAddress("glIsVertexArray"));
01923     glLabelObjectEXT = PFNGLLABELOBJECTEXTPROC (glfwGetProcAddress("glLabelObjectEXT"));
01924     glLineWidth = PFNGLLINEWIDTHPROC (glfwGetProcAddress("glLineWidth"));
01925     glLinkProgram = PFNGLLINKPROGRAMPROC (glfwGetProcAddress("glLinkProgram"));
01926     glListDrawCommandsStatesClientNV =
PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC (glfwGetProcAddress("glListDrawCommandsStatesClientNV"));
01927     glLogicOp = PFNGLLOGICOPPROC (glfwGetProcAddress("glLogicOp"));
01928     glMakeBufferNonResidentNV =
PFNGLMAKEBUFFERNONRESIDENTNVPROC (glfwGetProcAddress("glMakeBufferNonResidentNV"));
01929     glMakeBufferResidentNV =
PFNGLMAKEBUFFERRESIDENTNVPROC (glfwGetProcAddress("glMakeBufferResidentNV"));
01930     glMakeImageHandleNonResidentARB =
PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC (glfwGetProcAddress("glMakeImageHandleNonResidentARB"));
01931     glMakeImageHandleNonResidentNV =
PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC (glfwGetProcAddress("glMakeImageHandleNonResidentNV"));
01932     glMakeImageHandleResidentARB =
PFNGLMAKEIMAGEHANDLERESIDENTARBPROC (glfwGetProcAddress("glMakeImageHandleResidentARB"));
01933     glMakeImageHandleResidentNV =
PFNGLMAKEIMAGEHANDLERESIDENTNVPROC (glfwGetProcAddress("glMakeImageHandleResidentNV"));
01934     glMakeNamedBufferNonResidentNV =
PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC (glfwGetProcAddress("glMakeNamedBufferNonResidentNV"));
01935     glMakeNamedBufferResidentNV =
PFNGLMAKENAMEDBUFFERRESIDENTNVPROC (glfwGetProcAddress("glMakeNamedBufferResidentNV"));
01936     glMakeTextureHandleNonResidentARB =
PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC (glfwGetProcAddress("glMakeTextureHandleNonResidentARB"));
01937     glMakeTextureHandleNonResidentNV =
PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC (glfwGetProcAddress("glMakeTextureHandleNonResidentNV"));
01938     glMakeTextureHandleResidentARB =
PFNGLMAKETEXTUREHANDLERESIDENTARBPROC (glfwGetProcAddress("glMakeTextureHandleResidentARB"));
01939     glMakeTextureHandleResidentNV =
PFNGLMAKETEXTUREHANDLERESIDENTNVPROC (glfwGetProcAddress("glMakeTextureHandleResidentNV"));
01940     glMapBuffer = PFNGLMAPBUFFERPROC (glfwGetProcAddress("glMapBuffer"));
01941     glMapBufferRange = PFNGLMAPBUFFERRANGEPROC (glfwGetProcAddress("glMapBufferRange"));
01942     glMapNamedBuffer = PFNGLMAPNAMEDBUFFERPROC (glfwGetProcAddress("glMapNamedBuffer"));
01943     glMapNamedBufferEXT = PFNGLMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress("glMapNamedBufferEXT"));
01944     glMapNamedBufferRange = PFNGLMAPNAMEDBUFFERRANGEPROC (glfwGetProcAddress("glMapNamedBufferRange"));
01945     glMapNamedBufferRangeEXT =
PFNGLMAPNAMEDBUFFERRANGEEXTPROC (glfwGetProcAddress("glMapNamedBufferRangeEXT"));
01946     glMatrixFrustumEXT = PFNGLMATRIXFRUSTUMEXTPROC (glfwGetProcAddress("glMatrixFrustumEXT"));
01947     glMatrixLoad3x2fNV = PFNGLMATRIXLOAD3X2FNVPROC (glfwGetProcAddress("glMatrixLoad3x2fNV"));
01948     glMatrixLoad3x3fNV = PFNGLMATRIXLOAD3X3FNVPROC (glfwGetProcAddress("glMatrixLoad3x3fNV"));
01949     glMatrixLoadIdentityEXT =
PFNGLMATRIXLOADIDENTITYEXTPROC (glfwGetProcAddress("glMatrixLoadIdentityEXT"));
01950     glMatrixLoadTranspose3x3fNV =
PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC (glfwGetProcAddress("glMatrixLoadTranspose3x3fNV"));
01951     glMatrixLoadTransposedEXT =
PFNGLMATRIXLOADTRANSPOSEDEXTPROC (glfwGetProcAddress("glMatrixLoadTransposedEXT"));
01952     glMatrixLoadTransposefEXT =
PFNGLMATRIXLOADTRANSPOSEFEEXTPROC (glfwGetProcAddress("glMatrixLoadTransposefEXT"));
01953     glMatrixLoaddeEXT = PFNGLMATRIXLOADDEXTPROC (glfwGetProcAddress("glMatrixLoaddeEXT"));
01954     glMatrixLoadfEXT = PFNGLMATRIXLOADFEEXTPROC (glfwGetProcAddress("glMatrixLoadfEXT"));
01955     glMatrixMult3x2fNV = PFNGLMATRIXMULT3X2FNVPROC (glfwGetProcAddress("glMatrixMult3x2fNV"));
01956     glMatrixMult3x3fNV = PFNGLMATRIXMULT3X3FNVPROC (glfwGetProcAddress("glMatrixMult3x3fNV"));
01957     glMatrixMultTranspose3x3fNV =
PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC (glfwGetProcAddress("glMatrixMultTranspose3x3fNV"));
01958     glMatrixMultTransposedEXT =
PFNGLMATRIXMULTTRANSPOSEDEXTPROC (glfwGetProcAddress("glMatrixMultTransposedEXT"));
01959     glMatrixMultTransposefEXT =
PFNGLMATRIXMULTTRANSPOSEFEEXTPROC (glfwGetProcAddress("glMatrixMultTransposefEXT"));
01960     glMatrixMultdeEXT = PFNGLMATRIXMULTDEXTPROC (glfwGetProcAddress("glMatrixMultdeEXT"));
01961     glMatrixMultfEXT = PFNGLMATRIXMULTFEEXTPROC (glfwGetProcAddress("glMatrixMultfEXT"));
01962     glMatrixOrthoEXT = PFNGLMATRIXORTHOEXTPROC (glfwGetProcAddress("glMatrixOrthoEXT"));

```

```

01963     glMatrixPopEXT = PFNGLMATRIXPOPEXTPROC (glfwGetProcAddress ("glMatrixPopEXT"));
01964     glMatrixPushEXT = PFNGLMATRIXPUSHEXTPROC (glfwGetProcAddress ("glMatrixPushEXT"));
01965     glMatrixRotatedEXT = PFNGLMATRIXROTATEDEXTPROC (glfwGetProcAddress ("glMatrixRotatedEXT"));
01966     glMatrixRotatefEXT = PFNGLMATRIXROTATEFEXTPROC (glfwGetProcAddress ("glMatrixRotatefEXT"));
01967     glMatrixScaledEXT = PFNGLMATRIXSCALEDEXTPROC (glfwGetProcAddress ("glMatrixScaledEXT"));
01968     glMatrixScalefEXT = PFNGLMATRIXSCALEFEXTPROC (glfwGetProcAddress ("glMatrixScalefEXT"));
01969     glMatrixTranslatedEXT = PFNGLMATRIXTRANSLATEDEXTPROC (glfwGetProcAddress ("glMatrixTranslatedEXT"));
01970     glMatrixTranslatefEXT = PFNGLMATRIXTRANSLATEFEXTPROC (glfwGetProcAddress ("glMatrixTranslatefEXT"));
01971     glMaxShaderCompilerThreadsARB =
PFNGLMAXSHADERCOMPILERTHREADSARBPROC (glfwGetProcAddress ("glMaxShaderCompilerThreadsARB"));
01972     glMemoryBarrier = PFNGLMEMORYBARRIERPROC (glfwGetProcAddress ("glMemoryBarrier"));
01973     glMemoryBarrierByRegion =
PFNGLMEMORYBARRIERBYREGIONPROC (glfwGetProcAddress ("glMemoryBarrierByRegion"));
01974     glMinSampleShading = PFNGLMINSAMPLESHADINGPROC (glfwGetProcAddress ("glMinSampleShading"));
01975     glMinSampleShadingARB = PFNGLMINSAMPLESHADINGARBPROC (glfwGetProcAddress ("glMinSampleShadingARB"));
01976     glMultiDrawArrays = PFNGLMULTIDRAWARRAYSPROC (glfwGetProcAddress ("glMultiDrawArrays"));
01977     glMultiDrawArraysIndirect =
PFNGLMULTIDRAWARRAYSINDIRECTPROC (glfwGetProcAddress ("glMultiDrawArraysIndirect"));
01978     glMultiDrawArraysIndirectBindlessCountNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectBindlessCountNV"));
01979     glMultiDrawArraysIndirectBindlessNV =
PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectBindlessNV"));
01980     glMultiDrawArraysIndirectCountARB =
PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC (glfwGetProcAddress ("glMultiDrawArraysIndirectCountARB"));
01981     glMultiDrawElements = PFNGLMULTIDRAWELEMENTSPROC (glfwGetProcAddress ("glMultiDrawElements"));
01982     glMultiDrawElementsBaseVertex =
PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC (glfwGetProcAddress ("glMultiDrawElementsBaseVertex"));
01983     glMultiDrawElementsIndirect =
PFNGLMULTIDRAWELEMENTSINDIRECTPROC (glfwGetProcAddress ("glMultiDrawElementsIndirect"));
01984     glMultiDrawElementsIndirectBindlessCountNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectBindlessCountNV"));
01985     glMultiDrawElementsIndirectBindlessNV =
PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectBindlessNV"));
01986     glMultiDrawElementsIndirectCountARB =
PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC (glfwGetProcAddress ("glMultiDrawElementsIndirectCountARB"));
01987     glMultiTexBufferEXT = PFNGLMULTITEXBUFFEREXTPROC (glfwGetProcAddress ("glMultiTexBufferEXT"));
01988     glMultiTexCoordPointerEXT =
PFNGLMULTITEXCOORDPOINTEREXTPROC (glfwGetProcAddress ("glMultiTexCoordPointerEXT"));
01989     glMultiTexEnvfEXT = PFNGLMULTITEXENVFEXTPROC (glfwGetProcAddress ("glMultiTexEnvfEXT"));
01990     glMultiTexEnvfvEXT = PFNGLMULTITEXENVFVEXTPROC (glfwGetProcAddress ("glMultiTexEnvfvEXT"));
01991     glMultiTexEnviEXT = PFNGLMULTITEXENVIEXTPROC (glfwGetProcAddress ("glMultiTexEnviEXT"));
01992     glMultiTexEnvivEXT = PFNGLMULTITEXENVIVEXTPROC (glfwGetProcAddress ("glMultiTexEnvivEXT"));
01993     glMultiTexGendEXT = PFNGLMULTITEXGENDEXTPROC (glfwGetProcAddress ("glMultiTexGendEXT"));
01994     glMultiTexGendvEXT = PFNGLMULTITEXGENDVEXTPROC (glfwGetProcAddress ("glMultiTexGendvEXT"));
01995     glMultiTexGenfEXT = PFNGLMULTITEXGENFEXTPROC (glfwGetProcAddress ("glMultiTexGenfEXT"));
01996     glMultiTexGenfvEXT = PFNGLMULTITEXGENFVEXTPROC (glfwGetProcAddress ("glMultiTexGenfvEXT"));
01997     glMultiTexGeniEXT = PFNGLMULTITEXGENIEXTPROC (glfwGetProcAddress ("glMultiTexGeniEXT"));
01998     glMultiTexGenivEXT = PFNGLMULTITEXGENIVEXTPROC (glfwGetProcAddress ("glMultiTexGenivEXT"));
01999     glMultiTexImage1DEXT = PFNGLMULTITEXIMAGE1DEXTPROC (glfwGetProcAddress ("glMultiTexImage1DEXT"));
02000     glMultiTexImage2DEXT = PFNGLMULTITEXIMAGE2DEXTPROC (glfwGetProcAddress ("glMultiTexImage2DEXT"));
02001     glMultiTexImage3DEXT = PFNGLMULTITEXIMAGE3DEXTPROC (glfwGetProcAddress ("glMultiTexImage3DEXT"));
02002     glMultiTexParameterIivEXT =
PFNGLMULTITEXPARAMETERIIVEXTPROC (glfwGetProcAddress ("glMultiTexParameterIivEXT"));
02003     glMultiTexParameterIuivEXT =
PFNGLMULTITEXPARAMETERIUIVEXTPROC (glfwGetProcAddress ("glMultiTexParameterIuivEXT"));
02004     glMultiTexParameterfEXT =
PFNGLMULTITEXPARAMETERFEXTPROC (glfwGetProcAddress ("glMultiTexParameterfEXT"));
02005     glMultiTexParameterfvEXT =
PFNGLMULTITEXPARAMETERFVEXTPROC (glfwGetProcAddress ("glMultiTexParameterfvEXT"));
02006     glMultiTexParameterIiEXT =
PFNGLMULTITEXPARAMETERIEXTPROC (glfwGetProcAddress ("glMultiTexParameterIiEXT"));
02007     glMultiTexParameterivEXT =
PFNGLMULTITEXPARAMETERIVEXTPROC (glfwGetProcAddress ("glMultiTexParameterivEXT"));
02008     glMultiTexRenderbufferEXT =
PFNGLMULTITEXRENDERBUFFEREXTPROC (glfwGetProcAddress ("glMultiTexRenderbufferEXT"));
02009     glMultiTexSubImage1DEXT =
PFNGLMULTITEXSUBIMAGE1DEXTPROC (glfwGetProcAddress ("glMultiTexSubImage1DEXT"));
02010     glMultiTexSubImage2DEXT =
PFNGLMULTITEXSUBIMAGE2DEXTPROC (glfwGetProcAddress ("glMultiTexSubImage2DEXT"));
02011     glMultiTexSubImage3DEXT =
PFNGLMULTITEXSUBIMAGE3DEXTPROC (glfwGetProcAddress ("glMultiTexSubImage3DEXT"));
02012     glNamedBufferData = PFNGLNAMEDBUFFERDATAPROC (glfwGetProcAddress ("glNamedBufferData"));
02013     glNamedBufferDataEXT = PFNGLNAMEDBUFFERDATAEXTPROC (glfwGetProcAddress ("glNamedBufferDataEXT"));
02014     glNamedBufferPageCommitmentARB =
PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC (glfwGetProcAddress ("glNamedBufferPageCommitmentARB"));
02015     glNamedBufferPageCommitmentEXT =
PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC (glfwGetProcAddress ("glNamedBufferPageCommitmentEXT"));
02016     glNamedBufferStorage = PFNGLNAMEDBUFFERSTORAGEPROC (glfwGetProcAddress ("glNamedBufferStorage"));
02017     glNamedBufferStorageEXT =
PFNGLNAMEDBUFFERSTORAGEEXTPROC (glfwGetProcAddress ("glNamedBufferStorageEXT"));
02018     glNamedBufferSubData = PFNGLNAMEDBUFFERSUBDATAPROC (glfwGetProcAddress ("glNamedBufferSubData"));
02019     glNamedBufferSubDataEXT =
PFNGLNAMEDBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glNamedBufferSubDataEXT"));
02020     glNamedCopyBufferSubDataEXT =
PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC (glfwGetProcAddress ("glNamedCopyBufferSubDataEXT"));
02021     glNamedFramebuffersDrawBuffer =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC (glfwGetProcAddress ("glNamedFramebuffersDrawBuffer"));

```

```

02022     glNamedFramebufferDrawBuffers =
PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC (glfwGetProcAddress ("glNamedFramebufferDrawBuffers"));
02023     glNamedFramebufferParameteri =
PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC (glfwGetProcAddress ("glNamedFramebufferParameteri"));
02024     glNamedFramebufferParameteriEXT =
PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC (glfwGetProcAddress ("glNamedFramebufferParameteriEXT"));
02025     glNamedFramebufferReadBuffer =
PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferReadBuffer"));
02026     glNamedFramebufferRenderbuffer =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC (glfwGetProcAddress ("glNamedFramebufferRenderbuffer"));
02027     glNamedFramebufferRenderbufferEXT =
PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC (glfwGetProcAddress ("glNamedFramebufferRenderbufferEXT"));
02028     glNamedFramebufferSampleLocationsfvARB =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC (glfwGetProcAddress ("glNamedFramebufferSampleLocationsfvARB"));
02029     glNamedFramebufferSampleLocationsfvNV =
PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC (glfwGetProcAddress ("glNamedFramebufferSampleLocationsfvNV"));
02030     glNamedFramebufferTexture =
PFNGLNAMEDFRAMEBUFFERTEXTUREPROC (glfwGetProcAddress ("glNamedFramebufferTexture"));
02031     glNamedFramebufferTexture1DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture1DEXT"));
02032     glNamedFramebufferTexture2DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture2DEXT"));
02033     glNamedFramebufferTexture3DEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC (glfwGetProcAddress ("glNamedFramebufferTexture3DEXT"));
02034     glNamedFramebufferTextureEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureEXT"));
02035     glNamedFramebufferTextureFaceEXT =
PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureFaceEXT"));
02036     glNamedFramebufferTextureLayer =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayer"));
02037     glNamedFramebufferTextureLayerEXT =
PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC (glfwGetProcAddress ("glNamedFramebufferTextureLayerEXT"));
02038     glNamedProgramLocalParameter4dEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dEXT"));
02039     glNamedProgramLocalParameter4dvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4dvEXT"));
02040     glNamedProgramLocalParameter4fEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fEXT"));
02041     glNamedProgramLocalParameter4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameter4fvEXT"));
02042     glNamedProgramLocalParameterI4iEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4iEXT"));
02043     glNamedProgramLocalParameterI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4ivEXT"));
02044     glNamedProgramLocalParameterI4uiEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uiEXT"));
02045     glNamedProgramLocalParameterI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameterI4uivEXT"));
02046     glNamedProgramLocalParameters4fvEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParameters4fvEXT"));
02047     glNamedProgramLocalParametersI4ivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4ivEXT"));
02048     glNamedProgramLocalParametersI4uivEXT =
PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC (glfwGetProcAddress ("glNamedProgramLocalParametersI4uivEXT"));
02049     glNamedProgramStringEXT =
PFNGLNAMEDPROGRAMSTRINGEXTPROC (glfwGetProcAddress ("glNamedProgramStringEXT"));
02050     glNamedRenderbufferStorage =
PFNGLNAMEDRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glNamedRenderbufferStorage"));
02051     glNamedRenderbufferStorageEXT =
PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageEXT"));
02052     glNamedRenderbufferStorageMultisample =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisample"));
02053     glNamedRenderbufferStorageMultisampleCoverageEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLECOVERAGEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleCoverageEXT"));
02054     glNamedRenderbufferStorageMultisampleEXT =
PFNGLNAMEDRENDERBUFFERSTAGEMULTISAMPLEEXTPROC (glfwGetProcAddress ("glNamedRenderbufferStorageMultisampleEXT"));
02055     glNamedStringARB = PFNGLNAMEDSTRINGARBPROC (glfwGetProcAddress ("glNamedStringARB"));
02056     glNormalFormatNV = PFNGLNORMALFORMATNVPROC (glfwGetProcAddress ("glNormalFormatNV"));
02057     glObjectLabel = PFNGLOBJECTLABELPROC (glfwGetProcAddress ("glObjectLabel"));
02058     glObjectPtrLabel = PFNGLOBJECTPTRLABELPROC (glfwGetProcAddress ("glObjectPtrLabel"));
02059     glPatchParameterfv = PFNGLPATCHPARAMETERFVPROC (glfwGetProcAddress ("glPatchParameterfv"));
02060     glPatchParameteri = PFNGLPATCHPARAMETERIPROC (glfwGetProcAddress ("glPatchParameteri"));
02061     glPathCommandsNV = PFNGLPATHCOMMANDSNVPROC (glfwGetProcAddress ("glPathCommandsNV"));
02062     glPathCoordsNV = PFNGLPATHCOORDSNVPROC (glfwGetProcAddress ("glPathCoordsNV"));
02063     glPathCoverDepthFuncNV =
PFNGLPATHCOVERDEPTHFUNCNVPROC (glfwGetProcAddress ("glPathCoverDepthFuncNV"));
02064     glPathDashArrayNV = PFNGLPATHDASHARRAYNVPROC (glfwGetProcAddress ("glPathDashArrayNV"));
02065     glPathGlyphIndexArrayNV =
PFNGLPATHGLYPHINDEXARRAYNVPROC (glfwGetProcAddress ("glPathGlyphIndexArrayNV"));
02066     glPathGlyphIndexRangeNV =
PFNGLPATHGLYPHINDEXRANGENVPROC (glfwGetProcAddress ("glPathGlyphIndexRangeNV"));
02067     glPathGlyphRangeNV = PFNGLPATHGLYPHRANGENVPROC (glfwGetProcAddress ("glPathGlyphRangeNV"));
02068     glPathGlyphsNV = PFNGLPATHGLYPHSNVPROC (glfwGetProcAddress ("glPathGlyphsNV"));
02069     glPathMemoryGlyphIndexArrayNV =
PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC (glfwGetProcAddress ("glPathMemoryGlyphIndexArrayNV"));
02070     glPathParameterfNV = PFNGLPATHPARAMETERFNVPROC (glfwGetProcAddress ("glPathParameterfNV"));
02071     glPathParameterfvNV = PFNGLPATHPARAMETERFVNVPROC (glfwGetProcAddress ("glPathParameterfvNV"));

```

```

02072     glPathParameteriNV = PFNGLPATHPARAMETERINVPROC (glfwGetProcAddress("glPathParameteriNV"));
02073     glPathParameterivNV = PFNGLPATHPARAMETERIVNVPROC (glfwGetProcAddress("glPathParameterivNV"));
02074     glPathStencilDepthOffsetNV =
PFNGLPATHSTENCILDEPTHOFFSETNVPROC (glfwGetProcAddress("glPathStencilDepthOffsetNV"));
02075     glPathStencilFuncNV = PFNGLPATHSTENCILFUNCNVPROC (glfwGetProcAddress("glPathStencilFuncNV"));
02076     glPathStringNV = PFNGLPATHSTRINGNVPROC (glfwGetProcAddress("glPathStringNV"));
02077     glPathSubCommandsNV = PFNGLPATHSUBCOMMANDSNVPROC (glfwGetProcAddress("glPathSubCommandsNV"));
02078     glPathSubCoordsNV = PFNGLPATHSUBCOORDSNVPROC (glfwGetProcAddress("glPathSubCoordsNV"));
02079     glPauseTransformFeedback =
PFNGLPAUSETRANSFORMFEEDBACKPROC (glfwGetProcAddress("glPauseTransformFeedback"));
02080     glPixelStoref = PFNGLPIXELSTOREFPROC (glfwGetProcAddress("glPixelStoref"));
02081     glPixelStorei = PFNGLPIXELSTOREIPROC (glfwGetProcAddress("glPixelStorei"));
02082     glPointAlongPathNV = PFNGLPOINTALONGPATHNVPROC (glfwGetProcAddress("glPointAlongPathNV"));
02083     glPointParameterf = PFNGLPOINTPARAMETERFPROC (glfwGetProcAddress("glPointParameterf"));
02084     glPointParameterfv = PFNGLPOINTPARAMETERFVPROC (glfwGetProcAddress("glPointParameterfv"));
02085     glPointParameteri = PFNGLPOINTPARAMETERIPROC (glfwGetProcAddress("glPointParameteri"));
02086     glPointParameteriv = PFNGLPOINTPARAMETERIVPROC (glfwGetProcAddress("glPointParameteriv"));
02087     glPointSize = PFNGLPOINTSIZEPROC (glfwGetProcAddress("glPointSize"));
02088     glPolygonMode = PFNGLPOLYGONMODEPROC (glfwGetProcAddress("glPolygonMode"));
02089     glPolygonOffset = PFNGLPOLYGONOFFSETPROC (glfwGetProcAddress("glPolygonOffset"));
02090     glPolygonOffsetClampEXT =
PFNGLPOLYGONOFFSETCLAMPEXTPROC (glfwGetProcAddress("glPolygonOffsetClampEXT"));
02091     glPopDebugGroup = PFNGLPOPDEBUGGROUPPROC (glfwGetProcAddress("glPopDebugGroup"));
02092     glPopGroupMarkerEXT = PFNGLPOPGROUPMARKEREXTPROC (glfwGetProcAddress("glPopGroupMarkerEXT"));
02093     glPrimitiveBoundingBoxARB =
PFNGLPRIMITIVEBOUNDINGBOXARBPROC (glfwGetProcAddress("glPrimitiveBoundingBoxARB"));
02094     glPrimitiveRestartIndex =
PFNGLPRIMITIVERESTARTINDEXPROC (glfwGetProcAddress("glPrimitiveRestartIndex"));
02095     glProgramBinary = PFNGLPROGRAMBINARYPROC (glfwGetProcAddress("glProgramBinary"));
02096     glProgramParameteri = PFNGLPROGRAMPARAMETERIPROC (glfwGetProcAddress("glProgramParameteri"));
02097     glProgramParameteriARB =
PFNGLPROGRAMPARAMETERIARBPROC (glfwGetProcAddress("glProgramParameteriARB"));
02098     glProgramPathFragmentInputGenNV =
PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC (glfwGetProcAddress("glProgramPathFragmentInputGenNV"));
02099     glProgramUniformId = PFNGLPROGRAMUNIFORMIDPROC (glfwGetProcAddress("glProgramUniformId"));
02100     glProgramUniformIdEXT = PFNGLPROGRAMUNIFORMIDEXTPROC (glfwGetProcAddress("glProgramUniformIdEXT"));
02101     glProgramUniformIdv = PFNGLPROGRAMUNIFORMIDVPROC (glfwGetProcAddress("glProgramUniformIdv"));
02102     glProgramUniformIdvEXT =
PFNGLPROGRAMUNIFORMIDVEXTPROC (glfwGetProcAddress("glProgramUniformIdvEXT"));
02103     glProgramUniformI = PFNGLPROGRAMUNIFORMIFPROC (glfwGetProcAddress("glProgramUniformI"));
02104     glProgramUniformIEXT = PFNGLPROGRAMUNIFORMIEXTPROC (glfwGetProcAddress("glProgramUniformIEXT"));
02105     glProgramUniformIfv = PFNGLPROGRAMUNIFORMIFVPROC (glfwGetProcAddress("glProgramUniformIfv"));
02106     glProgramUniformIfvEXT =
PFNGLPROGRAMUNIFORMIFVEXTPROC (glfwGetProcAddress("glProgramUniformIfvEXT"));
02107     glProgramUniformli = PFNGLPROGRAMUNIFORMIIPROC (glfwGetProcAddress("glProgramUniformli"));
02108     glProgramUniformli64ARB =
PFNGLPROGRAMUNIFORMLI64ARBPROC (glfwGetProcAddress("glProgramUniformli64ARB"));
02109     glProgramUniformli64NV =
PFNGLPROGRAMUNIFORMLI64NVPROC (glfwGetProcAddress("glProgramUniformli64NV"));
02110     glProgramUniformli64vARB =
PFNGLPROGRAMUNIFORMLI64VARBPROC (glfwGetProcAddress("glProgramUniformli64vARB"));
02111     glProgramUniformli64vNV =
PFNGLPROGRAMUNIFORMLI64VNVPROC (glfwGetProcAddress("glProgramUniformli64vNV"));
02112     glProgramUniformliEXT = PFNGLPROGRAMUNIFORMIEXTPROC (glfwGetProcAddress("glProgramUniformliEXT"));
02113     glProgramUniformliv = PFNGLPROGRAMUNIFORMIIVPROC (glfwGetProcAddress("glProgramUniformliv"));
02114     glProgramUniformlivEXT =
PFNGLPROGRAMUNIFORMIIVEXTPROC (glfwGetProcAddress("glProgramUniformlivEXT"));
02115     glProgramUniformlui = PFNGLPROGRAMUNIFORMUIPROC (glfwGetProcAddress("glProgramUniformlui"));
02116     glProgramUniformlui64ARB =
PFNGLPROGRAMUNIFORMLUI64ARBPROC (glfwGetProcAddress("glProgramUniformlui64ARB"));
02117     glProgramUniformlui64NV =
PFNGLPROGRAMUNIFORMLUI64NVPROC (glfwGetProcAddress("glProgramUniformlui64NV"));
02118     glProgramUniformlui64vARB =
PFNGLPROGRAMUNIFORMLUI64VARBPROC (glfwGetProcAddress("glProgramUniformlui64vARB"));
02119     glProgramUniformlui64vNV =
PFNGLPROGRAMUNIFORMLUI64VNVPROC (glfwGetProcAddress("glProgramUniformlui64vNV"));
02120     glProgramUniformluiEXT =
PFNGLPROGRAMUNIFORMUIEXTPROC (glfwGetProcAddress("glProgramUniformluiEXT"));
02121     glProgramUniformliv = PFNGLPROGRAMUNIFORMIIVPROC (glfwGetProcAddress("glProgramUniformliv"));
02122     glProgramUniformliuivEXT =
PFNGLPROGRAMUNIFORMLUIVEXTPROC (glfwGetProcAddress("glProgramUniformliuivEXT"));
02123     glProgramUniform2d = PFNGLPROGRAMUNIFORM2DPROC (glfwGetProcAddress("glProgramUniform2d"));
02124     glProgramUniform2dEXT = PFNGLPROGRAMUNIFORM2DEXTPROC (glfwGetProcAddress("glProgramUniform2dEXT"));
02125     glProgramUniform2dv = PFNGLPROGRAMUNIFORM2DVPROC (glfwGetProcAddress("glProgramUniform2dv"));
02126     glProgramUniform2dvEXT =
PFNGLPROGRAMUNIFORM2DVEXTPROC (glfwGetProcAddress("glProgramUniform2dvEXT"));
02127     glProgramUniform2f = PFNGLPROGRAMUNIFORM2FPROC (glfwGetProcAddress("glProgramUniform2f"));
02128     glProgramUniform2fEXT = PFNGLPROGRAMUNIFORM2FEXTPROC (glfwGetProcAddress("glProgramUniform2fEXT"));
02129     glProgramUniform2fv = PFNGLPROGRAMUNIFORM2FVPROC (glfwGetProcAddress("glProgramUniform2fv"));
02130     glProgramUniform2fvEXT =
PFNGLPROGRAMUNIFORM2FVEXTPROC (glfwGetProcAddress("glProgramUniform2fvEXT"));
02131     glProgramUniform2i = PFNGLPROGRAMUNIFORM2IPROC (glfwGetProcAddress("glProgramUniform2i"));
02132     glProgramUniform2i64ARB =
PFNGLPROGRAMUNIFORM2I64ARBPROC (glfwGetProcAddress("glProgramUniform2i64ARB"));
02133     glProgramUniform2i64NV =
PFNGLPROGRAMUNIFORM2I64NVPROC (glfwGetProcAddress("glProgramUniform2i64NV"));
02134     glProgramUniform2i64vARB =

```



```

    PFNGLPROGRAMUNIFORM2I64VARBPROC (glfwGetProcAddress ("glProgramUniform2i64vARB"));
02135   glProgramUniform2i64vNV =
    PFNGLPROGRAMUNIFORM2I64VNVPROC (glfwGetProcAddress ("glProgramUniform2i64vNV"));
02136   glProgramUniform2iEXT = PFNGLPROGRAMUNIFORM2IEXTPROC (glfwGetProcAddress ("glProgramUniform2iEXT"));
02137   glProgramUniform2iv = PFNGLPROGRAMUNIFORM2IVPROC (glfwGetProcAddress ("glProgramUniform2iv"));
02138   glProgramUniform2ivEXT =
    PFNGLPROGRAMUNIFORM2IVEXTPROC (glfwGetProcAddress ("glProgramUniform2ivEXT"));
02139   glProgramUniform2ui = PFNGLPROGRAMUNIFORM2UIPROC (glfwGetProcAddress ("glProgramUniform2ui"));
02140   glProgramUniform2ui64ARB =
    PFNGLPROGRAMUNIFORM2UI64ARBPROC (glfwGetProcAddress ("glProgramUniform2ui64ARB"));
02141   glProgramUniform2ui64NV =
    PFNGLPROGRAMUNIFORM2UI64NVPROC (glfwGetProcAddress ("glProgramUniform2ui64NV"));
02142   glProgramUniform2ui64vARB =
    PFNGLPROGRAMUNIFORM2UI64VARBPROC (glfwGetProcAddress ("glProgramUniform2ui64vARB"));
02143   glProgramUniform2ui64vNV =
    PFNGLPROGRAMUNIFORM2UI64VNVPROC (glfwGetProcAddress ("glProgramUniform2ui64vNV"));
02144   glProgramUniform2uiEXT =
    PFNGLPROGRAMUNIFORM2UIEXTPROC (glfwGetProcAddress ("glProgramUniform2uiEXT"));
02145   glProgramUniform2uiv = PFNGLPROGRAMUNIFORM2UIVPROC (glfwGetProcAddress ("glProgramUniform2uiv"));
02146   glProgramUniform2uivEXT =
    PFNGLPROGRAMUNIFORM2UIVEXTPROC (glfwGetProcAddress ("glProgramUniform2uivEXT"));
02147   glProgramUniform3d = PFNGLPROGRAMUNIFORM3DPROC (glfwGetProcAddress ("glProgramUniform3d"));
02148   glProgramUniform3dEXT = PFNGLPROGRAMUNIFORM3DEXTPROC (glfwGetProcAddress ("glProgramUniform3dEXT"));
02149   glProgramUniform3dv = PFNGLPROGRAMUNIFORM3DVPROC (glfwGetProcAddress ("glProgramUniform3dv"));
02150   glProgramUniform3dvEXT =
    PFNGLPROGRAMUNIFORM3DVEXTPROC (glfwGetProcAddress ("glProgramUniform3dvEXT"));
02151   glProgramUniform3f = PFNGLPROGRAMUNIFORM3FPROC (glfwGetProcAddress ("glProgramUniform3f"));
02152   glProgramUniform3fEXT = PFNGLPROGRAMUNIFORM3FEXTPROC (glfwGetProcAddress ("glProgramUniform3fEXT"));
02153   glProgramUniform3fv = PFNGLPROGRAMUNIFORM3FVPROC (glfwGetProcAddress ("glProgramUniform3fv"));
02154   glProgramUniform3fvEXT =
    PFNGLPROGRAMUNIFORM3FVEXTPROC (glfwGetProcAddress ("glProgramUniform3fvEXT"));
02155   glProgramUniform3i = PFNGLPROGRAMUNIFORM3IPROC (glfwGetProcAddress ("glProgramUniform3i"));
02156   glProgramUniform3i64ARB =
    PFNGLPROGRAMUNIFORM3I64ARBPROC (glfwGetProcAddress ("glProgramUniform3i64ARB"));
02157   glProgramUniform3i64NV =
    PFNGLPROGRAMUNIFORM3I64NVPROC (glfwGetProcAddress ("glProgramUniform3i64NV"));
02158   glProgramUniform3i64vARB =
    PFNGLPROGRAMUNIFORM3I64VARBPROC (glfwGetProcAddress ("glProgramUniform3i64vARB"));
02159   glProgramUniform3i64vNV =
    PFNGLPROGRAMUNIFORM3I64VNVPROC (glfwGetProcAddress ("glProgramUniform3i64vNV"));
02160   glProgramUniform3iEXT = PFNGLPROGRAMUNIFORM3IEXTPROC (glfwGetProcAddress ("glProgramUniform3iEXT"));
02161   glProgramUniform3iv = PFNGLPROGRAMUNIFORM3IVPROC (glfwGetProcAddress ("glProgramUniform3iv"));
02162   glProgramUniform3ivEXT =
    PFNGLPROGRAMUNIFORM3IVEXTPROC (glfwGetProcAddress ("glProgramUniform3ivEXT"));
02163   glProgramUniform3ui = PFNGLPROGRAMUNIFORM3UIPROC (glfwGetProcAddress ("glProgramUniform3ui"));
02164   glProgramUniform3ui64ARB =
    PFNGLPROGRAMUNIFORM3UI64ARBPROC (glfwGetProcAddress ("glProgramUniform3ui64ARB"));
02165   glProgramUniform3ui64NV =
    PFNGLPROGRAMUNIFORM3UI64NVPROC (glfwGetProcAddress ("glProgramUniform3ui64NV"));
02166   glProgramUniform3ui64vARB =
    PFNGLPROGRAMUNIFORM3UI64VARBPROC (glfwGetProcAddress ("glProgramUniform3ui64vARB"));
02167   glProgramUniform3ui64vNV =
    PFNGLPROGRAMUNIFORM3UI64VNVPROC (glfwGetProcAddress ("glProgramUniform3ui64vNV"));
02168   glProgramUniform3uiEXT =
    PFNGLPROGRAMUNIFORM3UIEXTPROC (glfwGetProcAddress ("glProgramUniform3uiEXT"));
02169   glProgramUniform3uiv = PFNGLPROGRAMUNIFORM3UIVPROC (glfwGetProcAddress ("glProgramUniform3uiv"));
02170   glProgramUniform3uivEXT =
    PFNGLPROGRAMUNIFORM3UIVEXTPROC (glfwGetProcAddress ("glProgramUniform3uivEXT"));
02171   glProgramUniform4d = PFNGLPROGRAMUNIFORM4DPROC (glfwGetProcAddress ("glProgramUniform4d"));
02172   glProgramUniform4dEXT = PFNGLPROGRAMUNIFORM4DEXTPROC (glfwGetProcAddress ("glProgramUniform4dEXT"));
02173   glProgramUniform4dv = PFNGLPROGRAMUNIFORM4DVPROC (glfwGetProcAddress ("glProgramUniform4dv"));
02174   glProgramUniform4dvEXT =
    PFNGLPROGRAMUNIFORM4DVEXTPROC (glfwGetProcAddress ("glProgramUniform4dvEXT"));
02175   glProgramUniform4f = PFNGLPROGRAMUNIFORM4FPROC (glfwGetProcAddress ("glProgramUniform4f"));
02176   glProgramUniform4fEXT = PFNGLPROGRAMUNIFORM4FEXTPROC (glfwGetProcAddress ("glProgramUniform4fEXT"));
02177   glProgramUniform4fv = PFNGLPROGRAMUNIFORM4FVPROC (glfwGetProcAddress ("glProgramUniform4fv"));
02178   glProgramUniform4fvEXT =
    PFNGLPROGRAMUNIFORM4FVEXTPROC (glfwGetProcAddress ("glProgramUniform4fvEXT"));
02179   glProgramUniform4i = PFNGLPROGRAMUNIFORM4IPROC (glfwGetProcAddress ("glProgramUniform4i"));
02180   glProgramUniform4i64ARB =
    PFNGLPROGRAMUNIFORM4I64ARBPROC (glfwGetProcAddress ("glProgramUniform4i64ARB"));
02181   glProgramUniform4i64NV =
    PFNGLPROGRAMUNIFORM4I64NVPROC (glfwGetProcAddress ("glProgramUniform4i64NV"));
02182   glProgramUniform4i64vARB =
    PFNGLPROGRAMUNIFORM4I64VARBPROC (glfwGetProcAddress ("glProgramUniform4i64vARB"));
02183   glProgramUniform4i64vNV =
    PFNGLPROGRAMUNIFORM4I64VNVPROC (glfwGetProcAddress ("glProgramUniform4i64vNV"));
02184   glProgramUniform4iEXT = PFNGLPROGRAMUNIFORM4IEXTPROC (glfwGetProcAddress ("glProgramUniform4iEXT"));
02185   glProgramUniform4iv = PFNGLPROGRAMUNIFORM4IVPROC (glfwGetProcAddress ("glProgramUniform4iv"));
02186   glProgramUniform4ivEXT =
    PFNGLPROGRAMUNIFORM4IVEXTPROC (glfwGetProcAddress ("glProgramUniform4ivEXT"));
02187   glProgramUniform4ui = PFNGLPROGRAMUNIFORM4UIPROC (glfwGetProcAddress ("glProgramUniform4ui"));
02188   glProgramUniform4ui64ARB =
    PFNGLPROGRAMUNIFORM4UI64ARBPROC (glfwGetProcAddress ("glProgramUniform4ui64ARB"));
02189   glProgramUniform4ui64NV =
    PFNGLPROGRAMUNIFORM4UI64NVPROC (glfwGetProcAddress ("glProgramUniform4ui64NV"));
02190   glProgramUniform4ui64vARB =

```

```

PFNGLPROGRAMUNIFORM4UI64VARBPROC (glfwGetProcAddress ("glProgramUniform4ui64vARB"));
02191 glProgramUniform4ui64vNV =
PFNGLPROGRAMUNIFORM4UI64VNVPROC (glfwGetProcAddress ("glProgramUniform4ui64vNV"));
02192 glProgramUniform4uiEXT =
PFNGLPROGRAMUNIFORM4UIEXTPROC (glfwGetProcAddress ("glProgramUniform4uiEXT"));
02193 glProgramUniform4uiv = PFNGLPROGRAMUNIFORM4UIVPROC (glfwGetProcAddress ("glProgramUniform4uiv"));
02194 glProgramUniform4uivEXT =
PFNGLPROGRAMUNIFORM4UIVEXTPROC (glfwGetProcAddress ("glProgramUniform4uivEXT"));
02195 glProgramUniformHandleui64ARB =
PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64ARB"));
02196 glProgramUniformHandleui64NV =
PFNGLPROGRAMUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress ("glProgramUniformHandleui64NV"));
02197 glProgramUniformHandleui64vARB =
PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress ("glProgramUniformHandleui64vARB"));
02198 glProgramUniformHandleui64vNV =
PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress ("glProgramUniformHandleui64vNV"));
02199 glProgramUniformMatrix2dv =
PFNGLPROGRAMUNIFORMMATRIX2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2dv"));
02200 glProgramUniformMatrix2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2dvEXT"));
02201 glProgramUniformMatrix2fv =
PFNGLPROGRAMUNIFORMMATRIX2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2fv"));
02202 glProgramUniformMatrix2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2fvEXT"));
02203 glProgramUniformMatrix2x3dv =
PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dv"));
02204 glProgramUniformMatrix2x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3dvEXT"));
02205 glProgramUniformMatrix2x3fv =
PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fv"));
02206 glProgramUniformMatrix2x3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x3fvEXT"));
02207 glProgramUniformMatrix2x4dv =
PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dv"));
02208 glProgramUniformMatrix2x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4dvEXT"));
02209 glProgramUniformMatrix2x4fv =
PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fv"));
02210 glProgramUniformMatrix2x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix2x4fvEXT"));
02211 glProgramUniformMatrix3dv =
PFNGLPROGRAMUNIFORMMATRIX3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3dv"));
02212 glProgramUniformMatrix3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3dvEXT"));
02213 glProgramUniformMatrix3fv =
PFNGLPROGRAMUNIFORMMATRIX3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3fv"));
02214 glProgramUniformMatrix3fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3fvEXT"));
02215 glProgramUniformMatrix3x2dv =
PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dv"));
02216 glProgramUniformMatrix3x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2dvEXT"));
02217 glProgramUniformMatrix3x2fv =
PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fv"));
02218 glProgramUniformMatrix3x2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x2fvEXT"));
02219 glProgramUniformMatrix3x4dv =
PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dv"));
02220 glProgramUniformMatrix3x4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4dvEXT"));
02221 glProgramUniformMatrix3x4fv =
PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fv"));
02222 glProgramUniformMatrix3x4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix3x4fvEXT"));
02223 glProgramUniformMatrix4dv =
PFNGLPROGRAMUNIFORMMATRIX4DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4dv"));
02224 glProgramUniformMatrix4dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4dvEXT"));
02225 glProgramUniformMatrix4fv =
PFNGLPROGRAMUNIFORMMATRIX4FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4fv"));
02226 glProgramUniformMatrix4fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4fvEXT"));
02227 glProgramUniformMatrix4x2dv =
PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dv"));
02228 glProgramUniformMatrix4x2dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2dvEXT"));
02229 glProgramUniformMatrix4x2fv =
PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fv"));
02230 glProgramUniformMatrix4x2fvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x2fvEXT"));
02231 glProgramUniformMatrix4x3dv =
PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dv"));
02232 glProgramUniformMatrix4x3dvEXT =
PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3dvEXT"));
02233 glProgramUniformMatrix4x3fv =
PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fv"));
02234 glProgramUniformMatrix4x3fvEXT =

```

```
PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC (glfwGetProcAddress ("glProgramUniformMatrix4x3fvEXT"));
02235 glProgramUniformi64NV =
PFNGLPROGRAMUNIFORMUI64NVPROC (glfwGetProcAddress ("glProgramUniformui64NV"));
02236 glProgramUniformi64vNV =
PFNGLPROGRAMUNIFORMUI64VNVPROC (glfwGetProcAddress ("glProgramUniformui64vNV"));
02237 glProvokingVertex = PFNGLPROVOKINGVERTEXPROC (glfwGetProcAddress ("glProvokingVertex"));
02238 glPushClientAttribDefaultEXT =
PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC (glfwGetProcAddress ("glPushClientAttribDefaultEXT"));
02239 glPushDebugGroup = PFNGLPUSHDEBUGGROUPPPROC (glfwGetProcAddress ("glPushDebugGroup"));
02240 glPushGroupMarkerEXT = PFNGLPUSHGROUPMARKEREXTPROC (glfwGetProcAddress ("glPushGroupMarkerEXT"));
02241 glQueryCounter = PFNGLQUERYCOUNTERPROC (glfwGetProcAddress ("glQueryCounter"));
02242 glRasterSamplesEXT = PFNGLRASTERSAMPLESEXTPROC (glfwGetProcAddress ("glRasterSamplesEXT"));
02243 glReadBuffer = PFNGLREADBUFFERPROC (glfwGetProcAddress ("glReadBuffer"));
02244 glReadPixels = PFNGLREADPIXELSPROC (glfwGetProcAddress ("glReadPixels"));
02245 glReadnPixels = PFNGLREADNPIXELSPROC (glfwGetProcAddress ("glReadnPixels"));
02246 glReadnPixelsARB = PFNGLREADNPIXELSARBPROC (glfwGetProcAddress ("glReadnPixelsARB"));
02247 glReleaseShaderCompiler =
PFNGLRELEASESHADERCOMPILERPROC (glfwGetProcAddress ("glReleaseShaderCompiler"));
02248 glRenderbufferStorage = PFNGLRENDERBUFFERSTORAGEPROC (glfwGetProcAddress ("glRenderbufferStorage"));
02249 glRenderbufferStorageMultisample =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC (glfwGetProcAddress ("glRenderbufferStorageMultisample"));
02250 glRenderbufferStorageMultisampleCoverageNV =
PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEENVPROC (glfwGetProcAddress ("glRenderbufferStorageMultisampleCoverageNV"));
02251 glResolveDepthValuesNV =
PFNGLRESOLVEDEPTHVALUESNVPROC (glfwGetProcAddress ("glResolveDepthValuesNV"));
02252 glResumeTransformFeedback =
PFNGLRESUMETRANSFORMFEEDBACKPROC (glfwGetProcAddress ("glResumeTransformFeedback"));
02253 glSampleCoverage = PFNGLSAMPLECOVERAGEPROC (glfwGetProcAddress ("glSampleCoverage"));
02254 glSampleMaski = PFNGLSAMPLEMASKIPROC (glfwGetProcAddress ("glSampleMaski"));
02255 glSamplerParameterIiv = PFNGLSAMPLERPARAMETERIIVPROC (glfwGetProcAddress ("glSamplerParameterIiv"));
02256 glSamplerParameterIuiv =
PFNGLSAMPLERPARAMETERIUIVPROC (glfwGetProcAddress ("glSamplerParameterIuiv"));
02257 glSamplerParameterf = PFNGLSAMPLERPARAMETERFPROC (glfwGetProcAddress ("glSamplerParameterf"));
02258 glSamplerParameterfv = PFNGLSAMPLERPARAMETERFVPROC (glfwGetProcAddress ("glSamplerParameterfv"));
02259 glSamplerParameteri = PFNGLSAMPLERPARAMETERIPROC (glfwGetProcAddress ("glSamplerParameteri"));
02260 glSamplerParameteriv = PFNGLSAMPLERPARAMETERIVPROC (glfwGetProcAddress ("glSamplerParameteriv"));
02261 glScissor = PFNGLSCISSORPROC (glfwGetProcAddress ("glScissor"));
02262 glScissorArrayv = PFNGLSCISSORARRAYVPROC (glfwGetProcAddress ("glScissorArrayv"));
02263 glScissorIndexed = PFNGLSCISSORINDEXEDPROC (glfwGetProcAddress ("glScissorIndexed"));
02264 glScissorIndexedv = PFNGLSCISSORINDEXEDVPROC (glfwGetProcAddress ("glScissorIndexedv"));
02265 glSecondaryColorFormatNV =
PFNGLSECONDARYCOLORFORMATNVPROC (glfwGetProcAddress ("glSecondaryColorFormatNV"));
02266 glSelectPerfMonitorCountersAMD =
PFNGLSELECTPERFMONITORCOUNTERSAMDPROC (glfwGetProcAddress ("glSelectPerfMonitorCountersAMD"));
02267 glShaderBinary = PFNGLSHADERBINARYPROC (glfwGetProcAddress ("glShaderBinary"));
02268 glShaderSource = PFNGLSHADERSOURCEPROC (glfwGetProcAddress ("glShaderSource"));
02269 glShaderStorageBlockBinding =
PFNGLSHADERSTORAGEBLOCKBINDINGPROC (glfwGetProcAddress ("glShaderStorageBlockBinding"));
02270 glSignalVkfenceNV = PFNGLSIGNALVKFENCEENVPROC (glfwGetProcAddress ("glSignalVkfenceNV"));
02271 glSignalVkSemaphoreNV = PFNGLSIGNALVKSEMAPHORENVPROC (glfwGetProcAddress ("glSignalVkSemaphoreNV"));
02272 glSpecializeShaderARB = PFNGLSPECIALIZESHADERARBPROC (glfwGetProcAddress ("glSpecializeShaderARB"));
02273 glStateCaptureNV = PFNGLSTATECAPTURENVPROC (glfwGetProcAddress ("glStateCaptureNV"));
02274 glStencilFillPathInstancedNV =
PFNGLSTENCILFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilFillPathInstancedNV"));
02275 glStencilFillPathNV = PFNGLSTENCILFILLPATHNVPROC (glfwGetProcAddress ("glStencilFillPathNV"));
02276 glStencilFunc = PFNGLSTENCILFUNCPROC (glfwGetProcAddress ("glStencilFunc"));
02277 glStencilFuncSeparate = PFNGLSTENCILFUNCSEPARATEPROC (glfwGetProcAddress ("glStencilFuncSeparate"));
02278 glStencilMask = PFNGLSTENCILMASKPROC (glfwGetProcAddress ("glStencilMask"));
02279 glStencilMaskSeparate = PFNGLSTENCILMASKSEPARATEPROC (glfwGetProcAddress ("glStencilMaskSeparate"));
02280 glStencilOp = PFNGLSTENCILOPPROC (glfwGetProcAddress ("glStencilOp"));
02281 glStencilOpSeparate = PFNGLSTENCILOPSEPARATEPROC (glfwGetProcAddress ("glStencilOpSeparate"));
02282 glStencilStrokePathInstancedNV =
PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilStrokePathInstancedNV"));
02283 glStencilStrokePathNV = PFNGLSTENCILSTROKEPATHNVPROC (glfwGetProcAddress ("glStencilStrokePathNV"));
02284 glStencilThenCoverFillPathInstancedNV =
PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilThenCoverFillPathInstancedNV"));
02285 glStencilThenCoverFillPathNV =
PFNGLSTENCILTHENCOVERFILLPATHNVPROC (glfwGetProcAddress ("glStencilThenCoverFillPathNV"));
02286 glStencilThenCoverStrokePathInstancedNV =
PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC (glfwGetProcAddress ("glStencilThenCoverStrokePathInstancedNV"));
02287 glStencilThenCoverStrokePathNV =
PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC (glfwGetProcAddress ("glStencilThenCoverStrokePathNV"));
02288 glSubpixelPrecisionBiasNV =
PFNGLSUBPIXELPRECISIONBIASNVPROC (glfwGetProcAddress ("glSubpixelPrecisionBiasNV"));
02289 glTexBuffer = PFNGLTEXBUFFERPROC (glfwGetProcAddress ("glTexBuffer"));
02290 glTexBufferARB = PFNGLTEXBUFFERARBPROC (glfwGetProcAddress ("glTexBufferARB"));
02291 glTexBufferRange = PFNGLTEXBUFFERRANGEPROC (glfwGetProcAddress ("glTexBufferRange"));
02292 glTexCoordFormatNV = PFNGLTEXCOORDFORMATNVPROC (glfwGetProcAddress ("glTexCoordFormatNV"));
02293 glTexImage1D = PFNGLTEXIMAGE1DPROC (glfwGetProcAddress ("glTexImage1D"));
02294 glTexImage2D = PFNGLTEXIMAGE2DPROC (glfwGetProcAddress ("glTexImage2D"));
02295 glTexImage2DMultisample =
PFNGLTEXIMAGE2DMULTISAMPLEPROC (glfwGetProcAddress ("glTexImage2DMultisample"));
02296 glTexImage3D = PFNGLTEXIMAGE3DPROC (glfwGetProcAddress ("glTexImage3D"));
02297 glTexImage3DMultisample =
PFNGLTEXIMAGE3DMULTISAMPLEPROC (glfwGetProcAddress ("glTexImage3DMultisample"));
02298 glTexPageCommitmentARB =
PFNGLTEXPAGECOMMITMENTARBPROC (glfwGetProcAddress ("glTexPageCommitmentARB"));
```

```

02299     glTexParameterIiv = PFNGLTEXPARAMETERIIVPROC (glfwGetProcAddress("glTexParameterIiv"));
02300     glTexParameterIuiv = PFNGLTEXPARAMETERIUIVPROC (glfwGetProcAddress("glTexParameterIuiv"));
02301     glTexParameterf = PFNGLTEXPARAMETERFPROC (glfwGetProcAddress("glTexParameterf"));
02302     glTexParameterfv = PFNGLTEXPARAMETERFVPROC (glfwGetProcAddress("glTexParameterfv"));
02303     glTexParameteri = PFNGLTEXPARAMETERIPROC (glfwGetProcAddress("glTexParameteri"));
02304     glTexParameteriv = PFNGLTEXPARAMETERIVPROC (glfwGetProcAddress("glTexParameteriv"));
02305     glTexStorage1D = PFNGLTEXSTORAGE1DPROC (glfwGetProcAddress("glTexStorage1D"));
02306     glTexStorage2D = PFNGLTEXSTORAGE2DPROC (glfwGetProcAddress("glTexStorage2D"));
02307     glTexStorage2DMultisample =
PFNGLTEXSTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage2DMultisample"));
02308     glTexStorage3D = PFNGLTEXSTORAGE3DPROC (glfwGetProcAddress("glTexStorage3D"));
02309     glTexStorage3DMultisample =
PFNGLTEXSTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTexStorage3DMultisample"));
02310     glTexSubImage1D = PFNGLTEXSUBIMAGE1DPROC (glfwGetProcAddress("glTexSubImage1D"));
02311     glTexSubImage2D = PFNGLTEXSUBIMAGE2DPROC (glfwGetProcAddress("glTexSubImage2D"));
02312     glTexSubImage3D = PFNGLTEXSUBIMAGE3DPROC (glfwGetProcAddress("glTexSubImage3D"));
02313     glTextureBarrier = PFNGLTEXTUREBARRIERPROC (glfwGetProcAddress("glTextureBarrier"));
02314     glTextureBarrierNV = PFNGLTEXTUREBARRIERNVPROC (glfwGetProcAddress("glTextureBarrierNV"));
02315     glTextureBuffer = PFNGLTEXTUREBUFFERPROC (glfwGetProcAddress("glTextureBuffer"));
02316     glTextureBufferEXT = PFNGLTEXTUREBUFFEREXTPROC (glfwGetProcAddress("glTextureBufferEXT"));
02317     glTextureBufferRange = PFNGLTEXTUREBUFFERRANGEPROC (glfwGetProcAddress("glTextureBufferRange"));
02318     glTextureBufferRangeEXT =
PFNGLTEXTUREBUFFERRANGEEXTPROC (glfwGetProcAddress("glTextureBufferRangeEXT"));
02319     glTextureImage1DEXT = PFNGLTEXTUREIMAGE1DEXTPROC (glfwGetProcAddress("glTextureImage1DEXT"));
02320     glTextureImage2DEXT = PFNGLTEXTUREIMAGE2DEXTPROC (glfwGetProcAddress("glTextureImage2DEXT"));
02321     glTextureImage3DEXT = PFNGLTEXTUREIMAGE3DEXTPROC (glfwGetProcAddress("glTextureImage3DEXT"));
02322     glTexturePageCommitmentEXT =
PFNGLTEXTUREPAGECOMMITMENTEXTPROC (glfwGetProcAddress("glTexturePageCommitmentEXT"));
02323     glTextureParameterIiv = PFNGLTEXTUREPARAMETERIIVPROC (glfwGetProcAddress("glTextureParameterIiv"));
02324     glTextureParameterIivEXT =
PFNGLTEXTUREPARAMETERIIVEXTPROC (glfwGetProcAddress("glTextureParameterIivEXT"));
02325     glTextureParameterIuiv =
PFNGLTEXTUREPARAMETERIUIVPROC (glfwGetProcAddress("glTextureParameterIuiv"));
02326     glTextureParameterIuivEXT =
PFNGLTEXTUREPARAMETERIUIVEXTPROC (glfwGetProcAddress("glTextureParameterIuivEXT"));
02327     glTextureParameterf = PFNGLTEXTUREPARAMETERFPROC (glfwGetProcAddress("glTextureParameterf"));
02328     glTextureParameterfEXT =
PFNGLTEXTUREPARAMETERFEXTPROC (glfwGetProcAddress("glTextureParameterfEXT"));
02329     glTextureParameterfv = PFNGLTEXTUREPARAMETERFVPROC (glfwGetProcAddress("glTextureParameterfv"));
02330     glTextureParameterfvEXT =
PFNGLTEXTUREPARAMETERFVEXTPROC (glfwGetProcAddress("glTextureParameterfvEXT"));
02331     glTextureParameteri = PFNGLTEXTUREPARAMETERIPROC (glfwGetProcAddress("glTextureParameteri"));
02332     glTextureParameteriEXT =
PFNGLTEXTUREPARAMETERIEXTPROC (glfwGetProcAddress("glTextureParameteriEXT"));
02333     glTextureParameteriv = PFNGLTEXTUREPARAMETERIVPROC (glfwGetProcAddress("glTextureParameteriv"));
02334     glTextureParameterivEXT =
PFNGLTEXTUREPARAMETERIVEXTPROC (glfwGetProcAddress("glTextureParameterivEXT"));
02335     glTextureRenderbufferEXT =
PFNGLTEXTURERENDERBUFFEREXTPROC (glfwGetProcAddress("glTextureRenderbufferEXT"));
02336     glTextureStorage1D = PFNGLTEXTURESTORAGE1DPROC (glfwGetProcAddress("glTextureStorage1D"));
02337     glTextureStorage1DEXT = PFNGLTEXTURESTORAGE1DEXTPROC (glfwGetProcAddress("glTextureStorage1DEXT"));
02338     glTextureStorage2D = PFNGLTEXTURESTORAGE2DPROC (glfwGetProcAddress("glTextureStorage2D"));
02339     glTextureStorage2DEXT = PFNGLTEXTURESTORAGE2DEXTPROC (glfwGetProcAddress("glTextureStorage2DEXT"));
02340     glTextureStorage2DMultisample =
PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage2DMultisample"));
02341     glTextureStorage2DMultisampleEXT =
PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage2DMultisampleEXT"));
02342     glTextureStorage3D = PFNGLTEXTURESTORAGE3DPROC (glfwGetProcAddress("glTextureStorage3D"));
02343     glTextureStorage3DEXT = PFNGLTEXTURESTORAGE3DEXTPROC (glfwGetProcAddress("glTextureStorage3DEXT"));
02344     glTextureStorage3DMultisample =
PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC (glfwGetProcAddress("glTextureStorage3DMultisample"));
02345     glTextureStorage3DMultisampleEXT =
PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC (glfwGetProcAddress("glTextureStorage3DMultisampleEXT"));
02346     glTextureSubImage1D = PFNGLTEXTURESUBIMAGE1DPROC (glfwGetProcAddress("glTextureSubImage1D"));
02347     glTextureSubImage1DEXT =
PFNGLTEXTURESUBIMAGE1DEXTPROC (glfwGetProcAddress("glTextureSubImage1DEXT"));
02348     glTextureSubImage2D = PFNGLTEXTURESUBIMAGE2DPROC (glfwGetProcAddress("glTextureSubImage2D"));
02349     glTextureSubImage2DEXT =
PFNGLTEXTURESUBIMAGE2DEXTPROC (glfwGetProcAddress("glTextureSubImage2DEXT"));
02350     glTextureSubImage3D = PFNGLTEXTURESUBIMAGE3DPROC (glfwGetProcAddress("glTextureSubImage3D"));
02351     glTextureSubImage3DEXT =
PFNGLTEXTURESUBIMAGE3DEXTPROC (glfwGetProcAddress("glTextureSubImage3DEXT"));
02352     glTextureView = PFNGLTEXTUREVIEWPROC (glfwGetProcAddress("glTextureView"));
02353     glTransformFeedbackBufferBase =
PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC (glfwGetProcAddress("glTransformFeedbackBufferBase"));
02354     glTransformFeedbackBufferRange =
PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC (glfwGetProcAddress("glTransformFeedbackBufferRange"));
02355     glTransformFeedbackVaryings =
PFNGLTRANSFORMFEEDBACKVARYINGSPROC (glfwGetProcAddress("glTransformFeedbackVaryings"));
02356     glTransformPathNV = PFNGLTRANSFORMPATHNVPROC (glfwGetProcAddress("glTransformPathNV"));
02357     glUniform1d = PFNGLUNIFORM1DPROC (glfwGetProcAddress("glUniform1d"));
02358     glUniform1dv = PFNGLUNIFORM1DVPROC (glfwGetProcAddress("glUniform1dv"));
02359     glUniform1f = PFNGLUNIFORM1FPROC (glfwGetProcAddress("glUniform1f"));
02360     glUniform1fv = PFNGLUNIFORM1FVPROC (glfwGetProcAddress("glUniform1fv"));
02361     glUniform1i = PFNGLUNIFORM1IPROC (glfwGetProcAddress("glUniform1i"));
02362     glUniform1i64ARB = PFNGLUNIFORM1I64ARBPROC (glfwGetProcAddress("glUniform1i64ARB"));
02363     glUniform1i64NV = PFNGLUNIFORM1I64NVPROC (glfwGetProcAddress("glUniform1i64NV"));

```



```
02364 glUniform1i64vARB = PFNGLUNIFORM1I64VARBPROC (glfwGetProcAddress("glUniform1i64vARB"));
02365 glUniform1i64vNV = PFNGLUNIFORM1I64VNVPROC (glfwGetProcAddress("glUniform1i64vNV"));
02366 glUniform1iv = PFNGLUNIFORM1IVPROC (glfwGetProcAddress("glUniform1iv"));
02367 glUniform1ui = PFNGLUNIFORM1UIPROC (glfwGetProcAddress("glUniform1ui"));
02368 glUniform1ui64ARB = PFNGLUNIFORM1UI64ARBPROC (glfwGetProcAddress("glUniform1ui64ARB"));
02369 glUniform1ui64NV = PFNGLUNIFORM1UI64NVPROC (glfwGetProcAddress("glUniform1ui64NV"));
02370 glUniform1ui64vARB = PFNGLUNIFORM1UI64VARBPROC (glfwGetProcAddress("glUniform1ui64vARB"));
02371 glUniform1ui64vNV = PFNGLUNIFORM1UI64VNVPROC (glfwGetProcAddress("glUniform1ui64vNV"));
02372 glUniform1uiv = PFNGLUNIFORM1UIVPROC (glfwGetProcAddress("glUniform1uiv"));
02373 glUniform2d = PFNGLUNIFORM2DPROC (glfwGetProcAddress("glUniform2d"));
02374 glUniform2dv = PFNGLUNIFORM2DVPROC (glfwGetProcAddress("glUniform2dv"));
02375 glUniform2f = PFNGLUNIFORM2FPROC (glfwGetProcAddress("glUniform2f"));
02376 glUniform2fv = PFNGLUNIFORM2FVPROC (glfwGetProcAddress("glUniform2fv"));
02377 glUniform2i = PFNGLUNIFORM2IPROC (glfwGetProcAddress("glUniform2i"));
02378 glUniform2i64ARB = PFNGLUNIFORM2I64ARBPROC (glfwGetProcAddress("glUniform2i64ARB"));
02379 glUniform2i64NV = PFNGLUNIFORM2I64NVPROC (glfwGetProcAddress("glUniform2i64NV"));
02380 glUniform2i64vARB = PFNGLUNIFORM2I64VARBPROC (glfwGetProcAddress("glUniform2i64vARB"));
02381 glUniform2i64vNV = PFNGLUNIFORM2I64VNVPROC (glfwGetProcAddress("glUniform2i64vNV"));
02382 glUniform2iv = PFNGLUNIFORM2IVPROC (glfwGetProcAddress("glUniform2iv"));
02383 glUniform2ui = PFNGLUNIFORM2UIPROC (glfwGetProcAddress("glUniform2ui"));
02384 glUniform2ui64ARB = PFNGLUNIFORM2UI64ARBPROC (glfwGetProcAddress("glUniform2ui64ARB"));
02385 glUniform2ui64NV = PFNGLUNIFORM2UI64NVPROC (glfwGetProcAddress("glUniform2ui64NV"));
02386 glUniform2ui64vARB = PFNGLUNIFORM2UI64VARBPROC (glfwGetProcAddress("glUniform2ui64vARB"));
02387 glUniform2ui64vNV = PFNGLUNIFORM2UI64VNVPROC (glfwGetProcAddress("glUniform2ui64vNV"));
02388 glUniform2uiv = PFNGLUNIFORM2UIVPROC (glfwGetProcAddress("glUniform2uiv"));
02389 glUniform3d = PFNGLUNIFORM3DPROC (glfwGetProcAddress("glUniform3d"));
02390 glUniform3dv = PFNGLUNIFORM3DVPROC (glfwGetProcAddress("glUniform3dv"));
02391 glUniform3f = PFNGLUNIFORM3FPROC (glfwGetProcAddress("glUniform3f"));
02392 glUniform3fv = PFNGLUNIFORM3FVPROC (glfwGetProcAddress("glUniform3fv"));
02393 glUniform3i = PFNGLUNIFORM3IPROC (glfwGetProcAddress("glUniform3i"));
02394 glUniform3i64ARB = PFNGLUNIFORM3I64ARBPROC (glfwGetProcAddress("glUniform3i64ARB"));
02395 glUniform3i64NV = PFNGLUNIFORM3I64NVPROC (glfwGetProcAddress("glUniform3i64NV"));
02396 glUniform3i64vARB = PFNGLUNIFORM3I64VARBPROC (glfwGetProcAddress("glUniform3i64vARB"));
02397 glUniform3i64vNV = PFNGLUNIFORM3I64VNVPROC (glfwGetProcAddress("glUniform3i64vNV"));
02398 glUniform3iv = PFNGLUNIFORM3IVPROC (glfwGetProcAddress("glUniform3iv"));
02399 glUniform3ui = PFNGLUNIFORM3UIPROC (glfwGetProcAddress("glUniform3ui"));
02400 glUniform3ui64ARB = PFNGLUNIFORM3UI64ARBPROC (glfwGetProcAddress("glUniform3ui64ARB"));
02401 glUniform3ui64NV = PFNGLUNIFORM3UI64NVPROC (glfwGetProcAddress("glUniform3ui64NV"));
02402 glUniform3ui64vARB = PFNGLUNIFORM3UI64VARBPROC (glfwGetProcAddress("glUniform3ui64vARB"));
02403 glUniform3ui64vNV = PFNGLUNIFORM3UI64VNVPROC (glfwGetProcAddress("glUniform3ui64vNV"));
02404 glUniform3uiv = PFNGLUNIFORM3UIVPROC (glfwGetProcAddress("glUniform3uiv"));
02405 glUniform4d = PFNGLUNIFORM4DPROC (glfwGetProcAddress("glUniform4d"));
02406 glUniform4dv = PFNGLUNIFORM4DVPROC (glfwGetProcAddress("glUniform4dv"));
02407 glUniform4f = PFNGLUNIFORM4FPROC (glfwGetProcAddress("glUniform4f"));
02408 glUniform4fv = PFNGLUNIFORM4FVPROC (glfwGetProcAddress("glUniform4fv"));
02409 glUniform4i = PFNGLUNIFORM4IPROC (glfwGetProcAddress("glUniform4i"));
02410 glUniform4i64ARB = PFNGLUNIFORM4I64ARBPROC (glfwGetProcAddress("glUniform4i64ARB"));
02411 glUniform4i64NV = PFNGLUNIFORM4I64NVPROC (glfwGetProcAddress("glUniform4i64NV"));
02412 glUniform4i64vARB = PFNGLUNIFORM4I64VARBPROC (glfwGetProcAddress("glUniform4i64vARB"));
02413 glUniform4i64vNV = PFNGLUNIFORM4I64VNVPROC (glfwGetProcAddress("glUniform4i64vNV"));
02414 glUniform4iv = PFNGLUNIFORM4IVPROC (glfwGetProcAddress("glUniform4iv"));
02415 glUniform4ui = PFNGLUNIFORM4UIPROC (glfwGetProcAddress("glUniform4ui"));
02416 glUniform4ui64ARB = PFNGLUNIFORM4UI64ARBPROC (glfwGetProcAddress("glUniform4ui64ARB"));
02417 glUniform4ui64NV = PFNGLUNIFORM4UI64NVPROC (glfwGetProcAddress("glUniform4ui64NV"));
02418 glUniform4ui64vARB = PFNGLUNIFORM4UI64VARBPROC (glfwGetProcAddress("glUniform4ui64vARB"));
02419 glUniform4ui64vNV = PFNGLUNIFORM4UI64VNVPROC (glfwGetProcAddress("glUniform4ui64vNV"));
02420 glUniform4uiv = PFNGLUNIFORM4UIVPROC (glfwGetProcAddress("glUniform4uiv"));
02421 glUniformBlockBinding = PFNGLUNIFORMBLOCKBINDINGPROC (glfwGetProcAddress("glUniformBlockBinding"));
02422 glUniformHandleui64ARB = PFNGLUNIFORMHANDLEUI64ARBPROC (glfwGetProcAddress("glUniformHandleui64ARB"));
02423 glUniformHandleui64NV = PFNGLUNIFORMHANDLEUI64NVPROC (glfwGetProcAddress("glUniformHandleui64NV"));
02424 glUniformHandleui64vARB = PFNGLUNIFORMHANDLEUI64VARBPROC (glfwGetProcAddress("glUniformHandleui64vARB"));
02425 glUniformHandleui64vNV = PFNGLUNIFORMHANDLEUI64VNVPROC (glfwGetProcAddress("glUniformHandleui64vNV"));
02426 glUniformMatrix2dv = PFNGLUNIFORMMATRIX2DVPROC (glfwGetProcAddress("glUniformMatrix2dv"));
02427 glUniformMatrix2fv = PFNGLUNIFORMMATRIX2FVPROC (glfwGetProcAddress("glUniformMatrix2fv"));
02428 glUniformMatrix2x3dv = PFNGLUNIFORMMATRIX2X3DVPROC (glfwGetProcAddress("glUniformMatrix2x3dv"));
02429 glUniformMatrix2x3fv = PFNGLUNIFORMMATRIX2X3FVPROC (glfwGetProcAddress("glUniformMatrix2x3fv"));
02430 glUniformMatrix2x4dv = PFNGLUNIFORMMATRIX2X4DVPROC (glfwGetProcAddress("glUniformMatrix2x4dv"));
02431 glUniformMatrix2x4fv = PFNGLUNIFORMMATRIX2X4FVPROC (glfwGetProcAddress("glUniformMatrix2x4fv"));
02432 glUniformMatrix3dv = PFNGLUNIFORMMATRIX3DVPROC (glfwGetProcAddress("glUniformMatrix3dv"));
02433 glUniformMatrix3fv = PFNGLUNIFORMMATRIX3FVPROC (glfwGetProcAddress("glUniformMatrix3fv"));
02434 glUniformMatrix3x2dv = PFNGLUNIFORMMATRIX3X2DVPROC (glfwGetProcAddress("glUniformMatrix3x2dv"));
02435 glUniformMatrix3x2fv = PFNGLUNIFORMMATRIX3X2FVPROC (glfwGetProcAddress("glUniformMatrix3x2fv"));
02436 glUniformMatrix3x4dv = PFNGLUNIFORMMATRIX3X4DVPROC (glfwGetProcAddress("glUniformMatrix3x4dv"));
02437 glUniformMatrix3x4fv = PFNGLUNIFORMMATRIX3X4FVPROC (glfwGetProcAddress("glUniformMatrix3x4fv"));
02438 glUniformMatrix4dv = PFNGLUNIFORMMATRIX4DVPROC (glfwGetProcAddress("glUniformMatrix4dv"));
02439 glUniformMatrix4fv = PFNGLUNIFORMMATRIX4FVPROC (glfwGetProcAddress("glUniformMatrix4fv"));
02440 glUniformMatrix4x2dv = PFNGLUNIFORMMATRIX4X2DVPROC (glfwGetProcAddress("glUniformMatrix4x2dv"));
02441 glUniformMatrix4x2fv = PFNGLUNIFORMMATRIX4X2FVPROC (glfwGetProcAddress("glUniformMatrix4x2fv"));
02442 glUniformMatrix4x3dv = PFNGLUNIFORMMATRIX4X3DVPROC (glfwGetProcAddress("glUniformMatrix4x3dv"));
02443 glUniformMatrix4x3fv = PFNGLUNIFORMMATRIX4X3FVPROC (glfwGetProcAddress("glUniformMatrix4x3fv"));
02444 glUniformSubroutinesuiv = PFNGLUNIFORMSUBROUTINESUIVPROC (glfwGetProcAddress("glUniformSubroutinesuiv"));
02445 glUniformui64NV = PFNGLUNIFORMUI64NVPROC (glfwGetProcAddress("glUniformui64NV"));
02446 glUniformui64vNV = PFNGLUNIFORMUI64VNVPROC (glfwGetProcAddress("glUniformui64vNV"));
```

```

02447     glUnmapBuffer = PFNGLUNMAPBUFFERPROC (glfwGetProcAddress ("glUnmapBuffer"));
02448     glUnmapNamedBuffer = PFNGLUNMAPNAMEDBUFFERPROC (glfwGetProcAddress ("glUnmapNamedBuffer"));
02449     glUnmapNamedBufferEXT = PFNGLUNMAPNAMEDBUFFEREXTPROC (glfwGetProcAddress ("glUnmapNamedBufferEXT"));
02450     glUseProgram = PFNGLUSEPROGRAMPROC (glfwGetProcAddress ("glUseProgram"));
02451     glUseProgramStages = PFNGLUSEPROGRAMSTAGESPROC (glfwGetProcAddress ("glUseProgramStages"));
02452     glUseShaderProgramEXT = PFNGLUSESHADERPROGRAMEXTPROC (glfwGetProcAddress ("glUseShaderProgramEXT"));
02453     glValidateProgram = PFNGLVALIDATEPROGRAMPROC (glfwGetProcAddress ("glValidateProgram"));
02454     glValidateProgramPipeline =
PFNGLVALIDATEPROGRAMPIPELINEPROC (glfwGetProcAddress ("glValidateProgramPipeline"));
02455     glVertexArrayAttribBinding =
PFNGLVERTEXARRAYATTRIBBINDINGPROC (glfwGetProcAddress ("glVertexArrayAttribBinding"));
02456     glVertexArrayAttribFormat =
PFNGLVERTEXARRAYATTRIBFORMATPROC (glfwGetProcAddress ("glVertexArrayAttribFormat"));
02457     glVertexArrayAttribIFormat =
PFNGLVERTEXARRAYATTRIBIFORMATPROC (glfwGetProcAddress ("glVertexArrayAttribIFormat"));
02458     glVertexArrayAttribLFormat =
PFNGLVERTEXARRAYATTRIBLFORMATPROC (glfwGetProcAddress ("glVertexArrayAttribLFormat"));
02459     glVertexArrayBindVertexBufferEXT =
PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC (glfwGetProcAddress ("glVertexArrayBindVertexBufferEXT"));
02460     glVertexArrayBindingDivisor =
PFNGLVERTEXARRAYBINDINGDIVISORPROC (glfwGetProcAddress ("glVertexArrayBindingDivisor"));
02461     glVertexArrayColorOffsetEXT =
PFNGLVERTEXARRAYCOLOROFFSETPROC (glfwGetProcAddress ("glVertexArrayColorOffsetEXT"));
02462     glVertexArrayEdgeFlagOffsetEXT =
PFNGLVERTEXARRAYEDGEFLAGOFFSETPROC (glfwGetProcAddress ("glVertexArrayEdgeFlagOffsetEXT"));
02463     glVertexArrayElementBuffer =
PFNGLVERTEXARRAYELEMENTBUFFERPROC (glfwGetProcAddress ("glVertexArrayElementBuffer"));
02464     glVertexArrayFogCoordOffsetEXT =
PFNGLVERTEXARRAYFOGCOORDOFFSETPROC (glfwGetProcAddress ("glVertexArrayFogCoordOffsetEXT"));
02465     glVertexArrayIndexOffsetEXT =
PFNGLVERTEXARRAYINDEXOFFSETPROC (glfwGetProcAddress ("glVertexArrayIndexOffsetEXT"));
02466     glVertexArrayMultiTexCoordOffsetEXT =
PFNGLVERTEXARRAYMULTITEXCOORDOFFSETPROC (glfwGetProcAddress ("glVertexArrayMultiTexCoordOffsetEXT"));
02467     glVertexArrayNormalOffsetEXT =
PFNGLVERTEXARRAYNORMALOFFSETPROC (glfwGetProcAddress ("glVertexArrayNormalOffsetEXT"));
02468     glVertexArraySecondaryColorOffsetEXT =
PFNGLVERTEXARRAYSECONDARYCOLOROFFSETPROC (glfwGetProcAddress ("glVertexArraySecondaryColorOffsetEXT"));
02469     glVertexArrayTexCoordOffsetEXT =
PFNGLVERTEXARRAYTEXCOORDOFFSETPROC (glfwGetProcAddress ("glVertexArrayTexCoordOffsetEXT"));
02470     glVertexArrayVertexAttribBindingEXT =
PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribBindingEXT"));
02471     glVertexArrayVertexAttribDivisorEXT =
PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribDivisorEXT"));
02472     glVertexArrayVertexAttribFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribFormatEXT"));
02473     glVertexArrayVertexAttribIFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIFormatEXT"));
02474     glVertexArrayVertexAttribIOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETPROC (glfwGetProcAddress ("glVertexArrayVertexAttribIOffsetEXT"));
02475     glVertexArrayVertexAttribLFormatEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLFormatEXT"));
02476     glVertexArrayVertexAttribLOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETPROC (glfwGetProcAddress ("glVertexArrayVertexAttribLOffsetEXT"));
02477     glVertexArrayVertexAttribOffsetEXT =
PFNGLVERTEXARRAYVERTEXATTRIBOFFSETPROC (glfwGetProcAddress ("glVertexArrayVertexAttribOffsetEXT"));
02478     glVertexArrayVertexBindingDivisorEXT =
PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC (glfwGetProcAddress ("glVertexArrayVertexBindingDivisorEXT"));
02479     glVertexArrayVertexBuffer =
PFNGLVERTEXARRAYVERTEXBUFFERPROC (glfwGetProcAddress ("glVertexArrayVertexBuffer"));
02480     glVertexArrayVertexBuffers =
PFNGLVERTEXARRAYVERTEXBUFFERSPROC (glfwGetProcAddress ("glVertexArrayVertexBuffers"));
02481     glVertexArrayVertexOffsetEXT =
PFNGLVERTEXARRAYVERTEXOFFSETPROC (glfwGetProcAddress ("glVertexArrayVertexOffsetEXT"));
02482     glVertexAttrib1d = PFNGLVERTEXATTRIB1DPROC (glfwGetProcAddress ("glVertexAttrib1d"));
02483     glVertexAttrib1dv = PFNGLVERTEXATTRIB1DVPROC (glfwGetProcAddress ("glVertexAttrib1dv"));
02484     glVertexAttrib1f = PFNGLVERTEXATTRIB1FPROC (glfwGetProcAddress ("glVertexAttrib1f"));
02485     glVertexAttrib1fv = PFNGLVERTEXATTRIB1FVPROC (glfwGetProcAddress ("glVertexAttrib1fv"));
02486     glVertexAttrib1s = PFNGLVERTEXATTRIB1SPROC (glfwGetProcAddress ("glVertexAttrib1s"));
02487     glVertexAttrib1sv = PFNGLVERTEXATTRIB1SVPROC (glfwGetProcAddress ("glVertexAttrib1sv"));
02488     glVertexAttrib2d = PFNGLVERTEXATTRIB2DPROC (glfwGetProcAddress ("glVertexAttrib2d"));
02489     glVertexAttrib2dv = PFNGLVERTEXATTRIB2DVPROC (glfwGetProcAddress ("glVertexAttrib2dv"));
02490     glVertexAttrib2f = PFNGLVERTEXATTRIB2FPROC (glfwGetProcAddress ("glVertexAttrib2f"));
02491     glVertexAttrib2fv = PFNGLVERTEXATTRIB2FVPROC (glfwGetProcAddress ("glVertexAttrib2fv"));
02492     glVertexAttrib2s = PFNGLVERTEXATTRIB2SPROC (glfwGetProcAddress ("glVertexAttrib2s"));
02493     glVertexAttrib2sv = PFNGLVERTEXATTRIB2SVPROC (glfwGetProcAddress ("glVertexAttrib2sv"));
02494     glVertexAttrib3d = PFNGLVERTEXATTRIB3DPROC (glfwGetProcAddress ("glVertexAttrib3d"));
02495     glVertexAttrib3dv = PFNGLVERTEXATTRIB3DVPROC (glfwGetProcAddress ("glVertexAttrib3dv"));
02496     glVertexAttrib3f = PFNGLVERTEXATTRIB3FPROC (glfwGetProcAddress ("glVertexAttrib3f"));
02497     glVertexAttrib3fv = PFNGLVERTEXATTRIB3FVPROC (glfwGetProcAddress ("glVertexAttrib3fv"));
02498     glVertexAttrib3s = PFNGLVERTEXATTRIB3SPROC (glfwGetProcAddress ("glVertexAttrib3s"));
02499     glVertexAttrib3sv = PFNGLVERTEXATTRIB3SVPROC (glfwGetProcAddress ("glVertexAttrib3sv"));
02500     glVertexAttrib4Nbv = PFNGLVERTEXATTRIB4NBVPROC (glfwGetProcAddress ("glVertexAttrib4Nbv"));
02501     glVertexAttrib4Niv = PFNGLVERTEXATTRIB4NIVPROC (glfwGetProcAddress ("glVertexAttrib4Niv"));
02502     glVertexAttrib4Nsv = PFNGLVERTEXATTRIB4NSVPROC (glfwGetProcAddress ("glVertexAttrib4Nsv"));
02503     glVertexAttrib4Nub = PFNGLVERTEXATTRIB4NUBPROC (glfwGetProcAddress ("glVertexAttrib4Nub"));
02504     glVertexAttrib4Nubv = PFNGLVERTEXATTRIB4NUBVPROC (glfwGetProcAddress ("glVertexAttrib4Nubv"));
02505     glVertexAttrib4Nuiv = PFNGLVERTEXATTRIB4NUIVPROC (glfwGetProcAddress ("glVertexAttrib4Nuiv"));

```

```

02506     glVertexAttrib4Nusv = PFNGLVERTEXATTRIB4NUSVPROC (glfwGetProcAddress("glVertexAttrib4Nusv"));
02507     glVertexAttrib4bv = PFNGLVERTEXATTRIB4BVPROC (glfwGetProcAddress("glVertexAttrib4bv"));
02508     glVertexAttrib4d = PFNGLVERTEXATTRIB4DPROC (glfwGetProcAddress("glVertexAttrib4d"));
02509     glVertexAttrib4dv = PFNGLVERTEXATTRIB4DVPROC (glfwGetProcAddress("glVertexAttrib4dv"));
02510     glVertexAttrib4f = PFNGLVERTEXATTRIB4FPROC (glfwGetProcAddress("glVertexAttrib4f"));
02511     glVertexAttrib4fv = PFNGLVERTEXATTRIB4FVPROC (glfwGetProcAddress("glVertexAttrib4fv"));
02512     glVertexAttrib4iv = PFNGLVERTEXATTRIB4IVPROC (glfwGetProcAddress("glVertexAttrib4iv"));
02513     glVertexAttrib4s = PFNGLVERTEXATTRIB4SPROC (glfwGetProcAddress("glVertexAttrib4s"));
02514     glVertexAttrib4sv = PFNGLVERTEXATTRIB4SVPROC (glfwGetProcAddress("glVertexAttrib4sv"));
02515     glVertexAttrib4ubv = PFNGLVERTEXATTRIB4UBVPROC (glfwGetProcAddress("glVertexAttrib4ubv"));
02516     glVertexAttrib4uiv = PFNGLVERTEXATTRIB4UIVPROC (glfwGetProcAddress("glVertexAttrib4uiv"));
02517     glVertexAttrib4usv = PFNGLVERTEXATTRIB4USVPROC (glfwGetProcAddress("glVertexAttrib4usv"));
02518     glVertexAttribBinding = PFNGLVERTEXATTRIBBINDINGPROC (glfwGetProcAddress("glVertexAttribBinding"));
02519     glVertexAttribDivisor = PFNGLVERTEXATTRIBDIVISORPROC (glfwGetProcAddress("glVertexAttribDivisor"));
02520     glVertexAttribDivisorARB = PFNGLVERTEXATTRIBDIVISORARBPROC (glfwGetProcAddress("glVertexAttribDivisorARB"));
02521     glVertexAttribFormat = PFNGLVERTEXATTRIBFORMATPROC (glfwGetProcAddress("glVertexAttribFormat"));
02522     glVertexAttribFormatNV = PFNGLVERTEXATTRIBFORMATNVPROC (glfwGetProcAddress("glVertexAttribFormatNV"));
02523     glVertexAttribI1i = PFNGLVERTEXATTRIBI1IPROC (glfwGetProcAddress("glVertexAttribI1i"));
02524     glVertexAttribI1iv = PFNGLVERTEXATTRIBI1IVPROC (glfwGetProcAddress("glVertexAttribI1iv"));
02525     glVertexAttribI1ui = PFNGLVERTEXATTRIBI1UIPROC (glfwGetProcAddress("glVertexAttribI1ui"));
02526     glVertexAttribI1uiv = PFNGLVERTEXATTRIBI1UIVPROC (glfwGetProcAddress("glVertexAttribI1uiv"));
02527     glVertexAttribI2i = PFNGLVERTEXATTRIBI2IPROC (glfwGetProcAddress("glVertexAttribI2i"));
02528     glVertexAttribI2iv = PFNGLVERTEXATTRIBI2IVPROC (glfwGetProcAddress("glVertexAttribI2iv"));
02529     glVertexAttribI2ui = PFNGLVERTEXATTRIBI2UIPROC (glfwGetProcAddress("glVertexAttribI2ui"));
02530     glVertexAttribI2uiv = PFNGLVERTEXATTRIBI2UIVPROC (glfwGetProcAddress("glVertexAttribI2uiv"));
02531     glVertexAttribI3i = PFNGLVERTEXATTRIBI3IPROC (glfwGetProcAddress("glVertexAttribI3i"));
02532     glVertexAttribI3iv = PFNGLVERTEXATTRIBI3IVPROC (glfwGetProcAddress("glVertexAttribI3iv"));
02533     glVertexAttribI3ui = PFNGLVERTEXATTRIBI3UIPROC (glfwGetProcAddress("glVertexAttribI3ui"));
02534     glVertexAttribI3uiv = PFNGLVERTEXATTRIBI3UIVPROC (glfwGetProcAddress("glVertexAttribI3uiv"));
02535     glVertexAttribI4bv = PFNGLVERTEXATTRIBI4BVPROC (glfwGetProcAddress("glVertexAttribI4bv"));
02536     glVertexAttribI4i = PFNGLVERTEXATTRIBI4IPROC (glfwGetProcAddress("glVertexAttribI4i"));
02537     glVertexAttribI4iv = PFNGLVERTEXATTRIBI4IVPROC (glfwGetProcAddress("glVertexAttribI4iv"));
02538     glVertexAttribI4sv = PFNGLVERTEXATTRIBI4SVPROC (glfwGetProcAddress("glVertexAttribI4sv"));
02539     glVertexAttribI4ubv = PFNGLVERTEXATTRIBI4UBVPROC (glfwGetProcAddress("glVertexAttribI4ubv"));
02540     glVertexAttribI4ui = PFNGLVERTEXATTRIBI4UIPROC (glfwGetProcAddress("glVertexAttribI4ui"));
02541     glVertexAttribI4uiv = PFNGLVERTEXATTRIBI4UIVPROC (glfwGetProcAddress("glVertexAttribI4uiv"));
02542     glVertexAttribI4usv = PFNGLVERTEXATTRIBI4USVPROC (glfwGetProcAddress("glVertexAttribI4usv"));
02543     glVertexAttribIFormat = PFNGLVERTEXATTRIBIFORMATPROC (glfwGetProcAddress("glVertexAttribIFormat"));
02544     glVertexAttribIFormatNV = PFNGLVERTEXATTRIBIFORMATNVPROC (glfwGetProcAddress("glVertexAttribIFormatNV"));
02545     glVertexAttribIPointer = PFNGLVERTEXATTRIBIPOINTERPROC (glfwGetProcAddress("glVertexAttribIPointer"));
02546     glVertexAttribL1d = PFNGLVERTEXATTRIBL1DPROC (glfwGetProcAddress("glVertexAttribL1d"));
02547     glVertexAttribL1dv = PFNGLVERTEXATTRIBL1DVPROC (glfwGetProcAddress("glVertexAttribL1dv"));
02548     glVertexAttribL1i64NV = PFNGLVERTEXATTRIBL1I64NVPROC (glfwGetProcAddress("glVertexAttribL1i64NV"));
02549     glVertexAttribL1i64vNV = PFNGLVERTEXATTRIBL1I64VNVPROC (glfwGetProcAddress("glVertexAttribL1i64vNV"));
02550     glVertexAttribL1ui64ARB = PFNGLVERTEXATTRIBL1UI64ARBPROC (glfwGetProcAddress("glVertexAttribL1ui64ARB"));
02551     glVertexAttribL1ui64NV = PFNGLVERTEXATTRIBL1UI64NVPROC (glfwGetProcAddress("glVertexAttribL1ui64NV"));
02552     glVertexAttribL1ui64vARB = PFNGLVERTEXATTRIBL1UI64VARBPROC (glfwGetProcAddress("glVertexAttribL1ui64vARB"));
02553     glVertexAttribL1ui64vNV = PFNGLVERTEXATTRIBL1UI64VNVPROC (glfwGetProcAddress("glVertexAttribL1ui64vNV"));
02554     glVertexAttribL2d = PFNGLVERTEXATTRIBL2DPROC (glfwGetProcAddress("glVertexAttribL2d"));
02555     glVertexAttribL2dv = PFNGLVERTEXATTRIBL2DVPROC (glfwGetProcAddress("glVertexAttribL2dv"));
02556     glVertexAttribL2i64NV = PFNGLVERTEXATTRIBL2I64NVPROC (glfwGetProcAddress("glVertexAttribL2i64NV"));
02557     glVertexAttribL2i64vNV = PFNGLVERTEXATTRIBL2I64VNVPROC (glfwGetProcAddress("glVertexAttribL2i64vNV"));
02558     glVertexAttribL2ui64NV = PFNGLVERTEXATTRIBL2UI64NVPROC (glfwGetProcAddress("glVertexAttribL2ui64NV"));
02559     glVertexAttribL2ui64vNV = PFNGLVERTEXATTRIBL2UI64VNVPROC (glfwGetProcAddress("glVertexAttribL2ui64vNV"));
02560     glVertexAttribL3d = PFNGLVERTEXATTRIBL3DPROC (glfwGetProcAddress("glVertexAttribL3d"));
02561     glVertexAttribL3dv = PFNGLVERTEXATTRIBL3DVPROC (glfwGetProcAddress("glVertexAttribL3dv"));
02562     glVertexAttribL3i64NV = PFNGLVERTEXATTRIBL3I64NVPROC (glfwGetProcAddress("glVertexAttribL3i64NV"));
02563     glVertexAttribL3i64vNV = PFNGLVERTEXATTRIBL3I64VNVPROC (glfwGetProcAddress("glVertexAttribL3i64vNV"));
02564     glVertexAttribL3ui64NV = PFNGLVERTEXATTRIBL3UI64NVPROC (glfwGetProcAddress("glVertexAttribL3ui64NV"));
02565     glVertexAttribL3ui64vNV = PFNGLVERTEXATTRIBL3UI64VNVPROC (glfwGetProcAddress("glVertexAttribL3ui64vNV"));
02566     glVertexAttribL4d = PFNGLVERTEXATTRIBL4DPROC (glfwGetProcAddress("glVertexAttribL4d"));
02567     glVertexAttribL4dv = PFNGLVERTEXATTRIBL4DVPROC (glfwGetProcAddress("glVertexAttribL4dv"));
02568     glVertexAttribL4i64NV = PFNGLVERTEXATTRIBL4I64NVPROC (glfwGetProcAddress("glVertexAttribL4i64NV"));
02569     glVertexAttribL4i64vNV = PFNGLVERTEXATTRIBL4I64VNVPROC (glfwGetProcAddress("glVertexAttribL4i64vNV"));
02570     glVertexAttribL4ui64NV = PFNGLVERTEXATTRIBL4UI64NVPROC (glfwGetProcAddress("glVertexAttribL4ui64NV"));
02571     glVertexAttribL4ui64vNV = PFNGLVERTEXATTRIBL4UI64VNVPROC (glfwGetProcAddress("glVertexAttribL4ui64vNV"));
02572     glVertexAttribLFormat = PFNGLVERTEXATTRIBLFORMATPROC (glfwGetProcAddress("glVertexAttribLFormat"));
02573     glVertexAttribLFormatNV = PFNGLVERTEXATTRIBLFORMATNVPROC (glfwGetProcAddress("glVertexAttribLFormatNV"));

```

```

02574     glVertexAttribLPointer =
PFNGLVERTEXATTRIBLPOINTERPROC (glfwGetProcAddress("glVertexAttribLPointer"));
02575     glVertexAttribP1ui = PFNGLVERTEXATTRIBP1UIPROC (glfwGetProcAddress("glVertexAttribP1ui"));
02576     glVertexAttribP1uiv = PFNGLVERTEXATTRIBP1UIVPROC (glfwGetProcAddress("glVertexAttribP1uiv"));
02577     glVertexAttribP2ui = PFNGLVERTEXATTRIBP2UIPROC (glfwGetProcAddress("glVertexAttribP2ui"));
02578     glVertexAttribP2uiv = PFNGLVERTEXATTRIBP2UIVPROC (glfwGetProcAddress("glVertexAttribP2uiv"));
02579     glVertexAttribP3ui = PFNGLVERTEXATTRIBP3UIPROC (glfwGetProcAddress("glVertexAttribP3ui"));
02580     glVertexAttribP3uiv = PFNGLVERTEXATTRIBP3UIVPROC (glfwGetProcAddress("glVertexAttribP3uiv"));
02581     glVertexAttribP4ui = PFNGLVERTEXATTRIBP4UIPROC (glfwGetProcAddress("glVertexAttribP4ui"));
02582     glVertexAttribP4uiv = PFNGLVERTEXATTRIBP4UIVPROC (glfwGetProcAddress("glVertexAttribP4uiv"));
02583     glVertexAttribPointer = PFNGLVERTEXATTRIBPOINTERPROC (glfwGetProcAddress("glVertexAttribPointer"));
02584     glVertexBindingDivisor =
PFNGLVERTEXBINDINGDIVISORPROC (glfwGetProcAddress("glVertexBindingDivisor"));
02585     glVertexFormatNV = PFNGLVERTEXFORMATNVPROC (glfwGetProcAddress("glVertexFormatNV"));
02586     glViewport = PFNGLVIEWPORTPROC (glfwGetProcAddress("glViewport"));
02587     glViewportArrayv = PFNGLVIEWPORTARRAYVPROC (glfwGetProcAddress("glViewportArrayv"));
02588     glViewportIndexedf = PFNGLVIEWPORTINDEXEDFPROC (glfwGetProcAddress("glViewportIndexedf"));
02589     glViewportIndexedfv = PFNGLVIEWPORTINDEXEDFVPROC (glfwGetProcAddress("glViewportIndexedfv"));
02590     glViewportPositionWScaleNV =
PFNGLVIEWPORTPOSITIONWSCALENVPROC (glfwGetProcAddress("glViewportPositionWScaleNV"));
02591     glViewportSwizzleNV = PFNGLVIEWPORTSWIZZLENVPROC (glfwGetProcAddress("glViewportSwizzleNV"));
02592     glWaitSync = PFNGLWAITSYNCPROC (glfwGetProcAddress("glWaitSync"));
02593     glWaitVkSemaphoreNV = PFNGLWAITVKSEMAPHORENVPROC (glfwGetProcAddress("glWaitVkSemaphoreNV"));
02594     glWeightPathsNV = PFNGLWEIGHTPATHSNVPROC (glfwGetProcAddress("glWeightPathsNV"));
02595     glWindowRectanglesEXT = PFNGLWINDOWRECTANGLESEXTPROC (glfwGetProcAddress("glWindowRectanglesEXT"));
02596 #endif
02597
02598 // 使用している GPU のバッファアライメントを調べる
02599 glGetIntegerv(GL_UNIFORM_BUFFER_OFFSET_ALIGNMENT, &ggBufferAlignment);
02600 }
02601
02602 //
02603 // OpenGL のエラーをチェックする
02604 //
02605 //   OpenGL の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
02606 //
02607 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02608 //
02609 void gg::ggError(const std::string& name, unsigned int line)
02610 {
02611     const GLenum error{ glGetError() };
02612
02613     if (error != GL_NO_ERROR)
02614     {
02615         if (!name.empty())
02616         {
02617             std::cerr << name;
02618             if (line > 0) std::cerr << " (" << line << ")";
02619             std::cerr << ": ";
02620         }
02621
02622         switch (error)
02623         {
02624             case GL_INVALID_ENUM:
02625                 std::cerr << "An unacceptable value is specified for an enumerated argument" << std::endl;
02626                 break;
02627             case GL_INVALID_VALUE:
02628                 std::cerr << "A numeric argument is out of range" << std::endl;
02629                 break;
02630             case GL_INVALID_OPERATION:
02631                 std::cerr << "The specified operation is not allowed in the current state" << std::endl;
02632                 break;
02633             case GL_OUT_OF_MEMORY:
02634                 std::cerr << "There is not enough memory left to execute the command" << std::endl;
02635                 break;
02636             case GL_INVALID_FRAMEBUFFER_OPERATION:
02637                 std::cerr << "The specified operation is not allowed current frame buffer" << std::endl;
02638                 break;
02639             default:
02640                 std::cerr << "An OpenGL error has occurred: " << std::hex << std::showbase << error << std::endl;
02641                 break;
02642         }
02643     }
02644 }
02645
02646 //
02647 // FBO のエラーをチェックする
02648 //
02649 //   FBO の API を呼び出し直後に実行すればエラーのあるときにメッセージを表示する
02650 //
02651 //   msg エラー発生時に標準エラー出力に出力する文字列. nullptr なら何も出力しない
02652 //
02653 void gg::ggFBOError(const std::string& name, unsigned int line)
02654 {
02655     const auto status{ glCheckFramebufferStatus(GL_FRAMEBUFFER) };
02656
02657     if (status != GL_FRAMEBUFFER_COMPLETE)

```



```

02658 {
02659     if (!name.empty())
02660     {
02661         std::cerr << name;
02662         if (line > 0) std::cerr << " (" << line << ")";
02663         std::cerr << ": ";
02664     }
02665
02666     switch (status)
02667     {
02668     case GL_FRAMEBUFFER_UNDEFINED:
02669         std::cerr << "the default framebuffer does not exist" << std::endl;
02670         break;
02671     case GL_FRAMEBUFFER_INCOMPLETE_ATTACHMENT:
02672         std::cerr << "Framebuffer incomplete, duplicate attachment" << std::endl;
02673         break;
02674     case GL_FRAMEBUFFER_INCOMPLETE_MISSING_ATTACHMENT:
02675         std::cerr << "Framebuffer incomplete, missing attachment" << std::endl;
02676         break;
02677     case GL_FRAMEBUFFER_UNSUPPORTED:
02678         std::cerr << "Unsupported framebuffer internal" << std::endl;
02679         break;
02680     case GL_FRAMEBUFFER_INCOMPLETE_MULTISAMPLE:
02681         std::cerr << "The value of GL_RENDERBUFFER_SAMPLES is not"
02682             " the same for all attached renderbuffers or,"
02683             " if the attached images are a mix of renderbuffers and textures,"
02684             " the value of GL_RENDERBUFFER_SAMPLES is not zero" << std::endl;
02685         break;
02686     #if !defined(GL_GLES_PROTOTYPES)
02687     case GL_FRAMEBUFFER_INCOMPLETE_LAYER_TARGETS:
02688         std::cerr << "Any framebuffer attachment is layered,"
02689             " and any populated attachment is not layered,"
02690             " or if all populated color attachments are not from textures"
02691             " of the same target" << std::endl;
02692         break;
02693     case GL_FRAMEBUFFER_INCOMPLETE_DRAW_BUFFER:
02694         std::cerr << "Framebuffer incomplete, missing draw buffer" << std::endl;
02695         break;
02696     case GL_FRAMEBUFFER_INCOMPLETE_READ_BUFFER:
02697         std::cerr << "Framebuffer incomplete, missing read buffer" << std::endl;
02698         break;
02699     #endif
02700     default:
02701         std::cerr << "Programming error; will fail on all hardware: " << std::hex << std::showbase <<
02702             status << std::endl;
02703         break;
02704     }
02705 }
02706
02707 //
02708 // 変換行列：行列とベクトルの積  $c \leftarrow a \times b$ 
02709 //
02710 void gg::GgMatrix::projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02711 {
02712     for (int i = 0; i < 4; ++i)
02713     {
02714         c[i] = a[0 + i] * b[0] + a[4 + i] * b[1] + a[8 + i] * b[2] + a[12 + i] * b[3];
02715     }
02716 }
02717
02718 //
02719 // 変換行列：行列と行列の積  $c \leftarrow a \times b$ 
02720 //
02721 void gg::GgMatrix::multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const
02722 {
02723     for (int i = 0; i < 16; ++i)
02724     {
02725         int j = i & 3, k = i & ~3;
02726
02727         c[i] = a[0 + j] * b[k + 0] + a[4 + j] * b[k + 1] + a[8 + j] * b[k + 2] + a[12 + j] * b[k + 3];
02728     }
02729 }
02730
02731 //
02732 // 変換行列：単位行列を設定する
02733 //
02734 gg::GgMatrix& gg::GgMatrix::loadIdentity()
02735 {
02736     *this =
02737     {
02738         1.0f, 0.0f, 0.0f, 0.0f,
02739         0.0f, 1.0f, 0.0f, 0.0f,
02740         0.0f, 0.0f, 1.0f, 0.0f,
02741         0.0f, 0.0f, 0.0f, 1.0f
02742     };
02743 }

```

```

02744     return *this;
02745 }
02746
02747 //
02748 // 変換行列：平行移動変換行列を設定する
02749 //
02750 gg::GgMatrix& gg::GgMatrix::loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02751 {
02752     *this =
02753     {
02754         w,      0.0f, 0.0f, 0.0f,
02755         0.0f, w,      0.0f, 0.0f,
02756         0.0f, 0.0f, w,      0.0f,
02757         x,      y,      z,      w
02758     };
02759     return *this;
02760 }
02761
02762 //
02763 // 変換行列：拡大縮小変換行列を設定する
02764 //
02765 //
02766 gg::GgMatrix& gg::GgMatrix::loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
02767 {
02768     *this =
02769     {
02770         x,      0.0f, 0.0f, 0.0f,
02771         0.0f, y,      0.0f, 0.0f,
02772         0.0f, 0.0f, z,      0.0f,
02773         0.0f, 0.0f, 0.0f, w
02774     };
02775     return *this;
02776 }
02777
02778 //
02779 // 変換行列：x 軸中心の回転変換行列を設定する
02780 //
02781 //
02782 gg::GgMatrix& gg::GgMatrix::loadRotateX(GLfloat a)
02783 {
02784     const auto c{ cosf(a) };
02785     const auto s{ sinf(a) };
02786
02787     *this =
02788     {
02789         1.0f, 0.0f, 0.0f, 0.0f,
02790         0.0f, c,      s,      0.0f,
02791         0.0f, -s,     c,      0.0f,
02792         0.0f, 0.0f, 0.0f, 1.0f
02793     };
02794     return *this;
02795 }
02796
02797 //
02798 // 変換行列：y 軸中心の回転変換行列を設定する
02799 //
02800 //
02801 gg::GgMatrix& gg::GgMatrix::loadRotateY(GLfloat a)
02802 {
02803     const auto c{ cosf(a) };
02804     const auto s{ sinf(a) };
02805
02806     *this =
02807     {
02808         c,      0.0f, -s,      0.0f,
02809         0.0f, 1.0f, 0.0f, 0.0f,
02810         s,      0.0f, c,      0.0f,
02811         0.0f, 0.0f, 0.0f, 1.0f
02812     };
02813     return *this;
02814 }
02815
02816 //
02817 // 変換行列：z 軸中心の回転変換行列を設定する
02818 //
02819 //
02820 gg::GgMatrix& gg::GgMatrix::loadRotateZ(GLfloat a)
02821 {
02822     const auto c{ cosf(a) };
02823     const auto s{ sinf(a) };
02824
02825     *this =
02826     {
02827         c,      s,      0.0f, 0.0f,
02828         -s,     c,      0.0f, 0.0f,
02829         0.0f, 0.0f, 1.0f, 0.0f,
02830         0.0f, 0.0f, 0.0f, 1.0f

```

```

02831     };
02832
02833     return *this;
02834 }
02835
02836 //
02837 // 変換行列：任意軸中心の回転変換行列を設定する
02838 //
02839 gg::GgMatrix& gg::GgMatrix::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
02840 {
02841     const auto d{ sqrtf(x * x + y * y + z * z) };
02842
02843     if (d > 0.0f)
02844     {
02845         const auto l{ x / d }, m{ y / d }, n{ z / d };
02846         const auto l2{ l * l }, m2{ m * m }, n2{ n * n };
02847         const auto lm{ l * m }, mn{ m * n }, nl{ n * l };
02848         const auto c{ cosf(a) }, cl{ 1.0f - c };
02849         const auto s{ sinf(a) };
02850
02851         *this =
02852         {
02853             (1.0f - l2) * c + l2,
02854             lm * cl + n * s,
02855             nl * cl - m * s,
02856             0.0f,
02857
02858             lm * cl - n * s,
02859             (1.0f - m2) * c + m2,
02860             mn * cl + l * s,
02861             0.0f,
02862
02863             nl * cl + m * s,
02864             mn * cl - l * s,
02865             (1.0f - n2) * c + n2,
02866             0.0f,
02867
02868             0.0f,
02869             0.0f,
02870             0.0f,
02871             1.0f,
02872         };
02873     }
02874
02875     return *this;
02876 }
02877
02878 //
02879 // 変換行列：転置行列を設定する
02880 //
02881 gg::GgMatrix& gg::GgMatrix::loadTranspose(const GLfloat* marray)
02882 {
02883     *this =
02884     {
02885         marray[ 0],
02886         marray[ 4],
02887         marray[ 8],
02888         marray[12],
02889         marray[ 1],
02890         marray[ 5],
02891         marray[ 9],
02892         marray[13],
02893         marray[ 2],
02894         marray[ 6],
02895         marray[10],
02896         marray[14],
02897         marray[ 3],
02898         marray[ 7],
02899         marray[11],
02900         marray[15]
02901     };
02902
02903     return *this;
02904 }
02905
02906 //
02907 // 変換行列：逆行列を設定する
02908 //
02909 gg::GgMatrix& gg::GgMatrix::loadInvert(const GLfloat* marray)
02910 {
02911     GLfloat lu[20];
02912     GLfloat* plu[4];
02913
02914     // j 行の要素の値の絶対値の最大値を plu[j][4] に求める
02915     for (int j = 0; j < 4; ++j)
02916     {
02917         auto max{ fabs(*(plu[j] = lu + 5 * j) = *(marray++)) };

```

```

02918
02919     for (int i = 0; ++i < 4;)
02920     {
02921         auto a{ fabs(plu[j][i] = *(marray++)) };
02922         if (a > max) max = a;
02923     }
02924     if (max == 0.0f) return *this;
02925     plu[j][4] = 1.0f / max;
02926 }
02927
02928 // ピボットを考慮した LU 分解
02929 for (int j = 0; j < 4; ++j)
02930 {
02931     auto max{ fabs(plu[j][j] * plu[j][4]) };
02932     int i = j;
02933
02934     for (int k = j; ++k < 4;)
02935     {
02936         auto a{ fabs(plu[k][j] * plu[k][4]) };
02937         if (a > max)
02938         {
02939             max = a;
02940             i = k;
02941         }
02942     }
02943     if (i > j)
02944     {
02945         auto t{ plu[j] };
02946         plu[j] = plu[i];
02947         plu[i] = t;
02948     }
02949     if (plu[j][j] == 0.0f) return *this;
02950     for (int k = j; ++k < 4;)
02951     {
02952         plu[k][j] /= plu[j][j];
02953         for (int i = j; ++i < 4;)
02954         {
02955             plu[k][i] -= plu[j][i] * plu[k][j];
02956         }
02957     }
02958 }
02959
02960 // LU 分解から逆行列を求める
02961 for (int k = 0; k < 4; ++k)
02962 {
02963     // array に単位行列を設定する
02964     for (int i = 0; i < 4; ++i)
02965     {
02966         (*this)[i * 4 + k] = (plu[i] == lu + k * 5) ? 1.0f : 0.0f;
02967     }
02968     // lu から逆行列を求める
02969     for (int i = 0; i < 4; ++i)
02970     {
02971         for (int j = i; ++j < 4;)
02972         {
02973             (*this)[j * 4 + k] -= (*this)[i * 4 + k] * plu[j][i];
02974         }
02975     }
02976     for (int i = 4; --i >= 0;)
02977     {
02978         for (int j = i; ++j < 4;)
02979         {
02980             (*this)[i * 4 + k] -= plu[i][j] * (*this)[j * 4 + k];
02981         }
02982         (*this)[i * 4 + k] /= plu[i][i];
02983     }
02984 }
02985
02986 return *this;
02987 }
02988
02989 //
02990 // 変換行列：法線変換行列を設定する
02991 //
02992 gg::GgMatrix& gg::GgMatrix::loadNormal(const GLfloat* marray)
02993 {
02994     *this =
02995     {
02996         marray[ 5] * marray[10] - marray[ 6] * marray[ 9],
02997         marray[ 6] * marray[ 8] - marray[ 4] * marray[10],
02998         marray[ 4] * marray[ 9] - marray[ 5] * marray[ 8],
02999         0.0f,
03000
03001         marray[ 9] * marray[ 2] - marray[10] * marray[ 1],
03002         marray[10] * marray[ 0] - marray[ 8] * marray[ 2],
03003         marray[ 8] * marray[ 1] - marray[ 9] * marray[ 0],
03004         0.0f,

```

```

03005
03006     marray[ 1] * marray[ 6] - marray[ 2] * marray[ 5],
03007     marray[ 2] * marray[ 4] - marray[ 0] * marray[ 6],
03008     marray[ 0] * marray[ 5] - marray[ 1] * marray[ 4],
03009     0.0f,
03010
03011     0.0f,
03012     0.0f,
03013     0.0f,
03014     1.0f
03015 };
03016
03017 return *this;
03018 }
03019
03020 //
03021 // 変換行列：ビュー変換行列を設定する
03022 //
03023 gg::GgMatrix& gg::GgMatrix::loadLookat(
03024     GLfloat ex, GLfloat ey, GLfloat ez,
03025     GLfloat tx, GLfloat ty, GLfloat tz,
03026     GLfloat ux, GLfloat uy, GLfloat uz
03027 )
03028 {
03029     // z 軸 = e - t
03030     const auto zx{ ex - tx };
03031     const auto zy{ ey - ty };
03032     const auto zz{ ez - tz };
03033
03034     // z 軸の長さ
03035     const auto z{ sqrtf(zx * zx + zy * zy + zz * zz) };
03036
03037     // x 軸 = u × z 軸
03038     const auto xx{ uy * zz - uz * zy };
03039     const auto xy{ uz * zx - ux * zz };
03040     const auto xz{ ux * zy - uy * zx };
03041
03042     // x 軸の長さ
03043     const auto x{ sqrtf(xx * xx + xy * xy + xz * xz) };
03044
03045     // y 軸 = z 軸 × x 軸
03046     const auto yx{ zy * xz - zz * xy };
03047     const auto yy{ zz * xx - zx * xz };
03048     const auto yz{ zx * xy - zy * xx };
03049
03050     // y 軸の長さ
03051     const auto y{ sqrtf(yx * yx + yy * yy + yz * yz) };
03052
03053     // y 軸の長さをチェック
03054     if (fabs(y) < std::numeric_limits<float>::epsilon()) return *this;
03055
03056     (*this)[ 0] = xx / x;
03057     (*this)[ 1] = yx / y;
03058     (*this)[ 2] = zx / z;
03059     (*this)[ 3] = 0.0f;
03060
03061     (*this)[ 4] = xy / x;
03062     (*this)[ 5] = yy / y;
03063     (*this)[ 6] = zy / z;
03064     (*this)[ 7] = 0.0f;
03065
03066     (*this)[ 8] = xz / x;
03067     (*this)[ 9] = yz / y;
03068     (*this)[10] = zz / z;
03069     (*this)[11] = 0.0f;
03070
03071     (*this)[12] = -(ex * (*this)[ 0] + ey * (*this)[ 4] + ez * (*this)[ 8]);
03072     (*this)[13] = -(ex * (*this)[ 1] + ey * (*this)[ 5] + ez * (*this)[ 9]);
03073     (*this)[14] = -(ex * (*this)[ 2] + ey * (*this)[ 6] + ez * (*this)[10]);
03074     (*this)[15] = 1.0f;
03075
03076     return *this;
03077 }
03078
03079 //
03080 // 変換行列：平行投影変換行列を設定する
03081 //
03082 gg::GgMatrix& gg::GgMatrix::loadOrthogonal(
03083     GLfloat left, GLfloat right,
03084     GLfloat bottom, GLfloat top,
03085     GLfloat zNear, GLfloat zFar
03086 )
03087 {
03088     const auto dx{ right - left };
03089     const auto dy{ top - bottom };
03090     const auto dz{ zFar - zNear };
03091

```

```

03092     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03093
03094     *this =
03095     {
03096         2.0f / dx,
03097         0.0f,
03098         0.0f,
03099         0.0f,
03100
03101         0.0f,
03102         2.0f / dy,
03103         0.0f,
03104         0.0f,
03105
03106         0.0f,
03107         0.0f,
03108         -2.0f / dz,
03109         0.0f,
03110
03111         -(right + left) / dx,
03112         -(top + bottom) / dy,
03113         -(zFar + zNear) / dz,
03114         1.0f
03115     };
03116
03117     return *this;
03118 }
03119
03120 //
03121 // 変換行列：透視投影変換行列を設定する
03122 //
03123 gg::GgMatrix& gg::GgMatrix::loadFrustum(
03124     GLfloat left, GLfloat right,
03125     GLfloat bottom, GLfloat top,
03126     GLfloat zNear, GLfloat zFar
03127 )
03128 {
03129     const auto dx{ right - left };
03130     const auto dy{ top - bottom };
03131     const auto dz{ zFar - zNear };
03132
03133     if (dx == 0.0f || dy == 0.0f || dz == 0.0f) return *this;
03134
03135     *this =
03136     {
03137         2.0f * zNear / dx,
03138         0.0f,
03139         0.0f,
03140         0.0f,
03141
03142         0.0f,
03143         2.0f * zNear / dy,
03144         0.0f,
03145         0.0f,
03146
03147         (right + left) / dx,
03148         (top + bottom) / dy,
03149         -(zFar + zNear) / dz,
03150         -1.0f,
03151
03152         0.0f,
03153         0.0f,
03154         -2.0f * zFar * zNear / dz,
03155         0.0f
03156     };
03157
03158     return *this;
03159 }
03160
03161 //
03162 // 変換行列：画角から透視投影変換行列を設定する
03163 //
03164 gg::GgMatrix& gg::GgMatrix::loadPerspective(
03165     GLfloat fovy, GLfloat aspect,
03166     GLfloat zNear, GLfloat zFar
03167 )
03168 {
03169     const auto dz{ zFar - zNear };
03170
03171     if (dz == 0.0f) return *this;
03172
03173     const auto f{ 1.0f / tanf(fovy * 0.5f) };
03174
03175     *this =
03176     {
03177         f / aspect,
03178         0.0f,

```

```

03179     0.0f,
03180     0.0f,
03181
03182     0.0f,
03183     f,
03184     0.0f,
03185     0.0f,
03186
03187     0.0f,
03188     0.0f,
03189     -(zFar + zNear) / dz,
03190     -1.0f,
03191
03192     0.0f,
03193     0.0f,
03194     -2.0f * zFar * zNear / dz,
03195     0.0f
03196 };
03197
03198 return *this;
03199 }
03200
03201 //
03202 // 四元数: GgQuaternion 型の四元数 p, q の積を r に求める
03203 //
03204 void gg::GgQuaternion::multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const
03205 {
03206     r[0] = p[1] * q[2] - p[2] * q[1] + p[0] * q[3] + p[3] * q[0];
03207     r[1] = p[2] * q[0] - p[0] * q[2] + p[1] * q[3] + p[3] * q[1];
03208     r[2] = p[0] * q[1] - p[1] * q[0] + p[2] * q[3] + p[3] * q[2];
03209     r[3] = p[3] * q[3] - p[0] * q[0] - p[1] * q[1] - p[2] * q[2];
03210 }
03211
03212 //
03213 // 四元数: GgQuaternion 型の四元数 q が表す変換行列を m に求める
03214 //
03215 void gg::GgQuaternion::toMatrix(GLfloat* m, const GLfloat* q) const
03216 {
03217     const auto xx{ q[0] * q[0] * 2.0f };
03218     const auto yy{ q[1] * q[1] * 2.0f };
03219     const auto zz{ q[2] * q[2] * 2.0f };
03220     const auto xy{ q[0] * q[1] * 2.0f };
03221     const auto yz{ q[1] * q[2] * 2.0f };
03222     const auto zx{ q[2] * q[0] * 2.0f };
03223     const auto xw{ q[0] * q[3] * 2.0f };
03224     const auto yw{ q[1] * q[3] * 2.0f };
03225     const auto zw{ q[2] * q[3] * 2.0f };
03226
03227     m[ 0] = 1.0f - yy - zz;
03228     m[ 1] = xy + zw;
03229     m[ 2] = zx - yw;
03230     m[ 4] = xy - zw;
03231     m[ 5] = 1.0f - zz - xx;
03232     m[ 6] = yz + xw;
03233     m[ 8] = zx + yw;
03234     m[ 9] = yz - xw;
03235     m[10] = 1.0f - xx - yy;
03236     m[ 3] = m[ 7] = m[11] = m[12] = m[13] = m[14] = 0.0f;
03237     m[15] = 1.0f;
03238 }
03239
03240 //
03241 // 四元数: 回転変換行列 a が表す四元数を q に求める
03242 //
03243 void gg::GgQuaternion::toQuaternion(GLfloat* q, const GLfloat* a) const
03244 {
03245     const auto tr{ a[0] + a[5] + a[10] + a[15] };
03246
03247     if (tr > 0.0f)
03248     {
03249         q[3] = sqrtf(tr) * 0.5f;
03250         q[0] = (a[6] - a[9]) * 0.25f / q[3];
03251         q[1] = (a[8] - a[2]) * 0.25f / q[3];
03252         q[2] = (a[1] - a[4]) * 0.25f / q[3];
03253     }
03254 }
03255
03256 //
03257 // 四元数: 球面線形補間 p に q と r を t で補間した四元数を求める
03258 //
03259 void gg::GgQuaternion::slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const
03260 {
03261     auto qr{ ggDot4(q, r) };
03262     GLfloat s[4]{ r[0], r[1], r[2], r[3] };
03263
03264     // 内積が負なら最短経路を通るように符号を反転する
03265     if (qr < 0.0f)

```

```

03266 {
03267     qr = -qr;
03268     s[0] = -s[0];
03269     s[1] = -s[1];
03270     s[2] = -s[2];
03271     s[3] = -s[3];
03272 }
03273
03274 // 浮動小数点誤差で 1.0f を超える場合をクランプ
03275 if (qr > 1.0f) qr = 1.0f;
03276
03277 const auto ss{ 1.0f - qr * qr };
03278
03279 if (ss <= 0.0f)
03280 {
03281     if (p != q)
03282     {
03283         p[0] = q[0];
03284         p[1] = q[1];
03285         p[2] = q[2];
03286         p[3] = q[3];
03287     }
03288 }
03289 else
03290 {
03291     const auto sp{ sqrtf(ss) };
03292     const auto ph{ acosf(qr) };
03293     const auto pt{ ph * t };
03294     const auto t1{ sinf(pt) / sp };
03295     const auto t0{ sinf(ph - pt) / sp };
03296
03297     p[0] = q[0] * t0 + s[0] * t1;
03298     p[1] = q[1] * t0 + s[1] * t1;
03299     p[2] = q[2] * t0 + s[2] * t1;
03300     p[3] = q[3] * t0 + s[3] * t1;
03301 }
03302 }
03303
03304 //
03305 // 四元数: (x, y, z) を軸とし角度 a 回転する四元数を求める
03306 //
03307 gg::GgQuaternion& gg::GgQuaternion::loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03308 {
03309     const auto l{ x * x + y * y + z * z };
03310     const auto w{ cosf(a * 0.5f) };
03311
03312     if (fabs(l) > std::numeric_limits<float>::epsilon())
03313     {
03314         const auto s{ sinf(a) / sqrtf(l) };
03315
03316         *this = GgQuaternion{ x * s, y * s, z * s, w };
03317     }
03318     else
03319     {
03320         *this = GgQuaternion{ 0.0f, 0.0f, 0.0f, w };
03321     }
03322
03323     return *this;
03324 }
03325
03326 //
03327 // x 軸中心に角度 a 回転する四元数を格納する
03328 //
03329 gg::GgQuaternion& gg::GgQuaternion::loadRotateX(GLfloat a)
03330 {
03331     const auto t{ a * 0.5f };
03332
03333     *this = GgQuaternion{ sinf(t), 0.0f, 0.0f, cosf(t) };
03334
03335     return *this;
03336 }
03337
03338 //
03339 // y 軸中心に角度 a 回転する四元数を格納する
03340 //
03341 gg::GgQuaternion& gg::GgQuaternion::loadRotateY(GLfloat a)
03342 {
03343     const auto t{ a * 0.5f };
03344
03345     *this = GgQuaternion{ 0.0f, sinf(t), 0.0f, cosf(t) };
03346
03347     return *this;
03348 }
03349
03350 //
03351 // z 軸中心に角度 a 回転する四元数を格納する
03352 //

```



```

03353 gg::GgQuaternion& gg::GgQuaternion::loadRotateZ(GLfloat a)
03354 {
03355     const auto t{ a * 0.5f };
03356
03357     *this = GgQuaternion{ 0.0f, 0.0f, sinf(t), cosf(t) };
03358
03359     return *this;
03360 }
03361
03362 //
03363 // 四元数：オイラー角 (heading, pitch, roll) にもとづいて四元数を求める
03364 //
03365 gg::GgQuaternion& gg::GgQuaternion::loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll)
03366 {
03367     GgQuaternion h, p, r;
03368
03369     h.loadRotateY(heading);
03370     p.loadRotateX(pitch);
03371     r.loadRotateZ(roll);
03372
03373     *this = h * p * r;
03374
03375     return *this;
03376 }
03377
03378 //
03379 // 四元数：正規化して格納する
03380 //
03381 gg::GgQuaternion& gg::GgQuaternion::loadNormalize(const GLfloat* a)
03382 {
03383     *this = a;
03384
03385     ggNormalize4(data());
03386
03387     return *this;
03388 }
03389
03390 //
03391 // 四元数：共役四元数を格納する
03392 //
03393 gg::GgQuaternion& gg::GgQuaternion::loadConjugate(const GLfloat* a)
03394 {
03395     // w 要素を反転する
03396     data()[0] = a[0];
03397     data()[1] = a[1];
03398     data()[2] = a[2];
03399     data()[3] = -a[3];
03400
03401     return *this;
03402 }
03403
03404 //
03405 // 四元数：逆元を格納する
03406 //
03407 gg::GgQuaternion& gg::GgQuaternion::loadInvert(const GLfloat* a)
03408 {
03409     // ノルムの二乗を求める
03410     const auto l(ggDot4(a, a));
03411
03412     if (l > 0.0f)
03413     {
03414         // 共役四元数を求める
03415         GgQuaternion r;
03416         r.loadConjugate(a);
03417
03418         // ノルムの二乗で割る
03419         data()[0] = r[0] / l;
03420         data()[1] = r[1] / l;
03421         data()[2] = r[2] / l;
03422         data()[3] = r[3] / l;
03423     }
03424
03425     return *this;
03426 }
03427
03428 //
03429 // 簡易トラックボール処理：リセット
03430 //
03431 void gg::GgTrackball::reset(const GgQuaternion& q)
03432 {
03433     // ドラッグ中ではない
03434     drag = false;
03435
03436     // 単位クォーターニオンに初期値を与える
03437     *this = cq = q;
03438
03439     // 回転行列を初期化する

```

```

03440     GgQuaternion::getMatrix(rt);
03441 }
03442 //
03443 //
03444 // 簡易トラックボール処理：トラックボールする領域の設定
03445 //
03446 // Reshape コールバック (resize) の中で実行する
03447 // (w, h) ウィンドウサイズ
03448 //
03449 void gg::GgTrackball::region(GLfloat w, GLfloat h)
03450 {
03451     // マウスポインタ位置のウィンドウ内の相対的位置への換算用
03452     scale[0] = 2.0f / w;
03453     scale[1] = 2.0f / h;
03454 }
03455 //
03456 // 簡易トラックボール処理：ドラッグ開始時の処理
03457 //
03458 // マウスボタンを押したときに実行する
03459 // (x, y) 現在のマウス位置
03460 //
03461 //
03462 void gg::GgTrackball::begin(GLfloat x, GLfloat y)
03463 {
03464     // ドラッグ開始
03465     drag = true;
03466 //
03467 // ドラッグ開始点を記録する
03468     start[0] = x;
03469     start[1] = y;
03470 }
03471 //
03472 // 簡易トラックボール処理：ドラッグ中の処理
03473 //
03474 // マウスのドラッグ中に実行する
03475 // (x, y) 現在のマウス位置
03476 //
03477 //
03478 void gg::GgTrackball::motion(GLfloat x, GLfloat y)
03479 {
03480     // ドラッグ中でなければ何もしない
03481     if (!drag) return;
03482 //
03483 // マウスポインタの位置のドラッグ開始位置からの変位
03484     const auto dx{ (x - start[0]) * scale[0] };
03485     const auto dy{ (y - start[1]) * scale[1] };
03486 //
03487 // マウスポインタの位置のドラッグ開始位置からの距離
03488     const auto a{ hypot(dx, dy) };
03489 //
03490 // マウスポインタの位置がドラッグ開始位置から移動していれば
03491     if (a > 0.0)
03492     {
03493         // 現在の回転の四元数に作った四元数を掛けて合成する
03494         *this = ggRotateQuaternion(dy, dx, 0.0f, a * 3.1415926536f) * cq;
03495 //
03496 // 合成した四元数から回転の変換行列を求める
03497         GgQuaternion::getMatrix(rt);
03498     }
03499 }
03500 //
03501 // 簡易トラックボール処理：回転角の修正
03502 //
03503 //
03504 // 現在の回転角を修正する
03505 // q 修正分の回転角を表す四元数
03506 //
03507 void gg::GgTrackball::rotate(const GgQuaternion& q)
03508 {
03509     // ドラッグ中なら何もしない
03510     if (drag) return;
03511 //
03512 // 保存されている四元数に修正分の四元数を掛けて合成する
03513     *this = q * cq;
03514 //
03515 // 合成した四元数から回転の変換行列を求める
03516     GgQuaternion::getMatrix(rt);
03517 //
03518 // 誤差を吸収するために正規化して保存する
03519     cq = normalize();
03520 }
03521 //
03522 // 簡易トラックボール処理：停止時の処理
03523 //
03524 //
03525 // マウスボタンを離れたときに実行する
03526 // (x, y) 現在のマウス位置

```

```

03527 //
03528 void gg::GgTrackball::end(GLfloat x, GLfloat y)
03529 {
03530     // ドラック終了点における回転を求める
03531     motion(x, y);
03532
03533     // 誤差を吸収するために正規化して保存する
03534     cq = normalize();
03535
03536     // ドラック終了
03537     drag = false;
03538 }
03539
03540 //
03541 // 配列に格納された画像の内容を TGA ファイルに保存する
03542 //
03543 //     name ファイル名
03544 //     buffer 画像データ
03545 //     width 画像の横の画素数
03546 //     height 画像の縦の画素数
03547 //     depth 画像の 1 画素のバイト数
03548 //     戻り値 保存に成功すれば true, 失敗すれば false
03549 //
03550 bool gg::ggSaveTga(
03551     const std::string& name,
03552     const void* buffer,
03553     unsigned int width,
03554     unsigned int height,
03555     unsigned int depth
03556 )
03557 {
03558     // ファイルを開く
03559     std::ofstream file{ Utf8ToTChar(name), std::ios::binary };
03560
03561     // ファイルが開けなかったら戻る
03562     if (file.fail()) return false;
03563
03564     // 画像のヘッダ
03565     const unsigned char type{ static_cast<unsigned char>(depth == 0 ? 0 : depth < 3 ? 3 : 2) };
03566     const unsigned char alpha{ static_cast<unsigned char>(depth == 2 || depth == 4 ? 8 : 0) };
03567     const unsigned char header[18]
03568     {
03569         0,          // ID length
03570         0,          // Color map type (none)
03571         type,        // Image Type (2:RGB, 3:Grayscale)
03572         0, 0,        // Offset into the color map table
03573         0, 0,        // Number of color map entries
03574         0,          // Number of a color map entry bits per pixel
03575         0, 0,        // Horizontal image position
03576         0, 0,        // Vertical image position
03577         static_cast<unsigned char>(width & 0xff),
03578         static_cast<unsigned char>(width >> 8),
03579         static_cast<unsigned char>(height & 0xff),
03580         static_cast<unsigned char>(height >> 8),
03581         static_cast<unsigned char>(depth * 8),
03582         alpha        // Image descriptor
03583     };
03584
03585     // ヘッダを書き込む
03586     file.write(reinterpret_cast<const char*>(header), sizeof header);
03587
03588     // ヘッダの書き込みに失敗したら戻る
03589     if (file.bad()) return false;
03590
03591     // データを書き込む
03592     const unsigned int size{ width * height * depth };
03593     if (type == 2)
03594     {
03595         // フルカラー
03596         std::vector<char> temp(size);
03597         for (GLuint i = 0; i < size; i += depth)
03598         {
03599             temp[i + 2] = static_cast<const char*>(buffer)[i + 0];
03600             temp[i + 1] = static_cast<const char*>(buffer)[i + 1];
03601             temp[i + 0] = static_cast<const char*>(buffer)[i + 2];
03602             if (depth == 4) temp[i + 3] = static_cast<const char*>(buffer)[i + 3];
03603         }
03604         file.write(temp.data(), size);
03605     }
03606     else if (type == 3)
03607     {
03608         // グレースケール
03609         file.write(static_cast<const char*>(buffer), size);
03610     }
03611
03612     // フッタを書き込む
03613     constexpr char footer[]{ "\0\0\0\0\0\0\0TRUEVISION-XFILE." };

```

```

03614     file.write(footer, sizeof footer);
03615
03616     // データの書き込みに失敗していなければ true を返す
03617     return !file.bad();
03618 }
03619
03620 //
03621 // カラーバッファの内容を TGA ファイルに保存する
03622 //
03623 //     name 保存するファイル名
03624 //     戻り値 保存に成功すれば true, 失敗すれば false
03625 //
03626 bool gg::ggSaveColor(const std::string& name)
03627 {
03628     // 現在のビューポートのサイズを得る
03629     GLint viewport[4];
03630     glGetIntegerv(GL_VIEWPORT, viewport);
03631
03632     // ビューポートのサイズ分のメモリを確保する
03633     std::vector<GLubyte> buffer(viewport[2] * viewport[3] * 3);
03634
03635     // 画面表示の完了を待つ
03636     glFinish();
03637
03638     // カラーバッファを読み込む
03639     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03640                 GL_RGB, GL_UNSIGNED_BYTE, buffer.data());
03641
03642     // 読み込んだデータをファイルに書き込む
03643     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 3);
03644 }
03645
03646 //
03647 // デプスバッファの内容を TGA ファイルに保存する
03648 //
03649 //     name 保存するファイル名
03650 //     戻り値 保存に成功すれば true, 失敗すれば false
03651 //
03652 bool gg::ggSaveDepth(const std::string& name)
03653 {
03654     // 現在のビューポートのサイズを得る
03655     GLint viewport[4];
03656     glGetIntegerv(GL_VIEWPORT, viewport);
03657
03658     // ビューポートのサイズ分のメモリを確保する
03659     std::vector<GLubyte> buffer(viewport[2] * viewport[3]);
03660
03661     // 画面表示の完了を待つ
03662     glFinish();
03663
03664     // デプスバッファを読み込む
03665     glReadPixels(viewport[0], viewport[1], viewport[2], viewport[3],
03666                 GL_DEPTH_COMPONENT, GL_UNSIGNED_BYTE, buffer.data());
03667
03668     // 読み込んだデータをファイルに書き込む
03669     return ggSaveTga(name, buffer.data(), viewport[2], viewport[3], 1);
03670 }
03671
03672 //
03673 // TGA ファイル (8/16/24/32bit) を読み込む
03674 //
03675 //     name 読み込むファイル名
03676 //     pWidth 読み込んだファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03677 //     pHeight 読み込んだファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03678 //     pFormat 読み込んだファイルのフォーマットの格納先のポインタ (nullptr なら格納しない)
03679 //     image 読み込んだ画像を格納する vector
03680 //     戻り値 読み込みに成功すれば true, 失敗すれば false
03681 //
03682 bool gg::ggReadImage(
03683     const std::string& name,
03684     std::vector<GLubyte>& image,
03685     GLsizei* pWidth,
03686     GLsizei* pHeight,
03687     GLenum* pFormat
03688 )
03689 {
03690     // ファイルを開く
03691     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
03692
03693     // ファイルが開けなかったら戻る
03694     if (file.fail()) return false;
03695
03696     // ヘッダを読み込む
03697     unsigned char header[18];
03698     file.read(reinterpret_cast<char*>(header), sizeof header);
03699
03700     // ヘッダの読み込みに失敗したら戻る

```

```

03701     if (file.bad()) return false;
03702
03703     // 深度
03704     const auto depth{ header[16] / 8 };
03705     GLenum format;
03706     switch (depth)
03707     {
03708     case 1:
03709         format = GL_RED;
03710         break;
03711     case 2:
03712         format = GL_RG;
03713         break;
03714     case 3:
03715         format = GL_BGR;
03716         break;
03717     case 4:
03718         format = GL_BGRA;
03719         break;
03720     default:
03721         // 取り扱いえないフォーマットだったら戻る
03722         return false;
03723     }
03724
03725     // フォーマットを格納する
03726     if (pFormat) *pFormat = format;
03727
03728     // 画像の縦横の画素数
03729     const auto width{ static_cast<GLsizei>(header[13] << 8 | header[12]) };
03730     if (pWidth) *pWidth = width;
03731     const auto height{ static_cast<GLsizei>(header[15] << 8 | header[14]) };
03732     if (pHeight) *pHeight = height;
03733
03734     // データサイズ
03735     const auto size{ width * height * depth };
03736
03737     // サイズが小さすぎたら戻る
03738     if (size < 2) return false;
03739
03740     // 読み込みに使うメモリを確保する
03741     image.resize(size);
03742
03743     // データを読み込む
03744     if (header[2] & 8)
03745     {
03746         // RLE
03747         int p{ 0 };
03748         char c;
03749         while (file.get(c))
03750         {
03751             if (c & 0x80)
03752             {
03753                 // run-length packet
03754                 const auto count{ (c & 0x7f) + 1 };
03755                 if (p + depth * count > size) break;
03756                 char temp[4];
03757                 file.read(temp, depth);
03758                 for (int i = 0; i < count; ++i)
03759                 {
03760                     for (int j = 0; j < depth; j++) image[p++] = temp[j++];
03761                 }
03762             }
03763             else
03764             {
03765                 // raw packet
03766                 const auto count{ (c + 1) * depth };
03767                 if (p + count > size) break;
03768                 file.read(reinterpret_cast<char*>(image.data() + p), count);
03769                 p += count;
03770             }
03771         }
03772     }
03773     else
03774     {
03775         // 非圧縮
03776         file.read(reinterpret_cast<char*>(image.data()), size);
03777     }
03778
03779     // 読み込みに失敗していなければ true を返す
03780     return !file.bad();
03781 }
03782
03783 //
03784 // テクスチャを作成して画像を読み込む
03785 //
03786 // image 画像データ, nullptr ならメモリの確保だけを行う
03787 // width 画像の横の画素数

```

```

03788 // height 画像の縦の画素数
03789 // format 画像データのフォーマット
03790 // type 画像のデータ型
03791 // internal テクスチャの内部フォーマット
03792 // wrap テクスチャのラッピングモード
03793 // swizzle true ならテクスチャの赤と青を入れ替える
03794 // 戻り値 テクスチャ名
03795 //
03796 GLuint gg::ggLoadTexture(
03797     const GLvoid* image,
03798     GLsizei width,
03799     GLsizei height,
03800     GLenum format,
03801     GLenum type,
03802     GLenum internal,
03803     GLenum wrap,
03804     bool swizzle
03805 )
03806 {
03807     // テクスチャを作成する
03808     GLuint texture;
03809     glGenTextures(1, &texture);
03810
03811     // 作成したテクスチャを 2D テクスチャとしてバインドする
03812     glBindTexture(GL_TEXTURE_2D, texture);
03813
03814     // アルファチャンネルがなければ 4 バイト境界に設定する
03815     glPixelStorei(GL_UNPACK_ALIGNMENT, format == GL_RGBA || format == GL_BGRA ? 4 : 1);
03816
03817     // テクスチャを割り当てる
03818     glTexImage2D(GL_TEXTURE_2D, 0, internal, width, height, 0, format, type, image);
03819
03820     // バイリニア (ミップマップなし), エッジでクランプ
03821     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
03822     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
03823     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, wrap);
03824     glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, wrap);
03825
03826     if (swizzle)
03827     {
03828         // テクスチャのサンプリング時に赤と青を入れ替える
03829         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, GL_BLUE);
03830         glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, GL_RED);
03831     }
03832
03833     // テクスチャ名を返す
03834     return texture;
03835 }
03836
03837 //
03838 // テクスチャを作成して TGA フォーマットの画像ファイルを読み込む
03839 //
03840 // name TGA ファイル名
03841 // pWidth 読みだした画像ファイルの横の画素数の格納先のポインタ (nullptr なら格納しない)
03842 // pHeight 読みだした画像ファイルの縦の画素数の格納先のポインタ (nullptr なら格納しない)
03843 // internal テクスチャの内部フォーマット
03844 // wrap テクスチャのラッピングモード
03845 // 戻り値 テクスチャ名
03846 //
03847 GLuint gg::ggLoadImage(
03848     const std::string& name,
03849     GLsizei* pWidth,
03850     GLsizei* pHeight,
03851     GLenum internal,
03852     GLenum wrap
03853 )
03854 {
03855     // 画像データ
03856     std::vector<GLubyte> image;
03857
03858     // 画像サイズ
03859     GLsizei width, height;
03860
03861     // 画像フォーマット
03862     GLenum format;
03863
03864     // 画像を読み込む
03865     ggReadImage(name, image, &width, &height, &format);
03866
03867     // 画像が読み込めなかったら戻る
03868     if (image.empty()) return 0;
03869
03870     // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
03871     if (internal == 0) internal = format;
03872
03873     // テクスチャに読み込む (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
03874     const auto tex{ ggLoadTexture(image.data(), width, height,

```

```

03875     format, GL_UNSIGNED_BYTE, internal, wrap, false) };
03876
03877     // 画像サイズを返す
03878     if (pWidth) *pWidth = width;
03879     if (pHeight) *pHeight = height;
03880
03881     // テクスチャ名を返す
03882     return tex;
03883 }
03884
03885 //
03886 // グレースケール画像 (8bit) から法線マップのデータを作成する
03887 //
03888 // width 高さマップのグレースケール画像 hmap の横の画素数
03889 // height 高さマップのグレースケール画像のデータ hmap の縦の画素数
03890 // stride データの間隔
03891 // hmap グレースケール画像のデータ
03892 // nz 法線の z 成分の割合
03893 // internal テクスチャの内部フォーマット
03894 // nmap 法線マップを格納する vector
03895 //
03896 void gg::ggCreateNormalMap(
03897     const GLubyte* hmap,
03898     GLsizei width,
03899     GLsizei height,
03900     GLenum format,
03901     GLfloat nz,
03902     GLenum internal,
03903     std::vector<GgVector>& nmap
03904 )
03905 {
03906     // メモリサイズ
03907     const GLsizei size{ width * height };
03908
03909     // 法線マップのメモリを確保する
03910     nmap.resize(size);
03911
03912     // 画素のバイト数
03913     GLint stride;
03914     switch (format)
03915     {
03916     case GL_RED:
03917         stride = 1;
03918         break;
03919     case GL_RG:
03920         stride = 2;
03921         break;
03922     case GL_RGB:
03923         stride = 3;
03924         break;
03925     case GL_RGBA:
03926         stride = 4;
03927         break;
03928     default:
03929         stride = 1;
03930         break;
03931     }
03932
03933     // 法線マップの作成
03934     for (GLsizei i = 0; i < size; ++i)
03935     {
03936         const auto x{ i % width };
03937         const auto y{ i - x };
03938         const auto u0{ (y + (x - 1 + width) % width) * stride };
03939         const auto u1{ (y + (x + 1) % width) * stride };
03940         const auto v0{ ((y - width + size) % size + x) * stride };
03941         const auto v1{ ((y + width) % size + x) * stride };
03942
03943         // 隣接する画素との値の差を法線の成分に用いる
03944         nmap[i][0] = static_cast<GLfloat>(hmap[u0] - hmap[u1]);
03945         nmap[i][1] = static_cast<GLfloat>(hmap[v0] - hmap[v1]);
03946         nmap[i][2] = nz;
03947         nmap[i][3] = hmap[i * stride];
03948
03949         // 法線ベクトルを正規化する
03950         ggNormalize3(&nmap[i]);
03951     }
03952
03953     // 内部フォーマットが浮動小数点テクスチャでなければ [0,1] に正規化する
03954     if (
03955         internal != GL_RGB16F &&
03956         internal != GL_RGBA16F &&
03957         internal != GL_RGB32F &&
03958         internal != GL_RGBA32F
03959     )
03960     {
03961         for (GLsizei i = 0; i < size; ++i)

```

```

03962     {
03963         nmap[i][0] = nmap[i][0] * 0.5f + 0.5f;
03964         nmap[i][1] = nmap[i][1] * 0.5f + 0.5f;
03965         nmap[i][2] = nmap[i][2] * 0.5f + 0.5f;
03966         nmap[i][3] *= 0.0039215686f; // == 1/255
03967     }
03968 }
03969 }
03970
03971 //
03972 // TGA 画像ファイルの高さマップ読み込んで法線マップのテクスチャを作成する
03973 //
03974 // name TGA ファイル名
03975 // nz 作成した法線の z 成分の割合
03976 // pWidth 読み込んだ画像の横の画素数の格納先のポインタ (nullptr なら格納しない)
03977 // pHeight 読み込んだ画像の縦の画素数の格納先のポインタ (nullptr なら格納しない)
03978 // internal テクスチャの内部フォーマット
03979 // 戻り値 テクスチャ名
03980 //
03981 GLuint gg::ggLoadHeight(
03982     const std::string& name,
03983     GLfloat nz,
03984     GLsizei* pWidth,
03985     GLsizei* pHeight,
03986     GLenum internal
03987 )
03988 {
03989     // 画像データ
03990     std::vector<GLubyte> hmap;
03991
03992     // 画像サイズ
03993     GLsizei width, height;
03994
03995     // 画像フォーマット
03996     GLenum format;
03997
03998     // 高さマップの画像を読み込む
03999     ggReadImage(name, hmap, &width, &height, &format);
04000
04001     // 画像が読み込めなかったら戻る
04002     if (hmap.empty()) return 0;
04003
04004     // 法線マップ
04005     std::vector<GgVector> nmap;
04006
04007     // 法線マップを作成する
04008     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04009
04010     // 画像サイズを返す
04011     if (pWidth) *pWidth = width;
04012     if (pHeight) *pHeight = height;
04013
04014     // テクスチャを作成して返す
04015     return ggLoadTexture(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal, GL_REPEAT);
04016 }
04017
04018 //
04019 // テクスチャの赤と青を交換する
04020 //
04021 void gg::GgTexture::swapRandB(bool swap) const
04022 {
04023     const auto swizzle
04024     {
04025         swap ?
04026             std::pair<GLint, GLint>{ GL_BLUE, GL_RED } :
04027             std::pair<GLint, GLint>{ GL_RED, GL_BLUE }
04028     };
04029
04030     glBindTexture(GL_TEXTURE_2D, texture);
04031     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_R, swizzle.first);
04032     glTexParameterf(GL_TEXTURE_2D, GL_TEXTURE_SWIZZLE_B, swizzle.second);
04033     glBindTexture(GL_TEXTURE_2D, 0);
04034 }
04035
04036 //
04037 // TGA フォーマットの画像ファイルを読み込んでカラーのテクスチャを作成する
04038 //
04039 // name 読み込むファイル名
04040 // internal glTexImage2D() に指定するテクスチャの内部フォーマット. 0 なら外部フォーマットに合わせる
04041 // 戻り値 テクスチャの作成に成功すれば true, 失敗すれば false
04042 //
04043 void gg::GgColorTexture::load(
04044     const std::string& name,
04045     GLenum internal,
04046     GLenum wrap
04047 )
04048 {

```



```

04049 // 画像データ
04050 std::vector<GLubyte> image;
04051
04052 // 画像サイズ
04053 GLsizei width, height;
04054
04055 // 画像フォーマット
04056 GLenum format;
04057
04058 // 画像を読み込む
04059 ggReadImage(name, image, &width, &height, &format);
04060
04061 // internal == 0 なら内部フォーマットを読み込んだファイルに合わせる
04062 if (internal == 0) internal = format;
04063
04064 // テクスチャを作成する (ggReadImage() で読み込んだ画像は GL_BGR / GL_BGRA)
04065 texture = std::make_shared<GgTexture>(image.data(), width, height,
04066     format, GL_UNSIGNED_BYTE, internal, wrap, false);
04067 }
04068
04069 //
04070 // TGA フォーマットの画像ファイルから高さマップ読み込んで法線マップのテクスチャを作成する
04071 //
04072 // name 画像ファイル名
04073 // width テクスチャとして用いる画像データの横幅
04074 // height テクスチャとして用いる画像データの高さ
04075 // format テクスチャとして用いる画像データのフォーマット (GL_RED, GL_RG, GL_RGB, GL_RGBA)
04076 // nz 法線マップの z 成分の値
04077 // internal テクスチャの内部フォーマット
04078 //
04079 void gg::GgNormalTexture::load(
04080     const std::string& name,
04081     GLfloat nz,
04082     GLenum internal
04083 )
04084 {
04085     // 高さマップ
04086     std::vector<GLubyte> hmap;
04087
04088     // 画像サイズ
04089     GLsizei width, height;
04090
04091     // 画像フォーマット
04092     GLenum format;
04093
04094     // 高さマップの画像を読み込む
04095     ggReadImage(name, hmap, &width, &height, &format);
04096
04097     // 法線マップ
04098     std::vector<GgVector> nmap;
04099
04100     // 法線マップを作成する
04101     ggCreateNormalMap(hmap.data(), width, height, format, nz, internal, nmap);
04102
04103     // テクスチャを作成する
04104     texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,
04105         GL_REPEAT);
04106 }
04107
04108 //
04109 // OBJ ファイルの読み込みに使うデータ型と関数
04110 //
04111 //
04112 namespace gg
04113 {
04114     // GLfloat 型の 2 要素のベクトル
04115     using vec2 = std::array<GLfloat, 2>;
04116
04117     // GLfloat 型の 3 要素のベクトル
04118     using vec3 = std::array<GLfloat, 3>;
04119
04120     // 三角形データ
04121     struct fidx
04122     {
04123         GLuint p[3]; // 頂点座標番号
04124         GLuint n[3]; // 頂点法線番号
04125         GLuint t[3]; // テクスチャ座標番号
04126         bool smooth; // スムーズシェーディングの有無
04127     };
04128
04129     // ポリゴングループ
04130     struct fgrp
04131     {
04132         GLuint nextgroup; // 次のポリゴングループの最初の三角形番号
04133         GLuint mtlno; // このポリゴングループの材質番号
04134     };
04135     // コンストラクタ

```

```

04136     fgrp(GLuint nextgroup, GLuint mtlno) :
04137         nextgroup(nextgroup),
04138         mtlno(mtlno)
04139     {
04140     }
04141 };
04142
04143 // デフォルトの材質
04144 constexpr GgSimpleShader::Material defaultMaterial
04145 {
04146     { 0.1f, 0.1f, 0.1f, 1.0f },
04147     { 0.6f, 0.6f, 0.6f, 0.0f },
04148     { 0.3f, 0.3f, 0.3f, 0.0f },
04149     60.0f
04150 };
04151
04152 // デフォルトの材質名
04153 constexpr char defaultMaterialName[] = "_default-";
04154
04155 //
04156 // Alias OBJ 形式の MTL ファイルを読み込む
04157 //
04158 // mtlpath MTL ファイルのパス名
04159 // mtl 読み込んだ材質名をキー、材質番号を値にした map
04160 // material 材質データ
04161 //
04162 static bool ggLoadMtl(const std::string& mtlpath,
04163     std::map<std::string, GLuint>& mtl,
04164     std::vector<GgSimpleShader::Material>& material)
04165 {
04166     // MTL ファイルが無ければ戻る
04167     std::ifstream mtlfile{ Utf8ToTChar(mtlpath), std::ios::binary };
04168     if (!mtlfile)
04169     {
04170 #if defined(DEBUG)
04171         std::cerr << "Warning: Can't open MTL file: " << mtlpath << std::endl;
04172 #endif
04173         return false;
04174     }
04175
04176     // 一行読み込み用のバッファ
04177     std::string mtlline;
04178
04179     // 材質名（ループの外に置く）
04180     std::string mtlname{ defaultMaterialName };
04181
04182     // 現在の材質番号を登録する
04183     mtl[mtlname] = static_cast<GLuint>(material.size());
04184
04185     // 現在の材質にデフォルトの材質を設定する
04186     material.emplace_back(defaultMaterial);
04187
04188     // 材質データを読み込む
04189     while (std::getline(mtlfile, mtlline))
04190     {
04191         // 空行は読み飛ばす
04192         if (mtlline == "") continue;
04193
04194         // 最後の文字が '\r' なら
04195         if (*(mtlline.end() - 1) == '\r')
04196         {
04197             // 最後の文字を削除する
04198             mtlline.erase(mtlline.end() - 1, mtlline.end());
04199
04200             // 空行になったら読み飛ばす
04201             if (mtlline == "") continue;
04202         }
04203
04204         // 読み込んだ行を文字列ストリームにする
04205         std::istringstream mtlstr{ mtlline };
04206
04207         // オペレータ
04208         std::string mtlop;
04209
04210         // 文字列ストリームから材質パラメータの種類を取り出す
04211         mtlstr >> mtlop;
04212
04213         // '#' で始まる場合はコメントとして行末まで読み飛ばす
04214         if (mtlop[0] == '#') continue;
04215
04216         if (mtlop == "newmtl")
04217         {
04218             // 新規作成する材質名を取り出す
04219             mtlstr >> mtlname;
04220
04221             // 材質名が既に存在するかどうか調べる
04222             const auto m{ mtl.find(mtlname) };

```

```

04223         if (m == mtl.end())
04224         {
04225             // 存在しないので新規作成する材質の番号をその材質名に割り当てる
04226             mtl[mtlname] = static_cast<GLuint>(material.size());
04227
04228             // 新規作成する材質にデフォルトの材質を設定しておく
04229             material.emplace_back(defaultMaterial);
04230         }
04231
04232 #if defined(DEBUG)
04233         std::cerr << "newmtl: " << mtlname << std::endl;
04234 #endif
04235     }
04236     else if (mtlop == "Ka")
04237     {
04238         // 環境光の反射係数を登録する
04239         mtlstr
04240             >> material.back().ambient[0]
04241             >> material.back().ambient[1]
04242             >> material.back().ambient[2];
04243     }
04244     else if (mtlop == "Kd")
04245     {
04246         // 拡散反射係数を登録する
04247         mtlstr
04248             >> material.back().diffuse[0]
04249             >> material.back().diffuse[1]
04250             >> material.back().diffuse[2];
04251     }
04252     else if (mtlop == "Ks")
04253     {
04254         // 鏡面反射係数を登録する
04255         mtlstr
04256             >> material.back().specular[0]
04257             >> material.back().specular[1]
04258             >> material.back().specular[2];
04259     }
04260     else if (mtlop == "Ns")
04261     {
04262         // 輝き係数を登録する
04263         GLfloat shininess;
04264         mtlstr >> shininess;
04265         material.back().shininess = shininess * 0.1f;
04266     }
04267     else if (mtlop == "d")
04268     {
04269         // 不透明度を登録する
04270         mtlstr >> material.back().ambient[3];
04271     }
04272 }
04273
04274 // MTL ファイルの読み込みに失敗したら戻る
04275 if (mtlfile.bad())
04276 {
04277 #if defined(DEBUG)
04278     std::cerr << "Warning: Can't read MTL file: " << mtlpath << std::endl;
04279 #endif
04280     return false;
04281 }
04282
04283 // MTL ファイルの読み込みに成功した
04284 return true;
04285 }
04286
04287 //
04288 // Alias OBJ 形式のファイルを解析する
04289 //
04290 // name Alias OBJ 形式のファイルのファイル名
04291 // group 同じ材質を割り当てるポリゴングループ
04292 // mtl 読み込んだ材質名をキーにした map
04293 // pos 頂点の位置
04294 // norm 頂点の法線
04295 // tex 頂点のテクスチャ座標
04296 // face 三角形のデータ
04297 //
04298 static bool ggParseObj(
04299     const std::string& name,
04300     std::vector<fgrp>& group,
04301     std::vector<GgSimpleShader::Material>& material,
04302     std::vector<vec3>& pos,
04303     std::vector<vec3>& norm,
04304     std::vector<vec2>& tex,
04305     std::vector<fidx>& face,
04306     bool normalize
04307 )
04308 {
04309     // ファイルパスからディレクトリ名を取り出す

```

```

04310     const std::string path{ name };
04311     const size_t base{ path.find_last_of("/\\") };
04312     const std::string dirname{ (base == std::string::npos) ? "" : path.substr(0, base + 1) };
04313
04314     // OBJ ファイルを読み込む
04315     std::ifstream file{ Utf8ToTChar(path) };
04316
04317     // 読み込みに失敗したら戻る
04318     if (file.fail())
04319     {
04320         #if defined(DEBUG)
04321             std::cerr << "Error: Can't open OBJ file: " << path << std::endl;
04322         #endif
04323         return false;
04324     }
04325
04326     // ポリゴングループの最初の三角形番号
04327     GLsizei startgroup{static_cast<GLsizei>(group.size())};
04328
04329     // スムーズシェーディングのスイッチ
04330     bool smooth{ false };
04331
04332     // 材質のテーブル
04333     std::map<std::string, GLuint> mtl;
04334
04335     // 現在の材質名 (ループの外で宣言する)
04336     std::string mtlname;
04337
04338     // 座標値の最小値・最大値
04339     vec3 bmin{ FLT_MAX }, bmax{ -FLT_MAX };
04340
04341     // 一行読み込み用のバッファ
04342     std::string line;
04343
04344     // データの読み込み
04345     while (std::getline(file, line))
04346     {
04347         // 空行は読み飛ばす
04348         if (line == "") continue;
04349
04350         // 最後の文字が '\r' なら
04351         if (*(line.end() - 1) == '\r')
04352         {
04353             // 最後の文字を削除する
04354             line.erase(line.end() - 1, line.end());
04355
04356             // 空行になったら読み飛ばす
04357             if (line == "") continue;
04358         }
04359
04360         // 一行を文字列ストリームに入れる
04361         std::istringstream str(line);
04362
04363         // 最初のトークンを命令 (op) とみなす
04364         std::string op;
04365         str >> op;
04366
04367         if (op[0] == '#') continue;
04368
04369         if (op == "v")
04370         {
04371             // 頂点位置
04372             vec3 v;
04373
04374             // 頂点位置はスペースで区切られている
04375             str >> v[0] >> v[1] >> v[2];
04376
04377             // 頂点位置を記録する
04378             pos.emplace_back(v);
04379
04380             // 頂点位置の最小値と最大値を求める (AABB)
04381             for (int i = 0; i < 3; ++i)
04382             {
04383                 bmin[i] = std::min(bmin[i], v[i]);
04384                 bmax[i] = std::max(bmax[i], v[i]);
04385             }
04386         }
04387         else if (op == "vt")
04388         {
04389             // テクスチャ座標
04390             vec2 t;
04391
04392             // 頂点位置はスペースで区切られている
04393             str >> t[0] >> t[1];
04394
04395             // テクスチャ座標を記録する
04396             tex.emplace_back(t);

```

```

04397     }
04398     else if (op == "vn")
04399     {
04400         // 頂点法線
04401         vec3 n;
04402
04403         // 頂点法線はスペースで区切られている
04404         str >> n[0] >> n[1] >> n[2];
04405
04406         // 頂点法線を記録する
04407         norm.emplace_back(n);
04408     }
04409     else if (op == "f")
04410     {
04411         // 三角形データ
04412         fidx f;
04413
04414         // スムースシェーディング
04415         f.smooth = smooth;
04416
04417         // 三頂点のそれぞれについて
04418         for (int i = 0; i < 3; ++i)
04419         {
04420             // 1項目取り出す
04421             std::string s;
04422             str >> s;
04423
04424             // 文字の位置
04425             auto c{ s.begin() };
04426
04427             // テクスチャ座標と法線の番号は未定義を表す 0 にしておく
04428             f.p[i] = f.t[i] = f.n[i] = 0;
04429
04430             // 項目の最初の要素は頂点座標番号
04431             while (c != s.end() && isdigit(*c)) f.p[i] = f.p[i] * 10 + *c++ - '0';
04432             if (c == s.end() || *c++ != '/') continue;
04433
04434             // 二つ目の項目はテクスチャ座標
04435             while (c != s.end() && isdigit(*c)) f.t[i] = f.t[i] * 10 + *c++ - '0';
04436             if (c == s.end() || *c++ != '/') continue;
04437
04438             // 三つ目の項目は法線番号
04439             while (c != s.end() && isdigit(*c)) f.n[i] = f.n[i] * 10 + *c++ - '0';
04440         }
04441
04442         // 三角形データを登録する
04443         face.emplace_back(f);
04444     }
04445     else if (op == "s")
04446     {
04447         // '1' だったらスムースシェーディング有効
04448         std::string s;
04449         str >> s;
04450         smooth = s == "1";
04451     }
04452     else if (op == "usemtl")
04453     {
04454         // 次のポリゴングループの最初の三角形番号
04455         const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04456
04457         // ポリゴングループに三角形が存在すれば
04458         if (nextgroup > startgroup)
04459         {
04460             // ポリゴングループの三角形数と材質番号を記録する
04461             group.emplace_back(nextgroup, mtl[mtlname]);
04462
04463             // 次のポリゴングループの開始番号を保存しておく
04464             startgroup = nextgroup;
04465         }
04466
04467         // 次に usemtl が来るまで材質名を保持する
04468         str >> mtlname;
04469
04470         // 材質の存在チェック
04471         if (mtl.find(mtlname) == mtl.end())
04472         {
04473             #if defined(DEBUG)
04474                 std::cerr << "Warning: Undefined material: " << mtlname << std::endl;
04475             #endif
04476
04477             // デフォルトの材質を割り当てておく
04478             mtlname = defaultMaterialName;
04479         }
04480         #if defined(DEBUG)
04481         else std::cerr << "usemtl: " << mtlname << std::endl;
04482         #endif
04483     }

```

```

04484     else if (op == "mtllib")
04485     {
04486         // MTL ファイルのパス名を作る
04487         str >> std::ws;
04488         std::string mtlpath;
04489         std::getline(str, mtlpath);
04490
04491         // MTL ファイルを読み込む
04492         ggLoadMtl(dirname + mtlpath, mtl, material);
04493     }
04494 }
04495
04496 // OBJ ファイルの読み込みに失敗したら戻る
04497 if (file.bad())
04498 {
04499 #if defined(DEBUG)
04500     std::cerr << "Error: Can't read OBJ file: " << path << std::endl;
04501 #endif
04502     return false;
04503 }
04504
04505 // 最後のポリゴングループの次の三角形番号
04506 const GLsizei nextgroup(static_cast<GLsizei>(face.size()));
04507 if (nextgroup > startgroup)
04508 {
04509     // 最後のポリゴングループの三角形数と材質を記録する
04510     group.emplace_back(nextgroup, static_cast<GLuint>(mtl[mtlname]));
04511 }
04512
04513 // スムーズシェーディングしない三角形の頂点を追加する
04514 for (auto& f : face)
04515 {
04516     if (!f.smooth)
04517     {
04518         // 三頂点のそれぞれについて
04519         for (int i = 0; i < 3; ++i)
04520         {
04521             // 新しい頂点座標を生成する (std::array の要素は emplace_back できない)
04522             pos.emplace_back(pos[f.p[i] - 1]);
04523             f.p[i] = static_cast<GLuint>(pos.size());
04524
04525             if (f.t[i] > 0)
04526             {
04527                 // 新しいテクスチャ座標を生成する
04528                 tex.emplace_back(tex[f.t[i] - 1]);
04529                 f.t[i] = static_cast<GLuint>(tex.size());
04530             }
04531
04532             if (f.n[i] > 0)
04533             {
04534                 // 新しい法線を生成する
04535                 norm.emplace_back(norm[f.n[i] - 1]);
04536                 f.n[i] = static_cast<GLuint>(norm.size());
04537             }
04538         }
04539     }
04540 }
04541
04542 // 法線データがなければ算出しておく
04543 if (norm.empty())
04544 {
04545     // 法線データ数の初期値は頂点数と同じでスムーズシェーディングのために初期値は 0
04546     norm.resize(pos.size(), { 0.0f, 0.0f, 0.0f });
04547
04548     // 面の法線の算出と頂点法線の算出
04549     for (auto& f : face)
04550     {
04551         // 頂点座標番号
04552         const auto v0{ f.p[0] - 1 };
04553         const auto v1{ f.p[1] - 1 };
04554         const auto v2{ f.p[2] - 1 };
04555
04556         // v1 - v0, v2 - v0 を求める
04557         const GLfloat d1[]{ pos[v1][0] - pos[v0][0], pos[v1][1] - pos[v0][1], pos[v1][2] - pos[v0][2] };
04558         const GLfloat d2[]{ pos[v2][0] - pos[v0][0], pos[v2][1] - pos[v0][1], pos[v2][2] - pos[v0][2] };
04559
04560         // 外積により面法線を求める
04561         vec3 n;
04562         ggCross(n.data(), d1, d2);
04563
04564         if (f.smooth)
04565         {
04566             // スムーズシェーディングを行うときは
04567             for (int i = 0; i < 3; ++i)
04568             {

```

```

04569         // 面法線を頂点法線に積算する
04570         norm[v0][i] += n[i];
04571         norm[v1][i] += n[i];
04572         norm[v2][i] += n[i];
04573
04574         // 面の各頂点の法線番号は頂点番号と同じにする
04575         f.n[i] = f.p[i];
04576     }
04577 }
04578 else
04579 {
04580     // 面法線を最初の頂点に保存する
04581     norm[v0] = n;
04582     f.n[0] = f.p[0];
04583
04584     // 2 頂点追加
04585     for (int i = 1; i < 3; ++i)
04586     {
04587         norm.emplace_back(n);
04588         f.n[i] = static_cast<GLuint>(norm.size());
04589     }
04590 }
04591 }
04592
04593 // 頂点の法線ベクトルを正規化する
04594 for (auto& n : norm) ggNormalize3(n.data());
04595 }
04596
04597 // 図形の正規化
04598 if (normalize)
04599 {
04600     // 図形の大きさ
04601     const auto sx{ bmax[0] - bmin[0] };
04602     const auto sy{ bmax[1] - bmin[1] };
04603     const auto sz{ bmax[2] - bmin[2] };
04604
04605     // 図形のスケール
04606     GLfloat s{ sx };
04607     if (sy > s) s = sy;
04608     if (sz > s) s = sz;
04609     const auto scale{ s != 0.0f ? 2.0f / s : 1.0f };
04610
04611     // 図形の中心位置
04612     const auto cx{ (bmax[0] + bmin[0]) * 0.5f };
04613     const auto cy{ (bmax[1] + bmin[1]) * 0.5f };
04614     const auto cz{ (bmax[2] + bmin[2]) * 0.5f };
04615
04616     // 図形の大きさと位置を正規化する
04617     for (auto& p : pos)
04618     {
04619         p[0] = (p[0] - cx) * scale;
04620         p[1] = (p[1] - cy) * scale;
04621         p[2] = (p[2] - cz) * scale;
04622     }
04623 }
04624
04625 #if defined(DEBUG)
04626     std::cerr
04627     << "[" << name << "]\n(Parsed) Group: " << group.size() << ", Material: " << mtl.size()
04628     << ", Pos: " << pos.size() << ", Norm: " << norm.size() << ", Tex: " << tex.size()
04629     << ", Face: " << face.size() << "\n";
04630 #endif
04631
04632 // OBJ ファイルの読み込み成功
04633 return true;
04634 }
04635 }
04636
04637 //
04638 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Arrays 形式)
04639 //
04640 // name 読み込むOBJ ファイル名
04641 // group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04642 // material 読み込んだデータのポリゴングループごとの材質
04643 // vert 読み込んだデータの頂点属性
04644 // normalize true ならサイズを正規化する
04645 // 戻り値 読み込みに成功したら true
04646 //
04647 //
04648 bool gg::ggLoadSimpleObj(const std::string& name,
04649     std::vector<std::array<GLuint, 3>>& group,
04650     std::vector<GgSimpleShader::Material>& material,
04651     std::vector<GgVertex>& vert,
04652     bool normalize)
04653 {
04654     // 読み込み用の一時記憶領域
04655     std::vector<fgrp> tgroup;
04656     std::vector<vec3> tpos;

```

```

04657     std::vector<vec3> tnorm;
04658     std::vector<vec2> ttex;
04659     std::vector<idx> tface;
04660
04661     // OBJ ファイルを解析する
04662     if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04663
04664     // 頂点属性データのメモリを確保する
04665     vert.reserve(vert.size() + tface.size() * 3);
04666
04667     // ポリゴングループデータのメモリを確保する
04668     group.reserve(group.size() + tgroup.size());
04669     material.reserve(material.size() + tgroup.size());
04670
04671     // ポリゴングループの最初の三角形番号
04672     GLuint startgroup{ 0 };
04673
04674     // ポリゴングループデータの作成
04675     for (auto& g : tgroup)
04676     {
04677         // このポリゴングループの最初の頂点番号と頂点数・材質番号
04678         std::array<GLuint, 3> v;
04679
04680         // このポリゴングループの最初の頂点番号を保存する
04681         v[0] = static_cast<GLuint>(vert.size());
04682
04683         // 三角形ごとの頂点データの作成
04684         for (GLuint j = startgroup; j < g.nextgroup; ++j)
04685         {
04686             // 処理対象の三角形
04687             auto& f = tface[j];
04688
04689             // 三頂点のそれぞれについて
04690             for (int i = 0; i < 3; ++i)
04691             {
04692                 // テクスチャ座標
04693                 vec2 tex{ 0.0f, 0.0f };
04694                 if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04695
04696                 // 頂点法線
04697                 vec3 norm{ 0.0f, 0.0f, 0.0f };
04698                 if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04699
04700                 // 頂点属性の追加
04701                 vert.emplace_back(tpos[f.p[i] - 1].data(), norm.data());
04702             }
04703         }
04704
04705         // このポリゴングループの頂点数を保存する
04706         v[1] = static_cast<GLuint>(vert.size()) - v[0];
04707         v[2] = g.mtlno;
04708
04709         // このポリゴングループの最初の頂点番号と頂点数・材質番号を登録する
04710         group.emplace_back(v);
04711
04712         // 次のポリゴングループの最初の三角形番号を求める
04713         startgroup = g.nextgroup;
04714     }
04715
04716     #if defined(DEBUG)
04717     std::cerr
04718         << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04719         << ", Vertex: " << vert.size() << "\n";
04720     #endif
04721
04722     // OBJ ファイルの読み込み成功
04723     return true;
04724 }
04725
04726 //
04727 // 三角形分割された Alias OBJ 形式のファイルと MTL ファイルを読み込む (Elements 形式)
04728 //
04729 // name 読み込むOBJ ファイル名
04730 // group 読み込んだデータの各ポリゴングループの最初の三角形番号と三角形数
04731 // material 読み込んだデータのポリゴングループごとの材質
04732 // vert 読み込んだデータの頂点属性
04733 // face 読み込んだデータの三角形の頂点インデックス
04734 // normalize true ならサイズを正規化する
04735 // 戻り値 読み込みに成功したら true
04736 //
04737 bool gg::ggLoadSimpleObj(const std::string& name,
04738     std::vector<std::array<GLuint, 3>>& group,
04739     std::vector<GgSimpleShader::Material>& material,
04740     std::vector<GgVertex>& vert,
04741     std::vector<GLuint>& face,
04742     bool normalize)
04743 {

```



```

04744 // 読み込み用の一時記憶領域
04745 std::vector<fgrp> tgroup;
04746 std::vector<vec3> tpos;
04747 std::vector<vec3> tnorm;
04748 std::vector<vec2> ttex;
04749 std::vector<fidx> tface;
04750
04751 // OBJ ファイルを解析する
04752 if (!ggParseObj(name, tgroup, material, tpos, tnorm, ttex, tface, normalize)) return false;
04753
04754 // 頂点属性データの最初の頂点番号
04755 const auto vertbase{ static_cast<GLuint>(vert.size()) };
04756
04757 // 頂点属性データのメモリを確保する
04758 vert.resize(vertbase + tpos.size());
04759
04760 // 三角形データのメモリを確保する
04761 face.reserve(face.size() + tface.size());
04762
04763 // ポリゴングループデータのメモリを確保する
04764 group.reserve(group.size() + tgroup.size());
04765 material.reserve(material.size() + tgroup.size());
04766
04767 // ポリゴングループの最初の三角形番号
04768 GLuint startgroup{ 0 };
04769
04770 // ポリゴングループデータの作成
04771 for (auto& g : tgroup)
04772 {
04773     // このポリゴングループの最初の頂点番号
04774     const auto first{ static_cast<GLuint>(face.size()) };
04775
04776     // 三角形ごとの頂点データの作成
04777     for (GLuint j = startgroup; j < g.nextgroup; ++j)
04778     {
04779         // 処理対象の三角形
04780         auto& f{ tface[j] };
04781
04782         // 三頂点のそれぞれについて
04783         for (int i = 0; i < 3; ++i)
04784         {
04785             // 追加する三角形データの頂点番号
04786             const auto q{ f.p[i] - 1 + vertbase };
04787
04788             // 三角形データの追加
04789             face.emplace_back(q);
04790
04791             // テクスチャ座標番号
04792             vec2 tex{ 0.0f, 0.0f };
04793             if (f.t[i] > 0) tex = ttex[f.t[i] - 1];
04794
04795             // 頂点法線番号
04796             vec3 norm{ 0.0f, 0.0f, 0.0f };
04797             if (f.n[i] > 0) norm = tnorm[f.n[i] - 1];
04798
04799             // 頂点の格納
04800             vert[q] = GgVertex(tpos[f.p[i] - 1].data(), norm.data());
04801         }
04802     }
04803
04804     // このポリゴングループの最初の三角形番号と三角形数・材質番号を登録する
04805     group.emplace_back(std::array<GLuint, 3>{ first, static_cast<GLuint>(face.size()) - first, g.mtlno
04806 });
04807
04808     // 次のポリゴングループの最初の三角形番号を求める
04809     startgroup = g.nextgroup;
04810 }
04811
04812 #if defined(DEBUG)
04813     std::cerr
04814     << "(Stored) Group: " << group.size() << ", Material: " << material.size()
04815     << ", Vertex: " << vert.size() << ", Face: " << face.size() << "\n";
04816 #endif
04817 // OBJ ファイルの読み込み成功
04818 return true;
04819 }
04820
04821 //
04822 // シェーダオブジェクトのコンパイル結果を表示する
04823 //
04824 static GLboolean printShaderInfoLog(GLuint shader, const std::string& str)
04825 {
04826     // コンパイル結果を取得する
04827     GLint status;
04828     glGetShaderiv(shader, GL_COMPILE_STATUS, &status);
04829 #if defined(DEBUG)

```

```

04830     if (status == GL_FALSE) std::cerr << "Compile Error in " << str << std::endl;
04831 #endif
04832
04833     // シェーダのコンパイル時のログの長さを取得する
04834     GLsizei bufSize;
04835     glGetShaderiv(shader, GL_INFO_LOG_LENGTH, &bufSize);
04836
04837     if (bufSize > 1)
04838     {
04839         // シェーダのコンパイル時のログの内容を取得する
04840         std::vector<GLchar> infoLog(bufSize);
04841         GLsizei length;
04842         glGetShaderInfoLog(shader, bufSize, &length, infoLog.data());
04843         #if defined(DEBUG)
04844             std::cerr << infoLog.data();
04845         #endif
04846     }
04847
04848     // コンパイル結果を返す
04849     return static_cast<GLboolean>(status);
04850 }
04851
04852 //
04853 // プログラムオブジェクトのリンク結果を表示する
04854 //
04855 static GLboolean printProgramInfoLog(GLuint program)
04856 {
04857     // リンク結果を取得する
04858     GLint status;
04859     glGetProgramiv(program, GL_LINK_STATUS, &status);
04860     #if defined(DEBUG)
04861     if (status == GL_FALSE) std::cerr << "Link Error." << std::endl;
04862     #endif
04863
04864     // シェーダのリンク時のログの長さを取得する
04865     GLsizei bufSize;
04866     glGetProgramiv(program, GL_INFO_LOG_LENGTH, &bufSize);
04867
04868     // シェーダのリンク時のログの内容を取得する
04869     if (bufSize > 1)
04870     {
04871         std::vector<GLchar> infoLog(bufSize);
04872         GLsizei length;
04873         glGetProgramInfoLog(program, bufSize, &length, infoLog.data());
04874         #if defined(DEBUG)
04875             std::cerr << infoLog.data() << std::endl;
04876         #endif
04877     }
04878
04879     // リンク結果を返す
04880     return static_cast<GLboolean>(status);
04881 }
04882
04883 //
04884 // シェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
04885 //
04886 //     vsrc パーテックスシェーダのソースプログラムの文字列
04887 //     fsrc フラグメントシェーダのソースプログラムの文字列 (nullptr なら不使用)
04888 //     gsrc ジオメトリシェーダのソースプログラムの文字列 (nullptr なら不使用)
04889 //     nvarying フィードバックする varying 変数の数 (0 なら不使用)
04890 //     varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
04891 //     vtext パーテックスシェーダのコンパイル時のメッセージに追加する文字列
04892 //     ftext フラグメントシェーダのコンパイル時のメッセージに追加する文字列
04893 //     gtext ジオメトリシェーダのコンパイル時のメッセージに追加する文字列
04894 //     戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
04895 //
04896 GLuint gg::ggCreateShader(
04897     const std::string& vsrc,
04898     const std::string& fsrc,
04899     const std::string& gsrc,
04900     GLint nvarying,
04901     const char* const* varyings,
04902     const std::string& vtext,
04903     const std::string& ftext,
04904     const std::string& gtext)
04905 {
04906     // シェーダプログラムの作成
04907     const auto program{ glCreateProgram() };
04908
04909     if (program > 0)
04910     {
04911         bool status = true;
04912
04913         if (!vsrc.empty())
04914         {
04915             // パーテックスシェーダのシェーダオブジェクトを作成する
04916             const auto vertShader{ glCreateShader(GL_VERTEX_SHADER) };

```

```

04917     const auto* vsrcp{ vsrc.c_str() };
04918     glShaderSource(vertShader, 1, &vsrcp, nullptr);
04919     glCompileShader(vertShader);
04920
04921     // パーテックスシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04922     if (printShaderInfoLog(vertShader, vtext))
04923         glAttachShader(program, vertShader);
04924     else
04925         status = false;
04926     glDeleteShader(vertShader);
04927 }
04928
04929 if (!fsrc.empty())
04930 {
04931     // フラグメントシェーダのシェーダオブジェクトを作成する
04932     const auto fragShader{ glCreateShader(GL_FRAGMENT_SHADER) };
04933     const auto* fsrcp{ fsrc.c_str() };
04934     glShaderSource(fragShader, 1, &fsrcp, nullptr);
04935     glCompileShader(fragShader);
04936
04937     // フラグメントシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04938     if (printShaderInfoLog(fragShader, ftext))
04939         glAttachShader(program, fragShader);
04940     else
04941         status = false;
04942     glDeleteShader(fragShader);
04943 }
04944
04945 if (!gsrc.empty())
04946 {
04947     #if defined(GL_GLES_PROTOTYPES)
04948     #   if defined(DEBUG)
04949         std::cerr << gtext << ": The geometry shader is not supported." << std::endl;
04950         status = false;
04951     #   endif
04952     #else
04953         // ジオメトリシェーダのシェーダオブジェクトを作成する
04954         const auto geomShader{ glCreateShader(GL_GEOMETRY_SHADER) };
04955         const auto* gsrcp{ gsrc.c_str() };
04956         glShaderSource(geomShader, 1, &gsrcp, nullptr);
04957         glCompileShader(geomShader);
04958
04959         // ジオメトリシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
04960         if (printShaderInfoLog(geomShader, gtext))
04961             glAttachShader(program, geomShader);
04962         else
04963             status = false;
04964         glDeleteShader(geomShader);
04965     #endif
04966 }
04967
04968 // 全てのシェーダオブジェクトのコンパイルに成功したら
04969 if (status)
04970 {
04971     // feedback に使う varying 変数を指定する
04972     if (nvarying > 0)
04973         glTransformFeedbackVaryings(program, nvarying, varyings, GL_SEPARATE_ATTRIBS);
04974
04975     // シェーダプログラムをリンクする
04976     glLinkProgram(program);
04977
04978     // リンクに成功したらプログラムオブジェクト名を返す
04979     if (printProgramInfoLog(program) != GL_FALSE) return program;
04980 }
04981 }
04982
04983 // プログラムオブジェクトが作成できなかった
04984 glDeleteProgram(program);
04985 return 0;
04986 }
04987
04988 //
04989 // シェーダのソースファイルを読み込んだ vector を返す
04990 //
04991 // name ソースファイル名
04992 // src 読み込んだソースファイルの文字列
04993 // 戻り値 読み込みの成功すれば true, 失敗したら false
04994 //
04995 static bool readShaderSource(const std::string& name, std::string& src)
04996 {
04997     // ファイル名が nullptr ならそのまま戻る
04998     if (name.empty()) return true;
04999
05000     // ソースファイルを開く
05001     std::ifstream file{ Utf8ToTChar(name), std::ios::binary };
05002     if (file.fail())
05003     {

```

```

05004 // ファイルが開けなければエラーで戻る
05005 #if defined(DEBUG)
05006     std::cerr << "Error: Can't open source file: " << name << std::endl;
05007 #endif
05008     return false;
05009 }
05010 // ファイル全体を文字列として読み込む
05011 std::istreambuf_iterator<char> it{ file };
05012 std::istreambuf_iterator<char> last;
05013 src = std::string(it, last);
05014
05015 // ファイルがうまく読み込めなければ戻る
05016 if (file.bad())
05017 {
05018     #if defined(DEBUG)
05019         std::cerr << "Error: Could not read source file: " << name << std::endl;
05020     #endif
05021     return false;
05022 }
05023
05024 // ファイルを閉じて戻る
05025 return true;
05026 }
05027
05028 // シェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05029 //
05030 // vert バーテックスシェーダのソースファイル名
05031 // frag フラグメントシェーダのソースファイル名 (nullptr なら不使用)
05032 // geom ジオメトリシェーダのソースファイル名 (nullptr なら不使用)
05033 // nvarying フィードバックする varying 変数の数 (0 なら不使用)
05034 // varyings フィードバックする varying 変数のリスト (nullptr なら不使用)
05035 // 戻り値 シェーダプログラムのプログラム名 (作成できなければ 0)
05036
05037 GLuint gg::ggLoadShader(
05038     const std::string& vert,
05039     const std::string& frag,
05040     const std::string& geom,
05041     GLint nvarying,
05042     const char* const* varyings
05043 )
05044 {
05045     // 読み込んだシェーダのソースプログラム
05046     std::string vsrc, fsrc, gsrc;
05047
05048     // 読み込んだ結果
05049     bool status;
05050
05051     // シェーダのソースファイルを読み込む
05052     status = readShaderSource(vert, vsrc);
05053     status = readShaderSource(frag, fsrc) && status;
05054     status = readShaderSource(geom, gsrc) && status;
05055
05056     // 全てのソースファイルが読み込めていなかったらエラー
05057     if (!status) return 0;
05058
05059     // プログラムオブジェクトを作成する
05060     return ggCreateShader(vsrc, fsrc, gsrc, nvarying, varyings, vert, frag, geom);
05061 }
05062
05063 #if !defined(__APPLE__) && !defined(GL_GLES_PROTOTYPES)
05064 // コンピュートシェーダのソースプログラムの文字列を読み込んでプログラムオブジェクトを作成する
05065 //
05066 // csrc コンピュートシェーダのソースプログラムの文字列
05067 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05068
05069 GLuint gg::ggCreateComputeShader(const std::string& csrc, const std::string& ctext)
05070 {
05071     // シェーダプログラムの作成
05072     const auto program{ glCreateProgram() };
05073
05074     if (program > 0)
05075     {
05076         // ソースプログラムの文字列が空だったら 0 を返す
05077         if (csrc.empty()) return 0;
05078
05079         // コンピュートシェーダのシェーダオブジェクトを作成する
05080         const auto compShader{ glCreateShader(GL_COMPUTE_SHADER) };
05081         const auto* csrccp{ csrc.c_str() };
05082         glShaderSource(compShader, 1, &csrccp, nullptr);
05083         glCompileShader(compShader);
05084
05085         // コンピュートシェーダのシェーダオブジェクトをプログラムオブジェクトに組み込む
05086         if (printShaderInfoLog(compShader, ctext)) glAttachShader(program, compShader);
05087         glDeleteShader(compShader);
05088     }
05089 }

```

```

05091
05092 // シェーダプログラムをリンクする
05093 glLinkProgram(program);
05094
05095 // プログラムオブジェクトが作成できなければ 0 を返す
05096 if (printProgramInfoLog(program) == GL_FALSE)
05097 {
05098     glDeleteProgram(program);
05099     return 0;
05100 }
05101 }
05102
05103 // プログラムオブジェクトを返す
05104 return program;
05105 }
05106
05107 //
05108 // コンピュートシェーダのソースファイルを読み込んでプログラムオブジェクトを作成する
05109 //
05110 // comp コンピュートシェーダのソースファイル名
05111 // 戻り値 プログラムオブジェクトのプログラム名 (作成できなければ 0)
05112 //
05113 GLuint gg::ggLoadComputeShader(const std::string& comp)
05114 {
05115     // シェーダのソースファイルを読み込む
05116     std::string csrc;
05117     if (readShaderSource(comp, csrc))
05118     {
05119         // プログラムオブジェクトを作成する
05120         return ggCreateComputeShader(csrc.data(), comp);
05121     }
05122     // プログラムオブジェクト作成失敗
05123     return 0;
05124 }
05125 #endif
05126
05127 //
05128 // 点: データ作成
05129 //
05130 //
05131 void gg::GgPoints::load(const GgVector* pos, GLsizei count, GLenum usage)
05132 {
05133     // 頂点バッファオブジェクトを作成する
05134     position = std::make_shared<GgBuffer<GgVector>>(GL_ARRAY_BUFFER, pos,
05135         static_cast<GLsizei>(sizeof(GgVector)), count, usage);
05136     // このバッファオブジェクトは index == 0 の in 変数から入力する
05137     glVertexAttribPointer(0, static_cast<GLint>(pos->size()), GL_FLOAT, GL_FALSE, 0, 0);
05138     glEnableVertexAttribArray(0);
05139 }
05140
05141 //
05142 // 点: 描画
05143 //
05144 void gg::GgPoints::draw(GLint first, GLsizei count) const
05145 {
05146     // 頂点配列オブジェクトを指定する
05147     GgShape::draw(first, count);
05148     // 図形を描画する
05149     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05150 }
05151
05152 //
05153 // 三角形: データ作成
05154 //
05155 //
05156 void gg::GgTriangles::load(const GgVertex* vert, GLsizei count, GLenum usage)
05157 {
05158     // 頂点バッファオブジェクトを作成する
05159     vertex = std::make_shared<GgBuffer<GgVertex>>(GL_ARRAY_BUFFER, vert,
05160         static_cast<GLsizei>(sizeof(GgVertex)), count, usage);
05161     // 頂点の位置は index == 0 の in 変数から入力する
05162     glVertexAttribPointer(0, static_cast<GLint>(vert->position.size()), GL_FLOAT, GL_FALSE,
05163         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, position));
05164     glEnableVertexAttribArray(0);
05165     // 頂点の法線は index == 1 の in 変数から入力する
05166     glVertexAttribPointer(1, static_cast<GLint>(vert->normal.size()), GL_FLOAT, GL_FALSE,
05167         sizeof(GgVertex), static_cast<const char*>(0) + offsetof(GgVertex, normal));
05168     glEnableVertexAttribArray(1);
05169 }
05170
05171 //
05172 // 三角形: 描画
05173 //
05174 //
05175 void gg::GgTriangles::draw(GLint first, GLsizei count) const

```

```

05176 {
05177     // 頂点配列オブジェクトを指定する
05178     GgShape::draw(first, count);
05179
05180     // 図形を描画する
05181     glDrawArrays(getMode(), first, count > 0 ? count : getCount() - first);
05182 }
05183
05184 //
05185 // オブジェクト：描画
05186 //
05187 void gg::GgElements::draw(GLint first, GLsizei count) const
05188 {
05189     // 頂点配列オブジェクトを指定する
05190     GgShape::draw(first, count);
05191
05192     // 図形を描画する
05193     glDrawElements(getMode(), count > 0 ? count : getIndexCount() - first,
05194         GL_UNSIGNED_INT, static_cast<GLuint*>(0) + first);
05195 }
05196
05197 //
05198 // 点群を立方体状に生成する
05199 //
05200 std::shared_ptr<gg::GgPoints> gg::ggPointsCube(GLsizei count, GLfloat length, GLfloat cx, GLfloat cy,
    GLfloat cz)
05201 {
05202     // メモリを確保する
05203     std::vector<GgVector> pos;
05204     pos.reserve(count);
05205
05206     // 点を生成する
05207     for (GLsizei v = 0; v < count; ++v)
05208     {
05209         const GgVector p
05210         {
05211             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cx,
05212             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cy,
05213             (static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 0.5f) * length + cz,
05214             1.0f
05215         };
05216
05217         pos.emplace_back(p);
05218     }
05219
05220     // 点データの GgPoints オブジェクトを作成して返す
05221     return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05222 }
05223
05224 //
05225 // 点群を球状に生成する
05226 //
05227 std::shared_ptr<gg::GgPoints> gg::ggPointsSphere(GLsizei count, GLfloat radius,
    GLfloat cx, GLfloat cy, GLfloat cz)
05228 {
05229     // メモリを確保する
05230     std::vector<GgVector> pos;
05231     pos.reserve(count);
05232
05233     // 点を生成する
05234     for (GLsizei v = 0; v < count; ++v)
05235     {
05236         const auto r{ radius * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) };
05237         const auto t{ 6.28318530718f * static_cast<GLfloat>(rand()) / (static_cast<GLfloat>(RAND_MAX) +
05238             1.0f) };
05239         const auto cp{ 2.0f * static_cast<GLfloat>(rand()) / static_cast<GLfloat>(RAND_MAX) - 1.0f };
05240         const auto sp{ sqrtf(1.0f - cp * cp) };
05241         const auto ct{ cosf(t) };
05242         const auto st{ sinf(t) };
05243         const GgVector p{ r * sp * ct + cx, r * sp * st + cy, r * cp + cz, 1.0f };
05244
05245         pos.emplace_back(p);
05246     }
05247
05248     // 点データの GgPoints オブジェクトを作成して返す
05249     return std::make_shared<gg::GgPoints>(pos.data(), static_cast<GLsizei>(pos.size()), GL_POINTS);
05250 }
05251
05252 //
05253 // 矩形状に 2 枚の三角形を生成する
05254 //
05255 std::shared_ptr<gg::GgTriangles> gg::ggRectangle(GLfloat width, GLfloat height)
05256 {
05257     // 頂点属性
05258     std::array<gg::GgVertex, 4> vert;
05259
05260     // 頂点位置と法線を求める

```

```

05261     for (int v = 0; v < 4; ++v)
05262     {
05263         const auto x{ ((v & 1) * 2 - 1) * width };
05264         const auto y{ ((v & 2) - 1) * height };
05265
05266         vert[v] = gg::GgVertex{ x, y, 0.0f, 0.0f, 0.0f, 1.0f };
05267     }
05268
05269     // 矩形の GgTriangles オブジェクトを作成する
05270     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_STRIP);
05271 }
05272
05273 //
05274 // 楕円状に三角形を生成する
05275 //
05276 std::shared_ptr<gg::GgTriangles> gg::ggEllipse(GLfloat width, GLfloat height, GLuint slices)
05277 {
05278     // 楕円のスケール
05279     constexpr GLfloat scale{ 0.5f };
05280
05281     // 作業用のメモリ
05282     std::vector<gg::GgVertex> vert;
05283
05284     // 頂点位置と法線を求める
05285     for (GLuint v = 0; v < slices; ++v)
05286     {
05287         const auto t{ 6.28318530717f * static_cast<GLfloat>(v) / static_cast<GLfloat>(slices) };
05288         const auto x{ cosf(t) * width * scale };
05289         const auto y{ sinf(t) * height * scale };
05290
05291         vert.emplace_back(x, y, 0.0f, 0.0f, 0.0f, 1.0f);
05292     }
05293
05294     // GgTriangles オブジェクトを作成する
05295     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLE_FAN);
05296 }
05297
05298 //
05299 // Wavefront OBJ ファイルを読み込む (Arrays 形式)
05300 //
05301 std::shared_ptr<gg::GgTriangles> gg::ggArraysObj(const std::string& name, bool normalize)
05302 {
05303     std::vector<std::array<GLuint, 3>> group;
05304     std::vector<gg::GgSimpleShader::Material> material;
05305     std::vector<gg::GgVertex> vert;
05306
05307     // ファイルを読み込む
05308     if (!ggLoadSimpleObj(name, group, material, vert, normalize)) return nullptr;
05309
05310     // GgTriangles オブジェクトを作成する
05311     return std::make_shared<gg::GgTriangles>(vert.data(), static_cast<GLsizei>(vert.size()),
GL_TRIANGLES);
05312 }
05313
05314 //
05315 // Wavefront OBJ ファイルを読み込む (Elements 形式)
05316 //
05317 std::shared_ptr<gg::GgElements> gg::ggElementsObj(const std::string& name, bool normalize)
05318 {
05319     std::vector<std::array<GLuint, 3>> group;
05320     std::vector<gg::GgSimpleShader::Material> material;
05321     std::vector<gg::GgVertex> vert;
05322     std::vector<GLuint> face;
05323
05324     // ファイルを読み込む
05325     if (!ggLoadSimpleObj(name, group, material, vert, face, normalize)) return nullptr;
05326
05327     // GgElements オブジェクトを作成する
05328     return std::make_shared<gg::GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05329 }
05330
05331 //
05332 //
05333 // マッシュ形状を作成する (Elements 形式)
05334 //
05335 std::shared_ptr<gg::GgElements> gg::ggElementsMesh(GLuint slices, GLuint stacks, const
GLfloat (*pos)[3], const GLfloat (*norm)[3])
05336 {
05337     // 頂点属性
05338     std::vector<gg::GgVertex> vert((slices + 1) * (stacks + 1));
05339
05340     // 頂点の法線を求める
05341     for (GLuint j = 0; j <= stacks; ++j)
05342     {
05343         for (GLuint i = 0; i <= slices; ++i)

```

```

05344 {
05345     // 処理対象の頂点番号
05346     const auto k{ j * (slices + 1) + i };
05347
05348     // 頂点の法線
05349     gg::GgVector tnorm;
05350     tnorm[3] = 0.0f;
05351
05352     if (norm)
05353     {
05354         tnorm[0] = norm[k][0];
05355         tnorm[1] = norm[k][1];
05356         tnorm[2] = norm[k][2];
05357     }
05358     else
05359     {
05360         // 処理対象の頂点の周囲の頂点番号
05361         const auto kim{ i > 0 ? k - 1 : k };
05362         const auto kip{ i < slices ? k + 1 : k };
05363         const auto kjm{ j > 0 ? k - slices - 1 : k };
05364         const auto kjp{ j < stacks ? k + slices + 1 : k };
05365
05366         // 接線ベクトル
05367         const std::array<GLfloat, 3> t
05368         {
05369             pos[kip][0] - pos[kim][0],
05370             pos[kip][1] - pos[kim][1],
05371             pos[kip][2] - pos[kim][2]
05372         };
05373
05374         // 従接線ベクトル
05375         const std::array<GLfloat, 3> b
05376         {
05377             pos[kjp][0] - pos[kjm][0],
05378             pos[kjp][1] - pos[kjm][1],
05379             pos[kjp][2] - pos[kjm][2]
05380         };
05381
05382         // 法線
05383         tnorm[0] = t[1] * b[2] - t[2] * b[1];
05384         tnorm[1] = t[2] * b[0] - t[0] * b[2];
05385         tnorm[2] = t[0] * b[1] - t[1] * b[0];
05386
05387         // 法線の正規化
05388         ggNormalize3(tnorm);
05389     }
05390
05391     // 頂点の位置
05392     const gg::GgVector tpos{ pos[k][0], pos[k][1], pos[k][2], 1.0f };
05393
05394     // 頂点属性の保存
05395     vert[k] = gg::GgVertex{ tpos, tnorm };
05396 }
05397 }
05398
05399 // 頂点のインデックス (三角形データ)
05400 std::vector<GLuint> face;
05401
05402 // 頂点のインデックスを求める
05403 for (GLuint j = 0; j < stacks; ++j)
05404 {
05405     for (GLuint i = 0; i < slices; ++i)
05406     {
05407         // 処理対象のマス
05408         const auto k{ (slices + 1) * j + i };
05409
05410         // マスの上半分の三角形
05411         face.emplace_back(k);
05412         face.emplace_back(k + slices + 2);
05413         face.emplace_back(k + 1);
05414
05415         // マスの下半分の三角形
05416         face.emplace_back(k);
05417         face.emplace_back(k + slices + 1);
05418         face.emplace_back(k + slices + 2);
05419     }
05420 }
05421
05422 // GgElements オブジェクトを作成する
05423 return std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
05424     face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
05425 }
05426
05427 //
05428 // 球状に三角形データを生成する (Elements 形式)
05429 //
05430 std::shared_ptr<gg::GgElements> gg::ggElementsSphere(GLfloat radius, int slices, int stacks)

```



```

05431 {
05432     // 頂点の位置と法線
05433     std::vector<GLfloat> p, n;
05434
05435     // 頂点の位置と法線を求める
05436     for (int j = 0; j <= stacks; ++j)
05437     {
05438         const auto t{ static_cast<GLfloat>(j) / static_cast<GLfloat>(stacks) };
05439         const auto ph{ 3.1415926536f * t };
05440         const auto y{ cosf(ph) };
05441         const auto r{ sinf(ph) };
05442
05443         for (int i = 0; i <= slices; ++i)
05444         {
05445             const auto s{ static_cast<GLfloat>(i) / static_cast<GLfloat>(slices) };
05446             const auto th{ -2.0f * 3.1415926536f * s };
05447             const auto x{ r * cosf(th) };
05448             const auto z{ r * sinf(th) };
05449
05450             // 頂点の座標値
05451             p.emplace_back(x * radius);
05452             p.emplace_back(y * radius);
05453             p.emplace_back(z * radius);
05454
05455             // 頂点の法線
05456             n.emplace_back(x);
05457             n.emplace_back(y);
05458             n.emplace_back(z);
05459         }
05460     }
05461
05462     // GgElements オブジェクトを作成する
05463     return ggElementsMesh(slices, stacks, reinterpret_cast<GLfloat(*)[3]>(p.data()),
05464         reinterpret_cast<GLfloat(*)[3]>(n.data()));
05465 }
05466
05467 //
05468 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05469 //
05470 //   r 光源の強度の環境光成分の赤成分
05471 //   g 光源の強度の環境光成分の緑成分
05472 //   b 光源の強度の環境光成分の青成分
05473 //   a 光源の強度の環境光成分の不透明度
05474 //   first 値を設定する光源データの最初の番号
05475 //   count 値を設定する光源データの数
05476 //
05477 void gg::GgSimpleShader::LightBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05478     GLint first, GLsizei count) const
05479 {
05480     // データを格納するバッファオブジェクトの先頭のポインタ
05481     const auto start{ static_cast<char*>(map(first, count)) };
05482     for (GLsizei i = 0; i < count; ++i)
05483     {
05484         // バッファオブジェクトの i 番目のブロックのポインタ
05485         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05486
05487         // 光源の環境光成分を設定する
05488         light->ambient[0] = r;
05489         light->ambient[1] = g;
05490         light->ambient[2] = b;
05491         light->ambient[3] = a;
05492     }
05493     unmap();
05494 }
05495
05496 //
05497 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の環境光成分を設定する
05498 //
05499 //   ambient 光源の強度の環境光成分
05500 //   first 値を設定する光源データの最初の番号
05501 //   count 値を設定する光源データの数
05502 //
05503 void gg::GgSimpleShader::LightBuffer::loadAmbient(const GgVector& ambient,
05504     GLint first, GLsizei count) const
05505 {
05506     // データを格納するバッファオブジェクトの先頭のポインタ
05507     const auto start{ static_cast<char*>(map(first, count)) };
05508     for (GLsizei i = 0; i < count; ++i)
05509     {
05510         // バッファオブジェクトの i 番目のブロックのポインタ
05511         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05512
05513         // 光源の強度の環境光成分を設定する
05514         light->ambient = ambient;
05515     }
05516     unmap();
05517 }

```

```

05518
05519 //
05520 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05521 //
05522 //   r 光源の強度の拡散反射光成分の赤成分
05523 //   g 光源の強度の拡散反射光成分の緑成分
05524 //   b 光源の強度の拡散反射光成分の青成分
05525 //   a 光源の強度の拡散反射光成分の不透明度
05526 //   first 値を設定する光源データの最初の番号
05527 //   count 値を設定する光源データの数
05528 //
05529 void gg::GgSimpleShader::LightBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05530 GLint first, GLsizei count) const
05531 {
05532     // データを格納するバッファオブジェクトの先頭のポインタ
05533     const auto start{ static_cast<char*>(map(first, count)) };
05534     for (GLsizei i = 0; i < count; ++i)
05535     {
05536         // バッファオブジェクトの i 番目のブロックのポインタ
05537         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05538
05539         // 光源の拡散反射光成分を設定する
05540         light->diffuse[0] = r;
05541         light->diffuse[1] = g;
05542         light->diffuse[2] = b;
05543         light->diffuse[3] = a;
05544     }
05545     unmap();
05546 }
05547
05548 //
05549 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の拡散反射光成分を設定する
05550 //
05551 //   ambient 光源の強度の拡散反射光成分
05552 //   first 値を設定する光源データの最初の番号
05553 //   count 値を設定する光源データの数
05554 //
05555 void gg::GgSimpleShader::LightBuffer::loadDiffuse(const GgVector& diffuse,
05556 GLint first, GLsizei count) const
05557 {
05558     // データを格納するバッファオブジェクトの先頭のポインタ
05559     const auto start{ static_cast<char*>(map(first, count)) };
05560     for (GLsizei i = 0; i < count; ++i)
05561     {
05562         // バッファオブジェクトの i 番目のブロックのポインタ
05563         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05564
05565         // 光源の強度の拡散反射光成分を設定する
05566         light->diffuse = diffuse;
05567     }
05568     unmap();
05569 }
05570
05571 //
05572 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05573 //
05574 //   r 光源の強度の鏡面反射光成分の赤成分
05575 //   g 光源の強度の鏡面反射光成分の緑成分
05576 //   b 光源の強度の鏡面反射光成分の青成分
05577 //   a 光源の強度の鏡面反射光成分の不透明度
05578 //   first 値を設定する光源データの最初の番号
05579 //   count 値を設定する光源データの数
05580 //
05581 void gg::GgSimpleShader::LightBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05582 GLint first, GLsizei count) const
05583 {
05584     // データを格納するバッファオブジェクトの先頭のポインタ
05585     const auto start{ static_cast<char*>(map(first, count)) };
05586     for (GLsizei i = 0; i < count; ++i)
05587     {
05588         // バッファオブジェクトの i 番目のブロックのポインタ
05589         const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05590
05591         // 光源の鏡面反射光成分を設定する
05592         light->specular[0] = r;
05593         light->specular[1] = g;
05594         light->specular[2] = b;
05595         light->specular[3] = a;
05596     }
05597     unmap();
05598 }
05599
05600 //
05601 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の強度の鏡面反射光成分を設定する
05602 //
05603 //   ambient 光源の強度の鏡面反射光成分
05604 //   first 値を設定する光源データの最初の番号

```

```

05605 //   count 値を設定する光源データの数
05606 //
05607 void gg::GgSimpleShader::LightBuffer::loadSpecular(const GgVector& specular,
05608   GLint first, GLsizei count) const
05609 {
05610   // データを格納するバッファオブジェクトの先頭のポインタ
05611   const auto start{ static_cast<char*>(map(first, count)) };
05612   for (GLsizei i = 0; i < count; ++i)
05613   {
05614     // バッファオブジェクトの i 番目のブロックのポインタ
05615     const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05616
05617     // 光源の強度の鏡面反射光成分を設定する
05618     light->specular = specular;
05619   }
05620   unmap();
05621 }
05622
05623 //
05624 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の色を設定するか位置は変更しない
05625 //
05626 //   material 光源の特性の GgSimpleShader::Light 構造体
05627 //   first 値を設定する光源データの最初の番号
05628 //   count 値を設定する光源データの数
05629 //
05630 void gg::GgSimpleShader::LightBuffer::loadColor(const Light& color,
05631   GLint first, GLsizei count) const
05632 {
05633   // データを格納するバッファオブジェクトの先頭のポインタ
05634   const auto start{ static_cast<char*>(map(first, count)) };
05635   for (GLsizei i = 0; i < count; ++i)
05636   {
05637     // バッファオブジェクトの i 番目のブロックのポインタ
05638     const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05639
05640     // 光源の色を設定する
05641     light->ambient = color.ambient;
05642     light->diffuse = color.diffuse;
05643     light->specular = color.specular;
05644   }
05645   unmap();
05646 }
05647
05648 //
05649 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05650 //
05651 //   x 光源の位置の x 座標
05652 //   y 光源の位置の y 座標
05653 //   z 光源の位置の z 座標
05654 //   w 光源の位置の w 座標
05655 //   first 値を設定する光源データの最初の番号
05656 //   count 値を設定する光源データの数
05657 //
05658 void gg::GgSimpleShader::LightBuffer::loadPosition(GLfloat x, GLfloat y, GLfloat z, GLfloat w,
05659   GLint first, GLsizei count) const
05660 {
05661   // データを格納するバッファオブジェクトの先頭のポインタ
05662   const auto start{ static_cast<char*>(map(first, count)) };
05663   for (GLsizei i = 0; i < count; ++i)
05664   {
05665     // バッファオブジェクトの i 番目のブロックのポインタ
05666     const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05667
05668     // 光源の位置を設定する
05669     light->position[0] = x;
05670     light->position[1] = y;
05671     light->position[2] = z;
05672     light->position[3] = w;
05673   }
05674   unmap();
05675 }
05676
05677 //
05678 // 三角形に単純な陰影付けを行うシェーダが参照する光源データ：光源の位置を設定する
05679 //
05680 //   position 光源の位置
05681 //   first 値を設定する光源データの最初の番号
05682 //   count 値を設定する光源データの数
05683 //
05684 void gg::GgSimpleShader::LightBuffer::loadPosition(const GgVector& position,
05685   GLint first, GLsizei count) const
05686 {
05687   // データを格納するバッファオブジェクトの先頭のポインタ
05688   const auto start{ static_cast<char*>(map(first, count)) };
05689   for (GLsizei i = 0; i < count; ++i)
05690   {
05691     // バッファオブジェクトの i 番目のブロックのポインタ

```

```

05692     const auto light{ reinterpret_cast<Light*>(start + getStride() * i) };
05693
05694     // 光源の位置を設定する
05695     light->position = position;
05696 }
05697 unmap();
05698 }
05699
05700 //
05701 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05702 //
05703 //   r 環境光に対する反射係数の赤成分
05704 //   g 環境光に対する反射係数の緑成分
05705 //   b 環境光に対する反射係数の青成分
05706 //   a 環境光に対する反射係数の不透明度
05707 //   first 値を設定する材質データの最初の番号
05708 //   count 値を設定する材質データの数
05709 //
05710 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05711 GLint first, GLsizei count) const
05712 {
05713     // データを格納するバッファオブジェクトの先頭のポインタ
05714     const auto start{ static_cast<char*>(map(first, count)) };
05715     for (GLsizei i = 0; i < count; ++i)
05716     {
05717         // バッファオブジェクトの i 番目のブロックのポインタ
05718         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05719
05720         // 環境光に対する反射係数を設定する
05721         material->ambient[0] = r;
05722         material->ambient[1] = g;
05723         material->ambient[2] = b;
05724         material->ambient[3] = a;
05725     }
05726     unmap();
05727 }
05728
05729 //
05730 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数を設定する
05731 //
05732 //   ambient 環境光に対する反射係数
05733 //   first 値を設定する光源データの最初の番号
05734 //   count 値を設定する光源データの数
05735 //
05736 void gg::GgSimpleShader::MaterialBuffer::loadAmbient(const GgVector& ambient,
05737 GLint first, GLsizei count) const
05738 {
05739     // データを格納するバッファオブジェクトの先頭のポインタ
05740     const auto start{ static_cast<char*>(map(first, count)) };
05741     for (GLsizei i = 0; i < count; ++i)
05742     {
05743         // バッファオブジェクトの i 番目のブロックのポインタ
05744         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05745
05746         // 光源の強度の環境光成分を設定する
05747         material->ambient = ambient;
05748     }
05749     unmap();
05750 }
05751
05752 //
05753 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05754 //
05755 //   r 拡散反射係数の赤成分
05756 //   g 拡散反射係数の緑成分
05757 //   b 拡散反射係数の青成分
05758 //   a 拡散反射係数の不透明度
05759 //   first 値を設定する材質データの最初の番号
05760 //   count 値を設定する材質データの数
05761 //
05762 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05763 GLint first, GLsizei count) const
05764 {
05765     // データを格納するバッファオブジェクトの先頭のポインタ
05766     const auto start{ static_cast<char*>(map(first, count)) };
05767     for (GLsizei i = 0; i < count; ++i)
05768     {
05769         // バッファオブジェクトの i 番目のブロックのポインタ
05770         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05771
05772         // 拡散反射係数を設定する
05773         material->diffuse[0] = r;
05774         material->diffuse[1] = g;
05775         material->diffuse[2] = b;
05776         material->diffuse[3] = a;
05777     }
05778     unmap();

```

```

05779 }
05780
05781 //
05782 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：拡散反射係数を設定する
05783 //
05784 //   diffuse 拡散反射係数
05785 //   first   値を設定する光源データの最初の番号
05786 //   count   値を設定する光源データの数
05787 //
05788 void gg::GgSimpleShader::MaterialBuffer::loadDiffuse(const GgVector& diffuse,
05789 GLint first, GLsizei count) const
05790 {
05791     // データを格納するバッファオブジェクトの先頭のポインタ
05792     const auto start{ static_cast<char*>(map(first, count)) };
05793     for (GLsizei i = 0; i < count; ++i)
05794     {
05795         // バッファオブジェクトの i 番目のブロックのポインタ
05796         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05797
05798         // 光源の強度の環境光成分を設定する
05799         material->diffuse = diffuse;
05800     }
05801     unmap();
05802 }
05803
05804 //
05805 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05806 //
05807 //   r 環境光に対する反射係数と拡散反射係数の赤成分
05808 //   g 環境光に対する反射係数と拡散反射係数の緑成分
05809 //   b 環境光に対する反射係数と拡散反射係数の青成分
05810 //   a 環境光に対する反射係数と拡散反射係数の不透明度
05811 //   first 値を設定する材質データの最初の番号
05812 //   count 値を設定する材質データの数
05813 //
05814 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(GLfloat r, GLfloat g, GLfloat b,
GLfloat a,
GLint first, GLsizei count) const
05815 {
05816     // データを格納するバッファオブジェクトの先頭のポインタ
05817     const auto start{ static_cast<char*>(map(first, count)) };
05818     for (GLsizei i = 0; i < count; ++i)
05819     {
05820         // バッファオブジェクトの i 番目のブロックのポインタ
05821         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05822
05823         // 環境光に対する反射係数と拡散反射係数を設定する
05824         material->ambient[0] = material->diffuse[0] = r;
05825         material->ambient[1] = material->diffuse[1] = g;
05826         material->ambient[2] = material->diffuse[2] = b;
05827         material->ambient[3] = material->diffuse[3] = a;
05828     }
05829     unmap();
05830 }
05831
05832 //
05833 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05834 //
05835 //   color 環境光に対する反射係数と拡散反射係数
05836 //   first 値を設定する光源データの最初の番号
05837 //   count 値を設定する光源データの数
05838 //
05839 //
05840 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GgVector& color,
05841 GLint first, GLsizei count) const
05842 {
05843     // データを格納するバッファオブジェクトの先頭のポインタ
05844     const auto start{ static_cast<char*>(map(first, count)) };
05845     for (GLsizei i = 0; i < count; ++i)
05846     {
05847         // バッファオブジェクトの i 番目のブロックのポインタ
05848         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05849
05850         // 光源の強度の環境光成分を設定する
05851         material->ambient = material->diffuse = color;
05852     }
05853     unmap();
05854 }
05855
05856 //
05857 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：環境光に対する反射係数と拡散反射係数を設定する
05858 //
05859 //   color 環境光に対する反射係数と拡散反射係数を格納した GLfloat 型の 4 要素の配列
05860 //   first 値を設定する材質データの最初の番号
05861 //   count 値を設定する材質データの数
05862 //
05863 void gg::GgSimpleShader::MaterialBuffer::loadAmbientAndDiffuse(const GLfloat* color,
05864 GLint first, GLsizei count) const

```

```

05865 {
05866     // ambient 要素のバイトオフセット
05867     constexpr auto ambientOffset{ offsetof(Material, ambient) };
05868
05869     // ambient 要素のバイト数
05870     constexpr auto ambientSize{ sizeof(Material::diffuse) };
05871
05872     // diffuse 要素のバイトオフセット
05873     constexpr auto diffuseOffset{ offsetof(Material, diffuse) };
05874
05875     // diffuse 要素のバイト数
05876     constexpr auto diffuseSize{ sizeof(Material::diffuse) };
05877
05878     // 元のデータの先頭
05879     const auto* source{ reinterpret_cast<const char*>(color) };
05880
05881     // first 番目のブロックから count 個の ambient 要素と diffuse 要素に値を設定する
05882     bind();
05883     for (GLsizei i = 0; i < count; ++i)
05884     {
05885         // 格納先
05886         const auto destination{ getStride() * (first + i) };
05887
05888         // first + i 番目のブロックの ambient 要素に値を設定する
05889         glBufferSubData(getTarget(), destination + ambientOffset, ambientSize, source + i * ambientSize);
05890
05891         // first + i 番目のブロックの diffuse 要素に値を設定する
05892         glBufferSubData(getTarget(), destination + diffuseOffset, diffuseSize, source + i * diffuseSize);
05893     }
05894 }
05895
05896 //
05897 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05898 //
05899 //     r 鏡面反射係数の赤成分
05900 //     g 鏡面反射係数の緑成分
05901 //     b 鏡面反射係数の青成分
05902 //     a 鏡面反射係数の不透明度
05903 //     first 値を設定する材質データの最初の番号
05904 //     count 値を設定する材質データの数
05905 //
05906 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(GLfloat r, GLfloat g, GLfloat b, GLfloat a,
05907     GLint first, GLsizei count) const
05908 {
05909     // データを格納するバッファオブジェクトの先頭のポインタ
05910     const auto start{ static_cast<char*>(map(first, count)) };
05911     for (GLsizei i = 0; i < count; ++i)
05912     {
05913         // バッファオブジェクトの i 番目のブロックのポインタ
05914         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05915
05916         // 鏡面反射係数を設定する
05917         material->specular[0] = r;
05918         material->specular[1] = g;
05919         material->specular[2] = b;
05920         material->specular[3] = a;
05921     }
05922     unmap();
05923 }
05924
05925 //
05926 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：鏡面反射係数を設定する
05927 //
05928 //     specular 鏡面反射係数
05929 //     first 値を設定する光源データの最初の番号
05930 //     count 値を設定する光源データの数
05931 //
05932 void gg::GgSimpleShader::MaterialBuffer::loadSpecular(const GgVector& specular,
05933     GLint first, GLsizei count) const
05934 {
05935     // データを格納するバッファオブジェクトの先頭のポインタ
05936     const auto start{ static_cast<char*>(map(first, count)) };
05937     for (GLsizei i = 0; i < count; ++i)
05938     {
05939         // バッファオブジェクトの i 番目のブロックのポインタ
05940         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05941
05942         // 光源の強度の環境光成分を設定する
05943         material->specular = specular;
05944     }
05945     unmap();
05946 }
05947
05948 //
05949 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05950 //
05951 //     shininess 輝き係数

```

```

05952 // first 値を設定する材質データの最初の番号
05953 // count 値を設定する材質データの数
05954 //
05955 void gg::GgSimpleShader::MaterialBuffer::loadShininess(GLfloat shininess,
05956 GLint first, GLsizei count) const
05957 {
05958     // データを格納するバッファオブジェクトの先頭のポインタ
05959     const auto start{ static_cast<char*>(map(first, count)) };
05960     for (GLsizei i = 0; i < count; ++i)
05961     {
05962         // バッファオブジェクトの i 番目のブロックのポインタ
05963         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05964         material->shininess = shininess;
05965     }
05966     unmap();
05967 }
05968
05969 //
05970 // 三角形に単純な陰影付けを行うシェーダが参照する材質データ：輝き係数を設定する
05971 //
05972 // shininess 輝き係数
05973 // first 値を設定する材質データの最初の番号
05974 // count 値を設定する材質データの数
05975 //
05976 void gg::GgSimpleShader::MaterialBuffer::loadShininess(const GLfloat* shininess,
05977 GLint first, GLsizei count) const
05978 {
05979     // データを格納するバッファオブジェクトの先頭のポインタ
05980     const auto start{ static_cast<char*>(map(first, count)) };
05981     for (GLsizei i = 0; i < count; ++i)
05982     {
05983         // バッファオブジェクトの i 番目のブロックのポインタ
05984         const auto material{ reinterpret_cast<Material*>(start + getStride() * i) };
05985         material->shininess = shininess[i];
05986     }
05987     unmap();
05988 }
05989
05990 //
05991 // 三角形に単純な陰影付けを行うシェーダ：シェーダのソースファイルの読み込み
05992 //
05993 bool gg::GgSimpleShader::load(const std::string& vert, const std::string& frag, const std::string&
geom,
05994 GLint nvarying, const char* const* varyings)
05995 {
05996     if (!GgPointShader::load(vert, frag, geom, nvarying, varyings)) return false;
05997
05998     mnLoc = glGetUniformLocation(get(), "mn");
05999     const auto lightIndex{ glGetUniformLocation(get(), "Light") };
06000     glUniformBlockBinding(get(), lightIndex, LightBindingPoint);
06001     const auto materialIndex{ glGetUniformLocation(get(), "Material") };
06002     glUniformBlockBinding(get(), materialIndex, MaterialBindingPoint);
06003
06004     return true;
06005 }
06006
06007 //
06008 // Wavefront OBJ 形式のデータ：コンストラクタ
06009 //
06010 gg::GgSimpleObj::GgSimpleObj(const std::string& name, bool normalize)
06011 {
06012     // 作業用のメモリ
06013     std::vector<GgSimpleShader::Material> mat;
06014     std::vector<GgVertex> vert;
06015     std::vector<GLuint> face;
06016
06017     // グループのデータのメモリを確保する
06018     group = std::make_shared<std::vector<std::array<GLuint, 3>>>();
06019
06020     // ファイルを読み込む
06021     if (ggLoadSimpleObj(name, *group, mat, vert, face, normalize))
06022     {
06023         // 頂点バッファオブジェクトを作成する
06024         data = std::make_shared<GgElements>(vert.data(), static_cast<GLsizei>(vert.size()),
06025         face.data(), static_cast<GLsizei>(face.size()), GL_TRIANGLES);
06026
06027         // 描画するオブジェクトを切り替えるために頂点配列オブジェクトを閉じておく
06028         glBindVertexArray(0);
06029
06030         // 材質データを設定する
06031         material = std::make_shared<GgSimpleShader::MaterialBuffer>(mat.data(),
06032         static_cast<GLsizei>(mat.size()));
06033     }
06034 }
06035
06036 //
06037 // Wavefront OBJ 形式のデータ：図形の描画

```



```

06037 //
06038 void gg::GgSimpleObj::draw(GLint first, GLsizei count) const
06039 {
06040     // 保持しているグループの数
06041     const auto ng{ static_cast<GLsizei>(group->size()) };
06042
06043     // 描画する最後のグループの次
06044     auto last{ count <= 0 ? ng : first + count };
06045     if (last > ng) last = ng;
06046
06047     for (GLsizei i = first; i < last; ++i)
06048     {
06049         // グループのデータ
06050         const auto& g{ (*group)[i] };
06051
06052         // 材質を設定する
06053         material->select(g[2]);
06054
06055         // 図形を描画する
06056         data->draw(g[0], g[1]);
06057     }
06058 }

```

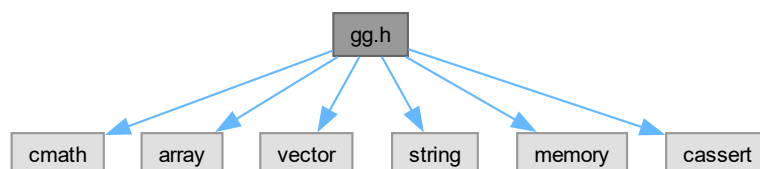
8.3 gg.h ファイル

```

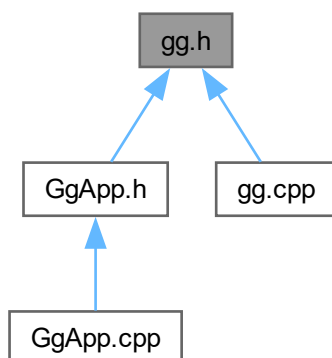
#include <cmath>
#include <array>
#include <vector>
#include <string>
#include <memory>
#include <cassert>

```

gg.h の依存先関係図:



被依存関係図:



クラス

- class `gg::GgVector`
- class `gg::GgMatrix`
- class `gg::GgQuaternion`
- class `gg::GgTrackball`
- class `gg::GgTexture`
- class `gg::GgColorTexture`
- class `gg::GgNormalTexture`
- class `gg::GgBuffer< T >`
- class `gg::GgUniformBuffer< T >`
- class `gg::GgVertexArray`
- class `gg::GgShape`
- class `gg::GgPoints`
- struct `gg::GgVertex`
- class `gg::GgTriangles`
- class `gg::GgElements`
- class `gg::GgShader`
- class `gg::GgPointShader`
- class `gg::GgSimpleShader`
- struct `gg::GgSimpleShader::Light`
- class `gg::GgSimpleShader::LightBuffer`
- struct `gg::GgSimpleShader::Material`
- class `gg::GgSimpleShader::MaterialBuffer`
- class `gg::GgSimpleObj`

名前空間

- namespace `gg`

マクロ定義

- `#define ggError()`
- `#define ggFBOError()`

型定義

- `using pathString = std::string`
- `using pathChar = char`

列挙型

- `enum gg::BindingPoints { gg::LightBindingPoint = 0 , gg::MaterialBindingPoint }`

関数

- `pathString Utf8ToTChar (const std::string &string)`
- `std::string TCharToUtf8 (const pathString &cstring)`
- `void gg::ggInit ()`
- `void gg::ggError (const std::string &name="", unsigned int line=0)`
- `void gg::ggFBOError (const std::string &name="", unsigned int line=0)`
- `void gg::ggCross (GLfloat *c, const GLfloat *a, const GLfloat *b)`
- `GLfloat gg::ggDot3 (const GLfloat *a, const GLfloat *b)`
- `GLfloat gg::ggLength3 (const GLfloat *a)`
- `GLfloat gg::ggDistance3 (const GLfloat *a, const GLfloat *b)`
- `void gg::ggNormalize3 (const GLfloat *a, GLfloat *b)`
- `void gg::ggNormalize3 (GLfloat *a)`
- `GLfloat gg::ggDot4 (const GLfloat *a, const GLfloat *b)`
- `GLfloat gg::ggLength4 (const GLfloat *a)`
- `GLfloat gg::ggDistance4 (const GLfloat *a, const GLfloat *b)`
- `void gg::ggNormalize4 (const GLfloat *a, GLfloat *b)`
- `void gg::ggNormalize4 (GLfloat *a)`
- `const GgVector & gg::operator+ (const GgVector &v)`
- `GgVector gg::operator+ (GLfloat a, const GgVector &b)`
- `const GgVector gg::operator- (const GgVector &v)`
- `GgVector gg::operator- (GLfloat a, const GgVector &b)`
- `GgVector gg::operator* (GLfloat a, const GgVector &b)`
- `GgVector gg::operator/ (GLfloat a, const GgVector &b)`
- `GgVector gg::ggCross (const GgVector &a, const GgVector &b)`
- `GLfloat gg::ggDot3 (const GgVector &a, const GgVector &b)`
- `GLfloat gg::ggLength3 (const GgVector &a)`
- `GLfloat gg::ggDistance3 (const GgVector &a, const GgVector &b)`
- `GgVector gg::ggNormalize3 (const GgVector &a)`
- `void gg::ggNormalize3 (GgVector *a)`
- `GLfloat gg::ggDot4 (const GgVector &a, const GgVector &b)`
- `GLfloat gg::ggLength4 (const GgVector &a)`
- `GLfloat gg::ggDistance4 (const GgVector &a, const GgVector &b)`
- `GgVector gg::ggNormalize4 (const GgVector &a)`
- `void gg::ggNormalize4 (GgVector *a)`
- `GgMatrix gg::ggIdentity ()`
- `GgMatrix gg::ggTranslate (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)`
- `GgMatrix gg::ggTranslate (const GLfloat *t)`

- `GgMatrix gg::ggTranslate` (const `GgVector` &t)
- `GgMatrix gg::ggScale` (GLfloat x, GLfloat y, GLfloat z, GLfloat w=1.0f)
- `GgMatrix gg::ggScale` (const GLfloat *s)
- `GgMatrix gg::ggScale` (const `GgVector` &s)
- `GgMatrix gg::ggRotateX` (GLfloat a)
- `GgMatrix gg::ggRotateY` (GLfloat a)
- `GgMatrix gg::ggRotateZ` (GLfloat a)
- `GgMatrix gg::ggRotate` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgMatrix gg::ggRotate` (const GLfloat *r, GLfloat a)
- `GgMatrix gg::ggRotate` (const `GgVector` &r, GLfloat a)
- `GgMatrix gg::ggRotate` (const GLfloat *r)
- `GgMatrix gg::ggRotate` (const `GgVector` &r)
- `GgMatrix gg::ggLookat` (GLfloat ex, GLfloat ey, GLfloat ez, GLfloat tx, GLfloat ty, GLfloat tz, GLfloat ux, GLfloat uy, GLfloat uz)
- `GgMatrix gg::ggLookat` (const GLfloat *e, const GLfloat *t, const GLfloat *u)
- `GgMatrix gg::ggLookat` (const `GgVector` &e, const `GgVector` &t, const `GgVector` &u)
- `GgMatrix gg::ggOrthogonal` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggFrustum` (GLfloat left, GLfloat right, GLfloat bottom, GLfloat top, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggPerspective` (GLfloat fovy, GLfloat aspect, GLfloat zNear, GLfloat zFar)
- `GgMatrix gg::ggTranspose` (const `GgMatrix` &m)
- `GgMatrix gg::ggInvert` (const `GgMatrix` &m)
- `GgMatrix gg::ggNormal` (const `GgMatrix` &m)
- `GgQuaternion gg::ggIdentityQuaternion` ()
- `GgQuaternion gg::ggMatrixQuaternion` (const GLfloat *a)
- `GgQuaternion gg::ggMatrixQuaternion` (const `GgMatrix` &m)
- `GgMatrix gg::ggQuaternionMatrix` (const `GgQuaternion` &q)
- `GgMatrix gg::ggQuaternionTransposeMatrix` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggRotateQuaternion` (GLfloat x, GLfloat y, GLfloat z, GLfloat a)
- `GgQuaternion gg::ggRotateQuaternion` (const GLfloat *v, GLfloat a)
- `GgQuaternion gg::ggRotateQuaternion` (const GLfloat *v)
- `GgQuaternion gg::ggEulerQuaternion` (GLfloat heading, GLfloat pitch, GLfloat roll)
- `GgQuaternion gg::ggEulerQuaternion` (const GLfloat *e)
- `GgQuaternion gg::ggEulerQuaternion` (const `GgVector` &e)
- `GgQuaternion gg::ggSlerp` (const GLfloat *a, const GLfloat *b, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const `GgQuaternion` &r, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const `GgQuaternion` &q, const GLfloat *a, GLfloat t)
- `GgQuaternion gg::ggSlerp` (const GLfloat *a, const `GgQuaternion` &q, GLfloat t)
- GLfloat `gg::ggNorm` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggNormalize` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggConjugate` (const `GgQuaternion` &q)
- `GgQuaternion gg::ggInvert` (const `GgQuaternion` &q)
- bool `gg::ggSaveTga` (const std::string &name, const void *buffer, unsigned int width, unsigned int height, unsigned int depth)
- bool `gg::ggSaveColor` (const std::string &name)
- bool `gg::ggSaveDepth` (const std::string &name)
- bool `gg::ggReadImage` (const std::string &name, std::vector< GLubyte > &image, GLsizei *pWidth, GLsizei *pHeight, GLenum *pFormat)
- GLuint `gg::ggLoadTexture` (const GLvoid *image, GLsizei width, GLsizei height, GLenum format=GL_RGBA, GLenum type=GL_UNSIGNED_BYTE, GLenum internal=GL_RGB, GLenum wrap=GL_CLAMP_TO_EDGE, bool swizzle=false)
- GLuint `gg::ggLoadImage` (const std::string &name, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA, GLenum wrap=GL_CLAMP_TO_EDGE)
- void `gg::ggCreateNormalMap` (const GLubyte *hmap, GLsizei width, GLsizei height, GLenum format, GLfloat nz, GLenum internal, std::vector< `GgVector` > &nmap)

- GLuint [gg::ggLoadHeight](#) (const std::string &name, GLfloat nz, GLsizei *pWidth=nullptr, GLsizei *pHeight=nullptr, GLenum internal=GL_RGBA)
- GLuint [gg::ggCreateShader](#) (const std::string &vsrsrc, const std::string &fsrsrc="", const std::string &gsrsrc="", GLint nvarying=0, const char *const *varyings=nullptr, const std::string &vtext="vertex shader", const std::string &ftext="fragment shader", const std::string >ext="geometry shader")
- GLuint [gg::ggLoadShader](#) (const std::string &vert, const std::string &frag="", const std::string &geom="", GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggLoadShader](#) (const std::array< std::string, 3 > &files, GLint nvarying=0, const char *const *varyings=nullptr)
- GLuint [gg::ggCreateComputeShader](#) (const std::string &csrsrc, const std::string &ctext="compute shader")
- GLuint [gg::ggLoadComputeShader](#) (const std::string &comp)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsCube](#) (GLsizei countv, GLfloat length=1.0f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgPoints](#) > [gg::ggPointsSphere](#) (GLsizei countv, GLfloat radius=0.5f, GLfloat cx=0.0f, GLfloat cy=0.0f, GLfloat cz=0.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggRectangle](#) (GLfloat width=1.0f, GLfloat height=1.0f)
- std::shared_ptr< [GgTriangles](#) > [gg::ggEllipse](#) (GLfloat width=1.0f, GLfloat height=1.0f, GLuint slices=16)
- std::shared_ptr< [GgTriangles](#) > [gg::ggArraysObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsObj](#) (const std::string &name, bool normalize=false)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsMesh](#) (GLuint slices, GLuint stacks, const GLfloat(*pos)[3], const GLfloat(*norm)[3]=nullptr)
- std::shared_ptr< [GgElements](#) > [gg::ggElementsSphere](#) (GLfloat radius=1.0f, int slices=16, int stacks=8)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, bool normalize=false)
- bool [gg::ggLoadSimpleObj](#) (const std::string &name, std::vector< std::array< GLuint, 3 > > &group, std::vector< [GgSimpleShader::Material](#) > &material, std::vector< [GgVertex](#) > &vert, std::vector< GLuint > &face, bool normalize=false)

変数

- GLint [gg::ggBufferAlignment](#)
使用している GPU のバッファアライメント。

8.3.1 詳解

ゲームグラフィックス特論の宿題用補助プログラム GLFW3 版の宣言。

著者

Kohe Tokoi

日付

July 17, 2025

[gg.h](#) に定義があります。

8.3.2 マクロ定義詳解

8.3.2.1 ggError

```
#define ggError()
```

OpenGL のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で OpenGL のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

gg.h の 1398 行目に定義があります。

8.3.2.2 ggFBOError

```
#define ggFBOError()
```

FBO のエラーの発生を検知したときにソースファイル名と行番号を表示する。

覚え書き

このマクロを置いた場所（より前）で FBO のエラーが発生していた時に、このマクロを置いたソースファイル名と行番号を出力する。リリースビルド時には無視される。

gg.h の 1425 行目に定義があります。

8.3.3 型定義詳解

8.3.3.1 pathChar

```
using pathChar = char
```

gg.h の 78 行目に定義があります。

8.3.3.2 pathString

```
using pathString = std::string
```

gg.h の 77 行目に定義があります。

8.3.4 関数詳解

8.3.4.1 TCharToUtf8()

```
std::string TCharToUtf8 (  
    const pathString & cstring) [inline]
```

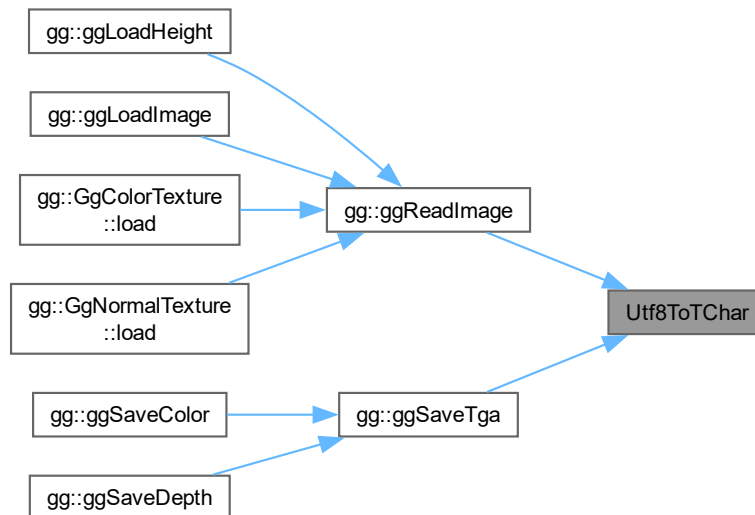
gg.h の 80 行目に定義があります。

8.3.4.2 Utf8ToTChar()

```
pathString Utf8ToTChar (
    const std::string & string) [inline]
```

gg.h の 79 行目に定義があります。

被呼び出し関係図:



8.4 gg.h

[詳解]

```

00001 #pragma once
00002
00026
00034
00035 // 標準ライブラリ
00036 #include <cmath>
00037 #include <array>
00038 #include <vector>
00039 #include <string>
00040 #include <memory>
00041 #include <cassert>
00042
00043 // Windows (Visual Studio) のとき
00044 #if defined(_MSC_VER)
00045 // 非推奨の警告を出さない
00046 # pragma warning(disable:4996)
00047 // 数学ライブラリの定数を使う
00048 # define _USE_MATH_DEFINES
00049 // MIN() / MAX マクロは使わない
00050 # define NOMINMAX
00051 // ファイルパスの文字コード
00052 #include <atlstr.h>
00053 using pathString = CString;
00054 using pathChar = wchar_t;
00055 extern pathString Utf8ToTChar(const std::string& string);
00056 extern std::string TCharToUtf8(const pathString& cstring);
00057 // デバッグビルドかどうか調べる
00058 # if defined(DEBUG)
00059 #   if !defined(DEBUG)
```

```

00060 #         define DEBUG
00061 #     endif
00062 #     define GLFW3_CONFIGURATION "Debug"
00063 # else
00064 #     if !defined(NDEBUG)
00065 #         define NDEBUG
00066 #     endif
00067 #     define GLFW3_CONFIGURATION "Release"
00068 #     endif
00069 // プラットフォームを調べる
00070 #     if defined(_WIN64)
00071 #         define GLFW3_PLATFORM "x64"
00072 #     else
00073 #         define GLFW3_PLATFORM "Win32"
00074 #     endif
00075 #else
00076 // ファイルパスの文字コード
00077 using pathString = std::string;
00078 using pathChar = char;
00079 inline pathString Utf8ToTChar(const std::string& string) { return string; }
00080 inline std::string TCharToUtf8(const pathString& cstring) { return cstring; }
00081 #endif
00082
00084
00085 // macOS で "OpenGL deprecated" の警告を出さない
00086 #if defined(__APPLE__)
00087 #     define GL_SILENCE_DEPRECATED
00088 #endif
00089
00090 // フレームワークに GLFW 3 を使う
00091 #if defined(IMGUI_IMPL_OPENGL_ES2)
00092 #     define GLFW_INCLUDE_ES2
00093 #elif defined(IMGUI_IMPL_OPENGL_ES3)
00094 #     define GLFW_INCLUDE_ES3
00095 #else
00096 #     define GLFW_INCLUDE_GLCOREARB
00097 #endif
00098 #include <GLFW/glfw3.h>
00099 #if !defined(__gl_h__)
00100 #     define __gl_h__
00101 #endif
00102 #if !defined(__GL_H__)
00103 #     define __GL_H__
00104 #endif
00105
00106 // OpenGL 3.2 の API のエントリポイント
00107 #if !defined(GL3_PROTOTYPES) && !defined(GL_GLES_PROTOTYPES)
00108 extern PFNGLACTIVEPROGRAMEXTPROC glActiveProgramEXT;
00109 extern PFNGLACTIVESHADERPROGRAMPROC glActiveShaderProgram;
00110 extern PFNGLACTIVETEXTUREPROC glActiveTexture;
00111 extern PFNGLAPPLYFRAMEBUFFERATTACHMENTCMAAINTELPROC glApplyFramebufferAttachmentCMAAINTEL;
00112 extern PFNGLATTACHSHADERPROC glAttachShader;
00113 extern PFNGLBEGINCONDITIONALRENDERNVPROC glBeginConditionalRenderNV;
00114 extern PFNGLBEGINCONDITIONALRENDERPROC glBeginConditionalRender;
00115 extern PFNGLBEGINPERFMONITORAMDPROC glBeginPerfMonitorAMD;
00116 extern PFNGLBEGINPERFQUERYINTELPROC glBeginPerfQueryINTEL;
00117 extern PFNGLBEGINQUERYINDEXEDPROC glBeginQueryIndexed;
00118 extern PFNGLBEGINQUERYPROC glBeginQuery;
00119 extern PFNGLBEGINTRANSFORMFEEDBACKPROC glBeginTransformFeedback;
00120 extern PFNGLBINDATTRIBLOCATIONPROC glBindAttribLocation;
00121 extern PFNGLBINDBUFFERBASEPROC glBindBufferBase;
00122 extern PFNGLBINDBUFFERPROC glBindBuffer;
00123 extern PFNGLBINDBUFFERRANGEPROC glBindBufferRange;
00124 extern PFNGLBINDBUFFERSBASEPROC glBindBuffersBase;
00125 extern PFNGLBINDBUFFERSRANGEPROC glBindBuffersRange;
00126 extern PFNGLBINDFRAGDATALOCATIONINDEXEDPROC glBindFragDataLocationIndexed;
00127 extern PFNGLBINDFRAGDATALOCATIONPROC glBindFragDataLocation;
00128 extern PFNGLBINDFRAMEBUFFERPROC glBindFramebuffer;
00129 extern PFNGLBINDIMAGETEXTUREPROC glBindImageTexture;
00130 extern PFNGLBINDIMAGETEXTURESPROC glBindImageTextures;
00131 extern PFNGLBINDMULTITEXTUREEXTPROC glBindMultiTextureEXT;
00132 extern PFNGLBINDPROGRAMPIPELINEPROC glBindProgramPipeline;
00133 extern PFNGLBINDRENDERBUFFERPROC glBindRenderbuffer;
00134 extern PFNGLBINDSAMPLERPROC glBindSampler;
00135 extern PFNGLBINDSAMPLERSPROC glBindSamplers;
00136 extern PFNGLBINDTEXTUREPROC glBindTexture;
00137 extern PFNGLBINDTEXTURESPROC glBindTextures;
00138 extern PFNGLBINDTEXTUREUNITPROC glBindTextureUnit;
00139 extern PFNGLBINDTRANSFORMFEEDBACKPROC glBindTransformFeedback;
00140 extern PFNGLBINDVERTEXARRAYPROC glBindVertexArray;
00141 extern PFNGLBINDVERTEXBUFFERPROC glBindVertexBuffer;
00142 extern PFNGLBINDVERTEXBUFFERSPROC glBindVertexBuffers;
00143 extern PFNGLBLENDBARRIERKHRPROC glBlendBarrierKHR;
00144 extern PFNGLBLENDBARRIERNVPROC glBlendBarrierNV;
00145 extern PFNGLBLENDCOLORPROC glBlendColor;
00146 extern PFNGLBLEND EQUATIONIARBPROC glBlendEquationIARB;
00147 extern PFNGLBLEND EQUATIONIPROC glBlendEquationI;

```

```

00148 extern PFNGLBLENDSEPARATEPROC glBlendEquation;
00149 extern PFNGLBLENDSEPARATEIARBPROC glBlendEquationSeparateIARB;
00150 extern PFNGLBLENDSEPARATEIPROC glBlendEquationSeparateI;
00151 extern PFNGLBLENDSEPARATEPROC glBlendEquationSeparate;
00152 extern PFNGLBLENDFUNCARBPROC glBlendFuncIARB;
00153 extern PFNGLBLENDFUNCIPROC glBlendFuncI;
00154 extern PFNGLBLENDFUNCPROC glBlendFunc;
00155 extern PFNGLBLENDFUNCSEPARATEIARBPROC glBlendFuncSeparateIARB;
00156 extern PFNGLBLENDFUNCSEPARATEIPROC glBlendFuncSeparateI;
00157 extern PFNGLBLENDFUNCSEPARATEPROC glBlendFuncSeparate;
00158 extern PFNGLBLENDPARAMETERINVPROC glBlendParameteriNV;
00159 extern PFNGLBLITFRAMEBUFFERPROC glBlitFramebuffer;
00160 extern PFNGLBLITNAMEDFRAMEBUFFERPROC glBlitNamedFramebuffer;
00161 extern PFNGLBUFFERADDRESSRANGENVPROC glBufferAddressRangeNV;
00162 extern PFNGLBUFFERDATAPROC glBufferData;
00163 extern PFNGLBUFFERPAGECOMMITMENTARBPROC glBufferPageCommitmentARB;
00164 extern PFNGLBUFFERSTORAGEPROC glBufferStorage;
00165 extern PFNGLBUFFERSUBDATAPROC glBufferSubData;
00166 extern PFNGLCALLCOMMANDLISTNVPROC glCallCommandListNV;
00167 extern PFNGLCHECKFRAMEBUFFERSTATUSPROC glCheckFramebufferStatus;
00168 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUSEXTPROC glCheckNamedFramebufferStatusEXT;
00169 extern PFNGLCHECKNAMEDFRAMEBUFFERSTATUSPROC glCheckNamedFramebufferStatus;
00170 extern PFNGLCLAMPColorPROC glClampColor;
00171 extern PFNGLCLEARBUFFERDATAPROC glClearBufferData;
00172 extern PFNGLCLEARBUFFERFIPROC glClearBufferfi;
00173 extern PFNGLCLEARBUFFERFVPROC glClearBufferfv;
00174 extern PFNGLCLEARBUFFERFIVPROC glClearBufferfiv;
00175 extern PFNGLCLEARBUFFERSUBDATAPROC glClearBufferSubData;
00176 extern PFNGLCLEARBUFFERUIPROC glClearBufferui;
00177 extern PFNGLCLEARCOLORPROC glClearColor;
00178 extern PFNGLCLEARDEPTHFPROC glClearDepthf;
00179 extern PFNGLCLEARDEPTHPROC glClearDepth;
00180 extern PFNGLCLEARNAMEDBUFFERDATAEXTPROC glClearNamedBufferDataEXT;
00181 extern PFNGLCLEARNAMEDBUFFERDATAPROC glClearNamedBufferData;
00182 extern PFNGLCLEARNAMEDBUFFERSUBDATAEXTPROC glClearNamedBufferSubDataEXT;
00183 extern PFNGLCLEARNAMEDBUFFERSUBDATAPROC glClearNamedBufferSubData;
00184 extern PFNGLCLEARNAMEDFRAMEBUFFERFIPROC glClearNamedFramebufferfi;
00185 extern PFNGLCLEARNAMEDFRAMEBUFFERFVPROC glClearNamedFramebufferfv;
00186 extern PFNGLCLEARNAMEDFRAMEBUFFERFIVPROC glClearNamedFramebufferfiv;
00187 extern PFNGLCLEARNAMEDFRAMEBUFFERUIPROC glClearNamedFramebufferui;
00188 extern PFNGLCLEARPROC glClear;
00189 extern PFNGLCLEARSTENCILPROC glClearStencil;
00190 extern PFNGLCLEARTEXIMAGEPROC glClearTexImage;
00191 extern PFNGLCLEARTEXSUBIMAGEPROC glClearTexSubImage;
00192 extern PFNGLCLIENTATTRIBDEFAULTTEXTPROC glClientAttribDefaultEXT;
00193 extern PFNGLCLIENTWAITSYNCPROC glClientWaitSync;
00194 extern PFNGLCLIPCONTROLPROC glClipControl;
00195 extern PFNGLCOLORFORMATNVPROC glColorFormatNV;
00196 extern PFNGLCOLORMASKIPROC glColorMaski;
00197 extern PFNGLCOLORMASKPROC glColorMask;
00198 extern PFNGLCOMMANDLISTSEGMENTSNVPROC glCommandListSegmentsNV;
00199 extern PFNGLCOMPILECOMMANDLISTNVPROC glCompileCommandListNV;
00200 extern PFNGLCOMPILESHADERINCLUDEARBPROC glCompileShaderIncludeARB;
00201 extern PFNGLCOMPILESHADERPROC glCompileShader;
00202 extern PFNGLCOMPRESSEDMULTITEXIMAGE1DTEXTPROC glCompressedMultiTexImage1DTEXT;
00203 extern PFNGLCOMPRESSEDMULTITEXIMAGE2DTEXTPROC glCompressedMultiTexImage2DTEXT;
00204 extern PFNGLCOMPRESSEDMULTITEXIMAGE3DTEXTPROC glCompressedMultiTexImage3DTEXT;
00205 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE1DTEXTPROC glCompressedMultiTexSubImage1DTEXT;
00206 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE2DTEXTPROC glCompressedMultiTexSubImage2DTEXT;
00207 extern PFNGLCOMPRESSEDMULTITEXSUBIMAGE3DTEXTPROC glCompressedMultiTexSubImage3DTEXT;
00208 extern PFNGLCOMPRESSEDTEXTIMAGE1DPROC glCompressedTexImage1D;
00209 extern PFNGLCOMPRESSEDTEXTIMAGE2DPROC glCompressedTexImage2D;
00210 extern PFNGLCOMPRESSEDTEXTIMAGE3DPROC glCompressedTexImage3D;
00211 extern PFNGLCOMPRESSEDTEXTSUBIMAGE1DPROC glCompressedTexSubImage1D;
00212 extern PFNGLCOMPRESSEDTEXTSUBIMAGE2DPROC glCompressedTexSubImage2D;
00213 extern PFNGLCOMPRESSEDTEXTSUBIMAGE3DPROC glCompressedTexSubImage3D;
00214 extern PFNGLCOMPRESSEDTEXTUREIMAGE1DTEXTPROC glCompressedTextureImage1DTEXT;
00215 extern PFNGLCOMPRESSEDTEXTUREIMAGE2DTEXTPROC glCompressedTextureImage2DTEXT;
00216 extern PFNGLCOMPRESSEDTEXTUREIMAGE3DTEXTPROC glCompressedTextureImage3DTEXT;
00217 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DTEXTPROC glCompressedTextureSubImage1DTEXT;
00218 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE1DPROC glCompressedTextureSubImage1D;
00219 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DTEXTPROC glCompressedTextureSubImage2DTEXT;
00220 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE2DPROC glCompressedTextureSubImage2D;
00221 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DTEXTPROC glCompressedTextureSubImage3DTEXT;
00222 extern PFNGLCOMPRESSEDTEXTURESUBIMAGE3DPROC glCompressedTextureSubImage3D;
00223 extern PFNGLCONSERVATIVERASTERPARAMETERFNVPROC glConservativeRasterParameterfNV;
00224 extern PFNGLCONSERVATIVERASTERPARAMETERINVPROC glConservativeRasterParameteriNV;
00225 extern PFNGLCOPYBUFFERSUBDATAPROC glCopyBufferSubData;
00226 extern PFNGLCOPYIMAGESUBDATAPROC glCopyImageSubData;
00227 extern PFNGLCOPYMULTITEXIMAGE1DTEXTPROC glCopyMultiTexImage1DTEXT;
00228 extern PFNGLCOPYMULTITEXIMAGE2DTEXTPROC glCopyMultiTexImage2DTEXT;
00229 extern PFNGLCOPYMULTITEXSUBIMAGE1DTEXTPROC glCopyMultiTexSubImage1DTEXT;
00230 extern PFNGLCOPYMULTITEXSUBIMAGE2DTEXTPROC glCopyMultiTexSubImage2DTEXT;
00231 extern PFNGLCOPYMULTITEXSUBIMAGE3DTEXTPROC glCopyMultiTexSubImage3DTEXT;
00232 extern PFNGLCOPYNAMEDBUFFERSUBDATAPROC glCopyNamedBufferSubData;
00233 extern PFNGLCOPYPATHNVPROC glCopyPathNV;
00234 extern PFNGLCOPYTEXTIMAGE1DPROC glCopyTexImage1D;

```



```
00235 extern PFNGLCOPYTEXIMAGE2DPROC glCopyTexImage2D;
00236 extern PFNGLCOPYTEXSUBIMAGE1DPROC glCopyTexSubImage1D;
00237 extern PFNGLCOPYTEXSUBIMAGE2DPROC glCopyTexSubImage2D;
00238 extern PFNGLCOPYTEXSUBIMAGE3DPROC glCopyTexSubImage3D;
00239 extern PFNGLCOPYTEXTUREIMAGE1DEXTPROC glCopyTextureImage1DEXT;
00240 extern PFNGLCOPYTEXTUREIMAGE2DEXTPROC glCopyTextureImage2DEXT;
00241 extern PFNGLCOPYTEXTURESUBIMAGE1DEXTPROC glCopyTextureSubImage1DEXT;
00242 extern PFNGLCOPYTEXTURESUBIMAGE1DPROC glCopyTextureSubImage1D;
00243 extern PFNGLCOPYTEXTURESUBIMAGE2DEXTPROC glCopyTextureSubImage2DEXT;
00244 extern PFNGLCOPYTEXTURESUBIMAGE2DPROC glCopyTextureSubImage2D;
00245 extern PFNGLCOPYTEXTURESUBIMAGE3DEXTPROC glCopyTextureSubImage3DEXT;
00246 extern PFNGLCOPYTEXTURESUBIMAGE3DPROC glCopyTextureSubImage3D;
00247 extern PFNGLCOVERAGEMODULATIONNVPROC glCoverageModulationNV;
00248 extern PFNGLCOVERAGEMODULATIONTABLENVPROC glCoverageModulationTableNV;
00249 extern PFNGLCOVERFILLPATHINSTANCEDNVPROC glCoverFillPathInstancedNV;
00250 extern PFNGLCOVERFILLPATHNVPROC glCoverFillPathNV;
00251 extern PFNGLCOVERSTROKEPATHINSTANCEDNVPROC glCoverStrokePathInstancedNV;
00252 extern PFNGLCOVERSTROKEPATHNVPROC glCoverStrokePathNV;
00253 extern PFNGLCREATEBUFFERSPROC glCreateBuffers;
00254 extern PFNGLCREATECOMMANDLISTSNVPROC glCreateCommandListsNV;
00255 extern PFNGLCREATEFRAMEBUFFERSPROC glCreateFramebuffers;
00256 extern PFNGLCREATEPERFQUERYINTELPROC glCreatePerfQueryINTEL;
00257 extern PFNGLCREATEPROGRAMPIPELINESPROC glCreateProgramPipelines;
00258 extern PFNGLCREATEPROGRAMPROC glCreateProgram;
00259 extern PFNGLCREATEQUERIESPROC glCreateQueries;
00260 extern PFNGLCREATERENDERBUFFERSPROC glCreateRenderbuffers;
00261 extern PFNGLCREATESAMPLERSPROC glCreateSamplers;
00262 extern PFNGLCREATESHADERPROC glCreateShader;
00263 extern PFNGLCREATESHADERPROGRAMEXTPROC glCreateShaderProgramEXT;
00264 extern PFNGLCREATESHADERPROGRAMVPROC glCreateShaderProgramv;
00265 extern PFNGLCREATESTATESNVPROC glCreateStatesNV;
00266 extern PFNGLCREATESYNCFROMCLEVENTARBPROC glCreateSyncFromCLeventARB;
00267 extern PFNGLCREATETEXTURESPROC glCreateTextures;
00268 extern PFNGLCREATETRANSFORMFEEDBACKSPROC glCreateTransformFeedbacks;
00269 extern PFNGLCREATEVERTEXARRAYSPROC glCreateVertexArrays;
00270 extern PFNGLCULLFACEPROC glCullFace;
00271 extern PFNGLDEBUGMESSAGECALLBACKARBPROC glDebugMessageCallbackARB;
00272 extern PFNGLDEBUGMESSAGECALLBACKPROC glDebugMessageCallback;
00273 extern PFNGLDEBUGMESSAGECONTROLARBPROC glDebugMessageControlARB;
00274 extern PFNGLDEBUGMESSAGECONTROLPROC glDebugMessageControl;
00275 extern PFNGLDEBUGMESSAGEINSERTARBPROC glDebugMessageInsertARB;
00276 extern PFNGLDEBUGMESSAGEINSERTPROC glDebugMessageInsert;
00277 extern PFNGLDELETEBUFFERSPROC glDeleteBuffers;
00278 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00279 extern PFNGLDELETEFRAMEBUFFERSPROC glDeleteFramebuffers;
00280 extern PFNGLDELETENAMEDSTRINGARBPROC glDeleteNamedStringARB;
00281 extern PFNGLDELETEPATHSNVPROC glDeletePathsNV;
00282 extern PFNGLDELETEPERFMONITORSAMDPROC glDeletePerfMonitorsAMD;
00283 extern PFNGLDELETEPERFQUERYINTELPROC glDeletePerfQueryINTEL;
00284 extern PFNGLDELETEPROGRAMPIPELINESPROC glDeleteProgramPipelines;
00285 extern PFNGLDELETEPROGRAMPROC glDeleteProgram;
00286 extern PFNGLDELETEQUERIESPROC glDeleteQueries;
00287 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00288 extern PFNGLDELETESAMPLERSPROC glDeleteSamplers;
00289 extern PFNGLDELETESHADERPROC glDeleteShader;
00290 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00291 extern PFNGLDELETESYNCPROC glDeleteSync;
00292 extern PFNGLDELETETEXTURESPROC glDeleteTextures;
00293 extern PFNGLDELETETRANSFORMFEEDBACKSPROC glDeleteTransformFeedbacks;
00294 extern PFNGLDELETEVERTEXARRAYSPROC glDeleteVertexArrays;
00295 extern PFNGLDEPTHFUNCPROC glDepthFunc;
00296 extern PFNGLDEPTHMASKPROC glDepthMask;
00297 extern PFNGLDEPTHRANGEARRAYVPROC glDepthRangeArrayv;
00298 extern PFNGLDEPTHRANGEFPROC glDepthRange;
00299 extern PFNGLDEPTHRANGEINDEXEDPROC glDepthRangeIndexed;
00300 extern PFNGLDEPTHRANGEPROC glDepthRange;
00301 extern PFNGLDETACHSHADERPROC glDetachShader;
00302 extern PFNGLDISABLECLIENTSTATEIEXTPROC glDisableClientStateiEXT;
00303 extern PFNGLDISABLECLIENTSTATEINDEXEDEXTPROC glDisableClientStateIndexedEXT;
00304 extern PFNGLDISABLEINDEXEDEXTPROC glDisableIndexedEXT;
00305 extern PFNGLDISABLEIPROC glDisablei;
00306 extern PFNGLDISABLEPROC glDisable;
00307 extern PFNGLDISABLEVERTEXARRAYATTRIBEXTPROC glDisableVertexArrayAttribEXT;
00308 extern PFNGLDISABLEVERTEXARRAYATTRIBPROC glDisableVertexArrayAttrib;
00309 extern PFNGLDISABLEVERTEXARRAYEXTPROC glDisableVertexArrayEXT;
00310 extern PFNGLDISABLEVERTEXATTRIBARRAYPROC glDisableVertexAttribArray;
00311 extern PFNGLDISPATCHCOMPUTEGROUPSIZEARBPROC glDispatchComputeGroupSizeARB;
00312 extern PFNGLDISPATCHCOMPUTEINDIRECTPROC glDispatchComputeIndirect;
00313 extern PFNGLDISPATCHCOMPUTEPROC glDispatchCompute;
00314 extern PFNGLDRAWARRAYSINDIRECTPROC glDrawArraysIndirect;
00315 extern PFNGLDRAWARRAYSINSTANCEDARBPROC glDrawArraysInstancedARB;
00316 extern PFNGLDRAWARRAYSINSTANCEDBASEINSTANCEPROC glDrawArraysInstancedBaseInstance;
00317 extern PFNGLDRAWARRAYSINSTANCEDEXTPROC glDrawArraysInstancedEXT;
00318 extern PFNGLDRAWARRAYSINSTANCEDPROC glDrawArraysInstanced;
00319 extern PFNGLDRAWARRAYSPROC glDrawArrays;
00320 extern PFNGLDRAWBUFFERPROC glDrawBuffer;
00321 extern PFNGLDRAWBUFFERSPROC glDrawBuffers;
```

```

00322 extern PFNGLDRAWCOMMANDSADDRESSNVPROC glDrawCommandsAddressNV;
00323 extern PFNGLDRAWCOMMANDSNDVPROC glDrawCommandsNV;
00324 extern PFNGLDRAWCOMMANDSSTATESADDRESSNVPROC glDrawCommandsStatesAddressNV;
00325 extern PFNGLDRAWCOMMANDSSTATESNDVPROC glDrawCommandsStatesNV;
00326 extern PFNGLDRAWELEMENTSBASEVERTEXPROC glDrawElementsBaseVertex;
00327 extern PFNGLDRAWELEMENTSINDIRECTPROC glDrawElementsIndirect;
00328 extern PFNGLDRAWELEMENTSINSTANCEDARBPROC glDrawElementsInstancedARB;
00329 extern PFNGLDRAWELEMENTSINSTANCEDBASEINSTANCEPROC glDrawElementsInstancedBaseInstance;
00330 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXBASEINSTANCEPROC
glDrawElementsInstancedBaseVertexBaseInstance;
00331 extern PFNGLDRAWELEMENTSINSTANCEDBASEVERTEXPROC glDrawElementsInstancedBaseVertex;
00332 extern PFNGLDRAWELEMENTSINSTANCEDEXTPROC glDrawElementsInstancedEXT;
00333 extern PFNGLDRAWELEMENTSINSTANCEDPROC glDrawElementsInstanced;
00334 extern PFNGLDRAWELEMENTSPROC glDrawElements;
00335 extern PFNGLDRAWRANGEELEMENTSBASEVERTEXPROC glDrawRangeElementsBaseVertex;
00336 extern PFNGLDRAWRANGEELEMENTSPROC glDrawRangeElements;
00337 extern PFNGLDRAWTRANSFORMFEEDBACKINSTANCEDPROC glDrawTransformFeedbackInstanced;
00338 extern PFNGLDRAWTRANSFORMFEEDBACKPROC glDrawTransformFeedback;
00339 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMINSTANCEDPROC glDrawTransformFeedbackStreamInstanced;
00340 extern PFNGLDRAWTRANSFORMFEEDBACKSTREAMPROC glDrawTransformFeedbackStream;
00341 extern PFNGLDRAWVKIMAGENVPROC glDrawVkImageNV;
00342 extern PFNGLEDGEFLAGFORMATNVPROC glEdgeFlagFormatNV;
00343 extern PFNGLENABLECLIENTSTATEIEXTPROC glEnableClientStateiEXT;
00344 extern PFNGLENABLECLIENTSTATEINDEXEDEXTPROC glEnableClientStateIndexedEXT;
00345 extern PFNGLENABLEINDEXEDEXTPROC glEnableIndexedEXT;
00346 extern PFNGLENABLEIPROC glEnablei;
00347 extern PFNGLENABLEPROC glEnable;
00348 extern PFNGLENABLEVERTEXARRAYATTRIBEXTPROC glEnableVertexArrayAttribEXT;
00349 extern PFNGLENABLEVERTEXARRAYATTRIBPROC glEnableVertexArrayAttrib;
00350 extern PFNGLENABLEVERTEXARRAYEXTPROC glEnableVertexArrayEXT;
00351 extern PFNGLENABLEVERTEXATTRIBARRAYPROC glEnableVertexAttribArray;
00352 extern PFNGLENDCONDITIONALRENDERNDVPROC glEndConditionalRenderNV;
00353 extern PFNGLENDCONDITIONALRENDERPROC glEndConditionalRender;
00354 extern PFNGLENDPERFMONITORAMDPROC glEndPerfMonitorAMD;
00355 extern PFNGLENDPERFQUERYINTELPROC glEndPerfQueryINTEL;
00356 extern PFNGLENDQUERYINDEXEDPROC glEndQueryIndexed;
00357 extern PFNGLENDQUERYPROC glEndQuery;
00358 extern PFNGLENDTRANSFORMFEEDBACKPROC glEndTransformFeedback;
00359 extern PFNGLEVALUATEDEPTHVALUESARBPROC glEvaluateDepthValuesARB;
00360 extern PFNGLFENCESYNCPROC glFenceSync;
00361 extern PFNGLFINISHPROC glFinish;
00362 extern PFNGLFLUSHMAPPEDBUFFERRANGEPROC glFlushMappedBufferRange;
00363 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEEXTPROC glFlushMappedNamedBufferRangeEXT;
00364 extern PFNGLFLUSHMAPPEDNAMEDBUFFERRANGEPROC glFlushMappedNamedBufferRange;
00365 extern PFNGLFLUSHPROC glFlush;
00366 extern PFNGLFOGCOORDFORMATNVPROC glFogCoordFormatNV;
00367 extern PFNGLFRAGMENTCOVERAGECOLORNVPROC glFragmentCoverageColorNV;
00368 extern PFNGLFRAMEBUFFERDRAWBUFFEREXTPROC glFramebufferDrawBufferEXT;
00369 extern PFNGLFRAMEBUFFERDRAWBUFFERSEXTPROC glFramebufferDrawBuffersEXT;
00370 extern PFNGLFRAMEBUFFERPARAMETERIPROC glFramebufferParameteri;
00371 extern PFNGLFRAMEBUFFERREADBUFFEREXTPROC glFramebufferReadBufferEXT;
00372 extern PFNGLFRAMEBUFFERRENDERBUFFERPROC glFramebufferRenderbuffer;
00373 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glFramebufferSampleLocationsfvARB;
00374 extern PFNGLFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glFramebufferSampleLocationsfvNV;
00375 extern PFNGLFRAMEBUFFERTEXTURE1DPROC glFramebufferTexture1D;
00376 extern PFNGLFRAMEBUFFERTEXTURE2DPROC glFramebufferTexture2D;
00377 extern PFNGLFRAMEBUFFERTEXTURE3DPROC glFramebufferTexture3D;
00378 extern PFNGLFRAMEBUFFERTEXTUREARBPROC glFramebufferTextureARB;
00379 extern PFNGLFRAMEBUFFERTEXTUREFACEARBPROC glFramebufferTextureFaceARB;
00380 extern PFNGLFRAMEBUFFERTEXTURELAYERARBPROC glFramebufferTextureLayerARB;
00381 extern PFNGLFRAMEBUFFERTEXTURELAYERPROC glFramebufferTextureLayer;
00382 extern PFNGLFRAMEBUFFERTEXTUREMULTIVIEWOVRPROC glFramebufferTextureMultiviewOVR;
00383 extern PFNGLFRAMEBUFFERTEXTUREPROC glFramebufferTexture;
00384 extern PFNGLFRONTFACEPROC glFrontFace;
00385 extern PFNGLGENBUFFERSPROC glGenBuffers;
00386 extern PFNGLGENERATEMIPMAPPROC glGenMipmap;
00387 extern PFNGLGENERATEMULTITEXMIPMAPEXTPROC glGenMultiTexMipmapEXT;
00388 extern PFNGLGENERATETEXTUREMIPMAPEXTPROC glGenTextureMipmapEXT;
00389 extern PFNGLGENERATETEXTUREMIPMAPPROC glGenTextureMipmap;
00390 extern PFNGLGENFRAMEBUFFERSPROC glGenFramebuffers;
00391 extern PFNGLGENPATHSNDVPROC glGenPathsNV;
00392 extern PFNGLGENPERFMONITORSAMDPROC glGenPerfMonitorsAMD;
00393 extern PFNGLGENPROGRAMPIPELINESPROC glGenProgramPipelines;
00394 extern PFNGLGENQUERIESPROC glGenQueries;
00395 extern PFNGLGENRENDERBUFFERSPROC glGenRenderbuffers;
00396 extern PFNGLGENSAMPLERSPROC glGenSamplers;
00397 extern PFNGLGENTEXTURESPROC glGenTextures;
00398 extern PFNGLGENTRANSFORMFEEDBACKSPROC glGenTransformFeedbacks;
00399 extern PFNGLGENVERTEXARRAYSPROC glGenVertexArrays;
00400 extern PFNGLGETACTIVEATOMICCOUNTERBUFFERIVPROC glGetActiveAtomicCounterBufferiv;
00401 extern PFNGLGETACTIVEATTRIBPROC glGetActiveAttrib;
00402 extern PFNGLGETACTIVESUBROUTINENAMEPROC glGetActiveSubroutineName;
00403 extern PFNGLGETACTIVESUBROUTINEUNIFORMIVPROC glGetActiveSubroutineUniformiv;
00404 extern PFNGLGETACTIVESUBROUTINEUNIFORMNAMEPROC glGetActiveSubroutineUniformName;
00405 extern PFNGLGETACTIVEUNIFORMBLOCKIVPROC glGetActiveUniformBlockiv;
00406 extern PFNGLGETACTIVEUNIFORMBLOCKNAMEPROC glGetActiveUniformBlockName;
00407 extern PFNGLGETACTIVEUNIFORMNAMEPROC glGetActiveUniformName;

```

```
00408 extern PFNGLGETACTIVEUNIFORMPROC glGetActiveUniform;
00409 extern PFNGLGETACTIVEUNIFORMSIVPROC glGetActiveUniformsiv;
00410 extern PFNGLGETATTACHEDSHADERSPROC glGetAttachedShaders;
00411 extern PFNGLGETATTRIBLOCATIONPROC glGetAttribLocation;
00412 extern PFNGLGETBOOLEANINDEXEDVEXTPROC glGetBooleanIndexedvEXT;
00413 extern PFNGLGETBOOLEANI_VPROC glGetBooleani_v;
00414 extern PFNGLGETBOOLEANVPROC glGetBooleanv;
00415 extern PFNGLGETBUFFERPARAMETERI64VPROC glGetBufferParameteri64v;
00416 extern PFNGLGETBUFFERPARAMETERIVPROC glGetBufferParameteriv;
00417 extern PFNGLGETBUFFERPARAMETERUI64VNVPROC glGetBufferParameterui64vNV;
00418 extern PFNGLGETBUFFERPOINTERVPROC glGetBufferPointerv;
00419 extern PFNGLGETBUFFERSUBDATAPROC glGetBufferSubData;
00420 extern PFNGLGETCOMMANDHEADERNVPROC glGetCommandHeaderNV;
00421 extern PFNGLGETCOMPRESSEDMULTITEXIMAGEEXTPROC glGetCompressedMultiTexImageEXT;
00422 extern PFNGLGETCOMPRESSEDTEXIMAGEPROC glGetCompressedTexImage;
00423 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEEXTPROC glGetCompressedTextureImageEXT;
00424 extern PFNGLGETCOMPRESSEDTEXTUREIMAGEPROC glGetCompressedTextureImage;
00425 extern PFNGLGETCOMPRESSEDTEXTURESUBIMAGEPROC glGetCompressedTextureSubImage;
00426 extern PFNGLGETCOVERAGEMODULATIONTABLENVPROC glGetCoverageModulationTableNV;
00427 extern PFNGLGETDEBUGMESSAGELOGARBPROC glGetDebugMessageLogARB;
00428 extern PFNGLGETDEBUGMESSAGELOGPROC glGetDebugMessageLog;
00429 extern PFNGLGETDOUBLEINDEXEDVEXTPROC glGetDoubleIndexedvEXT;
00430 extern PFNGLGETDOUBLEI_VEXTPROC glGetDoublei_vEXT;
00431 extern PFNGLGETDOUBLEI_VPROC glGetDoublei_v;
00432 extern PFNGLGETDOUBLEVPROC glGetDoublev;
00433 extern PFNGLGETERRORPROC glGetError;
00434 extern PFNGLGETFIRSTPERFQUERYIDINTELPROC glGetFirstPerfQueryIdINTEL;
00435 extern PFNGLGETFLOATINDEXEDVEXTPROC glGetFloatIndexedvEXT;
00436 extern PFNGLGETFLOATI_VEXTPROC glGetFloati_vEXT;
00437 extern PFNGLGETFLOATI_VPROC glGetFloati_v;
00438 extern PFNGLGETFLOATVPROC glGetFloatv;
00439 extern PFNGLGETFRAGDATAINDEXPROC glGetFragDataIndex;
00440 extern PFNGLGETFRAGDATALOCATIONPROC glGetFragDataLocation;
00441 extern PFNGLGETFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetFramebufferAttachmentParameteriv;
00442 extern PFNGLGETFRAMEBUFFERPARAMETERIVEXTPROC glGetFramebufferParameterivEXT;
00443 extern PFNGLGETFRAMEBUFFERPARAMETERIVPROC glGetFramebufferParameteriv;
00444 extern PFNGLGETGRAPHICSRESETSTATUSARBPROC glGetGraphicsResetStatusARB;
00445 extern PFNGLGETGRAPHICSRESETSTATUSPROC glGetGraphicsResetStatus;
00446 extern PFNGLGETIMAGEHANDLEARBPROC glGetImageHandleARB;
00447 extern PFNGLGETIMAGEHANDLENVPROC glGetImageHandleNV;
00448 extern PFNGLGETINTEGER64I_VPROC glGetInteger64i_v;
00449 extern PFNGLGETINTEGER64VPROC glGetInteger64v;
00450 extern PFNGLGETINTEGERINDEXEDVEXTPROC glGetIntegerIndexedvEXT;
00451 extern PFNGLGETINTEGERI_VPROC glGetIntegeri_v;
00452 extern PFNGLGETINTEGERUI64I_VNVPROC glGetIntegerui64i_vNV;
00453 extern PFNGLGETINTEGERUI64VNVPROC glGetIntegerui64vNV;
00454 extern PFNGLGETINTEGERVPROC glGetIntegerv;
00455 extern PFNGLGETINTERNALFORMATI64VPROC glGetInternalformati64v;
00456 extern PFNGLGETINTERNALFORMATIVPROC glGetInternalformativ;
00457 extern PFNGLGETINTERNALFORMATSAMPLEIVNVPROC glGetInternalformatSampleivNV;
00458 extern PFNGLGETMULTISAMPLEFVPROC glGetMultisamplefv;
00459 extern PFNGLGETMULTITEXENVFVEXTPROC glGetMultiTexEnvfvEXT;
00460 extern PFNGLGETMULTITEXENVIVEXTPROC glGetMultiTexEnvivEXT;
00461 extern PFNGLGETMULTITEXGENDVEXTPROC glGetMultiTexGendvEXT;
00462 extern PFNGLGETMULTITEXGENFVEXTPROC glGetMultiTexGenfvEXT;
00463 extern PFNGLGETMULTITEXGENIVEXTPROC glGetMultiTexGenivEXT;
00464 extern PFNGLGETMULTITEXIMAGEEXTPROC glGetMultiTexImageEXT;
00465 extern PFNGLGETMULTITEXLEVELPARAMETERFVEXTPROC glGetMultiTexLevelParameterfvEXT;
00466 extern PFNGLGETMULTITEXLEVELPARAMETERIVEXTPROC glGetMultiTexLevelParameterivEXT;
00467 extern PFNGLGETMULTITEXPARAMETERFVEXTPROC glGetMultiTexParameterfvEXT;
00468 extern PFNGLGETMULTITEXPARAMETERIIVEXTPROC glGetMultiTexParameterIivEXT;
00469 extern PFNGLGETMULTITEXPARAMETERIUIVEXTPROC glGetMultiTexParameterIuivEXT;
00470 extern PFNGLGETMULTITEXPARAMETERIVEXTPROC glGetMultiTexParameterivEXT;
00471 extern PFNGLGETNAMEDBUFFERPARAMETERI64VPROC glGetNamedBufferParameteri64v;
00472 extern PFNGLGETNAMEDBUFFERPARAMETERIVEXTPROC glGetNamedBufferParameterivEXT;
00473 extern PFNGLGETNAMEDBUFFERPARAMETERIVPROC glGetNamedBufferParameteriv;
00474 extern PFNGLGETNAMEDBUFFERPARAMETERUI64VNVPROC glGetNamedBufferParameterui64vNV;
00475 extern PFNGLGETNAMEDBUFFERPOINTERVEXTPROC glGetNamedBufferPointervEXT;
00476 extern PFNGLGETNAMEDBUFFERPOINTERVPROC glGetNamedBufferPointerv;
00477 extern PFNGLGETNAMEDBUFFERSUBDATAEXTPROC glGetNamedBufferSubDataEXT;
00478 extern PFNGLGETNAMEDBUFFERSUBDATAPROC glGetNamedBufferSubData;
00479 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVEXTPROC
    glGetNamedFramebufferAttachmentParameterivEXT;
00480 extern PFNGLGETNAMEDFRAMEBUFFERATTACHMENTPARAMETERIVPROC glGetNamedFramebufferAttachmentParameteriv;
00481 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVEXTPROC glGetNamedFramebufferParameterivEXT;
00482 extern PFNGLGETNAMEDFRAMEBUFFERPARAMETERIVPROC glGetNamedFramebufferParameteriv;
00483 extern PFNGLGETNAMEDPROGRAMIVEXTPROC glGetNamedProgramivEXT;
00484 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERDVEXTPROC glGetNamedProgramLocalParameterdvEXT;
00485 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERFVEXTPROC glGetNamedProgramLocalParameterfvEXT;
00486 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIIVEXTPROC glGetNamedProgramLocalParameterIivEXT;
00487 extern PFNGLGETNAMEDPROGRAMLOCALPARAMETERIUIVEXTPROC glGetNamedProgramLocalParameterIuivEXT;
00488 extern PFNGLGETNAMEDPROGRAMSTRINGEXTPROC glGetNamedProgramStringEXT;
00489 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVEXTPROC glGetNamedRenderbufferParameterivEXT;
00490 extern PFNGLGETNAMEDRENDERBUFFERPARAMETERIVPROC glGetNamedRenderbufferParameteriv;
00491 extern PFNGLGETNAMEDSTRINGARBPROC glGetNamedStringARB;
00492 extern PFNGLGETNAMEDSTRINGIVARBPROC glGetNamedStringivARB;
00493 extern PFNGLGETNCOMPRESSEDTEXIMAGEARBPROC glGetnCompressedTexImageARB;
```

```

00494 extern PFNGLGETNCOMPRESSEDTEXIMAGEPROC glGetnCompressedTexImage;
00495 extern PFNGLGETNEXTPERFQUERYIDINTELPROC glGetNextPerfQueryIdINTEL;
00496 extern PFNGLGETNTEXIMAGEARBPROC glGetnTexImageARB;
00497 extern PFNGLGETNTEXIMAGEPROC glGetnTexImage;
00498 extern PFNGLGETNUNIFORMDVARBPROC glGetnUniformdvarb;
00499 extern PFNGLGETNUNIFORMDVPROC glGetnUniformdvp;
00500 extern PFNGLGETNUNIFORMFVARBPROC glGetnUniformfvarb;
00501 extern PFNGLGETNUNIFORMFVPROC glGetnUniformfv;
00502 extern PFNGLGETNUNIFORMI64VARBPROC glGetnUniformi64varb;
00503 extern PFNGLGETNUNIFORMIVARBPROC glGetnUniformivarb;
00504 extern PFNGLGETNUNIFORMIVPROC glGetnUniformiv;
00505 extern PFNGLGETNUNIFORMUI64VARBPROC glGetnUniformui64varb;
00506 extern PFNGLGETNUNIFORMUIVARBPROC glGetnUniformuivarb;
00507 extern PFNGLGETNUNIFORMUIVPROC glGetnUniformuiv;
00508 extern PFNGLGETOBJECTLABELEXTPROC glGetObjectLabelEXT;
00509 extern PFNGLGETOBJECTLABELPROC glGetObjectLabel;
00510 extern PFNGLGETOBJECTPTRLABELPROC glGetObjectPtrLabel;
00511 extern PFNGLGETPATHCOMMANDSNVPROC glGetPathCommandsNV;
00512 extern PFNGLGETPATHCOORDSNVPROC glGetPathCoordsNV;
00513 extern PFNGLGETPATHDASHARRAYNVPROC glGetPathDashArrayNV;
00514 extern PFNGLGETPATHLENGTHNVPROC glGetPathLengthNV;
00515 extern PFNGLGETPATHMETRICRANGENVPROC glGetPathMetricRangeNV;
00516 extern PFNGLGETPATHMETRICSNVPROC glGetPathMetricsNV;
00517 extern PFNGLGETPATHPARAMETERFVNVPROC glGetPathParameterfvNV;
00518 extern PFNGLGETPATHPARAMETERIVNVPROC glGetPathParameterivNV;
00519 extern PFNGLGETPATHSPACINGNVPROC glGetPathSpacingNV;
00520 extern PFNGLGETPERFCOUNTERINFOINTELPROC glGetPerfCounterInfoINTEL;
00521 extern PFNGLGETPERFMONITORCOUNTERDATAAMDPROC glGetPerfMonitorCounterDataAMD;
00522 extern PFNGLGETPERFMONITORCOUNTERINFOAMDPROC glGetPerfMonitorCounterInfoAMD;
00523 extern PFNGLGETPERFMONITORCOUNTERSAMDPROC glGetPerfMonitorCountersAMD;
00524 extern PFNGLGETPERFMONITORCOUNTERSTRINGAMDPROC glGetPerfMonitorCounterStringAMD;
00525 extern PFNGLGETPERFMONITORGROUPSAMDPROC glGetPerfMonitorGroupsAMD;
00526 extern PFNGLGETPERFMONITORGROUPSTRINGAMDPROC glGetPerfMonitorGroupStringAMD;
00527 extern PFNGLGETPERFQUERYDATAINTELPROC glGetPerfQueryDataINTEL;
00528 extern PFNGLGETPERFQUERYIDBYNAMEINTELPROC glGetPerfQueryIdByNameINTEL;
00529 extern PFNGLGETPERFQUERYINFOINTELPROC glGetPerfQueryInfoINTEL;
00530 extern PFNGLGETPOINTERINDEXEDVEXTPROC glGetPointerIndexedvEXT;
00531 extern PFNGLGETPOINTERI_VEXTPROC glGetPointeri_vEXT;
00532 extern PFNGLGETPOINTERVPROC glGetPointeriv;
00533 extern PFNGLGETPROGRAMBINARYPROC glGetProgramBinary;
00534 extern PFNGLGETPROGRAMINFOLOGPROC glGetProgramInfoLog;
00535 extern PFNGLGETPROGRAMINTERFACEIVPROC glGetProgramInterfaceiv;
00536 extern PFNGLGETPROGRAMIVPROC glGetProgramiv;
00537 extern PFNGLGETPROGRAMPIPELINEINFOLOGPROC glGetProgramPipelineInfoLog;
00538 extern PFNGLGETPROGRAMPIPELINEIVPROC glGetProgramPipelineiv;
00539 extern PFNGLGETPROGRAMRESOURCEFVNVPROC glGetProgramResourcefvNV;
00540 extern PFNGLGETPROGRAMRESOURCEINDEXPROC glGetProgramResourceIndex;
00541 extern PFNGLGETPROGRAMRESOURCEIVPROC glGetProgramResourceiv;
00542 extern PFNGLGETPROGRAMRESOURCELOCATIONINDEXPROC glGetProgramResourceLocationIndex;
00543 extern PFNGLGETPROGRAMRESOURCELOCATIONPROC glGetProgramResourceLocation;
00544 extern PFNGLGETPROGRAMRESOURCENAMEPROC glGetProgramResourceName;
00545 extern PFNGLGETPROGRAMSTAGEIVPROC glGetProgramStageiv;
00546 extern PFNGLGETQUERYBUFFEROBJECTI64VPROC glGetQueryBufferObjecti64v;
00547 extern PFNGLGETQUERYBUFFEROBJECTIVPROC glGetQueryBufferObjectiv;
00548 extern PFNGLGETQUERYBUFFEROBJECTUI64VPROC glGetQueryBufferObjectui64v;
00549 extern PFNGLGETQUERYBUFFEROBJECTUIVPROC glGetQueryBufferObjectuiv;
00550 extern PFNGLGETQUERYINDEXEDIVPROC glGetQueryIndexediv;
00551 extern PFNGLGETQUERYIVPROC glGetQueryiv;
00552 extern PFNGLGETQUERYOBJECTI64VPROC glGetQueryObjecti64v;
00553 extern PFNGLGETQUERYOBJECTIVPROC glGetQueryObjectiv;
00554 extern PFNGLGETQUERYOBJECTUI64VPROC glGetQueryObjectui64v;
00555 extern PFNGLGETQUERYOBJECTUIVPROC glGetQueryObjectuiv;
00556 extern PFNGLGETRENDERBUFFERPARAMETERIVPROC glGetRenderbufferParameteriv;
00557 extern PFNGLGETSAMPLERPARAMETERFVPROC glGetSamplerParameterfv;
00558 extern PFNGLGETSAMPLERPARAMETERIIVPROC glGetSamplerParameteriiv;
00559 extern PFNGLGETSAMPLERPARAMETERIUIVPROC glGetSamplerParameteriui;
00560 extern PFNGLGETSAMPLERPARAMETERIVPROC glGetSamplerParameteriv;
00561 extern PFNGLGETSHADERINFOLOGPROC glGetShaderInfoLog;
00562 extern PFNGLGETSHADERIVPROC glGetShaderiv;
00563 extern PFNGLGETSHADERPRECISIONFORMATPROC glGetShaderPrecisionFormat;
00564 extern PFNGLGETSHADERSOURCEPROC glGetShaderSource;
00565 extern PFNGLGETSTAGEINDEXNVPROC glGetStageIndexNV;
00566 extern PFNGLGETSTRINGIPROC glGetStringi;
00567 extern PFNGLGETSTRINGPROC glGetString;
00568 extern PFNGLGETSUBROUTINEINDEXPROC glGetSubroutineIndex;
00569 extern PFNGLGETSUBROUTINEUNIFORMLOCATIONPROC glGetSubroutineUniformLocation;
00570 extern PFNGLGETSYNCIVPROC glGetSynciv;
00571 extern PFNGLGETTEXIMAGEPROC glGetTexImage;
00572 extern PFNGLGETTEXLEVELPARAMETERFVPROC glGetTexLevelParameterfv;
00573 extern PFNGLGETTEXLEVELPARAMETERIVPROC glGetTexLevelParameteriv;
00574 extern PFNGLGETTEXPARAMETERFVPROC glGetTexParameterfv;
00575 extern PFNGLGETTEXPARAMETERIIVPROC glGetTexParameteriiv;
00576 extern PFNGLGETTEXPARAMETERIUIVPROC glGetTexParameteriui;
00577 extern PFNGLGETTEXPARAMETERIVPROC glGetTexParameteriv;
00578 extern PFNGLGETTEXTUREHANDLEARBPROC glGetTextureHandleARB;
00579 extern PFNGLGETTEXTUREHANDLENVPROC glGetTextureHandleNV;
00580 extern PFNGLGETTEXTUREIMAGEEXTPROC glGetTextureImageEXT;

```



```
00581 extern PFNGLGETTEXTUREIMAGEPROC glGetTextureImage;
00582 extern PFNGLGETTEXTURELEVELPARAMETERFVEXTPROC glGetTextureLevelParameterfvEXT;
00583 extern PFNGLGETTEXTURELEVELPARAMETERFVPROC glGetTextureLevelParameterfv;
00584 extern PFNGLGETTEXTURELEVELPARAMETERIVEXTPROC glGetTextureLevelParameterivEXT;
00585 extern PFNGLGETTEXTURELEVELPARAMETERIVPROC glGetTextureLevelParameteriv;
00586 extern PFNGLGETTEXTUREPARAMETERFVEXTPROC glGetTextureParameterfvEXT;
00587 extern PFNGLGETTEXTUREPARAMETERFVPROC glGetTextureParameterfv;
00588 extern PFNGLGETTEXTUREPARAMETERIIVEXTPROC glGetTextureParameterIivEXT;
00589 extern PFNGLGETTEXTUREPARAMETERIIVPROC glGetTextureParameterIiv;
00590 extern PFNGLGETTEXTUREPARAMETERIUIVEXTPROC glGetTextureParameterIuivEXT;
00591 extern PFNGLGETTEXTUREPARAMETERIUIVPROC glGetTextureParameterIuiv;
00592 extern PFNGLGETTEXTUREPARAMETERIVEXTPROC glGetTextureParameterivEXT;
00593 extern PFNGLGETTEXTUREPARAMETERIVPROC glGetTextureParameteriv;
00594 extern PFNGLGETTEXTURESAMPLERHANDLEARBPROC glGetTextureSamplerHandleARB;
00595 extern PFNGLGETTEXTURESAMPLERHANDLENVPROC glGetTextureSamplerHandleNV;
00596 extern PFNGLGETTEXTURESUBIMAGEPROC glGetTextureSubImage;
00597 extern PFNGLGETTRANSFORMFEEDBACKI64VPROC glGetTransformFeedbacki64v;
00598 extern PFNGLGETTRANSFORMFEEDBACKIIVPROC glGetTransformFeedbackiv;
00599 extern PFNGLGETTRANSFORMFEEDBACKIIVPROC glGetTransformFeedbackiv;
00600 extern PFNGLGETTRANSFORMFEEDBACKVARYINGPROC glGetTransformFeedbackVarying;
00601 extern PFNGLGETUNIFORMBLOCKINDEXPROC glGetUniformBlockIndex;
00602 extern PFNGLGETUNIFORMDVPROC glGetUniformdv;
00603 extern PFNGLGETUNIFORMFVPROC glGetUniformfv;
00604 extern PFNGLGETUNIFORMI64VARBPROC glGetUniformi64vARB;
00605 extern PFNGLGETUNIFORMI64VNVPROC glGetUniformi64vNV;
00606 extern PFNGLGETUNIFORMINDICESPROC glGetUniformIndices;
00607 extern PFNGLGETUNIFORMIVPROC glGetUniformiv;
00608 extern PFNGLGETUNIFORMLOCATIONPROC glGetUniformLocation;
00609 extern PFNGLGETUNIFORMSUBROUTINEUIVPROC glGetUniformSubroutineuiv;
00610 extern PFNGLGETUNIFORMUI64VARBPROC glGetUniformui64vARB;
00611 extern PFNGLGETUNIFORMUI64VNVPROC glGetUniformui64vNV;
00612 extern PFNGLGETUNIFORMUIVPROC glGetUniformuiv;
00613 extern PFNGLGETVERTEXARRAYINDEXED64IVPROC glGetVertexArrayIndexed64iv;
00614 extern PFNGLGETVERTEXARRAYINDEXEDIVPROC glGetVertexArrayIndexediv;
00615 extern PFNGLGETVERTEXARRAYINTEGERI_VEXTPROC glGetVertexArrayIntegeri_vEXT;
00616 extern PFNGLGETVERTEXARRAYINTEGERVEXTPROC glGetVertexArrayIntegerivEXT;
00617 extern PFNGLGETVERTEXARRAYIVPROC glGetVertexArrayiv;
00618 extern PFNGLGETVERTEXARRAYPOINTERI_VEXTPROC glGetVertexArrayPointeri_vEXT;
00619 extern PFNGLGETVERTEXARRAYPOINTERVEXTPROC glGetVertexArrayPointerivEXT;
00620 extern PFNGLGETVERTEXATTRIBDVPROC glGetVertexAttribdv;
00621 extern PFNGLGETVERTEXATTRIBFVPROC glGetVertexAttribfv;
00622 extern PFNGLGETVERTEXATTRIBIIVPROC glGetVertexAttribIiv;
00623 extern PFNGLGETVERTEXATTRIBIUIVPROC glGetVertexAttribIuiv;
00624 extern PFNGLGETVERTEXATTRIBIVPROC glGetVertexAttribiv;
00625 extern PFNGLGETVERTEXATTRIBLDVPROC glGetVertexAttribLdv;
00626 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLi64vNV;
00627 extern PFNGLGETVERTEXATTRIBLUI64VARBPROC glGetVertexAttribLui64vARB;
00628 extern PFNGLGETVERTEXATTRIBLUI64VNVPROC glGetVertexAttribLui64vNV;
00629 extern PFNGLGETVERTEXATTRIBPOINTERVPROC glGetVertexAttribPointeriv;
00630 extern PFNGLGETVKPROCADDRNVPROC glGetVkProcAddrNV;
00631 extern PFNGLHINTPROC glHint;
00632 extern PFNGLINDEXFORMATNVPROC glIndexFormatNV;
00633 extern PFNGLINSERTEVENTMARKEREXTPROC glInsertEventMarkerEXT;
00634 extern PFNGLINTERPOLATEPATHSNVPROC glInterpolatePathsNV;
00635 extern PFNGLINVALIDATEBUFFERDATAPROC glInvalidateBufferData;
00636 extern PFNGLINVALIDATEBUFFERSUBDATAPROC glInvalidateBufferSubData;
00637 extern PFNGLINVALIDATEFRAMEBUFFERPROC glInvalidateFramebuffer;
00638 extern PFNGLINVALIDATENAMEDFRAMEBUFFERDATAPROC glInvalidateNamedFramebufferData;
00639 extern PFNGLINVALIDATENAMEDFRAMEBUFFERSUBDATAPROC glInvalidateNamedFramebufferSubData;
00640 extern PFNGLINVALIDATESUBFRAMEBUFFERPROC glInvalidateSubFramebuffer;
00641 extern PFNGLINVALIDATETEXIMAGEPROC glInvalidateTexImage;
00642 extern PFNGLINVALIDATETEXSUBIMAGEPROC glInvalidateTexSubImage;
00643 extern PFNGLISBUFFERPROC glIsBuffer;
00644 extern PFNGLISBUFFERRESIDENTNVPROC glIsBufferResidentNV;
00645 extern PFNGLISCOMMANDLISTNVPROC glIsCommandListNV;
00646 extern PFNGLISENABLEDINDEXEDEXTPROC glIsEnabledIndexedEXT;
00647 extern PFNGLISENABLEDIPROC glIsEnabledi;
00648 extern PFNGLISENABLEDIPROC glIsEnabled;
00649 extern PFNGLISFRAMEBUFFERPROC glIsFramebuffer;
00650 extern PFNGLISIMAGEHANDLERESIDENTARBPROC glIsImageHandleResidentARB;
00651 extern PFNGLISIMAGEHANDLERESIDENTNVPROC glIsImageHandleResidentNV;
00652 extern PFNGLISNAMEDBUFFERRESIDENTNVPROC glIsNamedBufferResidentNV;
00653 extern PFNGLISNAMEDSTRINGARBPROC glIsNamedStringARB;
00654 extern PFNGLISPATHNVPROC glIsPathNV;
00655 extern PFNGLISPOINTINFILLPATHNVPROC glIsPointInFillPathNV;
00656 extern PFNGLISPOINTINSTROKEPATHNVPROC glIsPointInStrokePathNV;
00657 extern PFNGLISPROGRAMPIPELINEPROC glIsProgramPipeline;
00658 extern PFNGLISPROGRAMPROC glIsProgram;
00659 extern PFNGLISQUERYPROC glIsQuery;
00660 extern PFNGLISRENDERBUFFERPROC glIsRenderbuffer;
00661 extern PFNGLISSAMPLERPROC glIsSampler;
00662 extern PFNGLISSHADERPROC glIsShader;
00663 extern PFNGLISSTATENVPROC glIsStateNV;
00664 extern PFNGLISSYNCPROC glIsSync;
00665 extern PFNGLISTEXTUREHANDLERESIDENTARBPROC glIsTextureHandleResidentARB;
00666 extern PFNGLISTEXTUREHANDLERESIDENTNVPROC glIsTextureHandleResidentNV;
00667 extern PFNGLISTEXTUREPROC glIsTexture;
```

```
00668 extern PFNGLISTRANSFORMFEEDBACKPROC glIsTransformFeedback;
00669 extern PFNGLISVERTEXARRAYPROC glIsVertexArray;
00670 extern PFNGLLABELOBJECTEXTPROC glLabelObjectEXT;
00671 extern PFNGLLINEWIDTHPROC glLineWidth;
00672 extern PFNGLLINKPROGRAMPROC glLinkProgram;
00673 extern PFNGLLISTDRAWCOMMANDSSTATESCLIENTNVPROC glListDrawCommandsStatesClientNV;
00674 extern PFNGLLOGICOPPROC glLogicOp;
00675 extern PFNGLMAKEBUFFERNONRESIDENTNVPROC glMakeBufferNonResidentNV;
00676 extern PFNGLMAKEBUFFERRESIDENTNVPROC glMakeBufferResidentNV;
00677 extern PFNGLMAKEIMAGEHANDLENONRESIDENTARBPROC glMakeImageHandleNonResidentARB;
00678 extern PFNGLMAKEIMAGEHANDLENONRESIDENTNVPROC glMakeImageHandleNonResidentNV;
00679 extern PFNGLMAKEIMAGEHANDLERESIDENTARBPROC glMakeImageHandleResidentARB;
00680 extern PFNGLMAKEIMAGEHANDLERESIDENTNVPROC glMakeImageHandleResidentNV;
00681 extern PFNGLMAKENAMEDBUFFERNONRESIDENTNVPROC glMakeNamedBufferNonResidentNV;
00682 extern PFNGLMAKENAMEDBUFFERRESIDENTNVPROC glMakeNamedBufferResidentNV;
00683 extern PFNGLMAKETEXTUREHANDLENONRESIDENTARBPROC glMakeTextureHandleNonResidentARB;
00684 extern PFNGLMAKETEXTUREHANDLENONRESIDENTNVPROC glMakeTextureHandleNonResidentNV;
00685 extern PFNGLMAKETEXTUREHANDLERESIDENTARBPROC glMakeTextureHandleResidentARB;
00686 extern PFNGLMAKETEXTUREHANDLERESIDENTNVPROC glMakeTextureHandleResidentNV;
00687 extern PFNGLMAPBUFFERPROC glMapBuffer;
00688 extern PFNGLMAPBUFFERRANGEPROC glMapBufferRange;
00689 extern PFNGLMAPNAMEDBUFFEREXTPROC glMapNamedBufferEXT;
00690 extern PFNGLMAPNAMEDBUFFERPROC glMapNamedBuffer;
00691 extern PFNGLMAPNAMEDBUFFERRANGEEXTPROC glMapNamedBufferRangeEXT;
00692 extern PFNGLMAPNAMEDBUFFERRANGEPROC glMapNamedBufferRange;
00693 extern PFNGLMATRIXFRUSTUMEXTPROC glMatrixFrustumEXT;
00694 extern PFNGLMATRIXLOAD3X2FNVPROC glMatrixLoad3x2fNV;
00695 extern PFNGLMATRIXLOAD3X3FNVPROC glMatrixLoad3x3fNV;
00696 extern PFNGLMATRIXLOADDEXTPROC glMatrixLoadEXT;
00697 extern PFNGLMATRIXLOADFEXTPROC glMatrixLoadfEXT;
00698 extern PFNGLMATRIXLOADIDENTITYEXTPROC glMatrixLoadIdentityEXT;
00699 extern PFNGLMATRIXLOADTRANSPOSE3X3FNVPROC glMatrixLoadTranspose3x3fNV;
00700 extern PFNGLMATRIXLOADTRANSPOSEDEXTPROC glMatrixLoadTransposedEXT;
00701 extern PFNGLMATRIXLOADTRANSPOSEFEXTPROC glMatrixLoadTransposefEXT;
00702 extern PFNGLMATRIXMULT3X2FNVPROC glMatrixMult3x2fNV;
00703 extern PFNGLMATRIXMULT3X3FNVPROC glMatrixMult3x3fNV;
00704 extern PFNGLMATRIXMULTDEXTPROC glMatrixMultEXT;
00705 extern PFNGLMATRIXMULTFEXTPROC glMatrixMultfEXT;
00706 extern PFNGLMATRIXMULTTRANSPOSE3X3FNVPROC glMatrixMultTranspose3x3fNV;
00707 extern PFNGLMATRIXMULTTRANSPOSEDEXTPROC glMatrixMultTransposedEXT;
00708 extern PFNGLMATRIXMULTTRANSPOSEFEXTPROC glMatrixMultTransposefEXT;
00709 extern PFNGLMATRIXORTHOEXTPROC glMatrixOrthoEXT;
00710 extern PFNGLMATRIXPOPEXTPROC glMatrixPopEXT;
00711 extern PFNGLMATRIXPUSHEXTPROC glMatrixPushEXT;
00712 extern PFNGLMATRIXROTATEDEXTPROC glMatrixRotatedEXT;
00713 extern PFNGLMATRIXROTATEFEXTPROC glMatrixRotatefEXT;
00714 extern PFNGLMATRIXSCALEDEXTPROC glMatrixScaledEXT;
00715 extern PFNGLMATRIXSCALEFEXTPROC glMatrixScalefEXT;
00716 extern PFNGLMATRIXTRANSLATEDEXTPROC glMatrixTranslatedEXT;
00717 extern PFNGLMATRIXTRANSLATEFEXTPROC glMatrixTranslatefEXT;
00718 extern PFNGLMAXSHADERCOMPILERTHREADSARBPROC glMaxShaderCompilerThreadsARB;
00719 extern PFNGLMEMORYBARRIERBYREGIONPROC glMemoryBarrierByRegion;
00720 extern PFNGLMEMORYBARRIERPROC glMemoryBarrier;
00721 extern PFNGLMINSAMPLESHADINGARBPROC glMinSampleShadingARB;
00722 extern PFNGLMINSAMPLESHADINGPROC glMinSampleShading;
00723 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawArraysIndirectBindlessCountNV;
00724 extern PFNGLMULTIDRAWARRAYSINDIRECTBINDLESSNVPROC glMultiDrawArraysIndirectBindlessNV;
00725 extern PFNGLMULTIDRAWARRAYSINDIRECTCOUNTARBPROC glMultiDrawArraysIndirectCountARB;
00726 extern PFNGLMULTIDRAWARRAYSINDIRECTPROC glMultiDrawArraysIndirect;
00727 extern PFNGLMULTIDRAWARRAYSPROC glMultiDrawArrays;
00728 extern PFNGLMULTIDRAWELEMENTSBASEVERTEXPROC glMultiDrawElementsBaseVertex;
00729 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSCOUNTNVPROC glMultiDrawElementsIndirectBindlessCountNV;
00730 extern PFNGLMULTIDRAWELEMENTSINDIRECTBINDLESSNVPROC glMultiDrawElementsIndirectBindlessNV;
00731 extern PFNGLMULTIDRAWELEMENTSINDIRECTCOUNTARBPROC glMultiDrawElementsIndirectCountARB;
00732 extern PFNGLMULTIDRAWELEMENTSINDIRECTPROC glMultiDrawElementsIndirect;
00733 extern PFNGLMULTIDRAWELEMENTSPROC glMultiDrawElements;
00734 extern PFNGLMULTITEXBUFFEREXTPROC glMultiTexBufferEXT;
00735 extern PFNGLMULTITEXCOORDPOINTINTEREXTPROC glMultiTexCoordPointerEXT;
00736 extern PFNGLMULTITEXENVFEXTPROC glMultiTexEnvfEXT;
00737 extern PFNGLMULTITEXENVFVEXTPROC glMultiTexEnvfvEXT;
00738 extern PFNGLMULTITEXENVIEXTPROC glMultiTexEnviEXT;
00739 extern PFNGLMULTITEXENVIVEXTPROC glMultiTexEnvivEXT;
00740 extern PFNGLMULTITEXGENDEXTPROC glMultiTexGendEXT;
00741 extern PFNGLMULTITEXGENDVEXTPROC glMultiTexGendvEXT;
00742 extern PFNGLMULTITEXGENFEXTPROC glMultiTexGenfEXT;
00743 extern PFNGLMULTITEXGENFVEXTPROC glMultiTexGenfvEXT;
00744 extern PFNGLMULTITEXGENIEXTPROC glMultiTexGeniEXT;
00745 extern PFNGLMULTITEXGENIVEXTPROC glMultiTexGenivEXT;
00746 extern PFNGLMULTITEXIMAGE1DEXTPROC glMultiTexImage1DTEXT;
00747 extern PFNGLMULTITEXIMAGE2DEXTPROC glMultiTexImage2DTEXT;
00748 extern PFNGLMULTITEXIMAGE3DEXTPROC glMultiTexImage3DTEXT;
00749 extern PFNGLMULTITEXPARAMETERFEXTPROC glMultiTexParameterfEXT;
00750 extern PFNGLMULTITEXPARAMETERFVEXTPROC glMultiTexParameterfvEXT;
00751 extern PFNGLMULTITEXPARAMETERIEXTPROC glMultiTexParameteriEXT;
00752 extern PFNGLMULTITEXPARAMETERIIVEXTPROC glMultiTexParameteriivEXT;
00753 extern PFNGLMULTITEXPARAMETERIUIVEXTPROC glMultiTexParameteriuiEXT;
00754 extern PFNGLMULTITEXPARAMETERIVEXTPROC glMultiTexParameterivEXT;
```

```
00755 extern PFNGLMULTITEXRENDERBUFFEREXTPROC glMultiTexRenderbufferEXT;
00756 extern PFNGLMULTITEXSUBIMAGE1DEXTPROC glMultiTexSubImage1EXT;
00757 extern PFNGLMULTITEXSUBIMAGE2DEXTPROC glMultiTexSubImage2EXT;
00758 extern PFNGLMULTITEXSUBIMAGE3DEXTPROC glMultiTexSubImage3EXT;
00759 extern PFNGLNAMEDBUFFERDATAEXTPROC glNamedBufferDataEXT;
00760 extern PFNGLNAMEDBUFFERDATAPROC glNamedBufferData;
00761 extern PFNGLNAMEDBUFFERPAGECOMMITMENTARBPROC glNamedBufferPageCommitmentARB;
00762 extern PFNGLNAMEDBUFFERPAGECOMMITMENTEXTPROC glNamedBufferPageCommitmentEXT;
00763 extern PFNGLNAMEDBUFFERSTORAGEEXTPROC glNamedBufferStorageEXT;
00764 extern PFNGLNAMEDBUFFERSTORAGEPROC glNamedBufferStorage;
00765 extern PFNGLNAMEDBUFFERSUBDATAEXTPROC glNamedBufferSubDataEXT;
00766 extern PFNGLNAMEDBUFFERSUBDATAPROC glNamedBufferSubData;
00767 extern PFNGLNAMEDCOPYBUFFERSUBDATAEXTPROC glNamedCopyBufferSubDataEXT;
00768 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERPROC glNamedFramebufferDrawBuffer;
00769 extern PFNGLNAMEDFRAMEBUFFERDRAWBUFFERSPROC glNamedFramebufferDrawBuffers;
00770 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIEXTPROC glNamedFramebufferParameteriEXT;
00771 extern PFNGLNAMEDFRAMEBUFFERPARAMETERIPROC glNamedFramebufferParameteri;
00772 extern PFNGLNAMEDFRAMEBUFFERREADBUFFERPROC glNamedFramebufferReadBuffer;
00773 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFEREXTPROC glNamedFramebufferRenderbufferEXT;
00774 extern PFNGLNAMEDFRAMEBUFFERRENDERBUFFERPROC glNamedFramebufferRenderbuffer;
00775 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVARBPROC glNamedFramebufferSampleLocationsfvARB;
00776 extern PFNGLNAMEDFRAMEBUFFERSAMPLELOCATIONSFVNVPROC glNamedFramebufferSampleLocationsfvNV;
00777 extern PFNGLNAMEDFRAMEBUFFERTEXTURE1DEXTPROC glNamedFramebufferTexture1EXT;
00778 extern PFNGLNAMEDFRAMEBUFFERTEXTURE2DEXTPROC glNamedFramebufferTexture2EXT;
00779 extern PFNGLNAMEDFRAMEBUFFERTEXTURE3DEXTPROC glNamedFramebufferTexture3EXT;
00780 extern PFNGLNAMEDFRAMEBUFFERTEXTUREEXTPROC glNamedFramebufferTextureEXT;
00781 extern PFNGLNAMEDFRAMEBUFFERTEXTUREFACEEXTPROC glNamedFramebufferTextureFaceEXT;
00782 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYEREXTPROC glNamedFramebufferTextureLayerEXT;
00783 extern PFNGLNAMEDFRAMEBUFFERTEXTURELAYERPROC glNamedFramebufferTextureLayer;
00784 extern PFNGLNAMEDFRAMEBUFFERTEXTUREPROC glNamedFramebufferTexture;
00785 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DEXTPROC glNamedProgramLocalParameter4dEXT;
00786 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4DVEXTPROC glNamedProgramLocalParameter4dvEXT;
00787 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FEXTPROC glNamedProgramLocalParameter4fEXT;
00788 extern PFNGLNAMEDPROGRAMLOCALPARAMETER4FVEXTPROC glNamedProgramLocalParameter4fvEXT;
00789 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IEXTPROC glNamedProgramLocalParameterI4iEXT;
00790 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4IVEXTPROC glNamedProgramLocalParameterI4ivEXT;
00791 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIEXTPROC glNamedProgramLocalParameterI4uiEXT;
00792 extern PFNGLNAMEDPROGRAMLOCALPARAMETERI4UIVEXTPROC glNamedProgramLocalParameterI4uivEXT;
00793 extern PFNGLNAMEDPROGRAMLOCALPARAMETERS4FVEXTPROC glNamedProgramLocalParameters4fvEXT;
00794 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4IEXTPROC glNamedProgramLocalParametersI4iEXT;
00795 extern PFNGLNAMEDPROGRAMLOCALPARAMETERSI4UIVEXTPROC glNamedProgramLocalParametersI4uivEXT;
00796 extern PFNGLNAMEDPROGRAMSTRINGEXTPROC glNamedProgramStringEXT;
00797 extern PFNGLNAMEDRENDERBUFFERSTORAGEEXTPROC glNamedRenderbufferStorageEXT;
00798 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLECOVERAGEEXTPROC
glNamedRenderbufferStorageMultisampleCoverageEXT;
00799 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEEXTPROC glNamedRenderbufferStorageMultisampleEXT;
00800 extern PFNGLNAMEDRENDERBUFFERSTORAGEMULTISAMPLEPROC glNamedRenderbufferStorageMultisample;
00801 extern PFNGLNAMEDRENDERBUFFERSTORAGEPROC glNamedRenderbufferStorage;
00802 extern PFNGLNAMEDSTRINGARBPROC glNamedStringARB;
00803 extern PFNGLNORMALFORMATNVPROC glNormalFormatNV;
00804 extern PFNGLOBJCTLABELPROC glObjectLabel;
00805 extern PFNGLOBJECTPTRLABELPROC glObjectPtrLabel;
00806 extern PFNGLPATCHPARAMETERFVPROC glPatchParameterfv;
00807 extern PFNGLPATCHPARAMETERIPROC glPatchParameteri;
00808 extern PFNGLPATHCOMMANDSNVPROC glPathCommandsNV;
00809 extern PFNGLPATHCOORDSNVPROC glPathCoordsNV;
00810 extern PFNGLPATHCOVERDEPTHFUNCNVPROC glPathCoverDepthFuncNV;
00811 extern PFNGLPATHDASHARRAYNVPROC glPathDashArrayNV;
00812 extern PFNGLPATHGLYPHINDEXARRAYNVPROC glPathGlyphIndexArrayNV;
00813 extern PFNGLPATHGLYPHINDEXRANGENVPROC glPathGlyphIndexRangeNV;
00814 extern PFNGLPATHGLYPHRANGENVPROC glPathGlyphRangeNV;
00815 extern PFNGLPATHGLYPHSNVPROC glPathGlyphsNV;
00816 extern PFNGLPATHMEMORYGLYPHINDEXARRAYNVPROC glPathMemoryGlyphIndexArrayNV;
00817 extern PFNGLPATHPARAMETERFNVPROC glPathParameterfNV;
00818 extern PFNGLPATHPARAMETERFVNVPROC glPathParameterfvNV;
00819 extern PFNGLPATHPARAMETERINVPROC glPathParameteriNV;
00820 extern PFNGLPATHPARAMETERIVNVPROC glPathParameterivNV;
00821 extern PFNGLPATHSTENCILDEPTHOFFSETNVPROC glPathStencilDepthOffsetNV;
00822 extern PFNGLPATHSTENCILFUNCNVPROC glPathStencilFuncNV;
00823 extern PFNGLPATHSTRINGNVPROC glPathStringNV;
00824 extern PFNGLPATHSUBCOMMANDSNVPROC glPathSubCommandsNV;
00825 extern PFNGLPATHSUBCOORDSNVPROC glPathSubCoordsNV;
00826 extern PFNGLPAUSETRANSFORMFEEDBACKPROC glPauseTransformFeedback;
00827 extern PFNGLPIXELSTOREFPROC glPixelStoref;
00828 extern PFNGLPIXELSTOREIPROC glPixelStorei;
00829 extern PFNGLPOINTALONGPATHNVPROC glPointAlongPathNV;
00830 extern PFNGLPOINTPARAMETERFPROC glPointParameterf;
00831 extern PFNGLPOINTPARAMETERFVPROC glPointParameterfv;
00832 extern PFNGLPOINTPARAMETERIPROC glPointParameteri;
00833 extern PFNGLPOINTPARAMETERIVPROC glPointParameteriv;
00834 extern PFNGLPOINTSIZEPROC glPointSize;
00835 extern PFNGLPOLYGONMODEPROC glPolygonMode;
00836 extern PFNGLPOLYGONOFFSETCLAMPEXTPROC glPolygonOffsetClampEXT;
00837 extern PFNGLPOLYGONOFFSETPROC glPolygonOffset;
00838 extern PFNGLPOPDEBUGGROUPPROC glPopDebugGroup;
00839 extern PFNGLPOPDEBUGMARKEREXTPROC glPopDebugMarkerEXT;
00840 extern PFNGLPRIMITIVEBOUNDINGBOXARBPROC glPrimitiveBoundingBoxARB;
```

```

00841 extern PFNGLPRIMITIVERESTARTINDEXPROC glPrimitiveRestartIndex;
00842 extern PFNGLPROGRAMBINARYPROC glProgramBinary;
00843 extern PFNGLPROGRAMPARAMETERIARBPROC glProgramParameteriARB;
00844 extern PFNGLPROGRAMPARAMETERIPROC glProgramParameteri;
00845 extern PFNGLPROGRAMPATHFRAGMENTINPUTGENNVPROC glProgramPathFragmentInputGenNV;
00846 extern PFNGLPROGRAMUNIFORM1DXTPROC glProgramUniform1dEXT;
00847 extern PFNGLPROGRAMUNIFORM1DPROC glProgramUniform1d;
00848 extern PFNGLPROGRAMUNIFORM1DVEXTPROC glProgramUniform1dvEXT;
00849 extern PFNGLPROGRAMUNIFORM1DVPROC glProgramUniform1dv;
00850 extern PFNGLPROGRAMUNIFORM1FEXTPROC glProgramUniform1fEXT;
00851 extern PFNGLPROGRAMUNIFORM1FPROC glProgramUniform1f;
00852 extern PFNGLPROGRAMUNIFORM1FVEXTPROC glProgramUniform1fvEXT;
00853 extern PFNGLPROGRAMUNIFORM1FVPROC glProgramUniform1fv;
00854 extern PFNGLPROGRAMUNIFORM1I64ARBPROC glProgramUniform1i64ARB;
00855 extern PFNGLPROGRAMUNIFORM1I64NVPROC glProgramUniform1i64NV;
00856 extern PFNGLPROGRAMUNIFORM1I64VARBPROC glProgramUniform1i64vARB;
00857 extern PFNGLPROGRAMUNIFORM1I64VNVPROC glProgramUniform1i64vNV;
00858 extern PFNGLPROGRAMUNIFORM1IEXTPROC glProgramUniform1iEXT;
00859 extern PFNGLPROGRAMUNIFORM1IPROC glProgramUniform1i;
00860 extern PFNGLPROGRAMUNIFORM1IVEXTPROC glProgramUniform1ivEXT;
00861 extern PFNGLPROGRAMUNIFORM1IVPROC glProgramUniform1iv;
00862 extern PFNGLPROGRAMUNIFORM1UI64ARBPROC glProgramUniform1ui64ARB;
00863 extern PFNGLPROGRAMUNIFORM1UI64NVPROC glProgramUniform1ui64NV;
00864 extern PFNGLPROGRAMUNIFORM1UI64VARBPROC glProgramUniform1ui64vARB;
00865 extern PFNGLPROGRAMUNIFORM1UI64VNVPROC glProgramUniform1ui64vNV;
00866 extern PFNGLPROGRAMUNIFORM1UIEXTPROC glProgramUniform1uiEXT;
00867 extern PFNGLPROGRAMUNIFORM1UIPROC glProgramUniform1ui;
00868 extern PFNGLPROGRAMUNIFORM1UIVEXTPROC glProgramUniform1uivEXT;
00869 extern PFNGLPROGRAMUNIFORM1UIVPROC glProgramUniform1uiv;
00870 extern PFNGLPROGRAMUNIFORM2DXTPROC glProgramUniform2dEXT;
00871 extern PFNGLPROGRAMUNIFORM2DPROC glProgramUniform2d;
00872 extern PFNGLPROGRAMUNIFORM2DVEXTPROC glProgramUniform2dvEXT;
00873 extern PFNGLPROGRAMUNIFORM2DVPROC glProgramUniform2dv;
00874 extern PFNGLPROGRAMUNIFORM2FEXTPROC glProgramUniform2fEXT;
00875 extern PFNGLPROGRAMUNIFORM2FPROC glProgramUniform2f;
00876 extern PFNGLPROGRAMUNIFORM2FVEXTPROC glProgramUniform2fvEXT;
00877 extern PFNGLPROGRAMUNIFORM2FVPROC glProgramUniform2fv;
00878 extern PFNGLPROGRAMUNIFORM2I64ARBPROC glProgramUniform2i64ARB;
00879 extern PFNGLPROGRAMUNIFORM2I64NVPROC glProgramUniform2i64NV;
00880 extern PFNGLPROGRAMUNIFORM2I64VARBPROC glProgramUniform2i64vARB;
00881 extern PFNGLPROGRAMUNIFORM2I64VNVPROC glProgramUniform2i64vNV;
00882 extern PFNGLPROGRAMUNIFORM2IEXTPROC glProgramUniform2iEXT;
00883 extern PFNGLPROGRAMUNIFORM2IPROC glProgramUniform2i;
00884 extern PFNGLPROGRAMUNIFORM2IVEXTPROC glProgramUniform2ivEXT;
00885 extern PFNGLPROGRAMUNIFORM2IVPROC glProgramUniform2iv;
00886 extern PFNGLPROGRAMUNIFORM2UI64ARBPROC glProgramUniform2ui64ARB;
00887 extern PFNGLPROGRAMUNIFORM2UI64NVPROC glProgramUniform2ui64NV;
00888 extern PFNGLPROGRAMUNIFORM2UI64VARBPROC glProgramUniform2ui64vARB;
00889 extern PFNGLPROGRAMUNIFORM2UI64VNVPROC glProgramUniform2ui64vNV;
00890 extern PFNGLPROGRAMUNIFORM2UIEXTPROC glProgramUniform2uiEXT;
00891 extern PFNGLPROGRAMUNIFORM2UIPROC glProgramUniform2ui;
00892 extern PFNGLPROGRAMUNIFORM2UIVEXTPROC glProgramUniform2uivEXT;
00893 extern PFNGLPROGRAMUNIFORM2UIVPROC glProgramUniform2uiv;
00894 extern PFNGLPROGRAMUNIFORM3DXTPROC glProgramUniform3dEXT;
00895 extern PFNGLPROGRAMUNIFORM3DPROC glProgramUniform3d;
00896 extern PFNGLPROGRAMUNIFORM3DVEXTPROC glProgramUniform3dvEXT;
00897 extern PFNGLPROGRAMUNIFORM3DVPROC glProgramUniform3dv;
00898 extern PFNGLPROGRAMUNIFORM3FEXTPROC glProgramUniform3fEXT;
00899 extern PFNGLPROGRAMUNIFORM3FPROC glProgramUniform3f;
00900 extern PFNGLPROGRAMUNIFORM3FVEXTPROC glProgramUniform3fvEXT;
00901 extern PFNGLPROGRAMUNIFORM3FVPROC glProgramUniform3fv;
00902 extern PFNGLPROGRAMUNIFORM3I64ARBPROC glProgramUniform3i64ARB;
00903 extern PFNGLPROGRAMUNIFORM3I64NVPROC glProgramUniform3i64NV;
00904 extern PFNGLPROGRAMUNIFORM3I64VARBPROC glProgramUniform3i64vARB;
00905 extern PFNGLPROGRAMUNIFORM3I64VNVPROC glProgramUniform3i64vNV;
00906 extern PFNGLPROGRAMUNIFORM3IEXTPROC glProgramUniform3iEXT;
00907 extern PFNGLPROGRAMUNIFORM3IPROC glProgramUniform3i;
00908 extern PFNGLPROGRAMUNIFORM3IVEXTPROC glProgramUniform3ivEXT;
00909 extern PFNGLPROGRAMUNIFORM3IVPROC glProgramUniform3iv;
00910 extern PFNGLPROGRAMUNIFORM3UI64ARBPROC glProgramUniform3ui64ARB;
00911 extern PFNGLPROGRAMUNIFORM3UI64NVPROC glProgramUniform3ui64NV;
00912 extern PFNGLPROGRAMUNIFORM3UI64VARBPROC glProgramUniform3ui64vARB;
00913 extern PFNGLPROGRAMUNIFORM3UI64VNVPROC glProgramUniform3ui64vNV;
00914 extern PFNGLPROGRAMUNIFORM3UIEXTPROC glProgramUniform3uiEXT;
00915 extern PFNGLPROGRAMUNIFORM3UIPROC glProgramUniform3ui;
00916 extern PFNGLPROGRAMUNIFORM3UIVEXTPROC glProgramUniform3uivEXT;
00917 extern PFNGLPROGRAMUNIFORM3UIVPROC glProgramUniform3uiv;
00918 extern PFNGLPROGRAMUNIFORM4DXTPROC glProgramUniform4dEXT;
00919 extern PFNGLPROGRAMUNIFORM4DPROC glProgramUniform4d;
00920 extern PFNGLPROGRAMUNIFORM4DVEXTPROC glProgramUniform4dvEXT;
00921 extern PFNGLPROGRAMUNIFORM4DVPROC glProgramUniform4dv;
00922 extern PFNGLPROGRAMUNIFORM4FEXTPROC glProgramUniform4fEXT;
00923 extern PFNGLPROGRAMUNIFORM4FPROC glProgramUniform4f;
00924 extern PFNGLPROGRAMUNIFORM4FVEXTPROC glProgramUniform4fvEXT;
00925 extern PFNGLPROGRAMUNIFORM4FVPROC glProgramUniform4fv;
00926 extern PFNGLPROGRAMUNIFORM4I64ARBPROC glProgramUniform4i64ARB;
00927 extern PFNGLPROGRAMUNIFORM4I64NVPROC glProgramUniform4i64NV;

```



```
00928 extern PFNGLPROGRAMUNIFORM4I64VARBPROC glProgramUniform4i64vARB;
00929 extern PFNGLPROGRAMUNIFORM4I64VNVPROC glProgramUniform4i64vNV;
00930 extern PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4iEXT;
00931 extern PFNGLPROGRAMUNIFORM4IPROC glProgramUniform4i;
00932 extern PFNGLPROGRAMUNIFORM4IEXTPROC glProgramUniform4ivEXT;
00933 extern PFNGLPROGRAMUNIFORM4IVPROC glProgramUniform4iv;
00934 extern PFNGLPROGRAMUNIFORM4UI64ARBPROC glProgramUniform4ui64ARB;
00935 extern PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00936 extern PFNGLPROGRAMUNIFORM4UI64VARBPROC glProgramUniform4ui64vARB;
00937 extern PFNGLPROGRAMUNIFORM4UI64VNVPROC glProgramUniform4ui64vNV;
00938 extern PFNGLPROGRAMUNIFORM4UIEXTPROC glProgramUniform4uiEXT;
00939 extern PFNGLPROGRAMUNIFORM4UIPROC glProgramUniform4ui;
00940 extern PFNGLPROGRAMUNIFORM4UIVEXTPROC glProgramUniform4uivEXT;
00941 extern PFNGLPROGRAMUNIFORM4UIVPROC glProgramUniform4uiv;
00942 extern PFNGLPROGRAMUNIFORMHANDLEUI64ARBPROC glProgramUniformHandleui64ARB;
00943 extern PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00944 extern PFNGLPROGRAMUNIFORMHANDLEUI64VARBPROC glProgramUniformHandleui64vARB;
00945 extern PFNGLPROGRAMUNIFORMHANDLEUI64VNVPROC glProgramUniformHandleui64vNV;
00946 extern PFNGLPROGRAMUNIFORMMATRIX2DVEXTPROC glProgramUniformMatrix2dvEXT;
00947 extern PFNGLPROGRAMUNIFORMMATRIX2DVPROC glProgramUniformMatrix2dv;
00948 extern PFNGLPROGRAMUNIFORMMATRIX2FVEXTPROC glProgramUniformMatrix2fvEXT;
00949 extern PFNGLPROGRAMUNIFORMMATRIX2FVPROC glProgramUniformMatrix2fv;
00950 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVEXTPROC glProgramUniformMatrix2x3dvEXT;
00951 extern PFNGLPROGRAMUNIFORMMATRIX2X3DVPROC glProgramUniformMatrix2x3dv;
00952 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVEXTPROC glProgramUniformMatrix2x3fvEXT;
00953 extern PFNGLPROGRAMUNIFORMMATRIX2X3FVPROC glProgramUniformMatrix2x3fv;
00954 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVEXTPROC glProgramUniformMatrix2x4dvEXT;
00955 extern PFNGLPROGRAMUNIFORMMATRIX2X4DVPROC glProgramUniformMatrix2x4dv;
00956 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVEXTPROC glProgramUniformMatrix2x4fvEXT;
00957 extern PFNGLPROGRAMUNIFORMMATRIX2X4FVPROC glProgramUniformMatrix2x4fv;
00958 extern PFNGLPROGRAMUNIFORMMATRIX3DVEXTPROC glProgramUniformMatrix3dvEXT;
00959 extern PFNGLPROGRAMUNIFORMMATRIX3DVPROC glProgramUniformMatrix3dv;
00960 extern PFNGLPROGRAMUNIFORMMATRIX3FVEXTPROC glProgramUniformMatrix3fvEXT;
00961 extern PFNGLPROGRAMUNIFORMMATRIX3FVPROC glProgramUniformMatrix3fv;
00962 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVEXTPROC glProgramUniformMatrix3x2dvEXT;
00963 extern PFNGLPROGRAMUNIFORMMATRIX3X2DVPROC glProgramUniformMatrix3x2dv;
00964 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVEXTPROC glProgramUniformMatrix3x2fvEXT;
00965 extern PFNGLPROGRAMUNIFORMMATRIX3X2FVPROC glProgramUniformMatrix3x2fv;
00966 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVEXTPROC glProgramUniformMatrix3x4dvEXT;
00967 extern PFNGLPROGRAMUNIFORMMATRIX3X4DVPROC glProgramUniformMatrix3x4dv;
00968 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVEXTPROC glProgramUniformMatrix3x4fvEXT;
00969 extern PFNGLPROGRAMUNIFORMMATRIX3X4FVPROC glProgramUniformMatrix3x4fv;
00970 extern PFNGLPROGRAMUNIFORMMATRIX4DVEXTPROC glProgramUniformMatrix4dvEXT;
00971 extern PFNGLPROGRAMUNIFORMMATRIX4DVPROC glProgramUniformMatrix4dv;
00972 extern PFNGLPROGRAMUNIFORMMATRIX4FVEXTPROC glProgramUniformMatrix4fvEXT;
00973 extern PFNGLPROGRAMUNIFORMMATRIX4FVPROC glProgramUniformMatrix4fv;
00974 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVEXTPROC glProgramUniformMatrix4x2dvEXT;
00975 extern PFNGLPROGRAMUNIFORMMATRIX4X2DVPROC glProgramUniformMatrix4x2dv;
00976 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVEXTPROC glProgramUniformMatrix4x2fvEXT;
00977 extern PFNGLPROGRAMUNIFORMMATRIX4X2FVPROC glProgramUniformMatrix4x2fv;
00978 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVEXTPROC glProgramUniformMatrix4x3dvEXT;
00979 extern PFNGLPROGRAMUNIFORMMATRIX4X3DVPROC glProgramUniformMatrix4x3dv;
00980 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVEXTPROC glProgramUniformMatrix4x3fvEXT;
00981 extern PFNGLPROGRAMUNIFORMMATRIX4X3FVPROC glProgramUniformMatrix4x3fv;
00982 extern PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00983 extern PFNGLPROGRAMUNIFORMUI64VNVPROC glProgramUniformui64vNV;
00984 extern PFNGLPROVOKINGVERTEXPROC glProvokingVertex;
00985 extern PFNGLPUSHCLIENTATTRIBDEFAULTEXTPROC glPushClientAttribDefaultEXT;
00986 extern PFNGLPUSHDEBUGGROUPPROC glPushDebugGroup;
00987 extern PFNGLPUSHGROUPMARKEREXTPROC glPushGroupMarkerEXT;
00988 extern PFNGLQUERYCOUNTERPROC glQueryCounter;
00989 extern PFNGLRASTERSAMPLESEXTPROC glRasterSamplesEXT;
00990 extern PFNGLREADBUFFERPROC glReadBuffer;
00991 extern PFNGLREADNPIXELSARBPROC glReadnPixelsARB;
00992 extern PFNGLREADNPIXELSPROC glReadnPixels;
00993 extern PFNGLREADPIXELSPROC glReadPixels;
00994 extern PFNGLRELEASESHADERCOMPILERPROC glReleaseShaderCompiler;
00995 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLECOVERGENVPROC glRenderbufferStorageMultisampleCoverageNV;
00996 extern PFNGLRENDERBUFFERSTORAGEMULTISAMPLEPROC glRenderbufferStorageMultisample;
00997 extern PFNGLRENDERBUFFERSTORAGEPROC glRenderbufferStorage;
00998 extern PFNGLRESOLVEDEPTHVALUESNVPROC glResolveDepthValuesNV;
00999 extern PFNGLRESUMETRANSFORMFEEDBACKPROC glResumeTransformFeedback;
01000 extern PFNGLSAMPLECOVERAGEPROC glSampleCoverage;
01001 extern PFNGLSAMPLERMASKIPROC glSamplerMaski;
01002 extern PFNGLSAMPLERPARAMETERFPROC glSamplerParameterf;
01003 extern PFNGLSAMPLERPARAMETERFVPROC glSamplerParameterfv;
01004 extern PFNGLSAMPLERPARAMETERIIVPROC glSamplerParameterIiv;
01005 extern PFNGLSAMPLERPARAMETERIPROC glSamplerParameteri;
01006 extern PFNGLSAMPLERPARAMETERIUIVPROC glSamplerParameterIuiv;
01007 extern PFNGLSAMPLERPARAMETERIVPROC glSamplerParameteriv;
01008 extern PFNGLSCISSORARRAYVPROC glScissorArrayv;
01009 extern PFNGLSCISSORINDEXEDPROC glScissorIndexed;
01010 extern PFNGLSCISSORINDEXEDVPROC glScissorIndexedv;
01011 extern PFNGLSCISSORPROC glScissor;
01012 extern PFNGLSECONDARYCOLORFORMATNVPROC glSecondaryColorFormatNV;
01013 extern PFNGLSELECTPERFMONITORCOUNTERSAMDPROC glSelectPerfMonitorCountersAMD;
01014 extern PFNGLSHADERBINARYPROC glShaderBinary;
```

```

01015 extern PFNGLSHADERSOURCEPROC glShaderSource;
01016 extern PFNGLSHADERSTORAGEBLOCKBINDINGPROC glShaderStorageBlockBinding;
01017 extern PFNGLSIGNALVKFENCEVPROC glSignalVkFenceNV;
01018 extern PFNGLSIGNALVKSEMAPHORENVPROC glSignalVkSemaphoreNV;
01019 extern PFNGLSPECIALIZESHADERARBPROC glSpecializeShaderARB;
01020 extern PFNGLSTATECAPTURENVPROC glStateCaptureNV;
01021 extern PFNGLSTENCILFILLPATHINSTANCEDNVPROC glStencilFillPathInstancedNV;
01022 extern PFNGLSTENCILFILLPATHNVPROC glStencilFillPathNV;
01023 extern PFNGLSTENCILFUNCPROC glStencilFunc;
01024 extern PFNGLSTENCILFUNCSEPARATEPROC glStencilFuncSeparate;
01025 extern PFNGLSTENCILMASKPROC glStencilMask;
01026 extern PFNGLSTENCILMASKSEPARATEPROC glStencilMaskSeparate;
01027 extern PFNGLSTENCILOPPROC glStencilOp;
01028 extern PFNGLSTENCILOPSEPARATEPROC glStencilOpSeparate;
01029 extern PFNGLSTENCILSTROKEPATHINSTANCEDNVPROC glStencilStrokePathInstancedNV;
01030 extern PFNGLSTENCILSTROKEPATHNVPROC glStencilStrokePathNV;
01031 extern PFNGLSTENCILTHENCOVERFILLPATHINSTANCEDNVPROC glStencilThenCoverFillPathInstancedNV;
01032 extern PFNGLSTENCILTHENCOVERFILLPATHNVPROC glStencilThenCoverFillPathNV;
01033 extern PFNGLSTENCILTHENCOVERSTROKEPATHINSTANCEDNVPROC glStencilThenCoverStrokePathInstancedNV;
01034 extern PFNGLSTENCILTHENCOVERSTROKEPATHNVPROC glStencilThenCoverStrokePathNV;
01035 extern PFNGLSUBPIXELPRECISIONBIASNVPROC glSubpixelPrecisionBiasNV;
01036 extern PFNGLTEXBUFFERARBPROC glTexBufferARB;
01037 extern PFNGLTEXBUFFERPROC glTexBuffer;
01038 extern PFNGLTEXBUFFERRANGEPROC glTexBufferRange;
01039 extern PFNGLTEXCOORDFORMATNVPROC glTexCoordFormatNV;
01040 extern PFNGLTEXIMAGE1DPROC glTexImage1D;
01041 extern PFNGLTEXIMAGE2DMULTISAMPLEPROC glTexImage2DMultisample;
01042 extern PFNGLTEXIMAGE2DPROC glTexImage2D;
01043 extern PFNGLTEXIMAGE3DMULTISAMPLEPROC glTexImage3DMultisample;
01044 extern PFNGLTEXIMAGE3DPROC glTexImage3D;
01045 extern PFNGLTEXPAGECOMMITMENTARBPROC glTexPageCommitmentARB;
01046 extern PFNGLTEXPARAMETERFPROC glTexParameterf;
01047 extern PFNGLTEXPARAMETERFVPROC glTexParameterfv;
01048 extern PFNGLTEXPARAMETERIIVPROC glTexParameterIiv;
01049 extern PFNGLTEXPARAMETERIPROC glTexParameteri;
01050 extern PFNGLTEXPARAMETERIUIVPROC glTexParameterIuiv;
01051 extern PFNGLTEXPARAMETERIVPROC glTexParameteriv;
01052 extern PFNGLTEXSTORAGE1DPROC glTexStorage1D;
01053 extern PFNGLTEXSTORAGE2DMULTISAMPLEPROC glTexStorage2DMultisample;
01054 extern PFNGLTEXSTORAGE2DPROC glTexStorage2D;
01055 extern PFNGLTEXSTORAGE3DMULTISAMPLEPROC glTexStorage3DMultisample;
01056 extern PFNGLTEXSTORAGE3DPROC glTexStorage3D;
01057 extern PFNGLTEXSUBIMAGE1DPROC glTexSubImage1D;
01058 extern PFNGLTEXSUBIMAGE2DPROC glTexSubImage2D;
01059 extern PFNGLTEXSUBIMAGE3DPROC glTexSubImage3D;
01060 extern PFNGLTEXTUREBARRIERNVPROC glTextureBarrierNV;
01061 extern PFNGLTEXTUREBARRIERPROC glTextureBarrier;
01062 extern PFNGLTEXTUREBUFFEREXTPROC glTextureBufferEXT;
01063 extern PFNGLTEXTUREBUFFERPROC glTextureBuffer;
01064 extern PFNGLTEXTUREBUFFERRANGEEXTPROC glTextureBufferRangeEXT;
01065 extern PFNGLTEXTUREBUFFERRANGEPROC glTextureBufferRange;
01066 extern PFNGLTEXTUREIMAGE1DEXTPROC glTextureImage1DEXT;
01067 extern PFNGLTEXTUREIMAGE2DEXTPROC glTextureImage2DEXT;
01068 extern PFNGLTEXTUREIMAGE3DEXTPROC glTextureImage3DEXT;
01069 extern PFNGLTEXTUREPAGECOMMITMENTEXTPROC glTexturePageCommitmentEXT;
01070 extern PFNGLTEXTUREPARAMETERFEXTPROC glTextureParameterfEXT;
01071 extern PFNGLTEXTUREPARAMETERFPROC glTextureParameterf;
01072 extern PFNGLTEXTUREPARAMETERFVEXTPROC glTextureParameterfvEXT;
01073 extern PFNGLTEXTUREPARAMETERFVPROC glTextureParameterfv;
01074 extern PFNGLTEXTUREPARAMETERIEXTPROC glTextureParameteriEXT;
01075 extern PFNGLTEXTUREPARAMETERIIVEXTPROC glTextureParameterIivEXT;
01076 extern PFNGLTEXTUREPARAMETERIIVPROC glTextureParameterIiv;
01077 extern PFNGLTEXTUREPARAMETERIPROC glTextureParameteri;
01078 extern PFNGLTEXTUREPARAMETERIUIVEXTPROC glTextureParameterIuivEXT;
01079 extern PFNGLTEXTUREPARAMETERIUIVPROC glTextureParameterIuiv;
01080 extern PFNGLTEXTUREPARAMETERIVEXTPROC glTextureParameterivEXT;
01081 extern PFNGLTEXTUREPARAMETERIVPROC glTextureParameteriv;
01082 extern PFNGLTEXTURERENDERBUFFEREXTPROC glTextureRenderbufferEXT;
01083 extern PFNGLTEXTURESTORAGE1DEXTPROC glTextureStorage1DEXT;
01084 extern PFNGLTEXTURESTORAGE1DPROC glTextureStorage1D;
01085 extern PFNGLTEXTURESTORAGE2DEXTPROC glTextureStorage2DEXT;
01086 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEEXTPROC glTextureStorage2DMultisampleEXT;
01087 extern PFNGLTEXTURESTORAGE2DMULTISAMPLEPROC glTextureStorage2DMultisample;
01088 extern PFNGLTEXTURESTORAGE2DPROC glTextureStorage2D;
01089 extern PFNGLTEXTURESTORAGE3DEXTPROC glTextureStorage3DEXT;
01090 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEEXTPROC glTextureStorage3DMultisampleEXT;
01091 extern PFNGLTEXTURESTORAGE3DMULTISAMPLEPROC glTextureStorage3DMultisample;
01092 extern PFNGLTEXTURESTORAGE3DPROC glTextureStorage3D;
01093 extern PFNGLTEXTURESUBIMAGE1DEXTPROC glTextureSubImage1DEXT;
01094 extern PFNGLTEXTURESUBIMAGE1DPROC glTextureSubImage1D;
01095 extern PFNGLTEXTURESUBIMAGE2DEXTPROC glTextureSubImage2DEXT;
01096 extern PFNGLTEXTURESUBIMAGE2DPROC glTextureSubImage2D;
01097 extern PFNGLTEXTURESUBIMAGE3DEXTPROC glTextureSubImage3DEXT;
01098 extern PFNGLTEXTURESUBIMAGE3DPROC glTextureSubImage3D;
01099 extern PFNGLTEXTUREVIEWPROC glTextureView;
01100 extern PFNGLTRANSFORMFEEDBACKBUFFERBASEPROC glTransformFeedbackBufferBase;
01101 extern PFNGLTRANSFORMFEEDBACKBUFFERRANGEPROC glTransformFeedbackBufferRange;

```

```
01102 extern PFNGLTRANSFORMFEEDBACKVARYINGSPROC glTransformFeedbackVaryings;
01103 extern PFNGLTRANSFORMPATHNVPROC glTransformPathNV;
01104 extern PFNGLUNIFORM1DPROC glUniform1d;
01105 extern PFNGLUNIFORM1DVPROC glUniform1dv;
01106 extern PFNGLUNIFORM1FPROC glUniform1f;
01107 extern PFNGLUNIFORM1FVPROC glUniform1fv;
01108 extern PFNGLUNIFORM1I64ARBPROC glUniform1i64ARB;
01109 extern PFNGLUNIFORM1I64NVPROC glUniform1i64NV;
01110 extern PFNGLUNIFORM1I64VARBPROC glUniform1i64vARB;
01111 extern PFNGLUNIFORM1I64VNVPROC glUniform1i64vNV;
01112 extern PFNGLUNIFORM1IPROC glUniform1i;
01113 extern PFNGLUNIFORM1IVPROC glUniform1iv;
01114 extern PFNGLUNIFORM1UI64ARBPROC glUniform1ui64ARB;
01115 extern PFNGLUNIFORM1UI64NVPROC glUniform1ui64NV;
01116 extern PFNGLUNIFORM1UI64VARBPROC glUniform1ui64vARB;
01117 extern PFNGLUNIFORM1UI64VNVPROC glUniform1ui64vNV;
01118 extern PFNGLUNIFORM1UIPROC glUniform1ui;
01119 extern PFNGLUNIFORM1UIVPROC glUniform1uiv;
01120 extern PFNGLUNIFORM2DPROC glUniform2d;
01121 extern PFNGLUNIFORM2DVPROC glUniform2dv;
01122 extern PFNGLUNIFORM2FPROC glUniform2f;
01123 extern PFNGLUNIFORM2FVPROC glUniform2fv;
01124 extern PFNGLUNIFORM2I64ARBPROC glUniform2i64ARB;
01125 extern PFNGLUNIFORM2I64NVPROC glUniform2i64NV;
01126 extern PFNGLUNIFORM2I64VARBPROC glUniform2i64vARB;
01127 extern PFNGLUNIFORM2I64VNVPROC glUniform2i64vNV;
01128 extern PFNGLUNIFORM2IPROC glUniform2i;
01129 extern PFNGLUNIFORM2IVPROC glUniform2iv;
01130 extern PFNGLUNIFORM2UI64ARBPROC glUniform2ui64ARB;
01131 extern PFNGLUNIFORM2UI64NVPROC glUniform2ui64NV;
01132 extern PFNGLUNIFORM2UI64VARBPROC glUniform2ui64vARB;
01133 extern PFNGLUNIFORM2UI64VNVPROC glUniform2ui64vNV;
01134 extern PFNGLUNIFORM2UIPROC glUniform2ui;
01135 extern PFNGLUNIFORM2UIVPROC glUniform2uiv;
01136 extern PFNGLUNIFORM3DPROC glUniform3d;
01137 extern PFNGLUNIFORM3DVPROC glUniform3dv;
01138 extern PFNGLUNIFORM3FPROC glUniform3f;
01139 extern PFNGLUNIFORM3FVPROC glUniform3fv;
01140 extern PFNGLUNIFORM3I64ARBPROC glUniform3i64ARB;
01141 extern PFNGLUNIFORM3I64NVPROC glUniform3i64NV;
01142 extern PFNGLUNIFORM3I64VARBPROC glUniform3i64vARB;
01143 extern PFNGLUNIFORM3I64VNVPROC glUniform3i64vNV;
01144 extern PFNGLUNIFORM3IPROC glUniform3i;
01145 extern PFNGLUNIFORM3IVPROC glUniform3iv;
01146 extern PFNGLUNIFORM3UI64ARBPROC glUniform3ui64ARB;
01147 extern PFNGLUNIFORM3UI64NVPROC glUniform3ui64NV;
01148 extern PFNGLUNIFORM3UI64VARBPROC glUniform3ui64vARB;
01149 extern PFNGLUNIFORM3UI64VNVPROC glUniform3ui64vNV;
01150 extern PFNGLUNIFORM3UIPROC glUniform3ui;
01151 extern PFNGLUNIFORM3UIVPROC glUniform3uiv;
01152 extern PFNGLUNIFORM4DPROC glUniform4d;
01153 extern PFNGLUNIFORM4DVPROC glUniform4dv;
01154 extern PFNGLUNIFORM4FPROC glUniform4f;
01155 extern PFNGLUNIFORM4FVPROC glUniform4fv;
01156 extern PFNGLUNIFORM4I64ARBPROC glUniform4i64ARB;
01157 extern PFNGLUNIFORM4I64NVPROC glUniform4i64NV;
01158 extern PFNGLUNIFORM4I64VARBPROC glUniform4i64vARB;
01159 extern PFNGLUNIFORM4I64VNVPROC glUniform4i64vNV;
01160 extern PFNGLUNIFORM4IPROC glUniform4i;
01161 extern PFNGLUNIFORM4IVPROC glUniform4iv;
01162 extern PFNGLUNIFORM4UI64ARBPROC glUniform4ui64ARB;
01163 extern PFNGLUNIFORM4UI64NVPROC glUniform4ui64NV;
01164 extern PFNGLUNIFORM4UI64VARBPROC glUniform4ui64vARB;
01165 extern PFNGLUNIFORM4UI64VNVPROC glUniform4ui64vNV;
01166 extern PFNGLUNIFORM4UIPROC glUniform4ui;
01167 extern PFNGLUNIFORM4UIVPROC glUniform4uiv;
01168 extern PFNGLUNIFORMBLOCKBINDINGPROC glUniformBlockBinding;
01169 extern PFNGLUNIFORMHANDLEUI64ARBPROC glUniformHandleui64ARB;
01170 extern PFNGLUNIFORMHANDLEUI64NVPROC glUniformHandleui64NV;
01171 extern PFNGLUNIFORMHANDLEUI64VARBPROC glUniformHandleui64vARB;
01172 extern PFNGLUNIFORMHANDLEUI64VNVPROC glUniformHandleui64vNV;
01173 extern PFNGLUNIFORMMATRIX2DVPROC glUniformMatrix2dv;
01174 extern PFNGLUNIFORMMATRIX2FVPROC glUniformMatrix2fv;
01175 extern PFNGLUNIFORMMATRIX2X3DVPROC glUniformMatrix2x3dv;
01176 extern PFNGLUNIFORMMATRIX2X3FVPROC glUniformMatrix2x3fv;
01177 extern PFNGLUNIFORMMATRIX2X4DVPROC glUniformMatrix2x4dv;
01178 extern PFNGLUNIFORMMATRIX2X4FVPROC glUniformMatrix2x4fv;
01179 extern PFNGLUNIFORMMATRIX3DVPROC glUniformMatrix3dv;
01180 extern PFNGLUNIFORMMATRIX3FVPROC glUniformMatrix3fv;
01181 extern PFNGLUNIFORMMATRIX3X2DVPROC glUniformMatrix3x2dv;
01182 extern PFNGLUNIFORMMATRIX3X2FVPROC glUniformMatrix3x2fv;
01183 extern PFNGLUNIFORMMATRIX3X4DVPROC glUniformMatrix3x4dv;
01184 extern PFNGLUNIFORMMATRIX3X4FVPROC glUniformMatrix3x4fv;
01185 extern PFNGLUNIFORMMATRIX4DVPROC glUniformMatrix4dv;
01186 extern PFNGLUNIFORMMATRIX4FVPROC glUniformMatrix4fv;
01187 extern PFNGLUNIFORMMATRIX4X2DVPROC glUniformMatrix4x2dv;
01188 extern PFNGLUNIFORMMATRIX4X2FVPROC glUniformMatrix4x2fv;
```

```

01189 extern PFNGLUNIFORMMATRIX4X3DVPROC glUniformMatrix4x3dv;
01190 extern PFNGLUNIFORMMATRIX4X3FVPROC glUniformMatrix4x3fv;
01191 extern PFNGLUNIFORMSUBROUTINESUIVPROC glUniformSubroutinesuiv;
01192 extern PFNGLUNIFORMUI64NVPROC glUniformui64NV;
01193 extern PFNGLUNIFORMUI64VNVPROC glUniformui64vNV;
01194 extern PFNGLUNMAPBUFFERPROC glUnmapBuffer;
01195 extern PFNGLUNMAPNAMEDBUFFEREXTPROC glUnmapNamedBufferEXT;
01196 extern PFNGLUNMAPNAMEDBUFFERPROC glUnmapNamedBuffer;
01197 extern PFNGLUSEPROGRAMPROC glUseProgram;
01198 extern PFNGLUSEPROGRAMSTAGESPROC glUseProgramStages;
01199 extern PFNGLUSESHADERPROGRAMEXTPROC glUseProgramEXT;
01200 extern PFNGLVALIDATEPROGRAMPIPELINEPROC glValidateProgramPipeline;
01201 extern PFNGLVALIDATEPROGRAMPROC glValidateProgram;
01202 extern PFNGLVERTEXARRAYATTRIBBINDINGPROC glVertexArrayAttribBinding;
01203 extern PFNGLVERTEXARRAYATTRIBFORMATPROC glVertexArrayAttribFormat;
01204 extern PFNGLVERTEXARRAYATTRIBIFORMATPROC glVertexArrayAttribIFormat;
01205 extern PFNGLVERTEXARRAYATTRIBLFORMATPROC glVertexArrayAttribLFormat;
01206 extern PFNGLVERTEXARRAYBINDINGDIVISORPROC glVertexArrayBindingDivisor;
01207 extern PFNGLVERTEXARRAYBINDVERTEXBUFFEREXTPROC glVertexArrayBindVertexBufferEXT;
01208 extern PFNGLVERTEXARRAYCOLOROFFSETEXTPROC glVertexArrayColorOffsetEXT;
01209 extern PFNGLVERTEXARRAYEDGEFLAGOFFSETEXTPROC glVertexArrayEdgeFlagOffsetEXT;
01210 extern PFNGLVERTEXARRAYELEMENTBUFFERPROC glVertexArrayElementBuffer;
01211 extern PFNGLVERTEXARRAYFOGCOORDOFFSETEXTPROC glVertexArrayFogCoordOffsetEXT;
01212 extern PFNGLVERTEXARRAYINDEXOFFSETEXTPROC glVertexArrayIndexOffsetEXT;
01213 extern PFNGLVERTEXARRAYMULTITEXCOORDOFFSETEXTPROC glVertexArrayMultiTexCoordOffsetEXT;
01214 extern PFNGLVERTEXARRAYNORMALOFFSETEXTPROC glVertexArrayNormalOffsetEXT;
01215 extern PFNGLVERTEXARRAYSECONDARYCOLOROFFSETEXTPROC glVertexArraySecondaryColorOffsetEXT;
01216 extern PFNGLVERTEXARRAYTEXCOORDOFFSETEXTPROC glVertexArrayTexCoordOffsetEXT;
01217 extern PFNGLVERTEXARRAYVERTEXATTRIBBINDINGEXTPROC glVertexArrayVertexAttribBindingEXT;
01218 extern PFNGLVERTEXARRAYVERTEXATTRIBDIVISOREXTPROC glVertexArrayVertexAttribDivisorEXT;
01219 extern PFNGLVERTEXARRAYVERTEXATTRIBFORMATEXTPROC glVertexArrayVertexAttribFormatEXT;
01220 extern PFNGLVERTEXARRAYVERTEXATTRIBIFORMATEXTPROC glVertexArrayVertexAttribIFormatEXT;
01221 extern PFNGLVERTEXARRAYVERTEXATTRIBIOFFSETEXTPROC glVertexArrayVertexAttribIOffsetEXT;
01222 extern PFNGLVERTEXARRAYVERTEXATTRIBLFORMATEXTPROC glVertexArrayVertexAttribLFormatEXT;
01223 extern PFNGLVERTEXARRAYVERTEXATTRIBLOFFSETEXTPROC glVertexArrayVertexAttribLOffsetEXT;
01224 extern PFNGLVERTEXARRAYVERTEXATTRIBOFFSETEXTPROC glVertexArrayVertexAttribOffsetEXT;
01225 extern PFNGLVERTEXARRAYVERTEXBINDINGDIVISOREXTPROC glVertexArrayVertexBindingDivisorEXT;
01226 extern PFNGLVERTEXARRAYVERTEXBUFFERPROC glVertexArrayVertexBuffer;
01227 extern PFNGLVERTEXARRAYVERTEXBUFFERSPROC glVertexArrayVertexBuffers;
01228 extern PFNGLVERTEXARRAYVERTEXOFFSETEXTPROC glVertexArrayVertexOffsetEXT;
01229 extern PFNGLVERTEXATTRIB1DPROC glVertexAttrib1d;
01230 extern PFNGLVERTEXATTRIB1DVPROC glVertexAttrib1dv;
01231 extern PFNGLVERTEXATTRIB1FPROC glVertexAttrib1f;
01232 extern PFNGLVERTEXATTRIB1FVPROC glVertexAttrib1fv;
01233 extern PFNGLVERTEXATTRIB1SPROC glVertexAttrib1s;
01234 extern PFNGLVERTEXATTRIB1SVPROC glVertexAttrib1sv;
01235 extern PFNGLVERTEXATTRIB2DPROC glVertexAttrib2d;
01236 extern PFNGLVERTEXATTRIB2DVPROC glVertexAttrib2dv;
01237 extern PFNGLVERTEXATTRIB2FPROC glVertexAttrib2f;
01238 extern PFNGLVERTEXATTRIB2FVPROC glVertexAttrib2fv;
01239 extern PFNGLVERTEXATTRIB2SPROC glVertexAttrib2s;
01240 extern PFNGLVERTEXATTRIB2SVPROC glVertexAttrib2sv;
01241 extern PFNGLVERTEXATTRIB3DPROC glVertexAttrib3d;
01242 extern PFNGLVERTEXATTRIB3DVPROC glVertexAttrib3dv;
01243 extern PFNGLVERTEXATTRIB3FPROC glVertexAttrib3f;
01244 extern PFNGLVERTEXATTRIB3FVPROC glVertexAttrib3fv;
01245 extern PFNGLVERTEXATTRIB3SPROC glVertexAttrib3s;
01246 extern PFNGLVERTEXATTRIB3SVPROC glVertexAttrib3sv;
01247 extern PFNGLVERTEXATTRIB4BVPROC glVertexAttrib4bv;
01248 extern PFNGLVERTEXATTRIB4DPROC glVertexAttrib4d;
01249 extern PFNGLVERTEXATTRIB4DVPROC glVertexAttrib4dv;
01250 extern PFNGLVERTEXATTRIB4FPROC glVertexAttrib4f;
01251 extern PFNGLVERTEXATTRIB4FVPROC glVertexAttrib4fv;
01252 extern PFNGLVERTEXATTRIB4IVPROC glVertexAttrib4iv;
01253 extern PFNGLVERTEXATTRIB4NBVPROC glVertexAttrib4Nbv;
01254 extern PFNGLVERTEXATTRIB4NIVPROC glVertexAttrib4Niv;
01255 extern PFNGLVERTEXATTRIB4NSVPROC glVertexAttrib4Nsv;
01256 extern PFNGLVERTEXATTRIB4NUBPROC glVertexAttrib4Nub;
01257 extern PFNGLVERTEXATTRIB4NUBVPROC glVertexAttrib4Nubv;
01258 extern PFNGLVERTEXATTRIB4NUIVPROC glVertexAttrib4Nuiv;
01259 extern PFNGLVERTEXATTRIB4NUSVPROC glVertexAttrib4Nusv;
01260 extern PFNGLVERTEXATTRIB4SPROC glVertexAttrib4s;
01261 extern PFNGLVERTEXATTRIB4SVPROC glVertexAttrib4sv;
01262 extern PFNGLVERTEXATTRIB4UBVPROC glVertexAttrib4ubv;
01263 extern PFNGLVERTEXATTRIB4UIVPROC glVertexAttrib4uiv;
01264 extern PFNGLVERTEXATTRIB4USVPROC glVertexAttrib4usv;
01265 extern PFNGLVERTEXATTRIBBINDINGPROC glVertexAttribBinding;
01266 extern PFNGLVERTEXATTRIBDIVISORARBPROC glVertexAttribDivisorARB;
01267 extern PFNGLVERTEXATTRIBDIVISORPROC glVertexAttribDivisor;
01268 extern PFNGLVERTEXATTRIBFORMATNVPROC glVertexAttribFormatNV;
01269 extern PFNGLVERTEXATTRIBFORMATPROC glVertexAttribFormat;
01270 extern PFNGLVERTEXATTRIBI1IPROC glVertexAttribI1i;
01271 extern PFNGLVERTEXATTRIBI1IVPROC glVertexAttribI1iv;
01272 extern PFNGLVERTEXATTRIBI1UIPROC glVertexAttribI1ui;
01273 extern PFNGLVERTEXATTRIBI1UIVPROC glVertexAttribI1uiv;
01274 extern PFNGLVERTEXATTRIBI2IPROC glVertexAttribI2i;
01275 extern PFNGLVERTEXATTRIBI2IVPROC glVertexAttribI2iv;

```



```

01276 extern PFNGLVERTEXATTRIBI2UIPROC glVertexAttribI2ui;
01277 extern PFNGLVERTEXATTRIBI2UIVPROC glVertexAttribI2uiv;
01278 extern PFNGLVERTEXATTRIBI3IPROC glVertexAttribI3i;
01279 extern PFNGLVERTEXATTRIBI3IVPROC glVertexAttribI3iv;
01280 extern PFNGLVERTEXATTRIBI3UIPROC glVertexAttribI3ui;
01281 extern PFNGLVERTEXATTRIBI3UIVPROC glVertexAttribI3uiv;
01282 extern PFNGLVERTEXATTRIBI4BVPROC glVertexAttribI4bv;
01283 extern PFNGLVERTEXATTRIBI4IPROC glVertexAttribI4i;
01284 extern PFNGLVERTEXATTRIBI4IVPROC glVertexAttribI4iv;
01285 extern PFNGLVERTEXATTRIBI4SVPROC glVertexAttribI4sv;
01286 extern PFNGLVERTEXATTRIBI4UBVPROC glVertexAttribI4ubv;
01287 extern PFNGLVERTEXATTRIBI4UIPROC glVertexAttribI4ui;
01288 extern PFNGLVERTEXATTRIBI4UIVPROC glVertexAttribI4uiv;
01289 extern PFNGLVERTEXATTRIBI4USVPROC glVertexAttribI4usv;
01290 extern PFNGLVERTEXATTRIBIFORMATNVPROC glVertexAttribIFormatNV;
01291 extern PFNGLVERTEXATTRIBIFORMATPROC glVertexAttribIFormat;
01292 extern PFNGLVERTEXATTRIBIPOINTERPROC glVertexAttribIPointer;
01293 extern PFNGLVERTEXATTRIBL1DPROC glVertexAttribL1d;
01294 extern PFNGLVERTEXATTRIBL1DVPROC glVertexAttribL1dv;
01295 extern PFNGLVERTEXATTRIBL1I64NVPROC glVertexAttribL1i64NV;
01296 extern PFNGLVERTEXATTRIBL1I64VNVPROC glVertexAttribL1i64vNV;
01297 extern PFNGLVERTEXATTRIBL1UI64ARBPROC glVertexAttribL1ui64ARB;
01298 extern PFNGLVERTEXATTRIBL1UI64NVPROC glVertexAttribL1ui64NV;
01299 extern PFNGLVERTEXATTRIBL1UI64VARBPROC glVertexAttribL1ui64vARB;
01300 extern PFNGLVERTEXATTRIBL1UI64VNVPROC glVertexAttribL1ui64vNV;
01301 extern PFNGLVERTEXATTRIBL2DPROC glVertexAttribL2d;
01302 extern PFNGLVERTEXATTRIBL2DVPROC glVertexAttribL2dv;
01303 extern PFNGLVERTEXATTRIBL2I64NVPROC glVertexAttribL2i64NV;
01304 extern PFNGLVERTEXATTRIBL2I64VNVPROC glVertexAttribL2i64vNV;
01305 extern PFNGLVERTEXATTRIBL2UI64NVPROC glVertexAttribL2ui64NV;
01306 extern PFNGLVERTEXATTRIBL2UI64VNVPROC glVertexAttribL2ui64vNV;
01307 extern PFNGLVERTEXATTRIBL3DPROC glVertexAttribL3d;
01308 extern PFNGLVERTEXATTRIBL3DVPROC glVertexAttribL3dv;
01309 extern PFNGLVERTEXATTRIBL3I64NVPROC glVertexAttribL3i64NV;
01310 extern PFNGLVERTEXATTRIBL3I64VNVPROC glVertexAttribL3i64vNV;
01311 extern PFNGLVERTEXATTRIBL3UI64NVPROC glVertexAttribL3ui64NV;
01312 extern PFNGLVERTEXATTRIBL3UI64VNVPROC glVertexAttribL3ui64vNV;
01313 extern PFNGLVERTEXATTRIBL4DPROC glVertexAttribL4d;
01314 extern PFNGLVERTEXATTRIBL4DVPROC glVertexAttribL4dv;
01315 extern PFNGLVERTEXATTRIBL4I64NVPROC glVertexAttribL4i64NV;
01316 extern PFNGLVERTEXATTRIBL4I64VNVPROC glVertexAttribL4i64vNV;
01317 extern PFNGLVERTEXATTRIBL4UI64NVPROC glVertexAttribL4ui64NV;
01318 extern PFNGLVERTEXATTRIBL4UI64VNVPROC glVertexAttribL4ui64vNV;
01319 extern PFNGLVERTEXATTRIBLFORMATNVPROC glVertexAttribLFormatNV;
01320 extern PFNGLVERTEXATTRIBLFORMATPROC glVertexAttribLFormat;
01321 extern PFNGLVERTEXATTRIBLPOINTERPROC glVertexAttribLPointer;
01322 extern PFNGLVERTEXATTRIBP1UIPROC glVertexAttribP1ui;
01323 extern PFNGLVERTEXATTRIBP1UIVPROC glVertexAttribP1uiv;
01324 extern PFNGLVERTEXATTRIBP2UIPROC glVertexAttribP2ui;
01325 extern PFNGLVERTEXATTRIBP2UIVPROC glVertexAttribP2uiv;
01326 extern PFNGLVERTEXATTRIBP3UIPROC glVertexAttribP3ui;
01327 extern PFNGLVERTEXATTRIBP3UIVPROC glVertexAttribP3uiv;
01328 extern PFNGLVERTEXATTRIBP4UIPROC glVertexAttribP4ui;
01329 extern PFNGLVERTEXATTRIBP4UIVPROC glVertexAttribP4uiv;
01330 extern PFNGLVERTEXATTRIBPOINTERPROC glVertexAttribPointer;
01331 extern PFNGLVERTEXBINDINGDIVISORPROC glVertexBindingDivisor;
01332 extern PFNGLVERTEXFORMATNVPROC glVertexFormatNV;
01333 extern PFNGLVIEWPORTARRAYVPROC glVertexArrayv;
01334 extern PFNGLVIEWPORTINDEXEDFPROC glVertexIndexedf;
01335 extern PFNGLVIEWPORTINDEXEDFVPROC glVertexIndexedfv;
01336 extern PFNGLVIEWPORTPOSITIONWSALENVPROC glVertexPositionWScaleNV;
01337 extern PFNGLVIEWPORTPROC glVertex;
01338 extern PFNGLVIEWPORTSWIZZLENVPROC glVertexSwizzleNV;
01339 extern PFNGLWAITSYNCPROC glWaitSync;
01340 extern PFNGLWAITVKSEMAPHORENVPROC glWaitVkSemaphoreNV;
01341 extern PFNGLWEIGHTPATHSNVPROC glWeightPathsNV;
01342 extern PFNGLWINDOWRECTANGLESEXTPROC glWindowRectanglesEXT;
01343 #endif
01344
01346
01350 namespace gg
01351 {
01355     enum BindingPoints
01356     {
01357         LightBindingPoint = 0,
01358         MaterialBindingPoint,
01359     };
01360
01364     extern GLint ggBufferAlignment;
01365
01372     extern void ggInit();
01373
01383     extern void _ggError(const std::string& name = "", unsigned int line = 0);
01384
01395 #if defined(DEBUG)
01396 #    define ggError() gg::_ggError(__FILE__, __LINE__)
01397 #else

```

```

01398 # define ggError()
01399 #endif
01400
01410 extern void _ggFBOError(const std::string& name = "", unsigned int line = 0);
01411
01422 #if defined(DEBUG)
01423 # define ggFBOError() gg::_ggFBOError(__FILE__, __LINE__)
01424 #else
01425 # define ggFBOError()
01426 #endif
01427
01435 inline void ggCross(GLfloat* c, const GLfloat* a, const GLfloat* b)
01436 {
01437     c[0] = a[1] * b[2] - a[2] * b[1];
01438     c[1] = a[2] * b[0] - a[0] * b[2];
01439     c[2] = a[0] * b[1] - a[1] * b[0];
01440 }
01441
01449 inline GLfloat ggDot3(const GLfloat* a, const GLfloat* b)
01450 {
01451     return a[0] * b[0] + a[1] * b[1] + a[2] * b[2];
01452 }
01453
01460 inline GLfloat ggLength3(const GLfloat* a)
01461 {
01462     return sqrtf(ggDot3(a, a));
01463 }
01464
01472 inline GLfloat ggDistance3(const GLfloat* a, const GLfloat* b)
01473 {
01474     const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], 0.0f };
01475     return ggLength3(c);
01476 }
01477
01484 inline void ggNormalize3(const GLfloat* a, GLfloat* b)
01485 {
01486     const GLfloat l { ggLength3(a) };
01487     assert(l > 0.0f);
01488     b[0] = a[0] / l;
01489     b[1] = a[1] / l;
01490     b[2] = a[2] / l;
01491 }
01492
01498 inline void ggNormalize3(GLfloat* a)
01499 {
01500     const GLfloat l { ggLength3(a) };
01501     assert(l > 0.0f);
01502     a[0] /= l;
01503     a[1] /= l;
01504     a[2] /= l;
01505 }
01506
01514 inline GLfloat ggDot4(const GLfloat* a, const GLfloat* b)
01515 {
01516     return a[0] * b[0] + a[1] * b[1] + a[2] * b[2] + a[3] * b[3];
01517 }
01518
01525 inline GLfloat ggLength4(const GLfloat* a)
01526 {
01527     return sqrtf(ggDot4(a, a));
01528 }
01529
01537 inline GLfloat ggDistance4(const GLfloat* a, const GLfloat* b)
01538 {
01539     const GLfloat c[] { a[0] - b[0], a[1] - b[1], a[2] - b[2], a[3] - b[3] };
01540     return ggLength4(c);
01541 }
01542
01549 inline void ggNormalize4(const GLfloat* a, GLfloat* b)
01550 {
01551     const GLfloat l { ggLength4(a) };
01552     assert(l > 0.0f);
01553     b[0] = a[0] / l;
01554     b[1] = a[1] / l;
01555     b[2] = a[2] / l;
01556     b[3] = a[3] / l;
01557 }
01558
01564 inline void ggNormalize4(GLfloat* a)
01565 {
01566     const GLfloat l { ggLength4(a) };
01567     assert(l > 0.0f);
01568     a[0] /= l;
01569     a[1] /= l;
01570     a[2] /= l;
01571     a[3] /= l;
01572 }

```

```

01573
01577 class GgVector : public std::array<GLfloat, 4>
01578 {
01579 public:
01580
01584     GgVector() = default;
01585
01594     constexpr GgVector(GLfloat v0, GLfloat v1, GLfloat v2, GLfloat v3) :
01595         std::array<GLfloat, 4>{ v0, v1, v2, v3 }
01596     {
01597     }
01598
01604     constexpr GgVector(const GLfloat* a) :
01605         GgVector{ a[0], a[1], a[2], a[3] }
01606     {
01607     }
01608
01614     constexpr GgVector(GLfloat c) :
01615         GgVector{ c, c, c, c }
01616     {
01617     }
01618
01624     GgVector(const GgVector& v) = default;
01625
01631     GgVector(GgVector&& v) = default;
01632
01636     ~GgVector() = default;
01637
01641     GgVector& operator=(const GgVector& v) = default;
01642
01646     GgVector& operator=(GgVector&& v) = default;
01647
01654     constexpr GgVector operator+(const GgVector& v) const noexcept
01655     {
01656         return GgVector{ (*this)[0] + v[0], (*this)[1] + v[1], (*this)[2] + v[2], (*this)[3] + v[3] };
01657     }
01658
01665     constexpr GgVector operator+(GLfloat c) const noexcept
01666     {
01667         return GgVector{ (*this)[0] + c, (*this)[1] + c, (*this)[2] + c, (*this)[3] + c };
01668     }
01669
01676     constexpr GgVector& operator+=(const GgVector& v) noexcept
01677     {
01678         (*this)[0] += v[0];
01679         (*this)[1] += v[1];
01680         (*this)[2] += v[2];
01681         (*this)[3] += v[3];
01682         return *this;
01683     }
01684
01691     constexpr GgVector& operator+=(GLfloat c) noexcept
01692     {
01693         (*this)[0] += c;
01694         (*this)[1] += c;
01695         (*this)[2] += c;
01696         (*this)[3] += c;
01697         return *this;
01698     }
01699
01706     constexpr GgVector operator-(const GgVector& v) const noexcept
01707     {
01708         return GgVector{ (*this)[0] - v[0], (*this)[1] - v[1], (*this)[2] - v[2], (*this)[3] - v[3] };
01709     }
01710
01717     constexpr GgVector operator-(GLfloat c) const noexcept
01718     {
01719         return GgVector{ (*this)[0] - c, (*this)[1] - c, (*this)[2] - c, (*this)[3] - c };
01720     }
01721
01728     constexpr GgVector& operator-=(const GgVector& v) noexcept
01729     {
01730         (*this)[0] -= v[0];
01731         (*this)[1] -= v[1];
01732         (*this)[2] -= v[2];
01733         (*this)[3] -= v[3];
01734         return *this;
01735     }
01736
01743     constexpr GgVector& operator-=(GLfloat c) noexcept
01744     {
01745         (*this)[0] -= c;
01746         (*this)[1] -= c;
01747         (*this)[2] -= c;
01748         (*this)[3] -= c;
01749         return *this;
01750     }

```

```

01751
01758 constexpr GgVector operator*(const GgVector& v) const noexcept
01759 {
01760     return GgVector{ (*this)[0] * v[0], (*this)[1] * v[1], (*this)[2] * v[2], (*this)[3] * v[3] };
01761 }
01762
01769 constexpr GgVector operator*(GLfloat c) const noexcept
01770 {
01771     return GgVector{ (*this)[0] * c, (*this)[1] * c, (*this)[2] * c, (*this)[3] * c };
01772 }
01773
01780 constexpr GgVector& operator*=(const GgVector& v) noexcept
01781 {
01782     (*this)[0] *= v[0];
01783     (*this)[1] *= v[1];
01784     (*this)[2] *= v[2];
01785     (*this)[3] *= v[3];
01786     return *this;
01787 }
01788
01795 constexpr GgVector& operator*=(GLfloat c) noexcept
01796 {
01797     (*this)[0] *= c;
01798     (*this)[1] *= c;
01799     (*this)[2] *= c;
01800     (*this)[3] *= c;
01801     return *this;
01802 }
01803
01810 constexpr GgVector operator/(const GgVector& v) const noexcept
01811 {
01812     return GgVector{ (*this)[0] / v[0], (*this)[1] / v[1], (*this)[2] / v[2], (*this)[3] / v[3] };
01813 }
01814
01821 constexpr GgVector operator/(GLfloat c) const noexcept
01822 {
01823     return GgVector{ (*this)[0] / c, (*this)[1] / c, (*this)[2] / c, (*this)[3] / c };
01824 }
01825
01832 constexpr GgVector& operator/=(const GgVector& v) noexcept
01833 {
01834     (*this)[0] /= v[0];
01835     (*this)[1] /= v[1];
01836     (*this)[2] /= v[2];
01837     (*this)[3] /= v[3];
01838     return *this;
01839 }
01840
01847 constexpr GgVector& operator/=(GLfloat c) noexcept
01848 {
01849     (*this)[0] /= c;
01850     (*this)[1] /= c;
01851     (*this)[2] /= c;
01852     (*this)[3] /= c;
01853     return *this;
01854 }
01855
01862 inline GLfloat dot3(const GgVector& v) const
01863 {
01864     return ggDot3(data(), v.data());
01865 }
01866
01872 inline GLfloat length3() const
01873 {
01874     return ggLength3(data());
01875 }
01876
01883 inline GLfloat distance3(const GgVector& v) const
01884 {
01885     return ggDistance3(data(), v.data());
01886 }
01887
01893 inline GgVector normalize3() const
01894 {
01895     GgVector b;
01896     ggNormalize3(data(), b.data());
01897     return b;
01898 }
01899
01906 inline GLfloat dot4(const GgVector& v) const
01907 {
01908     return ggDot4(data(), v.data());
01909 }
01910
01916 inline GLfloat length4() const
01917 {
01918     return ggLength4(data());

```



```

01919     }
01920
01927     inline GLfloat distance4(const GgVector& v) const
01928     {
01929         return ggDistance4(data(), v.data());
01930     }
01931
01937     inline GgVector normalize4() const
01938     {
01939         GgVector b;
01940         ggNormalize4(data(), b.data());
01941         return b;
01942     }
01943 };
01944
01951     inline const GgVector& operator+(const GgVector& v)
01952     {
01953         return v;
01954     }
01955
01963     inline GgVector operator+(GLfloat a, const GgVector& b)
01964     {
01965         return GgVector{ a + b[0], a + b[1], a + b[2], a + b[3] };
01966     }
01967
01974     inline const GgVector operator-(const GgVector& v)
01975     {
01976         return GgVector{ -v[0], -v[1], -v[2], -v[3] };
01977     }
01978
01986     inline GgVector operator-(GLfloat a, const GgVector& b)
01987     {
01988         return GgVector{ a - b[0], a - b[1], a - b[2], a - b[3] };
01989     }
01990
01998     inline GgVector operator*(GLfloat a, const GgVector& b)
01999     {
02000         return GgVector{ a * b[0], a * b[1], a * b[2], a * b[3] };
02001     }
02002
02010     inline GgVector operator/(GLfloat a, const GgVector& b)
02011     {
02012         return GgVector{ a / b[0], a / b[1], a / b[2], a / b[3] };
02013     }
02014
02025     inline GgVector ggCross(const GgVector& a, const GgVector& b)
02026     {
02027         GgVector c;
02028         ggCross(c.data(), a.data(), b.data());
02029         return c;
02030     }
02031
02039     inline GLfloat ggDot3(const GgVector& a, const GgVector& b)
02040     {
02041         return ggDot3(a.data(), b.data());
02042     }
02043
02050     inline GLfloat ggLength3(const GgVector& a)
02051     {
02052         return ggLength3(a.data());
02053     }
02054
02062     inline GLfloat ggDistance3(const GgVector& a, const GgVector& b)
02063     {
02064         return ggDistance3(a.data(), b.data());
02065     }
02066
02076     inline GgVector ggNormalize3(const GgVector& a)
02077     {
02078         GgVector b;
02079         ggNormalize3(a.data(), b.data());
02080         return b;
02081     }
02082
02091     inline void ggNormalize3(GgVector* a)
02092     {
02093         ggNormalize3(a->data());
02094     }
02095
02103     inline GLfloat ggDot4(const GgVector& a, const GgVector& b)
02104     {
02105         return ggDot4(a.data(), b.data());
02106     }
02107
02114     inline GLfloat ggLength4(const GgVector& a)
02115     {
02116         return ggLength4(a.data());

```

```

02117     }
02118
02126 inline GLfloat ggDistance4(const GgVector& a, const GgVector& b)
02127 {
02128     return ggDistance4(a.data(), b.data());
02129 }
02130
02137 inline GgVector ggNormalize4(const GgVector& a)
02138 {
02139     GgVector b;
02140     ggNormalize4(a.data(), b.data());
02141     return b;
02142 }
02143
02149 inline void ggNormalize4(GgVector* a)
02150 {
02151     ggNormalize4(a->data());
02152 }
02153
02157 class GgMatrix : public std::array<GLfloat, 16>
02158 {
02159     // 行列 a とベクトル b の積をベクトル c に格納する
02160     void projection(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02161
02162     // 行列 a と行列 b の積を行列 c に格納する
02163     void multiply(GLfloat* c, const GLfloat* a, const GLfloat* b) const;
02164
02165 public:
02166
02170     GgMatrix() = default;
02171
02192     constexpr GgMatrix(
02193         GLfloat m00, GLfloat m01, GLfloat m02, GLfloat m03,
02194         GLfloat m10, GLfloat m11, GLfloat m12, GLfloat m13,
02195         GLfloat m20, GLfloat m21, GLfloat m22, GLfloat m23,
02196         GLfloat m30, GLfloat m31, GLfloat m32, GLfloat m33
02197     ) :
02198         std::array<GLfloat, 16>{ m00, m01, m02, m03, m10, m11, m12, m13, m20, m21, m22, m23, m30, m31,
02199             m32, m33 }
02200     {
02201
02207     constexpr GgMatrix(const GLfloat* a) :
02208         GgMatrix{ a[0], a[1], a[2], a[3], a[4], a[5], a[6], a[7], a[8], a[9], a[10], a[11], a[12],
02209             a[13], a[14], a[15] }
02210     {
02211
02217     constexpr GgMatrix(GLfloat c) :
02218         GgMatrix{ c, c, c, c, c, c, c, c, c, c, c, c, c, c, c, c }
02219     {
02220     }
02221
02227     GgMatrix(const GgMatrix& m) = default;
02228
02234     GgMatrix(GgMatrix&& m) = default;
02235
02239     ~GgMatrix() = default;
02240
02244     GgMatrix& operator=(const GgMatrix& m) = default;
02245
02249     GgMatrix& operator=(GgMatrix&& m) = default;
02250
02257     GgMatrix& operator=(const GLfloat* a)
02258     {
02259         *this = GgMatrix(a);
02260         return *this;
02261     }
02262
02269     GgMatrix operator+(const GLfloat* a) const
02270     {
02271         GgMatrix t;
02272         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] + a[i];
02273         return t;
02274     }
02275
02282     GgMatrix operator+(const GgMatrix& m) const
02283     {
02284         return operator+(m.data());
02285     }
02286
02293     GgMatrix& operator+=(const GLfloat* a)
02294     {
02295         for (std::size_t i = 0; i < size(); ++i) data()[i] += a[i];
02296         return *this;
02297     }
02298

```

```

02305     GgMatrix& operator+=(const GgMatrix& m)
02306     {
02307         return operator+=(m.data());
02308     }
02309
02316     GgMatrix operator-(const GLfloat* a) const
02317     {
02318         GgMatrix t;
02319         for (std::size_t i = 0; i < size(); ++i) t[i] = data()[i] - a[i];
02320         return t;
02321     }
02322
02329     GgMatrix operator-(const GgMatrix& m) const
02330     {
02331         return operator-(m.data());
02332     }
02333
02340     GgMatrix& operator-=(const GLfloat* a)
02341     {
02342         for (std::size_t i = 0; i < size(); ++i) data()[i] -= a[i];
02343         return *this;
02344     }
02345
02352     GgMatrix& operator-=(const GgMatrix& m)
02353     {
02354         return operator-=(m.data());
02355     }
02356
02363     GgMatrix operator*(const GLfloat* a) const
02364     {
02365         GgMatrix t;
02366         multiply(t.data(), data(), a);
02367         return t;
02368     }
02369
02376     GgMatrix operator*(const GgMatrix& m) const
02377     {
02378         return operator*(m.data());
02379     }
02380
02387     GgMatrix& operator*=(const GLfloat* a)
02388     {
02389         *this = operator*(a);
02390         return *this;
02391     }
02392
02399     GgMatrix& operator*=(const GgMatrix& m)
02400     {
02401         return operator*=(m.data());
02402     }
02403
02410     GgMatrix operator/(const GLfloat* a) const
02411     {
02412         GgMatrix i, t;
02413         i.loadInvert(a);
02414         multiply(t.data(), data(), i.data());
02415         return t;
02416     }
02417
02424     GgMatrix operator/(const GgMatrix& m) const
02425     {
02426         return operator/(m.data());
02427     }
02428
02435     GgMatrix& operator/=(const GLfloat* a)
02436     {
02437         *this = operator/(a);
02438         return *this;
02439     }
02440
02447     GgMatrix& operator/=(const GgMatrix& m)
02448     {
02449         return operator/=(m.data());
02450     }
02451
02455     GgMatrix& loadIdentity();
02456
02466     GgMatrix& loadTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02467
02474     GgMatrix& loadTranslate(const GLfloat* t)
02475     {
02476         return loadTranslate(t[0], t[1], t[2]);
02477     }
02478
02485     GgMatrix& loadTranslate(const GgVector& t)
02486     {
02487         return loadTranslate(t[0], t[1], t[2], t[3]);

```

```

02488     }
02489
02499     GgMatrix& loadScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f);
02500
02507     GgMatrix& loadScale(const GLfloat* s)
02508     {
02509         return loadScale(s[0], s[1], s[2]);
02510     }
02511
02518     GgMatrix& loadScale(const GgVector& s)
02519     {
02520         return loadScale(s[0], s[1], s[2], s[3]);
02521     }
02522
02529     GgMatrix& loadRotateX(GLfloat a);
02530
02537     GgMatrix& loadRotateY(GLfloat a);
02538
02545     GgMatrix& loadRotateZ(GLfloat a);
02546
02556     GgMatrix& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
02557
02565     GgMatrix& loadRotate(const GLfloat* r, GLfloat a)
02566     {
02567         return loadRotate(r[0], r[1], r[2], a);
02568     }
02569
02577     GgMatrix& loadRotate(const GgVector& r, GLfloat a)
02578     {
02579         return loadRotate(r[0], r[1], r[2], a);
02580     }
02581
02588     GgMatrix& loadRotate(const GLfloat* r)
02589     {
02590         return loadRotate(r[0], r[1], r[2], r[3]);
02591     }
02592
02599     GgMatrix& loadRotate(const GgVector& r)
02600     {
02601         return loadRotate(r[0], r[1], r[2], r[3]);
02602     }
02603
02618     GgMatrix& loadLookat(GLfloat ex, GLfloat ey, GLfloat ez,
02619         GLfloat tx, GLfloat ty, GLfloat tz,
02620         GLfloat ux, GLfloat uy, GLfloat uz);
02621
02630     GgMatrix& loadLookat(const GLfloat* e, const GLfloat* t, const GLfloat* u)
02631     {
02632         return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02633     }
02634
02643     GgMatrix& loadLookat(const GgVector& e, const GgVector& t, const GgVector& u)
02644     {
02645         return loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02646     }
02647
02659     GgMatrix& loadOrthogonal(GLfloat left, GLfloat right,
02660         GLfloat bottom, GLfloat top,
02661         GLfloat zNear, GLfloat zFar);
02662
02674     GgMatrix& loadFrustum(GLfloat left, GLfloat right,
02675         GLfloat bottom, GLfloat top,
02676         GLfloat zNear, GLfloat zFar);
02677
02687     GgMatrix& loadPerspective(GLfloat fovy, GLfloat aspect,
02688         GLfloat zNear, GLfloat zFar);
02689
02696     GgMatrix& loadTranspose(const GLfloat* a);
02697
02704     GgMatrix& loadTranspose(const GgMatrix& m)
02705     {
02706         return loadTranspose(m.data());
02707     }
02708
02715     GgMatrix& loadInvert(const GLfloat* a);
02716
02723     GgMatrix& loadInvert(const GgMatrix& m)
02724     {
02725         return loadInvert(m.data());
02726     }
02727
02734     GgMatrix& loadNormal(const GLfloat* a);
02735
02742     GgMatrix& loadNormal(const GgMatrix& m)
02743     {
02744         return loadNormal(m.data());
02745     }

```

```

02746
02756 GgMatrix translate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02757 {
02758     GgMatrix m;
02759     return operator*(m.loadTranslate(x, y, z, w));
02760 }
02761
02768 GgMatrix translate(const GLfloat* t) const
02769 {
02770     return translate(t[0], t[1], t[2]);
02771 }
02772
02779 GgMatrix translate(const GgVector& t) const
02780 {
02781     return translate(t[0], t[1], t[2], t[3]);
02782 }
02783
02793 GgMatrix scale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f) const
02794 {
02795     GgMatrix m;
02796     return operator*(m.loadScale(x, y, z, w));
02797 }
02798
02805 GgMatrix scale(const GLfloat* s) const
02806 {
02807     return scale(s[0], s[1], s[2]);
02808 }
02809
02816 GgMatrix scale(const GgVector& s) const
02817 {
02818     return scale(s[0], s[1], s[2], s[3]);
02819 }
02820
02827 GgMatrix rotateX(GLfloat a) const
02828 {
02829     GgMatrix m;
02830     return operator*(m.loadRotateX(a));
02831 }
02832
02839 GgMatrix rotateY(GLfloat a) const
02840 {
02841     GgMatrix m;
02842     return operator*(m.loadRotateY(a));
02843 }
02844
02851 GgMatrix rotateZ(GLfloat a) const
02852 {
02853     GgMatrix m;
02854     return operator*(m.loadRotateZ(a));
02855 }
02856
02866 GgMatrix rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
02867 {
02868     GgMatrix m;
02869     return operator*(m.loadRotate(x, y, z, a));
02870 }
02871
02879 GgMatrix rotate(const GLfloat* r, GLfloat a) const
02880 {
02881     return rotate(r[0], r[1], r[2], a);
02882 }
02883
02891 GgMatrix rotate(const GgVector& r, GLfloat a) const
02892 {
02893     return rotate(r[0], r[1], r[2], a);
02894 }
02895
02902 GgMatrix rotate(const GLfloat* r) const
02903 {
02904     return rotate(r[0], r[1], r[2], r[3]);
02905 }
02906
02913 GgMatrix rotate(const GgVector& r) const
02914 {
02915     return rotate(r[0], r[1], r[2], r[3]);
02916 }
02917
02932 GgMatrix lookat(
02933     GLfloat ex, GLfloat ey, GLfloat ez,
02934     GLfloat tx, GLfloat ty, GLfloat tz,
02935     GLfloat ux, GLfloat uy, GLfloat uz
02936 ) const
02937 {
02938     GgMatrix m;
02939     return operator*(m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz));
02940 }
02941

```

```

02950 GgMatrix lookat(const GLfloat* e, const GLfloat* t, const GLfloat* u) const
02951 {
02952     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02953 }
02954
02963 GgMatrix lookat(const GgVector& e, const GgVector& t, const GgVector& u) const
02964 {
02965     return lookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
02966 }
02967
02979 GgMatrix orthogonal(
02980     GLfloat left, GLfloat right,
02981     GLfloat bottom, GLfloat top,
02982     GLfloat zNear, GLfloat zFar
02983 ) const
02984 {
02985     GgMatrix m;
02986     return operator*(m.loadOrthogonal(left, right, bottom, top, zNear, zFar));
02987 }
02988
03000 GgMatrix frustum(
03001     GLfloat left, GLfloat right,
03002     GLfloat bottom, GLfloat top,
03003     GLfloat zNear, GLfloat zFar
03004 ) const
03005 {
03006     GgMatrix m;
03007     return operator*(m.loadFrustum(left, right, bottom, top, zNear, zFar));
03008 }
03009
03019 GgMatrix perspective(
03020     GLfloat fovy, GLfloat aspect,
03021     GLfloat zNear, GLfloat zFar
03022 ) const
03023 {
03024     GgMatrix m;
03025     return operator*(m.loadPerspective(fovy, aspect, zNear, zFar));
03026 }
03027
03033 GgMatrix transpose() const
03034 {
03035     GgMatrix t;
03036     return t.loadTranspose(*this);
03037 }
03038
03044 GgMatrix invert() const
03045 {
03046     GgMatrix t;
03047     return t.loadInvert(*this);
03048 }
03049
03055 GgMatrix normal() const
03056 {
03057     GgMatrix t;
03058     return t.loadNormal(*this);
03059 }
03060
03067 void projection(GLfloat* c, const GLfloat* v) const
03068 {
03069     projection(c, data(), v);
03070 }
03071
03078 void projection(GLfloat* c, const GgVector& v) const
03079 {
03080     projection(c, v.data());
03081 }
03082
03089 void projection(GgVector& c, const GLfloat* v) const
03090 {
03091     projection(c.data(), v);
03092 }
03093
03100 void projection(GgVector& c, const GgVector& v) const
03101 {
03102     projection(c.data(), v.data());
03103 }
03104
03111 GgVector operator*(const GgVector& v) const
03112 {
03113     GgVector c;
03114     projection(c, v);
03115     return c;
03116 }
03117
03123 const GLfloat* get() const
03124 {
03125     return data();

```

```

03126     }
03127
03133     void get(GLfloat* a) const
03134     {
03135         std::copy(data(), data() + size(), a);
03136     }
03137 };
03138
03144 inline GgMatrix ggIdentity()
03145 {
03146     GgMatrix t;
03147     return t.loadIdentity();
03148 }
03149
03159 inline GgMatrix ggTranslate(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03160 {
03161     GgMatrix m;
03162     return m.loadTranslate(x, y, z, w);
03163 }
03164
03171 inline GgMatrix ggTranslate(const GLfloat* t)
03172 {
03173     GgMatrix m;
03174     return m.loadTranslate(t[0], t[1], t[2]);
03175 }
03176
03183 inline GgMatrix ggTranslate(const GgVector& t)
03184 {
03185     GgMatrix m;
03186     return m.loadTranslate(t[0], t[1], t[2], t[3]);
03187 }
03188
03198 inline GgMatrix ggScale(GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f)
03199 {
03200     GgMatrix m;
03201     return m.loadScale(x, y, z, w);
03202 }
03203
03210 inline GgMatrix ggScale(const GLfloat* s)
03211 {
03212     GgMatrix m;
03213     return m.loadScale(s[0], s[1], s[2]);
03214 }
03215
03222 inline GgMatrix ggScale(const GgVector& s)
03223 {
03224     GgMatrix m;
03225     return m.loadScale(s[0], s[1], s[2], s[3]);
03226 }
03227
03234 inline GgMatrix ggRotateX(GLfloat a)
03235 {
03236     GgMatrix m;
03237     return m.loadRotateX(a);
03238 }
03239
03246 inline GgMatrix ggRotateY(GLfloat a)
03247 {
03248     GgMatrix m;
03249     return m.loadRotateY(a);
03250 }
03251
03258 inline GgMatrix ggRotateZ(GLfloat a)
03259 {
03260     GgMatrix m;
03261     return m.loadRotateZ(a);
03262 }
03263
03273 inline GgMatrix ggRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
03274 {
03275     GgMatrix m;
03276     return m.loadRotate(x, y, z, a);
03277 }
03278
03286 inline GgMatrix ggRotate(const GLfloat* r, GLfloat a)
03287 {
03288     GgMatrix m;
03289     return m.loadRotate(r[0], r[1], r[2], a);
03290 }
03291
03299 inline GgMatrix ggRotate(const GgVector& r, GLfloat a)
03300 {
03301     GgMatrix m;
03302     return m.loadRotate(r[0], r[1], r[2], a);
03303 }
03304
03311 inline GgMatrix ggRotate(const GLfloat* r)

```

```

03312 {
03313     GgMatrix m;
03314     return m.loadRotate(r[0], r[1], r[2], r[3]);
03315 }
03316
03323 inline GgMatrix ggRotate(const GgVector& r)
03324 {
03325     GgMatrix m;
03326     return m.loadRotate(r[0], r[1], r[2], r[3]);
03327 }
03328
03343 inline GgMatrix ggLookat(
03344     GLfloat ex, GLfloat ey, GLfloat ez,    // 視点の位置
03345     GLfloat tx, GLfloat ty, GLfloat tz,    // 目標点の位置
03346     GLfloat ux, GLfloat uy, GLfloat uz     // 上方向のベクトル
03347 )
03348 {
03349     GgMatrix m;
03350     return m.loadLookat(ex, ey, ez, tx, ty, tz, ux, uy, uz);
03351 }
03352
03361 inline GgMatrix ggLookat(
03362     const GLfloat* e,                      // 視点の位置
03363     const GLfloat* t,                      // 目標点の位置
03364     const GLfloat* u                      // 上方向のベクトル
03365 )
03366 {
03367     GgMatrix m;
03368     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03369 }
03370
03379 inline GgMatrix ggLookat(
03380     const GgVector& e,                      // 視点の位置
03381     const GgVector& t,                      // 目標点の位置
03382     const GgVector& u                      // 上方向のベクトル
03383 )
03384 {
03385     GgMatrix m;
03386     return m.loadLookat(e[0], e[1], e[2], t[0], t[1], t[2], u[0], u[1], u[2]);
03387 }
03388
03400 inline GgMatrix ggOrthogonal(GLfloat left, GLfloat right,
03401     GLfloat bottom, GLfloat top,
03402     GLfloat zNear, GLfloat zFar)
03403 {
03404     GgMatrix m;
03405     return m.loadOrthogonal(left, right, bottom, top, zNear, zFar);
03406 }
03407
03419 inline GgMatrix ggFrustum(
03420     GLfloat left, GLfloat right,
03421     GLfloat bottom, GLfloat top,
03422     GLfloat zNear, GLfloat zFar
03423 )
03424 {
03425     GgMatrix m;
03426     return m.loadFrustum(left, right, bottom, top, zNear, zFar);
03427 }
03428
03438 inline GgMatrix ggPerspective(
03439     GLfloat fovy, GLfloat aspect,
03440     GLfloat zNear, GLfloat zFar
03441 )
03442 {
03443     GgMatrix m;
03444     return m.loadPerspective(fovy, aspect, zNear, zFar);
03445 }
03446
03453 inline GgMatrix ggTranspose(const GgMatrix& m)
03454 {
03455     return m.transpose();
03456 }
03457
03464 inline GgMatrix ggInvert(const GgMatrix& m)
03465 {
03466     return m.invert();
03467 }
03468
03475 inline GgMatrix ggNormal(const GgMatrix& m)
03476 {
03477     return m.normal();
03478 }
03479
03483 class GgQuaternion : public GgVector
03484 {
03485     // GgQuaternion 型の四元数 p と四元数 q の積を四元数 r に求める
03486     void multiply(GLfloat* r, const GLfloat* p, const GLfloat* q) const;

```



```

03487
03488 // GgQuaternion 型の四元数 q が表す回転の変換行列を m に求める
03489 void toMatrix(GLfloat* m, const GLfloat* q) const;
03490
03491 // 回転の変換行列 m が表す四元数を q に求める
03492 void toQuaternion(GLfloat* q, const GLfloat* m) const;
03493
03494 // 球面線形補間 q と r を t で補間した四元数を p に求める
03495 void slerp(GLfloat* p, const GLfloat* q, const GLfloat* r, GLfloat t) const;
03496
03497 public:
03498
03502 GgQuaternion() = default;
03503
03512 constexpr GgQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat w) :
03513     GgVector{ x, y, z, w }
03514 {
03515 }
03516
03522 constexpr GgQuaternion(GLfloat c) :
03523     GgQuaternion{ c, c, c, c }
03524 {
03525 }
03526
03532 GgQuaternion(const GLfloat* a) :
03533     GgQuaternion{ a[0], a[1], a[2], a[3] }
03534 {
03535 }
03536
03542 GgQuaternion(const GgVector& v) :
03543     GgQuaternion{ v[0], v[1], v[2], v[3] }
03544 {
03545 }
03546
03552 GgQuaternion(const GgQuaternion& q) = default;
03553
03559 GgQuaternion(GgQuaternion&& q) = default;
03560
03564 ~GgQuaternion() = default;
03565
03572 GgQuaternion& operator=(const GgQuaternion& q) = default;
03573
03580 GgQuaternion& operator=(GgQuaternion&& q) = default;
03581
03587 GLfloat norm() const
03588 {
03589     return gglength4(*this);
03590 }
03591
03601 GgQuaternion& loadAdd(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03602 {
03603     (*this)[0] += x;
03604     (*this)[1] += y;
03605     (*this)[2] += z;
03606     (*this)[3] += w;
03607
03608     return *this;
03609 }
03610
03617 GgQuaternion& loadAdd(const GLfloat* a)
03618 {
03619     return loadAdd(a[0], a[1], a[2], a[3]);
03620 }
03621
03628 GgQuaternion& loadAdd(const GgVector& v)
03629 {
03630     return loadAdd(v[0], v[1], v[2], v[3]);
03631 }
03632
03639 GgQuaternion& loadAdd(const GgQuaternion& q)
03640 {
03641     return loadAdd(q[0], q[1], q[2], q[3]);
03642 }
03643
03653 GgQuaternion& loadSubtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03654 {
03655     (*this)[0] -= x;
03656     (*this)[1] -= y;
03657     (*this)[2] -= z;
03658     (*this)[3] -= w;
03659
03660     return *this;
03661 }
03662
03669 GgQuaternion& loadSubtract(const GLfloat* a)
03670 {
03671     return loadSubtract(a[0], a[1], a[2], a[3]);

```

```

03672     }
03673
03680 GgQuaternion& loadSubtract(const GgVector& v)
03681 {
03682     return loadSubtract(v[0], v[1], v[2], v[3]);
03683 }
03684
03691 GgQuaternion& loadSubtract(const GgQuaternion& q)
03692 {
03693     return loadSubtract(q[0], q[1], q[2], q[3]);
03694 }
03695
03705 GgQuaternion& loadMultiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03706 {
03707     const GLfloat a[]={ x, y, z, w };
03708     return loadMultiply(a);
03709 }
03710
03717 GgQuaternion& loadMultiply(const GLfloat* a)
03718 {
03719     return *this = multiply(a);
03720 }
03721
03728 GgQuaternion& loadMultiply(const GgVector& v)
03729 {
03730     return loadMultiply(v.data());
03731 }
03732
03739 GgQuaternion& loadMultiply(const GgQuaternion& q)
03740 {
03741     return loadMultiply(q.data());
03742 }
03743
03753 GgQuaternion& loadDivide(GLfloat x, GLfloat y, GLfloat z, GLfloat w)
03754 {
03755     const GLfloat a[]={ x, y, z, w };
03756     return loadDivide(a);
03757 }
03758
03765 GgQuaternion& loadDivide(const GLfloat* a)
03766 {
03767     return *this = divide(a);
03768 }
03769
03776 GgQuaternion& loadDivide(const GgVector& v)
03777 {
03778     return loadDivide(v.data());
03779 }
03780
03787 GgQuaternion& loadDivide(const GgQuaternion& q)
03788 {
03789     return loadDivide(q.data());
03790 }
03791
03801 GgQuaternion add(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03802 {
03803     GgQuaternion s{ *this };
03804     return s.loadAdd(x, y, z, w);
03805 }
03806
03813 GgQuaternion add(const GLfloat* a) const
03814 {
03815     return add(a[0], a[1], a[2], a[3]);
03816 }
03817
03824 GgQuaternion add(const GgVector& v) const
03825 {
03826     return add(v[0], v[1], v[2], v[3]);
03827 }
03828
03835 GgQuaternion add(const GgQuaternion& q) const
03836 {
03837     return add(q[0], q[1], q[2], q[3]);
03838 }
03839
03849 GgQuaternion subtract(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03850 {
03851     GgQuaternion s{ *this };
03852     return s.loadSubtract(x, y, z, w);
03853 }
03854
03861 GgQuaternion subtract(const GLfloat* a) const
03862 {
03863     return subtract(a[0], a[1], a[2], a[3]);
03864 }
03865
03872 GgQuaternion subtract(const GgVector& v) const

```

```

03873     {
03874         return subtract(v[0], v[1], v[2], v[3]);
03875     }
03876
03883 GgQuaternion subtract(const GgQuaternion& q) const
03884 {
03885     return subtract(q[0], q[1], q[2], q[3]);
03886 }
03887
03897 GgQuaternion multiply(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03898 {
03899     const GLfloat a[]={ x, y, z, w };
03900     return multiply(a);
03901 }
03902
03909 GgQuaternion multiply(const GLfloat* a) const
03910 {
03911     GgQuaternion s;
03912     multiply(s.data(), data(), a);
03913     return s;
03914 }
03915
03922 GgQuaternion multiply(const GgVector& v) const
03923 {
03924     return multiply(v.data());
03925 }
03926
03933 GgQuaternion multiply(const GgQuaternion& q) const
03934 {
03935     return multiply(q.data());
03936 }
03937
03947 GgQuaternion divide(GLfloat x, GLfloat y, GLfloat z, GLfloat w) const
03948 {
03949     const GLfloat a[]={ x, y, z, w };
03950     return divide(a);
03951 }
03952
03959 GgQuaternion divide(const GLfloat* a) const
03960 {
03961     GgQuaternion s, ia;
03962     ia.loadInvert(a);
03963     multiply(s.data(), data(), ia.data());
03964     return s;
03965 }
03966
03973 GgQuaternion divide(const GgVector& v) const
03974 {
03975     return divide(v.data());
03976 }
03977
03984 GgQuaternion divide(const GgQuaternion& q) const
03985 {
03986     return divide(q.data());
03987 }
03988
03989 // 演算子
03990 GgQuaternion& operator=(const GLfloat* a)
03991 {
03992     return *this = GgQuaternion(a);
03993 }
03994 GgQuaternion& operator=(const GgVector& v)
03995 {
03996     return *this = GgQuaternion(v);
03997 }
03998 GgQuaternion& operator+=(const GLfloat* a)
03999 {
04000     return loadAdd(a);
04001 }
04002 GgQuaternion& operator+=(const GgVector& v)
04003 {
04004     return loadAdd(v);
04005 }
04006 GgQuaternion& operator+=(const GgQuaternion& q)
04007 {
04008     return loadAdd(q);
04009 }
04010 GgQuaternion& operator-=(const GLfloat* a)
04011 {
04012     return loadSubtract(a);
04013 }
04014 GgQuaternion& operator-=(const GgVector& v)
04015 {
04016     return loadSubtract(v);
04017 }
04018 GgQuaternion& operator-=(const GgQuaternion& q)
04019 {

```

```

04020         return operator==(q.data());
04021     }
04022     GgQuaternion& operator*=(const GLfloat* a)
04023     {
04024         return loadMultiply(a);
04025     }
04026     GgQuaternion& operator*=(const GgVector& v)
04027     {
04028         return loadMultiply(v);
04029     }
04030     GgQuaternion& operator*=(const GgQuaternion& q)
04031     {
04032         return operator==(q.data());
04033     }
04034     GgQuaternion& operator/=(const GLfloat* a)
04035     {
04036         return loadDivide(a);
04037     }
04038     GgQuaternion& operator/=(const GgVector& v)
04039     {
04040         return loadDivide(v);
04041     }
04042     GgQuaternion& operator/=(const GgQuaternion& q)
04043     {
04044         return operator/=(q.data());
04045     }
04046     GgQuaternion operator+(const GLfloat* a) const
04047     {
04048         return add(a);
04049     }
04050     GgQuaternion operator+(const GgVector& v) const
04051     {
04052         return add(v);
04053     }
04054     GgQuaternion operator+(const GgQuaternion& q) const
04055     {
04056         return operator+(q.data());
04057     }
04058     GgQuaternion operator-(const GLfloat* a) const
04059     {
04060         return subtract(a);
04061     }
04062     GgQuaternion operator-(const GgVector& v) const
04063     {
04064         return subtract(v);
04065     }
04066     GgQuaternion operator-(const GgQuaternion& q) const
04067     {
04068         return operator-(q.data());
04069     }
04070     GgQuaternion operator*(const GLfloat* a) const
04071     {
04072         return multiply(a);
04073     }
04074     GgQuaternion operator*(const GgVector& v) const
04075     {
04076         return multiply(v);
04077     }
04078     GgQuaternion operator*(const GgQuaternion& q) const
04079     {
04080         return operator*(q.data());
04081     }
04082     GgQuaternion operator/(const GLfloat* a) const
04083     {
04084         return divide(a);
04085     }
04086     GgQuaternion operator/(const GgVector& v) const
04087     {
04088         return divide(v);
04089     }
04090     GgQuaternion operator/(const GgQuaternion& q) const
04091     {
04092         return operator/(q.data());
04093     }
04094     GgQuaternion& loadMatrix(const GLfloat* a)
04102     {
04103         toQuaternion(data(), a);
04104         return *this;
04105     }
04106     GgQuaternion& loadMatrix(const GgMatrix& m)
04114     {
04115         return loadMatrix(m.get());
04116     }
04117     GgQuaternion& loadIdentity()
04123

```

```

04124     {
04125         return *this = GgQuaternion(0.0f, 0.0f, 0.0f, 1.0f);
04126     }
04127
04137     GgQuaternion& loadRotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a);
04138
04146     GgQuaternion& loadRotate(const GLfloat* v, GLfloat a)
04147     {
04148         return loadRotate(v[0], v[1], v[2], a);
04149     }
04150
04157     GgQuaternion& loadRotate(const GLfloat* v)
04158     {
04159         return loadRotate(v[0], v[1], v[2], v[3]);
04160     }
04161
04168     GgQuaternion& loadRotateX(GLfloat a);
04169
04176     GgQuaternion& loadRotateY(GLfloat a);
04177
04184     GgQuaternion& loadRotateZ(GLfloat a);
04185
04195     GgQuaternion rotate(GLfloat x, GLfloat y, GLfloat z, GLfloat a) const
04196     {
04197         GgQuaternion q;
04198         return multiply(q.loadRotate(x, y, z, a));
04199     }
04200
04208     GgQuaternion rotate(const GLfloat* v, GLfloat a) const
04209     {
04210         return rotate(v[0], v[1], v[2], a);
04211     }
04212
04219     GgQuaternion rotate(const GLfloat* v) const
04220     {
04221         return rotate(v[0], v[1], v[2], v[3]);
04222     }
04223
04230     GgQuaternion rotateX(GLfloat a) const
04231     {
04232         return rotate(1.0f, 0.0f, 0.0f, a);
04233     }
04234
04241     GgQuaternion rotateY(GLfloat a) const
04242     {
04243         return rotate(0.0f, 1.0f, 0.0f, a);
04244     }
04245
04252     GgQuaternion rotateZ(GLfloat a) const
04253     {
04254         return rotate(0.0f, 0.0f, 1.0f, a);
04255     }
04256
04265     GgQuaternion& loadEuler(GLfloat heading, GLfloat pitch, GLfloat roll);
04266
04273     GgQuaternion& loadEuler(const GLfloat* e)
04274     {
04275         return loadEuler(e[0], e[1], e[2]);
04276     }
04277
04286     GgQuaternion euler(GLfloat heading, GLfloat pitch, GLfloat roll) const
04287     {
04288         GgQuaternion r;
04289         return multiply(r.loadEuler(heading, pitch, roll));
04290     }
04291
04298     GgQuaternion euler(const GLfloat* e) const
04299     {
04300         return euler(e[0], e[1], e[2]);
04301     }
04302
04312     GgQuaternion euler(const GgVector& e) const
04313     {
04314         return euler(e[0], e[1], e[2]);
04315     }
04316
04325     GgQuaternion& loadSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04326     {
04327         slerp(data(), a, b, t);
04328         return *this;
04329     }
04330
04339     GgQuaternion& loadSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04340     {
04341         return loadSlerp(q.data(), r.data(), t);
04342     }
04343

```

```

04352 GgQuaternion& loadSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04353 {
04354     return loadSlerp(q.data(), a, t);
04355 }
04356
04365 GgQuaternion& loadSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04366 {
04367     return loadSlerp(a, q.data(), t);
04368 }
04369
04376 GgQuaternion& loadNormalize(const GLfloat* a);
04377
04384 GgQuaternion& loadNormalize(const GgQuaternion& q)
04385 {
04386     return loadNormalize(q.data());
04387 }
04388
04395 GgQuaternion& loadConjugate(const GLfloat* a);
04396
04403 GgQuaternion& loadConjugate(const GgQuaternion& q)
04404 {
04405     return loadConjugate(q.data());
04406 }
04407
04414 GgQuaternion& loadInvert(const GLfloat* a);
04415
04422 GgQuaternion& loadInvert(const GgQuaternion& q)
04423 {
04424     return loadInvert(q.data());
04425 }
04426
04434 GgQuaternion slerp(GLfloat* a, GLfloat t) const
04435 {
04436     GgQuaternion p;
04437     slerp(p.data(), data(), a, t);
04438     return p;
04439 }
04440
04448 GgQuaternion slerp(const GgQuaternion& q, GLfloat t) const
04449 {
04450     GgQuaternion p;
04451     slerp(p.data(), data(), q.data(), t);
04452     return p;
04453 }
04454
04460 GgQuaternion normalize() const
04461 {
04462     GgQuaternion q;
04463     q.loadNormalize(data());
04464     return q;
04465 }
04466
04472 GgQuaternion conjugate() const
04473 {
04474     GgQuaternion q;
04475     q.loadConjugate(data());
04476     return q;
04477 }
04478
04484 GgQuaternion invert() const
04485 {
04486     GgQuaternion q;
04487     q.loadInvert(data());
04488     return q;
04489 }
04490
04496 void get(GLfloat* a) const
04497 {
04498     a[0] = data()[0];
04499     a[1] = data()[1];
04500     a[2] = data()[2];
04501     a[3] = data()[3];
04502 }
04503
04509 void getMatrix(GLfloat* a) const
04510 {
04511     toMatrix(a, data());
04512 }
04513
04519 void getMatrix(GgMatrix& m) const
04520 {
04521     getMatrix(m.data());
04522 }
04523
04529 GgMatrix getMatrix() const
04530 {
04531     GgMatrix m;

```

```

04532     getMatrix(m);
04533     return m;
04534 }
04535
04541 void getConjugateMatrix(GLfloat* a) const
04542 {
04543     GgQuaternion c;
04544     c.loadConjugate(data());
04545     toMatrix(a, c.data());
04546 }
04547
04553 void getConjugateMatrix(GgMatrix& m) const
04554 {
04555     getConjugateMatrix(m.data());
04556 }
04557
04563 GgMatrix getConjugateMatrix() const
04564 {
04565     GgMatrix m;
04566     getConjugateMatrix(m);
04567     return m;
04568 }
04569 };
04570
04576 inline GgQuaternion ggIdentityQuaternion()
04577 {
04578     GgQuaternion q;
04579     return q.loadIdentity();
04580 }
04581
04588 inline GgQuaternion ggMatrixQuaternion(const GLfloat* a)
04589 {
04590     GgQuaternion q;
04591     return q.loadMatrix(a);
04592 }
04593
04600 inline GgQuaternion ggMatrixQuaternion(const GgMatrix& m)
04601 {
04602     return ggMatrixQuaternion(m.get());
04603 }
04604
04611 inline GgMatrix ggQuaternionMatrix(const GgQuaternion& q)
04612 {
04613     GLfloat m[16];
04614     q.getMatrix(m);
04615     return GgMatrix{ m };
04616 }
04617
04624 inline GgMatrix ggQuaternionTransposeMatrix(const GgQuaternion& q)
04625 {
04626     GLfloat m[16];
04627     q.getMatrix(m);
04628     GgMatrix t;
04629     return t.loadTranspose(m);
04630 }
04631
04641 inline GgQuaternion ggRotateQuaternion(GLfloat x, GLfloat y, GLfloat z, GLfloat a)
04642 {
04643     GgQuaternion q;
04644     return q.loadRotate(x, y, z, a);
04645 }
04646
04654 inline GgQuaternion ggRotateQuaternion(const GLfloat* v, GLfloat a)
04655 {
04656     return ggRotateQuaternion(v[0], v[1], v[2], a);
04657 }
04658
04665 inline GgQuaternion ggRotateQuaternion(const GLfloat* v)
04666 {
04667     return ggRotateQuaternion(v[0], v[1], v[2], v[3]);
04668 }
04669
04678 inline GgQuaternion ggEulerQuaternion(GLfloat heading, GLfloat pitch, GLfloat roll)
04679 {
04680     GgQuaternion q;
04681     return q.loadEuler(heading, pitch, roll);
04682 }
04683
04690 inline GgQuaternion ggEulerQuaternion(const GLfloat* e)
04691 {
04692     return ggEulerQuaternion(e[0], e[1], e[2]);
04693 }
04694
04701 inline GgQuaternion ggEulerQuaternion(const GgVector& e)
04702 {
04703     return ggEulerQuaternion(e[0], e[1], e[2]);
04704 }

```

```

04705
04714 inline GgQuaternion ggSlerp(const GLfloat* a, const GLfloat* b, GLfloat t)
04715 {
04716     GgQuaternion r;
04717     return r.loadSlerp(a, b, t);
04718 }
04719
04728 inline GgQuaternion ggSlerp(const GgQuaternion& q, const GgQuaternion& r, GLfloat t)
04729 {
04730     return ggSlerp(q.data(), r.data(), t);
04731 }
04732
04741 inline GgQuaternion ggSlerp(const GgQuaternion& q, const GLfloat* a, GLfloat t)
04742 {
04743     return ggSlerp(q.data(), a, t);
04744 }
04745
04754 inline GgQuaternion ggSlerp(const GLfloat* a, const GgQuaternion& q, GLfloat t)
04755 {
04756     return ggSlerp(a, q.data(), t);
04757 }
04758
04765 inline GLfloat ggNorm(const GgQuaternion& q)
04766 {
04767     return q.norm();
04768 }
04769
04776 inline GgQuaternion ggNormalize(const GgQuaternion& q)
04777 {
04778     return q.normalize();
04779 }
04780
04787 inline GgQuaternion ggConjugate(const GgQuaternion& q)
04788 {
04789     return q.conjugate();
04790 }
04791
04798 inline GgQuaternion ggInvert(const GgQuaternion& q)
04799 {
04800     return q.invert();
04801 }
04802
04806 class GgTrackball : public GgQuaternion
04807 {
04808     bool drag;           // ドラッグ中か否か
04809     GLfloat start[2];    // ドラッグ開始位置
04810     GLfloat scale[2];    // マウスの絶対位置→ウィンドウ内での相対位置の換算係数
04811     GgQuaternion cq;     // 回転の初期値 (四元数)
04812     GgMatrix rt;         // 回転の変換行列
04813
04814 public:
04815
04821     GgTrackball(const GgQuaternion& q = ggIdentityQuaternion())
04822     {
04823         reset(q);
04824     }
04825
04829     ~GgTrackball() = default;
04830
04836     GgTrackball& operator=(const GgQuaternion& q)
04837     {
04838         GgQuaternion::operator=(q);
04839         return *this;
04840     }
04841
04851     void region(GLfloat w, GLfloat h);
04852
04862     void region(int w, int h)
04863     {
04864         region(static_cast<GLfloat>(w), static_cast<GLfloat>(h));
04865     }
04866
04876     void begin(GLfloat x, GLfloat y);
04877
04887     void motion(GLfloat x, GLfloat y);
04888
04894     void rotate(const GgQuaternion& q);
04895
04905     void end(GLfloat x, GLfloat y);
04906
04912     void reset(const GgQuaternion& q = ggIdentityQuaternion());
04913
04919     const GLfloat* getStart() const
04920     {
04921         return start;
04922     }
04923

```



```

04927     const GLfloat& getStart(int direction) const
04928     {
04929         return start[direction];
04930     }
04931
04937     void getStart(GLfloat* position) const
04938     {
04939         position[0] = start[0];
04940         position[1] = start[1];
04941     }
04942
04948     const GLfloat* getScale() const
04949     {
04950         return scale;
04951     }
04952
04958     const GLfloat getScale(int direction) const
04959     {
04960         return scale[direction];
04961     }
04962
04968     void getScale(GLfloat* factor) const
04969     {
04970         factor[0] = scale[0];
04971         factor[1] = scale[1];
04972     }
04973
04979     const GgQuaternion& getQuaternion() const
04980     {
04981         return *this;
04982     }
04983
04989     const GgMatrix& getMatrix() const
04990     {
04991         return rt;
04992     }
04993
04999     const GLfloat* get() const
05000     {
05001         return rt.get();
05002     }
05003 };
05004
05015 extern bool ggSaveTga(
05016     const std::string& name,
05017     const void* buffer,
05018     unsigned int width,
05019     unsigned int height,
05020     unsigned int depth
05021 );
05022
05029 extern bool ggSaveColor(const std::string& name);
05030
05037 extern bool ggSaveDepth(const std::string& name);
05038
05049 extern bool ggReadImage(
05050     const std::string& name,
05051     std::vector<GLubyte>& image,
05052     GLsizei* pWidth,
05053     GLsizei* pHeight,
05054     GLenum* pFormat
05055 );
05056
05070 extern GLuint ggLoadTexture(
05071     const GLvoid* image,
05072     GLsizei width,
05073     GLsizei height,
05074     GLenum format = GL_RGB,
05075     GLenum type = GL_UNSIGNED_BYTE,
05076     GLenum internal = GL_RGB,
05077     GLenum wrap = GL_CLAMP_TO_EDGE,
05078     bool swizzle = false
05079 );
05080
05091 extern GLuint ggLoadImage(
05092     const std::string& name,
05093     GLsizei* pWidth = nullptr,
05094     GLsizei* pHeight = nullptr,
05095     GLenum internal = GL_RGBA,
05096     GLenum wrap = GL_CLAMP_TO_EDGE
05097 );
05098
05110 extern void ggCreateNormalMap(
05111     const GLubyte* hmap,
05112     GLsizei width,
05113     GLsizei height,
05114     GLenum format,

```

```

05115     GLfloat nz,
05116     GLenum internal,
05117     std::vector<GgVector>& nmap
05118 );
05119
05130 extern GLuint ggLoadHeight(
05131     const std::string& name,
05132     GLfloat nz,
05133     GLsizei* pWidth = nullptr,
05134     GLsizei* pHeight = nullptr,
05135     GLenum internal = GL_RGBA
05136 );
05137
05151 extern GLuint ggCreateShader(
05152     const std::string& vsrc,
05153     const std::string& fsrc = "",
05154     const std::string& gsrc = "",
05155     GLint nvarying = 0,
05156     const char* const* varyings = nullptr,
05157     const std::string& vtext = "vertex shader",
05158     const std::string& ftext = "fragment shader",
05159     const std::string& gtext = "geometry shader");
05160
05171 extern GLuint ggLoadShader(
05172     const std::string& vert,
05173     const std::string& frag = "",
05174     const std::string& geom = "",
05175     GLint nvarying = 0,
05176     const char* const* varyings = nullptr
05177 );
05178
05187 inline GLuint ggLoadShader(
05188     const std::array<std::string, 3>& files,
05189     GLint nvarying = 0,
05190     const char* const* varyings = nullptr
05191 )
05192 {
05193     return ggLoadShader(files[0], files[1], files[2], nvarying, varyings);
05194 }
05195
05196 #if !defined(__APPLE__)
05204 extern GLuint ggCreateComputeShader(
05205     const std::string& csrc,
05206     const std::string& ctext = "compute shader"
05207 );
05208
05215 extern GLuint ggLoadComputeShader(const std::string& comp);
05216 #endif
05217
05224 class GgTexture
05225 {
05226     // テクスチャ名
05227     GLuint texture;
05228
05229     // テクスチャの縦横の画素数
05230     GLsizei size[2];
05231
05232 public:
05233
05246     GgTexture(
05247         const GLvoid* image,
05248         GLsizei width,
05249         GLsizei height,
05250         GLenum format = GL_RGB,
05251         GLenum type = GL_UNSIGNED_BYTE,
05252         GLenum internal = GL_RGBA,
05253         GLenum wrap = GL_CLAMP_TO_EDGE,
05254         bool swizzle = false
05255     ) :
05256         texture{ ggLoadTexture(image, width, height, format, type, internal, wrap, swizzle) },
05257         size{ width, height }
05258     {
05259     }
05260
05266     GgTexture(const GgTexture& texture) = delete;
05267
05277     GgTexture(GgTexture&& o) noexcept :
05278         texture{ o.texture },
05279         size{ o.size[0], o.size[1] }
05280     {
05281         o.texture = 0;
05282     }
05283
05287     virtual ~GgTexture()
05288     {
05289         glBindTexture(GL_TEXTURE_2D, 0);
05290         glDeleteTextures(1, &texture);

```

```

05291     }
05292
05299     GgTexture& operator=(const GgTexture& texture) = delete;
05300
05310     GgTexture& operator=(GgTexture&& o) noexcept
05311     {
05312         if (&o != this)
05313         {
05314             // 保持しているテクスチャを削除する
05315             glBindTexture(GL_TEXTURE_2D, 0);
05316             glDeleteTextures(1, &texture);
05317
05318             // テクスチャ名の所有権を移動する
05319             texture = o.texture;
05320             size[0] = o.size[0];
05321             size[1] = o.size[1];
05322             o.texture = 0;
05323         }
05324
05325         return *this;
05326     }
05327
05331     void bind() const
05332     {
05333         glBindTexture(GL_TEXTURE_2D, texture);
05334     }
05335
05339     void unbind() const
05340     {
05341         glBindTexture(GL_TEXTURE_2D, 0);
05342     }
05343
05349     void swapRandB(bool swizzle) const;
05350
05356     const GLsizei& getWidth() const
05357     {
05358         return size[0];
05359     }
05360
05366     const GLsizei& getHeight() const
05367     {
05368         return size[1];
05369     }
05370
05376     void getSize(GLsizei* size) const
05377     {
05378         size[0] = getWidth();
05379         size[1] = getHeight();
05380     }
05381
05387     const GLsizei* getSize() const
05388     {
05389         return size;
05390     }
05391
05397     const GLuint& getTexture() const
05398     {
05399         return texture;
05400     }
05401 };
05402
05409 class GgColorTexture
05410 {
05411     // テクスチャ
05412     std::shared_ptr<GgTexture> texture;
05413
05414 public:
05415
05419     GgColorTexture()
05420     {
05421     }
05422
05435     GgColorTexture(
05436         const GLvoid* image,
05437         GLsizei width,
05438         GLsizei height,
05439         GLenum format = GL_RGB,
05440         GLenum type = GL_UNSIGNED_BYTE,
05441         GLenum internal = GL_RGB,
05442         GLenum wrap = GL_CLAMP_TO_EDGE,
05443         bool swizzle = true
05444     )
05445     {
05446         load(image, width, height, format, type, internal, wrap, swizzle);
05447     }
05448
05456     GgColorTexture(

```

```

05457     const std::string& name,
05458     GLenum internal = 0,
05459     GLenum wrap = GL_CLAMP_TO_EDGE
05460 )
05461 {
05462     load(name, internal, wrap);
05463 }
05464
05468 virtual ~GgColorTexture()
05469 {
05470 }
05471
05484 void load(
05485     const GLvoid* image,
05486     GLsizei width,
05487     GLsizei height,
05488     GLenum format = GL_RGB,
05489     GLenum type = GL_UNSIGNED_BYTE,
05490     GLenum internal = GL_RGB,
05491     GLenum wrap = GL_CLAMP_TO_EDGE,
05492     bool swizzle = true
05493 )
05494 {
05495     // テクスチャを作成する
05496     texture = std::make_shared<GgTexture>(image, width, height, format, type, internal, wrap,
05497     swizzle);
05498 }
05499
05506 void load(
05507     const std::string& name,
05508     GLenum internal = 0,
05509     GLenum wrap = GL_CLAMP_TO_EDGE
05510 );
05511
05517 explicit operator bool() const noexcept
05518 {
05519     return texture != nullptr;
05520 }
05521
05527 bool operator!() const noexcept
05528 {
05529     return !static_cast<bool>(*this);
05530 }
05531
05537 const GgTexture* get() const
05538 {
05539     return texture.get();
05540 }
05541
05547 GLuint getTexture() const
05548 {
05549     return texture ? texture->getTexture() : 0;
05550 }
05551
05557 GLsizei getWidth() const
05558 {
05559     return texture ? texture->getWidth() : 0;
05560 }
05561
05567 GLsizei getHeight() const
05568 {
05569     return texture ? texture->getHeight() : 0;
05570 }
05571
05577 void getSize(GLsizei* size) const
05578 {
05579     if (texture) texture->getSize(size);
05580 }
05581
05587 const GLsizei* getSize() const
05588 {
05589     return texture ? texture->getSize() : nullptr;
05590 }
05591
05595 void bind() const
05596 {
05597     if (texture) texture->bind();
05598 }
05599
05603 void unbind() const
05604 {
05605     if (texture) texture->unbind();
05606 }
05607 };
05608
05615 class GgNormalTexture
05616 {

```

```

05617     // テクスチャ
05618     std::shared_ptr<GgTexture> texture;
05619
05620 public:
05621
05625     GgNormalTexture()
05626     {
05627     }
05628
05639     GgNormalTexture(
05640         const GLubyte* image,
05641         GLsizei width,
05642         GLsizei height,
05643         GLenum format = GL_RED,
05644         GLfloat nz = 1.0f,
05645         GLenum internal = GL_RGBA
05646     )
05647     {
05648         // 法線マップのテクスチャを作成する
05649         load(image, width, height, format, nz, internal);
05650     }
05651
05659     GgNormalTexture(
05660         const std::string& name,
05661         GLfloat nz = 1.0f,
05662         GLenum internal = GL_RGBA
05663     )
05664     {
05665         // 法線マップのテクスチャを作成する
05666         load(name, nz, internal);
05667     }
05668
05672     virtual ~GgNormalTexture()
05673     {
05674     }
05675
05686     void load(
05687         const GLubyte* hmap,
05688         GLsizei width,
05689         GLsizei height,
05690         GLenum format = GL_RED,
05691         GLfloat nz = 1.0f,
05692         GLenum internal = GL_RGBA
05693     )
05694     {
05695         // 法線マップ
05696         std::vector<GgVector> nmap;
05697
05698         // 法線マップを作成する
05699         ggCreateNormalMap(hmap, width, height, format, nz, internal, nmap);
05700
05701         // テクスチャを作成する
05702         texture = std::make_shared<GgTexture>(nmap.data(), width, height, GL_RGBA, GL_FLOAT, internal,
GL_REPEAT);
05703     }
05704
05712     void load(
05713         const std::string& name,
05714         GLfloat nz = 1.0f,
05715         GLenum internal = GL_RGBA
05716     );
05717
05723     explicit operator bool() const noexcept
05724     {
05725         return texture != nullptr;
05726     }
05727
05733     bool operator!() const noexcept
05734     {
05735         return !static_cast<bool>(*this);
05736     }
05737
05743     const GgTexture* get() const
05744     {
05745         return texture.get();
05746     }
05747
05753     GLuint getTexture() const
05754     {
05755         return texture ? texture->getTexture() : 0;
05756     }
05757
05763     GLsizei getWidth() const
05764     {
05765         return texture ? texture->getWidth() : 0;
05766     }
05767

```

```

05773     GLsizei getHeight() const
05774     {
05775         return texture ? texture->getHeight() : 0;
05776     }
05777
05783     void getSize(GLsizei* size) const
05784     {
05785         if (texture) texture->getSize(size);
05786     }
05787
05793     const GLsizei* getSize() const
05794     {
05795         return texture ? texture->getSize() : nullptr;
05796     }
05797
05801     void bind() const
05802     {
05803         if (texture) texture->bind();
05804     }
05805
05809     void unbind() const
05810     {
05811         if (texture) texture->unbind();
05812     }
05813 };
05814
05821 template <typename T>
05822 class GgBuffer
05823 {
05824     // ターゲット
05825     const GLenum target;
05826
05827     // バッファオブジェクトのアライメントを考慮したデータの間隔
05828     const GLsizei stride;
05829
05830     // データの数
05831     const GLsizei count;
05832
05833     // バッファオブジェクト (ムーブしたら 0 になる)
05834     GLuint buffer;
05835
05836 public:
05837
05847     GgBuffer(
05848         GLenum target,
05849         const T* data,
05850         GLsizei stride,
05851         GLsizei count,
05852         GLenum usage
05853     ) :
05854         target{ target },
05855         stride{ stride },
05856         count{ count },
05857         buffer{ [] { GLuint buffer; glGenBuffers(1, &buffer); return buffer; } () }
05858     {
05859         // バッファオブジェクトのメモリを確保してデータを転送する
05860         glBindBuffer(target, buffer);
05861         glBufferData(target, getStride() * count, data, usage);
05862     }
05863
05869     GgBuffer(const GgBuffer<T>& buffer) = delete;
05870
05881     GgBuffer(GgBuffer<T>&& o) noexcept :
05882         target{ o.target },
05883         stride{ o.stride },
05884         count{ o.count },
05885         buffer{ o.buffer }
05886     {
05887         o.buffer = 0;
05888     }
05889
05893     virtual ~GgBuffer()
05894     {
05895         // バッファオブジェクトを削除する (ムーブ済みなら buffer は 0 で無視される)
05896         glBindBuffer(target, 0);
05897         glDeleteBuffers(1, &buffer);
05898     }
05899
05906     GgBuffer<T>& operator=(const GgBuffer<T>& buffer) = delete;
05907
05913     const GLuint& getTarget() const
05914     {
05915         return target;
05916     }
05917
05923     GLsizeiptr getStride() const
05924     {

```

```

05925     return static_cast<GLsizeiptr>(stride);
05926 }
05927
05933 const GLsizei& getCount() const
05934 {
05935     return count;
05936 }
05937
05943 const GLuint& getBuffer() const
05944 {
05945     return buffer;
05946 }
05947
05951 void bind() const
05952 {
05953     glBindBuffer(target, buffer);
05954 }
05955
05959 void unbind() const
05960 {
05961     glBindBuffer(target, 0);
05962 }
05963
05969 void* map() const
05970 {
05971     glBindBuffer(target, buffer);
05972 #if defined(GL_GLES_PROTOTYPES)
05973     return glMapBufferRange(target, 0, getStride() * count, GL_MAP_WRITE_BIT);
05974 #else
05975     return glMapBuffer(target, GL_WRITE_ONLY);
05976 #endif
05977 }
05978
05986 void* map(GLint first, GLsizei count) const
05987 {
05988     // count が 0 なら全データをマップする
05989     if (count == 0) count = getCount();
05990     if (first + count > getCount()) count = getCount() - first;
05991
05992     glBindBuffer(target, buffer);
05993     return glMapBufferRange(target, getStride() * first, getStride() * count, GL_MAP_WRITE_BIT);
05994 }
05995
05999 void unmap() const
06000 {
06001     glUnmapBuffer(target);
06002 }
06003
06011 void send(const T* data, GLint first, GLsizei count) const
06012 {
06013     // count が 0 なら全データを転送する
06014     if (count == 0) count = getCount();
06015     if (first + count > getCount()) count = getCount() - first;
06016
06017     // データを既存のバッファオブジェクトに転送する
06018     glBindBuffer(target, buffer);
06019     glBufferSubData(target, getStride() * first, getStride() * count, data);
06020 }
06021
06029 void read(T* data, GLint first, GLsizei count) const
06030 {
06031     // count が 0 なら全データを抽出する
06032     if (count == 0) count = getCount();
06033     if (first + count > getCount()) count = getCount() - first;
06034
06035     // データをバッファオブジェクトから抽出する
06036     glBindBuffer(target, buffer);
06037 #if defined(GL_GLES_PROTOTYPES)
06038     const GLsizeiptr begin{ getStride() * first };
06039     const GLsizeiptr range{ getStride() * count };
06040     T* const source{ glMapBufferRange(target, begin, range, GL_MAP_READ_BIT) };
06041     std::copy(source, source + count, data);
06042     glUnmapBuffer(target);
06043 #else
06044     glGetBufferSubData(target, getStride() * first, getStride() * count, data);
06045 #endif
06046 }
06047
06056 void copy(GLuint src_buffer, GLint src_first = 0, GLint dst_first = 0, GLsizei count = 0) const
06057 {
06058     // count が 0 なら全データを複写する
06059     if (count == 0) count = getCount();
06060     if (src_first + count > getCount()) count = getCount() - src_first;
06061     if (dst_first + count > getCount()) count = getCount() - dst_first;
06062
06063     // データの間隔
06064     const GLsizeiptr stride{ getStride() };

```

```

06065
06066     glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06067     glBindBuffer(GL_COPY_WRITE_BUFFER, buffer);
06068     glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
06069         stride * src.first, stride * dst.first, stride * count);
06070     glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06071     glBindBuffer(GL_COPY_READ_BUFFER, 0);
06072 }
06073 };
06074
06075 template <typename T>
06082 class GgUniformBuffer
06083 {
06084     // ユニフォームバッファオブジェクト
06085     std::shared_ptr<GgBuffer<T>> uniform;
06086
06087 public:
06088
06092     GgUniformBuffer()
06093     {
06094     }
06095
06103     GgUniformBuffer(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06104     {
06105         load(data, count, usage);
06106     }
06107
06115     GgUniformBuffer(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06116     {
06117         load(data, count, usage);
06118     }
06119
06123     virtual ~GgUniformBuffer()
06124     {
06125     }
06126
06132     const GLuint& getTarget() const
06133     {
06134         return uniform->getTarget();
06135     }
06136
06142     GLsizeiptr getStride() const
06143     {
06144         return uniform->getStride();
06145     }
06146
06152     const GLsizei& getCount() const
06153     {
06154         return uniform->getCount();
06155     }
06156
06162     const GLuint& getBuffer() const
06163     {
06164         return uniform->getBuffer();
06165     }
06166
06170     void bind() const
06171     {
06172         uniform->bind();
06173     }
06174
06178     void unbind() const
06179     {
06180         uniform->unbind();
06181     }
06182
06188     void* map() const
06189     {
06190         return uniform->map();
06191     }
06192
06200     void* map(GLint first, GLsizei count) const
06201     {
06202         return uniform->map(first, count);
06203     }
06204
06208     void unmap() const
06209     {
06210         uniform->unmap();
06211     }
06212
06220     void load(const T* data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06221     {
06222         // バッファオブジェクト上のデータの間隔
06223         const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment ) };
06224

```



```

06225     // ユニフォームバッファオブジェクトを確保する
06226     uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
06227
06228     // 確保したユニフォームバッファオブジェクトにデータを転送する
06229     if (data) send(data, 0, sizeof(T), 0, count);
06230 }
06231
06232 void load(const T& data, GLsizei count, GLenum usage = GL_STATIC_DRAW)
06233 {
06234     // バッファオブジェクト上のデータの間の
06235     const GLsizei stride{ (((static_cast<GLint>(sizeof(T)) - 1) / ggBufferAlignment) + 1) *
ggBufferAlignment )};
06236
06237     // ユニフォームバッファオブジェクトを確保する
06238     uniform = std::make_shared<GgBuffer<T>>(GL_UNIFORM_BUFFER, nullptr, stride, count, usage);
06239
06240     // 確保したユニフォームバッファオブジェクトにデータを転送する
06241     fill(&data, 0, sizeof(T), 0, count);
06242 }
06243
06244 void send(
06245     const GLvoid* data,
06246     GLint offset = 0,
06247     GLsizei size = sizeof(T),
06248     GLint first = 0,
06249     GLsizei count = 0
06250 ) const
06251 {
06252     // count が 0 なら全データを転送する
06253     if (count == 0) count = getCount();
06254     if (first + count > getCount()) count = getCount() - first;
06255
06256     // 転送元のデータの先頭
06257     const char* source{ reinterpret_cast<const char*>(data) };
06258
06259     // ターゲット
06260     const GLuint target{ getTarget() };
06261
06262     // データの間の
06263     const GLsizeiptr stride{ getStride() };
06264
06265     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06266     bind();
06267     for (GLsizei i = 0; i < count; ++i)
06268     {
06269         glBufferSubData(target, stride * (first + i) + offset, size, source + size * i);
06270     }
06271 }
06272
06273 void fill(
06274     const GLvoid* data,
06275     GLint offset = 0,
06276     GLsizei size = sizeof(T),
06277     GLint first = 0,
06278     GLsizei count = 0
06279 ) const
06280 {
06281     // count が 0 なら全データを転送する
06282     if (count == 0) count = getCount();
06283     if (first + count > getCount()) count = getCount() - first;
06284
06285     // ターゲット
06286     const GLuint target{ getTarget() };
06287
06288     // データの間の
06289     const GLsizeiptr stride{ getStride() };
06290
06291     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06292     bind();
06293     for (GLsizei i = 0; i < count; ++i)
06294     {
06295         glBufferSubData(target, stride * (first + i) + offset, size, data);
06296     }
06297 }
06298
06299 void read(
06300     GLvoid* data,
06301     GLint offset = 0,
06302     GLsizei size = sizeof(T),
06303     GLint first = 0,
06304     GLsizei count = 0
06305 ) const
06306 {
06307     // count が 0 なら全データを転送する
06308     if (count == 0) count = getCount();
06309     if (first + count > getCount()) count = getCount() - first;
06310
06311     // ターゲット
06312     const GLuint target{ getTarget() };
06313
06314     // データの間の
06315     const GLsizeiptr stride{ getStride() };
06316
06317     // first 番目のブロックから count 個の各ブロックの先頭から offset バイトの位置にデータを転送する
06318     bind();
06319     for (GLsizei i = 0; i < count; ++i)
06320     {
06321         glBufferSubData(target, stride * (first + i) + offset, size, data);
06322     }
06323 }
06324
06325 void read(
06326     GLvoid* data,
06327     GLint offset = 0,
06328     GLsizei size = sizeof(T),
06329     GLint first = 0,
06330     GLsizei count = 0
06331 ) const
06332 {
06333     // count が 0 なら全データを転送する
06334     if (count == 0) count = getCount();
06335     if (first + count > getCount()) count = getCount() - first;
06336 }
06337

```

```

06345 // 抽出先のデータの先頭
06346 char* const destination{ reinterpret_cast<char*>(data) };
06347
06348 // ターゲット
06349 const GLuint target{ getTarget() };
06350
06351 // データの間隔
06352 const GLsizeiptr stride{ getStride() };
06353
06354 // データをユニフォームバッファオブジェクトから抽出する
06355 bind();
06356 #if defined(GL_GLES_PROTOTYPES)
06357 const char* const source{ glMapBufferRange(target, stride * first, stride * count,
GLMAP_READ_BIT) };
06358 for (GLsizei i = 0; i < count; ++i)
06359 {
06360     const char* const begin{ source + stride * i + offset };
06361     const char* const end{ begin + size };
06362     std::copy(begin, end, destination + sizeof(T) * i);
06363 }
06364 glUnmapBuffer(target);
06365 #else
06366 for (GLsizei i = 0; i < count; ++i)
06367 {
06368     glGetBufferSubData(target, stride * (first + i) + offset, size, destination + sizeof(T) * i);
06369 }
06370 #endif
06371 }
06372
06381 void copy(
06382     GLuint src_buffer,
06383     GLint src_first = 0,
06384     GLint dst_first = 0,
06385     GLsizei count = 0
06386 ) const
06387 {
06388     // count が 0 なら全データを複写する
06389     if (count == 0) count = getCount();
06390     if (src_first + count > getCount()) count = getCount() - src_first;
06391     if (dst_first + count > getCount()) count = getCount() - dst_first;
06392
06393     // データの間隔
06394     const GLsizeiptr stride{ getStride() };
06395
06396     // ユニフォームバッファオブジェクトではブロックごとに転送する
06397     glBindBuffer(GL_COPY_READ_BUFFER, src_buffer);
06398     glBindBuffer(GL_COPY_WRITE_BUFFER, getBuffer());
06399     for (GLsizei i = 0; i < count; ++i)
06400     {
06401         glCopyBufferSubData(GL_COPY_READ_BUFFER, GL_COPY_WRITE_BUFFER,
stride * (src_first + i), stride * (dst_first + i), sizeof(T));
06402     }
06403     glBindBuffer(GL_COPY_WRITE_BUFFER, 0);
06404     glBindBuffer(GL_COPY_READ_BUFFER, 0);
06405 }
06406 };
06407
06412 class GgVertexArray
06413 {
06414     // 頂点配列オブジェクト (ムーブしたら 0 になる)
06415     GLuint vao;
06416
06417 public:
06424 GgVertexArray(GLenum mode = 0) :
06425     vao{ [] { GLuint vao; glGenVertexArrays(1, &vao); return vao; } () }
06426 {
06427     glBindVertexArray(vao);
06428 }
06429
06435 GgVertexArray(const GgVertexArray& array) = delete;
06436
06447 GgVertexArray(GgVertexArray&& o) noexcept :
06448     vao{ o.vao }
06449 {
06450     o.vao = 0;
06451 }
06452
06456 virtual ~GgVertexArray()
06457 {
06458     // 頂点配列オブジェクトを削除する (ムーブ済みなら vao は 0 で無視される)
06459     glBindVertexArray(0);
06460     glDeleteVertexArrays(1, &vao);
06461 }
06462
06469 GgVertexArray& operator=(const GgVertexArray& array) = delete;
06470

```

```

06481     GgVertexArray& operator=(GgVertexArray&& o) noexcept
06482     {
06483         if (&o != this)
06484         {
06485             // 保持している頂点配列オブジェクトを削除する
06486             glBindVertexArray(0);
06487             glDeleteVertexArrays(1, &vao);
06488
06489             // 頂点配列オブジェクト名の所有権を移動する
06490             vao = o.vao;
06491             o.vao = 0;
06492         }
06493         return *this;
06494     }
06495
06502     const GLuint& get() const
06503     {
06504         return vao;
06505     }
06506
06510     void bind() const
06511     {
06512         glBindVertexArray(vao);
06513     }
06514 };
06515
06523 class GgShape
06524 {
06525     // 頂点配列オブジェクト
06526     std::shared_ptr<GgVertexArray> object;
06527
06528     // 基本図形の種類
06529     GLenum mode;
06530
06531 public:
06532
06538     GgShape(GLenum mode = 0) :
06539         object{ std::make_shared<GgVertexArray>() },
06540         mode{ mode }
06541     {
06542     }
06543
06547     virtual ~GgShape()
06548     {
06549     }
06550
06556     virtual explicit operator bool() const noexcept
06557     {
06558         return object.get() != nullptr;
06559     }
06560
06566     virtual bool operator!() const noexcept
06567     {
06568         return !static_cast<bool>(*this);
06569     }
06570
06576     const GLuint& get() const
06577     {
06578         return object->get();
06579     }
06580
06586     void setMode(GLenum mode)
06587     {
06588         this->mode = mode;
06589     }
06590
06596     const GLenum& getMode() const
06597     {
06598         return this->mode;
06599     }
06600
06607     virtual void draw(GLint first = 0, GLsizei count = 0) const
06608     {
06609         object->bind();
06610     }
06611 };
06612
06616 class GgPoints
06617     : public GgShape
06618 {
06619     // 頂点バッファオブジェクト
06620     std::shared_ptr<GgBuffer<GgVector>> position;
06621
06622 public:
06623
06627     GgPoints(GLenum mode = GL_POINTS) :

```

```

06628     GgShape(mode)
06629     {
06630     }
06631
06640     GgPoints(
06641         const GgVector* pos,
06642         GLsizei countv,
06643         GLenum mode = GL_POINTS,
06644         GLenum usage = GL_STATIC_DRAW
06645     ) :
06646         GgPoints(mode)
06647     {
06648         load(pos, countv, usage);
06649     }
06650
06654     virtual ~GgPoints()
06655     {
06656     }
06657
06663     explicit operator bool() const noexcept
06664     {
06665         return position.get() != nullptr;
06666     }
06667
06673     bool operator!() const noexcept
06674     {
06675         return !static_cast<bool>(*this);
06676     }
06677
06683     const GLsizei& getCount() const
06684     {
06685         return position->getCount();
06686     }
06687
06693     const GLuint& getBuffer() const
06694     {
06695         return position->getBuffer();
06696     }
06697
06705     void send(const GgVector* pos, GLint first = 0, GLsizei count = 0) const
06706     {
06707         position->send(pos, first, count);
06708     }
06709
06717     void load(const GgVector* pos, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06718
06725     virtual void draw(GLint first = 0, GLsizei count = 0) const;
06726 };
06727
06731 struct GgVertex
06732 {
06734     GgVector position;
06735
06737     GgVector normal;
06738
06742     GgVertex()
06743     {
06744     }
06745
06752     GgVertex(const GgVector& pos, const GgVector& norm) :
06753         position(pos),
06754         normal(norm)
06755     {
06756     }
06757
06768     GgVertex(
06769         GLfloat px, GLfloat py, GLfloat pz,
06770         GLfloat nx, GLfloat ny, GLfloat nz
06771     ) :
06772         position{ px, py, pz, 1.0f },
06773         normal{ nx, ny, nz, 0.0f }
06774     {
06775     }
06776
06783     GgVertex(const GLfloat* pos, const GLfloat* norm) :
06784         GgVertex(pos[0], pos[1], pos[2], norm[0], norm[1], norm[2])
06785     {
06786     }
06787 };
06788
06792 class GgTriangles
06793 : public GgShape
06794 {
06795     // 頂点属性
06796     std::shared_ptr<GgBuffer<GgVertex>> vertex;
06797
06798 public:

```

```

06799
06805     GgTriangles(GLenum mode = GL_TRIANGLES) :
06806         GgShape(mode)
06807     {
06808     }
06809
06818     GgTriangles(
06819         const GgVertex* vert,
06820         GLsizei count,
06821         GLenum mode = GL_TRIANGLES,
06822         GLenum usage = GL_STATIC_DRAW
06823     ) :
06824         GgTriangles(mode)
06825     {
06826         load(vert, count, usage);
06827     }
06828
06832     virtual ~GgTriangles()
06833     {
06834     }
06835
06841     const GLsizei& getCount() const
06842     {
06843         return vertex->getCount();
06844     }
06845
06851     const GLuint& getBuffer() const
06852     {
06853         return vertex->getBuffer();
06854     }
06855
06863     void send(const GgVertex* vert, GLint first = 0, GLsizei count = 0) const
06864     {
06865         vertex->send(vert, first, count);
06866     }
06867
06875     void load(const GgVertex* vert, GLsizei count, GLenum usage = GL_STATIC_DRAW);
06876
06883     virtual void draw(GLint first = 0, GLsizei count = 0) const;
06884 };
06885
06889 class GgElements
06890     : public GgTriangles
06891 {
06892     // インデックスを格納する頂点バッファオブジェクト
06893     std::shared_ptr<GgBuffer<GLuint>> index;
06894
06895 public:
06896
06902     GgElements(GLenum mode = GL_TRIANGLES) :
06903         GgTriangles(mode)
06904     {
06905     }
06906
06917     GgElements(
06918         const GgVertex* vert,
06919         GLsizei countv,
06920         const GLuint* face,
06921         GLsizei countf,
06922         GLenum mode = GL_TRIANGLES,
06923         GLenum usage = GL_STATIC_DRAW
06924     ) :
06925         GgElements(mode)
06926     {
06927         load(vert, countv, face, countf, usage);
06928     }
06929
06933     virtual ~GgElements()
06934     {
06935     }
06936
06941     const GLsizei& getIndexCount() const
06942     {
06943         return index->getCount();
06944     }
06945
06951     const GLuint& getIndexBuffer() const
06952     {
06953         return index->getBuffer();
06954     }
06955
06966     void send(
06967         const GgVertex* vert,
06968         GLuint firstv,
06969         GLsizei countv,
06970         const GLuint* face = nullptr,
06971         GLuint firstf = 0,

```

```

06972     GLsizei countf = 0
06973 ) const
06974 {
06975     GgTriangles::send(vert, firstv, countv);
06976     if (face != nullptr && countf > 0) index->send(face, firstf, countf);
06977 }
06978
06988 void load(
06989     const GgVertex* vert,
06990     GLsizei countv,
06991     const GLuint* face,
06992     GLsizei countf,
06993     GLenum usage = GL_STATIC_DRAW
06994 )
06995 {
06996     // 頂点バッファオブジェクトを作成する
06997     GgTriangles::load(vert, countv, usage);
06998
06999     // インデックスの頂点バッファオブジェクトを作成する
07000     index = std::make_shared<GgBuffer<GLuint>>(GL_ELEMENT_ARRAY_BUFFER, face,
static_cast<GLsizei>(sizeof(GLuint)), countf, usage);
07001 }
07002
07009 virtual void draw(GLint first = 0, GLsizei count = 0) const;
07010 };
07011
07022 extern std::shared_ptr<GgPoints> ggPointsCube(
07023     GLsizei countv,
07024     GLfloat length = 1.0f,
07025     GLfloat cx = 0.0f,
07026     GLfloat cy = 0.0f,
07027     GLfloat cz = 0.0f
07028 );
07029
07040 extern std::shared_ptr<GgPoints> ggPointsSphere(
07041     GLsizei countv,
07042     GLfloat radius = 0.5f,
07043     GLfloat cx = 0.0f,
07044     GLfloat cy = 0.0f,
07045     GLfloat cz = 0.0f
07046 );
07047
07055 extern std::shared_ptr<GgTriangles> ggRectangle(
07056     GLfloat width = 1.0f,
07057     GLfloat height = 1.0f
07058 );
07059
07068 extern std::shared_ptr<GgTriangles> ggEllipse(
07069     GLfloat width = 1.0f,
07070     GLfloat height = 1.0f,
07071     GLuint slices = 16
07072 );
07073
07085 extern std::shared_ptr<GgTriangles> ggArraysObj(
07086     const std::string& name,
07087     bool normalize = false
07088 );
07089
07101 extern std::shared_ptr<GgElements> ggElementsObj(
07102     const std::string& name,
07103     bool normalize = false
07104 );
07105
07118 extern std::shared_ptr<GgElements> ggElementsMesh(
07119     GLuint slices,
07120     GLuint stacks,
07121     const GLfloat(*pos)[3],
07122     const GLfloat(*norm)[3] = nullptr
07123 );
07124
07135 extern std::shared_ptr<GgElements> ggElementsSphere(
07136     GLfloat radius = 1.0f,
07137     int slices = 16,
07138     int stacks = 8
07139 );
07140
07147 class GgShader
07148 {
07149     // プログラム名 (ムーブしたら 0 になる)
07150     GLuint program;
07151
07152 public:
07153
07163     GgShader(
07164         const std::string& vert,
07165         const std::string& frag = "",
07166         const std::string& geom = "",

```

```

07167     int nvarying = 0,
07168     const char* const* varyings = nullptr
07169 ) :
07170     program(ggLoadShader(vert, frag, geom, nvarying, varyings))
07171 {
07172 }
07173
07181 GgShader(
07182     const std::array<std::string, 3>& files,
07183     int nvarying = 0,
07184     const char* const* varyings = nullptr
07185 ) :
07186     GgShader(files[0], files[1], files[2], nvarying, varyings)
07187 {
07188 }
07189
07195 GgShader(const GgShader& shader) = delete;
07196
07206 GgShader(GgShader&& o) noexcept :
07207     program{ o.program }
07208 {
07209     o.program = 0;
07210 }
07211
07215 virtual ~GgShader()
07216 {
07217     // シェーダプログラムを削除する (ムーブ済みなら program は 0 で無視される)
07218     glUseProgram(0);
07219     glDeleteProgram(program);
07220 }
07221
07228 GgShader& operator=(const GgShader& shader) = delete;
07229
07240 GgShader& operator=(GgShader&& o) noexcept
07241 {
07242     if (&o != this)
07243     {
07244         // 保持しているシェーダプログラムを削除する
07245         glUseProgram(0);
07246         glDeleteProgram(program);
07247
07248         // プログラム名の所有権を移動する
07249         program = o.program;
07250         o.program = 0;
07251     }
07252
07253     return *this;
07254 }
07255
07259 void use() const
07260 {
07261     glUseProgram(program);
07262 }
07263
07267 void unuse() const
07268 {
07269     glUseProgram(0);
07270 }
07271
07277 GLuint get() const
07278 {
07279     return program;
07280 }
07281 };
07282
07286 class GgPointShader
07287 {
07288     // シェーダー
07289     std::shared_ptr<GgShader> shader;
07290
07291     // 投影変換行列の uniform 変数の場所
07292     GLint mpLoc;
07293
07294     // モデルビュー変換行列の uniform 変数の場所
07295     GLint mvLoc;
07296
07297 public:
07298
07302 GgPointShader() :
07303     mpLoc{ -1 },
07304     mvLoc{ -1 }
07305 {
07306 }
07307
07317 GgPointShader(
07318     const std::string& vert,
07319     const std::string& frag = "",

```

```

07320     const std::string& geom = "",
07321     GLint nvarying = 0,
07322     const char* const* varyings = nullptr
07323 ) :
07324     GgPointShader()
07325 {
07326     load(vert, frag, geom, nvarying, varyings);
07327 }
07328
07336 GgPointShader(
07337     const std::array<std::string, 3>& files,
07338     int nvarying = 0,
07339     const char* const* varyings = nullptr
07340 ) :
07341     GgPointShader(files[0], files[1], files[2], nvarying, varyings)
07342 {
07343 }
07344
07348 virtual ~GgPointShader()
07349 {
07350 }
07351
07362 bool load(
07363     const std::string& vert,
07364     const std::string& frag = "",
07365     const std::string& geom = "",
07366     GLint nvarying = 0,
07367     const char* const* varyings = nullptr
07368 )
07369 {
07370     // シェーダを作成する
07371     shader = std::make_shared<GgShader>(vert, frag, geom, nvarying, varyings);
07372
07373     // プログラム名を取り出す
07374     const GLuint program(shader->get());
07375
07376     // プログラムオブジェクトが作成できていなければ戻る
07377     if (program == 0) return false;
07378
07379     // 変換行列の uniform 変数の場所
07380     mpLoc = glGetUniformLocation(program, "mp");
07381     mvLoc = glGetUniformLocation(program, "mv");
07382
07383     // プログラムオブジェクトの作成に成功した
07384     return true;
07385 }
07386
07395 bool load(
07396     const std::array<std::string, 3>& files,
07397     GLint nvarying = 0,
07398     const char* const* varyings = nullptr
07399 )
07400 {
07401     return load(files[0], files[1], files[2], nvarying, varyings);
07402 }
07403
07409 virtual void loadProjectionMatrix(const GLfloat* mp) const
07410 {
07411     glUniformMatrix4fv(mpLoc, 1, GL_FALSE, mp);
07412 }
07413
07419 virtual void loadProjectionMatrix(const GgMatrix& mp) const
07420 {
07421     loadProjectionMatrix(mp.get());
07422 }
07423
07429 virtual void loadModelviewMatrix(const GLfloat* mv) const
07430 {
07431     glUniformMatrix4fv(mvLoc, 1, GL_FALSE, mv);
07432 }
07433
07439 virtual void loadModelviewMatrix(const GgMatrix& mv) const
07440 {
07441     loadModelviewMatrix(mv.get());
07442 }
07443
07450 virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07451 {
07452     loadProjectionMatrix(mp);
07453     loadModelviewMatrix(mv);
07454 }
07455
07462 virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07463 {
07464     loadMatrix(mp.get(), mv.get());
07465 }
07466

```



```

07470     virtual void use() const
07471     {
07472         shader->use();
07473     }
07474
07480     void use(const GLfloat* mp) const
07481     {
07482         use();
07483         loadProjectionMatrix(mp);
07484     }
07485
07491     void use(const GgMatrix& mp) const
07492     {
07493         use(mp.get());
07494     }
07495
07502     void use(const GLfloat* mp, const GLfloat* mv) const
07503     {
07504         use(mp);
07505         loadModelviewMatrix(mv);
07506     }
07507
07514     void use(const GgMatrix& mp, const GgMatrix& mv) const
07515     {
07516         use(mp.get(), mv.get());
07517     }
07518
07522     void unuse() const
07523     {
07524         shader->unuse();
07525     }
07526
07532     GLuint get() const
07533     {
07534         return shader->get();
07535     }
07536 };
07537
07541 class GgSimpleShader
07542 : public GgPointShader
07543 {
07544     // モデルビュー変換の法線変換行列の uniform 変数の場所
07545     GLint mnLoc;
07546
07547 public:
07548
07552     GgSimpleShader() :
07553         GgPointShader(),
07554         mnLoc{ -1 }
07555     {
07556     }
07557
07567     GgSimpleShader(
07568         const std::string& vert,
07569         const std::string& frag = "",
07570         const std::string& geom = "",
07571         GLint nvarying = 0,
07572         const char* const* varyings = nullptr
07573     )
07574     {
07575         load(vert, frag, geom, nvarying, varyings);
07576     }
07577
07585     GgSimpleShader(
07586         const std::array<std::string, 3>& files,
07587         GLint nvarying = 0,
07588         const char* const* varyings = nullptr
07589     ) :
07590         GgSimpleShader(files[0], files[1], files[2], nvarying, varyings)
07591     {
07592     }
07593
07597     GgSimpleShader(const GgSimpleShader& o) :
07598         GgPointShader(o),
07599         mnLoc{ o.mnLoc }
07600     {
07601     }
07602
07606     virtual ~GgSimpleShader()
07607     {
07608     }
07609
07616     GgSimpleShader& operator=(const GgSimpleShader& o)
07617     {
07618         if (&o != this)
07619         {
07620             GgPointShader::operator=(o);

```

```

07621         mnLoc = o.mnLoc;
07622     }
07623
07624     return *this;
07625 }
07626
07637 bool load(
07638     const std::string& vert,
07639     const std::string& frag = "",
07640     const std::string& geom = "",
07641     GLint nvarying = 0,
07642     const char* const* varyings = nullptr
07643 );
07644
07653 bool load(
07654     const std::array<std::string, 3>& files,
07655     GLint nvarying = 0,
07656     const char* const* varyings = nullptr
07657 )
07658 {
07659     return load(files[0], files[1], files[2], nvarying, varyings);
07660 }
07661
07668 virtual void loadModelviewMatrix(const GLfloat* mv, const GLfloat* mn) const
07669 {
07670     GgPointShader::loadModelviewMatrix(mv);
07671     glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07672 }
07673
07680 virtual void loadModelviewMatrix(const GgMatrix& mv, const GgMatrix& mn) const
07681 {
07682     loadModelviewMatrix(mv.get(), mn.get());
07683 }
07684
07690 virtual void loadModelviewMatrix(const GLfloat* mv) const
07691 {
07692     loadModelviewMatrix(mv, GgMatrix(mv).normal().get());
07693 }
07694
07700 virtual void loadModelviewMatrix(const GgMatrix& mv) const
07701 {
07702     loadModelviewMatrix(mv.get());
07703 }
07704
07712 virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
07713 {
07714     GgPointShader::loadMatrix(mp, mv);
07715     glUniformMatrix4fv(mnLoc, 1, GL_FALSE, mn);
07716 }
07717
07725 virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
07726 {
07727     loadMatrix(mp.get(), mv.get(), mn.get());
07728 }
07729
07736 virtual void loadMatrix(const GLfloat* mp, const GLfloat* mv) const
07737 {
07738     loadMatrix(mp, mv, GgMatrix(mv).normal());
07739 }
07740
07747 virtual void loadMatrix(const GgMatrix& mp, const GgMatrix& mv) const
07748 {
07749     loadMatrix(mp, mv, mv.normal());
07750 }
07751
07755 struct Light
07756 {
07757     GgVector ambient;
07758     GgVector diffuse;
07759     GgVector specular;
07760     GgVector position;
07761 };
07762
07766 class LightBuffer
07767     : public GgUniformBuffer<Light>
07768 {
07769 public:
07770
07778     LightBuffer(
07779         const Light* light = nullptr,
07780         GLsizei count = 1,
07781         GLenum usage = GL_STATIC_DRAW
07782     ) :
07783         GgUniformBuffer<Light>(light, count, usage)
07784     {
07785     }
07786

```

```

07794     LightBuffer(
07795         const Light& light,
07796         GLsizei count = 1,
07797         GLenum usage = GL_STATIC_DRAW
07798     ) :
07799         GgUniformBuffer<Light>(light, count, usage)
07800     {
07801     }
07802
07813     LightBuffer(
07814         GgVector ambient,
07815         GgVector diffuse,
07816         GgVector specular,
07817         GgVector position,
07818         GLsizei count = 1,
07819         GLenum usage = GL_STATIC_DRAW
07820     ) :
07821         GgUniformBuffer<Light>(Light{ ambient, diffuse, specular, position }, count, usage)
07822     {
07823     }
07824
07828     virtual ~LightBuffer()
07829     {
07830     }
07831
07842     void loadAmbient(
07843         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07844         GLint first = 0, GLsizei count = 1
07845     ) const;
07846
07854     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
07855
07863     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
07864     {
07865         // first 番目のブロックから count 個の ambient 要素に値を設定する
07866         send(ambient, offsetof(Light, ambient), sizeof(Light::ambient), first, count);
07867     }
07868
07879     void loadDiffuse(
07880         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07881         GLint first = 0, GLsizei count = 1
07882     ) const;
07883
07891     void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
07892
07900     void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
07901     {
07902         // first 番目のブロックから count 個の diffuse 要素に値を設定する
07903         send(diffuse, offsetof(Light, diffuse), sizeof(Light::diffuse), first, count);
07904     }
07905
07916     void loadSpecular(
07917         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
07918         GLint first = 0, GLsizei count = 1
07919     ) const;
07920
07928     void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
07929
07937     void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
07938     {
07939         // first 番目のブロックから count 個の specular 要素に値を設定する
07940         send(specular, offsetof(Light, specular), sizeof(Light::specular), first, count);
07941     }
07942
07950     void loadColor(const Light& color, GLint first = 0, GLsizei count = 1) const;
07951
07962     void loadPosition(
07963         GLfloat x, GLfloat y, GLfloat z, GLfloat w = 1.0f,
07964         GLint first = 0, GLsizei count = 1
07965     ) const;
07966
07974     void loadPosition(const GgVector& position, GLint first = 0, GLsizei count = 1) const;
07975
07983     void loadPosition(const GLfloat* position, GLint first = 0, GLsizei count = 1) const
07984     {
07985         // first 番目のブロックから count 個の position 要素に値を設定する
07986         send(position, offsetof(Light, position), sizeof(Light::position), first, count);
07987     }
07988
07996     void loadPosition(const GgVector* position, GLint first = 0, GLsizei count = 1) const
07997     {
07998         loadPosition(position->data(), first, count);
07999     }
08000
08008     void load(const Light* light, GLint first = 0, GLsizei count = 1) const
08009     {
08010         send(light, 0, sizeof(Light), first, count);

```

```

08011     }
08012
08020 void load(const Light& light, GLint first = 0, GLsizei count = 1) const
08021 {
08022     load(&light, first, count);
08023 }
08024
08030 void select(GLint i = 0) const
08031 {
08032     // バッファオブジェクトの i 番目のブロックの位置
08033     const GLintptr offset(static_cast<GLintptr>(getStride()) * i);
08034     glBindBufferRange(getTarget(), LightBindingPoint, getBuffer(), offset, sizeof(Light));
08035 }
08036 };
08037
08041 struct Material
08042 {
08043     GgVector ambient;
08044     GgVector diffuse;
08045     GgVector specular;
08046     GLfloat shininess;
08047 };
08048
08052 class MaterialBuffer
08053     : public GgUniformBuffer<Material>
08054 {
08055 public:
08056
08064     MaterialBuffer(
08065         const Material* material = nullptr,
08066         GLsizei count = 1,
08067         GLenum usage = GL_STATIC_DRAW
08068     ) :
08069         GgUniformBuffer<Material>(material, count, usage)
08070     {
08071     }
08072
08080     MaterialBuffer(
08081         const Material& material,
08082         GLsizei count = 1,
08083         GLenum usage = GL_STATIC_DRAW
08084     ) :
08085         GgUniformBuffer<Material>(material, count, usage)
08086     {
08087     }
08088
08099     MaterialBuffer(
08100         GgVector ambient,
08101         GgVector diffuse,
08102         GgVector specular,
08103         GLfloat shininess,
08104         GLsizei count = 1,
08105         GLenum usage = GL_STATIC_DRAW
08106     ) :
08107         GgUniformBuffer<Material>(Material{ ambient, diffuse, specular, shininess }, count, usage)
08108     {
08109     }
08110
08114     virtual ~MaterialBuffer()
08115     {
08116     }
08117
08128     void loadAmbient(
08129         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08130         GLint first = 0, GLsizei count = 1
08131     ) const;
08132
08140     void loadAmbient(const GgVector& ambient, GLint first = 0, GLsizei count = 1) const;
08141
08149     void loadAmbient(const GLfloat* ambient, GLint first = 0, GLsizei count = 1) const
08150     {
08151         // first 番目のブロックから count 個のブロックの ambient 要素に値を設定する
08152         send(ambient, offsetof(Material, ambient), sizeof(Material::ambient), first, count);
08153     }
08154
08165     void loadDiffuse(
08166         GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08167         GLint first = 0, GLsizei count = 1
08168     ) const;
08169
08177     void loadDiffuse(const GgVector& diffuse, GLint first = 0, GLsizei count = 1) const;
08178
08186     void loadDiffuse(const GLfloat* diffuse, GLint first = 0, GLsizei count = 1) const
08187     {
08188         // first 番目のブロックから count 個の diffuse 要素に値を設定する
08189         send(diffuse, offsetof(Material, diffuse), sizeof(Material::diffuse), first, count);
08190     }

```

```

08191
08202 void loadAmbientAndDiffuse(
08203     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08204     GLint first = 0, GLsizei count = 1
08205 ) const;
08206
08214 void loadAmbientAndDiffuse(const GgVector& color, GLint first = 0, GLsizei count = 1) const;
08215
08223 void loadAmbientAndDiffuse(const GLfloat* color, GLint first = 0, GLsizei count = 1) const;
08224
08235 void loadSpecular(
08236     GLfloat r, GLfloat g, GLfloat b, GLfloat a = 1.0f,
08237     GLint first = 0, GLsizei count = 1
08238 ) const;
08239
08247 void loadSpecular(const GgVector& specular, GLint first = 0, GLsizei count = 1) const;
08248
08256 void loadSpecular(const GLfloat* specular, GLint first = 0, GLsizei count = 1) const
08257 {
08258     // first 番目のブロックから count 個の specular 要素に値を設定する
08259     send(specular, offsetof(Material, specular), sizeof(Material::specular), first, count);
08260 }
08261
08269 void loadShininess(GLfloat shininess, GLint first = 0, GLsizei count = 1) const;
08270
08278 void loadShininess(const GLfloat* shininess, GLint first = 0, GLsizei count = 1) const;
08279
08287 void load(const Material* material, GLint first = 0, GLsizei count = 1) const
08288 {
08289     send(material, 0, sizeof(Material), first, count);
08290 }
08291
08299 void load(const Material& material, GLint first = 0, GLsizei count = 1) const
08300 {
08301     load(&material, first, count);
08302 }
08303
08309 void select(GLint i = 0) const
08310 {
08311     // バッファオブジェクトの i 番目のブロックの位置
08312     const GLintptr offset{ static_cast<GLintptr>(getStride()) * i };
08313     glBindBufferRange(getTarget(), MaterialBindingPoint, getBuffer(), offset, sizeof(Material));
08314 }
08315 };
08316
08320 void use() const
08321 {
08322     // プログラムオブジェクトは基底クラスで指定する
08323     GgPointShader::use();
08324 }
08325
08333 void use(const GLfloat* mp, const GLfloat* mv, const GLfloat* mn) const
08334 {
08335     // プログラムオブジェクトを指定する
08336     use();
08337
08338     // 変換行列を設定する
08339     loadMatrix(mp, mv, mn);
08340 }
08341
08349 void use(const GgMatrix& mp, const GgMatrix& mv, const GgMatrix& mn) const
08350 {
08351     use(mp.get(), mv.get(), mn.get());
08352 }
08353
08360 void use(const GLfloat* mp, const GLfloat* mv) const
08361 {
08362     use(mp, mv, GgMatrix(mv).normal().get());
08363 }
08364
08371 void use(const GgMatrix& mp, const GgMatrix& mv) const
08372 {
08373     use(mp, mv, mv.normal());
08374 }
08375
08382 void use(const LightBuffer* light, GLint i = 0) const
08383 {
08384     // プログラムオブジェクトを指定する
08385     use();
08386
08387     // 光源を設定する
08388     light->select(i);
08389 }
08390
08397 void use(const LightBuffer& light, GLint i = 0) const
08398 {
08399     use(&light, i);

```

```

08400     }
08401
08411 void use(
08412     const GLfloat* mp,
08413     const GLfloat* mv,
08414     const GLfloat* mn,
08415     const LightBuffer* light,
08416     GLint i = 0
08417 ) const
08418 {
08419     // 光源を指定してプログラムオブジェクトを指定する
08420     use(light, i);
08421
08422     // 変換行列を設定する
08423     loadMatrix(mp, mv, mn);
08424 }
08425
08435 void use(
08436     const GgMatrix& mp,
08437     const GgMatrix& mv,
08438     const GgMatrix& mn,
08439     const LightBuffer& light,
08440     GLint i = 0
08441 ) const
08442 {
08443     use(mp.get(), mv.get(), mn.get(), &light, i);
08444 }
08445
08454 void use(
08455     const GLfloat* mp,
08456     const GLfloat* mv,
08457     const LightBuffer* light,
08458     GLint i = 0
08459 ) const
08460 {
08461     use(mp, mv, GgMatrix(mv).normal().get(), light, i);
08462 }
08463
08472 void use(
08473     const GgMatrix& mp,
08474     const GgMatrix& mv,
08475     const LightBuffer& light,
08476     GLint i = 0
08477 ) const
08478 {
08479     use(mp, mv, mv.normal(), light, i);
08480 }
08481
08489 void use(const GLfloat* mp, const LightBuffer* light, GLint i = 0) const
08490 {
08491     // 光源を指定してプログラムオブジェクトを指定する
08492     use(light, i);
08493
08494     // 投影変換行列を設定する
08495     loadProjectionMatrix(mp);
08496 }
08497
08505 void use(const GgMatrix& mp, const LightBuffer& light, GLint i = 0) const
08506 {
08507     // 光源を指定してプログラムオブジェクトを指定する
08508     use(mp.get(), &light, i);
08509 }
08510 };
08511
08522 extern bool ggLoadSimpleObj(
08523     const std::string& name,
08524     std::vector<std::array<GLuint, 3>>& group,
08525     std::vector<GgSimpleShader::Material>& material,
08526     std::vector<GgVertex>& vert,
08527     bool normalize = false
08528 );
08529
08541 extern bool ggLoadSimpleObj(
08542     const std::string& name,
08543     std::vector<std::array<GLuint, 3>>& group,
08544     std::vector<GgSimpleShader::Material>& material,
08545     std::vector<GgVertex>& vert,
08546     std::vector<GLuint>& face,
08547     bool normalize = false
08548 );
08549
08553 class GgSimpleObj
08554 {
08555     // 同じ材質を割り当てるポリゴングループごとの三角形数
08556     std::shared_ptr<std::vector<std::array<GLuint, 3>>> group;
08557
08558     // ポリゴングループごとの材質のユニフォームバッファ

```

```

08559     std::shared_ptr<GgSimpleShader::MaterialBuffer> material;
08560
08561     // この図形の形状データ
08562     std::shared_ptr<GgElements> data;
08563
08564 public:
08565
08566     GgSimpleObj(const std::string& name, bool normalize = false);
08573
08576     virtual ~GgSimpleObj()
08577     {
08578     }
08579
08585     explicit operator bool() const noexcept
08586     {
08587         return data.get() != nullptr;
08588     }
08589
08595     bool operator!() const noexcept
08596     {
08597         return !static_cast<bool>(*this);
08598     }
08599
08605     const GgTriangles* get() const
08606     {
08607         return data.get();
08608     }
08609
08616     virtual void draw(GLint first = 0, GLsizei count = 0) const;
08617 };
08618 }

```

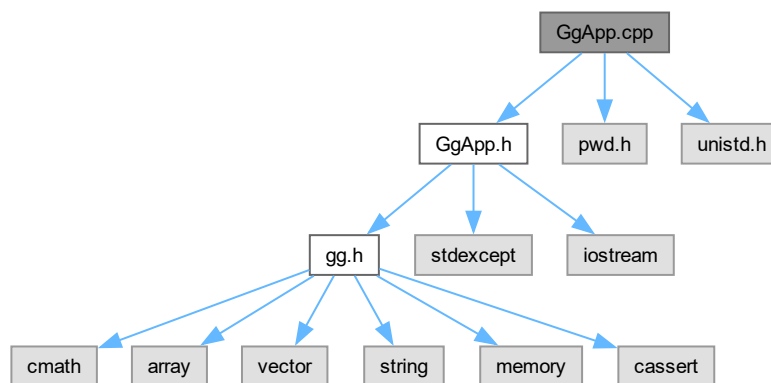
8.5 GgApp.cpp ファイル

```

#include "GgApp.h"
#include <pwd.h>
#include <unistd.h>

```

GgApp.cpp の依存先関係図:



8.5.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの実装.

著者

Kohe Tokoi

日付

July 27, 2025

[GgApp.cpp](#) に定義があります。

8.6 GgApp.cpp

[詳解]

```
00001 /*
00002
00003 ゲームグラフィックス特論用補助プログラム GLFW3 版
00004
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00025 #include "GgApp.h"
00026 using namespace gg;
00027
00028 //
00029 // GLFW のエラー表示
00030 //
00031 static void glfwErrorCallback(int error, const char* description)
00032 {
00033     #if defined(__aarch64__)
00034         if (error == 65544) return;
00035     #endif
00036     throw std::runtime_error(description);
00037 }
00038
00039 //
00040 // GgApp クラスのコンストラクタ
00041 //
00042 GgApp::GgApp(int major, int minor)
00043 {
00044     // GLFW のエラー処理関数を登録する
00045     glfwSetErrorCallback(glfwErrorCallback);
00046
00047     // GLFW を初期化する
00048     if (glfwInit() == GLFW_FALSE) throw std::runtime_error("Can't initialize GLFW");
00049
00050     // OpenGL の major 番号が指定されていれば
00051     if (major > 0)
00052     {
00053         // OpenGL のバージョンを指定する
00054         glfwWindowHint(GLFW_CONTEXT_VERSION_MAJOR, major);
00055         glfwWindowHint(GLFW_CONTEXT_VERSION_MINOR, minor);
00056     }
00057
00058     #if defined(GL_GLES_PROTOTYPES)
00059         // OpenGL ES 3 のコンテキストを指定する
00060         glfwWindowHint(GLFW_CLIENT_API, GLFW_OPENGL_ES_API);
00061         glfwWindowHint(GLFW_CONTEXT_CREATION_API, GLFW_EGL_CONTEXT_API);
00062     #else
00063         // OpenGL Version 3.2 以降なら
00064     #endif
00065 }
```



```

00070     if (major * 10 + minor >= 32)
00071     {
00072         // Core Profile を選択する (macOS の都合)
00073         glfwWindowHint(GLFW_OPENGL_FORWARD_COMPAT, GL_TRUE);
00074         glfwWindowHint(GLFW_OPENGL_PROFILE, GLFW_OPENGL_CORE_PROFILE);
00075     }
00076 #endif
00077 }
00078
00079 #if defined(IMGUI_VERSION)
00080 // ImGui のバージョンをチェックする
00081 ImGui::CheckVersion();
00082
00083 // ImGui のコンテキストを作成する
00084 ImGui::CreateContext();
00085 #endif
00086 }
00087
00088 //
00089 // デストラクタ
00090 //
00091 GgApp::~GgApp()
00092 {
00093     #if defined(IMGUI_VERSION)
00094     // Shutdown Platform/Renderer bindings
00095     ImGui_ImplOpenGL3_Shutdown();
00096     ImGui_ImplGlfw_Shutdown();
00097     ImGui::DestroyContext();
00098     #endif
00099
00100     // プログラム終了時に GLFW を終了する
00101     glfwTerminate();
00102 }
00103
00104 //
00105 // マウスや矢印キーによる平行移動量を初期化する
00106 //
00107 void GgApp::Window::HumanInterface::resetTranslation()
00108 {
00109     // 平行移動量を初期化する
00110     for (auto& t : translation)
00111     {
00112         std::fill(t.begin(), t.end(), GgVector{ 0.0f, 0.0f, 0.0f, 1.0f });
00113     }
00114
00115     // 矢印キーの設定値を初期化する
00116     std::fill(arrow.begin(), arrow.end(), std::array<int, 2>{ 0, 0 });
00117
00118     // マウスホイールの回転量を初期化する
00119     std::fill(wheel.begin(), wheel.end(), 0.0f);
00120 }
00121
00122 //
00123 // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00124 //
00125 void GgApp::Window::HumanInterface::calcTranslation(int button, const std::array<GLfloat, 3>&
velocity)
00126 {
00127     // マウスの相対変位
00128     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00129     const auto dx{ (mouse[0] - rotation[button].getStart(0)) * rotation[button].getScale(0) };
00130     const auto dy{ (rotation[button].getStart(1) - mouse[1]) * rotation[button].getScale(1) };
00131
00132     // 平行移動量
00133     auto& t{ translation[button] };
00134
00135     // 平行移動量の更新
00136     t[1][0] = dx * velocity[0] + t[0][0];
00137     t[1][1] = dy * velocity[1] + t[0][1];
00138
00139     // 回転量の更新
00140     rotation[button].motion(mouse[0], mouse[1]);
00141 }
00142
00143 //
00144 // ウィンドウのサイズ変更時の処理
00145 //
00146 void GgApp::Window::resize(GLFWwindow* window, int width, int height)
00147 {
00148     // このインスタンスの this ポインタを得る
00149     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00150
00151     if (instance)
00152     {
00153         // ウィンドウのサイズを保存する
00154         instance->size[0] = width;
00155         instance->size[1] = height;

```

```

00156
00157 // トラックボール処理の範囲を設定する
00158 for (auto& current_if : instance->interfaceData)
00159 {
00160     for (auto& t : current_if.rotation)
00161     {
00162         t.region(width, height);
00163     }
00164 }
00165
00166 // ビューポートを更新する
00167 instance->updateViewport();
00168
00169 // ユーザー定義のコールバック関数の呼び出し
00170 if (instance->resizeFunc) (*instance->resizeFunc)(instance, width, height);
00171 }
00172 }
00173
00174 //
00175 // キーボードをタイプした時の処理
00176 //
00177 void GgApp::Window::keyboard(GLFWwindow* window, int key, int scancode, int action, int mods)
00178 {
00179     #if defined(IMGUI_VERSION)
00180     // ImGui がキーボードを使うときはキーボードの処理を行わない
00181     if (ImGui::GetIO().WantCaptureKeyboard) return;
00182     #endif
00183
00184     // このインスタンスの this ポインタを得る
00185     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00186
00187     if (instance && action)
00188     {
00189         // ユーザー定義のコールバック関数の呼び出し
00190         if (instance->keyboardFunc) (*instance->keyboardFunc)(instance, key, scancode, action, mods);
00191
00192         // 対象のユーザインタフェース
00193         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00194
00195         switch (key)
00196         {
00197             case GLFW_KEY_HOME:
00198
00199                 // トラックボールを初期化する
00200                 instance->resetRotation();
00201                 [[fallthrough]];
00202
00203             case GLFW_KEY_END:
00204
00205                 // 平行移動量を初期化する
00206                 instance->resetTranslation();
00207                 break;
00208
00209             case GLFW_KEY_UP:
00210
00211                 if (mods & GLFW_MOD_SHIFT)
00212                     current_if.arrow[1][1]++;
00213                 else if (mods & GLFW_MOD_CONTROL)
00214                     current_if.arrow[2][1]++;
00215                 else if (mods & GLFW_MOD_ALT)
00216                     current_if.arrow[3][1]++;
00217                 else
00218                     current_if.arrow[0][1]++;
00219                 break;
00220
00221             case GLFW_KEY_DOWN:
00222
00223                 if (mods & GLFW_MOD_SHIFT)
00224                     current_if.arrow[1][1]--;
00225                 else if (mods & GLFW_MOD_CONTROL)
00226                     current_if.arrow[2][1]--;
00227                 else if (mods & GLFW_MOD_ALT)
00228                     current_if.arrow[3][1]--;
00229                 else
00230                     current_if.arrow[0][1]--;
00231                 break;
00232
00233             case GLFW_KEY_RIGHT:
00234
00235                 if (mods & GLFW_MOD_SHIFT)
00236                     current_if.arrow[1][0]++;
00237                 else if (mods & GLFW_MOD_CONTROL)
00238                     current_if.arrow[2][0]++;
00239                 else if (mods & GLFW_MOD_ALT)
00240                     current_if.arrow[3][0]++;
00241                 else
00242                     current_if.arrow[0][0]++;

```

```

00243         break;
00244
00245     case GLFW_KEY_LEFT:
00246
00247         if (mods & GLFW_MOD_SHIFT)
00248             current_if.arrow[1][0]--;
00249         else if (mods & GLFW_MOD_CONTROL)
00250             current_if.arrow[2][0]--;
00251         else if (mods & GLFW_MOD_ALT)
00252             current_if.arrow[3][0]--;
00253         else
00254             current_if.arrow[0][0]--;
00255         break;
00256
00257     default:
00258         break;
00259     }
00260
00261     current_if.lastKey = key;
00262 }
00263 }
00264
00265 //
00266 // マウスボタンを操作したときの処理
00267 //
00268 void GgApp::Window::mouse(GLFWwindow* window, int button, int action, int mods)
00269 {
00270     #if defined(IMGUI_VERSION)
00271         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00272         if (ImGui::GetIO().WantCaptureMouse) return;
00273     #endif
00274
00275     // このインスタンスの this ポインタを得る
00276     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00277
00278     // マウスボタンの状態を記録する
00279     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00280     instance->status[button] = action != GLFW_RELEASE;
00281
00282     if (instance)
00283     {
00284         // ユーザー定義のコールバック関数の呼び出し
00285         if (instance->mouseFunc) (*instance->mouseFunc)(instance, button, action, mods);
00286
00287         // 対象のユーザインタフェース
00288         auto& current_if{ instance->interfaceData[instance->interfaceNo] };
00289
00290         // マウスの現在位置を得る
00291         const auto x{ current_if.mouse[0] };
00292         const auto y{ current_if.mouse[1] };
00293
00294         if (x < 0 || x >= instance->size[0] || y < 0 || y >= instance->size[1]) return;
00295
00296         if (action)
00297         {
00298             // ドラッグ開始
00299             current_if.rotation[button].begin(x, y);
00300         }
00301         else
00302         {
00303             // ドラッグ終了
00304             current_if.translation[button][0] = current_if.translation[button][1];
00305             current_if.rotation[button].end(x, y);
00306         }
00307     }
00308 }
00309
00310 //
00311 // マウスホイールを操作した時の処理
00312 //
00313 void GgApp::Window::wheel(GLFWwindow* window, double x, double y)
00314 {
00315     #if defined(IMGUI_VERSION)
00316         // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00317         if (ImGui::GetIO().WantCaptureMouse) return;
00318     #endif
00319
00320     // このインスタンスの this ポインタを得る
00321     auto* const instance{ static_cast<Window*>(glfwGetWindowUserPointer(window)) };
00322
00323     if (instance)
00324     {
00325         // ユーザー定義のコールバック関数の呼び出し
00326         if (instance->wheelFunc) (*instance->wheelFunc)(instance, x, y);
00327
00328         // 対象のユーザインタフェース
00329         auto& current_if{ instance->interfaceData[instance->interfaceNo] };

```

```

00330
00331 // マウスホイールの回転量の保存
00332 current.if.wheel[0] += static_cast<GLfloat>(x);
00333 current.if.wheel[1] += static_cast<GLfloat>(y);
00334
00335 // マウスによる平行移動量の z 値の更新
00336 const auto z{ current.if.wheel[1] * instance->velocity[2] };
00337 for (auto& t : current.if.translation) t[1][2] = z;
00338 }
00339 }
00340
00341 //
00342 // Window クラスのコンストラクタ
00343 //
00344 GgApp::Window::Window(const std::string& title, int width, int height, int fullscreen, GLFWwindow*
share) :
00345     size{ width, height },
00346     fboSize{ width, height }
00347 {
00348     // ディスプレイの情報
00349     GLFWmonitor* monitor{ nullptr };
00350
00351     // フルスクリーン表示
00352     if (fullscreen > 0)
00353     {
00354         // 接続されているモニタの数を数える
00355         int mcount;
00356         auto** const monitors{ glfwGetMonitors(&mcount) };
00357
00358         // セカンダリモニタがあればそれを使う
00359         if (fullscreen > mcount) fullscreen = mcount;
00360         monitor = monitors[fullscreen - 1];
00361
00362         // モニタのモードを調べる
00363         const auto* mode{ glfwGetVideoMode(monitor) };
00364
00365         // ウィンドウのサイズをディスプレイのサイズにする
00366         width = mode->width;
00367         height = mode->height;
00368     }
00369
00370     // GLFW のウィンドウを作成する
00371     window = glfwCreateWindow(width, height, title.c_str(), monitor, share);
00372
00373     // ウィンドウが作成できなければエラー
00374     if (!window) throw std::runtime_error("Unable to open the GLFW window.");
00375
00376     // 現在のウィンドウを処理対象にする
00377     glfwMakeContextCurrent(window);
00378
00379     // ゲームグラフィックス特論の都合による初期化を行う
00380     ggInit();
00381
00382     // このインスタンスの this ポインタを記録しておく
00383     glfwSetWindowUserPointer(window, this);
00384
00385     // キーボードを操作した時の処理を登録する
00386     glfwSetKeyCallback(window, keyboard);
00387
00388     // マウスボタンを操作したときの処理を登録する
00389     glfwSetMouseButtonCallback(window, mouse);
00390
00391     // マウスホイール操作時に呼び出す処理を登録する
00392     glfwSetScrollCallback(window, wheel);
00393
00394     // ウィンドウのサイズ変更時に呼び出す処理を登録する
00395     glfwSetFramebufferSizeCallback(window, resize);
00396
00397     // 垂直同期タイミングに合わせる
00398     glfwSwapInterval(1);
00399
00400     // 実際のフレームバッファのサイズを取得する
00401     glfwGetFramebufferSize(window, &width, &height);
00402
00403     // ビューポートと投影変換行列を初期化する
00404     resize(window, width, height);
00405
00406 #if defined(IMGUI_VERSION)
00407     // 最初のウィンドウを開いたとき
00408     static bool firstTime{ true };
00409     if (firstTime)
00410     {
00411         // Setup Platform/Renderer bindings
00412         ImGui_ImplGlfw_InitForOpenGL(window, true);
00413         ImGui_ImplOpenGL3_Init(nullptr);
00414
00415         // 実行済みであることを記録する

```

```

00416     firstTime = false;
00417 }
00418 #endif
00419 }
00420
00421 //
00422 // Window クラスのムーブコンストラクタ
00423 //
00424 GgApp::Window::Window(Window&& w) noexcept :
00425     window{ w.window },
00426     size{ w.size },
00427     fboSize{ w.fboSize },
00428     interfaceData{ std::move(w.interfaceData) },
00429     interfaceNo{ w.interfaceNo },
00430     userPointer{ w.userPointer },
00431     resizeFunc{ w.resizeFunc },
00432     keyboardFunc{ w.keyboardFunc },
00433     mouseFunc{ w.mouseFunc },
00434     wheelFunc{ w.wheelFunc }
00435 {
00436     w.window = nullptr;
00437     if (window)
00438     {
00439         glfwSetWindowUserPointer(window, this);
00440     }
00441 }
00442
00443 //
00444 // Window クラスのムーブ代入演算子
00445 //
00446 GgApp::Window& GgApp::Window::operator=(Window&& w) noexcept
00447 {
00448     if (&w != this)
00449     {
00450         if (window)
00451         {
00452             glfwDestroyWindow(window);
00453         }
00454         window = w.window;
00455         size = w.size;
00456         fboSize = w.fboSize;
00457         interfaceData = std::move(w.interfaceData);
00458         interfaceNo = w.interfaceNo;
00459         userPointer = w.userPointer;
00460         resizeFunc = w.resizeFunc;
00461         keyboardFunc = w.keyboardFunc;
00462         mouseFunc = w.mouseFunc;
00463         wheelFunc = w.wheelFunc;
00464
00465         w.window = nullptr;
00466         if (window)
00467         {
00468             glfwSetWindowUserPointer(window, this);
00469         }
00470     }
00471     return *this;
00472 }
00473
00474 //
00475 // イベントを取得してループを継続すべきかどうか調べる
00476 //
00477 GgApp::Window::operator bool()
00478 {
00479     // イベントを取り出す
00480     glfwPollEvents();
00481
00482     // ウィンドウを閉じるべきなら false を返す
00483     if (glfwShouldClose()) return false;
00484
00485     // 対象のユーザインタフェース
00486     auto& current_if{ interfaceData[interfaceNo] };
00487
00488     #if defined(IMGUI_VERSION)
00489     // ImGui の新規フレームを作成する
00490     ImGui_ImplOpenGL3_NewFrame();
00491     ImGui_ImplGlfw_NewFrame();
00492     ImGui::NewFrame();
00493
00494     // ImGui の状態を取り出す
00495     const ImGuiIO& io{ ImGui::GetIO() };
00496
00497     // ImGui がマウスを使うときは Window クラスのマウス位置を更新しない
00498     if (io.WantCaptureMouse) return true;
00499
00500     // マウスの位置を更新する
00501     current_if.mouse = std::array<GLfloat, 2>{ io.MousePos.x, io.MousePos.y };
00502     #else

```

```

00503 // マウスの現在位置を調べる
00504 double x, y;
00505 glfwGetCursorPos(window, &x, &y);
00506
00507 // マウスの位置を更新する
00508 current_if.mouse = std::array<GLfloat, 2>{ static_cast<GLfloat>(x), static_cast<GLfloat>(y) };
00509 #endif
00510
00511 // マウスドラッグ
00512 for (int button = GLFW_MOUSE_BUTTON_1; button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT; ++button)
00513 {
00514     // マウスボタンを押していたら
00515     if (status[button])
00516     {
00517         // 現在位置と平行移動量を更新する
00518         current_if.calcTranslation(button, velocity);
00519     }
00520 }
00521
00522 return true;
00523 }
00524
00525 //
00526 // カラーバッファを入れ替える
00527 //
00528 void GgApp::Window::swapBuffers() const
00529 {
00530     #if defined(IMGUI_VERSION)
00531         // ImGui の描画データがあればフレームをレンダリングする
00532         ImGui::Render();
00533         ImDrawData* data{ ImGui::GetDrawData() };
00534         if (data) ImGui::ImplOpenGL3.RenderDrawData(data);
00535     #endif
00536
00537     // エラーチェック
00538     ggError();
00539
00540     // カラーバッファを入れ替える
00541     glfwSwapBuffers(window);
00542 }
00543
00544 //
00545 // ビューポートのサイズを更新する
00546 //
00547 void GgApp::Window::updateViewport()
00548 {
00549     // フレームバッファの大きさを求める
00550     glfwGetFramebufferSize(window, &fboSize[0], &fboSize[1]);
00551
00552     #if defined(IMGUI_VERSION)
00553         // フレームバッファの高さからメニューバーの高さを減じる
00554         fboSize[1] -= menubarHeight;
00555     #endif
00556
00557     // ウィンドウの縦横比を保存する
00558     aspect = static_cast<GLfloat>(fboSize[0]) / static_cast<GLfloat>(fboSize[1]);
00559
00560     // ビューポートを設定する
00561     restoreViewport();
00562 }
00563
00564 #if defined(GG_USE_OPENXR)
00565 namespace
00566 {
00567     //
00568     // OpenXR の関数の戻り値を文字列にする
00569     //
00570     std::string xrMessage(XrInstance instance, XrResult result, const std::string& message)
00571     {
00572         char buffer[XR_MAX_RESULT_STRING_SIZE]{ '\0' };
00573         if (instance == XR_NULL_HANDLE
00574             || XR_FAILED(xrResultToString(instance, result, buffer))
00575             || buffer[0] == '\0')
00576         {
00577             std::snprintf(buffer, sizeof buffer, "XrResult(%d)", static_cast<int>(result));
00578         }
00579         return message + ": " + buffer;
00580     }
00581 }
00582
00583 //
00584 // OpenXR の関数の戻り値を検査して、エラーなら例外を投げる
00585 //
00586 void xrCheck(XrInstance instance, XrResult result, const std::string& message)
00587 {
00588     if (XR_SUCCEEDED(result)) return;
00589     throw std::runtime_error(xrMessage(instance, result, message));

```

```

00590     }
00591
00592     //
00593     // OpenXR の関数の戻り値を検査して、エラーなら標準エラー出力に報告する
00594     //
00595     bool xrWarn(XrInstance instance, XrResult result, const std::string& message)
00596     {
00597         if (XR_SUCCEEDED(result)) return true;
00598         std::cerr << "OpenXR: " << xrMessage(instance, result, message) << '\n';
00599         return false;
00600     }
00601
00602     //
00603     // 固定長の文字列に安全にコピーする
00604     //
00605     void xrCopyString(char* destination, size_t size, const char* source)
00606     {
00607         if (size == 0) return;
00608         const auto length{ std::min(std::strlen(source), size - 1) };
00609         std::memcpy(destination, source, length);
00610         destination[length] = '\0';
00611     }
00612 }
00613
00614 //
00615 // コンストラクタ
00616 //
00617 GgApp::OpenXR::OpenXR()
00618 {
00619 }
00620
00621 //
00622 // デストラクタ
00623 //
00624 GgApp::OpenXR::~OpenXR()
00625 {
00626     // このオブジェクトは関数内 static なので、破棄されるのは main() が
00627     // 終了した後、すなわち OpenGL のコンテキストが失われた後である。
00628     // したがってここでは OpenGL の資源には触れず、OpenXR のハンドルだけを
00629     // 解放する (OpenGL の資源は terminate() で解放しておくこと)。
00630     destroyXr();
00631 }
00632
00633 //
00634 // アクションシステムを初期化する
00635 //
00636 void GgApp::OpenXR::initActions()
00637 {
00638     // アクションセットの作成
00639     XrActionSetCreateInfo actionSetInfo{ XR_TYPE_ACTION_SET_CREATE_INFO };
00640     xrCopyString(actionSetInfo.actionSetName, sizeof actionSetInfo.actionSetName, "gameplay");
00641     xrCopyString(actionSetInfo.localizedActionSetName, sizeof actionSetInfo.localizedActionSetName,
00642 "Gameplay");
00643     actionSetInfo.priority = 0;
00644     xrCheck(instance, xrCreateActionSet(instance, &actionSetInfo, &actionSet),
00645 "Can't create the OpenXR action set");
00646
00647     // サブアクションパスの取得
00648     xrCheck(instance, xrStringToPath(instance, "/user/hand/left", &handSubactionPath[Hand::Left]),
00649 "Can't convert the path of the left hand");
00650     xrCheck(instance, xrStringToPath(instance, "/user/hand/right", &handSubactionPath[Hand::Right]),
00651 "Can't convert the path of the right hand");
00652
00653     // アクション作成ヘルパー
00654     auto createAction = [this](const char* name, const char* localizedName, XrActionType type, XrAction&
00655 action)
00656     {
00657         XrActionCreateInfo createInfo{ XR_TYPE_ACTION_CREATE_INFO };
00658         xrCopyString(createInfo.actionName, sizeof createInfo.actionName, name);
00659         xrCopyString(createInfo.localizedActionName, sizeof createInfo.localizedActionName,
00660 localizedName);
00661         createInfo.actionType = type;
00662         createInfo.countSubactionPaths = Hand::Count;
00663         createInfo.subactionPaths = handSubactionPath;
00664         xrCheck(instance, xrCreateAction(actionSet, &createInfo, &action),
00665 std::string("Can't create the OpenXR action \"") + name + "\"");
00666     };
00667
00668     createAction("aim_pose", "Aim Pose", XR_ACTION_TYPE_POSE_INPUT, aimPoseAction);
00669     createAction("grip_pose", "Grip Pose", XR_ACTION_TYPE_POSE_INPUT, gripPoseAction);
00670     createAction("trigger", "Trigger", XR_ACTION_TYPE_FLOAT_INPUT, triggerAction);
00671     createAction("grip", "Grip", XR_ACTION_TYPE_FLOAT_INPUT, gripAction);
00672     createAction("thumbstick", "Thumbstick", XR_ACTION_TYPE_VECTOR2F_INPUT, thumbstickAction);
00673     createAction("thumbstick_click", "Thumbstick Click", XR_ACTION_TYPE_BOOLEAN_INPUT,
00674 thumbstickClickAction);
00675     createAction("primary_button", "Primary Button", XR_ACTION_TYPE_BOOLEAN_INPUT, primaryButtonAction);
00676     createAction("secondary_button", "Secondary Button", XR_ACTION_TYPE_BOOLEAN_INPUT,

```

```

secondaryButtonAction);
00673 createAction("menu.button", "Menu Button", XR_ACTION_TYPE_BOOLEAN_INPUT, menuButtonAction);
00674 createAction("haptic", "Haptic Vibration", XR_ACTION_TYPE_VIBRATION_OUTPUT, hapticAction);
00675
00676 // バインディング設定ヘルパー (対応していない対話プロファイルは読み飛ばす)
00677 auto suggestBindings = [this](const char* profileStr, const std::vector<std::pair<XrAction, const
char*>>& bindings)
00678 {
00679     XrPath profilePath{ XR_NULL_PATH };
00680     if (XR_FAILED(xrStringToPath(instance, profileStr, &profilePath))) return;
00681
00682     std::vector<XrActionSuggestedBinding> suggestedBindings;
00683     suggestedBindings.reserve(bindings.size());
00684     for (const auto& [action, pathStr] : bindings)
00685     {
00686         XrPath path{ XR_NULL_PATH };
00687         if (XR_FAILED(xrStringToPath(instance, pathStr, &path))) continue;
00688         suggestedBindings.push_back(XrActionSuggestedBinding{ action, path });
00689     }
00690     if (suggestedBindings.empty()) return;
00691
00692     XrInteractionProfileSuggestedBinding profileSuggestedBindings{
XR_TYPE_INTERACTION_PROFILE_SUGGESTED_BINDING };
00693     profileSuggestedBindings.interactionProfile = profilePath;
00694     profileSuggestedBindings.suggestedBindings = suggestedBindings.data();
00695     profileSuggestedBindings.countSuggestedBindings = static_cast<uint32_t>(suggestedBindings.size());
00696     xrWarn(instance, xrSuggestInteractionProfileBindings(instance, &profileSuggestedBindings),
std::string("Can't suggest the bindings for ") + profileStr);
00697 };
00698
00699 // Simple Controller のバインディング (すべてのランタイムが対応する最小限のもの)
00700 suggestBindings("/interaction_profiles/khr/simple_controller", {
00701     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00702     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00703     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00704     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00705     { triggerAction, "/user/hand/left/input/select/click" },
00706     { triggerAction, "/user/hand/right/input/select/click" },
00707     { menuButtonAction, "/user/hand/left/input/menu/click" },
00708     { menuButtonAction, "/user/hand/right/input/menu/click" },
00709     { hapticAction, "/user/hand/left/output/haptic" },
00710     { hapticAction, "/user/hand/right/output/haptic" }
00711 });
00712
00713 // Meta (Oculus) Touch コントローラーのバインディング
00714 suggestBindings("/interaction_profiles/oculus/touch_controller", {
00715     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00716     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00717     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00718     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00719     { triggerAction, "/user/hand/left/input/trigger/value" },
00720     { triggerAction, "/user/hand/right/input/trigger/value" },
00721     { gripAction, "/user/hand/left/input/squeeze/value" },
00722     { gripAction, "/user/hand/right/input/squeeze/value" },
00723     { thumbstickAction, "/user/hand/left/input/thumbstick" },
00724     { thumbstickAction, "/user/hand/right/input/thumbstick" },
00725     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },
00726     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
00727     { primaryButtonAction, "/user/hand/left/input/x/click" },
00728     { primaryButtonAction, "/user/hand/right/input/a/click" },
00729     { secondaryButtonAction, "/user/hand/left/input/y/click" },
00730     { secondaryButtonAction, "/user/hand/right/input/b/click" },
00731     { menuButtonAction, "/user/hand/left/input/menu/click" },
00732     { hapticAction, "/user/hand/left/output/haptic" },
00733     { hapticAction, "/user/hand/right/output/haptic" }
00734 });
00735
00736 // HTC Vive コントローラーのバインディング
00737 suggestBindings("/interaction_profiles/htc/vive_controller", {
00738     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00739     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00740     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00741     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00742     { triggerAction, "/user/hand/left/input/trigger/value" },
00743     { triggerAction, "/user/hand/right/input/trigger/value" },
00744     { gripAction, "/user/hand/left/input/squeeze/click" },
00745     { gripAction, "/user/hand/right/input/squeeze/click" },
00746     { thumbstickAction, "/user/hand/left/input/trackpad" },
00747     { thumbstickAction, "/user/hand/right/input/trackpad" },
00748     { thumbstickClickAction, "/user/hand/left/input/trackpad/click" },
00749     { thumbstickClickAction, "/user/hand/right/input/trackpad/click" },
00750     { menuButtonAction, "/user/hand/left/input/menu/click" },
00751     { menuButtonAction, "/user/hand/right/input/menu/click" },
00752     { hapticAction, "/user/hand/left/output/haptic" },
00753     { hapticAction, "/user/hand/right/output/haptic" }
00754 });
00755
00756

```



```

00757 // Valve Index コントローラーのバインディング
00758 suggestBindings("/interaction_profiles/valve/index.controller", {
00759     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00760     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00761     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00762     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00763     { triggerAction, "/user/hand/left/input/trigger/value" },
00764     { triggerAction, "/user/hand/right/input/trigger/value" },
00765     { gripAction, "/user/hand/left/input/squeeze/value" },
00766     { gripAction, "/user/hand/right/input/squeeze/value" },
00767     { thumbstickAction, "/user/hand/left/input/thumbstick" },
00768     { thumbstickAction, "/user/hand/right/input/thumbstick" },
00769     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },
00770     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
00771     { primaryButtonAction, "/user/hand/left/input/a/click" },
00772     { primaryButtonAction, "/user/hand/right/input/a/click" },
00773     { secondaryButtonAction, "/user/hand/left/input/b/click" },
00774     { secondaryButtonAction, "/user/hand/right/input/b/click" },
00775     { hapticAction, "/user/hand/left/output/haptic" },
00776     { hapticAction, "/user/hand/right/output/haptic" }
00777 });
00778
00779 // Microsoft Mixed Reality モーションコントローラーのバインディング
00780 suggestBindings("/interaction_profiles/microsoft/motion.controller", {
00781     { aimPoseAction, "/user/hand/left/input/aim/pose" },
00782     { aimPoseAction, "/user/hand/right/input/aim/pose" },
00783     { gripPoseAction, "/user/hand/left/input/grip/pose" },
00784     { gripPoseAction, "/user/hand/right/input/grip/pose" },
00785     { triggerAction, "/user/hand/left/input/trigger/value" },
00786     { triggerAction, "/user/hand/right/input/trigger/value" },
00787     { gripAction, "/user/hand/left/input/squeeze/click" },
00788     { gripAction, "/user/hand/right/input/squeeze/click" },
00789     { thumbstickAction, "/user/hand/left/input/thumbstick" },
00790     { thumbstickAction, "/user/hand/right/input/thumbstick" },
00791     { thumbstickClickAction, "/user/hand/left/input/thumbstick/click" },
00792     { thumbstickClickAction, "/user/hand/right/input/thumbstick/click" },
00793     { menuButtonAction, "/user/hand/left/input/menu/click" },
00794     { menuButtonAction, "/user/hand/right/input/menu/click" },
00795     { hapticAction, "/user/hand/left/output/haptic" },
00796     { hapticAction, "/user/hand/right/output/haptic" }
00797 });
00798
00799 // セッションにアクションセットをアタッチする (これ以降はバインディングを追加できない)
00800 XrSessionActionSetsAttachInfo attachInfo{ XR_TYPE_SESSION_ACTION_SETS_ATTACH_INFO };
00801 attachInfo.countActionSets = 1;
00802 attachInfo.actionSets = &actionSet;
00803 xrCheck(instance, xrAttachSessionActionSets(session, &attachInfo),
00804     "Can't attach the OpenXR action set to the session");
00805
00806 // アクションスペースの作成
00807 for (int i = 0; i < Hand::Count; ++i)
00808 {
00809     XrActionSpaceCreateInfo spaceInfo{ XR_TYPE_ACTION_SPACE_CREATE_INFO };
00810     spaceInfo.poseInActionSpace.orientation.w = 1.0f;
00811     spaceInfo.subactionPath = handSubactionPath[i];
00812
00813     spaceInfo.action = aimPoseAction;
00814     xrCheck(instance, xrCreateActionSpace(session, &spaceInfo, &aimSpace[i]),
00815         "Can't create the OpenXR action space for the aim pose");
00816
00817     spaceInfo.action = gripPoseAction;
00818     xrCheck(instance, xrCreateActionSpace(session, &spaceInfo, &gripSpace[i]),
00819         "Can't create the OpenXR action space for the grip pose");
00820 }
00821 }
00822
00823 //
00824 // アクション状態を更新する
00825 //
00826 void GgApp::OpenXR::pollActions()
00827 {
00828     // 入力を受け付けていなければコントローラーの状態を無効にする
00829     if (!isSessionRunning || actionSet == XR_NULL_HANDLE || sessionState != XR_SESSION_STATE_FOCUSED)
00830     {
00831         for (auto& state : controllerStates) state = ControllerState{};
00832         return;
00833     }
00834
00835     XrActiveActionSet activeActionSet{ actionSet, XR_NULL_PATH };
00836     XrActionsSyncInfo syncInfo{ XR_TYPE_ACTIONS_SYNC_INFO };
00837     syncInfo.countActiveActionSets = 1;
00838     syncInfo.activeActionSets = &activeActionSet;
00839     if (!xrWarn(instance, xrSyncActions(session, &syncInfo),
00840         "Can't synchronize the OpenXR actions")) return;
00841
00842     // 空間の姿勢を取り出すヘルパー (位置と向きの両方が有効なときだけ更新する)
00843     auto locate = [this](XrSpace space, XrPosef& pose)

```

```

00844 {
00845     if (space == XR_NULL_HANDLE) return false;
00846
00847     XrSpaceLocation location{ XR_TYPE_SPACE_LOCATION };
00848     if (XR_FAILED(xrLocateSpace(space, appSpace, frameState.predictedDisplayTime, &location)))
00849         return false;
00850
00851     constexpr XrSpaceLocationFlags valid
00852     {
00853         XR_SPACE_LOCATION_POSITION_VALID_BIT | XR_SPACE_LOCATION_ORIENTATION_VALID_BIT
00854     };
00855     if ((location.locationFlags & valid) != valid) return false;
00856
00857     pose = location.pose;
00858     return true;
00859 };
00860
00861 for (int i = 0; i < Hand::Count; ++i)
00862 {
00863     auto& state = controllerStates[i];
00864     const XrPath subaction = handSubactionPath[i];
00865
00866     XrActionStateGetInfo getInfo{ XR_TYPE_ACTION_STATE_GET_INFO };
00867     getInfo.subactionPath = subaction;
00868
00869     // グリップポーズの取得
00870     XrActionStatePose gripPoseState{ XR_TYPE_ACTION_STATE_POSE };
00871     getInfo.action = gripPoseAction;
00872     const bool gripActive
00873     {
00874         XR_SUCCEEDED(xrGetActionStatePose(session, &getInfo, &gripPoseState))
00875         && gripPoseState.isActive != XR_FALSE
00876     };
00877
00878     // エイムポーズの取得
00879     XrActionStatePose aimPoseState{ XR_TYPE_ACTION_STATE_POSE };
00880     getInfo.action = aimPoseAction;
00881     const bool aimActive
00882     {
00883         XR_SUCCEEDED(xrGetActionStatePose(session, &getInfo, &aimPoseState))
00884         && aimPoseState.isActive != XR_FALSE
00885     };
00886
00887     // どちらかの姿勢が有効ならコントローラーが接続されている
00888     state.isTracked = gripActive || aimActive;
00889     if (gripActive) locate(gripSpace[i], state.gripPose);
00890     if (aimActive) locate(aimSpace[i], state.aimPose);
00891
00892     // 連続値のアクションを取り出すヘルパー
00893     auto getFloat = [this, &getInfo](XrAction action, float& value)
00894     {
00895         getInfo.action = action;
00896         XrActionStateFloat floatState{ XR_TYPE_ACTION_STATE_FLOAT };
00897         value = XR_SUCCEEDED(xrGetActionStateFloat(session, &getInfo, &floatState))
00898             && floatState.isActive != XR_FALSE ? floatState.currentState : 0.0f;
00899     };
00900
00901     // 論理値のアクションを取り出すヘルパー
00902     auto getBoolean = [this, &getInfo](XrAction action, bool& value)
00903     {
00904         getInfo.action = action;
00905         XrActionStateBoolean booleanState{ XR_TYPE_ACTION_STATE_BOOLEAN };
00906         value = XR_SUCCEEDED(xrGetActionStateBoolean(session, &getInfo, &booleanState))
00907             && booleanState.isActive != XR_FALSE && booleanState.currentState != XR_FALSE;
00908     };
00909
00910     // トリガーとグリップ
00911     getFloat(triggerAction, state.trigger);
00912     getFloat(gripAction, state.grip);
00913
00914     // スティック
00915     getInfo.action = thumbstickAction;
00916     XrActionStateVector2f thumbstickState{ XR_TYPE_ACTION_STATE_VECTOR2F };
00917     if (XR_SUCCEEDED(xrGetActionStateVector2f(session, &getInfo, &thumbstickState))
00918         && thumbstickState.isActive != XR_FALSE)
00919         state.thumbstick = { thumbstickState.currentState.x, thumbstickState.currentState.y };
00920     else
00921         state.thumbstick = { 0.0f, 0.0f };
00922
00923     // ボタン
00924     getBoolean(thumbstickClickAction, state.thumbstickClick);
00925     getBoolean(primaryButtonAction, state.primaryButton);
00926     getBoolean(secondaryButtonAction, state.secondaryButton);
00927     getBoolean(menuButtonAction, state.menuButton);
00928 }
00929 }
00930

```

```

00931 //
00932 // OpenXR のイベントを処理する
00933 //
00934 void GgApp::OpenXR::pollEvents()
00935 {
00936     XrEventDataBuffer eventData{ XR_TYPE_EVENT_DATA_BUFFER };
00937     while (xrPollEvent(instance, &eventData) == XR_SUCCESS)
00938     {
00939         switch (eventData.type)
00940         {
00941             case XR_TYPE_EVENT_DATA_INSTANCE_LOSS_PENDING:
00942                 // OpenXR のインスタンスが失われるのでアプリケーションを終了する
00943                 isSessionRunning = false;
00944                 if (window) window->setClose(GLFW_TRUE);
00945                 break;
00946             case XR_TYPE_EVENT_DATA_SESSION_STATE_CHANGED:
00947             {
00948                 const auto* stateChanged{ reinterpret_cast<const XrEventDataSessionStateChanged*>(&eventData) };
00949                 sessionState = stateChanged->state;
00950                 switch (sessionState)
00951                 {
00952                     case XR_SESSION_STATE_READY:
00953                     {
00954                         // セッションを開始する
00955                         XrSessionBeginInfo beginInfo{ XR_TYPE_SESSION_BEGIN_INFO };
00956                         beginInfo.primaryViewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
00957                         if (xrWarn(instance, xrBeginSession(session, &beginInfo),
00958                             "Can't begin the OpenXR session")) isSessionRunning = true;
00959                         break;
00960                     }
00961                     case XR_SESSION_STATE_STOPPING:
00962                     {
00963                         // セッションを終了する
00964                         isSessionRunning = false;
00965                         frameBegun = false;
00966                         xrWarn(instance, xrEndSession(session), "Can't end the OpenXR session");
00967                         break;
00968                     }
00969                     case XR_SESSION_STATE_EXITING:
00970                     case XR_SESSION_STATE_LOSS_PENDING:
00971                     {
00972                         // アプリケーションを終了する
00973                         isSessionRunning = false;
00974                         if (window) window->setClose(GLFW_TRUE);
00975                         break;
00976                     }
00977                     default:
00978                     {
00979                         break;
00980                     }
00981                 }
00982             }
00983             default:
00984             {
00985                 break;
00986             }
00987         }
00988         // 次のイベントを取り出す準備をする
00989         eventData = XrEventDataBuffer{ XR_TYPE_EVENT_DATA_BUFFER };
00990     }
00991 }
00992 //
00993 // スワップチェーンを作成する
00994 //
00995 void GgApp::OpenXR::createSwapchains()
00996 {
00997     // ビュー構成を取得する
00998     uint32_t viewCount{ 0 };
00999     xrCheck(instance, xrEnumerateViewConfigurationViews(instance, systemId,
01000         XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, 0, &viewCount, nullptr),
01001         "Can't count the OpenXR view configuration views");
01002     views.assign(viewCount, XrViewConfigurationView{ XR_TYPE_VIEW_CONFIGURATION_VIEW });
01003     xrCheck(instance, xrEnumerateViewConfigurationViews(instance, systemId,
01004         XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, viewCount, &viewCount, views.data()),
01005         "Can't enumerate the OpenXR view configuration views");
01006     if (viewCount == 0) throw std::runtime_error("The OpenXR system has no view");
01007     viewStates.assign(viewCount, XrView{ XR_TYPE_VIEW });
01008     currentImageIndex.assign(viewCount, 0);
01009     imageAcquired.assign(viewCount, false);
01010 }
01011 // 環境の合成方法を取得する (最初のものが最も推奨される)

```

```

01018     uint32_t blendModeCount{ 0 };
01019     if (XR_SUCCEEDED(xrEnumerateEnvironmentBlendModes(instance, systemId,
01020     XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, 0, &blendModeCount, nullptr))
01021         && blendModeCount > 0)
01022     {
01023         std::vector<XrEnvironmentBlendMode> blendModes(blendModeCount);
01024         if (XR_SUCCEEDED(xrEnumerateEnvironmentBlendModes(instance, systemId,
01025     XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO, blendModeCount,
01026     &blendModeCount, blendModes.data()))
01027         {
01028             blendMode = blendModes[0];
01029         }
01030     }
01031
01032     // 利用可能なスワップチェーンのカラーフォーマットを取得する
01033     uint32_t formatCount{ 0 };
01034     xrCheck(instance, xrEnumerateSwapchainFormats(session, 0, &formatCount, nullptr),
01035     "Can't count the OpenXR swapchain formats");
01036     std::vector<int64_t> formats(formatCount);
01037     xrCheck(instance, xrEnumerateSwapchainFormats(session, formatCount, &formatCount, formats.data()),
01038     "Can't enumerate the OpenXR swapchain formats");
01039     if (formats.empty()) throw std::runtime_error("The OpenXR runtime has no swapchain format");
01040
01041     // 使用したいカラーフォーマットの候補 (前にあるものを優先する)
01042     static const int64_t preferred[]{ GL_SRGB8_ALPHA8, GL_SRGB8, GL_RGBA8, GL_RGB10_A2 };
01043
01044     // 利用可能なカラーフォーマットの中から使用するものを選ぶ
01045     int64_t format{ formats[0] };
01046     for (const auto candidate : preferred)
01047     {
01048         if (std::find(formats.begin(), formats.end(), candidate) != formats.end())
01049         {
01050             format = candidate;
01051             break;
01052         }
01053     }
01054
01055     // sRGB のフォーマットならリニア色空間で描画してガンマ補正をランタイムに任せる
01056     swapchainIsSrgb = format == GL_SRGB8_ALPHA8 || format == GL_SRGB8;
01057
01058     // ビューの数だけ FBO とデプスバッファを作成する
01059     openxrFbo.assign(viewCount, 0);
01060     openxrDepth.assign(viewCount, 0);
01061     glGenFramebuffers(static_cast<GLsizei>(viewCount), openxrFbo.data());
01062     glGenRenderbuffers(static_cast<GLsizei>(viewCount), openxrDepth.data());
01063
01064     for (uint32_t i = 0; i < viewCount; ++i)
01065     {
01066         // スワップチェーンを作成する
01067         XrSwapchainCreateInfo swapchainCreateInfo{ XR_TYPE_SWAPCHAIN_CREATE_INFO };
01068         swapchainCreateInfo.usageFlags = XR_SWAPCHAIN_USAGE_COLOR_ATTACHMENT_BIT
01069         | XR_SWAPCHAIN_USAGE_SAMPLED_BIT;
01070         swapchainCreateInfo.format = format;
01071         swapchainCreateInfo.sampleCount = 1;
01072         swapchainCreateInfo.width = views[i].recommendedImageRectWidth;
01073         swapchainCreateInfo.height = views[i].recommendedImageRectHeight;
01074         swapchainCreateInfo.faceCount = 1;
01075         swapchainCreateInfo.arraySize = 1;
01076         swapchainCreateInfo.mipCount = 1;
01077
01078         XrSwapchain swapchain{ XR_NULL_HANDLE };
01079         xrCheck(instance, xrCreateSwapchain(session, &swapchainCreateInfo, &swapchain),
01080         "Can't create the OpenXR swapchain");
01081         swapchains.push_back(swapchain);
01082
01083         // スワップチェーンのイメージを取得する
01084         uint32_t imageCount{ 0 };
01085         xrCheck(instance, xrEnumerateSwapchainImages(swapchain, 0, &imageCount, nullptr),
01086         "Can't count the OpenXR swapchain images");
01087         std::vector<XrSwapchainImageOpenGLKHR> images(imageCount,
01088         XrSwapchainImageOpenGLKHR{ XR_TYPE_SWAPCHAIN_IMAGE_OPENGL_KHR });
01089         xrCheck(instance, xrEnumerateSwapchainImages(swapchain, imageCount, &imageCount,
01090         reinterpret_cast<XrSwapchainImageBaseHeader*>(images.data())),
01091         "Can't enumerate the OpenXR swapchain images");
01092         swapchainImages.push_back(std::move(images));
01093
01094         // 隠面消去処理に使うデプスバッファを作成する
01095         glBindRenderbuffer(GL_RENDERBUFFER, openxrDepth[i]);
01096         glRenderbufferStorage(GL_RENDERBUFFER, GL_DEPTH24_STENCIL8,
01097         static_cast<GLsizei>(swapchainCreateInfo.width),
01098         static_cast<GLsizei>(swapchainCreateInfo.height));
01099         glBindRenderbuffer(GL_RENDERBUFFER, 0);
01100
01101         // FBO にデプスバッファを取り付けておく (カラーバッファは select() で取り付ける)
01102         glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[i]);
01103         glFramebufferRenderbuffer(GL_FRAMEBUFFER, GL_DEPTH_STENCIL_ATTACHMENT,
01104         GL_RENDERBUFFER, openxrDepth[i]);

```

```

01105     }
01106
01107     glBindFramebuffer(GL_FRAMEBUFFER, 0);
01108 }
01109
01110 //
01111 // OpenXR のセッションを作成する
01112 //
01113 GgApp::OpenXR& GgApp::OpenXR::initialize(const Window& window,
01114     XrReferenceSpaceType spaceType, const char* appName)
01115 {
01116     static OpenXR openxr;
01117
01118     // 初期化済みならそのまま返す
01119     if (openxr.initialized) return openxr;
01120
01121     // 初期化に失敗していた場合に備えて後始末をしておく
01122     openxr.terminate();
01123
01124     openxr.window = &window;
01125     openxr.referenceSpaceType = spaceType;
01126
01127     try
01128     {
01129         // 利用可能な拡張機能を調べる
01130         uint32_t extensionCount{ 0 };
01131         xrCheck(XR_NULL_HANDLE,
01132             xrEnumerateInstanceExtensionProperties(nullptr, 0, &extensionCount, nullptr),
01133             "Can't count the OpenXR instance extensions");
01134         std::vector<XrExtensionProperties> extensionProperties(extensionCount,
01135             XrExtensionProperties{ XR_TYPE_EXTENSION_PROPERTIES });
01136         xrCheck(XR_NULL_HANDLE,
01137             xrEnumerateInstanceExtensionProperties(nullptr, extensionCount,
01138                 &extensionCount, extensionProperties.data()),
01139             "Can't enumerate the OpenXR instance extensions");
01140
01141         // OpenGL との連携に必要な拡張機能が使えなければあきらめる
01142         const auto found{ std::any_of(extensionProperties.begin(), extensionProperties.end(),
01143             [](const XrExtensionProperties& p)
01144             {
01145                 return std::strcmp(p.extensionName, XR_KHR_OPENGL_ENABLE_EXTENSION_NAME) == 0;
01146             }) };
01147         if (!found)
01148         {
01149             throw std::runtime_error(
01150                 "The OpenXR runtime does not support " XR_KHR_OPENGL_ENABLE_EXTENSION_NAME);
01151         }
01152
01153         // XrInstance の作成
01154         XrInstanceCreateInfo createInfo{ XR_TYPE_INSTANCE_CREATE_INFO };
01155         xrCopyString(createInfo.applicationInfo.applicationName,
01156             sizeof createInfo.applicationInfo.applicationName, appName ? appName : "GgApp");
01157         createInfo.applicationInfo.applicationVersion = 1;
01158         xrCopyString(createInfo.applicationInfo.engineName,
01159             sizeof createInfo.applicationInfo.engineName, "GgApp");
01160         createInfo.applicationInfo.engineVersion = 1;
01161         createInfo.applicationInfo.apiVersion = XR_CURRENT_API_VERSION;
01162
01163         const char* const extensions[]{ XR_KHR_OPENGL_ENABLE_EXTENSION_NAME };
01164         createInfo.enabledExtensionCount = 1;
01165         createInfo.enabledExtensionNames = extensions;
01166
01167         xrCheck(XR_NULL_HANDLE, xrCreateInstance(&createInfo, &openxr.instance),
01168             "Can't create the OpenXR instance (is an OpenXR runtime installed and active?)");
01169
01170         // システムの取得
01171         XrSystemGetInfo systemInfo{ XR_TYPE_SYSTEM_GET_INFO };
01172         systemInfo.formFactor = XR_FORM_FACTOR_HEAD_MOUNTED_DISPLAY;
01173         xrCheck(openxr.instance, xrGetSystem(openxr.instance, &systemInfo, &openxr.systemId),
01174             "Can't get the OpenXR system (is the head mounted display connected?)");
01175
01176         // システムの名前の取得
01177         XrSystemProperties systemProperties{ XR_TYPE_SYSTEM_PROPERTIES };
01178         if (XR_SUCCEEDED(xrGetSystemProperties(openxr.instance, openxr.systemId, &systemProperties)))
01179         {
01180             openxr.systemName = systemProperties.systemName;
01181         }
01182
01183         // OpenGL との連携に必要な拡張機能の関数の取得
01184         PFN_xrGetOpenGLGraphicsRequirementsKHR pfnGetOpenGLGraphicsRequirementsKHR{ nullptr };
01185         xrCheck(openxr.instance, xrGetInstanceProcAddr(openxr.instance,
01186             "xrGetOpenGLGraphicsRequirementsKHR",
01187             reinterpret_cast<PFN_xrVoidFunction*>(&pfnGetOpenGLGraphicsRequirementsKHR)),
01188             "Can't get the address of xrGetOpenGLGraphicsRequirementsKHR");
01189         if (!pfnGetOpenGLGraphicsRequirementsKHR)
01190             throw std::runtime_error("Can't get the address of xrGetOpenGLGraphicsRequirementsKHR");
01191
01192     }

```

```

01192 // OpenGL の要件の取得 (セッションの作成前に必ず呼ばなければならない)
01193 XrGraphicsRequirementsOpenGDKHR graphicsRequirements{ XR_TYPE_GRAPHICS_REQUIREMENTS_OPENGL_KHR };
01194 xrCheck(openxr.instance, pfnGetOpenGDKGraphicsRequirementsKHR(openxr.instance,
01195     openxr.systemId, &graphicsRequirements),
01196     "Can't get the OpenGL graphics requirements");
01197
01198 // OpenGL のバージョンが要件を満たしているかどうか調べる
01199 GLint major{ 0 }, minor{ 0 };
01200 glGetIntegerv(GL_MAJOR_VERSION, &major);
01201 glGetIntegerv(GL_MINOR_VERSION, &minor);
01202 if (XR_MAKE_VERSION(major, minor, 0) < graphicsRequirements.minApiVersionSupported)
01203 {
01204     char message[128];
01205     std::snprintf(message, sizeof message,
01206         "The OpenXR runtime requires OpenGL %d.%d or later, but %d.%d is current",
01207         static_cast<int>(XR_VERSION_MAJOR(graphicsRequirements.minApiVersionSupported)),
01208         static_cast<int>(XR_VERSION_MINOR(graphicsRequirements.minApiVersionSupported)),
01209         major, minor);
01210     throw std::runtime_error(message);
01211 }
01212
01213 // OpenGL のコンテキストをセッションに結びつける
01214 #if defined(XR_USE_PLATFORM_WIN32)
01215 XrGraphicsBindingOpenGDKWin32KHR graphicsBinding{ XR_TYPE_GRAPHICS_BINDING_OPENGL_WIN32_KHR };
01216 graphicsBinding.hDC = wglGetCurrentDC();
01217 graphicsBinding.hGLRC = glfwGetWGLContext(window.get());
01218 #elif defined(XR_USE_PLATFORM_XLIB)
01219 XrGraphicsBindingOpenGDKXlibKHR graphicsBinding{ XR_TYPE_GRAPHICS_BINDING_OPENGL_XLIB_KHR };
01220 graphicsBinding.xDisplay = glfwGetXlibDisplay();
01221 graphicsBinding.visualid = 0;
01222 graphicsBinding.glxFBConfig = nullptr;
01223 graphicsBinding.glxDrawable = glfwGetGLXWindow(window.get());
01224 graphicsBinding.glxContext = glfwGetGLXContext(window.get());
01225 #else
01226 #error "GG-USE-OPENXR is not supported on this platform"
01227 #endif
01228
01229 // セッションの作成
01230 XrSessionCreateInfo sessionCreateInfo{ XR_TYPE_SESSION_CREATE_INFO };
01231 sessionCreateInfo.next = &graphicsBinding;
01232 sessionCreateInfo.systemId = openxr.systemId;
01233 xrCheck(openxr.instance, xrCreateSession(openxr.instance, &sessionCreateInfo, &openxr.session),
01234     "Can't create the OpenXR session");
01235
01236 // 参照空間の作成 (要求されたものが使えなければ LOCAL にフォールバックする)
01237 XrReferenceSpaceCreateInfo spaceCreateInfo{ XR_TYPE_REFERENCE_SPACE_CREATE_INFO };
01238 spaceCreateInfo.referenceSpaceType = openxr.referenceSpaceType;
01239 spaceCreateInfo.poseInReferenceSpace.orientation.w = 1.0f;
01240 if (XR_FAILED(xrCreateReferenceSpace(openxr.session, &spaceCreateInfo, &openxr.appSpace)))
01241 {
01242     openxr.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
01243     spaceCreateInfo.referenceSpaceType = XR_REFERENCE_SPACE_TYPE_LOCAL;
01244     xrCheck(openxr.instance,
01245         xrCreateReferenceSpace(openxr.session, &spaceCreateInfo, &openxr.appSpace),
01246         "Can't create the OpenXR reference space");
01247 }
01248
01249 // アクションシステムの初期化
01250 openxr.initActions();
01251
01252 // スワップチェーンの作成
01253 openxr.createSwapchains();
01254 }
01255 catch (...)
01256 {
01257     // 途中で確保した資源を解放してから例外を投げ直す
01258     openxr.terminate();
01259     throw;
01260 }
01261
01262 // OpenXR は xrWaitFrame() でフレームの表示速度を制御するので
01263 // ウィンドウ側の垂直同期の待ち合わせは行わない
01264 glfwSwapInterval(0);
01265
01266 openxr.initialized = true;
01267
01268 return openxr;
01269 }
01270
01271 //
01272 // OpenXR のハンドルを破棄する (OpenGL の資源には触れない)
01273 //
01274 void GgApp::OpenXR::destroyXr()
01275 {
01276     for (int i = 0; i < Hand::Count; ++i)
01277     {
01278         if (aimSpace[i] != XR_NULL_HANDLE) { xrDestroySpace(aimSpace[i]); aimSpace[i] = XR_NULL_HANDLE; }

```

```

01279     if (gripSpace[i] != XR_NULL_HANDLE) { xrDestroySpace(gripSpace[i]); gripSpace[i] = XR_NULL_HANDLE; }
01280 }
01281
01282 // アクションはアクションセットと一緒に破棄される
01283 if (actionSet != XR_NULL_HANDLE) xrDestroyActionSet(actionSet);
01284 actionSet = XR_NULL_HANDLE;
01285 aimPoseAction = gripPoseAction = XR_NULL_HANDLE;
01286 triggerAction = gripAction = XR_NULL_HANDLE;
01287 thumbstickAction = thumbstickClickAction = XR_NULL_HANDLE;
01288 primaryButtonAction = secondaryButtonAction = menuButtonAction = XR_NULL_HANDLE;
01289 hapticAction = XR_NULL_HANDLE;
01290
01291 for (auto swapchain : swapchains) xrDestroySwapchain(swapchain);
01292 swapchains.clear();
01293 swapchainImages.clear();
01294
01295 if (appSpace != XR_NULL_HANDLE) { xrDestroySpace(appSpace); appSpace = XR_NULL_HANDLE; }
01296 if (session != XR_NULL_HANDLE) { xrDestroySession(session); session = XR_NULL_HANDLE; }
01297 if (instance != XR_NULL_HANDLE) { xrDestroyInstance(instance); instance = XR_NULL_HANDLE; }
01298
01299 systemId = XR_NULL_SYSTEM_ID;
01300 sessionState = XR_SESSION_STATE_UNKNOWN;
01301 isSessionRunning = false;
01302 frameBegun = false;
01303 viewPoseValid = false;
01304 initialized = false;
01305 }
01306
01307 //
01308 // OpenXR のセッションを破棄する
01309 //
01310 void GgApp::OpenXR::terminate()
01311 {
01312     // 取得中のスワップチェーンイメージがあれば解放する (xrEndFrame() より前に行う)
01313     for (size_t i = 0; i < imageAcquired.size(); ++i)
01314     {
01315         if (!imageAcquired[i]) continue;
01316         XrSwapchainImageReleaseInfo releaseInfo{ XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO };
01317         xrReleaseSwapchainImage(swapchains[i], &releaseInfo);
01318         imageAcquired[i] = false;
01319     }
01320
01321     // 描画中のフレームがあれば完了しておく
01322     if (frameBegun) endFrame();
01323
01324     // OpenGL の資源を解放する
01325     if (!openxrFbo.empty())
01326     {
01327         glDeleteFramebuffers(static_cast<GLsizei>(openxrFbo.size()), openxrFbo.data());
01328         openxrFbo.clear();
01329     }
01330     if (!openxrDepth.empty())
01331     {
01332         glDeleteRenderbuffers(static_cast<GLsizei>(openxrDepth.size()), openxrDepth.data());
01333         openxrDepth.clear();
01334     }
01335
01336     // OpenXR のハンドルを破棄する
01337     destroyXr();
01338
01339     views.clear();
01340     viewStates.clear();
01341     currentImageIndex.clear();
01342     imageAcquired.clear();
01343     systemName.clear();
01344
01345     for (auto& state : controllerStates) state = ControllerState{};
01346
01347     // ウィンドウ側の設定を元に戻す
01348     if (window)
01349     {
01350         glDisable(GL_FRAMEBUFFER_SRGB);
01351         glfwSwapInterval(1);
01352         window = nullptr;
01353     }
01354 }
01355
01356 //
01357 // OpenXR による描画開始
01358 //
01359 bool GgApp::OpenXR::begin()
01360 {
01361     // 初期化されていなければ何もしない
01362     if (instance == XR_NULL_HANDLE) return false;
01363
01364     // OpenXR のイベントを処理する
01365     pollEvents();

```



```

01366
01367 // セッションが実行中でなければ描画しない
01368 if (!isSessionRunning) return false;
01369
01370 // 合成器がこのフレームの描画を始めるべき時刻まで待つ
01371 XrFrameWaitInfo waitInfo{ XR_TYPE_FRAME_WAIT_INFO };
01372 frameState = XrFrameState{ XR_TYPE_FRAME_STATE };
01373 if (!xrWarn(instance, xrWaitFrame(session, &waitInfo, &frameState),
01374     "Can't wait for the OpenXR frame")) return false;
01375
01376 // フレームの描画を開始する
01377 XrFrameBeginInfo beginInfo{ XR_TYPE_FRAME_BEGIN_INFO };
01378 if (!xrWarn(instance, xrBeginFrame(session, &beginInfo),
01379     "Can't begin the OpenXR frame")) return false;
01380
01381 // ここから先は必ず xrEndFrame() を呼ばなければならない
01382 frameBegun = true;
01383 viewPoseValid = false;
01384
01385 // コントローラーの状態を更新する
01386 pollActions();
01387
01388 // 描画すべきフレームなら視点の姿勢を取得する
01389 if (frameState.shouldRender != XR_FALSE)
01390 {
01391     XrViewLocateInfo viewLocateInfo{ XR_TYPE_VIEW_LOCATE_INFO };
01392     viewLocateInfo.viewConfigurationType = XR_VIEW_CONFIGURATION_TYPE_PRIMARY_STEREO;
01393     viewLocateInfo.displayTime = frameState.predictedDisplayTime;
01394     viewLocateInfo.space = appSpace;
01395
01396     XrViewState viewState{ XR_TYPE_VIEW_STATE };
01397     uint32_t viewCount{ 0 };
01398     if (XR_SUCCEEDED(xrLocateViews(session, &viewLocateInfo, &viewState,
01399         static_cast<uint32_t>(viewStates.size()), &viewCount, viewStates.data())))
01400     {
01401         // 位置と向きの両方が有効なときだけ描画する
01402         constexpr XrViewStateFlags valid
01403         {
01404             XR_VIEW_STATE_POSITION_VALID_BIT | XR_VIEW_STATE_ORIENTATION_VALID_BIT
01405         };
01406         viewPoseValid = (viewState.viewStateFlags & valid) == valid
01407             && viewCount == static_cast<uint32_t>(viewStates.size());
01408     }
01409 }
01410
01411 // 描画するなら true を返す
01412 if (viewPoseValid) return true;
01413
01414 // 描画しないフレームでもここで xrEndFrame() を呼んで辻褄を合わせる
01415 endFrame();
01416
01417 return false;
01418 }
01419
01420 //
01421 // 描画対象の目を指定してフレームバッファとビューポートを設定する
01422 //
01423 void GgApp::OpenXR::select(int eye)
01424 {
01425     // 描画すべきフレームでなければ何もしない
01426     if (!frameBegun || !viewPoseValid) return;
01427
01428     // 視点の番号が範囲を外れていたら何もしない
01429     assert(eye >= 0 && eye < static_cast<int>(swapchains.size()));
01430     if (eye < 0 || eye >= static_cast<int>(swapchains.size())) return;
01431
01432     // 取得済みなら描画先を結合し直すだけにする
01433     if (imageAcquired[eye])
01434     {
01435         glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[eye]);
01436         glViewport(0, 0,
01437             static_cast<GLsizei>(views[eye].recommendedImageRectWidth),
01438             static_cast<GLsizei>(views[eye].recommendedImageRectHeight));
01439         if (swapchainIsSrgb) glEnable(GL_FRAMEBUFFER_SRGB);
01440         return;
01441     }
01442
01443     // 描画可能なスワップチェーンイメージを取得する
01444     XrSwapchainImageAcquireInfo acquireInfo{ XR_TYPE_SWAPCHAIN_IMAGE_ACQUIRE_INFO };
01445     if (!xrWarn(instance, xrAcquireSwapchainImage(swapchains[eye], &acquireInfo,
01446         &currentImageIndex[eye]), "Can't acquire the OpenXR swapchain image")) return;
01447
01448     // そのスワップチェーンイメージが描画可能になるのを待つ
01449     XrSwapchainImageWaitInfo waitInfo{ XR_TYPE_SWAPCHAIN_IMAGE_WAIT_INFO };
01450     waitInfo.timeout = XR_INFINITE_DURATION;
01451     if (!xrWarn(instance, xrWaitSwapchainImage(swapchains[eye], &waitInfo),
01452         "Can't wait for the OpenXR swapchain image"))

```



```

01453 {
01454     XrSwapchainImageReleaseInfo releaseInfo{ XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO };
01455     xrReleaseSwapchainImage(swapchains[eye], &releaseInfo);
01456     return;
01457 }
01458
01459 imageAcquired[eye] = true;
01460
01461 // 描画先をこのスワップチェーンイメージに切り替える
01462 const GLuint texture{ swapchainImages[eye][currentImageIndex[eye]].image };
01463 glBindFramebuffer(GL_FRAMEBUFFER, openxrFbo[eye]);
01464 glFramebufferTexture2D(GL_FRAMEBUFFER, GL_COLOR_ATTACHMENT0, GL_TEXTURE_2D, texture, 0);
01465
01466 // フレームバッファオブジェクトが完成しているか確かめる (最初の一度だけ報告する)
01467 static bool reported{ false };
01468 if (!reported && glCheckFramebufferStatus(GL_FRAMEBUFFER) != GL_FRAMEBUFFER_COMPLETE)
01469 {
01470     reported = true;
01471     std::cerr << "OpenXR: The framebuffer object for the swapchain image is not complete\n";
01472 }
01473
01474 glViewport(0, 0,
01475     static_cast<GLsizei>(views[eye].recommendedImageRectWidth),
01476     static_cast<GLsizei>(views[eye].recommendedImageRectHeight));
01477
01478 // sRGB のスワップチェーンならリニア色空間で描画する
01479 if (swapchainIsSrgb) glEnable(GL_FRAMEBUFFER_SRGB);
01480 }
01481
01482 //
01483 // 描画対象の目を指定する (旧 LibOVR 仕様互換)
01484 //
01485 void GgApp::OpenXR::select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation)
01486 {
01487     select(eye);
01488
01489     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01490     const auto& pose = viewStates[eye].pose;
01491     const auto& fov = viewStates[eye].fov;
01492
01493     screen[0] = tanf(fov.angleLeft);
01494     screen[1] = tanf(fov.angleRight);
01495     screen[2] = tanf(fov.angleDown);
01496     screen[3] = tanf(fov.angleUp);
01497
01498     position[0] = pose.position.x;
01499     position[1] = pose.position.y;
01500     position[2] = pose.position.z;
01501
01502     orientation[0] = pose.orientation.x;
01503     orientation[1] = pose.orientation.y;
01504     orientation[2] = pose.orientation.z;
01505     orientation[3] = pose.orientation.w;
01506 }
01507
01508 //
01509 // 指定した目の描画を完了する
01510 //
01511 void GgApp::OpenXR::commit(int eye)
01512 {
01513     if (!frameBegun) return;
01514     assert(eye >= 0 && eye < static_cast<int>(swapchains.size()));
01515     if (eye < 0 || eye >= static_cast<int>(swapchains.size())) return;
01516     if (!imageAcquired[eye]) return;
01517
01518     // ガンマ補正を元に戻して描画先をウィンドウに戻す
01519     if (swapchainIsSrgb) glDisable(GL_FRAMEBUFFER_SRGB);
01520     glBindFramebuffer(GL_FRAMEBUFFER, 0);
01521
01522     // スワップチェーンイメージはミラー表示に使うので、ここでは解放しない
01523     // (解放は submit() の中でミラー表示を行った後に実施する)
01524 }
01525
01526 //
01527 // ミラー表示を行う
01528 //
01529 void GgApp::OpenXR::blitMirror() const
01530 {
01531     // ミラー表示を行わないなら何もしない
01532     if (mirrorView < 0 || !window) return;
01533     const auto eye{ static_cast<size_t>(mirrorView) };
01534     if (eye >= swapchains.size() || !imageAcquired[eye]) return;
01535
01536     // 転送元の大きさ
01537     const auto srcWidth{ static_cast<GLint>(views[eye].recommendedImageRectWidth) };
01538     const auto srcHeight{ static_cast<GLint>(views[eye].recommendedImageRectHeight) };
01539     if (srcWidth <= 0 || srcHeight <= 0) return;

```

```

01540
01541 // 転送先 (ウィンドウ) の大きさ
01542 const auto& fboSize{ window->getFboSize() };
01543 if (fboSize[0] <= 0 || fboSize[1] <= 0) return;
01544
01545 // 縦横比を保ったままウィンドウに収まる転送先の矩形を求める
01546 const auto scale{ std::min(
01547     static_cast<float>(fboSize[0]) / static_cast<float>(srcWidth),
01548     static_cast<float>(fboSize[1]) / static_cast<float>(srcHeight)) };
01549 const auto dstWidth{ static_cast<GLint>(static_cast<float>(srcWidth) * scale) };
01550 const auto dstHeight{ static_cast<GLint>(static_cast<float>(srcHeight) * scale) };
01551 const auto dstLeft{ (static_cast<GLint>(fboSize[0]) - dstWidth) / 2 };
01552 const auto dstBottom{ (static_cast<GLint>(fboSize[1]) - dstHeight) / 2 };
01553
01554 // sRGB の再変換を避けるためにガンマ補正を無効にする
01555 if (swapchainIsSrgb) glDisable(GL_FRAMEBUFFER_SRGB);
01556
01557 // 上下左右の余白を黒で塗りつぶす (消去色は元に戻す)
01558 GLfloat clearColor[4];
01559 glGetFloatv(GL_COLOR_CLEAR_VALUE, clearColor);
01560 glBindFramebuffer(GL_DRAW_FRAMEBUFFER, 0);
01561 glDisable(GL_SCISSOR_TEST);
01562 glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
01563 glClear(GL_COLOR_BUFFER_BIT);
01564 glClearColor(clearColor[0], clearColor[1], clearColor[2], clearColor[3]);
01565
01566 // スワップチェーンイメージをウィンドウに転送する
01567 glBindFramebuffer(GL_READ_FRAMEBUFFER, openxrFbo[eye]);
01568 glBlitFramebuffer(0, 0, srcWidth, srcHeight,
01569     dstLeft, dstBottom, dstLeft + dstWidth, dstBottom + dstHeight,
01570     GL_COLOR_BUFFER_BIT, GL_LINEAR);
01571 glBindFramebuffer(GL_READ_FRAMEBUFFER, 0);
01572 }
01573
01574 //
01575 // 描画中のフレームを合成器に転送する
01576 //
01577 void GgApp::OpenXR::endFrame()
01578 {
01579     if (!frameBegun) return;
01580
01581     // 合成する層
01582     std::vector<XrCompositionLayerProjectionView> projectionViews;
01583     XrCompositionLayerProjection layer{ XR_TYPE_COMPOSITION_LAYER_PROJECTION };
01584     const XrCompositionLayerBaseHeader* layers[1]{ nullptr };
01585
01586     // 描画したのなら層を用意する
01587     if (viewPoseValid && frameState.shouldRender != XR_FALSE)
01588     {
01589         projectionViews.resize(swapchains.size());
01590         for (size_t i = 0; i < swapchains.size(); ++i)
01591         {
01592             auto& projectionView{ projectionViews[i] };
01593             projectionView = XrCompositionLayerProjectionView{ XR_TYPE_COMPOSITION_LAYER_PROJECTION_VIEW };
01594             projectionView.pose = viewStates[i].pose;
01595             projectionView.fov = viewStates[i].fov;
01596             projectionView.subImage.swapchain = swapchains[i];
01597             projectionView.subImage.imageRect.offset = { 0, 0 };
01598             projectionView.subImage.imageRect.extent = {
01599                 static_cast<int32_t>(views[i].recommendedImageRectWidth),
01600                 static_cast<int32_t>(views[i].recommendedImageRectHeight)
01601             };
01602             projectionView.subImage.imageArrayIndex = 0;
01603         }
01604
01605         // 環境の合成方法に応じて層の属性を設定する
01606         layer.layerFlags = blendMode == XR_ENVIRONMENT_BLEND_MODE_OPAQUE ? 0
01607             : XR_COMPOSITION_LAYER_BLEND_TEXTURE_SOURCE_ALPHA_BIT
01608             | XR_COMPOSITION_LAYER_UNPREMULTIPLIED_ALPHA_BIT;
01609         layer.space = appSpace;
01610         layer.viewCount = static_cast<uint32_t>(projectionViews.size());
01611         layer.views = projectionViews.data();
01612         layers[0] = reinterpret_cast<const XrCompositionLayerBaseHeader*>(&layer);
01613     }
01614
01615     // フレームを合成器に転送する
01616     XrFrameEndInfo endInfo{ XR_TYPE_FRAME_END_INFO };
01617     endInfo.displayTime = frameState.predictedDisplayTime;
01618     endInfo.environmentBlendMode = blendMode;
01619     endInfo.layerCount = layers[0] ? 1u : 0u;
01620     endInfo.layers = layers;
01621     xrWarn(instance, xrEndFrame(session, &endInfo), "Can't end the OpenXR frame");
01622
01623     frameBegun = false;
01624 }
01625
01626 //

```

```

01627 // フレームを転送して HMD に表示する
01628 //
01629 bool GgApp::OpenXR::submit(bool mirror)
01630 {
01631     // 描画中のフレームがなければ何もしない
01632     if (!frameBegun) return false;
01633
01634     // ミラー表示の有無を設定する
01635     if (!mirror) mirrorView = -1;
01636     else if (mirrorView < 0) mirrorView = 0;
01637
01638     // スワップチェーンイメージを解放する前にミラー表示を行う
01639     blitMirror();
01640
01641     // ウィンドウのビューポートを復帰して Dear ImGui などの描画に備える
01642     if (window) window->restoreViewport();
01643
01644     // 取得したスワップチェーンイメージを解放する
01645     for (size_t i = 0; i < imageAcquired.size(); ++i)
01646     {
01647         if (!imageAcquired[i]) continue;
01648         XrSwapchainImageReleaseInfo releaseInfo{ XR_TYPE_SWAPCHAIN_IMAGE_RELEASE_INFO };
01649         xrWarn(instance, xrReleaseSwapchainImage(swapchains[i], &releaseInfo),
01650             "Can't release the OpenXR swapchain image");
01651         imageAcquired[i] = false;
01652     }
01653
01654     // 合成器にフレームを転送する
01655     endFrame();
01656
01657     return true;
01658 }
01659
01660 //
01661 // ミラー表示を行うビューの番号を設定する
01662 //
01663 void GgApp::OpenXR::setMirror(int eye)
01664 {
01665     mirrorView = eye;
01666 }
01667
01668 //
01669 // ミラー表示を行うビューの番号を取得する
01670 //
01671 int GgApp::OpenXR::getMirror() const
01672 {
01673     return mirrorView;
01674 }
01675
01676 //
01677 // セッションが実行中かどうか調べる
01678 //
01679 bool GgApp::OpenXR::isRunning() const
01680 {
01681     return isSessionRunning;
01682 }
01683
01684 //
01685 // アプリケーションが入力を受け付けているかどうか調べる
01686 //
01687 bool GgApp::OpenXR::isFocused() const
01688 {
01689     return sessionState == XR_SESSION_STATE_FOCUSED;
01690 }
01691
01692 //
01693 // OpenXR のシステム (HMD) の名前を取得する
01694 //
01695 const std::string& GgApp::OpenXR::getSystemName() const
01696 {
01697     return systemName;
01698 }
01699
01700 //
01701 // 指定した目の透視投影変換行列を取得する
01702 //
01703 GgMatrix GgApp::OpenXR::getProjectionMatrix(int eye, GLfloat zNear, GLfloat zFar) const
01704 {
01705     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01706     const auto& fov{ viewStates[eye].fov };
01707     const GLfloat l{ tanf(fov.angleLeft) * zNear };
01708     const GLfloat r{ tanf(fov.angleRight) * zNear };
01709     const GLfloat b{ tanf(fov.angleDown) * zNear };
01710     const GLfloat t{ tanf(fov.angleUp) * zNear };
01711     return ggFrustum(l, r, b, t, zNear, zFar);
01712 }
01713

```

```

01714 //
01715 // 指定した目のビュー変換行列を取得する
01716 //
01717 GgMatrix GgApp::OpenXR::getViewMatrix(int eye) const
01718 {
01719     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01720     const auto& pose{ viewStates[eye].pose };
01721     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01722     return q.getConjugateMatrix() * ggTranslate(-pose.position.x, -pose.position.y, -pose.position.z);
01723 }
01724 //
01725 //
01726 // 指定した目の姿勢行列を取得する
01727 //
01728 GgMatrix GgApp::OpenXR::getPoseMatrix(int eye) const
01729 {
01730     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01731     const auto& pose{ viewStates[eye].pose };
01732     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01733     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
01734 }
01735 //
01736 //
01737 // 指定した目の視点位置を取得する
01738 //
01739 GgVector GgApp::OpenXR::getPosition(int eye) const
01740 {
01741     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01742     const auto& pos{ viewStates[eye].pose.position };
01743     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
01744 }
01745 //
01746 //
01747 // 指定した目の視線方向の回転四元数を取得する
01748 //
01749 GgQuaternion GgApp::OpenXR::getOrientation(int eye) const
01750 {
01751     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01752     const auto& ori{ viewStates[eye].pose.orientation };
01753     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
01754 }
01755 //
01756 //
01757 // 指定した目の視野角情報 (XrFovf) を取得する
01758 //
01759 const XrFovf& GgApp::OpenXR::getFov(int eye) const
01760 {
01761     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01762     return viewStates[eye].fov;
01763 }
01764 //
01765 //
01766 // 指定した目の姿勢情報 (XrPosef) を取得する
01767 //
01768 const XrPosef& GgApp::OpenXR::getPose(int eye) const
01769 {
01770     assert(eye >= 0 && eye < static_cast<int>(viewStates.size()));
01771     return viewStates[eye].pose;
01772 }
01773 //
01774 //
01775 // 視点の姿勢が有効かどうか調べる
01776 //
01777 bool GgApp::OpenXR::isPoseValid() const
01778 {
01779     return viewPoseValid;
01780 }
01781 //
01782 //
01783 // レンダリング推奨解像度の横幅を取得する
01784 //
01785 GLsizei GgApp::OpenXR::getWidth(int eye) const
01786 {
01787     assert(eye >= 0 && eye < static_cast<int>(views.size()));
01788     return static_cast<GLsizei>(views[eye].recommendedImageRectWidth);
01789 }
01790 //
01791 //
01792 // レンダリング推奨解像度の高さを取得する
01793 //
01794 GLsizei GgApp::OpenXR::getHeight(int eye) const
01795 {
01796     assert(eye >= 0 && eye < static_cast<int>(views.size()));
01797     return static_cast<GLsizei>(views[eye].recommendedImageRectHeight);
01798 }

```

```

01799
01800 //
01801 // アスペクト比を取得する
01802 //
01803 GLfloat GgApp::OpenXR::getAspect(int eye) const
01804 {
01805     assert(eye >= 0 && eye < static_cast<int>(views.size()));
01806     return static_cast<GLfloat>(views[eye].recommendedImageRectWidth)
01807         / static_cast<GLfloat>(views[eye].recommendedImageRectHeight);
01808 }
01809
01810 //
01811 // ビューの総数を取得する
01812 //
01813 uint32_t GgApp::OpenXR::getViewCount() const
01814 {
01815     return static_cast<uint32_t>(views.size());
01816 }
01817
01818 //
01819 // 現在の参照空間タイプを取得する
01820 //
01821 XrReferenceSpaceType GgApp::OpenXR::getReferenceSpaceType() const
01822 {
01823     return referenceSpaceType;
01824 }
01825
01826 //
01827 // コントローラーがトラッキングされているか取得する
01828 //
01829 bool GgApp::OpenXR::isTracked(int hand) const
01830 {
01831     assert(hand >= 0 && hand < Hand::Count);
01832     return controllerStates[hand].isTracked;
01833 }
01834
01835 //
01836 // コントローラーのグリップ変換行列を取得する
01837 //
01838 GgMatrix GgApp::OpenXR::getGripMatrix(int hand) const
01839 {
01840     assert(hand >= 0 && hand < Hand::Count);
01841     const auto& pose = controllerStates[hand].gripPose;
01842     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01843     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
01844 }
01845
01846 //
01847 // コントローラーのエイム変換行列を取得する
01848 //
01849 GgMatrix GgApp::OpenXR::getAimMatrix(int hand) const
01850 {
01851     assert(hand >= 0 && hand < Hand::Count);
01852     const auto& pose = controllerStates[hand].aimPose;
01853     const GgQuaternion q{ pose.orientation.x, pose.orientation.y, pose.orientation.z, pose.orientation.w };
01854     return ggTranslate(pose.position.x, pose.position.y, pose.position.z) * q.getMatrix();
01855 }
01856
01857 //
01858 // コントローラーのグリップ位置を取得する
01859 //
01860 GgVector GgApp::OpenXR::getGripPosition(int hand) const
01861 {
01862     assert(hand >= 0 && hand < Hand::Count);
01863     const auto& pos = controllerStates[hand].gripPose.position;
01864     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
01865 }
01866
01867 //
01868 // コントローラーのグリップ回転四元数を取得する
01869 //
01870 GgQuaternion GgApp::OpenXR::getGripOrientation(int hand) const
01871 {
01872     assert(hand >= 0 && hand < Hand::Count);
01873     const auto& ori = controllerStates[hand].gripPose.orientation;
01874     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
01875 }
01876
01877 //
01878 // コントローラーのエイム位置を取得する
01879 //
01880 GgVector GgApp::OpenXR::getAimPosition(int hand) const
01881 {
01882     assert(hand >= 0 && hand < Hand::Count);
01883     const auto& pos = controllerStates[hand].aimPose.position;

```

```

01884     return GgVector{ pos.x, pos.y, pos.z, 1.0f };
01885 }
01886
01887 //
01888 // コントローラーのエイム回転四元数を取得する
01889 //
01890 GgQuaternion GgApp::OpenXR::getAimOrientation(int hand) const
01891 {
01892     assert(hand >= 0 && hand < Hand::Count);
01893     const auto& ori = controllerStates[hand].aimPose.orientation;
01894     return GgQuaternion{ ori.x, ori.y, ori.z, ori.w };
01895 }
01896
01897 //
01898 // トリガーの押し込み量を取得する
01899 //
01900 float GgApp::OpenXR::getTrigger(int hand) const
01901 {
01902     assert(hand >= 0 && hand < Hand::Count);
01903     return controllerStates[hand].trigger;
01904 }
01905
01906 //
01907 // グリップの押し込み量を取得する
01908 //
01909 float GgApp::OpenXR::getGrip(int hand) const
01910 {
01911     assert(hand >= 0 && hand < Hand::Count);
01912     return controllerStates[hand].grip;
01913 }
01914
01915 //
01916 // アナログスティックの入力値を取得する
01917 //
01918 std::array<float, 2> GgApp::OpenXR::getThumbstick(int hand) const
01919 {
01920     assert(hand >= 0 && hand < Hand::Count);
01921     return controllerStates[hand].thumbstick;
01922 }
01923
01924 //
01925 // アナログスティックのクリック状態を取得する
01926 //
01927 bool GgApp::OpenXR::getThumbstickClick(int hand) const
01928 {
01929     assert(hand >= 0 && hand < Hand::Count);
01930     return controllerStates[hand].thumbstickClick;
01931 }
01932
01933 //
01934 // プライマリボタンの押下状態を取得する
01935 //
01936 bool GgApp::OpenXR::getPrimaryButton(int hand) const
01937 {
01938     assert(hand >= 0 && hand < Hand::Count);
01939     return controllerStates[hand].primaryButton;
01940 }
01941
01942 //
01943 // セカンダリボタンの押下状態を取得する
01944 //
01945 bool GgApp::OpenXR::getSecondaryButton(int hand) const
01946 {
01947     assert(hand >= 0 && hand < Hand::Count);
01948     return controllerStates[hand].secondaryButton;
01949 }
01950
01951 //
01952 // メニューボタンの押下状態を取得する
01953 //
01954 bool GgApp::OpenXR::getMenuButton(int hand) const
01955 {
01956     assert(hand >= 0 && hand < Hand::Count);
01957     return controllerStates[hand].menuButton;
01958 }
01959
01960 //
01961 // コントローラーに振動を出力する
01962 //
01963 void GgApp::OpenXR::applyHapticVibration(int hand, float durationSeconds, float frequency, float
    amplitude)
01964 {
01965     assert(hand >= 0 && hand < Hand::Count);
01966     if (session == XR_NULL_HANDLE || hapticAction == XR_NULL_HANDLE) return;
01967
01968     XrHapticVibration vibration{ XR_TYPE_HAPTIC_VIBRATION };
01969     vibration.duration = durationSeconds > 0.0f

```

```

01970     ? static_cast<XrDuration>(static_cast<double>(durationSeconds) * 1.0e9)
01971     : XR_MIN_HAPTIC_DURATION;
01972     vibration.frequency = frequency;
01973     vibration.amplitude = std::min(std::max(amplitude, 0.0f), 1.0f);
01974
01975     XrHapticActionInfo actionInfo{ XR_TYPE_HAPTIC_ACTION_INFO };
01976     actionInfo.action = hapticAction;
01977     actionInfo.subactionPath = handSubactionPath[hand];
01978
01979     xrWarn(instance, xrApplyHapticFeedback(session, &actionInfo,
01980         reinterpret_cast<const XrHapticBaseHeader*>(&vibration)),
01981         "Can't apply the haptic feedback");
01982 }
01983
01984 #endif
01985
01986 #if defined(_WIN32)
01987 #   if !defined(_INC_WINDOWS) && !defined(_WINDOWS_)
01988 #       include <windows.h>
01989 #   endif
01990 #else
01991 #   include <pwd.h>
01992 #   include <unistd.h>
01993 #endif
01994 //
01995 // ユーザ名を得る
01996 //
01997 std::string GgApp::getUsername()
01998 {
01999     // 環境変数からユーザ名を得る
02000     const char* user{
02001         #if defined(_WIN32)
02002             std::getenv("USERNAME")
02003         #else
02004             std::getenv("USER")
02005         #endif
02006     };
02007
02008     // 環境変数からユーザ名が得られたらそれを返す
02009     if (user) return user;
02010
02011     #if defined(_WIN32)
02012         // Win32 API を使ってユーザ名を得る
02013         char username[256];
02014         DWORD size{ sizeof(username) };
02015         if (GetUserNameA(username, &size)) return std::string(username);
02016     #else
02017         struct passwd* pw{ getpwuid(getuid()) };
02018         if (pw) return std::string(pw->pw_name);
02019     #endif
02020
02021     // ユーザ名が得られなかった
02022     return "unknown";
02023 }

```

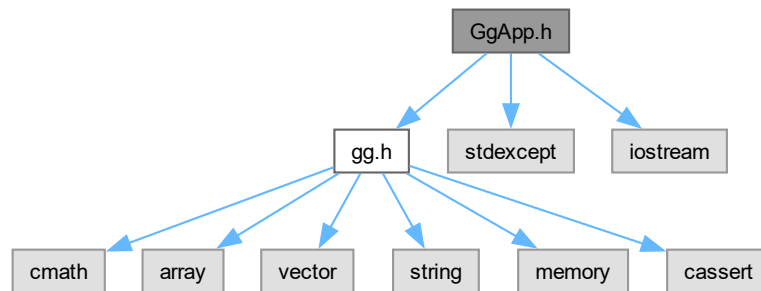
8.7 GgApp.h ファイル

```

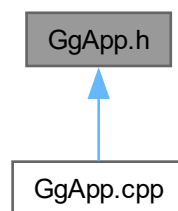
#include "gg.h"
#include <stdexcept>
#include <iostream>

```

GgApp.h の依存先関係図:



被依存関係図:



クラス

- class [GgApp](#)
- class [GgApp::Window](#)

マクロ定義

- #define [GG.BUTTON_COUNT](#) 3
- #define [GG.INTERFACE_COUNT](#) 5

8.7.1 詳解

ゲームグラフィックス特論宿題アプリケーションクラスの定義.

著者

Kohe Tokoi

日付

July 17, 2025

[GgApp.h](#) に定義があります。

8.7.2 マクロ定義詳解

8.7.2.1 GG_BUTTON_COUNT

```
#define GG_BUTTON_COUNT 3
```

GgApp.h の 49 行目に定義があります。

8.7.2.2 GG_INTERFACE_COUNT

```
#define GG_INTERFACE_COUNT 5
```

GgApp.h の 54 行目に定義があります。

8.8 GgApp.h

[詳解]

```
00001 #pragma once
00002
00003 /*
00004 ゲームグラフィックス特論用補助プログラム GLFW3 版
00005 Copyright (c) 2011-2025 Kohe Tokoi. All Rights Reserved.
00006
00007 Permission is hereby granted, free of charge, to any person obtaining a copy
00008 of this software and associated documentation files (the "Software"), to deal
00009 in the Software without restriction, including without limitation the rights
00010 to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
00011 copies or substantial portions of the Software.
00012
00013 The above copyright notice and this permission notice shall be included in
00014 all copies or substantial portions of the Software.
00015
00016 THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
00017 IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
00018 FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL
00019 KOHE TOKOI BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN
00020 AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN
00021 CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.
00022
00023 */
00024
00025 // Dear ImGui を使うなら
00026 #if !defined(GG_USE_IMGUI) && !defined(GG_NO_IMGUI)
00027 #   if defined(_has_include)
00028 #       if _has_include(<imgui.h>) || _has_include("imgui.h")
00029 #           define GG_USE_IMGUI
00030 #       endif
00031 #   endif
00032 #endif
00033
00034 // OpenXR を使うなら
00035 #define GG_USE_OPENXR
00036
00037 // 使用するマウスのボタン数
00038 #if !defined(GG_BUTTON_COUNT)
00039 #   define GG_BUTTON_COUNT 3
00040 #endif
00041
00042 // 使用するユーザインタフェースの数
00043 #if !defined(GG_INTERFACE_COUNT)
00044 #   define GG_INTERFACE_COUNT 5
00045 #endif
00046
00047 // 補助プログラム
00048 #include "gg.h"
00049 using namespace gg;
00050
```

```

00061 // 標準ライブラリ
00062 #include <stdexcept>
00063 #include <iostream>
00064
00065 // ImGui の組み込み
00066 #if defined(GG_USE_IMGUI)
00067 #   include "imgui.h"
00068 #   include "imgui_impl_glfw.h"
00069 #   include "imgui_impl_opengl3.h"
00070 #endif
00071
00072 // OpenXR ライブラリの組み込み
00073 #if defined(GG_USE_OPENXR)
00074 #   if defined(_WIN32)
00075 #       define XR_USE_PLATFORM_WIN32
00076 #       define XR_USE_GRAPHICS_API_OPENGL
00077 #       define GLFW_EXPOSE_NATIVE_WIN32
00078 #       define GLFW_EXPOSE_NATIVE_WGL
00079 #       include <GLFW/glfw3native.h>
00080 #       include <windows.h>
00081 #       include <unknwn.h>
00082 #       if defined(_MSC_VER)
00083 #           pragma comment(lib, "openxr_loader.lib")
00084 #       endif
00085 #   else
00086 #       if !defined(__gl_h_)
00087 #           define __gl_h_
00088 #       endif
00089 #       define XR_USE_PLATFORM_XLIB
00090 #       define XR_USE_GRAPHICS_API_OPENGL
00091 #       define GLFW_EXPOSE_NATIVE_X11
00092 #       define GLFW_EXPOSE_NATIVE_GLX
00093 #       include <GLFW/glfw3native.h>
00094 #   endif
00095 #   include <openxr/openxr.h>
00096 #   include <openxr/openxr_platform.h>
00097 #   include <algorithm>
00098 #   include <cstdint>
00099 #   include <cstring>
00100 #   include <string>
00101 #   include <utility>
00102 #   include <vector>
00103 #endif
00104
00108 class GgApp
00109 {
00110 public:
00111
00118     GgApp(int major = 0, int minor = 1);
00119
00125     GgApp(const GgApp& w) = delete;
00126
00132     GgApp(GgApp&& w) = default;
00133
00137     virtual ~GgApp();
00138
00145     GgApp& operator=(const GgApp& w) = delete;
00146
00153     GgApp& operator=(GgApp&& w) = default;
00154
00162     int main(int argc, const char* const* argv);
00163
00170     class Window
00171     {
00172     // ウィンドウの識別子
00173         GLFWwindow* window{ nullptr };
00174
00175         // ビューポートの横幅と高さ
00176         std::array<GLsizei, 2> size;
00177
00178         // フレームバッファの横幅と高さ
00179         std::array<GLsizei, 2> fboSize;
00180
00181     #if defined(IMGUI_VERSION)
00182         // メニューバーの高さ
00183         GLsizei menubarHeight{ 0 };
00184     #endif
00185
00186         // ビューポートの縦横比
00187         GLfloat aspect{ 1.0f };
00188
00189         // マウスの移動速度[X/Y/Z]
00190         std::array<GLfloat, 3> velocity{ 1.0f, 1.0f, 0.1f };
00191
00192         // マウスボタンの状態
00193         std::array<bool, GG_BUTTON_COUNT> status{};
00194

```

```

00195 // ユーザインタフェースのデータ構造
00196 struct HumanInterface
00197 {
00198     // 最後にタイプしたキー
00199     int lastKey{ 0 };
00200
00201     // 矢印キー
00202     std::array<std::array<int, 2>, 4> arrow{};
00203
00204     // マウスの現在位置
00205     std::array<GLfloat, 2> mouse{};
00206
00207     // マウスホイールの回転量
00208     std::array<GLfloat, 2> wheel{};
00209
00210     // 平行移動量[ボタン][直前/更新][X/Y/Z]
00211     std::array<std::array<gg::GgVector, 2>, GG.BUTTON_COUNT> translation{};
00212
00213     // トラックボール
00214     std::array<gg::GgTrackball, GG.BUTTON_COUNT> rotation;
00215
00216     // コンストラクタ
00217     HumanInterface()
00218     {
00219         resetTranslation();
00220     }
00221
00222     //
00223     // マウスや矢印キーによる平行移動量を初期化する
00224     //
00225     void resetTranslation();
00226
00227     //
00228     // 平行移動量と回転量を更新する (X, Y のみ, Z は wheel() で計算する)
00229     //
00230     void calcTranslation(int button, const std::array<GLfloat, 3>& velocity);
00231 };
00232
00233 // ヒューマンインタフェースデバイスのデータ
00234 std::array<HumanInterface, GG.INTERFACE_COUNT> interfaceData;
00235
00236 // ヒューマンインタフェースデバイスの番号
00237 int interfaceNo{ 0 };
00238
00239 //
00240 // ユーザー定義のコールバック関数へのポインタ
00241 //
00242 void* userPointer{ nullptr };
00243 void (*resizeFunc)(const Window* window, int width, int height){ nullptr };
00244 void (*keyboardFunc)(const Window* window, int key, int scancode, int action, int mods){ nullptr };
00245 };
00246 void (*mouseFunc)(const Window* window, int button, int action, int mods){ nullptr };
00247 void (*wheelFunc)(const Window* window, double x, double y){ nullptr };
00248
00249 //
00250 // ウィンドウのサイズ変更時の処理
00251 //
00252 static void resize(GLFWwindow* window, int width, int height);
00253
00254 //
00255 // キーボードをタイプした時の処理
00256 //
00257 static void keyboard(GLFWwindow* window, int key, int scancode, int action, int mods);
00258
00259 //
00260 // マウスボタンを操作したときの処理
00261 //
00262 static void mouse(GLFWwindow* window, int button, int action, int mods);
00263
00264 //
00265 // マウスホイールを操作した時の処理
00266 //
00267 static void wheel(GLFWwindow* window, double x, double y);
00268 public:
00269
00270     Window(const std::string& title = "GLFW Window", int width = 640, int height = 480,
00280         int fullscreen = 0, GLFWwindow* share = nullptr);
00281
00282     Window(const Window& w) = delete;
00283
00284     Window(Window&& w) noexcept;
00285
00286     virtual ~Window()
00287     {
00288         // ウィンドウが作成されていない場合は戻る
00289         if (!window) return;

```

```

00303
00304     // ウィンドウを破棄する
00305     glfwDestroyWindow(window);
00306 }
00307
00314 Window& operator=(const Window& w) = delete;
00315
00322 Window& operator=(Window&& w) noexcept;
00323
00329 auto* get() const
00330 {
00331     return window;
00332 }
00333
00339 void setClose(int flag = GLFW_TRUE) const
00340 {
00341     glfwSetWindowShouldClose(window, flag);
00342 }
00343
00349 bool shouldClose() const
00350 {
00351     // ウィンドウを閉じるべきなら true を返す
00352     return glfwWindowShouldClose(window) != GLFW_FALSE;
00353 }
00354
00360 explicit operator bool();
00361
00365 void swapBuffers() const;
00366
00370 void restoreViewport() const
00371 {
00372     if (!glfwGetWindowAttrib(window, GLFW_ICONIFIED)) glViewport(0, 0, fboSize[0], fboSize[1]);
00373 }
00374
00378 void updateViewport();
00379
00380 #if defined(IMGUI_VERSION)
00386 void setMenuBarHeight(GLsizei height)
00387 {
00388     // メニューバーの高さを保存する
00389     menubarHeight = height;
00390
00391     // ビューポートを復帰する
00392     updateViewport();
00393 }
00394 #endif
00395
00401 auto getWidth() const
00402 {
00403     return size[0];
00404 }
00405
00411 auto getHeight() const
00412 {
00413     return size[1];
00414 }
00415
00421 auto getFboWidth() const
00422 {
00423     return fboSize[0];
00424 }
00425
00431 auto getFboHeight() const
00432 {
00433     return fboSize[1];
00434 }
00435
00441 const auto& getSize() const
00442 {
00443     return size;
00444 }
00445
00451 void getSize(GLsizei* size) const
00452 {
00453     size[0] = getWidth();
00454     size[1] = getHeight();
00455 }
00456
00462 const auto& getFboSize() const
00463 {
00464     return fboSize;
00465 }
00466
00472 void getFboSize(GLsizei* fboSize) const
00473 {
00474     fboSize[0] = getFboWidth();
00475     fboSize[1] = getFboHeight();

```

```

00476     }
00477
00483     auto getAspect() const
00484     {
00485         return aspect;
00486     }
00487
00493     bool getKey(int key) const
00494     {
00495         #if defined(IMGUI_VERSION)
00496             // ImGui がキーボードを使うときはキーボードの処理を行わない
00497             if (ImGui::GetIO().WantCaptureKeyboard) return false;
00498         #endif
00499
00500         return glfwGetKey(window, key) != GLFW_RELEASE;
00501     }
00502
00508     void selectInterface(int no)
00509     {
00510         assert(static_cast<size_t>(no) < interfaceData.size());
00511         interfaceNo = no;
00512     }
00513
00521     void setVelocity(GLfloat vx, GLfloat vy, GLfloat vz = 0.1f)
00522     {
00523         velocity = std::array<GLfloat, 3>{ vx, vy, vz };
00524     }
00525
00531     int getLastKey()
00532     {
00533         auto& current_if{ interfaceData[interfaceNo] };
00534         const int key{ current_if.lastKey };
00535         current_if.lastKey = 0;
00536         return key;
00537     }
00538
00546     auto getArrow(int direction = 0, int mods = 0) const
00547     {
00548         const auto& current_if{ interfaceData[interfaceNo] };
00549         return static_cast<GLfloat>(current_if.arrow[mods & 3][direction & 1]);
00550     }
00551
00558     auto getArrowX(int mods = 0) const
00559     {
00560         return getArrow(0, mods);
00561     }
00562
00569     auto getArrowY(int mods = 0) const
00570     {
00571         return getArrow(1, mods);
00572     }
00573
00580     void getArrow(GLfloat* arrow, int mods = 0) const
00581     {
00582         arrow[0] = getArrowX(mods);
00583         arrow[1] = getArrowY(mods);
00584     }
00585
00591     auto getShiftArrowX() const
00592     {
00593         return getArrow(0, 1);
00594     }
00595
00601     auto getShiftArrowY() const
00602     {
00603         return getArrow(1, 1);
00604     }
00605
00611     void getShiftArrow(GLfloat* shift_arrow) const
00612     {
00613         shift_arrow[0] = getShiftArrowX();
00614         shift_arrow[1] = getShiftArrowY();
00615     }
00616
00622     auto getControlArrowX() const
00623     {
00624         return getArrow(0, 2);
00625     }
00626
00632     auto getControlArrowY() const
00633     {
00634         return getArrow(1, 2);
00635     }
00636
00642     void getControlArrow(GLfloat* control_arrow) const
00643     {
00644         control_arrow[0] = getControlArrowX();

```

```

00645     control_arrow[1] = getControlArrowY();
00646 }
00647
00653 auto getAltArrowX() const
00654 {
00655     return getArrow(0, 3);
00656 }
00657
00663 auto getAltArrowY() const
00664 {
00665     return getArrow(1, 3);
00666 }
00667
00673 void getAltArrow(GLfloat* alt_arrow) const
00674 {
00675     alt_arrow[0] = getAltArrowX();
00676     alt_arrow[1] = getAltArrowY();
00677 }
00678
00684 const auto* getMouse() const
00685 {
00686     const auto& current_if{ interfaceData[interfaceNo] };
00687     return current_if.mouse.data();
00688 }
00689
00695 void getMouse(GLfloat* position) const
00696 {
00697     const auto& current_if{ interfaceData[interfaceNo] };
00698     position[0] = current_if.mouse[0];
00699     position[1] = current_if.mouse[1];
00700 }
00701
00708 auto getMouse(int direction) const
00709 {
00710     const auto& current_if{ interfaceData[interfaceNo] };
00711     return current_if.mouse[direction & 1];
00712 }
00713
00719 auto getMouseX() const
00720 {
00721     const auto& current_if{ interfaceData[interfaceNo] };
00722     return current_if.mouse[0];
00723 }
00724
00730 auto getMouseY() const
00731 {
00732     const auto& current_if{ interfaceData[interfaceNo] };
00733     return current_if.mouse[1];
00734 }
00735
00741 const auto* getWheel() const
00742 {
00743     const auto& current_if{ interfaceData[interfaceNo] };
00744     return current_if.wheel.data();
00745 }
00746
00752 void getWheel(GLfloat* rotation) const
00753 {
00754     const auto& current_if{ interfaceData[interfaceNo] };
00755     rotation[0] = current_if.wheel[0];
00756     rotation[1] = current_if.wheel[1];
00757 }
00758
00765 auto getWheel(int direction) const
00766 {
00767     const auto& current_if{ interfaceData[interfaceNo] };
00768     return current_if.wheel[direction & 1];
00769 }
00770
00776 auto getWheelX() const
00777 {
00778     const auto& current_if{ interfaceData[interfaceNo] };
00779     return current_if.wheel[0];
00780 }
00781
00787 auto getWheelY() const
00788 {
00789     const auto& current_if{ interfaceData[interfaceNo] };
00790     return current_if.wheel[1];
00791 }
00792
00799 const auto& getTranslation(int button = GLFW_MOUSE_BUTTON_1) const
00800 {
00801     const auto& current_if{ interfaceData[interfaceNo] };
00802     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00803     return current_if.translation[button][1];
00804 }

```

```

00805
00812 auto getTranslationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00813 {
00814     const auto& current_if{ interfaceData[interfaceNo] };
00815     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00816     const auto& t{ current_if.translation[button][1] };
00817
00818     gg::GgMatrix m
00819     {
00820         1.0f, 0.0f, 0.0f, 0.0f,
00821         0.0f, 1.0f, 0.0f, 0.0f,
00822         0.0f, 0.0f, 1.0f, 0.0f,
00823         t[0], t[1], t[2], 1.0f
00824     };
00825
00826     return m;
00827 }
00828
00835 auto getScrollMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00836 {
00837     const auto& current_if{ interfaceData[interfaceNo] };
00838     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00839     const auto& t{ current_if.translation[button][1] };
00840
00841     gg::GgMatrix m
00842     {
00843         t[2] + 1.0f, 0.0f, 0.0f, 0.0f,
00844         0.0f, t[2] + 1.0f, 0.0f, 0.0f,
00845         0.0f, 0.0f, 1.0f, 0.0f,
00846         t[0], t[1], 0.0f, 1.0f
00847     };
00848
00849     return m;
00850 }
00851
00858 auto getRotation(int button = GLFW_MOUSE_BUTTON_1) const
00859 {
00860     const auto& current_if{ interfaceData[interfaceNo] };
00861     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00862     return current_if.rotation[button].getQuaternion();
00863 }
00864
00871 auto getRotationMatrix(int button = GLFW_MOUSE_BUTTON_1) const
00872 {
00873     const auto& current_if{ interfaceData[interfaceNo] };
00874     assert(button >= GLFW_MOUSE_BUTTON_1 && button < GLFW_MOUSE_BUTTON_1 + GG_BUTTON_COUNT);
00875     return current_if.rotation[button].getMatrix();
00876 }
00877
00881 void resetRotation()
00882 {
00883     // トラックボールを初期化する
00884     for (auto& tb : interfaceData[interfaceNo].rotation) tb.reset();
00885 }
00886
00890 void resetTranslation()
00891 {
00892     // 現在のインターフェースの平行移動量を初期化する
00893     interfaceData[interfaceNo].resetTranslation();
00894 }
00895
00899 void reset()
00900 {
00901     // トラックボール処理を初期化する
00902     resetRotation();
00903
00904     // 平行移動量を初期化する
00905     resetTranslation();
00906 }
00907
00913 void* getUserPointer() const
00914 {
00915     return userPointer;
00916 }
00917
00923 void setUserPointer(void* pointer)
00924 {
00925     userPointer = pointer;
00926 }
00927
00933 void setResizeFunc(void (*func)(const Window* window, int width, int height))
00934 {
00935     resizeFunc = func;
00936 }
00937
00943 void setKeyboardFunc(void (*func)(const Window* window, int key, int scancode, int action, int
mods))

```

```

00944     {
00945         keyboardFunc = func;
00946     }
00947
00953     void setMouseFunc(void (*func)(const Window* window, int button, int action, int mods))
00954     {
00955         mouseFunc = func;
00956     }
00957
00963     void setWheelFunc(void (*func)(const Window* window, double x, double y))
00964     {
00965         wheelFunc = func;
00966     }
00967 };
00968
00969 #if defined(GG_USE_OPENXR)
00978 class OpenXR
00979 {
00980 public:
00981
00985     enum Hand
00986     {
00987         Left = 0,
00988         Right = 1,
00989         Count = 2
00990     };
00991
00992 private:
00993
00994     // OpenXR のインスタンスとシステム
00995     XrInstance instance{ XR_NULL_HANDLE };
00996     XrSystemId systemId{ XR_NULL_SYSTEM_ID };
00997
00998     // OpenXR のシステムの名前
00999     std::string systemName;
01000
01001     // OpenXR のセッション
01002     XrSession session{ XR_NULL_HANDLE };
01003     XrSessionState sessionState{ XR_SESSION_STATE_UNKNOWN };
01004
01005     // OpenXR の参照空間
01006     XrSpace appSpace{ XR_NULL_HANDLE };
01007     XrReferenceSpaceType referenceSpaceType{ XR_REFERENCE_SPACE_TYPE_STAGE };
01008
01009     // ビュー設定とステート
01010     std::vector<XrViewConfigurationView> views;
01011     std::vector<XrView> viewStates;
01012
01013     // OpenXR のスワップチェーン
01014     std::vector<XrSwapchain> swapchains;
01015     std::vector<std::vector<XrSwapchainImageOpenGLKHR>> swapchainImages;
01016
01017     // OpenXR へのレンダリングに使う FBO とデプスバッファ (ビューの数だけ確保する)
01018     std::vector<GLuint> openxrFbo;
01019     std::vector<GLuint> openxrDepth;
01020
01021     // スワップチェーンのカラーフォーマットが sRGB なら true
01022     bool swapchainIsSrgb{ true };
01023
01024     // 環境の合成方法
01025     XrEnvironmentBlendMode blendMode{ XR_ENVIRONMENT_BLEND_MODE_OPAQUE };
01026
01027     // フレームの同期状態
01028     XrFrameState frameState{ XR_TYPE_FRAME_STATE };
01029     bool isSessionRunning{ false };
01030
01031     // xrBeginFrame() を呼んで xrEndFrame() を呼んでいない状態なら true
01032     bool frameBegun{ false };
01033
01034     // 視点の姿勢が取得できていれば true
01035     bool viewPoseValid{ false };
01036
01037     // 初期化が完了していれば true
01038     bool initialized{ false };
01039
01040     // 各フレームで取得したスワップチェーンイメージのインデックス
01041     std::vector<uint32_t> currentIndex;
01042
01043     // スワップチェーンイメージを取得中のビューなら true
01044     std::vector<bool> imageAcquired;
01045
01046     // ミラー表示を行うビューの番号, ミラー表示を行わないなら -1
01047     int mirrorView{ 0 };
01048
01049     // OpenXR のミラー表示を行うウィンドウ
01050     const Window* window{ nullptr };
01051

```



```

01052 // アクションセット
01053 XrActionSet actionSet{ XR_NULL_HANDLE };
01054
01055 // アクション定義
01056 XrAction aimPoseAction{ XR_NULL_HANDLE };
01057 XrAction gripPoseAction{ XR_NULL_HANDLE };
01058 XrAction triggerAction{ XR_NULL_HANDLE };
01059 XrAction gripAction{ XR_NULL_HANDLE };
01060 XrAction thumbstickAction{ XR_NULL_HANDLE };
01061 XrAction thumbstickClickAction{ XR_NULL_HANDLE };
01062 XrAction primaryButtonAction{ XR_NULL_HANDLE };
01063 XrAction secondaryButtonAction{ XR_NULL_HANDLE };
01064 XrAction menuButtonAction{ XR_NULL_HANDLE };
01065 XrAction hapticAction{ XR_NULL_HANDLE };
01066
01067 // コントローラーのスペース
01068 XrSpace aimSpace[Hand::Count]{ XR_NULL_HANDLE, XR_NULL_HANDLE };
01069 XrSpace gripSpace[Hand::Count]{ XR_NULL_HANDLE, XR_NULL_HANDLE };
01070
01071 // コントローラーの状態キャッシュ
01072 struct ControllerState
01073 {
01074     bool isTracked{ false };
01075     XrPosef gripPose{ { 0.0f, 0.0f, 0.0f, 1.0f }, { 0.0f, 0.0f, 0.0f } };
01076     XrPosef aimPose{ { 0.0f, 0.0f, 0.0f, 1.0f }, { 0.0f, 0.0f, 0.0f } };
01077     float trigger{ 0.0f };
01078     float grip{ 0.0f };
01079     std::array<float, 2> thumbstick{ 0.0f, 0.0f };
01080     bool thumbstickClick{ false };
01081     bool primaryButton{ false };
01082     bool secondaryButton{ false };
01083     bool menuButton{ false };
01084 };
01085 ControllerState controllerStates[Hand::Count];
01086
01087 // サポートするバスの文字列
01088 XrPath handSubactionPath[Hand::Count]{ XR_NULL_PATH, XR_NULL_PATH };
01089
01090 //
01091 // アクションシステムを初期化する
01092 //
01093 void initActions();
01094
01095 //
01096 // アクション状態を更新する
01097 //
01098 void pollActions();
01099
01100 //
01101 // OpenXR のイベントを処理する
01102 //
01103 void pollEvents();
01104
01105 //
01106 // スワップチェーンを作成する
01107 //
01108 void createSwapchains();
01109
01110 //
01111 // 描画中のフレームを合成器に転送する
01112 //
01113 void endFrame();
01114
01115 //
01116 // OpenXR のハンドルを破棄する (OpenGL の資源には触れない)
01117 //
01118 void destroyXr();
01119
01120 //
01121 // ミラー表示を行う
01122 //
01123 void blitMirror() const;
01124
01125 //
01126 // コンストラクタ
01127 //
01128 OpenXR();
01129
01130 //
01131 // デストラクタ
01132 //
01133 virtual ~OpenXR();
01134
01135 public:
01136
01137 // シングルトンなのでコピー・ムーブ禁止
01138 OpenXR(const OpenXR&) = delete;

```

```

01139     OpenXR& operator=(const OpenXR&) = delete;
01140     OpenXR(OpenXR&&) = delete;
01141     OpenXR& operator=(OpenXR&&) = delete;
01142
01156     static OpenXR& initialize(const Window& window,
01157         XrReferenceSpaceType spaceType = XR_REFERENCE_SPACE_TYPE_STAGE,
01158         const char* appName = "GgApp");
01159
01167     void terminate();
01168
01181     bool begin();
01182
01188     void select(int eye);
01189
01198     void select(int eye, GLfloat* screen, GLfloat* position, GLfloat* orientation);
01199
01209     void commit(int eye);
01210
01222     bool submit(bool mirror = true);
01223
01229     void setMirror(int eye);
01230
01236     int getMirror() const;
01237
01243     bool isRunning() const;
01244
01250     bool isFocused() const;
01251
01257     const std::string& getSystemName() const;
01258
01267     gg::GgMatrix getProjectionMatrix(int eye, GLfloat zNear = 0.1f, GLfloat zFar = 100.0f) const;
01268
01275     gg::GgMatrix getViewMatrix(int eye) const;
01276
01283     gg::GgMatrix getPoseMatrix(int eye) const;
01284
01291     gg::GgVector getPosition(int eye) const;
01292
01299     gg::GgQuaternion getOrientation(int eye) const;
01300
01307     const XrFovf& getFov(int eye) const;
01308
01315     const XrPosef& getPose(int eye) const;
01316
01322     bool isPoseValid() const;
01323
01330     GLsizei getWidth(int eye = 0) const;
01331
01338     GLsizei getHeight(int eye = 0) const;
01339
01346     GLfloat getAspect(int eye = 0) const;
01347
01353     uint32_t getViewCount() const;
01354
01360     XrReferenceSpaceType getReferenceSpaceType() const;
01361
01368     bool isTracked(int hand) const;
01369
01376     gg::GgMatrix getGripMatrix(int hand) const;
01377
01384     gg::GgMatrix getAimMatrix(int hand) const;
01385
01392     gg::GgVector getGripPosition(int hand) const;
01393
01400     gg::GgQuaternion getGripOrientation(int hand) const;
01401
01408     gg::GgVector getAimPosition(int hand) const;
01409
01416     gg::GgQuaternion getAimOrientation(int hand) const;
01417
01424     float getTrigger(int hand) const;
01425
01432     float getGrip(int hand) const;
01433
01440     std::array<float, 2> getThumbstick(int hand) const;
01441
01448     bool getThumbstickClick(int hand) const;
01449
01456     bool getPrimaryButton(int hand) const;
01457
01464     bool getSecondaryButton(int hand) const;
01465
01477     bool getMenuButton(int hand = Hand::Left) const;
01478
01487     void applyHapticVibration(int hand, float durationSeconds = 0.1f, float frequency =
XR_FREQUENCY_UNSPECIFIED, float amplitude = 0.5f);
01488 };
```

```
01489 #endif
01490
01496     static std::string getUsername();
01497 };
```


Index

- `_ggError`
 - `gg`, [14](#)
 - `_ggFBOError`
 - `gg`, [14](#)
 - `~GgApp`
 - `GgApp`, [85](#)
 - `~GgBuffer`
 - `gg::GgBuffer< T >`, [89](#)
 - `~GgColorTexture`
 - `gg::GgColorTexture`, [98](#)
 - `~GgElements`
 - `gg::GgElements`, [105](#)
 - `~GgMatrix`
 - `gg::GgMatrix`, [115](#)
 - `~GgNormalTexture`
 - `gg::GgNormalTexture`, [173](#)
 - `~GgPointShader`
 - `gg::GgPointShader`, [186](#)
 - `~GgPoints`
 - `gg::GgPoints`, [180](#)
 - `~GgQuaternion`
 - `gg::GgQuaternion`, [205](#)
 - `~GgShader`
 - `gg::GgShader`, [276](#)
 - `~GgShape`
 - `gg::GgShape`, [279](#)
 - `~GgSimpleObj`
 - `gg::GgSimpleObj`, [283](#)
 - `~GgSimpleShader`
 - `gg::GgSimpleShader`, [289](#)
 - `~GgTexture`
 - `gg::GgTexture`, [310](#)
 - `~GgTrackball`
 - `gg::GgTrackball`, [319](#)
 - `~GgTriangles`
 - `gg::GgTriangles`, [329](#)
 - `~GgUniformBuffer`
 - `gg::GgUniformBuffer< T >`, [335](#)
 - `~GgVector`
 - `gg::GgVector`, [350](#)
 - `~GgVertexArray`
 - `gg::GgVertexArray`, [377](#)
 - `~LightBuffer`
 - `gg::GgSimpleShader::LightBuffer`, [385](#)
 - `~MaterialBuffer`
 - `gg::GgSimpleShader::MaterialBuffer`, [404](#)
 - `~Window`
 - `GgApp::Window`, [422](#)
- `add`
 - `gg::GgQuaternion`, [205–207](#)
- `ambient`
 - `gg::GgSimpleShader::Light`, [381](#)
 - `gg::GgSimpleShader::Material`, [400](#)
- `begin`
 - `gg::GgTrackball`, [319](#)
- `bind`
 - `gg::GgBuffer< T >`, [90](#)
 - `gg::GgColorTexture`, [99](#)
 - `gg::GgNormalTexture`, [173](#)
 - `gg::GgTexture`, [311](#)
 - `gg::GgUniformBuffer< T >`, [335](#)
 - `gg::GgVertexArray`, [377](#)
- `BindingPoints`
 - `gg`, [14](#)
- `conjugate`
 - `gg::GgQuaternion`, [208](#)
- `copy`
 - `gg::GgBuffer< T >`, [90](#)
 - `gg::GgUniformBuffer< T >`, [335](#)
- `diffuse`
 - `gg::GgSimpleShader::Light`, [381](#)
 - `gg::GgSimpleShader::Material`, [400](#)
- `distance3`
 - `gg::GgVector`, [350](#)
- `distance4`
 - `gg::GgVector`, [350](#)
- `divide`
 - `gg::GgQuaternion`, [208–210](#)
- `dot3`
 - `gg::GgVector`, [351](#)
- `dot4`
 - `gg::GgVector`, [352](#)
- `draw`
 - `gg::GgElements`, [106](#)
 - `gg::GgPoints`, [180](#)
 - `gg::GgShape`, [279](#)
 - `gg::GgSimpleObj`, [283](#)
 - `gg::GgTriangles`, [329](#)
- `end`
 - `gg::GgTrackball`, [319](#)
- `euler`
 - `gg::GgQuaternion`, [211–213](#)
- `fill`
 - `gg::GgUniformBuffer< T >`, [336](#)
- `frustum`

- gg::GgMatrix, 115
- get
 - gg::GgColorTexture, 99
 - gg::GgMatrix, 116, 117
 - gg::GgNormalTexture, 173
 - gg::GgPointShader, 187
 - gg::GgQuaternion, 214
 - gg::GgShader, 276
 - gg::GgShape, 280
 - gg::GgSimpleObj, 283
 - gg::GgTrackball, 320
 - gg::GgVertexArray, 377
 - GgApp::Window, 422
- getAltArrow
 - GgApp::Window, 422
- getAltArrowX
 - GgApp::Window, 423
- getAltArrowY
 - GgApp::Window, 423
- getArrow
 - GgApp::Window, 424, 425
- getArrowX
 - GgApp::Window, 425
- getArrowY
 - GgApp::Window, 426
- getAspect
 - GgApp::Window, 427
- getBuffer
 - gg::GgBuffer< T >, 90
 - gg::GgPoints, 181
 - gg::GgTriangles, 329
 - gg::GgUniformBuffer< T >, 337
- getConjugateMatrix
 - gg::GgQuaternion, 214, 215
- getControlArrow
 - GgApp::Window, 427
- getControlArrowX
 - GgApp::Window, 428
- getControlArrowY
 - GgApp::Window, 428
- getCount
 - gg::GgBuffer< T >, 91
 - gg::GgPoints, 181
 - gg::GgTriangles, 330
 - gg::GgUniformBuffer< T >, 338
- getFboHeight
 - GgApp::Window, 429
- getFboSize
 - GgApp::Window, 429, 430
- getFboWidth
 - GgApp::Window, 430
- getHeight
 - gg::GgColorTexture, 99
 - gg::GgNormalTexture, 173
 - gg::GgTexture, 311
 - GgApp::Window, 431
- getIndexBuffer
 - gg::GgElements, 106
- getIndexCount
 - gg::GgElements, 107
- getKey
 - GgApp::Window, 431
- getLastKey
 - GgApp::Window, 431
- getMatrix
 - gg::GgQuaternion, 216, 218
 - gg::GgTrackball, 320
- getMode
 - gg::GgShape, 280
- getMouse
 - GgApp::Window, 432
- getMouseX
 - GgApp::Window, 433
- getMouseY
 - GgApp::Window, 433
- getQuaternion
 - gg::GgTrackball, 320
- getRotation
 - GgApp::Window, 433
- getRotationMatrix
 - GgApp::Window, 433
- getScale
 - gg::GgTrackball, 321
- getScrollMatrix
 - GgApp::Window, 434
- getShiftArrow
 - GgApp::Window, 434
- getShiftArrowX
 - GgApp::Window, 434
- getShiftArrowY
 - GgApp::Window, 435
- getSize
 - gg::GgColorTexture, 99
 - gg::GgNormalTexture, 174
 - gg::GgTexture, 311
 - GgApp::Window, 436
- getStart
 - gg::GgTrackball, 322
- getStride
 - gg::GgBuffer< T >, 91
 - gg::GgUniformBuffer< T >, 338
- getTarget
 - gg::GgBuffer< T >, 92
 - gg::GgUniformBuffer< T >, 339
- getTexture
 - gg::GgColorTexture, 100
 - gg::GgNormalTexture, 174
 - gg::GgTexture, 312
- getTranslation
 - GgApp::Window, 436
- getTranslationMatrix
 - GgApp::Window, 438
- getUsername
 - GgApp, 85
- getUserPointer
 - GgApp::Window, 438

getWheel
 GgApp::Window, 438, 439
getWheelX
 GgApp::Window, 439
getWheelY
 GgApp::Window, 439
getWidth
 gg::GgColorTexture, 100
 gg::GgNormalTexture, 174
 gg::GgTexture, 312
 GgApp::Window, 439
gg, 11
 _ggError, 14
 _ggFBOError, 14
 BindingPoints, 14
 ggArraysObj, 15
 ggBufferAlignment, 82
 ggConjugate, 15
 ggCreateComputeShader, 16
 ggCreateNormalMap, 17
 ggCreateShader, 18
 ggCross, 19
 ggDistance3, 20, 21
 ggDistance4, 22
 ggDot3, 23, 24
 ggDot4, 24, 25
 ggElementsMesh, 26
 ggElementsObj, 27
 ggElementsSphere, 28
 ggEllipse, 28
 ggEulerQuaternion, 29, 31
 ggFrustum, 32
 ggIdentity, 32
 ggIdentityQuaternion, 33
 ggInit, 33
 ggInvert, 34, 35
 ggLength3, 35, 36
 ggLength4, 37
 ggLoadComputeShader, 38
 ggLoadHeight, 39
 ggLoadImage, 39
 ggLoadShader, 40, 41
 ggLoadSimpleObj, 42, 43
 ggLoadTexture, 44
 ggLookat, 45, 46
 ggMatrixQuaternion, 47, 48
 ggNorm, 48
 ggNormal, 49
 ggNormalize, 49
 ggNormalize3, 50–52
 ggNormalize4, 52–54
 ggOrthogonal, 55
 ggPerspective, 55
 ggPointsCube, 56
 ggPointsSphere, 57
 ggQuaternionMatrix, 57
 ggQuaternionTransposeMatrix, 58
 ggReadImage, 58
 ggRectangle, 60
 ggRotate, 60–63
 ggRotateQuaternion, 63, 64
 ggRotateX, 65
 ggRotateY, 66
 ggRotateZ, 66
 ggSaveColor, 68
 ggSaveDepth, 68
 ggSaveTga, 70
 ggScale, 71, 72
 ggSlerp, 73, 75
 ggTranslate, 76, 78
 ggTranspose, 79
 LightBindingPoint, 14
 MaterialBindingPoint, 14
 operator+, 80
 operator-, 81
 operator/, 82
 operator*, 80
gg.cpp, 449
gg.h, 526
 ggError, 531
 ggFBOError, 531
 pathChar, 531
 pathString, 531
 TCharToUtf8, 531
 Utf8ToTChar, 531
gg::GgBuffer< T >, 87
 ~GgBuffer, 89
 bind, 90
 copy, 90
 getBuffer, 90
 getCount, 91
 getStride, 91
 getTarget, 92
 GgBuffer, 88, 89
 map, 92, 93
 operator=, 94
 read, 94
 send, 95
 unbind, 95
 unmap, 96
gg::GgColorTexture, 96
 ~GgColorTexture, 98
 bind, 99
 get, 99
 getHeight, 99
 getSize, 99
 getTexture, 100
 getWidth, 100
 GgColorTexture, 97, 98
 load, 100, 101
 operator bool, 102
 operator!, 102
 unbind, 102
gg::GgElements, 103
 ~GgElements, 105
 draw, 106

- getIndexBuffer, 106
- getIndexCount, 107
- GgElements, 104, 105
- load, 107
- send, 108
- gg::GgMatrix, 109
 - ~GgMatrix, 115
 - frustum, 115
 - get, 116, 117
 - GgMatrix, 112–114
 - invert, 118
 - loadFrustum, 118
 - loadIdentity, 119
 - loadInvert, 120
 - loadLookat, 121, 122
 - loadNormal, 123, 124
 - loadOrthogonal, 124
 - loadPerspective, 125
 - loadRotate, 126, 127, 129, 130
 - loadRotateX, 131
 - loadRotateY, 132
 - loadRotateZ, 132
 - loadScale, 133, 134
 - loadTranslate, 135, 136
 - loadTranspose, 137
 - lookat, 138, 139
 - normal, 141
 - operator+, 146, 148
 - operator+=, 149
 - operator-, 150, 151
 - operator-=, 151, 152
 - operator/, 153
 - operator/=: 154, 155
 - operator=, 156, 157
 - operator*, 142, 144
 - operator*=, 145, 146
 - orthogonal, 157
 - perspective, 158
 - projection, 158, 159
 - rotate, 160–162
 - rotateX, 163
 - rotateY, 164
 - rotateZ, 164
 - scale, 165, 167
 - translate, 168, 169
 - transpose, 170
- gg::GgNormalTexture, 171
 - ~GgNormalTexture, 173
 - bind, 173
 - get, 173
 - getHeight, 173
 - getSize, 174
 - getTexture, 174
 - getWidth, 174
 - GgNormalTexture, 172
 - load, 175, 176
 - operator bool, 176
 - operator!, 176
 - unbind, 177
- gg::GgPoints, 177
 - ~GgPoints, 180
 - draw, 180
 - getBuffer, 181
 - getCount, 181
 - GgPoints, 179
 - load, 181
 - operator bool, 182
 - operator!, 182
 - send, 182
- gg::GgPointShader, 183
 - ~GgPointShader, 186
 - get, 187
 - GgPointShader, 184–186
 - load, 187, 188
 - loadMatrix, 189
 - loadModelviewMatrix, 191, 192
 - loadProjectionMatrix, 192, 193
 - unuse, 194
 - use, 194–197
- gg::GgQuaternion, 198
 - ~GgQuaternion, 205
 - add, 205–207
 - conjugate, 208
 - divide, 208–210
 - euler, 211–213
 - get, 214
 - getConjugateMatrix, 214, 215
 - getMatrix, 216, 218
 - GgQuaternion, 201, 203–205
 - invert, 218
 - loadAdd, 219–221
 - loadConjugate, 222
 - loadDivide, 224, 226
 - loadEuler, 227, 229
 - loadIdentity, 230
 - loadInvert, 230, 231
 - loadMatrix, 232
 - loadMultiply, 233, 234
 - loadNormalize, 235, 237
 - loadRotate, 237, 238
 - loadRotateX, 239
 - loadRotateY, 240
 - loadRotateZ, 241
 - loadSlerp, 242–244
 - loadSubtract, 245, 246
 - multiply, 248, 249, 251
 - norm, 251
 - normalize, 252
 - operator+, 256
 - operator+=, 257, 258
 - operator-, 258, 259
 - operator-=, 259, 260
 - operator/, 261
 - operator/=: 262, 263
 - operator=, 263–265
 - operator*, 253, 254

- operator*=, 254, 255
- rotate, 265, 266
- rotateX, 267
- rotateY, 268
- rotateZ, 268
- slerp, 269
- subtract, 270, 271
- gg::GgShader, 273
 - ~GgShader, 276
 - get, 276
 - GgShader, 273–275
 - operator=, 276, 277
 - unuse, 277
 - use, 277
- gg::GgShape, 278
 - ~GgShape, 279
 - draw, 279
 - get, 280
 - getMode, 280
 - GgShape, 279
 - operator bool, 281
 - operator!, 281
 - setMode, 281
- gg::GgSimpleObj, 282
 - ~GgSimpleObj, 283
 - draw, 283
 - get, 283
 - GgSimpleObj, 282
 - operator bool, 284
 - operator!, 284
- gg::GgSimpleShader, 285
 - ~GgSimpleShader, 289
 - GgSimpleShader, 287, 288
 - load, 289, 290
 - loadMatrix, 291–293
 - loadModelviewMatrix, 294, 295
 - operator=, 296
 - use, 298–302, 304–307
- gg::GgSimpleShader::Light, 380
 - ambient, 381
 - diffuse, 381
 - position, 381
 - specular, 381
- gg::GgSimpleShader::LightBuffer, 382
 - ~LightBuffer, 385
 - LightBuffer, 383, 384
 - load, 385, 386
 - loadAmbient, 387, 388
 - loadColor, 389
 - loadDiffuse, 390, 391
 - loadPosition, 392–394
 - loadSpecular, 396, 397
 - select, 398
- gg::GgSimpleShader::Material, 399
 - ambient, 400
 - diffuse, 400
 - shininess, 400
 - specular, 400
- gg::GgSimpleShader::MaterialBuffer, 401
 - ~MaterialBuffer, 404
 - load, 405
 - loadAmbient, 406, 407
 - loadAmbientAndDiffuse, 408–410
 - loadDiffuse, 411, 412
 - loadShininess, 413, 414
 - loadSpecular, 415, 416
 - MaterialBuffer, 403, 404
 - select, 417
- gg::GgTexture, 308
 - ~GgTexture, 310
 - bind, 311
 - getHeight, 311
 - getSize, 311
 - getTexture, 312
 - getWidth, 312
 - GgTexture, 308–310
 - operator=, 312, 313
 - swapRandB, 313
 - unbind, 314
- gg::GgTrackball, 314
 - ~GgTrackball, 319
 - begin, 319
 - end, 319
 - get, 320
 - getMatrix, 320
 - getQuaternion, 320
 - getScale, 321
 - getStart, 322
 - GgTrackball, 318
 - motion, 322
 - operator=, 323
 - region, 323, 324
 - reset, 325
 - rotate, 325
- gg::GgTriangles, 326
 - ~GgTriangles, 329
 - draw, 329
 - getBuffer, 329
 - getCount, 330
 - GgTriangles, 327, 328
 - load, 330
 - send, 331
- gg::GgUniformBuffer< T >, 332
 - ~GgUniformBuffer, 335
 - bind, 335
 - copy, 335
 - fill, 336
 - getBuffer, 337
 - getCount, 338
 - getStride, 338
 - getTarget, 339
 - GgUniformBuffer, 332, 334
 - load, 339, 340
 - map, 341
 - read, 342
 - send, 343

- unbind, 344
- unmap, 344
- gg::GgVector, 345
 - ~GgVector, 350
 - distance3, 350
 - distance4, 350
 - dot3, 351
 - dot4, 352
 - GgVector, 346–349
 - length3, 352
 - length4, 353
 - normalize3, 353
 - normalize4, 354
 - operator+, 357, 359
 - operator+=", 359, 361
 - operator-, 361, 363
 - operator-=, 363, 365
 - operator/, 365, 367
 - operator/=", 367, 369
 - operator=, 369, 370
 - operator*, 354, 355
 - operator*=", 355, 357
- gg::GgVertex, 370
 - GgVertex, 372, 373
 - normal, 373
 - position, 373
- gg::GgVertexArray, 374
 - ~GgVertexArray, 377
 - bind, 377
 - get, 377
 - GgVertexArray, 374, 375
 - operator=, 377, 379
- GG_BUTTON_COUNT
 - GgApp.h, 615
- GG_INTERFACE_COUNT
 - GgApp.h, 615
- GgApp, 83
 - ~GgApp, 85
 - getUsername, 85
 - GgApp, 83, 84
 - main, 85
 - operator=, 86
- GgApp.cpp, 589
- GgApp.h, 613
 - GG_BUTTON_COUNT, 615
 - GG_INTERFACE_COUNT, 615
- GgApp::Window, 418
 - ~Window, 422
 - get, 422
 - getAltArrow, 422
 - getAltArrowX, 423
 - getAltArrowY, 423
 - getArrow, 424, 425
 - getArrowX, 425
 - getArrowY, 426
 - getAspect, 427
 - getControlArrow, 427
 - getControlArrowX, 428
 - getControlArrowY, 428
 - getFboHeight, 429
 - getFboSize, 429, 430
 - getFboWidth, 430
 - getHeight, 431
 - getKey, 431
 - getLastKey, 431
 - getMouse, 432
 - getMouseX, 433
 - getMouseY, 433
 - getRotation, 433
 - getRotationMatrix, 433
 - getScrollMatrix, 434
 - getShiftArrow, 434
 - getShiftArrowX, 434
 - getShiftArrowY, 435
 - getSize, 436
 - getTranslation, 436
 - getTranslationMatrix, 438
 - getUserPointer, 438
 - getWheel, 438, 439
 - getWheelX, 439
 - getWheelY, 439
 - getWidth, 439
 - operator bool, 440
 - operator=, 440, 441
 - reset, 441
 - resetRotation, 442
 - resetTranslation, 442
 - restoreViewport, 442
 - selectInterface, 443
 - setClose, 443
 - setKeyboardFunc, 443
 - setMouseFunc, 444
 - setResizeFunc, 444
 - setUserPointer, 445
 - setVelocity, 445
 - setWheelFunc, 445
 - shouldClose, 446
 - swapBuffers, 446
 - updateViewport, 446
 - Window, 420, 421
- ggArraysObj
 - gg, 15
- GgBuffer
 - gg::GgBuffer< T >, 88, 89
- ggBufferAlignment
 - gg, 82
- GgColorTexture
 - gg::GgColorTexture, 97, 98
- ggConjugate
 - gg, 15
- ggCreateComputeShader
 - gg, 16
- ggCreateNormalMap
 - gg, 17
- ggCreateShader
 - gg, 18

ggCross
 gg, 19
ggDistance3
 gg, 20, 21
ggDistance4
 gg, 22
ggDot3
 gg, 23, 24
ggDot4
 gg, 24, 25
GgElements
 gg::GgElements, 104, 105
ggElementsMesh
 gg, 26
ggElementsObj
 gg, 27
ggElementsSphere
 gg, 28
ggEllipse
 gg, 28
ggError
 gg.h, 531
ggEulerQuaternion
 gg, 29, 31
ggFBOError
 gg.h, 531
ggFrustum
 gg, 32
ggIdentity
 gg, 32
ggIdentityQuaternion
 gg, 33
ggInit
 gg, 33
ggInvert
 gg, 34, 35
ggLength3
 gg, 35, 36
ggLength4
 gg, 37
ggLoadComputeShader
 gg, 38
ggLoadHeight
 gg, 39
ggLoadImage
 gg, 39
ggLoadShader
 gg, 40, 41
ggLoadSimpleObj
 gg, 42, 43
ggLoadTexture
 gg, 44
ggLookat
 gg, 45, 46
GgMatrix
 gg::GgMatrix, 112–114
ggMatrixQuaternion
 gg, 47, 48
ggNorm
 gg, 48
ggNormal
 gg, 49
ggNormalize
 gg, 49
ggNormalize3
 gg, 50–52
ggNormalize4
 gg, 52–54
GgNormalTexture
 gg::GgNormalTexture, 172
ggOrthogonal
 gg, 55
ggPerspective
 gg, 55
GgPoints
 gg::GgPoints, 179
ggPointsCube
 gg, 56
GgPointShader
 gg::GgPointShader, 184–186
ggPointsSphere
 gg, 57
GgQuaternion
 gg::GgQuaternion, 201, 203–205
ggQuaternionMatrix
 gg, 57
ggQuaternionTransposeMatrix
 gg, 58
ggReadImage
 gg, 58
ggRectangle
 gg, 60
ggRotate
 gg, 60–63
ggRotateQuaternion
 gg, 63, 64
ggRotateX
 gg, 65
ggRotateY
 gg, 66
ggRotateZ
 gg, 66
ggSaveColor
 gg, 68
ggSaveDepth
 gg, 68
ggSaveTga
 gg, 70
ggScale
 gg, 71, 72
GgShader
 gg::GgShader, 273–275
GgShape
 gg::GgShape, 279
GgSimpleObj
 gg::GgSimpleObj, 282

- GgSimpleShader
 - gg::GgSimpleShader, 287, 288
- ggSlerp
 - gg, 73, 75
- GgTexture
 - gg::GgTexture, 308–310
- GgTrackball
 - gg::GgTrackball, 318
- ggTranslate
 - gg, 76, 78
- ggTranspose
 - gg, 79
- GgTriangles
 - gg::GgTriangles, 327, 328
- GgUniformBuffer
 - gg::GgUniformBuffer< T >, 332, 334
- GgVector
 - gg::GgVector, 346–349
- GgVertex
 - gg::GgVertex, 372, 373
- GgVertexArray
 - gg::GgVertexArray, 374, 375
- invert
 - gg::GgMatrix, 118
 - gg::GgQuaternion, 218
- length3
 - gg::GgVector, 352
- length4
 - gg::GgVector, 353
- LightBindingPoint
 - gg, 14
- LightBuffer
 - gg::GgSimpleShader::LightBuffer, 383, 384
- load
 - gg::GgColorTexture, 100, 101
 - gg::GgElements, 107
 - gg::GgNormalTexture, 175, 176
 - gg::GgPoints, 181
 - gg::GgPointShader, 187, 188
 - gg::GgSimpleShader, 289, 290
 - gg::GgSimpleShader::LightBuffer, 385, 386
 - gg::GgSimpleShader::MaterialBuffer, 405
 - gg::GgTriangles, 330
 - gg::GgUniformBuffer< T >, 339, 340
- loadAdd
 - gg::GgQuaternion, 219–221
- loadAmbient
 - gg::GgSimpleShader::LightBuffer, 387, 388
 - gg::GgSimpleShader::MaterialBuffer, 406, 407
- loadAmbientAndDiffuse
 - gg::GgSimpleShader::MaterialBuffer, 408–410
- loadColor
 - gg::GgSimpleShader::LightBuffer, 389
- loadConjugate
 - gg::GgQuaternion, 222
- loadDiffuse
 - gg::GgSimpleShader::LightBuffer, 390, 391
- gg::GgSimpleShader::MaterialBuffer, 411, 412
- loadDivide
 - gg::GgQuaternion, 224, 226
- loadEuler
 - gg::GgQuaternion, 227, 229
- loadFrustum
 - gg::GgMatrix, 118
- loadIdentity
 - gg::GgMatrix, 119
 - gg::GgQuaternion, 230
- loadInvert
 - gg::GgMatrix, 120
 - gg::GgQuaternion, 230, 231
- loadLookat
 - gg::GgMatrix, 121, 122
- loadMatrix
 - gg::GgPointShader, 189
 - gg::GgQuaternion, 232
 - gg::GgSimpleShader, 291–293
- loadModelviewMatrix
 - gg::GgPointShader, 191, 192
 - gg::GgSimpleShader, 294, 295
- loadMultiply
 - gg::GgQuaternion, 233, 234
- loadNormal
 - gg::GgMatrix, 123, 124
- loadNormalize
 - gg::GgQuaternion, 235, 237
- loadOrthogonal
 - gg::GgMatrix, 124
- loadPerspective
 - gg::GgMatrix, 125
- loadPosition
 - gg::GgSimpleShader::LightBuffer, 392–394
- loadProjectionMatrix
 - gg::GgPointShader, 192, 193
- loadRotate
 - gg::GgMatrix, 126, 127, 129, 130
 - gg::GgQuaternion, 237, 238
- loadRotateX
 - gg::GgMatrix, 131
 - gg::GgQuaternion, 239
- loadRotateY
 - gg::GgMatrix, 132
 - gg::GgQuaternion, 240
- loadRotateZ
 - gg::GgMatrix, 132
 - gg::GgQuaternion, 241
- loadScale
 - gg::GgMatrix, 133, 134
- loadShininess
 - gg::GgSimpleShader::MaterialBuffer, 413, 414
- loadSlerp
 - gg::GgQuaternion, 242–244
- loadSpecular
 - gg::GgSimpleShader::LightBuffer, 396, 397
 - gg::GgSimpleShader::MaterialBuffer, 415, 416
- loadSubtract

- gg::GgQuaternion, 245, 246
- loadTranslate
 - gg::GgMatrix, 135, 136
- loadTranspose
 - gg::GgMatrix, 137
- lookat
 - gg::GgMatrix, 138, 139
- main
 - GgApp, 85
- map
 - gg::GgBuffer< T >, 92, 93
 - gg::GgUniformBuffer< T >, 341
- MaterialBindingPoint
 - gg, 14
- MaterialBuffer
 - gg::GgSimpleShader::MaterialBuffer, 403, 404
- motion
 - gg::GgTrackball, 322
- multiply
 - gg::GgQuaternion, 248, 249, 251
- norm
 - gg::GgQuaternion, 251
- normal
 - gg::GgMatrix, 141
 - gg::GgVertex, 373
- normalize
 - gg::GgQuaternion, 252
- normalize3
 - gg::GgVector, 353
- normalize4
 - gg::GgVector, 354
- operator bool
 - gg::GgColorTexture, 102
 - gg::GgNormalTexture, 176
 - gg::GgPoints, 182
 - gg::GgShape, 281
 - gg::GgSimpleObj, 284
 - GgApp::Window, 440
- operator!
 - gg::GgColorTexture, 102
 - gg::GgNormalTexture, 176
 - gg::GgPoints, 182
 - gg::GgShape, 281
 - gg::GgSimpleObj, 284
- operator+
 - gg, 80
 - gg::GgMatrix, 146, 148
 - gg::GgQuaternion, 256
 - gg::GgVector, 357, 359
- operator+=
 - gg::GgMatrix, 149
 - gg::GgQuaternion, 257, 258
 - gg::GgVector, 359, 361
- operator-
 - gg, 81
 - gg::GgMatrix, 150, 151
 - gg::GgQuaternion, 258, 259
 - gg::GgVector, 361, 363
- operator-=
 - gg::GgMatrix, 151, 152
 - gg::GgQuaternion, 259, 260
 - gg::GgVector, 363, 365
- operator/
 - gg, 82
 - gg::GgMatrix, 153
 - gg::GgQuaternion, 261
 - gg::GgVector, 365, 367
- operator/=
 - gg::GgMatrix, 154, 155
 - gg::GgQuaternion, 262, 263
 - gg::GgVector, 367, 369
- operator=
 - gg::GgBuffer< T >, 94
 - gg::GgMatrix, 156, 157
 - gg::GgQuaternion, 263–265
 - gg::GgShader, 276, 277
 - gg::GgSimpleShader, 296
 - gg::GgTexture, 312, 313
 - gg::GgTrackball, 323
 - gg::GgVector, 369, 370
 - gg::GgVertexArray, 377, 379
 - GgApp, 86
 - GgApp::Window, 440, 441
- operator*
 - gg, 80
 - gg::GgMatrix, 142, 144
 - gg::GgQuaternion, 253, 254
 - gg::GgVector, 354, 355
- operator*=
 - gg::GgMatrix, 145, 146
 - gg::GgQuaternion, 254, 255
 - gg::GgVector, 355, 357
- orthogonal
 - gg::GgMatrix, 157
- pathChar
 - gg.h, 531
- pathString
 - gg.h, 531
- perspective
 - gg::GgMatrix, 158
- position
 - gg::GgSimpleShader::Light, 381
 - gg::GgVertex, 373
- projection
 - gg::GgMatrix, 158, 159
- read
 - gg::GgBuffer< T >, 94
 - gg::GgUniformBuffer< T >, 342
- region
 - gg::GgTrackball, 323, 324
- reset
 - gg::GgTrackball, 325
 - GgApp::Window, 441

- resetRotation
 - GgApp::Window, 442
 - resetTranslation
 - GgApp::Window, 442
 - restoreViewport
 - GgApp::Window, 442
 - rotate
 - gg::GgMatrix, 160–162
 - gg::GgQuaternion, 265, 266
 - gg::GgTrackball, 325
 - rotateX
 - gg::GgMatrix, 163
 - gg::GgQuaternion, 267
 - rotateY
 - gg::GgMatrix, 164
 - gg::GgQuaternion, 268
 - rotateZ
 - gg::GgMatrix, 164
 - gg::GgQuaternion, 268
 - scale
 - gg::GgMatrix, 165, 167
 - select
 - gg::GgSimpleShader::LightBuffer, 398
 - gg::GgSimpleShader::MaterialBuffer, 417
 - selectInterface
 - GgApp::Window, 443
 - send
 - gg::GgBuffer< T >, 95
 - gg::GgElements, 108
 - gg::GgPoints, 182
 - gg::GgTriangles, 331
 - gg::GgUniformBuffer< T >, 343
 - setClose
 - GgApp::Window, 443
 - setKeyboardFunc
 - GgApp::Window, 443
 - setMode
 - gg::GgShape, 281
 - setMouseFunc
 - GgApp::Window, 444
 - setResizeFunc
 - GgApp::Window, 444
 - setUserPointer
 - GgApp::Window, 445
 - setVelocity
 - GgApp::Window, 445
 - setWheelFunc
 - GgApp::Window, 445
 - shininess
 - gg::GgSimpleShader::Material, 400
 - shouldClose
 - GgApp::Window, 446
 - slerp
 - gg::GgQuaternion, 269
 - specular
 - gg::GgSimpleShader::Light, 381
 - gg::GgSimpleShader::Material, 400
 - subtract
 - gg::GgQuaternion, 270, 271
 - swapBuffers
 - GgApp::Window, 446
 - swapRandB
 - gg::GgTexture, 313
 - TCharToUtf8
 - gg.h, 531
 - translate
 - gg::GgMatrix, 168, 169
 - transpose
 - gg::GgMatrix, 170
 - unbind
 - gg::GgBuffer< T >, 95
 - gg::GgColorTexture, 102
 - gg::GgNormalTexture, 177
 - gg::GgTexture, 314
 - gg::GgUniformBuffer< T >, 344
 - unmap
 - gg::GgBuffer< T >, 96
 - gg::GgUniformBuffer< T >, 344
 - unuse
 - gg::GgPointShader, 194
 - gg::GgShader, 277
 - updateViewport
 - GgApp::Window, 446
 - use
 - gg::GgPointShader, 194–197
 - gg::GgShader, 277
 - gg::GgSimpleShader, 298–302, 304–307
 - Utf8ToTChar
 - gg.h, 531
 - Window
 - GgApp::Window, 420, 421
- ゲームグラフィックス特論の宿題用補助プログラム
GLFW3 版., 1